Google

SWE Coding Questions

196 questions candidates reported | Hard, then Medium, then Easy

Collected 2026-10-05 | one optimal answer each, brute force as pseudo-code, follow-ups

41	135	20
HARD	MEDIUM	EASY

Optimal answers: 88 from the interview site, 37 from your app, 71 written here.
★ = top 75 Medium/Hard or top 10 Easy by Google frequency (46 in this PDF).
Study tiers by Google frequency on LeetCode: TYPE 72, TALK 99, READ 25 (in this PDF).
25 design-style questions are mixed in under their topics, last in each topic and marked DESIGN-STYLE.

Topics inside each difficulty

- BFS/DFS Graph and Tree Traversal and Shortest Paths (48)
- Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (70)
- Dynamic Programming and State-Space Optimization (23)
- Expression Parsing and Evaluation (6)
- Other: Design, Heaps, Intervals and Misc (43)
- Trie and Prefix Indexing (6)

Sources

- hacktherounds.com/questions/google
- prachub.com Google SWE Coding & Algorithms, Technical Screen only
- aced.io/questions?company=google&role=swe&type=coding
- 1point3acres.com/interview/problems/company/google (SWE coding, no OA)

How to use

- Tap any title in the Contents to jump to it.
- Every page has < Prev | Contents | Next > at the bottom.
- Prev and Next move one question at a time; Contents returns here.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Google - Contents

196 questions | tap a title to jump to it

Hard questions (41) Go >

BFS/DFS Graph and Tree Traversal and Shortest Paths (9) Go >

H 1. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree — IN APP — TALK | LC 1325 | PracHub
H 2. Infection Spread on a Grid (Cellular Automaton) — TYPE | LC 994 | PracHub
H 3. Find minimum threshold enabling grid path — IN APP — TYPE | LC 1631 | PracHub
H 4. Compute distance-sum from every tree node — IN APP — TALK | LC 834 | PracHub
H 5. Compute minimax grid path and network delay — TALK | LC 1631 | PracHub
H 6. Minimize travel time with optional meeting point — TYPE | LC 743 | PracHub
H 7. Minimize maximum height along a grid path — IN APP — TYPE | LC 778 | PracHub
H 8. Implement a decode and encode function of a general N-ary tree. — TALK | LC 428 | Exponent
H 9. Friend Graph with Timeline (Friend / Unfriend Events) — TALK | 1Point3Acres

Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (17) Go >

H 10. Count of Smaller Numbers After Self — IN APP — TALK | LC 315 | HackTheRounds
H 11. Smallest Missing Positive Integer in Linear Time and Constant Extra Space — IN APP — TYPE | LC 41 | PracHub
H 12. Count Overlapping Rectangle Updates on a Grid — IN APP — TALK | LC 2536 | PracHub
H 13. Find Dictionary Matches for a Jammed String — TALK | LC 809 | PracHub
H 14. Simulate In-Place Cellular State Updates — IN APP — TALK | LC 289 | PracHub
★ H 15. Check Digits and Combine Ranges — TYPE | LC 9 | PracHub
★ H 16. Find Maximum Envelope Nesting — IN APP — TYPE | LC 354 | PracHub
H 17. Find longest duplicated substring — IN APP — TALK | LC 1044 | PracHub
★ H 18. Find safe travel intervals between planet influences — TYPE | LC 56 | PracHub
H 19. Shuffle array using random integer API — TALK | LC 384 | PracHub
H 20. Given an integer array nums and an integer k, return true if nums has a subarray... — IN APP — TALK | LC 523 | Exponent
★ H 21. Find the median of two sorted arrays. — IN APP — TYPE | LC 4 | Exponent
★ H 22. Implement a function to justify a list of words within a given width. — IN APP — TYPE | LC 68 | Exponent
H 23. Convert Valid Snake Case to Lower Camel Case — TALK | PracHub
H 24. Given two strings and two allowed operations, swapping positions of letters and converti... — TALK | LC 1153 | Exponent
H 25. Necklace Cut into Two Halves with Equal D/R Counts — TALK | 1Point3Acres
H 26. Implement a function that processes a stream of interval data. — TALK | LC 352 | Exponent

Dynamic Programming and State-Space Optimization (6) Go >

H 27. Find the Longest Straight Run of One Color — TALK | LC 562 | PracHub
H 28. Determine winner in stack-merging game — TALK | LC 292 | PracHub
H 29. Count same-color squares in a character grid — TYPE | LC 1277 | PracHub
H 30. Edit distance — IN APP — TYPE | LC 127 | Exponent
H 31. Count Perfect Wake Numbers up to N — TALK | 1Point3Acres
H 32. Matrix Flower Placement with House Adjacency — TALK | 1Point3Acres

Other: Design, Heaps, Intervals and Misc (8) Go >

H 33. Compute Turnstile Crossing Times — TALK | PracHub
H 34. Find secret word with match-count feedback — IN APP — TALK | LC 843 | PracHub
H 35. Compute city skyline outline — IN APP — TALK | LC 218 | PracHub
H 36. Return top-k subset sums in descending order — TALK | LC 2386 | PracHub
H 37. Merge k sorted linked lists. — IN APP — TYPE | LC 23 | Exponent

Google - Contents

196 questions | tap a title to jump to it

H 38. Find maximum value in huge text file — TALK | PracHub

H 39. Huffman Tree Construction — TALK | 1Point3Acres

★ H 40. Implement LRU Cache. — IN APP — TYPE | LC 146 | Exponent

Trie and Prefix Indexing (1) — Go >

H 41. Implement Longest-Match Text Replacement — IN APP — TALK | LC 648 | PracHub

Medium questions (135) — Go >

BFS/DFS Graph and Tree Traversal and Shortest Paths (35) — Go >

M 42. Lowest Common Ancestor of a Binary Tree III — IN APP — TALK | LC 1650 | HackTheRounds

M 43. Count Islands in a Grid and Track the Count as Land Cells Are Added — IN APP — TALK | LC 305 | PracHub

M 44. Find A Threshold-Limited Path With Minimum Required Safety — IN APP — TALK | LC 1102 | PracHub

M 45. Count Clusters of 2D Points Within a Radius — TALK | LC 547 | PracHub

M 46. Match people to questions using tags and priorities — TALK | LC 1820 | PracHub

★ M 47. Solve Grid and Counting Problems — TYPE | LC 200 | PracHub

M 48. Validate parent array forms a tree — IN APP — TALK | LC 261 | PracHub

M 49. Find shortest relationship path using BFS — IN APP — TALK | LC 127 | PracHub

★ M 50. Compute shortest paths with blocked nodes — TYPE | LC 743 | PracHub

M 51. Find mode in a trinary search tree — IN APP — TALK | LC 501 | PracHub

M 52. Determine whether two nodes are related — TALK | LC 1971 | PracHub

M 53. Compute distance to nearest taxi in grid — IN APP — TALK | LC 542 | PracHub

★ M 54. Solve tree, graph, sliding-window problems — TYPE | LC 103 | PracHub

★ M 55. Validate course catalog dependencies — IN APP — TYPE | LC 207 | PracHub

M 56. Implement zigzag level-order traversal — IN APP — TALK | LC 103 | PracHub

★ M 57. Course Schedule — IN APP — TYPE | LC 207 | Exponent

M 58. Given the root of a binary tree, return the length of the diameter of the tree. — IN APP — TYPE | LC 543 | Exponent

M 59. Construct a binary tree given its preorder and inorder traversals. — IN APP — TALK | LC 105 | Exponent

M 60. Determine if a given binary tree is a binary search tree (BST). — IN APP — TYPE | LC 98 | Exponent

M 61. Given a large n-ary tree where the root node can pass information to one child p... — IN APP — TALK | LC 1376 | Exponent

M 62. Minimum Direction Violations in a Directed Graph — TALK | LC 1466 | 1Point3Acres

M 63. Longest Consecutive Path in Binary Tree — TALK | LC 549 | 1Point3Acres

M 64. Shortest Directed Cycle Through a Node — TALK | LC 2608 | 1Point3Acres

M 65. Rotten Oranges / Multi-Source BFS (taxis) — IN APP — TYPE | LC 994 | 1Point3Acres

★ M 66. Number of Islands — IN APP — TYPE | LC 200 | 1Point3Acres

M 67. Lowest Common Ancestor — IN APP — TALK | LC 236 | HackTheRounds

M 68. Count Prefix-Matching Simple Paths in a Labeled Graph — TALK | LC 79 | PracHub

M 69. Solve matrix groups and recipe inventory — TALK | LC 2115 | PracHub

M 70. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread t... — TALK | LC 1102 | Exponent

M 71. Given a train schedule with departure city, arrival city, start time, end time, and duration, a... — TYPE | LC 743 | Exponent

M 72. Find the shortest path in a graph. — TYPE | LC 743 | Exponent

M 73. Minimum-Cost Tree Disconnection — READ | 1Point3Acres

M 74. Compute File System Total Size — READ | 1Point3Acres

M 75. Determine Reporting Relationships — TYPE | LC 721 | PracHub

M 76. Employee Hierarchy Add / Delete — READ | 1Point3Acres

Google - Contents

196 questions | tap a title to jump to it

Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (43) Go >

	M	77. Sliding-Window Mean After Removing the Largest K Values		TALK | LC 480 | PracHub
	M	78. Find the Largest Monotone Increasing Number		TALK | LC 738 | PracHub
★	M	79. Filter Repeated Robot Status Messages		TYPE | LC 359 | PracHub
	M	80. Can board states be transformed?		TALK | LC 777 | PracHub
★	M	81. Find shared objects across two log files		TYPE | LC 349 | PracHub
★	M	82. Solve array-sum and city-community problems		TYPE | LC 560 | PracHub
★	M	83. Add two big integers from digit lists	IN APP	TYPE | LC 2 | PracHub
★	M	84. Find median of merged RLE arrays		TYPE | LC 4 | PracHub
	M	85. Check pivot after removing one element		TALK | LC 724 | PracHub
	M	86. Find longest increasing contiguous subarray		TALK | LC 674 | PracHub
	M	87. Decide string transform with directional tokens and walls		TALK | LC 777 | PracHub
	M	88. Reconstruct original array from doubled shuffle		TALK | LC 2007 | PracHub
★	M	89. Solve Banana Speed and Interval Merge		TYPE | LC 875 | PracHub
★	M	90. Merge Intervals	IN APP	TYPE | LC 56 | Exponent
★	M	91. Group anagrams	IN APP	TYPE | LC 49 | Exponent
	M	92. Write functions to serialize and deserialize a list of strings.	IN APP	TALK | LC 271 | Exponent
	M	93. Determine if two sentences are similar.		TALK | LC 1813 | Exponent
	M	94. Count the number of ways to split a string.		TALK | LC 1573 | Exponent
★	M	95. Find the minimum window substring.	IN APP	TYPE | LC 76 | Exponent
	M	96. Given an array of strings, return groups of "buddy strings," where the strings have...	IN APP	TALK | LC 249 | Exponent
	M	97. Cutting Ribbons (LC 1891)		TALK | LC 1891 | 1Point3Acres
	M	98. Union of Sorted Interval Lists		TALK | LC 986 | 1Point3Acres
★	M	99. First Bad Version with Parallel Search Follow-up		TYPE | LC 278 | 1Point3Acres
	M	100. Run-Length Vector Storage + Dot Product		TALK | LC 1570 | 1Point3Acres
	M	101. Rotate Any m×n 2D Matrix		TYPE | LC 48 | 1Point3Acres
★	M	102. Shortest Subarray with K Distinct Integers		TYPE | LC 992 | HackTheRounds
	M	103. Check if all substrings are dictionary words		TALK | LC 139 | PracHub
★	M	104. Find Containing Range		TYPE | LC 56 | PracHub
	M	105. Check Whether Digits Are Monotone Increasing		READ | PracHub
★	M	106. Check if all substrings are anagrams of words		TYPE | LC 49 | PracHub
	M	107. Maximize 1D deliveries within distance and minimize total distance		TALK | LC 455 | PracHub
	M	108. Minimize calls to find all bad test pairs		READ | PracHub
	M	109. Min deletions to avoid overlap in first k		READ | PracHub
★	M	110. Given a list with response time for each server, calculate how long time it will take to pro...		TYPE | LC 875 | Exponent
	M	111. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance...		TALK | LC 475 | Exponent
	M	112. Progressive Array Partitioning		READ | 1Point3Acres
	M	113. Fence Painting Minimum Operations (Gemini Team)		READ | 1Point3Acres
	M	114. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)		READ | 1Point3Acres
	M	115. Sum of Two Squares in Two Ways		READ | HackTheRounds
	M	116. Hit Counter	IN APP	TALK | LC 362 | HackTheRounds
	M	117. Design set with O(1) random access	IN APP	TYPE | LC 380 | PracHub
	M	118. Design Tic-Tac-Toe With K Players		TALK | LC 348 | PracHub

Google - Contents

196 questions | tap a title to jump to it

M 119. Active Communication Top Users — READ | 1Point3Acres

Dynamic Programming and State-Space Optimization (15) — Go >

★ M 120. Maximize sum without choosing adjacent elements — IN APP — TYPE | LC 198 | PracHub

★ M 121. Given n houses in a line with money in each, find the maximum amount a robber... — IN APP — TYPE | LC 198 | Exponent

M 122. Explain how to find a target sum in an array. — IN APP — TALK | LC 494 | Exponent

★ M 123. Generate Parentheses — IN APP — TYPE | LC 22 | Exponent

★ M 124. Find the maximum subarray sum. — IN APP — TYPE | LC 53 | Exponent

M 125. Partition an array into two sub-arrays with equal sum. — IN APP — TYPE | LC 416 | Exponent

M 126. Permutations — IN APP — TYPE | LC 46 | Exponent

★ M 127. Find the longest palindromic subsequence using dynamic programming. — IN APP — TYPE | LC 516 | Exponent

M 128. Coin Change — IN APP — TYPE | LC 322 | Exponent

M 129. Count Word Decompositions into Chemical Element Symbols — TALK | LC 139 | 1Point3Acres

M 130. Jump Game with Profit — TYPE | LC 55 | 1Point3Acres

M 131. Trip Expense Settlement — IN APP — TALK | LC 465 | HackTheRounds

M 132. Reverse Coin Change — TALK | LC 518 | HackTheRounds

M 133. Determine if a 14-tile hand is winning — TALK | LC 659 | PracHub

M 134. Gold Chain Split into Two Equal Halves — READ | 1Point3Acres

Expression Parsing and Evaluation (6) — Go >

M 135. Nested List Weight Sum (LC 339) with Follow-Ups — TALK | LC 339 | 1Point3Acres

M 136. Target Expressions from Ordered Digits — TALK | LC 282 | 1Point3Acres

★ M 137. Decode String — IN APP — TYPE | LC 394 | 1Point3Acres

M 138. Nested Function Expression Evaluator — TALK | LC 770 | 1Point3Acres

★ M 139. Expand Recursive Key-Value Placeholders in a Template — TYPE | LC 394 | PracHub

M 140. Expression Tree Single-Leaf Mutation — READ | 1Point3Acres

Other: Design, Heaps, Intervals and Misc (32) — Go >

★ M 141. Meeting Rooms III Variant — TYPE | LC 253 | HackTheRounds

M 142. Playlist Shuffle No Adjacent Artists — TALK | LC 767 | HackTheRounds

★ M 143. Implement frequency + distance top-K queries — TYPE | LC 347 | PracHub

M 144. Rearrange Tasks With Cooldown — IN APP — TALK | LC 621 | PracHub

★ M 145. Solve Rooms and Top-K Streams — TYPE | LC 253 | PracHub

★ M 146. Compute servers needed for daily recurring jobs — IN APP — TYPE | LC 253 | PracHub

★ M 147. Compute minimum number of rooms needed — IN APP — TYPE | LC 253 | PracHub

★ M 148. Solve chat, grid paths, and car rentals — TYPE | LC 692 | PracHub

M 149. Simulate room assignments for scheduled meetings — IN APP — TALK | LC 2402 | PracHub

★ M 150. Top-K Frequent in Large Logs — IN APP — TYPE | LC 347 | 1Point3Acres

M 151. Gate Entry / Exit Time Resolution with Tie-Break Rules — TALK | LC 2534 | 1Point3Acres

M 152. Find a Horizontal Cut That Bisects Rectangle Area — TALK | LC 850 | PracHub

M 153. Min boards to cover roof holes — TALK | LC 1326 | PracHub

M 154. Solve several coding interview problems — TALK | LC 415 | PracHub

M 155. Choose optimal guesses for green-only Wordle — TALK | LC 843 | PracHub

M 156. Write SQL code to publish the Fibonacci series. — READ | Exponent

★ M 157. Sort a nearly sorted (or K sorted) array. — TYPE | LC 215 | Exponent

Google - Contents

196 questions | tap a title to jump to it

M 158. Streaming Points: Emit K Within Distance — READ | 1Point3Acres

M 159. Piano Hand Movement Grouping — READ | 1Point3Acres

★ M 160. Design a logger that returns top-K messages — TYPE | LC 359 | PracHub

M 161. Find top/bottom-k words in list or stream — TALK | LC 692 | PracHub

M 162. Implement iterator merging lists with filtering — TALK | LC 281 | PracHub

M 163. Implement a Reusable Locker Allocation System — TALK | LC 1845 | PracHub

M 164. Streaming Median (LC 295) with Follow-ups — IN APP — TYPE | LC 295 | 1Point3Acres

★ M 165. Streaming Insert + Find K-th Largest — IN APP — TALK | LC 703 | 1Point3Acres

★ M 166. Design a restaurant waitlist system — TYPE | LC 146 | PracHub

M 167. Design server allocation for multi-type nodes — READ | LC 1912 | PracHub

M 168. Design an elevator control system — READ | PracHub

M 169. Temperature Monitoring Data Structure — READ | 1Point3Acres

M 170. Next-Word Predictor — READ | 1Point3Acres

M 171. Implement Python's itertools.tee — READ | 1Point3Acres

M 172. Batch and Streaming Log Deduplication — READ | 1Point3Acres

Trie and Prefix Indexing (4) — Go >

M 173. Return dictionary words matching a prefix — IN APP — TALK | LC 208 | PracHub

M 174. Build Prefix Lookup with a Trie — IN APP — TALK | LC 208 | PracHub

M 175. Implement Trie Insert and Exact-Word Search — IN APP — TALK | LC 208 | PracHub

M 176. URL Request Handler Router — READ | 1Point3Acres

Easy questions (20) — Go >

BFS/DFS Graph and Tree Traversal and Shortest Paths (4) — Go >

E 177. Find right-side view of binary tree — IN APP — TALK | LC 199 | PracHub

E 178. Given two trees, determine if the pre-order traversals of two trees are same or not. — TYPE | LC 100 | Exponent

E 179. Calculate the height of a binary tree. — IN APP — TYPE | LC 104 | Exponent

E 180. Tree Node. — READ | Exponent

Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (10) — Go >

E 181. Check an Interval Against Existing Intervals for Conflicts — IN APP — TALK | LC 729 | PracHub

E 182. Implement string formatting and request tracking tasks — TALK | LC 246 | PracHub

E 183. Is this a valid palindrome? — IN APP — TYPE | LC 125 | Exponent

E 184. Reverse a Sentence — IN APP — TALK | LC 186 | Exponent

E 185. Given an array of strings, return the words that can be typed using letters of the alphabet... — TALK | LC 500 | Exponent

E 186. Find the Duplicates — TALK | LC 350 | Exponent

★ E 187. Maximum Profit — TYPE | LC 121 | Exponent

E 188. Sort a binary array in linear time. — TYPE | LC 75 | Exponent

E 189. Determine if two words are sorted in lexicographic order. — IN APP — TALK | LC 953 | Exponent

E 190. Parity Alternating Subarray Query — TALK | LC 303 | HackTheRounds

Dynamic Programming and State-Space Optimization (2) — Go >

★ E 191. Climbing Stairs — IN APP — TYPE | LC 70 | Exponent

E 192. Fibonacci Numbers — IN APP — TYPE | LC 509 | Exponent

Other: Design, Heaps, Intervals and Misc (3) — Go >

E 193. Find k pairs with smallest sums — IN APP — TALK | LC 373 | PracHub

Google - Contents

196 questions | tap a title to jump to it

E 194. Reverse a linked list. IN APP TYPE | LC 206 | Exponent

E 195. Implement Batched Undo/Redo Layer TALK | LC 1472 | PracHub

Trie and Prefix Indexing (1) Go >

E 196. Return all unique substrings of a given string. READ | Exponent

Hard questions (41)

BFS/DFS Graph and Tree Traversal and Shortest Paths 9 questions

HARD | IN APP LC 1325 | PracHub | Technical Screen | 1 of 196

Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

STUDY: TALK | Google frequency on LeetCode: all time 10.6, last 30 days 0 | book priority 150 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #1325 Delete Leaves With a Given Value

LeetCode match: LC 1325 Delete Leaves With a Given Value

Original: https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

APPROACH

Post-order DFS: process children first, then delete the node if it became a leaf with value == target. O(n).

HOW TO THINK ABOUT IT

'Repeatedly' hides an order: a node becomes deletable only after its children are processed. That is a post-order dependency, so a single bottom-up pass does what many passes would do. Whenever a rule says 'repeat until nothing changes' on a tree, ask whether children-first order collapses it to one pass.

Basis: LeetCode 1325, Delete Leaves With a Given Value.

BRUTE FORCE - PSEUDO-CODE

```
repeat: delete every leaf whose value equals the target (one pass over the tree)
stop when a pass deletes nothing
```

Time: O(n * h) | Space: O(h)

Why it is slow

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def remove_target_leaves(tree, target):
    n = len(tree)
    if n == 0 or tree[0] is None:
        return []
    left = [-1] * n
    right = [-1] * n
    order = [0]
    idx = 1
    head = 0
    while head < len(order):
        i = order[head]
        head += 1
        if idx < n:
            if tree[idx] is not None:
                left[i] = idx
                order.append(idx)
            idx += 1
        if idx < n:
            if tree[idx] is not None:
                right[i] = idx
                order.append(idx)
            idx += 1
    keep = [True] * n
    stack = [(0, False)]
    while stack:
        i, expanded = stack.pop()
        if expanded:
            if left[i] != -1 and not keep[left[i]]:
                left[i] = -1
            if right[i] != -1 and not keep[right[i]]:
                right[i] = -1
```

```
            if left[i] == -1 and right[i] == -1 and tree[i
] == target:
                keep[i] = False
        else:
            stack.append((i, True))
            if left[i] != -1:
                stack.append((left[i], False))
            if right[i] != -1:
                stack.append((right[i], False))
    if not keep[0]:
        return []
    out = [tree[0]]
    queue = [0]
    head = 0
    while head < len(queue):
        i = queue[head]
        head += 1
        for c in (left[i], right[i]):
            if c == -1:
                out.append(None)
            else:
                out.append(tree[c])
                queue.append(c)
    while out and out[-1] is None:
        out.pop()
    return out
```

Time: O(n), where n is the length of the input encoding: decoding, the post-order pruning pass and the re-encoding each touch every slot a constant number of times. | Space: O(n) for the child-link arrays, the traversal stack/queue and the output encoding.

Explanation

Decode the encoding into explicit links, prune bottom-up, then re-encode.

Decode. The first element is the root. Keep a cursor idx = 1 and a queue of the positions of present nodes in breadth-first order; for each dequeued node consume the next two slots as its left child and then its right child, recording a child only when that slot holds a value, and never consuming slots for absent nodes. That reproduces the encoding rule exactly and rebuilds every parent/child link in one linear pass.

Prune. Process the nodes in post-order using an explicit stack (each node is pushed once unexpanded and once expanded), because the height may reach 500 and recursion is unnecessary. Invariant: when a node is finalised, both of its subtrees have already been finalised, so the children that still count are exactly those marked kept. Detach every non-kept child, then mark the node itself not kept if and only if it now has no children and its value equals target. This is literally the cascade in the statement: a parent can only become a deletion candidate after its children are gone, and a single bottom-up pass settles every cascade because a node's status depends only on its own subtree. A node whose value equals target but that retains at least one child is therefore kept.

Re-encode. If the root was not kept, the answer is []. Otherwise emit the root's value and breadth-first traverse the surviving tree, appending two slots for every present node (None for an absent child) and enqueueing only present children; finally strip trailing Nones. Kept nodes keep their original parent and side by construction, since pruning only removes edges pointing at deleted nodes.

Edge cases: a one-node tree equal to target returns [] rather than [None]; a target-valued node with a surviving child stays; interior Nones must be preserved while trailing Nones must be dropped; an all-target chain collapses to the empty tree; target may be absent, zero or negative, and values may sit on either bound of [-1000000000, 1000000000]; the root's value is always present, so a non-empty answer never starts with None.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def removeLeafNodes_iterative(root, target):
    if root is None:
        return None
    parent = {id(root): None}
    order, stack = [], [root]
    while stack:
        n = stack.pop()
        order.append(n)
        for child in (n.left, n.right):
            if child:
                parent[id(child)] = n
                stack.append(child)
    for n in reversed(order):  # children always come befo
re their parent
        if n.left is None and n.right is None and n.val ==
 target:
            p = parent[id(n)]
            if p is None:
                return None
            if p.left is n:
                p.left = None
            else:
                p.right = None
    return root
```

Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. Cost: one post-order pass, O(n) time and O(h) space.

Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. Cost: O(1) extra, counted inside the same O(n) pass.

FUNCTION AND TEST CASES

Signature

```
def remove_target_leaves(tree: list[int | None], target: i
nt) -> list[int | None]:
```

Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

Test cases

Input	Expected
([5, 7, 7, 7, 3, None, 7], 7)	[5, 7, None, None, 3]
([3, 3, 1], 3)	[3, None, 1]

HARD | PracHub | Technical Screen | 2 of 196

Infection Spread on a Grid (Cellular Automaton)

STUDY: TYPE | Google frequency on LeetCode: all time 51.2, last 30 days 0 | book priority 58 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #994 Rotting Oranges, LeetCode #289 Game of Life

LeetCode match: LC 994 Rotting Oranges / LC 289 Game of Life (inferred from overview)

Original: https://prachub.com/coding-questions/infection-spread-on-a-grid-cellular-automaton

QUESTION

Overview: This question tests a candidate's ability to implement and optimize a synchronous cellular automaton simulation on a 2D grid, assessing proficiency in BFS-based propagation, state machine design, and incremental computation. It falls under algorithms and grid simulation, evaluating practical coding skill and the ability to handle progressively complex constraints including recovery delays, barriers, and early-termination fixed-point detection.

APPROACH

Keep a frontier of newly changed cells and update only their neighbors each step (multi-source BFS style) instead of rescanning the grid; use a state machine for cell transitions.

INTERVIEW-DAY PLAN

Clarify first

My copy has only the overview, so ask for the rules: the states (healthy, infected, recovered or immune), how infection spreads (four neighbors per step, synchronously), the recovery delay, whether barriers block spread, and what to return (the number of steps until nothing changes, or the final grid).

Brute force to say out loud

Rescan the entire grid on every step and compute the next grid from the old one. O(steps * mn).

Optimal idea

Keep a frontier of cells that changed in the last step. Each step only their neighbors can change, so process the frontier like a multi-source BFS and update a separate next frontier to keep the update synchronous. Store the infection time per cell to apply the recovery delay (recover when step minus infected time reaches the delay). Stop when the frontier is empty and no recoveries are pending, since the grid has reached a fixed point. O(mn) overall.

Dry-run these cases

No infected cells (0 steps), an isolated healthy cell behind barriers, a delay of 1, all cells infected at the start, two sources reaching a cell at the same time.

Likely follow-ups

Different neighborhoods, probabilistic spread, or very large grids (sparse frontier).

Pitfalls

Updating cells in place during a step, which makes the spread asynchronous. Missing pending recoveries when deciding to stop.

HOW TO THINK ABOUT IT

A synchronous cellular automaton is a BFS in disguise: the step number is the BFS level. Only cells next to cells that just changed can change next, so keep a frontier instead of rescanning the grid. Always compute the next state from the old state (not in place) so all cells update together.

Basis: Written. Assumed rules (confirm them): '.' healthy, 'I' infected, 'X' barrier; each step every healthy cell with an infected 4-neighbour becomes infected, all at once; the answer is the number of steps until nothing changes.

BRUTE FORCE - PSEUDO-CODE

```
steps = 0; repeat: build next = a copy of the grid where e
very healthy cell with an
    infected neighbour becomes infected
    if next equals the grid: stop; else grid = next, steps
 = steps + 1
```

Time: O(steps * m * n) | Space: O(m * n)

Why it is slow

Every step scans the whole grid even when only a thin edge is changing, O(steps * m * n). Keeping a frontier of newly infected cells (BFS) touches each cell once, O(m * n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(grid):
    R = len(grid)
    if R == 0:
        return []
    C = len(grid[0])
    rows = [list(r) for r in grid]
    out = [row[:] for row in rows]
    for i in range(R):
        for j in range(C):
            if rows[i][j] == "H":
                inf = False
                for di, dj in ((-1, 0), (1, 0), (0, -1), (
0, 1)):
                    ni, nj = i + di, j + dj
                    if 0 <= ni < R and 0 <= nj < C and row
s[ni][nj] == "I":
                        inf = True
```

```
                    break
            if inf:
                out[i][j] = "I"
    return ["".join(row) for row in out]
```

Time: O(R*C) | Space: O(R*C)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

```
def state_after(grid, t, delay=None):
    """Grid after t steps; with a recovery delay a cell in
fected at step s becomes 'R' at step s + delay."""
    g = [list(r) for r in grid]
    infected_at = {
        (r, c): 0 for r in range(len(g)) for c in range(le
n(g[0])) if g[r][c] == "I"
    }
    frontier, step = list(infected_at), 0
    while frontier and step < t:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1
)):
                nr, nc = r + dr, c + dc
                if (
                    0 <= nr < len(g)
                    and 0 <= nc < len(g[0])
                    and g[nr][nc] == "."
                    and (nr, nc) not in infected_at
                ):
                    infected_at[(nr, nc)] = step
                    nxt.append((nr, nc))
        frontier = nxt
    for (r, c), s in infected_at.items():
        g[r][c] = "R" if delay is not None and t - s >= de
lay else "I"
    return ["".join(row) for row in g]
```

Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> list[str]:
```

Sample call

```
solution(['HHH', 'HIH', 'HHH'])
```

Test cases

Input	Expected
`(['HHH', 'HIH', 'HHH'],)`	`['HIH', 'III', 'HIH']`
`(['HHH'],)`	`['HHH']`

HARD | IN APP LC 1631, 778 | PracHub | Technical Screen | 3 of 196

Find minimum threshold enabling grid path

STUDY: TYPE | Google frequency on LeetCode: all time 40, last 30 days 33 | book priority 47 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #1631 Path with Minimum Effort, LeetCode #778 Swim in Rising Water

LeetCode match: LC 1631 Path With Minimum Effort / LC 778 Swim in Rising Water (inferred from overview)

Original: https://prachub.com/coding-questions/find-minimum-threshold-enabling-grid-path

QUESTION

Overview: This question evaluates a candidate's understanding of grid pathfinding, reachability under constraints, and algorithmic optimization, focusing on skills such as graph traversal and handling threshold-based feasibility.

APPROACH

Binary search the threshold and BFS/DFS for reachability, or run Dijkstra/union-find where path cost is the max cell value along the path.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest threshold such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE - PSEUDO-CODE

```
depth-first search over every simple path from the top-lef
t to the bottom-right
for each path take its largest cell; answer = the smallest
 of those
```

Time: exponential | Space: O(path)

Why it is slow

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
import heapq


def solution(grid):
    m = len(grid)
    n = len(grid[0])

    best = [[float("inf")] * n for _ in range(m)]
    best[0][0] = grid[0][0]

    heap = [(grid[0][0], 0, 0)]
    directions = ((1, 0), (-1, 0), (0, 1), (0, -1))

    while heap:
        threshold, r, c = heapq.heappop(heap)

        if threshold != best[r][c]:
            continue

        if r == m - 1 and c == n - 1:
            return threshold

        for dr, dc in directions:
            nr = r + dr
            nc = c + dc

            if 0 <= nr < m and 0 <= nc < n:
                new_threshold = max(threshold, grid[nr][nc
])
                if new_threshold < best[nr][nc]:
                    best[nr][nc] = new_threshold
                    heapq.heappush(heap, (new_threshold, n
r, nc))

    return best[m - 1][n - 1]
```

Time: O(m * n * log(m * n)) | Space: O(m * n)

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Alternative: binary search the limit with a flood fill

```
def min_max_path_binary_search(H):
    m, n = len(H), len(H[0])

    def ok(limit):
        if H[0][0] > limit:
            return False
        seen, stack = {(0, 0)}, [(0, 0)]
        while stack:
            r, c = stack.pop()
            if (r, c) == (m - 1, n - 1):
                return True
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c +
 1), (r, c - 1)):
                if (
                    0 <= nr < m
                    and 0 <= nc < n
                    and (nr, nc) not in seen
                    and H[nr][nc] <= limit
                ):
                    seen.add((nr, nc))
                    stack.append((nr, nc))
        return False

    values = sorted({v for row in H for v in row})
    lo, hi = 0, len(values) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(values[mid]):
            hi = mid
        else:
            lo = mid + 1
    return values[lo]
```

Time: O(m * n log(m * n)) | Space: O(m * n)

Why

Sort the distinct values; the check 'can we walk from start to goal using only cells at or below this value' is monotone in the value, so binary search the smallest value for which it is true.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

```
def min_effort_path(H):
    """LeetCode 1631: minimise the largest absolute differ
ence between consecutive cells on the path."""
    m, n = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if 0 <= nr < m and 0 <= nc < n:
                nxt = max(cost, abs(H[nr][nc] - H[r][c]))
                if nxt < best.get((nr, nc), float("inf")):
                    best[(nr, nc)] = nxt
                    heapq.heappush(heap, (nxt, nr, nc))
    return 0
```

Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 2], [1, 3]])
```

Test cases

Input	Expected
([[0, 2], [1, 3]],)	3
([[0, 100, 2], [1, 2, 2], [3, 4, 1]],)	2

HARD | IN APP LC 834 | PracHub | Technical Screen | 4 of 196

Compute distance-sum from every tree node

STUDY: TALK | Google frequency on LeetCode: all time 30.5, last 30 days 0 | book priority 112 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #834 Sum of Distances in Tree

LeetCode match: LC 834 Sum of Distances in Tree

Original: https://prachub.com/coding-questions/compute-distance-sum-from-every-tree-node

QUESTION

Overview: This question evaluates understanding of tree and graph algorithms, including computation of pairwise distances and techniques for aggregating path lengths across nodes.

APPROACH

Rerooting DP: one DFS computes subtree sizes and the root's distance sum, a second DFS derives each child as ans[child] = ans[parent] - size[child] + (n - size[child]). O(n).

HOW TO THINK ABOUT IT

Computing the sum of distances from each node separately costs O(n) per node. Instead compute it once for the root, then derive the neighbour's answer in O(1): moving the root from a parent to its child makes every node in the child's subtree one step closer and every other node one step farther. That needs only the subtree sizes.

Basis: LeetCode 834, Sum of Distances in Tree (re-rooting dynamic programming).

BRUTE FORCE - PSEUDO-CODE

```
for every node: run a BFS and add up the distance to all o
ther nodes
```

Time: O(n^2) | Space: O(n)

Why it is slow

n searches of O(n) each, O(n^2). Compute the answer for the root, then move the root edge by edge using subtree sizes (re-rooting), O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(n, edges):
    if n == 1:
        return [0]

    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)

    parent = [-1] * n
    depth = [0] * n
    order = [0]
    parent[0] = 0

    i = 0
    while i < len(order):
        u = order[i]
        i += 1
        for v in adj[u]:
            if v == parent[u]:
                continue
            parent[v] = u
            depth[v] = depth[u] + 1
            order.append(v)

    subtree_size = [1] * n
    root_sum = sum(depth)

    for u in reversed(order[1:]):
        subtree_size[parent[u]] += subtree_size[u]

    ans = [0] * n
    ans[0] = root_sum

    i = 0
    while i < len(order):
        u = order[i]
        i += 1
        for v in adj[u]:
            if v == parent[u]:
                continue
            ans[v] = ans[u] + n - 2 * subtree_size[v]

    return ans
```

Time: O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(n).

```
def tree_eccentricities(n, edges):
    """Maximum distance from every node: the farthest node
```

```
from any node is one end of a diameter."""
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)

    def bfs(s):
        dist, queue = {s: 0}, deque([s])
        last = s
        while queue:
            u = queue.popleft()
            last = u
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        return last, dist

    a, _ = bfs(0)
    b, da = bfs(a)
    _, db = bfs(b)
    return [max(da[v], db[v]) for v in range(n)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]]) -> list[int]:
```

Sample call

```
solution(1, [])
```

Test cases

Input	Expected
(1, [])	[0]
(4, [[0, 1], [1, 2], [1, 3]])	[5, 3, 5, 5]

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Tree Distance Sum (Re-root DP) | 1Point3Acres | onsite-coding | asked 2026-01-19 | Single report

https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

The interviewer explicitly liked the "baseline first, optimize later" structure — name the O(n²) and move on within the first ~10 minutes.

Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].

Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

Preparation: Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.

Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.

Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

HARD | PracHub | Technical Screen | 5 of 196

Compute minimax grid path and network delay

STUDY: TALK | Google frequency on LeetCode: all time 37.9, last 30 days 0 | book priority 89 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1631 Path with Minimum Effort, LeetCode #743 Network Delay Time

LeetCode match: LC 1631 Path With Minimum Effort + LC 743 Network Delay Time (inferred from overview)

Original: https://prachub.com/coding-questions/compute-minimax-grid-path-and-network-delay

QUESTION

Overview: This question evaluates algorithmic problem-solving in shortest-path and path-optimization contexts, specifically minimax pathfinding on a grid and single-source reachability/timing in a weighted directed graph, testing graph modeling, alternative path cost metrics, edge-case handling, and working with large numeric weights.

APPROACH

Part 1: Dijkstra (or binary search + BFS) where cost is the max edge/cell along the path. Part 2: Dijkstra from the source; answer is the max distance, or -1 if some node is unreachable.

INTERVIEW-DAY PLAN

Clarify first

Two parts. Part 1: ask what the path cost is (the maximum cell value along the path, including the start), the allowed moves (4-directional) and the endpoints (top-left to bottom-right). Part 2: ask whether the graph is directed, what to return if some node is unreachable (-1), and the size of weights. The scraped solution covers only part 1.

Brute force to say out loud

Part 1: DFS over all paths, O(4^(mn)). Part 2: Bellman-Ford, which is fine for small graphs.

Optimal idea

Part 1: Dijkstra where the cost of a path is the maximum seen so far, with a min-heap keyed on that maximum: new_cost = max(cost, cell). O(mn log mn). Equivalent: binary search the cap and BFS, or sort cells and union-find until start and end connect. Part 2: Dijkstra from the source; the time for a signal to reach everyone is the largest shortest distance, or -1 if any node is unreachable. O((V + E) log V).

Dry-run these cases

Part 1: [[1,10,6],[1,3,2],[7,2,2]] gives 3 (as in the example), a 1 x 1 grid, a grid where the best path takes a detour. Part 2: source with no outgoing edges, a disconnected node, parallel edges.

Likely follow-ups

Part 1 with the cost as the sum minus something, or paths that may revisit cells. Part 2 with large weights (use 64-bit integers).

Pitfalls

Using the sum instead of the maximum when relaxing. Not counting the start cell's own value.

HOW TO THINK ABOUT IT

Both are 'shortest path' with a different notion of cost. For the grid the cost of a path is its maximum, which still works in Dijkstra: the best way to reach a cell is the one with the smallest maximum, and extending a path can never lower it. For the network, Dijkstra from the source gives every arrival time, and the answer is the largest of them (-1 if some node is never reached).

Basis: Two parts: LeetCode 1631-style minimax path (path cost = the largest cell value on the path, start cell included) and LeetCode 743, Network Delay Time (time for a signal to reach every node, or -1).

BRUTE FORCE - PSEUDO-CODE: PART 1: DFS OVER EVERY SIMPLE PATH

```
depth-first search over every simple path from the top-lef
t to the bottom-right
for each path take the largest cell value on it; answer =
the smallest such maximum
```

Time: exponential | Space: O(path)

Why it is slow

The number of simple paths is exponential. Dijkstra where the cost of a path is its maximum cell (or a union-find over cells added in increasing order) is O(m * n log(m * n)).

BRUTE FORCE - PSEUDO-CODE: PART 2: BELLMAN-FORD FROM THE SOURCE

```
dist[source] = 0, others infinity
repeat V - 1 times: relax every edge (u, v, w)
answer = the largest dist, or -1 if any node is unreachabl
e
```

Time: O(V * E) | Space: O(V)

Why it is slow

V - 1 rounds over all edges, O(V * E). Dijkstra with a heap is O((V + E) log V) for non-negative weights.

OPTIMAL SOLUTION - PART 1: DIJKSTRA WHERE THE COST OF A PATH IS ITS MAXIMUM CELL

```
def minimax_path(H):
    m, n = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if 0 <= nr < m and 0 <= nc < n:
                nxt = max(cost, H[nr][nc])
                if nxt < best.get((nr, nc), float("inf")):
                    best[(nr, nc)] = nxt
                    heapq.heappush(heap, (nxt, nr, nc))
    return -1
```

Time: O(m * n log(m * n)) | Space: O(m * n)

Why

Heap of (best maximum so far, cell). Pop the cell with the smallest bottleneck, and for each neighbour the new bottleneck is max(current, neighbour). Improve if smaller. The first time the goal is popped, its bottleneck is optimal.

OPTIMAL SOLUTION - PART 2: DIJKSTRA FROM THE SOURCE, ANSWER IS THE LARGEST DISTANCE

```
def networkDelayTime(times, n, k):
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))
    dist, heap = {}, [(0, k)]
    while heap:
        d, u = heapq.heappop(heap)
        if u in dist:
            continue
        dist[u] = d
        for v, w in graph[u]:
            if v not in dist:
                heapq.heappush(heap, (d + w, v))
    return max(dist.values()) if len(dist) == n else -1
```

Time: O((V + E) log V) | Space: O(V + E)

Why

Compute shortest distances with a heap. The signal reaches everyone at the time of the farthest node; if fewer than n nodes are reached the answer is -1.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

```
def minimax_path_union_find(H):
    """Alternative: add cells in increasing order of value
 and stop when start and goal are connected."""
    m, n = len(H), len(H[0])
    parent = list(range(m * n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    on = set()
    for val, r, c in sorted((H[r][c], r, c) for r in range
(m) for c in range(n)):
        on.add((r, c))
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if (nr, nc) in on:
                parent[find(r * n + c)] = find(nr * n + nc
)
        if (
            (0, 0) in on
            and (m - 1, n - 1) in on
            and find(0) == find((m - 1) * n + n - 1)
```

```
        ):
            return val
    return -1
```

Part 2: weights are huge.
Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. Cost: unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.
A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

FUNCTION AND TEST CASES

Signature

```
def solution(H: list[list[int]]) -> int:
SAMPLE INPUT
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]
```

Test cases

Input	Expected
`[[1, 10, 6], [1, 3, 2], [7, 2, 2]]`	`3`
`[[5, 1, 7], [4, 8, 2], [3, 6, 0]]`	`6`

HARD | PracHub | Technical Screen | 6 of 196

Minimize travel time with optional meeting point

STUDY: TYPE | Google frequency on LeetCode: all time 37.9, last 30 days 33 | book priority 52 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: the interview site's own solution
Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #2203 Minimum Weighted Subgraph With the Required Paths
LeetCode match: LC 743 Network Delay Time (multi-source Dijkstra); related LC 2203 (inferred from overview)
Original: https://prachub.com/coding-questions/minimize-travel-time-with-optional-meeting-point

QUESTION

Overview: This question evaluates a candidate's understanding of graph shortest-path computation and meeting-point optimization, testing skills in handling weighted edges, aggregating path costs, and reasoning about simultaneous movements on a network.

APPROACH

Run Dijkstra from each traveler's start, then pick the node that minimizes the total (or max, per the statement) travel time; compare against the no-meeting option.

INTERVIEW-DAY PLAN

Clarify first
From the scraped solution the setup is two travelers A and B and a destination. Confirm: do they have to travel together after meeting, do they wait for each other (the meeting time is the later arrival), is the graph directed or undirected, and what to return (the minimum time and the meeting node, with ties going to the smallest node, or -1 and -1 if impossible). My copy has only the overview, so verify these.

Brute force to say out loud
Try every node as a meeting point with a fresh shortest path computation each time. O(n * (E log V)).

Optimal idea
Run Dijkstra from A, Dijkstra from B, and Dijkstra from the destination on the reversed graph (so you get every node's distance to the destination in one pass). For each node m the time is max(distA[m], distB[m]) + distToDest[m]. Take the minimum, with ties to the smallest node. Compare with the no-meeting option if each can also go directly. O((V + E) log V).

Dry-run these cases
A and B start at the same node, a node that only one of them can reach, a directed graph where the way back is longer, two nodes with equal time.

Likely follow-ups
More travelers (Dijkstra from each, take the max), minimize total distance instead of arrival, or an optional meeting point.

Pitfalls
Forgetting to reverse the graph for the destination distances in a directed graph. Using sum instead of max for waiting.

HOW TO THINK ABOUT IT

Break it into shortest paths from three places: from A, from B, and into the destination (a Dijkstra on the reversed graph if directed). For each node m the cost is max(distA[m], distB[m]) + dist(m to destination); take the minimum. Three single-source searches replace trying every meeting node with its own search.

Basis: Written from the scraped solution (confirm it): two travellers A and B, a destination; they may meet at a node (the meeting happens when the later one arrives) and then travel together to the destination; return [best time, best node] with ties to the smallest node, or [-1, -1].

BRUTE FORCE - PSEUDO-CODE

```
d[i][j] = the shortest distance for every pair of nodes (t
riple loop)
for every node m: cost = max(d[A][m], d[B][m]) + d[m][dest
ination]
answer = the smallest cost (ties to the smallest m)
```

Time: O(V^3) | Space: O(V^2)

Why it is slow

All-pairs distances cost O(V^3) when only three single-source distances are needed. Dijkstra from A, from B and into the destination is O((V + E) log V).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(n, edges, startA, startB, dest, directed):
    import heapq

    if n <= 0:
        return [-1, -1]

    adj = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]

    for u, v, w in edges:
        adj[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            adj[v].append((u, w))
            rev[u].append((v, w))

    INF = float("inf")

    def dijkstra(source, graph):
        dist = [INF] * n
        dist[source] = 0
        heap = [(0, source)]

        while heap:
            cur_dist, node = heapq.heappop(heap)
            if cur_dist != dist[node]:
                continue

            for nei, w in graph[node]:
                nd = cur_dist + w
                if nd < dist[nei]:
                    dist[nei] = nd
                    heapq.heappush(heap, (nd, nei))

        return dist

    dist_a = dijkstra(startA, adj)
    dist_b = dijkstra(startB, adj)
    dist_to_dest = dijkstra(dest, rev)

    best_time = INF
    best_node = -1

    for m in range(n):
        if dist_a[m] == INF or dist_b[m] == INF or dist_to
_dest[m] == INF:
            continue

        total = max(dist_a[m], dist_b[m]) + dist_to_dest[m
]

        if total < best_time or (total == best_time and m
< best_node):
            best_time = total
            best_node = m

    if best_node == -1:
        return [-1, -1]

    return [best_time, best_node]
```

Time: O((n + e) log n), where e is the number of edges | Space: O(n + e)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

```
def _dijkstra_all(n, adj, s):
    dist = [float("inf")] * n
    dist[s] = 0
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def meeting_point_total_distance(n, edges, travelers, dest
):
    """Follow-up: K travellers who each walk to the meetin
g node, then everyone goes on together; minimise the sum o
f the walks plus the shared leg (undirected)."""
    adj = [[] for _ in range(n)]
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dists = [_dijkstra_all(n, adj, t) for t in travelers]
    dd = _dijkstra_all(n, adj, dest)
    return min(sum(d[m] for d in dists) + dd[m] for m in r
ange(n))
```

Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]], startA: int,
startB: int, dest: int, directed: bool = False) -> list[in
t]:
```

Sample call

```
solution(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)],
0, 2, 3, False)
```

Test cases

Input	Expected
`(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False)`	`[4, 1]`

Input	Expected
`(4, [(0, 1, 2), (2, 1, 3), (1, 3, 2), (0, 3, 10), (2, 3, 5)], 0, 2, 3, True)`	`[5, 1]`

HARD | IN APP LC 778, 1631 | PracHub | Technical Screen | 7 of 196

Minimize maximum height along a grid path

STUDY: TYPE | Google frequency on LeetCode: all time 40, last 30 days 33 | book priority 48 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #778 Swim in Rising Water, LeetCode #1631 Path with Minimum Effort

LeetCode match: LC 778 Swim in Rising Water / LC 1631

Original: https://prachub.com/coding-questions/minimize-maximum-height-along-a-grid-path

QUESTION

Overview: This question evaluates algorithmic pathfinding and graph-modeling skills, specifically reasoning about grid-based route selection where the objective is defined by an aggregate maximum over visited cell heights.

APPROACH

Min-heap Dijkstra keyed by the max height seen so far (or binary search the cap + BFS, or union-find by sorted cell heights). O(mn log mn).

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest maximum height such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE - PSEUDO-CODE

```
depth-first search over every simple path from the top-left to the bottom-right
for each path take its largest cell; answer = the smallest of those
```

Time: exponential | Space: O(path)

Why it is slow

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(heights):
    import heapq

    if not heights or not heights[0]:
        return 0
    m, n = len(heights), len(heights[0])
    best = [[None] * n for _ in range(m)]
    start = heights[0][0]
    best[0][0] = start
    pq = [(start, 0, 0)]
    while pq:
        cost, r, c = heapq.heappop(pq)
        if cost > best[r][c]:
            continue
        if r == m - 1 and c == n - 1:
            return cost
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < m and 0 <= nc < n:
                ncost = cost if cost > heights[nr][nc] els
e heights[nr][nc]
                if best[nr][nc] is None or ncost < best[nr
][nc]:
                    best[nr][nc] = ncost
                    heapq.heappush(pq, (ncost, nr, nc))
    return best[m - 1][n - 1]
```

Time: O(m*n*log(m*n)) — Dijkstra over the grid graph with m*n nodes and O(m*n) edges, each relaxation costing a log-factor heap operation. | Space: O(m*n) for the best-cost matrix and the priority queue.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Alternative: binary search the limit with a flood fill

```python
def min_max_path_binary_search(H):
    m, n = len(H), len(H[0])

    def ok(limit):
        if H[0][0] > limit:
            return False
        seen, stack = {(0, 0)}, [(0, 0)]
        while stack:
            r, c = stack.pop()
            if (r, c) == (m - 1, n - 1):
                return True
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c +
 1), (r, c - 1)):
                if (
                    0 <= nr < m
                    and 0 <= nc < n
                    and (nr, nc) not in seen
                    and H[nr][nc] <= limit
                ):
                    seen.add((nr, nc))
                    stack.append((nr, nc))
        return False

    values = sorted({v for row in H for v in row})
    lo, hi = 0, len(values) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(values[mid]):
            hi = mid
        else:
            lo = mid + 1
    return values[lo]
```

Time: O(m * n log(m * n)) | Space: O(m * n)

Why

Sort the distinct values; the check 'can we walk from start to goal using only cells at or below this value' is monotone in the value, so binary search the smallest value for which it is true.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

```
def min_effort_path(H):
    """LeetCode 1631: minimise the largest absolute differ
ence between consecutive cells on the path."""
    m, n = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if 0 <= nr < m and 0 <= nc < n:
                nxt = max(cost, abs(H[nr][nc] - H[r][c]))
                if nxt < best.get((nr, nc), float("inf")):
                    best[(nr, nc)] = nxt
                    heapq.heappush(heap, (nxt, nr, nc))
    return 0
```

Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FUNCTION AND TEST CASES

Sample call

```
solution([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Test cases

Input	Expected
([[1, 2, 3], [4, 5, 6], [7, 8, 9]],)	9
([[8, 4, 7], [6, 5, 9], [3, 2, 1]],)	8

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Path Minimizing the Maximum Cell (Swim in Rising Water) | 1Point3Acres | onsite-coding | asked 2026-07-08 | Low

https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

Preparation: Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.

Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

HARD | Exponent | 8 of 196

Implement a decode and encode function of a general N-ary tree.

STUDY: TALK | Google frequency on LeetCode: all time 18.1, last 30 days 0 | book priority 139 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 428 Serialize and Deserialize N-ary Tree (inferred from short statement)

Original: https://www.aced.io/questions/4516/decode-encode-function-general-tree

QUESTION

Implement a decode and encode function of a general N-ary tree.

APPROACH

Pre-order with child counts (value, numChildren) or a sentinel marker for end of children; rebuild recursively/with a queue. O(n).

INTERVIEW-DAY PLAN

Clarify first

Ask for the node shape (value plus list of children), the value type and range, whether values can contain delimiters, how to represent a null tree, and whether the format must be a string.

Brute force to say out loud

Serialize with nested brackets like 1[2,3[4]]. Easy to say but needs a parser and breaks if values contain brackets.

Optimal idea

Pre-order with child counts: write value then number of children, then each child recursively. To decode, read a value and a count, then recursively decode count children. Using a queue or iterator over tokens avoids index bookkeeping. O(n) time and space. For arbitrary string values length-prefix each value.

Dry-run these cases

Empty tree, single node, a deep chain (watch recursion depth), a root with many children, negative or multi-digit values.

Likely follow-ups

Make the format compact, avoid recursion for deep trees (explicit stack), or encode in BFS order.

Pitfalls

Using a plain delimiter that can appear in the data. Not handling the empty tree. Reading tokens past the end on malformed input.

HOW TO THINK ABOUT IT

Serialization questions are about a format that can be read back unambiguously. Ask what a node holds and whether values can contain your delimiters. A node can have any number of children, so the decoder must be told where the children end: either with a closing marker or by writing the child count next to each value. The count form needs no closing markers and can be decoded with an explicit stack, so deep trees are safe.

Basis: LeetCode 428, Serialize and Deserialize N-ary Tree.

BRUTE FORCE - PSEUDO-CODE

```
encode(node) = value + '[' + encodings of its children joi
ned by ',' + ']'
decode: parse the value up to '[', then parse children rec
ursively until the matching
    ']'
```

Time: O(n * h) for building strings (O(n) with a list buffer) | Space: O(n)

Why it is slow

Joining child strings copies text at every level (O(n * h) in the worst case) and the recursive parser can overflow on a deep tree; values containing brackets or commas break it. Pre-order with child counts is O(n) and needs no markers.

OPTIMAL SOLUTION - PRE-ORDER WITH CHILD COUNTS, ITERATIVE DECODE

```
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None e
lse []


def encode(root):
    out, stack = [], [root] if root else []
    while stack:
        n = stack.pop()
        out.append(str(n.val))
        out.append(str(len(n.children)))
        stack.extend(reversed(n.children))
    return " ".join(out)
```

Time: O(n) | Space: O(n)

Why

Write (value, number of children) in pre-order. The decoder reads a value and a count, then must read exactly that many children before returning to the parent, which a stack of (node, children still to read) handles with no recursion. No closing markers are needed.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The values are arbitrary strings that may contain spaces, commas or brackets.

Do not invent delimiters. Put the values in a real container: a JSON list of alternating value and child count, which handles quoting for you, or length-prefix every value (5#hello). The structure part (the counts) stays the same.

```
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None e
lse []


def encode_any(root):
    flat, stack = [], [root] if root else []
    while stack:
        n = stack.pop()
        flat.extend([n.val, len(n.children)])
        stack.extend(reversed(n.children))
    return json.dumps(flat)
```

The tree is huge and you want a compact binary encoding.

Counts and node values can be packed with variable-length integers (varint), which makes small values one byte. Keep the same pre-order count layout so decoding is still one pass. Add a version byte at the front so the format can change later. Cost: still O(n) time and space; variable-length integers cost about one byte per small value.

Encode in level order instead.

Write each node's value and child count in BFS order and decode with a queue: pop a parent, read its count, attach that many new nodes, and push them. BFS order is handy when the consumer wants to stream levels.

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

HARD | 1Point3Acres | onsite-coding, phone-screen | 9 of 196

Friend Graph with Timeline (Friend / Unfriend Events)

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 167 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

QUESTION

Examples:
Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] -> answer 5.
Follow-up 1: after the above events, query (0, A, C) -> false, (5, A, C) -> true.

APPROACH

Have weighted/path-compressed union-find ready and warm.
Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.
Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

INTERVIEW-DAY PLAN

Clarify first

Ask what the answer is for the base question (earliest time all N people are connected), what to return if it never happens, whether unfriend events exist, and whether queries ask about a past time.

Brute force to say out loud

For each query rebuild the graph from the events up to that time and run BFS. O(E + N) per query.

Optimal idea

Base: sort events and union-find, return the time the component count hits 1. Query at a time t: union by size without path compression and record each union's time; two people are connected at t if the largest time on the path between them is at most t. Unfriend: offline, place each edge's lifetime into a segment tree over time and run DFS with a rollback union-find, checking connectivity at the leaves.

Dry-run these cases

The example giving 5. A graph that never connects. Two events at the same time. Unfriend then refriend.

Likely follow-ups

Online unfriend (Holm-de Lichtenberg-Thorup or link-cut trees), counting components over time.

Pitfalls

Using path compression where rollback is needed. Treating unfriend like union-find undo.

HOW TO THINK ABOUT IT

Everyone becomes connected at the first moment the graph has one component. If friendships only appear, sort by time and use union-find (components only merge). Unfriending breaks that: removing an edge can split a component and union-find cannot undo. With all events known up front, handle each edge's lifetime in a segment tree over time and use a rollback union-find.

Basis: LeetCode 1101, The Earliest Moment When Everyone Become Friends, plus connectivity queries over time and unfriend events (offline dynamic connectivity). Assumption: events are (time, a, b, 'friend'|'unfriend').

BRUTE FORCE - PSEUDO-CODE

```
for each distinct event time t in order:
    replay all events up to t to get the current set of fr
iendships
    run a DFS from person 0; if all people are reached: re
turn t
return -1
```

Time: O(T * (N + E)) for T distinct times | Space: O(N + E)

Why it is slow

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

OPTIMAL SOLUTION - BASE: EARLIEST TIME EVERYONE IS CONNECTED (UNION-FIND ON SORTED EVENTS)

```python
def earliest_all_connected(n, events):
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    components = n
    if n == 1:
        return 0
    for t, a, b, kind in sorted(events, key=lambda e: e[0]
):
        if kind != "friend":
            continue
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            components -= 1
            if components == 1:
                return t
    return -1
```

Time: O(E log E) | Space: O(N)

Why

Sort by time and union the two people for each friend event. The first time the component count reaches 1 is the answer; -1 if it never does. Union by size with path compression keeps each operation near constant.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Follow-up 1: were two people connected at time t? (union with timestamps, no path compression)

```python
class TimedUnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.when = [0] * n  # time at which this node was
 attached to its parent

    def union(self, a, b, t):
        ra, rb = self._root(a), self._root(b)
        if ra == rb:
            return
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb], self.when[rb] = ra, t
        self.size[ra] += self.size[rb]
```

```
    def _root(self, x):
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def connected_at(self, a, b, t):
        while a != b:
            # climb the node that was attached earlier (it
 has the smaller 'when' unless it is a root)
            if self.parent[a] == a and self.parent[b] == b
:
                return False
            ka = self.when[a] if self.parent[a] != a else
float("inf")
            kb = self.when[b] if self.parent[b] != b else
float("inf")
            if ka <= kb:
                if ka > t:
                    return False
                a = self.parent[a]
            else:
                if kb > t:
                    return False
                b = self.parent[b]
        return True
```

Time: O(log N) per query | Space: O(N)

Why

Union by size without path compression keeps the union tree shallow (height O(log N)) and lets every union edge carry the time it was added. Two people are connected at time t exactly when the largest edge time on the tree path between them is at most t. Path compression would destroy those per-edge times, which is why it is left out here.

Follow-up 2: unfriend events (offline dynamic connectivity: segment tree over time + rollback DSU)

```
def earliest_all_connected_with_unfriend(n, events):
    if n == 1:
        return 0
    times = sorted({t for t, *_ in events})
    index = {t: i for i, t in enumerate(times)}
    alive, intervals = {}, []
    for t, a, b, kind in sorted(events, key=lambda e: e[0]
):
        key = (min(a, b), max(a, b))
        if kind == "friend":
            alive.setdefault(key, index[t])
        elif key in alive:
            intervals.append((alive.pop(key), index[t] - 1
, key))
    for key, start in alive.items():
        intervals.append((start, len(times) - 1, key))

    tree = [[] for _ in range(4 * max(1, len(times)))]

    def add(node, lo, hi, l, r, edge):
        if l > hi or r < lo or l > r:
            return
        if l <= lo and hi <= r:
            tree[node].append(edge)
            return
        mid = (lo + hi) // 2
        add(2 * node, lo, mid, l, r, edge)
        add(2 * node + 1, mid + 1, hi, l, r, edge)

    for l, r, edge in intervals:
        add(1, 0, len(times) - 1, l, r, edge)
```

```
    parent = list(range(n))
    size = [1] * n
    state = {"components": n}
    history = []
```

```
    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return
        if size[ra] < size[rb]:
            ra, rb = rb, ra
        parent[rb] = ra
        size[ra] += size[rb]
        state["components"] -= 1
        history.append(rb)

    def rollback(mark):
        while len(history) > mark:
            rb = history.pop()
            ra = parent[rb]
            size[ra] -= size[rb]
            parent[rb] = rb
            state["components"] += 1

    answer = [-1]

    def dfs(node, lo, hi):
        if answer[0] != -1:
            return
        mark = len(history)
        for a, b in tree[node]:
            union(a, b)
        if lo == hi:
            if state["components"] == 1:
                answer[0] = times[lo]
```

```
        else:
            mid = (lo + hi) // 2
            dfs(2 * node, lo, mid)
            dfs(2 * node + 1, mid + 1, hi)
        rollback(mark)

    if times:
        dfs(1, 0, len(times) - 1)
    return answer[0]
```

Time: O(E log E log N) | Space: O(N + E log E)

Why

Union-Find cannot undo a union in general, so handle deletions offline. Each friendship is alive for a time interval; insert that interval into O(log T) nodes of a segment tree over the distinct event times. Walk the tree depth first, applying the unions stored at each node (union by size, no path compression, with a rollback stack) and undoing them on the way back up. At each leaf the DSU is exactly the graph at that timestamp, so check whether it has one component.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

```
def components_over_time(n, events):
    """Insert-only events: the number of components after
each event time."""
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    comps, out = n, []
    for t, a, b, kind in sorted(events, key=lambda e: e[0]
):
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb
            comps -= 1
        out.append((t, comps))
    return out
```

Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

NOTES FROM CANDIDATES

The base problem looks innocent; the follow-up about unfriending is the actual signal. In practice, interviewers accept "rebuild graph from scratch per query + BFS" as a working answer if you also articulate why the optimal algorithm is fully dynamic connectivity.
The same family shows up as "friends form a chain over time" or "chess pieces R / L that can move plus walls" — the connectivity backbone is identical.
The follow-up the interviewer is fishing for is consistently a segment tree built over the event timeline; have the offline algorithm ready.
Two algorithmic families converge here: (1) offline — sort all events by time, run union-find, snapshot connectivity at each query T; (2) online with unfriend support — vanilla union-find doesn't handle deletion, so either rebuild per query, use link-cut trees, or apply offline reverse-time processing (treat unfriends as adds in reverse). Clarify with the interviewer which they want.
In an L4 R1 variant, the interviewer requested a graph-and-DFS implementation for the unfriend extension, then continued asking for an optimization that reused the earlier union-find work. Completing the rebuild approach did not end that follow-up.

PREPARATION

Have weighted/path-compressed union-find ready and warm.
Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.
Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns 17 questions

HARD | IN APP LC 315 | HackTheRounds | Phone Screen | 10 of 196

Count of Smaller Numbers After Self

STUDY: TALK | Google frequency on LeetCode: all time 33.2, last 30 days 0 | book priority 103 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #315 Count of Smaller Numbers After Self

LeetCode match: LC 315

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of smaller elements to the right of nums[i].

EXAMPLES

Example 1	
Input	`nums = [5, 2, 6, 1]`
Output	`[2, 1, 1, 0]`
Why	`To the right of 5 there are 2 smaller: [2, 1]. To the right of 2 there is 1 smaller: [1]. To the right of 6 there is 1 smaller: [1]. To the right of 1 there are 0 smaller.`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`[0, 0, 0, 0]`

Example 3	
Input	`nums = [4, 3, 2, 1]`
Output	`[3, 2, 1, 0]`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

APPROACH

Merge sort, binary indexed tree, or segment tree

HOW TO THINK ABOUT IT

For each element you want the count of smaller elements to its right: an order-statistics query on the elements seen so far if you scan right to left. A Fenwick tree over compressed values gives rank counts in O(log n). The same count falls out of merge sort: when a left element is placed after right elements that are smaller, those right elements are exactly the smaller numbers after it.

Basis: LeetCode 315, Count of Smaller Numbers After Self.

BRUTE FORCE - PSEUDO-CODE

```
for each index i: count the elements after i that are smal
ler than nums[i]
```

Time: O(n^2) | Space: O(1) extra

Why it is slow

Two nested loops, O(n^2): too slow at n = 10^5. A Fenwick tree over ranks scanned right to left (or merge sort) is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: MERGE SORT

```python
from typing import List


class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [0] * n
        indices = list(range(n))

        def merge_sort(start, end):
            if end - start <= 1:
                return

            mid = (start + end) // 2
            merge_sort(start, mid)
            merge_sort(mid, end)

            merged = []
            i, j = start, mid
            right_count = 0

            while i < mid and j < end:
                if nums[indices[j]] < nums[indices[i]]:
                    right_count += 1
                    merged.append(indices[j])
                    j += 1
                else:
                    result[indices[i]] += right_count
                    merged.append(indices[i])
                    i += 1

            while i < mid:
                result[indices[i]] += right_count
                merged.append(indices[i])
                i += 1

            while j < end:
                merged.append(indices[j])
                j += 1

            indices[start:end] = merged

        merge_sort(0, n)
        return result
```

Time: O(n log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Do it with merge sort instead of a Fenwick tree.

Sort indices by value with merge sort; whenever you place an element from the left half, add the number of right-half elements already placed (they are smaller). Same O(n log n), and it avoids coordinate compression.

```python
def count_smaller_merge_sort(nums):
    n = len(nums)
    counts = [0] * n
    idx = list(range(n))

    def sort(lo, hi):
        if hi - lo <= 1:
            return
        mid = (lo + hi) // 2
        sort(lo, mid)
        sort(mid, hi)
        merged, i, j, smaller_right = [], lo, mid, 0
        while i < mid or j < hi:
            if j >= hi or (i < mid and nums[idx[i]] <= num
s[idx[j]]):
                counts[
                    idx[i]
                ] += smaller_right  # right elements alrea
dy placed are smaller than this left element
                merged.append(idx[i])
                i += 1
            else:
                smaller_right += 1
                merged.append(idx[j])
                j += 1
        idx[lo:hi] = merged

    sort(0, n)
    return counts
```

Count larger numbers after self, or smaller numbers before self.

Larger after: query the suffix instead of the prefix of the Fenwick tree. Smaller before: scan left to right instead of right to left. Cost: still O(n log n) with the Fenwick tree.

Count inversions in an array.

The sum of the answers above is the number of inversions; merge sort counts them directly in O(n log n).

HARD | IN APP LC 41 | PracHub | Technical Screen | 11 of 196

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

STUDY: TYPE | Google frequency on LeetCode: all time 48, last 30 days 0 | book priority 65 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

APPROACH

Cyclic placement: swap each value v in 1..n into index v-1 until settled, then the first index i where nums[i] != i+1 gives the answer (else n+1). O(n) time, O(1) space.

HOW TO THINK ABOUT IT

The answer is between 1 and n + 1, because n numbers can cover at most n of the positive integers. So the array itself can be used as the 'seen' table: put each value v (1 <= v <= n) at index v - 1. After that, the first index i where nums[i] != i + 1 gives the answer i + 1.

Basis: LeetCode 41, First Missing Positive.

BRUTE FORCE - PSEUDO-CODE

```
seen = the set of all numbers
x = 1; while x is in seen: x = x + 1; return x
```

Time: O(n) | Space: O(n)

Why it is slow

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def first_missing_positive(nums):
    n = len(nums)
    i = 0
    while i < n:
        v = nums[i]
        if 1 <= v <= n and nums[v - 1] != v:
            nums[i], nums[v - 1] = nums[v - 1], v
        else:
            i += 1
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1
```

Time: O(n) | Space: O(1)

Explanation

The answer always lies in [1, n + 1]: if every value 1..n is present the answer is n + 1, otherwise it is the smallest gap. So only values in [1, n] matter, and the list itself can act as the presence table. The reference performs a cyclic placement: at index i, if v = nums[i] is in [1, n] and its home slot nums[v - 1] does not already hold v, swap v into slot v - 1 and re-examine the value that arrived at i; otherwise advance i. Each swap puts at least one value into its final home, and the check nums[v - 1] != v stops duplicates from swapping forever, so there are at most n swaps and the loop is O(n) overall. Afterwards, the first index k with nums[k] != k + 1 gives the answer k + 1; if every slot is correct, the answer is n + 1. Only a few index variables are used beyond the input, which is O(1) auxiliary space.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

```
def first_missing_sign_marking(nums):
    a, n = list(nums), len(nums)
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1
```

The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i]

- (i + 1).

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
`([4, -2, 1, 2],)`	`3`
`([5, 6, 100, -3],)`	`1`

HARD | IN APP LC 2536 | PracHub | Technical Screen | 12 of 196

Count Overlapping Rectangle Updates on a Grid

STUDY: TALK | **Google frequency on LeetCode: all time 18.1, last 30 days 0 | book priority 138 of 196**

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #2536 Increment Submatrices by One

LeetCode match: LC 2536 Increment Submatrices by One

Original: https://prachub.com/coding-questions/count-overlapping-rectangle-updates-on-a-grid

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement `range_add_counts(n, rectangles)`. Start with an `n x n` matrix of zeroes. Each rectangle is represented as `[row1, col1, row2, col2]` with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For `n = 3` and `rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- `1 <= n <= 500`
- `0 <= len(rectangles) <= 100_000`
- `0 <= row1 <= row2 < n`
- `0 <= col1 <= col2 < n`

APPROACH

2D difference array: for each rectangle do +1 at (r1,c1), -1 at (r1,c2+1) and (r2+1,c1), +1 at (r2+1,c2+1); then take 2D prefix sums once. O(n^2 + q) instead of O(n^2*q).

HOW TO THINK ABOUT IT

Adding 1 to every cell of up to 100,000 rectangles one cell at a time is too slow. Record only the corners of each rectangle (+1 at the top-left, -1 just beyond the right and bottom edges, +1 at the bottom-right corner beyond), and rebuild the counts with one pass of 2D prefix sums. Each rectangle then costs O(1) instead of its area.

Basis: LeetCode 2536, Increment Submatrices by One (2D difference array).

BRUTE FORCE - PSEUDO-CODE

```
for every rectangle: for every cell inside it: add 1
```

Time: O(R * area) | Space: O(n^2)

Why it is slow

O(R * area): 100,000 rectangles of area up to 250,000 is billions of cell updates. A 2D difference array touches four corners per rectangle and one prefix-sum pass rebuilds the counts, O(R + n^2).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def range_add_counts(n, rectangles):
    difference = [[0] * (n + 1) for _ in range(n + 1)]
    for row1, col1, row2, col2 in rectangles:
        difference[row1][col1] += 1
        difference[row2 + 1][col1] -= 1
        difference[row1][col2 + 1] -= 1
        difference[row2 + 1][col2 + 1] += 1
    answer = [[0] * n for _ in range(n)]
    for row in range(n):
        for col in range(n):
            above = difference[row - 1][col] if row else 0
            left = difference[row][col - 1] if col else 0
            diagonal = difference[row - 1][col - 1] if row
 and col else 0
            difference[row][col] += above + left - diagona
l
            answer[row][col] = difference[row][col]
    return answer
```

Time: O(n^2 + r), where r is the number of rectangles. | Space: O(n^2) for the difference and result matrices.

Explanation

A rectangle update is represented by adding at its top-left corner, subtracting immediately below and right of its extent, and restoring the overlap at the diagonal corner. After all O(1) updates, a two-dimensional prefix sum turns the difference grid into the exact coverage count for every cell.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How many cells are covered by at least k rectangles?

Build the count matrix with the difference array and count the cells with value at least k. O(R + n^2).

```
def range_add_counts(n, rects):
    d = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    for r in range(n + 1):
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    return [row[:n] for row in d[:n]]


def count_cells_covered_at_least(n, rects, k):
    g = range_add_counts(n, rects)
    return sum(1 for row in g for v in row if v >= k)
```

The grid is huge (10^9 x 10^9) but there are few rectangles.

Coordinate-compress the corner coordinates and apply the same difference array on the compressed grid (at most (2R)^2 cells), or use a sweep line with a segment tree over columns. Cost: O(R log R) to sort the corner coordinates plus up to O(R^2) compressed cells, instead of 10^18 cells.

Rectangles are added and the count of a cell is queried online.

Use a 2D Fenwick tree with range update and point query (the difference-array idea turned into a BIT): O(log^2 n) per operation.

FUNCTION AND TEST CASES

Signature

```
def range_add_counts(n: int, rectangles: list[list[int]])
-> list[list[int]]:
```

Sample call

```
range_add_counts(1, [])
```

Test cases

Input	Expected
(1, [])	[[0]]
(1, [[0, 0, 0, 0]])	[[1]]

HARD | PracHub | Technical Screen | 13 of 196

Find Dictionary Matches for a Jammed String

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 122 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 809 Expressive Words (inferred from overview)

Original: https://prachub.com/coding-questions/find-dictionary-matches-for-a-jammed-string

QUESTION

Overview: This question evaluates string-processing and pattern-matching skills, specifically run-length grouping of consecutive characters and efficient dictionary lookup, within the Coding & Algorithms domain.

APPROACH

Run-length encode the query and every dictionary word; a word matches if run characters align and each run length is compatible. Pre-group dictionary by compressed signature in a hash map for fast lookup.

INTERVIEW-DAY PLAN

Clarify first

Ask what jammed means exactly: a letter repeated more than it should be, and whether the repeat count must be at least the word's count or at least some threshold. Confirm case, empty strings and whether a match requires the same letter order and same number of runs.

Brute force to say out loud

For each dictionary word, compare it with the query run by run. O(W * L) per query.

Optimal idea

Compress each word into runs of (letter, count). Use the letter sequence (without counts) as a hash key so only words with the same run letters are candidates, then compare the counts under the agreed rule. For repeated queries a trie over run letters keeps lookups proportional to the number of runs.

Dry-run these cases

Query 'heeelllo' against 'hello' and 'helo'. A query with fewer runs than the word. A single-letter word. A word whose runs are longer than the query's.

Likely follow-ups

Stretch only allowed for runs of three or more, return all matches in input order, handle a huge dictionary with a trie.

Pitfalls

Treating 'ee' to 'e' as a match without checking the rule. Comparing total length instead of per-run counts.

HOW TO THINK ABOUT IT

'Same letters, stretched' means compare run-length encodings, not characters. The letter order of the runs has to match exactly, so the sequence of distinct run letters is a hash key that cuts the dictionary down before any counts are compared.

Basis: LeetCode 809, Expressive Words. Assumed rule (confirm it): a query run may be longer than the word's run only when the query run has length at least 3; otherwise the lengths must be equal.

BRUTE FORCE - PSEUDO-CODE

```
runs_q = the (letter, count) runs of the query
for each dictionary word: compare its runs with runs_q (sa
me letters in order, counts
    compatible); keep it if all match
```

Time: O(W * L) per query | Space: O(L)

Why it is slow

Every query touches every word, O(W * L), even words whose letters cannot match. Grouping words by their run letters in a hash map checks only the right bucket.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(dictionary_words, jammed_string):
    def compress(s):
        if not s:
            return "", []

        chars = [s[0]]
        counts = [1]
        for ch in s[1:]:
            if ch == chars[-1]:
                counts[-1] += 1
            else:
                chars.append(ch)
                counts.append(1)
        return "".join(chars), counts

    target_sig, target_counts = compress(jammed_string)
    matches = []

    for word in dictionary_words:
        sig, counts = compress(word)
        if sig != target_sig or len(counts) != len(target_
counts):
            continue

        ok = True
        for wc, jc in zip(counts, target_counts):
            if wc > jc:
                ok = False
                break

        if ok:
            matches.append(word)

    return matches
```

Time: O(L + J), where L is the total length of all dictionary words and J is the length of jammed_string | Space: O(G + K), where G is the number of groups in the current string being processed and K is the size of the output list

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

```
class JammedIndex:
    def __init__(self, words):
        self.groups = {}
        for w in words:
            r = _runs(w)
            self.groups.setdefault("".join(ch for ch, _ in
 r), []).append(
                (w, [c for _, c in r])
            )

    def matches(self, query, min_stretch=3):
        r = _runs(query)
        key = "".join(ch for ch, _ in r)
        counts = [c for _, c in r]
        return [
            w
            for w, wc in self.groups.get(key, [])
            if all(
                qc == c or (qc > c and qc >= min_stretch)
for qc, c in zip(counts, wc)
            )
        ]
```

```
def _runs(s):
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return out


def jammed_any_stretch(words, query):
    return JammedIndex(words).matches(query, min_stretch=2
)
```

Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. Cost: O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

FUNCTION AND TEST CASES

Signature

```
def solution(dictionary_words: list[str], jammed_string: s
tr) -> list[str]:
```

Sample call

```
solution(['help', 'hello', 'shell', 'hellp'], 'hellp')
```

Test cases

Input	Expected
`(['help', 'hello', 'shell', 'hellp'], 'hellp')`	`['help', 'hellp']`
`(['hello', 'halo', 'heeello'], 'hheeellllo')`	`['hello', 'heeello']`

HARD | IN APP LC 289 | PracHub | Technical Screen | 14 of 196

Simulate In-Place Cellular State Updates

STUDY: TALK | Google frequency on LeetCode: all time 23.4, last 30 days 0 | book priority 131 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #289 Game of Life

LeetCode match: LC 289 Game of Life (inferred from overview)

Original: https://prachub.com/coding-questions/simulate-in-place-cellular-state-updates

QUESTION

Overview: This question evaluates a candidate's ability to implement grid-based state transitions and manage in-place updates with strict space constraints, assessing skills in matrix manipulation, neighbor traversal, and correctness under simultaneous update rules.

APPROACH

Encode old and new state together in each cell (extra bits or special values like 2 and 3) so neighbors still read the old state, then finalize in a second pass. O(mn) time, O(1) extra space.

HOW TO THINK ABOUT IT

'Update every cell at once, in place' means a cell's new value must not disturb its neighbors' reads. The trick is to store both the old and the new state in one cell (two bits), then decode in a second pass. Whenever you need old and new values together with no extra grid, think 'pack two states into one number'.

Basis: LeetCode 289, Game of Life.

BRUTE FORCE - PSEUDO-CODE

```
old = a copy of the board
for every cell: count live neighbours in old; set the cell
 in the board from the rules
    (live with 2 or 3 neighbours stays, dead with 3 become
s live)
```

Time: O(m * n) | Space: O(m * n)

Why it is slow

Time O(m * n) is already optimal; the cost is a second full board, O(m * n) extra space. Storing the next state in a spare bit of each cell removes the copy.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(grid):
    if not grid or not grid[0]:
        return grid

    rows, cols = len(grid), len(grid[0])
    directions = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0,
 1), (1, -1), (1, 0), (1, 1)]

    for r in range(rows):
        for c in range(cols):
            live_neighbors = 0

            for dr, dc in directions:
```

```
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and g
rid[nr][nc] in (1, 2):
                    live_neighbors += 1

            if grid[r][c] == 1:
                if live_neighbors < 2 or live_neighbors >
3:
                    grid[r][c] = 2  # originally active, w
ill become inactive
            else:
                if live_neighbors == 3:
                    grid[r][c] = 3  # originally inactive,
 will become active

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                grid[r][c] = 0
            elif grid[r][c] == 3:
                grid[r][c] = 1

    return grid
```

Time: O(m * n) | Space: O(1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The board is infinite, or huge and mostly empty.

Store only the live cells in a set. A cell's next state depends on its live neighbor count, so count how many live neighbors every candidate cell has with a Counter and keep cells with 3, or with 2 if already live. Work is proportional to the number of live cells.

```
def life_infinite(live):
    """One generation on an unbounded board, storing only
the live cells."""
    from collections import Counter

    counts = Counter(
        (r + dr, c + dc)
        for r, c in live
        for dr in (-1, 0, 1)
        for dc in (-1, 0, 1)
        if (dr, dc) != (0, 0)
    )
    return {cell for cell, k in counts.items() if k == 3 o
r (k == 2 and cell in live)}
```

Why is bit 0 used when counting neighbors?

Neighbors that were already processed in this pass carry their next state in bit 1. Masking with & 1 always reads the original generation, which is the whole point of the encoding.

Cells wrap around the edges (a torus).

Replace the bounds check with modular indexing: (r + dr) % m and (c + dc) % n. Be careful on very small boards where the same neighbor can be reached through two offsets.

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]])
```

Test cases

Input	Expected
`([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]],)`	`[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]`
`([[1, 1], [1, 1]],)`	`[[1, 1], [1, 1]]`

HARD | PracHub | Technical Screen | 15 of 196

★ TOP 75: rank 4 in the Medium and Hard priority list

★ Check Digits and Combine Ranges

STUDY: TYPE | Google frequency on LeetCode: all time 73.6, last 30 days 63.4 | book priority 2 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #9 Palindrome Number, LeetCode #56 Merge Intervals, LeetCode #228 Summary Ranges

LeetCode match: LC 9 Palindrome Number + LC 56 Merge Intervals / LC 228 Summary Ranges

Original: https://prachub.com/coding-questions/check-digits-and-combine-ranges

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer **`x`**, return **`true`** if its decimal representation reads the same from left to right and from right to left; otherwise return **`false`**. Requirements and edge cases:

- Negative integers should return **`false`** because of the leading minus sign.
- You may assume **`x`** fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals **`intervals`**, where each interval is represented as **`[start, end]`**, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, **`[1, 4]`** and **`[4, 5]`** should be combined into **`[1, 5]`**.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion
Examples
Input: 121
Expected Output: True
Explanation: 121 reads the same forward and backward.
Input: -121
Expected Output: False
Explanation: Negative numbers are not palindromes.
Hints
A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.
Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
Input: [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
Input: [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

APPROACH

Task 1: reverse half the digits and compare (no string, handle negatives and trailing zeros). Task 2: sort ranges and merge or combine consecutive ones in one pass.

INTERVIEW-DAY PLAN

Clarify first

Two independent tasks. For the palindrome: negatives are false, no string conversion, 32-bit input. For the ranges: ask whether touching ranges such as [1, 4] and [4, 5] merge (the statement says they do) and whether adjacent integer ranges like [1, 2] and [3, 4] should also merge.

Brute force to say out loud

Palindrome: convert to a string and compare with its reverse. Ranges: compare every pair repeatedly until nothing merges, O(n^2).

Optimal idea

Palindrome: return false for negatives and for any positive number ending in 0; then build a reversed number from the low digits while x > rev, and compare x with rev or rev // 10 (to skip the middle digit). Only half the digits are processed, so overflow cannot happen. Ranges: sort by start, then sweep, extending the last merged range when the next start is at most its end, else append. O(n log n).

Dry-run these cases

121 true, -121 false, 10 false, 0 true, 1221, 12321. Ranges: [[1,3],[2,6],[8,10],[15,18]] gives [[1,6],[8,10],[15,18]]; [[1,4],[4,5]] gives [[1,5]]; unsorted input; one range; a range fully inside another.

Likely follow-ups

Palindrome in another base, or palindrome linked list. Ranges: insert a new interval into a sorted list (LeetCode 57), summary ranges of sorted integers (LeetCode 228).

Pitfalls

Reversing the whole number and overflowing. Forgetting to sort before merging. Using strict less-than when touching ranges must merge.

HOW TO THINK ABOUT IT

Task 1: a number is a palindrome if its second half reversed equals its first half, so reverse only half the digits (stop when the reversed part catches up) and avoid overflow. Negatives and numbers ending in 0 are never palindromes. Task 2: after sorting by start, an overlap can only be with the last merged range, so one pass with a 'current' range is enough.

Basis: Two independent tasks: LeetCode 9 (Palindrome Number, no string conversion) and LeetCode 56 (Merge Intervals; ranges sharing a point merge).

BRUTE FORCE - PSEUDO-CODE: TASK 1: CONVERT TO A STRING AND COMPARE WITH ITS REVERSE

```
return x >= 0 and str(x) == reverse of str(x)
```

Time: O(d) | Space: O(d)

Why it is slow

O(d) time but it builds strings, which the question forbids. Reversing only the second half of the digits arithmetically (x % 10, x // 10) uses O(1) space.

BRUTE FORCE - PSEUDO-CODE: TASK 2: MERGE ANY TWO OVERLAPPING RANGES UNTIL NONE OVERLAP

```
repeat: find two ranges that share a point; replace them w
ith their union
stop when no pair overlaps; return the ranges sorted
```

Time: O(n^2) | Space: O(n)

Why it is slow

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION - TASK 1: REVERSE ONLY THE SECOND HALF ARITHMETICALLY

```python
def isPalindrome(x):
    if x < 0 or (x % 10 == 0 and x != 0):
        return False
    rev = 0
    while x > rev:
        rev = rev * 10 + x % 10
        x //= 10
    return x == rev or x == rev // 10
```

Time: O(d) | Space: O(1)

Why

Reject negatives and positive numbers ending in 0. Move digits from x to rev (rev = rev * 10 + x % 10, x //= 10) until rev >= x; then x equals rev (even number of digits) or rev // 10 (odd, middle digit dropped). Only half the digits are processed, so rev cannot overflow.

OPTIMAL SOLUTION - TASK 2: SORT BY START, THEN ONE SWEEP

```python
def merge(ranges):
    out = []
    for s, e in sorted(ranges):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Time: O(n log n) | Space: O(n)

Why

Sort by start. For each range, if it starts at or before the end of the last merged range extend that range's end with max(); otherwise append it as a new range. Sorting guarantees nothing earlier can overlap the new range.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

```python
def summary_ranges(nums):
    out = []
    for x in nums:
        if out and x == out[-1][1] + 1:
            out[-1][1] = x
        else:
            out.append([x, x])
    return [f"{a}->{b}" if a != b else str(a) for a, b in
out]
```

Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

FUNCTION AND TEST CASES

Signature

```
def solution(x: int) -> bool:
SAMPLE INPUT
121
```

Test cases

Input	Expected
121	True
-121	False

HARD | IN APP LC 354 | PracHub | Technical Screen | 16 of 196

★ TOP 75: rank 14 in the Medium and Hard priority list

★ Find Maximum Envelope Nesting

STUDY: TYPE | Google frequency on LeetCode: all time 52.8, last 30 days 33 | book priority 25 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #354 Russian Doll Envelopes

LeetCode match: LC 354 Russian Doll Envelopes

Original: https://prachub.com/coding-questions/find-maximum-envelope-nesting

QUESTION

Overview: This question evaluates algorithmic problem-solving skills, particularly reasoning about multi-dimensional ordering, sequence optimization, and time-complexity analysis.

APPROACH

Sort by width asc and height desc on ties, then find the strictly increasing subsequence on heights with binary-search LIS (patience sorting). O(n log n).

HOW TO THINK ABOUT IT

Two dimensions are hard, one is easy. Sort by width so that width order is settled, then the question becomes a longest strictly increasing subsequence of heights. The trap is envelopes with equal width: they cannot nest, so sort equal widths by height descending, which prevents two of them from forming an increasing pair. The LIS is then done with patience sorting (binary search over tails).

Basis: LeetCode 354, Russian Doll Envelopes (strictly larger in both width and height).

BRUTE FORCE - PSEUDO-CODE

```
sort envelopes by width
best[i] = 1 + the largest best[j] over earlier envelopes j
 that are strictly smaller
    in both width and height
answer = the largest best
```

Time: O(n^2) | Space: O(n)

Why it is slow

Every envelope compares with all earlier ones, O(n^2). Sorting by width ascending and height descending, then a longest increasing subsequence on heights with binary search, is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(envelopes):
    from bisect import bisect_left

    if not envelopes:
        return 0

    # Sort by width ascending, and by height descending wh
en widths are equal.
    # This prevents envelopes with the same width from bei
ng counted in the LIS.
    envelopes = sorted(envelopes, key=lambda x: (x[0], -x[
1]))

    lis = []
    for _, height in envelopes:
        idx = bisect_left(lis, height)
        if idx == len(lis):
            lis.append(height)
        else:
            lis[idx] = height

    return len(lis)
```

Time: O(n log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

```python
def nesting_chain(envelopes):
    env = sorted(envelopes, key=lambda e: (e[0], -e[1]))
    tails, tail_idx, parent = [], [], [-1] * len(env)
    for k, (_, h) in enumerate(env):
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
            tail_idx.append(k)
        else:
            tails[i], tail_idx[i] = h, k
        parent[k] = tail_idx[i - 1] if i else -1
    chain, k = [], tail_idx[-1] if tail_idx else -1
    while k != -1:
        chain.append(env[k])
        k = parent[k]
    return chain[::-1]
```

Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

FUNCTION AND TEST CASES

Signature

```
def solution(envelopes: list[list[int]]) -> int:
SAMPLE INPUT
[[5, 4], [6, 4], [6, 7], [2, 3]]
```

Test cases

Input	Expected
[[5, 4], [6, 4], [6, 7], [2, 3]]	3
[]	0

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Russian Doll Envelopes | 1Point3Acres | phone-screen | Single report

https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

Preparation: Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

HARD | IN APP LC 1044 | PracHub | Technical Screen | 17 of 196

Find longest duplicated substring

STUDY: TALK | Google frequency on LeetCode: all time 30.5, last 30 days 0 | book priority 111 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #1044 Longest Duplicate Substring

LeetCode match: LC 1044 Longest Duplicate Substring

Original: https://prachub.com/coding-questions/find-longest-duplicated-substring

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., **`s[i..j)`** for **`0 <= i < j <= len(s)`**. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., **`1 <= n <= 2 × 10^5`**).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive **`O(n^2)`** substring comparison approach.

Examples

- **`s = "banana"`**
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include **`"ana"`**.
- **`s = "abcd"`**
- No duplicated substring, so output should be **`""`** (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

You are given a string s consisting of lowercase English letters. A substring is any contiguous non-empty sequence of characters in s. Return any one of the longest substrings that appears at least twice in s. The two occurrences may overlap. If no non-empty substring appears at least twice, return the empty string. Your algorithm should be substantially better than comparing all pairs of substrings naively.

Constraints

1 <= len(s) <= 2 * 10^5s consists only of lowercase English lettersOverlapping occurrences are allowed

Examples

Input: ('banana',)

Expected Output: 'ana'Explanation: 'ana' appears twice: at indices 1..3 and 3..5. No duplicated substring of length 4 exists.

Input: ('abcd',)

Expected Output: ''Explanation: Every non-empty substring appears only once.

Hints

All duplicated substrings can be represented compactly using a suffix-based structure such as a suffix array or suffix automaton.In a suffix automaton, each state represents substrings with the same set of ending positions. If a state is reached by at least two end positions, the longest substring represented by that state is duplicated.

APPROACH

Binary search on the substring length; check each length with a Rabin-Karp rolling hash plus a set (verify on collision). Alternative: suffix array + LCP. O(n log n).

HOW TO THINK ABOUT IT

If a substring of length L repeats, then a substring of every shorter length repeats too, so 'is there a duplicate of length L' is monotone and the answer length can be binary searched. Each check slides a window of length L over the string with a rolling hash and looks for a repeated hash, verifying the match to guard against collisions.

Basis: LeetCode 1044, Longest Duplicate Substring (binary search on the length with a rolling hash).

BRUTE FORCE - PSEUDO-CODE

```
for length L from n - 1 down to 1: put every substring of
that length into a set; if
    one repeats: return it
```

Time: O(n^3) | Space: O(n^2)

Why it is slow

Each length builds up to n substrings of up to n characters: O(n^3) time and O(n^2) space. If a duplicate of length L exists then so does one of every shorter length, so binary search on L with a rolling hash is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(s):
    n = len(s)
    if n < 2:
        return ""

    # Suffix Automaton fields:
    # transitions[v]: outgoing transitions from state v
    # link[v]: suffix link of state v
    # length[v]: maximum length of a substring represented
 by state v
    # first_pos[v]: one ending position of the longest sub
string represented by state v
    # occ[v]: number of end positions represented by state
 v, computed after propagation
    transitions = [{}]
    link = [-1]
    length = [0]
    first_pos = [-1]
    occ = [0]
    last = 0

    for i, ch in enumerate(s):
        cur = len(length)
        transitions.append({})
        length.append(length[last] + 1)
        link.append(0)
        first_pos.append(i)
        occ.append(1)

        p = last
        while p != -1 and ch not in transitions[p]:
            transitions[p][ch] = cur
            p = link[p]
```

```
        if p == -1:
            link[cur] = 0
        else:
            q = transitions[p][ch]
```

```
            if length[p] + 1 == length[q]:
                link[cur] = q
            else:
                clone = len(length)
                transitions.append(transitions[q].copy())
                length.append(length[p] + 1)
                link.append(link[q])
                first_pos.append(first_pos[q])
                occ.append(0)

                while p != -1 and transitions[p].get(ch) =
= q:
                    transitions[p][ch] = clone
                    p = link[p]

                link[q] = clone
                link[cur] = clone

        last = cur

    # Propagate occurrence counts from longer states to th
eir suffix-link parents.
    # States have max length at most n, so bucket by lengt
h for linear processing.
    buckets = [[] for _ in range(n + 1)]
    for v, l in enumerate(length):
        buckets[l].append(v)

    for l in range(n, 0, -1):
        for v in buckets[l]:
            parent = link[v]
            if parent != -1:
                occ[parent] += occ[v]

    best_len = 0
    best_end = -1

    for v in range(1, len(length)):
```

```
        if occ[v] >= 2 and length[v] > best_len:
            best_len = length[v]
            best_end = first_pos[v]

    if best_len == 0:
        return ""

    return s[best_end - best_len + 1 : best_end + 1]
```

Time: O(n), where n is the length of s. The suffix automaton has at most 2n - 1 states, and occurrence propagation is linear using length buckets. | Space: O(n), for the suffix automaton states, transitions, suffix links, occurrence counts, and buckets.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

```
def longest_dup_suffix_array(s):
    sa = sorted(range(len(s)), key=lambda i: s[i:])
    best = ""
```

```
    for a, b in zip(sa, sa[1:]):
        k = 0
        while a + k < len(s) and b + k < len(s) and s[a +
k] == s[b + k]:
            k += 1
        if k > len(best):
            best = s[a : a + k]
    return best
```

The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. Cost: O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> str:
```

Sample call

```
solution('banana')
```

Test cases

Input	Expected
`('banana',)`	`'ana'`
`('abcd',)`	`''`

HARD | PracHub | Technical Screen | 18 of 196

★ TOP 75: rank 10 in the Medium and Hard priority list

★ Find safe travel intervals between planet influences

STUDY: TYPE | Google frequency on LeetCode: all time 65.2, last 30 days 41.1 | book priority 8 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals (then take gaps)

Original: https://prachub.com/coding-questions/find-safe-travel-intervals-between-planet-influences

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9
Examples
Input: []
Expected Output: [(None, None)]
Explanation: With no planets, no point is unsafe, so the entire real line is safe.
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8, 12]. Their complement is (-infinity, 2), (4, 8), and (12, +infinity).
Hints
First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

APPROACH

Turn each planet into [c-r, c+r], sort, merge overlaps, and the safe intervals are the open gaps between merged intervals plus the two infinite ends. Watch endpoint inclusivity (unsafe intervals are closed).

HOW TO THINK ABOUT IT

The safe points are the complement of the union of the unsafe intervals. Merge the unsafe intervals first (sort by start, extend while they overlap or touch), and the safe intervals are exactly the gaps between consecutive merged intervals plus the two unbounded ends. Endpoints themselves are unsafe because the influence is closed (<=).

Basis: Interval complement (LeetCode 56, Merge Intervals, then take the gaps). Each planet (c, r) makes the closed interval [c - r, c + r] unsafe; safe intervals are the open gaps, possibly unbounded.

BRUTE FORCE - PSEUDO-CODE

```
collect every endpoint c - r and c + r, sorted
for each pair of neighbouring endpoints (and the two outer
 rays): the segment is safe
    if its midpoint is covered by no planet
```

Time: O(n^2) | Space: O(n)

Why it is slow

Each midpoint is tested against every planet, O(n^2). Merging the unsafe intervals after sorting and reading the gaps is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(planets):
    if not planets:
        return [(None, None)]

    intervals = [(c - r, c + r) for c, r in planets]
    intervals.sort()

    merged = []
    for left, right in intervals:
        if not merged or left > merged[-1][1]:
            merged.append([left, right])
        else:
            if right > merged[-1][1]:
                merged[-1][1] = right

    safe = []
    safe.append((None, merged[0][0]))

    for i in range(1, len(merged)):
        prev_right = merged[i - 1][1]
        curr_left = merged[i][0]
        if prev_right < curr_left:
            safe.append((prev_right, curr_left))

    safe.append((merged[-1][1], None))
    return safe
```

Time: O(n log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

```
def safe_intervals(planets):
    merged = []
    for a, b in sorted((c - r, c + r) for c, r in planets)
:
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out


def safe_within_range(planets, lo, hi):
    out = []
    for a, b in safe_intervals(planets):
        s, e = max(a, lo), min(b, hi)
        if s < e:
            out.append((s, e))
    return out
```

Are the endpoints safe?

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

Planets arrive over time.

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

FUNCTION AND TEST CASES

Signature

```
def solution(planets: list[list[int]]) -> list[list[int]]:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`[(None, None)]`
`[(3, 1), (10, 2)]`	`[(None, 2), (4, 8), (12, None)]`

HARD | PracHub | Technical Screen | 19 of 196

Shuffle array using random integer API

STUDY: TALK | Google frequency on LeetCode: all time 14.7, last 30 days 0 | book priority 148 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 384 Shuffle an Array

Original: https://prachub.com/coding-questions/shuffle-array-using-random-integer-api

QUESTION

You are given an array **nums** of length **n** representing a deck of distinct cards. You **do not** have access to a built-in shuffle function, but you are given the following API:

```
rand_int(l, r) # returns a uniformly random integer in [l,
 r], inclusive
```

Task

Implement a function **`shuffle(nums)`** that randomly permutes the array **in place** such that **every one of the `n!` permutations is equally likely**.

Requirements

- You may only use **`rand_int(l, r)`** as your source of randomness.
- The algorithm should run in **`O(n)`** time and **`O(1)`** extra space.
- You may assume **`rand_int`** itself is perfectly uniform and independent between calls. Describe your algorithm, argue why the resulting permutation is uniform, and give the time and space complexity.

APPROACH

Fisher-Yates: for i from n-1 down to 1, j = rand_int(0, i), swap nums[i], nums[j]. Uniform because each permutation has probability 1/n!. Avoid the biased 'swap with random index in [0,n)' version.

INTERVIEW-DAY PLAN

Clarify first

Ask whether rand_int(l, r) is inclusive of r, whether to shuffle in place, and whether the result must be uniformly random over all n! permutations.

Brute force to say out loud

Assign each element a random key and sort. It is uniform but O(n log n).

Optimal idea

Fisher-Yates: for i from n - 1 down to 1, pick j = rand_int(0, i) and swap nums[i] with nums[j]. Each of the n! permutations has probability 1 / n! because position i is filled by one of i + 1 equally likely elements. O(n) time, O(1) extra space.

Dry-run these cases

n = 0 and n = 1. n = 3 with many trials: all six permutations should appear about equally. Say why swapping with a random index in [0, n) is biased (n^n outcomes cannot divide evenly into n! permutations).

Likely follow-ups

Shuffle only k elements (stop the loop early), uniform integer from a coin flip only (rejection sampling), or a weighted shuffle.

Pitfalls

Using j in [0, n) every time. An off-by-one on the inclusive bound.

HOW TO THINK ABOUT IT

'Shuffle' means every one of the n! orders must be equally likely. Count outcomes: a procedure with n^n equally likely random choices cannot give n! equal buckets, so it is biased. Fisher-Yates makes n choices with sizes n, n-1, ..., 1, which multiply to exactly n!.

Basis: LeetCode 384, Shuffle an Array (Fisher-Yates).

BRUTE FORCE - PSEUDO-CODE

```
pair every element with a random number from a large range
sort the pairs by the random number; return the elements i
n that order
```

Time: O(n log n) | Space: O(n)

Why it is slow

O(n log n) time and O(n) extra space, and equal random keys make the result very slightly non-uniform. Fisher-Yates (swap position i with a random position in 0..i) is O(n), in place and exactly uniform.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(nums, random_indices):
    pos = 0

    def rand_int(l, r):
        nonlocal pos
        value = random_indices[pos]
        pos += 1
        return value

    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]

    return nums
```

Time: O(n) | Space: O(1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return only k random elements.

Run Fisher-Yates for k steps from the front (swap position i with a random index in i..n-1) and return the first k. O(k) after copying, still uniform.

```
def sample(nums, k):
    """k uniformly random distinct elements: stop Fisher-Y
ates after k steps."""
    a = list(nums)
    for i in range(k):
        j = rand_int(i, len(a) - 1)
        a[i], a[j] = a[j], a[i]
    return a[:k]
```

Only a coin-flip API exists, no rand_int.

Build a uniform integer in [0, m) by rejection sampling: draw ceil(log2 m) random bits and retry if the number is at least m. That keeps every value equally likely, unlike taking a remainder.

Why is the biased version biased?

It makes n choices of n options, n^n equally likely paths, and n! does not divide n^n for n >= 3, so some permutations must appear more often than others.

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], random_indices: list[int]) -
> list[int]:
```

Sample call

```
solution([1, 2, 3, 4], [1, 1, 0])
```

Test cases

Input	Expected
([1, 2, 3, 4], [1, 1, 0])	[3, 1, 4, 2]
([], [])	[]

HARD | IN APP LC 523 | Exponent | 20 of 196

Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 123 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #523)

LeetCode match: LC 523 Continuous Subarray Sum

+610 interviews

Original:
https://www.aced.io/questions/2765/subarray-sum-multiple-of-k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

APPROACH

Prefix sum mod k with a hash map of first-seen index; a repeat remainder at distance >= 2 means a valid subarray. Handle k = 0 and the remainder-0 case with {0: -1}. O(n).

HOW TO THINK ABOUT IT

A subarray sum is a difference of two prefix sums, and a multiple of k means the two prefix sums have the same remainder mod k. So remember the first index at which each remainder appeared; seeing it again at least two positions later gives a subarray of length at least two. Seed the table with remainder 0 at index -1 for subarrays that start at the beginning.

Basis: LeetCode 523, Continuous Subarray Sum: a subarray of at least two elements whose sum is a multiple of k. Assumed edge cases: nums null or shorter than 2 gives false; k = 0 means the sum must be 0.

BRUTE FORCE - PSEUDO-CODE

```
for every start i: running sum; for every end j > i: if th
e sum is a multiple of k:
    return true
return false
```

Time: O(n^2) | Space: O(1)

Why it is slow

O(n^2) pairs. Two prefix sums with the same remainder mod k bound a subarray whose sum is a multiple of k, so a dictionary of first-seen remainders is O(n).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #523 CONTINUOUS SUBARRAY SUM)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Return true if there's a continuous subarray of length >= 2 whose sum is a multiple of k. Right?"

"A few quick questions:
k can be 0? If k=0, check if any subarray sums to 0. k<0? Treat as |k|."

"Let me walk: nums=[23,2,4,6,7], k=6 -> [2,4] sums to 6=6*1 ok -> true ok

PHASE 3 — OPTIMIZE

Say: "Prefix sum mod k + hash map — O(n). Walk-through:
prefix_mod = 0. seen = {0: -1}. (mod 0 seen at index -1: empty prefix)
For i in 0..n-1:
prefix_mod = (prefix_mod + nums[i]) % k.
If prefix_mod in seen:
If i - seen[prefix_mod] >= 2: return True. (subarray length >= 2)
Else: seen[prefix_mod] = i. (store first occurrence only)
Return False.
WHY prefix mod: sum(nums[l..r]) % k == 0 prefix[r] % k == prefix[l-1] % k.
WHY store first occurrence: maximize subarray length -> earliest index with same mod.
WHY {0:-1}: handles case where prefix sum from index 0..i is divisible by k.
WHY length >= 2: problem requires at least 2 elements.
Time: O(n). Space: O(k).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _523_ContinuousSubarraySum:
    def checkSubarraySum(self, nums, k):
        remainder_to_index = {0: -1}
        prefix_sum = 0
        for i, num in enumerate(nums):
            prefix_sum += num
            remainder = prefix_sum % k if k != 0 else pr
efix_sum
            if remainder in remainder_to_index:
                if i - remainder_to_index[remainder] >=
2:
                    return True
            else:
                remainder_to_index[remainder] = i
        return False
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace the derived formula and confirm it on the example numbers."

nums=[23,2,4,6,7], k=6: rem_map={0:-1}.
i=0(23):rem=23%6=5,add{5:0}. i=1(2):prefix=25,rem=1,add{1:1}.
i=2(4):prefix=29,rem=5. 5 in map at 0. i-0=2>=2 -> True ok

Seeding {0:-1} handles subarrays starting at index 0
Edge: nums=[23,2,6,4,7], k=6 -> False ok. Prefix=0 seed at index -1 enables subarrays from index 0 to be caught.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n). Space O(min(n,k)) for the remainder map.
Follow-up: Subarray Sum Equals K (#560) — exact sum (not divisible); same prefix-sum pattern.
Follow-up: Subarray Sums Divisible by K (#974) — count such subarrays.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Count the subarrays whose sum is a multiple of k (any length, LeetCode 974).

Count how many times each remainder has occurred and add that count each time the remainder repeats (Python's % is already non-negative for negative sums). O(n).

```
def count_good_subarrays(nums, k):
    seen, total, count = Counter({0: 1}), 0, 0
    for x in nums:
        total += x
        key = total % k
```

```
        count += seen[key]
        seen[key] += 1
    return count
```

Why store only the first index of each remainder?

A later occurrence of the same remainder would make the subarray shorter, never longer, so the first index is the best partner for the length-at-least-two condition.

The values can be negative or k huge.

Python's modulo gives a non-negative remainder; in languages where % can be negative normalise with ((s % k) + k) % k. A huge k just means the dictionary holds up to n entries. Cost: O(n) time with up to min(n, k) dictionary entries.

OFFICIAL PAGE TEXT

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples

```
nums = [23, 2, 4, 7], k = 6
output: true

nums = [5, 0, 0, 0], k = 3
output: true

nums = null, k = 1
output: false
```

In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?Our solution processes the input array by iterating through it while maintaining a hash map (remainder_index_map) of remainders and their corresponding indices. For each element in the array, we accumulate the sum (sum_so_far) and compute the remainder when divided by k. We check if this remainder has been encountered before in the hash map. If the remainder is not in the hash map, we store the current index associated with this remainder. If the remainder has been seen before, we calculate the distance between the current index and the index of the previously seen remainder. If this distance is greater than or equal to the minimum subarray length (MIN_GOOD_SUBARRAY_LIMIT), we return True, indicating that a subarray with the required properties exists. If the loop completes without finding such a subarray, we return False. The use of a hash map ensures that each element is processed efficiently, as it allows constant-time lookups and insertions. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (hash map lookups and insertions) for each element. Space Complexity: The solution uses O(n) space for the hash map that stores the remainders and their indices. In the worst case, all remainders are unique, resulting in the hash map containing n entries.Let's say you are given an integer array nums and an integer k. Return true if nums has a subarray where its length is at least two, and the sum of the elements in the subarray is a multiple of k. Watch Neeraj, a Senior Engineering Manager, give his answer to this software engineering interview question.No answers contributed by the community yet.Related questionsGiven an array of n integers and a number k, find the maximum number of elements whose sum is less than k, and return their sum.Given an array of integers and an integer k, find the number of continuous subarrays whose sum equals k.Given an array of integers and a number N, find the length of the longest contiguous subarray such that the difference between any two elements in the subarray is less than N.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

---

HARD | IN APP LC 4 | Exponent | 21 of 196

★ TOP 75: rank 6 in the Medium and Hard priority list

★ Find the median of two sorted arrays.

STUDY: TYPE | Google frequency on LeetCode: all time 74.2, last 30 days 61 | book priority 3 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays (inferred from short statement)

Original:
https://www.aced.io/questions/4322/median-of-two-sorted-arrays

QUESTION

Find the median of two sorted arrays.

APPROACH

Binary search the partition on the shorter array so left halves hold the lower half; compare cross elements. O(log(min(m, n))).

HOW TO THINK ABOUT IT

Because both arrays are sorted you never need to merge them. The median splits the combined data into a left half and a right half of equal size; you only have to decide how many elements the shorter array contributes to the left half. Whether a split is right depends on four border values, and moving the split one way or the other is a binary search.

Basis: LeetCode 4, Median of Two Sorted Arrays.

BRUTE FORCE - PSEUDO-CODE

```
merged = a + b, then sort merged
if len(merged) is odd: return the middle element
else: return the average of the two middle elements
```

Time: O((m + n) log(m + n)) | Space: O(m + n)

Why it is slow

It sorts m + n numbers from scratch, O((m + n) log(m + n)), and stores them all, even though both inputs were already sorted. The partition binary search needs only O(log min(m, n)).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #4 MEDIAN OF TWO SORTED ARRAYS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Merge two sorted arrays conceptually and find the median. O(log(m+n)) required. Right?
Median: if total length is odd -> middle element. Even -> average of two middles."
What should I return if the target is not found? Is the array always sorted?

PHASE 3 — OPTIMIZE (binary search on partition — true O(log(min(m,n))))

Partition both arrays into left and right halves such that:
total left size = (m+n+1)//2, max(left) <= min(right).
Binary search on the partition point of the smaller array.
At each partition: check if nums1[mid-1] <= nums2[r-mid] and nums2[r-mid-1] <= nums1[mid].
Median = max(left halves) for odd total, or average of max(left)/min(right) for even."
Key insight: binary search on the partition index (not the value) finds the balanced split where max(left) <= min(right) in O(log min(m,n)) — no merging needed.

PHASE 4 — CLEAN CODE

```python
class _4_MedianTwoArrays:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1
        m, n   = len(nums1), len(nums2)
        half   = (m + n + 1) // 2
        lo, hi = 0, m

        while lo <= hi:
            i = (lo + hi) // 2          # partition of nums1
            j = half - i                # partition of nums2
            left1  = nums1[i - 1] if i > 0 else float('-inf')
            right1 = nums1[i]     if i < m else float('inf')
            left2  = nums2[j - 1] if j > 0 else float('-inf')
            right2 = nums2[j]     if j < n else float('inf')

            if left1 <= right2 and left2 <= right1:
                if (m + n) % 2 == 1:
                    return float(max(left1, left2))
                return (max(left1, left2) + min(right1, right2)) / 2
            elif left1 > right2:
                hi = i - 1
            else:
                lo = i + 1
        return 0.0
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace findMedianSortedArrays([1,3],[2])."
Merged would be [1,2,3]. Median = 2.0. But we binary search on the shorter array.
Partition A=[1,3] at mid=1: A_left=[1], A_right=[3]. Partition B=[2] at 1: B_left=[2], B_right=[].
maxLeftA=1, minRightA=3, maxLeftB=2, minRightB=inf.
1<=inf ok and 2<=3 ok -> valid partition.
total_left=(2+1)//2=1? No: (2+1)//2=1 element each side. Hmm n=3, total_left=2.
Retrace: len(A)=2,len(B)=1. Binary search on A: lo=0,hi=2.
mid=1. B_part=2-1=1.
maxLeftA=A[0]=1, minRightA=A[1]=3. maxLeftB=B[0]=2, minRightB=inf.
Check: maxLeftA(1)<=minRightB(inf) ok.
maxLeftB(2)<=minRightA(3) ok. Valid.
total n=3 (odd): median=max(maxLeftA,maxLeftB)=max(1,2)=2.0 ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(log(min(m,n))). Space O(1). Binary search on the smaller array's partition.
The +inf/-inf sentinels handle edge cases (empty left/right) cleanly.
The brute-force merge approach is often acceptable in interviews — present both and let the interviewer choose.
Given more time I'd ask: kth element of two sorted arrays? Binary search on k — eliminate k//2 elements from one array per

step. Same O(log k) idea.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if there are k sorted arrays instead of two?

Binary search on the answer value instead of on a position. For a candidate value v, count how many elements are <= v in each array with bisect; the median is the smallest v whose total count reaches the middle rank. That costs O(k log n log(range)) time and O(1) extra space and needs no merging.

```
def _kth_across(arrays, k):
    lo = min(a[0] for a in arrays if a)
    hi = max(a[-1] for a in arrays if a)
    while lo < hi:
        mid = (lo + hi) // 2
        if sum(bisect.bisect_right(a, mid) for a in arrays
) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


def find_median_of_k_sorted_arrays(arrays):
    total = sum(len(a) for a in arrays)
    if total % 2:
        return float(_kth_across(arrays, total // 2 + 1))
    return (_kth_across(arrays, total // 2) + _kth_across(
arrays, total // 2 + 1)) / 2
```

What if the arrays are too large for memory?

The two-array binary search only touches O(log n) positions, so with random access (an indexed file or a remote array) it reads almost nothing. If only sequential access is possible, do a streaming selection: a two-pointer merge that stops after (m + n) / 2 elements, using O(1) memory and O(m + n) time.

Find the k-th smallest element, not just the median.

The same partition search: choose i elements from the shorter array and k - i from the other, and move i until the borders are in order. The median is just k = (m + n + 1) // 2.

```
def kth_of_two(a, b, k):
    """k-th smallest (1-based) of two sorted arrays in O(l
og min(m, n))."""
    if len(a) > len(b):
        a, b = b, a
    lo, hi = max(0, k - len(b)), min(k, len(a))
    while lo < hi:
        i = (lo + hi) // 2
        if a[i] < b[k - i - 1]:
            lo = i + 1
        else:
            hi = i
    i = lo
    return max(([a[i - 1]] if i else []) + ([b[k - i - 1]]
 if k - i else []))
```

OFFICIAL PAGE TEXT

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade now

---

HARD | IN APP LC 68 | Exponent | 22 of 196

★ TOP 75: rank 56 in the Medium and Hard priority list

★ Implement a function to justify a list of words within a given width.

STUDY: TYPE | Google frequency on LeetCode: all time 44.7, last 30 days 0 | book priority 71 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #68)

LeetCode match: LC 68 Text Justification (inferred from short statement)

Original: https://www.aced.io/questions/4903/function-justify-words-within-width

QUESTION

Implement a function to justify a list of words within a given width.

APPROACH

Greedily pack words per line; distribute spaces evenly (extra to the leftmost gaps), left-justify the last line and single-word lines. O(total chars).

HOW TO THINK ABOUT IT

Two decisions, kept separate: which words go on each line (pack greedily while the words plus one space between them fit), and how to spread the leftover spaces (an equal share in each gap, the extra spaces to the leftmost gaps). The last line and one-word lines are the special cases to name before coding.

Basis: LeetCode 68, Text Justification.

BRUTE FORCE - PSEUDO-CODE

```
i = 0; while i < len(words):
    j = i + 1; while joining words[i..j] with single space
s still fits in width: j
        = j + 1
    spread the spare spaces over the gaps (extra ones to t
he left); the last line
        and one-word lines are left-aligned
    i = j
```

Time: O(total characters * words per line) | Space: O(width)

Why it is slow

Every attempt to add a word re-joins the whole line to measure it, so each line costs about (words per line) times its length. Tracking only a running length makes it linear in the total characters.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #68 TEXT JUSTIFICATION)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Arrange words into lines of exactly maxWidth chars. Spaces distributed as evenly as possible.
Extra spaces go to the LEFT. Last line: left-justified with single spaces. Right?"

"A few quick questions: Each word fits in maxWidth? Yes per constraints.
At least one word? Yes."

"Let me walk: words=['This','is','an','example'], maxWidth=16.
Line 1: 'This is an' (total=16) ok. Last: left-justified.

PHASE 3 — OPTIMIZE

Say: "Greedy line filling + space distribution — O(n * L).
Walk-through:
Greedily fill each line: pack as many words as fit (with single spaces between).
For each filled line:
If last line OR only one word: left-justify (spaces on right, single space between words).
Else: distribute remaining spaces evenly:
gaps = len(words) - 1. extra = maxWidth - sum(word lengths).
spaces_per_gap = extra // gaps. leftover = extra % gaps.
First 'leftover' gaps get one extra space.
WHY leftover to left gaps: problem specifies extra spaces go to left gaps first.
HOW to fill a line: maintain curr_width, keep adding words until next word would overflow.
Time: O(n * maxWidth). Space: O(maxWidth) per line.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _68_TextJustification:
    def fullJustify(self, words, maxWidth):
        result = []
        i = 0
        while i < len(words):
            line = [words[i]]; length = len(words[i]); i
 += 1
            while i < len(words) and length + 1 + len(wo
rds[i]) <= maxWidth:
                length += 1 + len(words[i]); line.append
(words[i]); i += 1
            if i == len(words) or len(line) == 1:
                result.append(' '.join(line).ljust(maxWi
dth))
            else:
                spaces = maxWidth - sum(len(w) for w in
line)
                gaps = len(line) - 1
                base, extra = divmod(spaces, gaps)
                row = line[0]
                for j in range(1, len(line)):
                    row += ' ' * (base + (1 if j <= extr
a else 0)) + line[j]
                result.append(row)
        return result
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace the derived formula and confirm it on the example numbers."

words=['This','is','an','example'], maxWidth=16:
line=['This','is','an'], length=10. spaces=6, gaps=2.
base=3,extra=0.
row='This is an'. len=16 ok
Last line='example '.ljust(16) ok

Single word line: 'word'.ljust(16) -> padded with spaces ok

Base + (1 if j <= extra else 0) distributes extra spaces to left gaps

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n*maxWidth): O(n) lines, each O(maxWidth) to build.
Space O(n) output.
Follow-up: minimize raggedness (aesthetic) — DP approach balances line lengths.
Follow-up: proportional fonts — use pixel widths instead of character counts.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How would you change it to fully justify the last line too, or to center the text?

Only the final-line branch changes. For full justification drop the special case for the last line (it would be spread like every other line). For centering, split the leftover spaces between the two sides, giving the extra space to the right. The greedy packing step stays identical.

What if the words contain wide characters or different pixel widths?

Replace len(word) by a measure(word) function (for example font metrics or East Asian width) and compare against the pixel width.

Packing and distribution logic stay the same; spaces are measured in the same unit, so the spare width may not divide evenly and the remainder must be assigned to gaps in whole space units.

Fully justify the last line too.

Drop the special case for the last line (a line with more than one word is spread like the others). Single-word lines stay left-aligned.

```
def fullJustify(words, max_width):
    lines, i = [], 0
    while i < len(words):
        j, length = i, 0
        while j < len(words) and length + len(words[j]) +
(j - i) <= max_width:
            length += len(words[j])
            j += 1
        gaps = j - i - 1
        if j == len(words) or gaps == 0:
            line = " ".join(words[i:j])
            line += " " * (max_width - len(line))
        else:
            spaces, extra = divmod(max_width - length, gap
s)
            line = ""
            for k in range(i, j - 1):
                line += words[k] + " " * (spaces + (1 if k
 - i < extra else 0))
            line += words[j - 1]
        lines.append(line)
        i = j
    return lines


def justify_all_lines(words, W):
    lines = fullJustify(words, W)
    if not lines:
        return lines
    # recompute which words are on the last line and sprea
d them like the others
    j, rows = 0, []
    while j < len(words):
        k, length = j, 0
        while k < len(words) and length + len(words[k]) +
(k - j) <= W:
            length += len(words[k])
            k += 1
        rows.append(words[j:k])
        j = k
    last = rows[-1]
    if len(last) > 1:
        letters = sum(len(w) for w in last)
        sp, extra = divmod(W - letters, len(last) - 1)
        lines[-1] = (
            "".join(
                w + " " * (sp + (1 if t < extra else 0))
                for t, w in enumerate(last[:-1])
            )
            + last[-1]
        )
    return lines
```

Words have pixel widths (a proportional font).

Replace len(word) with a measure function and compare against the pixel width; the extra space is distributed in whole space units, so the remainder rule stays the same.

Minimise raggedness instead of packing greedily.

That is the Knuth-Plass style DP: cost of a line = (spare space)^2, minimise the total over all ways to break the text. O(n * words per line).

HARD | PracHub | Technical Screen | 23 of 196

Convert Valid Snake Case to Lower Camel Case

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 165 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy string manipulation)

Original: https://prachub.com/coding-questions/convert-valid-snake-case-to-lower-camel-case

QUESTION

Convert Valid Snake Case to

Lower Camel Case Given a valid snake-case identifier, convert it to lower camel case. The input consists of one or more lowercase ASCII words separated by single underscores. There is no leading underscore, trailing underscore, or pair of adjacent underscores. The output keeps the first word lowercase, removes each underscore, and capitalizes the first letter of every later word. Implement:

```
def solve(name: str) -> str: ...
```

Constraints

- `1 <= len(name) <= 100_000`
- `name` matches `[a-z]+(?:_[a-z]+)*`

Examples

```
"foo_bar" -> "fooBar" "already" -> "already" "http_api_nam
e" -> "httpApiName"
```

APPROACH

Split on '_', keep the first word lowercase, capitalize the first letter of the rest, join. O(n). The Hard tag on PracHub looks like a labeling quirk.

INTERVIEW-DAY PLAN

Clarify first

Ask whether input can have digits, uppercase, leading or trailing underscores, or double underscores. The problem states it is valid, so say you will rely on that and mention what you would add if not.

Brute force to say out loud

Loop over characters and capitalize the character after each underscore. Linear and simple.

Optimal idea

Split on '_', keep the first word as is, capitalize the first letter of each other word and join. O(n). The Hard label on PracHub looks like a labeling quirk; do not over-engineer it.

Dry-run these cases

A single word, two words, many words, a one-letter word.

Likely follow-ups

Convert back from camel to snake, or handle acronyms.

Pitfalls

Capitalizing the first word. Using replace on every underscore without capitalizing.

HOW TO THINK ABOUT IT

The statement says the input is valid (single underscores, no edge underscores), so rely on that and say what you would add if it were not. The pattern is 'first word unchanged, capitalize the first letter of each later word'.

Basis: Written. The PracHub Hard label looks like a labeling quirk; this is an easy string task.

BRUTE FORCE - PSEUDO-CODE

```
up = False; for each character: if it is '_': up = True
else: append the character (uppercased if up); up = False
```

Time: O(n) | Space: O(n)

Why it is slow

Already O(n); it is just longer than needed. Splitting on '_' and capitalising each word after the first is the short form.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solve(name):
    words = name.split("_")
    return words[0] + "".join(word[0].upper() + word[1:] f
or word in words[1:])
```

Time: O(n) time. | Space: O(n) for the returned string.

Explanation

Scan once. Copy ordinary letters, omit underscores, and uppercase exactly the letter following each underscore. The first word is copied unchanged.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

```
def snake_to_camel_robust(s):
    words = [w for w in s.split("_") if w]
    return (
        (words[0].lower() + "".join(w.capitalize() for w i
n words[1:])) if words else ""
    )
```

Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

```
def camel_to_snake(s):
    out = []
    for ch in s:
        if ch.isupper():
            out.append("_")
        out.append(ch.lower())
    return "".join(out)
```

FUNCTION AND TEST CASES

Signature

```
def solve(name) -> str:
```

Sample call

```
solve('a')
```

Test cases

Input	Expected
`('a',)`	`'a'`
`('already',)`	`'already'`

ALSO REPORTED - 2 MORE REPORTS OF THIS QUESTION

Define the scope and requirements for a snake_case to camelCase converter function. | Exponent

https://www.aced.io/questions/6328/define-scope-snake-case-camel-case-converter

Snake Case to Camel Case with Repository-Wide Refactor | 1Point3Acres | tech-screen | asked 2026-07-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.

Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.

A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

Preparation: Implement the identifier conversion and enumerate edge cases before expanding scope.

Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

HARD | Exponent | 24 of 196

Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 147 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1153 String Transforms Into Another String

LeetCode match: No direct match; related LC 1153 String Transforms Into Another String (inferred from short statement)

Original: https://www.aced.io/questions/6330/determine-string-transformability-allowed-operations

QUESTION

Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

APPROACH

Position swaps make order irrelevant, so only the multiset/character mapping matters: build a char -> char map (must be consistent), and the conversion graph must have a free letter to break cycles if all 26 letters are used.

INTERVIEW-DAY PLAN

Clarify first

The wording fits LeetCode 1657, Close Strings (swap any two characters, or swap the identities of two letters that exist). If the second operation is a one-directional conversion of all occurrences of a letter (LeetCode 1153), the answer changes, so ask which one.

Brute force to say out loud

BFS over all strings reachable by the allowed operations. Exponential.

Optimal idea

1657 reading: the answer is true when both strings use the same set of letters and their sorted frequency lists are equal, because swaps reorder freely and the second operation permutes frequencies among existing letters. 1153 reading: each source letter must map to a single target letter, and if the strings differ and the target uses all 26 letters there is no spare letter to break a cycle, so it is impossible. Both are O(n).

Dry-run these cases

abc and bca (true), a and aa (false), cabbba and abbccc (true), uau and ssx (false); for 1153, aabcc to ccdee (true) and a full alphabet rotation (false).

Likely follow-ups

Both operations combined with one-directional conversion, which reduces to grouping letter counts, or returning the sequence of operations.

Pitfalls

Using the 1657 test when the conversion is one-directional. Comparing frequencies without comparing the letter sets.

HOW TO THINK ABOUT IT

Decide which operations you really have. If you can swap any two letters (positions) and swap the identities of two letters, the answer depends only on which letters appear and how often (the multiset of counts). If conversion is one-way (every a becomes b), letters can merge but never split, and cycles in the mapping need a spare letter. Name the reading before you code.

Basis: The wording matches LeetCode 1657, Close Strings (swap any two characters, or swap the identities of two existing letters). The dataset also relates LeetCode 1153 (one-directional conversion, no swaps), so both readings are solved.

BRUTE FORCE - PSEUDO-CODE

```
start from the first string; queue = [start], seen = {star
t}
for each string popped: apply every swap of two positions
and every swap of two
    letters' identities; add unseen results
return whether the second string was reached
```

Time: exponential | Space: exponential

Why it is slow

The number of reachable strings grows exponentially with the length. Because swaps make order irrelevant, comparing the sets of letters and their sorted counts decides it in O(n).

OPTIMAL SOLUTION - READING 1 (LEETCODE 1657): SAME LETTER SET AND SAME MULTISET OF FREQUENCIES

```python
def close_strings(s, t):
    return set(s) == set(t) and sorted(Counter(s).values()
) == sorted(
        Counter(t).values()
    )
```

Time: O(n) | Space: O(1) for a fixed alphabet

Why

Operation 1 reorders positions freely, so only the multiset of letters matters. Operation 2 exchanges the identities of two letters that exist, which permutes the frequencies among existing letters. Therefore s can become t exactly when both use the same set of letters (a letter cannot be created or destroyed) and their sorted frequency lists are equal.

OPTIMAL SOLUTION - READING 2 (LEETCODE 1153): ONE-DIRECTIONAL CONVERSION, NO SWAPS

```python
def can_convert(s, t, alphabet_size=26):
    if s == t:
        return True
    if len(s) != len(t):
        return False
    mapping = {}
    for a, b in zip(s, t):
        if mapping.setdefault(a, b) != b:
            return False
    return len(set(t)) < alphabet_size
```

Time: O(n) | Space: O(1)

Why

Each source letter must map to exactly one target letter (a function), otherwise it is impossible. If s != t and t already uses all 26 letters,

no spare letter exists to break mapping cycles, so it is impossible. Otherwise convert in an order that resolves chains, using a spare letter for any cycle.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if the two operations are combined (one-directional conversion plus swaps)?

Swaps make positions irrelevant, so only letter counts matter. Conversions can only merge letter classes, never split one. So t is reachable only if the counts of t can be formed by grouping the counts of s into the t letters (each s letter's count goes wholly to one t letter) and, unless the relabeling is an identity or chain, there is a spare letter. With at most 26 classes the grouping is a small bin-packing search.

Why does a full 26-letter target make the one-directional version impossible?

Every conversion rewrites a letter into another letter, and if a mapping contains a cycle (a to b and b to a) you need a temporary letter not currently in use to break it. If t uses all 26 letters then s must use all 26 as well (the mapping is a function onto 26 distinct letters), so the mapping is a permutation and any non-identity permutation has a cycle. The only possible case is therefore s equal to t.

Only position swaps are allowed.

Then s can become t exactly when they are anagrams: compare the letter counts. O(n).

```
def only_swaps_possible(s, t):
    return Counter(s) == Counter(t)
```

Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode 1153).

Each source letter must map to one target letter, and if the strings differ and the target already uses all 26 letters there is no spare letter to break mapping cycles, so it is impossible. Verified against brute force on a small alphabet.

```
def can_convert(s, t, alphabet_size=26):
    if s == t:
        return True
    if len(s) != len(t):
        return False
    mapping = {}
    for a, b in zip(s, t):
        if mapping.setdefault(a, b) != b:
            return False
    return len(set(t)) < alphabet_size
```

Return the sequence of operations, or the minimum number.

Minimum operations for the swap-only case is n minus the number of cycles in the permutation that sorts s into t; for letter-identity swaps it is the number of cycles in the letter mapping minus one per cycle of length greater than one.

HARD | 1Point3Acres | onsite-coding | 25 of 196

Necklace Cut into Two Halves with Equal D/R Counts

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 169 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

QUESTION

Examples:

```
s = "DRDR"    -> cut between index 1 and 2: "DR" + "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR"); rotate
, two cuts at offsets 1 and 3:
               pieces "DR" and "DR".
```

APPROACH

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

INTERVIEW-DAY PLAN

Clarify first

Ask whether the string is circular, whether the two pieces must be contiguous arcs, whether equal counts of each letter or equal strings are needed, and whether one cut or two are allowed.

Brute force to say out loud

Try every pair of cut positions and count letters. O(n^3) or O(n^2) with prefix counts.

Optimal idea

Each arc must hold half of the D's and half of the R's, so the total counts of both must be even and each arc has length n / 2. Slide a window of length n / 2 around s + s tracking its D count; the window starting at i and the one starting at i + n / 2 are complementary, and counts change by at most one per step, so some start hits exactly half. O(n).

Dry-run these cases

DRDR, DDRR (two cuts needed), an odd D count (impossible), a string of one letter.

Likely follow-ups

Equal strings instead of equal counts, or three pieces.

Pitfalls

Assuming a linear cut is always enough. Ignoring parity.

HOW TO THINK ABOUT IT

Two arcs of a circle with the same letter counts must each have exactly half the D's and half the R's, so the total counts must be even and each arc has length n / 2. Slide a window of length n / 2 around the circle: the window and its complement have D counts adding to the total, and the count changes by at most one per step, so it must hit half somewhere.

Basis: Circular sliding window / prefix balance (cf. LeetCode 525, Contiguous Array). Assumption: cut the circular necklace at two points into two arcs, each with the same number of D and R.

BRUTE FORCE - PSEUDO-CODE

```
for every pair of cut positions i < j: arc1 = s[i..j), arc
2 = the rest of the circle
if arc1 and arc2 have the same number of D and the same nu
mber of R: return true
return false
```

Time: O(n^3) | Space: O(n)

Why it is slow

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

OPTIMAL SOLUTION - FIXED-LENGTH CIRCULAR WINDOW WITH THE DISCRETE INTERMEDIATE VALUE ARGUMENT

```python
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            return i, (i + half) % n, doubled[i : i + half
], doubled[i + half : i + n]
        window += (doubled[i + half] == "D") - (doubled[i]
 == "D")
    return None
```

Time: O(n) | Space: O(n)

Why

If each arc has half of the D's and half of the R's, each arc has length n/2, and the total D and R counts must both be even. Slide a window of length n/2 around the circle (use s + s), tracking its D count. The window starting at i and the one starting at i + n/2 are complementary arcs, so their D counts sum to the total. Counts change by at most one per step, so if one is below half and the other above half the count must hit exactly half somewhere. Hence a valid pair of cuts always exists when parity allows.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

```python
def necklace_equal_strings(s):
    n = len(s)
    if n % 2:
        return None
    half = n // 2
    for i in range(n):
        rot = s[i:] + s[:i]
        if rot[:half] == rot[half:]:
            return i, rot[:half]
    return None
```

NOTES FROM CANDIDATES

Why 2 cuts always suffice: think of a running balance cnt(D) - cnt(R); over the full string it returns to 0. Any prefix splits the necklace into two arcs whose imbalances sum to 0; if the imbalance at some prefix is already 0, one cut suffices, otherwise a sliding window of length n/2 proves a 2-cut works.

Sliding-window approach: fix one cut at index i, look for j such that the window [i, j) has half the Ds and half the Rs. As i advances, j moves monotonically.

Don't get distracted by the necklace framing; once unwrapped onto a line of length 2n and constrained to half-length windows, the problem is standard.

Verbalize the 2-cut bound out loud before coding — interviewers ask why.

PREPARATION

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

HARD | DESIGN-STYLE (skippable) | Exponent | 26 of 196

Implement a function that processes a stream of interval data.

STUDY: TALK | Google frequency on LeetCode: all time 8.1, last 30 days 0 | book priority 152 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 352 Data Stream as Disjoint Intervals (likely) (inferred from short statement)

Original: https://www.aced.io/questions/6245/implement-function-process-interval-data

QUESTION

Implement a function that processes a stream of interval data.

APPROACH

Maintain sorted disjoint intervals in a TreeMap/sorted list; on each new interval find neighbors with binary search and merge. O(log n) per update.

INTERVIEW-DAY PLAN

Clarify first

Ask what the function must do per interval: merge and return the current union, return total covered length, or answer point queries. Ask whether touching intervals (end equals next start) merge, whether endpoints are inclusive, and whether intervals can arrive unsorted.

Brute force to say out loud

Keep all intervals in a list and re-sort and merge on each arrival. O(n log n) per interval.

Optimal idea

Keep disjoint intervals sorted by start. For a new interval, binary search for the first interval whose end is at least the new start and the last whose start is at most the new end; replace that slice with one merged interval. A sorted list gives O(log n) search with O(n) splice, a balanced tree gives O(log n) overall.

Dry-run these cases

Interval inside an existing one, one that bridges two, one that touches exactly, one before everything and one after everything.

Likely follow-ups

Removing a range (LeetCode 715), counting covered length incrementally, or a memory bound on the stream.

Pitfalls

Off-by-one on touching intervals. Splicing the wrong slice. Forgetting to keep the total length updated.

HOW TO THINK ABOUT IT

Keep the data in its simplest canonical form: merged intervals sorted by start with gaps between them. A new interval can only touch a contiguous block of existing ones, found with two binary searches, and that block collapses into one interval. State what 'touching' means before you code (shared endpoint, or only real overlap).

Basis: LeetCode 352, Data Stream as Disjoint Intervals (generalized from single numbers to whole intervals).

BRUTE FORCE - PSEUDO-CODE

```
store every interval ever added in a list
on each query: sort the list by start
fold it: if an interval starts at or before the end of the
 last merged one, extend
    that end, else start a new interval
```

Time: O(n log n) per query | Space: O(n)

Why it is slow

Every query re-sorts and re-merges the whole history, O(n log n) per query. Keeping the merged intervals sorted and only splicing the affected block makes an add cost O(log n) to find plus the shifted block.

OPTIMAL SOLUTION - SORTED DISJOINT INTERVALS WITH BINARY SEARCH

```python
class IntervalStream:
    def __init__(self):
        self.starts = []
        self.ends = []

    def add(self, start, end):
        lo = bisect.bisect_left(
            self.ends, start
        )  # first interval ending at or after start
        hi = bisect.bisect_right(self.starts, end)  # firs
t interval starting after end
        if lo < hi:
            start = min(start, self.starts[lo])
            end = max(end, self.ends[hi - 1])
        self.starts[lo:hi] = [start]
        self.ends[lo:hi] = [end]

    def intervals(self):
        return list(zip(self.starts, self.ends))
```

Time: O(log n) to find the spot, O(n) worst-case list shift | Space: O(n)

Why

Keep the merged intervals sorted by start. A new interval only interacts with the intervals found by two binary searches; replace that slice by the single merged interval. Touching closed intervals merge.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How do you keep adds fast for a very large number of intervals?

Use an ordered map (TreeMap, or a balanced tree / skip list in Python via the sortedcontainers package) keyed by start so a merge removes the swallowed intervals in amortized O(log n) instead of shifting an array. Each interval is inserted once and removed at most once, so n adds cost O(n log n) overall.

Should touching intervals such as [1, 3] and [4, 5] merge?
Agree on the semantics first. For integer points, [1, 3] and [4, 5] cover every point from 1 to 5, so they merge; use bisect_left(self.ends, start - 1). For real-valued closed intervals only overlap or a shared endpoint merges, which is what the code above does. For half-open intervals [a, b), a shared endpoint merges but nothing else does.

Answer 'is this point covered?' and 'how much is covered in total?'
The sorted disjoint intervals already support both: a point is covered if the interval with the largest start at or before it ends at or after it (one bisect), and the total length is the sum of the interval lengths (or keep it updated on each add).

```
class IntervalStream:
    def __init__(self):
        self.starts = []
        self.ends = []

    def add(self, start, end):
        lo = bisect.bisect_left(
            self.ends, start
        )  # first interval ending at or after start
        hi = bisect.bisect_right(self.starts, end)  # firs
t interval starting after end
        if lo < hi:
            start = min(start, self.starts[lo])
            end = max(end, self.ends[hi - 1])
        self.starts[lo:hi] = [start]
        self.ends[lo:hi] = [end]

    def intervals(self):
        return list(zip(self.starts, self.ends))


class CoverageStream(IntervalStream):
    """Adds point queries and total covered length to the
same structure."""

    def covers(self, x):
        import bisect

        i = bisect.bisect_right(self.starts, x) - 1
        return i >= 0 and self.ends[i] >= x

    def total_length(self):
        return sum(e - s for s, e in self.intervals())
```

Dynamic Programming and State-Space Optimization 6 questions

HARD | PracHub | Technical Screen | 27 of 196

Find the Longest Straight Run of One Color

STUDY: TALK | Google frequency on LeetCode: all time 19.6, last 30 days 0 | book priority 133 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: the interview site's own solution
LeetCode match: LC 562 Longest Line of Consecutive One in Matrix
Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **`R`** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **`G`** cells. No horizontal or vertical sequence has that length.

APPROACH

Per-cell DP over four directions (horizontal, vertical, diagonal, anti-diagonal): run[d][i][j] = run[d][prev]+1 if same color else 1. O(m*n).

INTERVIEW-DAY PLAN

Clarify first

Ask whether diagonals count in both directions (yes, anti-diagonals too), whether cells must share an exact color value, and what to return for an empty grid. Confirm that a run cannot bend or skip a cell.

Brute force to say out loud

For every cell and each of the four directions, walk forward while the color matches. That is O(m * n * min(m, n)) and is a fine first answer to say out loud.

Optimal idea

Let run[d][i][j] be the length of the same-color run that ends at (i, j) going in direction d. It is run[d][prev] + 1 if the previous cell in that direction has the same color, else 1. Four directions need only the previous row, so memory can be O(n). Answer is the maximum over everything. Time O(m * n).

Dry-run these cases

A single cell (answer 1). A 1 x n row of one color. A grid where the longest run is on the anti-diagonal. A grid where every neighbor differs (answer 1).

Likely follow-ups

Return the cells of the run, not just the length. Allow one wildcard cell. Make the run length at least K and count how many such runs exist.

Pitfalls

Forgetting the anti-diagonal needs the up-right previous cell, so it must be filled in the right scan order. Counting a bent path as a run.

HOW TO THINK ABOUT IT

A run in a fixed direction depends only on the cell before it in that direction, so each of the four directions is a one-line recurrence (like longest common suffix). Only four directions are needed because a line read backwards is the same line. Rolling only the previous row keeps memory at O(n).

Basis: Written; the same recurrence as LeetCode 562, Longest Line of Consecutive One in Matrix, with any color instead of 1.

BRUTE FORCE - PSEUDO-CODE

```
best = 1; for every cell and each of the 4 directions (rig
ht, down, diagonal, anti-
    diagonal):
    walk while the next cell has the same color, counting
the length
    best = max(best, length)
```

Time: O(m * n * min(m, n)) | Space: O(1)

Why it is slow

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def longestColorRun(grid):
    rows, cols = len(grid), len(grid[0])
    answer = 1
    for row in range(rows):
        for col in range(cols):
            color = grid[row][col]
            for dr, dc in ((0, 1), (1, 0), (1, 1), (1, -1)
):
                previous_r, previous_c = row - dr, col - d
c
                if (
                    0 <= previous_r < rows
                    and 0 <= previous_c < cols
                    and grid[previous_r][previous_c] == co
lor
                ):
                    continue
                length = 0
                r, c = row, col
                while 0 <= r < rows and 0 <= c < cols and
grid[r][c] == color:
                    length += 1
                    r += dr
                    c += dc
                answer = max(answer, length)
    return answer
```

Time: O(rows * columns). | Space: O(1) auxiliary space.

Explanation

Inspect four forward directions: right, down, down-right and down-left. These cover every permitted straight line without needing its reverse direction. For a cell and direction, first inspect the immediately preceding cell in that direction. If it exists and has the same color, this cell is inside a run that starts earlier, so skip a fresh traversal here. Otherwise, walk forward while cells stay inside the grid and keep the starting color, and update the maximum length.

Every maximal same-color run has exactly one starting cell whose predecessor is outside the grid or a different color. The algorithm visits that start and counts the full run. It skips only interior starting points, which could not give a longer result than the maximal run already considered. Thus every possible best straight run is counted, and no count changes direction or crosses a color/boundary interruption.

For each of the four directions, maximal runs partition the cells. Each cell is traversed once when its run is counted and receives only constant work for the predecessor check. Total time is O(rows * columns), despite the nested-looking forward walks. The fixed direction arrays and scanning indices use O(1) auxiliary space, and the input grid is unchanged. This bound also covers a legal million-cell single row or column.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step

back by its length.

```
def longest_run_cells(grid):
    best = (0, None, None)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            for dr, dc in DIRS4:
                if (
                    0 <= r - dr < len(grid)
                    and 0 <= c - dc < len(grid[0])
                    and grid[r - dr][c - dc] == grid[r][c]
                ):
                    continue  # not the start of a run in
this direction
                k = 0
                while (
                    0 <= r + dr * k < len(grid)
                    and 0 <= c + dc * k < len(grid[0])
                    and grid[r + dr * k][c + dc * k] == gr
id[r][c]
                ):
                    k += 1
                if k > best[0]:
                    best = (k, (r, c), (dr, dc))
    return best
```

Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
`(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)`	4
`(['RBG', 'BGR', 'GRB'],)`	3

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one. Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.
Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HARD | PracHub | Technical Screen | 28 of 196

Determine winner in stack-merging game

STUDY: TALK | Google frequency on LeetCode: all time 31.2, last 30 days 0 | book priority 110 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: the interview site's own solution
Related in your app (a neighbouring problem, not this exact question): LeetCode #877 Stone Game
LeetCode match: Related: LC 292 Nim Game, LC 464 Can I Win, LC 877 Stone Game (inferred from overview)
Original: https://prachub.com/coding-questions/determine-winner-in-stack-merging-game

QUESTION

Overview: This question evaluates understanding of combinatorial game theory, impartial game analysis, state-space encoding, and efficient state-space search techniques.

APPROACH

Impartial game: encode the state (tuple of stack sizes), memoize win/lose with DFS over all legal merges, or use Sprague-Grundy values. Prune with symmetry (sorted state).

INTERVIEW-DAY PLAN

Clarify first

My copy of this question has only the overview, so before anything ask for the exact rules: what a move is (for example merge two stacks, possibly adjacent ones, with a size limit), who moves first, whether the player who cannot move loses (normal play) or wins, and the number and size of stacks.

Brute force to say out loud

Recursive search over every legal move with no memory. Exponential.

Optimal idea

Treat it as an impartial game. Encode the state as the multiset of stack sizes (a sorted tuple, which removes symmetric duplicates). A state is winning if some move leads to a losing state, and a state with no moves is losing for the player to move under normal play. Memoize on the sorted tuple. If the game splits into independent parts, use Sprague-Grundy numbers (mex of reachable values) and XOR them instead of searching the product.

Dry-run these cases

No legal moves, one stack, two equal stacks, a state reachable by two move orders (checks memoization).

Likely follow-ups

Larger states (find a pattern from small cases with a brute-force table, then prove it), misere play, or counting winning first moves.

Pitfalls

Starting to code before the rules are confirmed. Memoizing on an unsorted state and blowing up the state space.

HOW TO THINK ABOUT IT

Two-player game with perfect play and no draws: define win(state) = some move leads to a state where the opponent loses. A state with no moves is a loss for whoever must move. If a move does not care about stack order, sort the state so equal positions share one cache entry. If the game splits into independent parts, use Grundy numbers and XOR them.

Basis: Written. The statement is not available in my copy, so this uses an example rule (confirm the real one): any two stacks may be merged if their combined size is at most a cap; a player who cannot move loses. The method (memoized win/lose search, Sprague-Grundy for independent games) is the part that carries over.

BRUTE FORCE - PSEUDO-CODE

```
wins(state): for every legal merge: if wins(state after th
e merge) is false: return
    true
return false (no merge leads the opponent into a loss, or
there is no move)
```

Time: exponential in the number of stacks | Space: O(depth)

Why it is slow

The same position is reached through many different move orders, so the work is exponential. Remembering the result per position (sorted state as key) solves each position once.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(colors):
    from functools import lru_cache

    initial = tuple(sorted((1, c) for c in colors))

    @lru_cache(maxsize=None)
    def can_win(state):
        m = len(state)
        seen_next = set()

        for i in range(m):
            hi, ci = state[i]
            for j in range(m):
                if i == j:
                    continue

                hj, cj = state[j]
                if hi == hj or ci == cj:
                    next_state = [state[k] for k in range(
m) if k != i and k != j]
                    next_state.append((hi + hj, ci))
                    next_state = tuple(sorted(next_state))

                    if next_state in seen_next:
                        continue
                    seen_next.add(next_state)
                    if not can_win(next_state):
                        return True

        return False

    return can_win(initial)
```

Time: O(S * n^3 log n), where S is the number of distinct reachable states and n is the current number of stacks (n <= 12) | Space: O(S * n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The game is a sum of independent games (several separate boards or piles).

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

```
def grundy(state, moves, memo=None):
    memo = {} if memo is None else memo
    if state in memo:
        return memo[state]
    seen = {grundy(nxt, moves, memo) for nxt in moves(stat
e)}
    g = 0
    while g in seen:
        g += 1
    memo[state] = g
    return g
```

Misere play (the player who cannot move wins).

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

Count the number of winning first moves.

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

FUNCTION AND TEST CASES

Signature

```
def solution(colors: list[str]) -> bool:
SAMPLE INPUT
['Y']
```

Test cases

Input	Expected
['Y']	False
['Y', 'W']	True

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Babylon Tile-Stack Merge Game | 1Point3Acres | onsite-coding | asked 2025-12-29 | Single report

https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API. Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top". Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).
Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

Preparation: Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

HARD | PracHub | Technical Screen | 29 of 196

Count same-color squares in a character grid

STUDY: TYPE | Google frequency on LeetCode: all time 51.2, last 30 days 0 | book priority 59 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: the interview site's own solution
Related in your app (a neighbouring problem, not this exact question): LeetCode #1277 Count Square Submatrices with All Ones, LeetCode #221 Maximal Square
LeetCode match: LC 1277 Count Square Submatrices with All Ones / LC 221 Maximal Square
Original: https://prachub.com/coding-questions/count-same-color-squares-in-a-character-grid

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **n** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by **`'a'`**. Your function should return the total count of such monochromatic squares for a given input grid.

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.
Constraints
0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000
Examples
Input: ([])
Expected Output: 0
Explanation: An empty grid contains no squares.
Input: (["aab", "aab", "bbb"])
Expected Output: 10
Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.
Hints
Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

APPROACH

dp[i][j] = 1 + min(top, left, top-left) when those neighbors share the same char as (i,j), else 1. Sum all dp values = number of monochromatic squares. O(mn).

INTERVIEW-DAY PLAN

Clarify first

Ask for the grid size, whether every k x k square counts (k from 1 up), and whether to count squares of every size (the statement says all).

Brute force to say out loud

For every top-left corner and size, check all cells. O(mn * min(m, n)^3).

Optimal idea

dp[i][j] is the size of the largest same-color square whose bottom-right corner is (i, j), and it equals the number of such squares ending there. If i and j are positive and the cells above, left and diagonal match the current character, dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]); otherwise it is 1. The answer is the sum of all dp values. O(mn) time, O(n) with a rolling row.

Dry-run these cases

A 1 x 1 grid (1), a grid of one color (sum of squares counts), a checkerboard (only 1 x 1 squares), a rectangle grid.

Likely follow-ups

The largest square only, or squares allowing at most one different cell.

Pitfalls

Forgetting to compare the three neighbors to the current character, not just to each other.

HOW TO THINK ABOUT IT

Let dp[i][j] be the side of the largest same-colour square whose bottom-right corner is (i, j). That is also the number of squares ending at that corner (one per size from 1 up to it). A square of side k at (i, j) needs squares of side k - 1 at the three cells above, left and above-left, all of the same colour, hence the min of those three plus one.

Basis: LeetCode 1277 (Count Square Submatrices with All Ones) with 'same character' instead of 1. Counts squares of every size k >= 1.

BRUTE FORCE - PSEUDO-CODE

```
for every top-left corner and every size k that fits: chec
k that all k * k cells are
    the same colour; count it if so
```

Time: O(m * n * min(m, n)^3) | Space: O(1)

Why it is slow

The same cells are rechecked for every size: O(m * n * min(m, n)^3). A table of the largest same-colour square ending at each cell, summed over all cells, is O(m * n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(grid):
    if not grid:
        return 0

    m = len(grid)
    n = len(grid[0])
    if n == 0:
        return 0

    total = 0
    prev = [0] * n

    for i in range(m):
        curr = [0] * n
        for j in range(n):
            if i == 0 or j == 0:
                curr[j] = 1
            else:
                c = grid[i][j]
                if (
                    c == grid[i - 1][j]
                    and c == grid[i][j - 1]
                    and c == grid[i - 1][j - 1]
                ):
                    curr[j] = 1 + min(prev[j], curr[j - 1]
, prev[j - 1])
                else:
                    curr[j] = 1

            total += curr[j]

        prev = curr

    return total
```

Time: O(m * n), where m is the number of rows and n is the number of columns. | Space: O(n), because only the previous DP row and current DP row are stored.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Only the largest square (LeetCode 221).

Keep the maximum of the same dp table instead of the sum.

```
def largest_same_color_square(grid):
    m, n, best = len(grid), len(grid[0]), 1
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            if grid[i][j] == grid[i - 1][j] == grid[i][j -
 1] == grid[i - 1][j - 1]:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j -
 1], dp[i - 1][j - 1])
                best = max(best, dp[i][j])
    return best
```

Allow at most one different cell inside a square.

The simple min-of-three recurrence no longer works. Use 2D prefix counts per colour and test each (corner, size) with binary search on the size, or DP with a second layer 'one defect used'.

Count only squares of colour c, or only side >= 2.

Filter while summing: add dp only for cells of colour c, or add max(0, dp - 1) for sizes at least 2.

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> int:
SAMPLE INPUT
([])
```

Test cases

Input	Expected
([])	0
(["aab", "aab", "bbb"])	10

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Count Same-Color Squares in Matrix | 1Point3Acres | onsite-coding | Low

https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

Variant reported in both intern VO and an additional NG round. Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

Preparation: Write the all-1-squares DP from memory in under 8 min.
Generalize to multi-color in <2 min — only the equality check changes.

HARD | IN APP LC 127, 72 | Exponent | 30 of 196

Edit distance

STUDY: TYPE | Google frequency on LeetCode: all time 46, last 30 days 0 | book priority 69 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #127)

LeetCode match: LC 127 Word Ladder (statement) / LC 72 Edit Distance (title)

+612 interviews

Original: https://www.aced.io/questions/3891/edit-distance

QUESTION

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples beginWord = 'hit' endWord = 'cog' wordList = ["hit", "hot", "dot", "dog", "cog"] output: 4

hit -> hot -> dot

-> dog -> cog beginWord = 'word' endWord = 'word' wordList = ['word', 'ward'] output: 0 beginWord = 'hit' endWord = 'cog' wordList = ["hit", "hot", "dot", "dog"] output: None

because no 'cog' in list

APPROACH

BFS over words where edges connect words differing by one letter (use wildcard buckets like h*t to build neighbors in O(n*L)); return the path length. For true edit distance use 2D DP: dp[i][j] = min(insert, delete, replace).

HOW TO THINK ABOUT IT

Compare the last characters. If they match, nothing is lost: the cost is the cost of the shorter prefixes. If not, one of three operations was used, so take the cheapest of replace, delete and insert, each followed by the cost of a smaller problem. The same smaller problems repeat, so tabulate them in a grid dp[i][j] = cost of turning the first i characters of a into the first j of b; each cell needs only its left, top and top-left neighbours.

Basis: LeetCode 72, Edit Distance (Levenshtein): insert, delete or replace one character.

BRUTE FORCE - PSEUDO-CODE

```
dist(i, j): if i == 0: return j; if j == 0: return i
if a[i-1] == b[j-1]: return dist(i - 1, j - 1)
else return 1 + the smallest of dist(i - 1, j - 1) (replac
e), dist(i - 1, j) (delete),
    dist(i, j - 1) (insert)
```

Time: O(3^(m + n)) | Space: O(m + n)

Why it is slow

Three calls per step and the same (i, j) pair is solved many times: about 3^(m + n). A table of all (i, j) values is O(m * n), with a rolling row for O(min(m, n)) memory.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #127 WORD LADDER)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Shortest word transformation from beginWord to endWord, changing one letter at a time.
Each intermediate word must be in wordList. Return total words in shortest path, or 0. Right?"
Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 3 — OPTIMIZE (BFS with pattern matching)

Say: "BFS on word transformations — already O(N*L*26).
Walk-through of how BFS finds min steps:
Build a set from wordList for O(1) lookup.
Queue: [(beginWord, 1)] — (current_word, steps_so_far).
For each word popped:
For each position i (0..L-1): try replacing with each of 26 letters.
candidate = word[:i] + letter + word[i+1:]
If candidate == endWord: return steps+1 immediately.
If candidate in word_set and not visited: enqueue (candidate, steps+1). Remove from set.
WHY BFS gives minimum steps: BFS explores words at distance 1, then 2, then 3...
First time we reach endWord = fewest transformations needed.
WHY remove from set instead of a visited set: saves memory and achieves the same effect
(each word can only be visited once on the shortest path).
Return 0 if queue exhausted without reaching endWord.
Time: O(N*L*26). Space: O(N*L).

PHASE 4 — CLEAN CODE

```
class _127_WordLadder:
    def ladderLength(self, beginWord: str, endWord: str,
 wordList: List[str]) -> int:
        word_set = set(wordList)
        if endWord not in word_set:
            return 0
        queue   = deque([(beginWord, 1)])
        visited = {beginWord}
        while queue:
            word, steps = queue.popleft()
            for i in range(len(word)):
                for c in 'abcdefghijklmnopqrstuvwxyz':
                    next_word = word[:i] + c + word[i+1:
]
                    if next_word == endWord:
                        return steps + 1
                    if next_word in word_set and next_wo
rd not in visited:
                        visited.add(next_word)
                        queue.append((next_word, steps +
 1))
        return 0
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace this BFS level-by-level using the queue state."
beginWord='hit', endWord='cog',
wordList=['hot','dot','dog','lot','log','cog']:
word_set={'hot','dot','dog','lot','log','cog'}. 'cog' in set ok.
BFS: queue=[('hit',1)]. visited={'hit'}.
'hit': h->h: 'ait'? No. 'bit'? No. 'hot'! -> enqueue ('hot',2).
i->_: 'hat'? No. 'hut'? No. None in set.
t->_: 'hia'? No. etc.
'hot': h->_: 'aot'..'dot'! (2), 'lot'! (2). -> enqueue ('dot',3),('lot',3).
'dot': d->_: 'aot'..'lot' visited. o->_: 'dat'..'dog'! -> ('dog',4).
'dog': d->_:'aog'..'log'! ->('log',5). o->_:'dag'..'cog'!'cog'==endWord -> return 5 ok

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n * L * 26) = O(n*L). Space O(n*L). Standard BFS shortest path.
Bidirectional BFS: search simultaneously from beginWord and endWord — roughly halves depth.
Given more time I'd ask: return all shortest paths?
That's Word Ladder II (#126) — BFS to build levels, then DFS to backtrack paths.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the actual operations.

Keep the full table and walk back from dp[m][n]: if the characters match step diagonally; otherwise take the neighbour whose value is one smaller and record replace, delete or insert. O(m * n) memory is needed for the walk back.

```
def edit_operations(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            dp[i][j] = (
                dp[i - 1][j - 1]
                if a[i - 1] == b[j - 1]
                else 1 + min(dp[i - 1][j - 1], dp[i - 1][j
], dp[i][j - 1])
            )
    ops, i, j = [], m, n
    while i or j:
        if i and j and a[i - 1] == b[j - 1]:
            i, j = i - 1, j - 1
        elif i and j and dp[i][j] == dp[i - 1][j - 1] + 1:
            ops.append(f"replace {a[i - 1]} with {b[j - 1]
}")
            i, j = i - 1, j - 1
        elif i and dp[i][j] == dp[i - 1][j] + 1:
            ops.append(f"delete {a[i - 1]}")
            i -= 1
        else:
            ops.append(f"insert {b[j - 1]}")
            j -= 1
    return ops[::-1]
```

Are the strings exactly one edit apart (LeetCode 161)?

No table needed: if the lengths differ by more than one answer false; scan to the first mismatch and compare the remainders (equal lengths: skip one char in each; otherwise skip one in the longer). O(n), O(1).

```
def one_edit_apart(a, b):
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) > len(b):
        a, b = b, a
    for i in range(len(a)):
        if a[i] != b[i]:
            return a[i + 1 :] == b[i + 1 :] if len(a) == l
en(b) else a[i:] == b[i + 1 :]
    return len(a) != len(b) or False
```

Operations have different costs, or only a small edit distance is of interest (at most k).

Weighted costs just change the numbers added in the recurrence. For 'at most k' only compute a band of width 2k + 1 around the diagonal: O(k * n).

OFFICIAL PAGE TEXT

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples

```
beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog", "cog"]
output: 4 # hit -> hot -> dot -> dog -> cog

beginWord = 'word'
endWord = 'word'
wordList = ['word', 'ward']
output: 0

beginWord = 'hit'
endWord = 'cog'
wordList = ["hit", "hot", "dot", "dog"]
output: None  # because no 'cog' in list
```

Our solution tackles the problem by first constructing a graph that represents all possible transformations between words in the list, where an edge exists between two words if they differ by exactly one character (i.e., they are "one edit distance" apart). This is done by iterating over the list of words and comparing each pair to determine if they meet the one edit distance condition. If they do, an edge is added between them in the graph. After constructing the graph, we use a breadth-first search (BFS) approach to find the shortest path from the start_word to the end_word. We initiate a queue that starts with the start_word and its associated edit distance (initially 0). As we traverse the graph, for each word, we check if it's the end_word. If so, we return the current edit distance incremented by one. If not, we add the word's neighbors to the queue for further exploration, ensuring that we don't revisit previously seen words. The BFS guarantees that the first time we

reach the end_word, it is via the shortest path, and we can stop the search early. Time Complexity: The time complexity of the solution is O(n^2 * m), where n is the number of words and m is the length of each word. This complexity arises from the double loop that compares each word to every other word (O(n^2)), and for each comparison, we check if the words differ by exactly one character (O(m)). Space Complexity: The space complexity is O(n^2), where n is the number of words. This space is required to store the graph, where each word can potentially have edges to all other words. Additionally, the BFS queue and the visited set require O(n) space in the worst case.In this video, Thomas, Google SWE, answers the Edit Distance coding interview question.No answers contributed by the community yet.

HARD | 1Point3Acres | phone-screen, onsite-coding | 31 of 196

Count Perfect Wake Numbers up to N

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 168 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

QUESTION

Examples:

```
196   -> perfect (digits 1,9,6 all distinct, no 0, 9 is not a valley)
23    -> perfect
12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 — 6 is not a valley because its right neighbor 3 is smaller)
1546  -> not perfect (4 is adjacent to 5 and 6, both strictly larger)
320   -> not perfect (contains 0)
34321 -> not perfect (digit 3 repeats)
```

APPROACH

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate."
Practice bitmask-over-digits + tight-flag state machines.
Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

INTERVIEW-DAY PLAN

Clarify first

Ask for the exact definition: no zero digit, all digits distinct, no interior digit smaller than both neighbors. Ask for the range of N and whether to count 1..N inclusive.

Brute force to say out loud

Test every number from 1 to N with a digit check. O(N * d), fine for small N.

Optimal idea

Digit DP over positions with state (used-digit mask, last digit, whether the digits have started falling, tight flag). Numbers with fewer digits than N are counted without a tight constraint, and zero is never allowed, so there are no leading-zero cases. Digits must rise then fall, which is exactly the no-valley rule. There is also a closed form of C(9, k) * 2^(k - 1) for k-digit numbers.

Dry-run these cases

196, 23, 12463 perfect; 1546, 320, 34321 not perfect. N = 9, N = 10, N a power of ten.

Likely follow-ups

A range [A, B] (inclusion-exclusion), or different digit rules.

Pitfalls

Forgetting the zero rule. Letting a digit repeat across the tight and free branches.

HOW TO THINK ABOUT IT

Distinct digits with no zero and no valley means the digits go up and then down (they can only change direction once). That structure is both a closed form for each length and a state for a digit DP: position, which digits are used, the last digit, and whether the digits have started falling. The tight flag handles the bound N.

Basis: Digit DP, like LeetCode 1012 (Numbers With Repeated Digits) and 233. Definition from the report: no zero digit, all digits distinct, and no interior digit is a valley (strictly smaller than both neighbours).

BRUTE FORCE - PSEUDO-CODE

```
count = 0; for x from 1 to N: if x has no zero digit, all digits distinct, and no
    interior digit smaller than both neighbours: count = count + 1
```

Time: O(N * d) | Space: O(1)

Why it is slow

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

OPTIMAL SOLUTION - PART 1: TEST A SINGLE NUMBER

```
def is_perfect(n):
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1]) for i in range(1, len(d) - 1))
```

Time: O(d) | Space: O(d)

Why

Check the three rules directly on the digit list: no '0', all digits distinct, and for each interior digit not (left > digit < right). The first and last digits are never valleys.

OPTIMAL SOLUTION - PART 2: DIGIT DP WITH TIGHT FLAG, USED-DIGIT MASK, LAST DIGIT AND DIRECTION

```
def count_perfect_up_to(N):
    if N < 1:
        return 0
    digits = list(map(int, str(N)))
    L = len(digits)

    @lru_cache(maxsize=None)
    def count_free(remaining, mask, last, falling):
        if remaining == 0:
            return 1
        total = 0
        for nxt in range(1, 10):
            if mask >> nxt & 1:
                continue
            if last == 0:
                total += count_free(remaining - 1, mask |
1 << nxt, nxt, False)
            elif nxt > last and not falling:
                total += count_free(remaining - 1, mask |
1 << nxt, nxt, False)
            elif nxt < last:
                total += count_free(remaining - 1, mask |
1 << nxt, nxt, True)
        return total

    total = sum(count_free(length, 0, 0, False) for length
 in range(1, L))

    def count_tight(pos, mask, last, falling):
        if pos == L:
            return 1
        res = 0
        for nxt in range(1, digits[pos] + 1):
            if mask >> nxt & 1:
                continue
            if last == 0 or (nxt > last and not falling):
                new_fall = False
            elif nxt < last:
                new_fall = True
            else:
                continue
            if nxt == digits[pos]:
                res += count_tight(pos + 1, mask | 1 << nx
t, nxt, new_fall)
            else:
                res += count_free(L - pos - 1, mask | 1 <<
 nxt, nxt, new_fall)
        return res

    return total + count_tight(0, 0, 0, False)
```

Time: O(d * 2^9 * 9 * 9) | Space: O(2^9 * 9)

Why

Distinct digits with no valley means the digits rise and then fall (unimodal). Build the number left to right with state (position, mask of used digits, last digit, whether we have started falling, tight). While rising any larger unused digit is allowed, once falling only smaller unused digits are. Numbers with fewer digits than N are free (not tight), so start the DP once for each shorter length and once tight for len(N). Zero is never allowed, which also removes the leading-zero bookkeeping.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

```
def perfect_with_k_digits(k):
    return math.comb(9, k) * 2 ** (k - 1) if 1 <= k <= 9 e
lse 0
```

Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

NOTES FROM CANDIDATES

Part 1 is a linear scan; Part 2 is digit DP with state (position, mask of used digits, last digit, is_tight against n). Track whether the previous step went up or down to detect a future valley.

Multiple candidates report not finishing Part 2 in the 45-minute window. The interviewer is generally patient and dispenses hints ("think DP" / "think digit DP") when you stall.

Watch out: the "no valley" rule is about interior digits, so the first and last digit are never themselves the valley; they can still create one for their neighbor.

The full prompt is short and recent — expect the follow-up to vary (e.g. count exactly-K-digit Perfect Wake Numbers).

PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate."

Practice bitmask-over-digits + tight-flag state machines.

Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

HARD | 1Point3Acres | onsite-coding | 32 of 196

Matrix Flower Placement with House Adjacency

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 171 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-05-31 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/matrix-flower-placement-with-house-adjacency

QUESTION

Examples:

```
H 0 0 0
H 0 0 0
0 0 0 H
0 0 0 0
```

The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.

APPROACH

Practice N-Queens with row/column sets and a recursive placement order.

Drill grid-neighbor helpers and constraint pruning before writing the full DFS.

Prepare the boolean version first, then generalize to collect all solutions.

INTERVIEW-DAY PLAN

Clarify first

Ask whether each row and column holds at most one or exactly one flower, whether a house needs at least one or exactly one adjacent flower, whether adjacency is four-way, and whether to return one solution, a boolean, or all.

Brute force to say out loud

Try every subset of empty cells. Exponential.

Optimal idea

Row-by-row backtracking like N-Queens: place at most one flower per row, track used columns, and check each house as soon as the row below it is decided (its neighbors are in rows h - 1 to h + 1). Prune at the first failed house. The boolean version stops at the first solution; the all-solutions version continues.

Dry-run these cases

No houses, a house with no empty neighbor (impossible), two houses sharing one neighbor, a 1 x 1 grid.

Likely follow-ups

Exactly-one rule, all solutions, or a faster exact-cover search.

Pitfalls

Checking a house before the next row is decided. Not restoring the cell on backtrack.

HOW TO THINK ABOUT IT

At most one flower per row and column is the N-Queens pattern: decide row by row which column (if any) gets the flower and remember the used columns. The extra rule about houses is local, so check a house as soon as the last row that can affect it is decided, and prune the branch if it fails.

Basis: N-Queens style backtracking (LeetCode 51/52) with extra local constraints. Assumptions stated explicitly: every house must have at least one flower in a 4-neighbour cell; rows and columns hold at most one flower (or exactly one if requested).

BRUTE FORCE - PSEUDO-CODE

```
for every subset of empty cells:
    check at most one flower per row and column
    check every house has a flower in one of its four neig
hbouring cells
    count it if both hold
```

Time: O(2^cells) | Space: O(cells)

Why it is slow

2^cells subsets: exponential. Backtracking row by row with a used-columns set and early house checks prunes almost all of them (still exponential in the worst case).

OPTIMAL SOLUTION - ROW-BY-ROW BACKTRACKING WITH USED-COLUMN SET AND DELAYED HOUSE CHECKS

```
def place_flowers(grid, exactly_one=False, find_all=False)
:
    R, C = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C) i
f cells[r][c] == "H"]
    solutions = []

    def house_ok(r, c):
        return any(
            0 <= r + dr < R and 0 <= c + dc < C and cells[
r + dr][c + dc] == "F"
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1
))
        )

    def check_row_houses(r):
        return all(house_ok(hr, hc) for hr, hc in houses i
f hr == r)

    def solve(r, used_cols):
        if not find_all and solutions:
            return
        if r >= 2 and not check_row_houses(
            r - 2
        ):  # rows up to r - 1 are decided, so row r - 2 h
ouses are final
            return
        if r == R:
            if check_row_houses(R - 1) and (not exactly_on
e or len(used_cols) == C):
                solutions.append(["".join(row) for row in
cells])
            return
        for c in range(C):
            if c not in used_cols and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used_cols | {c})
                cells[r][c] = "0"
        if not exactly_one:
            solve(r + 1, used_cols)
```

```
    solve(0, frozenset())
    return solutions if find_all else (solutions[0] if sol
utions else None)
```

Time: exponential worst case, heavily pruned | Space: O(rows + cols)

Why

Place at most one flower per row, tracking used columns, and only on empty cells ('0'). A house at row h has neighbours in rows h - 1, h and h + 1, so its requirement is final once row h + 1 is decided; failing there prunes the whole subtree. At the last row check all remaining houses. The 'exactly one' variant just forbids skipping a row and a column. Collecting every solution instead of the first one only changes whether the recursion stops at the first success.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. Cost: the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

```
def place_flowers(grid, exactly_one=False, find_all=False)
:
    R, C = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C) i
f cells[r][c] == "H"]
    solutions = []

    def house_ok(r, c):
        return any(
            0 <= r + dr < R and 0 <= c + dc < C and cells[
r + dr][c + dc] == "F"
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1
))
        )

    def check_row_houses(r):
        return all(house_ok(hr, hc) for hr, hc in houses i
f hr == r)

    def solve(r, used_cols):
        if not find_all and solutions:
            return
        if r >= 2 and not check_row_houses(
            r - 2
        ):  # rows up to r - 1 are decided, so row r - 2 h
ouses are final
            return
        if r == R:
            if check_row_houses(R - 1) and (not exactly_on
e or len(used_cols) == C):
                solutions.append(["".join(row) for row in
cells])
            return
        for c in range(C):
            if c not in used_cols and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used_cols | {c})
                cells[r][c] = "0"
        if not exactly_one:
            solve(r + 1, used_cols)

    solve(0, frozenset())
    return solutions if find_all else (solutions[0] if sol
utions else None)
```

Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

NOTES FROM CANDIDATES

This is an N-Queens-style DFS / backtracking problem with extra local constraints around houses.

Track occupied rows and columns, then validate the four-neighbor requirement either incrementally or at the leaf.

The two variants differ on at most one vs exactly one flower per row/column; state the assumption before coding.

PREPARATION

Practice N-Queens with row/column sets and a recursive placement order.

Drill grid-neighbor helpers and constraint pruning before writing the full DFS.

Prepare the boolean version first, then generalize to collect all solutions.

Other: Design, Heaps, Intervals and Misc 8 questions

HARD | PracHub | Technical Screen | 33 of 196

Compute Turnstile Crossing Times

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 162 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

LeetCode match: LeetCode 'Time Taken to Cross the Door' (exact number unverified) (inferred from overview)

Original: https://prachub.com/coding-questions/compute-turnstile-crossing-times

QUESTION

Overview: This question evaluates proficiency in queue simulation, event-driven scheduling, and handling priority and tie-breaking rules while achieving linear time complexity.

APPROACH

Two queues (entering, exiting) ordered by arrival; simulate time with the direction-priority and tie-break rules, jumping the clock when idle. O(n).

INTERVIEW-DAY PLAN

Clarify first

Ask: how many people can pass per second, which direction goes first when both are waiting, whether the rule depends on what happened in the previous second, and how ties on arrival time break (by id). Clarify what to return (time per person).

Brute force to say out loud

Simulate every second from the earliest to the latest time, checking both queues. Correct but its cost grows with the size of the timestamps, not the number of people.

Optimal idea

Keep two FIFO queues, enter and exit, and a variable for what the turnstile did in the previous second. Each step apply the priority rule to pick a queue. When both queues are empty, jump the clock to the next arrival and reset the previous-state to idle if at least one second was skipped. That makes the simulation O(n) after sorting.

Dry-run these cases

Two people arriving at the same second with opposite directions. A long idle gap followed by a simultaneous pair. All enters, then all exits. A person who arrives while the turnstile was just used by the other direction.

Likely follow-ups

Ask what happens when the idle jump leaves exactly one empty second versus more. Count that the 'previous second' state can be none, not only enter or exit.

Pitfalls

Resetting state only when the gap is two or more seconds. Forgetting to push arrivals that happened during the jump. Comparing queues by id instead of arrival order.

HOW TO THINK ABOUT IT

A scheduling rule that looks at 'what happened last second' is a state machine: write the state down (previous direction: enter, exit, none) before coding. If time values can be huge, simulate events, not seconds: jump the clock to the next arrival when both queues are empty, and remember an idle gap resets the state.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumed rules (confirm them): one person per second; if an enter and an exit compete, the previous second decides (enter if it was an enter, else exit, and exit if the door was idle).

BRUTE FORCE - PSEUDO-CODE

```
t = 0; queues for entering and exiting; prev = what happen
ed last second
each second: add arrivals; choose who passes by the priori
ty rule (using prev); record
    the time; t = t + 1
```

Time: O(T + n) where T is the largest timestamp | Space: O(n)

Why it is slow

A timestamp of 10^9 means a billion iterations even if nothing happens. Jumping to the next arrival when both queues are empty is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
from collections import deque


def solution(time, direction):
    n = len(time)
    answer = [-1] * n
    enter_q = deque()
    exit_q = deque()
    i = 0
    current_time = 0
    # last_state: -1 = the previous time unit was NOT used
 by anyone
    #              0 = previous user entered, 1 = previous
 user exited
    last_state = -1

    while i < n or enter_q or exit_q:
        if not enter_q and not exit_q:
            # Nobody is waiting. We must wait for the next
 arrival.
            # Only reset last_state when there is a genuin
e idle gap:
            # if the next person arrives strictly after cu
rrent_time, the
            # turnstile sits unused for at least one time
unit, so the
            # "previous time unit" is unused. If they arri
ve exactly at
            # current_time, the previous unit's direction
context still holds.
            if time[i] > current_time:
                last_state = -1
                current_time = time[i]

        # Admit everyone who has arrived by current_time.
        while i < n and time[i] <= current_time:
            if direction[i] == 0:
                enter_q.append(i)
            else:
```

```
                exit_q.append(i)
            i += 1

        if last_state == 1:
            # Previous user exited -> exiting has priority
.
            if exit_q:
                idx = exit_q.popleft()
                last_state = 1
            else:
                idx = enter_q.popleft()
                last_state = 0
        elif last_state == 0:
            # Previous user entered -> entering has priori
ty.
            if enter_q:
                idx = enter_q.popleft()
                last_state = 0
            else:
                idx = exit_q.popleft()
                last_state = 1
        else:
            # Previous time unit unused -> exiting has pri
ority.
            if exit_q:
                idx = exit_q.popleft()
                last_state = 1
            else:
                idx = enter_q.popleft()
                last_state = 0

        answer[idx] = current_time
        current_time += 1

    return answer
```

Time: O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

```
def turnstile_times_priority(events, first="exit"):
    """Same simulation with a configurable idle priority (
some versions let entering go first when the door was idle
)."""
    n = len(events)
    order = sorted(range(n), key=lambda i: (events[i][0],
i))
    ent, ext, res = deque(), deque(), [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or ent or ext:
        if not ent and not ext:
            t = max(t, events[order[i]][0])
            if t > last_used + 1:
                prev = None
        while i < n and events[order[i]][0] <= t:
            (ent if events[order[i]][1] == "enter" else ex
t).append(order[i])
            i += 1
        if ent and ext:
            want = prev if prev is not None else first
            queue = ent if want == "enter" else ext
        else:
            queue = ent or ext
        person = queue.popleft()
        res[person], prev, last_used = t, events[person][1
], t
        t += 1
    return res
```

Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

FUNCTION AND TEST CASES

Signature

```
def solution(time: list[int], direction: list[int]) -> lis
t[int]:
SAMPLE INPUT
[[0,0,1,5],[0,1,1,0]]
```

Test cases

Input	Expected
[[0, 0, 1, 5], [0, 1, 1, 0]]	[2, 0, 1, 5]
[[0, 0, 0], [0, 1, 0]]	[1, 0, 2]

HARD | IN APP LC 843 | PracHub | Technical Screen | 34 of 196

Find secret word with match-count feedback

STUDY: TALK | Google frequency on LeetCode: all time 42.2, last 30 days 0 | book priority 76 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word

Original: https://prachub.com/coding-questions/find-secret-word-with-match-count-feedback

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- `matchCount(guess) -> int`: when you submit a word `guess` from the list, the API returns the number of positions `i` such that `guess[i] == secret[i]` (i.e., exact character match at the same index). You are allowed to call `matchCount` **at most 10 times**.

Task

Design/implement a function (e.g., `findSecretWord(words)`) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- `words` contains between 1 and 100 words.
- All words have equal length `L` (commonly `L = 6`).
- Every guess must be a word from `words`.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once `matchCount(guess) == L` (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length `L`. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- `matchCount(guess) -> int`
- It returns how many positions `i` satisfy `guess[i] == secret[i]` You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement `solution(words, secret)` and simulate the API inside your function. The parameter `secret` is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string `""`.

Constraints`1 <= len(words) <= 1001 <= len(words[i]) <= 10`All words are unique, lowercase, and have the same length `Lsecret` is guaranteed to be one of the words in `words`Your algorithm may use at most 10 simulated `matchCount` callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

Examples

Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until `acckzz` is identified.

Input: (["secret"], "secret")

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns **k**, any valid remaining candidate must also have exactly **k** matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

APPROACH

Pick a guess, read matchCount, keep only candidates with the same match count against that guess. Choosing the guess that minimizes the worst-case remaining set (minimax) keeps queries low.

HOW TO THINK ABOUT IT

Each guess returns a number that every remaining candidate must agree with: a candidate can still be the secret only if it has the same match count with the guess as the secret does. So after each guess, filter the candidate list. To shrink fast, guess a word that splits the candidates evenly by match count (minimise the largest group); that is what keeps you inside 10 guesses.

Basis: LeetCode 843, Guess the Word (match count = positions with the same letter).

BRUTE FORCE - PSEUDO-CODE

```
candidates = all words; repeat: guess = any candidate; m =
 matchCount(guess)
if m equals the word length: done; else keep only the cand
idates whose match count
    with the guess equals m
```

Time: O(guesses * W * L) | Space: O(W)

Why it is slow

A poor guess may remove very few candidates, so there is no guarantee of finishing in 10 guesses. Guessing the word whose largest match-count group is smallest (minimax) keeps the list shrinking fast.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(words, secret):
    if not words or secret not in words:
        return ""

    n = len(words)
    L = len(words[0])

    def exact_matches(a, b):
        return sum(ch1 == ch2 for ch1, ch2 in zip(a, b))
```

```
    guesses_used = 0

    def matchCount(guess):
        nonlocal guesses_used
        guesses_used += 1
        return exact_matches(guess, secret)

    # Precompute pairwise match counts between every pair
of words.
    match_matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            m = exact_matches(words[i], words[j])
            match_matrix[i][j] = m
            match_matrix[j][i] = m

    candidates = list(range(n))
    used = set()

    while candidates and guesses_used < 10:
        if len(candidates) == 1:
            guess_idx = candidates[0]
        else:
            candidate_set = set(candidates)
            best_idx = None
            best_worst_bucket = float("inf")
```

```
            best_is_candidate = False

            for i in range(n):
                if i in used:
                    continue

                buckets = [0] * (L + 1)
                for j in candidates:
                    buckets[match_matrix[i][j]] += 1

                worst_bucket = max(buckets)
                is_candidate = i in candidate_set

                if (
                    best_idx is None
                    or worst_bucket < best_worst_bucket
                    or (
                        worst_bucket == best_worst_bucket
                        and is_candidate
                        and not best_is_candidate
                    )
                ):
                    best_idx = i
                    best_worst_bucket = worst_bucket
                    best_is_candidate = is_candidate

            guess_idx = best_idx

        used.add(guess_idx)
        feedback = matchCount(words[guess_idx])

        if feedback == L:
            return words[guess_idx]

        candidates = [j for j in candidates if match_matri
x[guess_idx][j] == feedback]
```

```
    return ""
```

Time: O(n^2 * L + 10 * n^2), which simplifies to O(n^2 * L) for the given constraints | Space: O(n^2)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Show how the candidate list shrinks.

After a guess with answer m, only words with the same match count against the guess survive (the secret and the candidate agree on every comparison). Printing the sizes round by round shows why a balanced guess finishes in a few rounds.

```
def candidates_after(words, guess, count):
    return [w for w in words if _match(guess, w) == count]
```

Guess at random instead of the minimax choice.

Expected to finish in far fewer than W guesses, but not guaranteed within 10 on a hard list. The minimax choice is the safe answer; random with a 'zero-match' preference (guess words that share no letters with earlier guesses) is a cheap improvement.

The word list is much larger.

Computing all pairwise match counts is O(W^2 L). Sample candidates when choosing a guess, or precompute buckets by letter at each position to count matches faster.

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], secret: str) -> str:
```

Sample call

```
solution(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz
")
```

Test cases

Input	Expected
`(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")`	`'acckzz'`
`(["secret"], "secret")`	`'secret'`

HARD | IN APP LC 218 | PracHub | Technical Screen | 35 of 196

Compute city skyline outline

STUDY: TALK | Google frequency on LeetCode: all time 29.8, last 30 days 0 | book priority 113 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #218 The Skyline Problem

LeetCode match: LC 218 The Skyline Problem

Original:
https://prachub.com/coding-questions/compute-city-skyline-outline

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers **`[L, R, H]`**:

- **`L`**: the x-coordinate of the left edge of the building,
- **`R`**: the x-coordinate of the right edge of the building (**`L < R`**),
- **`H`**: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height **`H`**. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **`x`**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **n** — the number of buildings.
- A list of **n** buildings, where each building is given as a triple **`[L, R, H]`** of integers. You may assume:
- **`1 <= n <= 10^4`**
- **`0 <= L < R <= 10^9`**
- **`1 <= H <= 10^9`**

Output

Return a list of key points representing the skyline. Each key point is a pair **`[x, height]`** (or a 2-element array/tuple) of integers.

Example

Input:

- **`n = 3`** - buildings = **`[[2, 9, 10], [3, 7, 15], [5, 12, 12]]`** One valid output:
- **`[[2, 10], [3, 15], [7, 12], [12, 0]]`** Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

APPROACH

Sweep line over building edges with a max-heap of active heights (lazy deletion); emit a point whenever the max height changes. O(n log n). Divide and conquer also works.

HOW TO THINK ABOUT IT

The skyline height at x is the tallest building covering x, and it can only change at a building's left or right edge. So sweep over those x positions in order, keep the active buildings in a max-heap by height, and report a key point whenever the maximum changes. The details that matter are the event order at the same x and removing finished buildings lazily.

Basis: LeetCode 218, The Skyline Problem.

BRUTE FORCE - PSEUDO-CODE

```
xs = all building edges, sorted
for each x: height = the largest height among buildings wi
th left <= x < right
output [x, height] whenever the height differs from the pr
evious one
```

Time: O(n^2) | Space: O(n)

Why it is slow

Each edge scans all buildings, O(n^2). A sweep with a max-heap of active buildings and lazy deletion is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
import heapq


def solution(n, buildings):
    # Sweep-line + max-heap. Create events at each left ed
ge (start) and
    # right edge (end). Process events left-to-right; the
current skyline
    # height is the tallest currently-live building.
    events = []
    for L, R, H in buildings:
        events.append((L, -H, R))  # start event: negative
 height marks a start
        events.append((R, 0, 0))  # end event: marks an x
where buildings may end
    events.sort()

    result = []
    # Max-heap keyed by negative height; each entry is (-H
, end_x).
    # Sentinel (0, inf) keeps ground level always availabl
e.
    live = [(0, float("inf"))]
    for x, negH, R in events:
        # Lazily evict buildings that have already ended a
t or before x.
        while live[0][1] <= x:
            heapq.heappop(live)
        if negH < 0:
            heapq.heappush(live, (negH, R))
        cur = -live[0][0]
        if not result or result[-1][1] != cur:
            result.append([x, cur])

    # Multiple events can share an x; keep the final heigh
t at each x.
    cleaned = []
    for x, h in result:
        if cleaned and cleaned[-1][0] == x:
            cleaned[-1][1] = h
        else:
            cleaned.append([x, h])
    # Drop any consecutive equal heights introduced by the
 collapse.
    final = []
```

```
    for x, h in cleaned:
        if final and final[-1][1] == h:
            continue
        final.append([x, h])
    return final
```

Time: O(n log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Divide and conquer instead of a heap.

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

```
def skyline_divide_conquer(buildings):
    def merge(a, b):
        out, i, j, h1, h2 = [], 0, 0, 0, 0
        while i < len(a) or j < len(b):
            if j >= len(b) or (i < len(a) and a[i][0] < b[
j][0]):
                x, h1 = a[i]
                i += 1
            elif i >= len(a) or b[j][0] < a[i][0]:
                x, h2 = b[j]
                j += 1
            else:
                x, h1 = a[i]
                h2 = b[j][1]
                i += 1
                j += 1
            h = max(h1, h2)
            if not out or out[-1][1] != h:
                out.append([x, h])
        return out

    def solve(lo, hi):
        if hi - lo == 0:
            return []
        if hi - lo == 1:
            l, r, h = buildings[lo]
            return [[l, h], [r, 0]]
        mid = (lo + hi) // 2
        return merge(solve(lo, mid), solve(mid, hi))

    return solve(0, len(buildings))
```

Coordinates up to 10^9 or buildings of a very large count.

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

Output the area under the skyline.

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

FUNCTION AND TEST CASES

Sample call

```
getSkyline(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])
```

Test cases

Input	Expected
`(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])`	`[[2, 10], [3, 15], [7, 12], [12, 0]]`
`(1, [[0, 2, 3]])`	`[[0, 3], [2, 0]]`

HARD | PracHub | Technical Screen | 36 of 196

Return top-k subset sums in descending order

STUDY: TALK | **Google frequency on LeetCode: all time 5, last 30 days 0 | book priority 156 of 196**

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

LeetCode match: LC 2386 Find the K-Sum of an Array

Original: https://prachub.com/coding-questions/return-top-k-subset-sums-in-descending-order

QUESTION

Overview: This question evaluates proficiency in combinatorial reasoning, algorithmic optimization for large search spaces, and efficient extraction of top-k values within the coding & algorithms domain.

APPROACH

Sort values, start from the all-positives sum, and use a max/min-heap of (sum, index) states that either drop the next element or swap it for the following one, popping k times. O(n log n + k log k).

INTERVIEW-DAY PLAN

Clarify first

Ask whether the empty subset counts, whether values can be negative or repeated, how large k is compared with 2^n, and whether equal sums count separately.

Brute force to say out loud

Enumerate all 2^n subsets, sort, take k. Fine for n <= 20; say that out loud.

Optimal idea

Reduce negatives: let base be the sum of the positive values, and work with absolute values. Every subset sum of the original equals base minus a subset sum of the absolute values, so the k largest sums are base minus the k smallest subset sums of the sorted absolute values. Generate the k smallest sums with a min-heap of (sum, last index): from a state you either add the next element or replace the last element with the next one. Each pop yields the next smallest sum and creates at most two states, so O(n log n + k log k).

Dry-run these cases

All positive values. All negative values (best is the empty subset). A zero in the array. k = 1 and k = 2^n.

Likely follow-ups

Need k sums without storing all of them, subsets themselves (store parent pointers), or a limit on subset size.

Pitfalls

Forgetting to sort first, which breaks the add-next/replace-last move set. Pushing duplicate states.

HOW TO THINK ABOUT IT

k is small compared with 2^n, so never enumerate all subsets. Turn the problem into 'k smallest sums of non-negative numbers': subtract everything from the sum of the positive values. Then grow answers in increasing order with a heap where each state has exactly two successors (extend with the next element, or swap the last element for the next one).

Basis: Written; the classic 'k smallest subset sums' heap technique (also behind LeetCode 2386, K-Sum of an Array).

BRUTE FORCE - PSEUDO-CODE

```
sums = the sum of every subset of the numbers
sort sums in descending order; return the first k
```

Time: O(2^n * n) | Space: O(2^n)

Why it is slow

2^n subsets and a sort of them: O(2^n * n) time and O(2^n) space, hopeless past about n = 20. A heap that grows the k best sums step by step is O(n log n + k log k).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(w, k):
    import heapq

    if k <= 0:
        return []

    n = len(w)
    if n == 0:
        return [0] if k >= 1 else []

    a = sorted(w)
    total = sum(a)

    # The largest subset sum is always the sum of all weig
hts.
    result = [total]
    if k == 1:
        return result

    # Generate the smallest sums of excluded elements.
    # Each heap entry is (excluded_sum, largest_index_used
).
    heap = [(a[0], 0)]

    for _ in range(k - 1):
        excluded_sum, i = heapq.heappop(heap)
        result.append(total - excluded_sum)

        if i + 1 < n:
            j = i + 1
            # Option 1: extend the current excluded subset
 with a[j].
            heapq.heappush(heap, (excluded_sum + a[j], j))
            # Option 2: replace a[i] with a[j].
            heapq.heappush(heap, (excluded_sum - a[i] + a[
j], j))

    return result
```

Time: O(n log n + k log k) | Space: O(n + k)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

```python
def top_k_subsets(nums, k):
    """Non-negative inputs: also return the subsets, using
 parent pointers instead of copying lists."""
    a = sorted(nums)
    nodes = [(None, None)]  # node id -> (index added, par
ent id)
    heap = [(a[0], 0, 1)] if a else []
    nodes.append((0, 0))
    out = [(0, [])]
    while heap and len(out) < k:
        s, i, nid = heapq.heappop(heap)
        chosen, cur = [], nid
        while cur:
            idx, cur = nodes[cur]
            chosen.append(a[idx])
        out.append((s, chosen[::-1]))
        if i + 1 < len(a):
            nodes.append((i + 1, nid))
            heapq.heappush(heap, (s + a[i + 1], i + 1, len
(nodes) - 1))
            nodes.append((i + 1, nodes[nid][1]))
            heapq.heappush(heap, (s - a[i] + a[i + 1], i +
 1, len(nodes) - 1))
    return out[:k]
```

k can be as large as 2^n.

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

Limit subsets to at most m elements.

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

FUNCTION AND TEST CASES

Signature

```python
def solution(w: list[int], k: int) -> list[int]:
```

Sample call

```
solution([3, 2, 1], 4)
```

Test cases

Input	Expected
([3, 2, 1], 4)	[6, 5, 4, 3]
([5], 2)	[5, 0]

HARD | IN APP LC 23 | Exponent | 37 of 196

Merge k sorted linked lists.

STUDY: TYPE | Google frequency on LeetCode: all time 49.4, last 30 days 0 | book priority 63 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #23)

LeetCode match: LC 23 Merge k Sorted Lists

+816 interviews

Original:

https://www.aced.io/questions/1167/merge-k-sorted-linked-lists

QUESTION

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.Interview experiences6 shared‹›Revolut · 2 months ago"Its good, but I was bad and dont really remember much. But all rounds was good. HR was nice. Tech guy on tech interview was nice too."Software Engineer · Entry Level / L3FFloQast · 2 months ago"Screening and the system design with director are virtual, while on-site loop in the office. The process is well organi

APPROACH

Min-heap of the k list heads (or divide and conquer pairwise merging). O(N log k).

HOW TO THINK ABOUT IT

At any moment the next output node is the smallest among the current heads of the k lists, which is a min-heap of size k. Each node is pushed and popped once, so the total cost is N log k. A second way is to merge lists in pairs, halving the number of lists each round, with the same bound.

Basis: LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE - PSEUDO-CODE

```
values = every value from every list
sort the values and build a new list from them
```

Time: O(N log N) | Space: O(N)

Why it is slow

O(N log N) time and O(N) extra memory, ignoring that each list is already sorted. A min-heap of the k current heads is O(N log k) with O(k) memory.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #23 MERGE K SORTED LISTS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Given k sorted linked lists, merge them into one sorted linked list. Right?"

"A few quick questions:
k can be 0 (empty input)? Yes — return None. Lists can be empty? Yes."

"Let me walk: lists=[[1,4,5],[1,3,4],[2,6]] -> [1,1,2,3,4,4,5,6] ok

PHASE 3 — OPTIMIZE

The bottleneck is building all values then sorting — we discard the sorted structure.
Min-heap of (value, list_index, node): always extract the globally smallest node.
Push each list's head. Pop min, add to result, push its next.

Time: O(n log k): n nodes, heap of size k for log k per pop/push.
Space: O(k) heap + O(n) output."
Key insight: the min-heap preserves the sorted order across all k lists — each node is pushed and popped exactly once for O(n log k) total.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _23_MergeKSortedLists:
    def mergeKLists(self, lists):
        heap = []
        for i, node in enumerate(lists):
            if node:
                heapq.heappush(heap, (node.val, i, node)
)
        dummy = cur = ListNode(0)
        while heap:
            val, i, node = heapq.heappop(heap)
            cur.next = node; cur = cur.next
            if node.next:
                heapq.heappush(heap, (node.next.val, i,
node.next))
        return dummy.next
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace mergeKLists([[1,4,5],[1,3,4],[2,6]])."
Min-heap of (val, list_id, node). Initial:
[(1,0,node1),(1,1,node1),(2,2,node2)].
Pop (1,0,node(1)): curr.next=node(1). Push (4,0,node(4)).
heap:[(1,1,node(1)),(4,0),(2,2)].
Pop (1,1,node(1)): curr.next=node(1). Push (3,1,node(3)).
heap:[(2,2),(3,1),(4,0)].
Pop (2,2,node(2)): curr.next=node(2). Push (6,2,node(6)).
Pop (3,1): Push (4,1). Pop (4,0): Push (5,0). Pop (4,1). Pop (5,0).

Pop (6,2).
Result: 1->1->2->3->4->4->5->6 ok

WHY min-heap: each pop gives the globally smallest node across all k lists in O(log k) — heap size stays k so it never grows with n.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n log k): each of n nodes pushed/popped, heap stays at size k.
Space O(k) heap — far better than O(n) from the brute sort.
Divide-and-conquer: pair lists and merge pairs repeatedly — also O(n log k).
Follow-up: k=2 reduces to LC 21 Merge Two Sorted Lists.
Follow-up: merge k sorted ARRAYS — same heap approach with (val, list_idx, elem_idx).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Merge the lists in pairs instead of using a heap.

Repeatedly merge neighbouring pairs with the two-list merge; there are log k rounds, each touching all N nodes: O(N log k) time and O(1) extra space (no heap).

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def merge_two(a, b):
    dummy = tail = ListNode()
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next


def merge_k_divide_and_conquer(lists):
    lists = [x for x in lists if x]
    while len(lists) > 1:
        lists = [
            merge_two(lists[i], lists[i + 1]) if i + 1 < l
en(lists) else lists[i]
            for i in range(0, len(lists), 2)
        ]
    return lists[0] if lists else None
```

The lists are too large for memory (files or streams).

The heap approach already works on streams: it holds one item per list. Read each input through an iterator and write the output as you go; this is the merge phase of an external sort (heapq.merge does it). Cost: O(N log k) time and O(k) memory; one sequential read per list.

Why not merge the lists one by one into a running result?

Merging list i into a result of size about i * (N / k) costs O(i * N / k), so the total is O(k * N), far worse than O(N log k).

OFFICIAL PAGE TEXT

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.

HARD | PracHub | Technical Screen | 38 of 196

Find maximum value in huge text file

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 166 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: written and tested here for this exact question
This question has several parts, so each part has its own answer below.
LeetCode match: No LC match (systems / scalability question; the Hard tag looks inflated)
Original: https://prachub.com/interview-questions/find-maximum-value-in-huge-text-file

QUESTION

You are given the path to a **very large** text file on disk. The file has the following properties:

- Each line contains exactly one signed 64-bit integer (in decimal).
- The file size can be tens or hundreds of gigabytes, so it **cannot** be fully loaded into memory.
- You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks).

Tasks

1.
Single-machine solution Design an algorithm to compute the **maximum integer value** in this file.

- Assume you have limited RAM (much smaller than the file size).
- Discuss how you would read the file and track the result.
- Give high-level pseudocode.
- Analyze time and space complexity. 2.

Performance improvement / parallelization Suppose you now have a multi-core machine or multiple machines available and want to speed up the computation.

- How would you modify your approach to take advantage of parallelism (e.g., processing the file in chunks)?

- Explain how you would combine partial results to get the global maximum.
- Mention any edge cases or failure scenarios you would consider (e.g., very large values, empty file). Assume you do not have a database; you are working directly with the file system and your programming language's standard I/O libraries.

APPROACH

Stream the file line by line (or in large buffered chunks) keeping a single running max initialized to None/-2^63; O(n) time, O(1) memory. Parallelize by splitting byte ranges aligned to line boundaries across cores/machines (map), then take the max of the partial maxima (reduce). Edge cases: empty file, trailing newline, malformed lines, negative-only values, retries when a worker fails.

INTERVIEW-DAY PLAN

Clarify first

Ask for the file size relative to RAM, whether a line can be malformed, whether the file can be empty, and whether there is one machine or a cluster.

Brute force to say out loud

Load the file and call max. Say why it fails (memory), then move on.

Optimal idea

Stream line by line with one running maximum initialized to None (not zero, so negative-only files work). O(n) time, O(1) memory. To parallelize split the file into byte ranges aligned to line boundaries: each worker discards its partial first line (the previous worker reads it) and finishes the line that crosses its end. Reduce with max of the partial results; retry failed ranges since each task is idempotent.

Dry-run these cases

Empty file, one line, all negative values, last line without newline, values at the 64-bit limits.

Likely follow-ups

Top K values, the median of a huge file, or distributed execution with failures.

Pitfalls

Starting the max at zero. Splitting ranges mid-line.

HOW TO THINK ABOUT IT

The answer is a running maximum, which needs one variable, so memory should not depend on the file size. Read the file as a stream. To go faster across cores, give each worker a byte range that starts and ends on line boundaries; the global maximum is the maximum of the partial maxima.

Basis: Systems / streaming question, no LeetCode match: a streaming reduce and its map-reduce parallelization.

BRUTE FORCE - PSEUDO-CODE

```
numbers = every line parsed to an integer, stored in a lis
t
return max(numbers)
```

Time: O(n) | Space: O(n)

Why it is slow

Needs memory for the whole file, which is exactly what is not available at tens of gigabytes. Streaming with one running maximum uses O(1) memory.

OPTIMAL SOLUTION - SINGLE MACHINE: STREAM THE FILE, KEEP ONE RUNNING MAXIMUM

```python
def max_in_file(path):
    best = None
    with open(path, "r", buffering=1 << 20) as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            value = int(line)
            if best is None or value > best:
                best = value
    return best
```

Time: O(n) | Space: O(1) plus the read buffer

Why

Never load the file. Read it through a buffered iterator and keep one variable, the maximum so far, started at None so that an all-negative file or a one-line file is handled correctly (do not start at 0). Python ints are unbounded, but in a fixed-width language parse into 64 bits and treat overflow as bad input. Disk read speed is the bottleneck, not the comparison.

OPTIMAL SOLUTION - PARALLEL: SPLIT BYTE RANGES ON LINE BOUNDARIES (MAP), MAX OF THE MAXIMA (REDUCE)

```python
def _max_in_range(args):
    path, start, end = args
    best = None
    with open(path, "rb") as f:
        if start > 0:
            f.seek(start - 1)
            f.readline()  # finish the line that began bef
ore this range
        while f.tell() <= end:
            line = f.readline()
            if not line:
                break
            line = line.strip()
            if line:
                value = int(line)
                if best is None or value > best:
                    best = value
    return best
```

Time: O(n / workers) plus I/O limits | Space: O(workers)

Why

Divide the file into roughly equal byte ranges. Each worker seeks to its start and, unless it starts at byte 0, discards the partial line it landed in (the previous worker finishes that line). It reads lines until the line it just read began past its end offset. Every line is therefore processed exactly once. The reducer takes the max of the partial results and ignores workers whose range was empty.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Edge cases to mention.

Empty file (return None or raise, by contract); a final line without a newline; blank lines; malformed lines; values at the 64-bit extremes; an all-negative file (do not initialize the max to 0); and Windows line endings. Decide whether a bad line fails the job or is skipped and counted.

How do you scale to many machines and handle failures?

Assign byte ranges (map tasks) to workers through a coordinator, and make each task idempotent (it only returns its maximum), so a failed or slow worker is simply retried or speculatively re-executed elsewhere. The reduce step is a max over a tiny list. On a distributed filesystem, align ranges to its block size for locality. Cost: O(n / workers) wall-clock time plus O(workers) to combine; total work stays O(n).

Is it I/O-bound or CPU-bound, and why does that matter?

Parsing integers is cheap compared with disk throughput, so one sequential reader often saturates the disk and extra threads help only if the data sits on several disks or on fast SSD or network storage. Measure first; the parallel version pays off mainly across machines or storage devices. Cost: reading takes about n / disk throughput and parsing is O(n) as well, so extra threads help only beyond the disk's speed.

The file has billions of lines and one machine is too slow.

Split by byte offsets across machines (each skips to the next newline), compute local maxima, and take the maximum of those few numbers. Tasks are idempotent, so a failed worker can simply be retried. Cost: O(n / machines) per machine and O(machines) to combine; a retry re-runs only the failed byte range.

Now return the K largest values instead of only the maximum.

Keep a min-heap of size K while streaming: push a value if the heap is not full, or replace the root when the value is larger. Memory O(K), time O(n log K); merge the per-worker heaps for a parallel version.

```python
def top_k_in_file(path, k):
    heap = []
    with open(path) as f:
        for line in f:
            if line.strip():
                v = int(line)
                if len(heap) < k:
                    heapq.heappush(heap, v)
                elif v > heap[0]:
                    heapq.heapreplace(heap, v)
    return sorted(heap, reverse=True)
```

HARD | 1Point3Acres | onsite-coding | 39 of 196

Huffman Tree Construction

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 170 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

QUESTION

Examples:
{"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45} -> standard Huffman tree with f near the root.

APPROACH

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

INTERVIEW-DAY PLAN

Clarify first

Ask whether to return the tree, a code table, or the encoded message, how ties are broken, and what to do with one symbol.

Brute force to say out loud

Repeatedly scan for the two smallest weights. O(n^2).

Optimal idea

Min-heap of nodes keyed by (weight, counter). Pop two, merge under a new node with the summed weight, push back, until one node remains. Walk the tree assigning '0' to left and '1' to right. O(n log n). Decode by walking the tree bit by bit. The counter prevents comparing nodes on ties.

Dry-run these cases

The CLRS example (total cost 224, 'f' gets a one-bit code). One symbol (code '0'). Equal weights.

Likely follow-ups

Canonical codes, decoding, or the two-queue O(n) method on sorted input.

Pitfalls

Comparing tree nodes on equal weights. Forgetting the single symbol case.

HOW TO THINK ABOUT IT

The two rarest symbols belong at the deepest level as siblings, so merge them first, treat the merged node as a new symbol with the combined weight, and repeat. 'Take the two smallest' is the job of a min-heap; tie-breaking with a counter keeps the heap from comparing tree nodes.

Basis: Classic greedy algorithm (CLRS 16.3); related to LeetCode 1167, Minimum Cost to Connect Sticks.

BRUTE FORCE - PSEUDO-CODE

```
items = the symbol weights
while more than one item: find the two smallest weights, r
emove them, add their sum
    back (cost += sum)
return the total cost
```

Time: O(n^2) | Space: O(n)

Why it is slow

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

OPTIMAL SOLUTION - MIN-HEAP MERGE OF THE TWO LIGHTEST NODES

```
def huffman_codes(freq):
    heap = [(w, i, sym, None, None) for i, (sym, w) in enu
merate(sorted(freq.items()))]
    if not heap:
        return {}
    heapq.heapify(heap)
    counter = len(heap)
    while len(heap) > 1:
        a = heapq.heappop(heap)
        b = heapq.heappop(heap)
        heapq.heappush(heap, (a[0] + b[0], counter, None,
a, b))
        counter += 1
    root = heap[0]
    codes = {}
    stack = [(root, "")]
    while stack:
        node, prefix = stack.pop()
        if node[2] is not None:
            codes[node[2]] = prefix or "0"
        else:
            stack.append((node[4], prefix + "1"))
            stack.append((node[3], prefix + "0"))
    return codes
```

Time: O(n log n) | Space: O(n)

Why

Put every symbol in a min-heap keyed by frequency, repeatedly pop the two lightest nodes, merge them under a new internal node with the summed weight and push it back. The last node is the root. A counter is the second heap key so ties never compare tree nodes. Codes come from a DFS: left edge '0', right edge '1'. A single-symbol alphabet needs the special case of code '0'.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

```
def huffman_codes(freq):
    heap = [(w, i, sym, None, None) for i, (sym, w) in enu
merate(sorted(freq.items()))]
    if not heap:
        return {}
    heapq.heapify(heap)
    counter = len(heap)
    while len(heap) > 1:
        a = heapq.heappop(heap)
        b = heapq.heappop(heap)
        heapq.heappush(heap, (a[0] + b[0], counter, None,
a, b))
        counter += 1
    root = heap[0]
    codes = {}
    stack = [(root, "")]
    while stack:
        node, prefix = stack.pop()
        if node[2] is not None:
            codes[node[2]] = prefix or "0"
        else:
            stack.append((node[4], prefix + "1"))
            stack.append((node[3], prefix + "0"))
    return codes


def canonical_codes(freq):
    lengths = {c: len(code) for c, code in huffman_codes(f
req).items()}
    out, code, prev = {}, 0, 0
    for c in sorted(lengths, key=lambda x: (lengths[x], x)
):
        code <<= lengths[c] - prev
        out[c] = format(code, f"0{lengths[c]}b")
        code, prev = code + 1, lengths[c]
    return out
```

NOTES FROM CANDIDATES

L6 onsite — interviewer expects clean code with a documented tree node, plus a discussion of encoding / decoding correctness.
Common follow-up: how to produce the code table (DFS from root, accumulating bits).
Trade-off discussion: Huffman is optimal for known per-symbol probability but not for adaptive / streaming inputs.

PREPARATION

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

HARD | IN APP LC 146 | DESIGN-STYLE (skippable) | Exponent | 40 of 196

★ TOP 75: rank 64 in the Medium and Hard priority list

★ Implement LRU Cache.

STUDY: TYPE | Google frequency on LeetCode: all time 57.3, last 30 days 0 | book priority 42 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #146)

LeetCode match: LC 146 LRU Cache

+2562 interviews

Original: https://www.aced.io/questions/3625/lru-cache

QUESTION

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time? Pythonclass LRUCache(n): set(key, value) get(key) Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated. Pythoncache = LRUCache(2)

Limit of 2 items cache.set('user1',

'Alex') cache.set('user2', 'Brian') cache.set('user3', 'Chris') Here user1 is empty because it was the least recently used and thus removed to make room for user3: Pythoncache.get('user1')

=> None cache.get('user2')

=> 'Brian' cache.get('user3')

=> 'Chris'Hint 1If you're having

trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array

APPROACH

Hash map + doubly linked list (or OrderedDict): get/put move the node to the front, evict from the tail when over capacity. O(1) per op.

HOW TO THINK ABOUT IT

You need two things at once: find a key quickly (a hash map) and know which key is the oldest use (an ordering you can update in O(1)). A doubly linked list gives O(1) move-to-front and remove-from-back, and each hash map entry points at its list node. Python's OrderedDict is exactly that pair; in an interview be ready to build the linked list by hand.

Basis: LeetCode 146, LRU Cache: get and put in O(1), evicting the least recently used key when over capacity.

BRUTE FORCE - PSEUDO-CODE

```
keep a list of (key, value) with the most recently used la
st
get(k): search the list; if found, move it to the end and
return the value
put(k, v): search and remove an existing entry or drop the
 first (oldest) when full;
    append (k, v)
```

Time: O(n) per operation | Space: O(capacity)

Why it is slow

Every operation scans the list, O(capacity). A hash map to nodes of a doubly linked list gives O(1) lookup and O(1) move-to-front and eviction.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #146 LRU CACHE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Design an LRU cache with a fixed capacity: get(key)->value or -1, and put(key,value).
When capacity is exceeded, evict the least recently used item.
Both operations must be O(1) average time. Right?"

"A few quick questions:
Does get count as a use (making the item recently used)? Yes.
Does put update an existing key's value and recency? Yes."

"Let me walk capacity=2: put(1,1), put(2,2), get(1)->1 (1 is now MRU (Most Recently Used)),
put(3,3) evicts 2, get(2)->-1 ok

PHASE 3 — OPTIMIZE

The bottleneck is the O(n) list shift on every access.
HashMap + Doubly Linked List gives O(1) for both:
HashMap: key -> DLL node (O(1) lookup).
DLL: maintains access order — head=LRU, tail=MRU.
get: move node to tail. put: add at tail; if over capacity, remove from head.
Use sentinel head/tail nodes to avoid edge-case pointer checks.

Time: O(1) all ops. Space: O(capacity).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _146_LRUCache:
    class _Node:
        def __init__(self, key=0, val=0):
            self.key = key; self.val = val; self.prev =
self.next = None

    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.head = self._Node(); self.tail = self._Node
()
        self.head.next = self.tail; self.tail.prev = sel
f.head

    def _remove(self, node):
        node.prev.next = node.next; node.next.prev = nod
e.prev

    def _insert_tail(self, node):
        node.prev = self.tail.prev; node.next = self.tai
l
        self.tail.prev.next = node; self.tail.prev = nod
e

    def get(self, key):
        if key not in self.map: return -1
        node = self.map[key]
        self._remove(node); self._insert_tail(node)
        return node.val

    def put(self, key, value):
        if key in self.map: self._remove(self.map[key])
        node = self._Node(key, value)
        self.map[key] = node; self._insert_tail(node)
        if len(self.map) > self.cap:
            lru = self.head.next
            self._remove(lru); del self.map[lru.key]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace LRUCache(capacity=2): put(1,1),put(2,2),get(1),put(3,3),get(2)."
OrderedDict (or doubly linked list + hash map).
put(1,1): cache={1:1}. Order:[1].
put(2,2): cache={1:1,2:2}. Order:[1,2].
get(1): move 1 to end (most recent). Order:[2,1]. Return 1 ok
put(3,3): capacity full. Evict LRU = front of order = 2.
cache={1:1,3:3}. Order:[1,3].
get(2): 2 not in cache -> return -1 ok
WHY LRU = front of OrderedDict: least recently used is at the "oldest" end.
move_to_end() on access keeps ordering correct in O(1).
Doubly linked list: head.next=LRU_node...MRU_node.next=tail.
After get(1): move node(1) to MRU:
node(1).prev.next=node(1).next; node(1).next=tail;
tail.prev=node(1). head.next=node(2) now.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(1) all ops: HashMap O(1) lookup + DLL O(1) insert/remove.
Space O(capacity): at most capacity nodes in the DLL and map.
Python's OrderedDict implements this internally — valid in an interview to mention.
Follow-up: LFU Cache (#460) — evict least FREQUENTLY used; needs two hashmaps + DLL per frequency.
Follow-up: thread-safe LRU — wrap get/put with a threading.Lock.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

OrderedDict version

```
class LRUCacheOrdered:
    def __init__(self, capacity):
        self.cap, self.d = capacity, OrderedDict()

    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]

    def put(self, key, value):
        if key in self.d:
            self.d.move_to_end(key)
        self.d[key] = value
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

Time: O(1) per operation | Space: O(capacity)

Why

OrderedDict remembers insertion order and supports move_to_end and popitem(last=False) in O(1), so it is the hash map and the recency list in one. Good for a quick answer once you can explain what it does underneath.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Many threads share the cache.

Wrap get and put in one lock (both reorder the list, so even get is a write). For more throughput shard the cache by key hash, each shard with its own lock and capacity.

```
class LRUCacheOrdered:
    def __init__(self, capacity):
        self.cap, self.d = capacity, OrderedDict()

    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]

    def put(self, key, value):
        if key in self.d:
            self.d.move_to_end(key)
        self.d[key] = value
        if len(self.d) > self.cap:
            self.d.popitem(last=False)


class LRUCacheThreadSafe(LRUCacheOrdered):
    def __init__(self, capacity):
        super().__init__(capacity)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return super().get(key)

    def put(self, key, value):
        with self.lock:
```

```
            super().put(key, value)
```

Entries expire after a time to live (TTL).
Store an expiry time with each entry, check it on get (a stale entry is treated as missing and removed), and optionally sweep from the old end of the list on put. The recency order and the O(1) operations are unchanged.

Least frequently used (LFU) instead of least recently used.
Keep a frequency per key and buckets of keys per frequency (each bucket an ordered dict) with a pointer to the minimum frequency; get and put move a key to the next bucket in O(1) (LeetCode 460).

OFFICIAL PAGE TEXT

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time?

```
Pythonclass LRUCache(n):
  set(key, value)
  get(key)
```

Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated.

```
Pythoncache = LRUCache(2) # Limit of 2 items

cache.set('user1', 'Alex')
cache.set('user2', 'Brian')
cache.set('user3', 'Chris')
```

Here user1 is empty because it was the least recently used and thus removed to make room for user3:

```
Pythoncache.get('user1') # => None
cache.get('user2') # => 'Brian'
cache.get('user3') # => 'Chris'
```

Hint 1If you're having trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array won't work because moving items around takes O(n) time. Could you combine two different data structures to maintain this order and also get fast lookups?Hint 3You can use a doubly linked list to store the items in the correct order. When an item is used, just move its node to the front of the list. The least recently used node will always end up at the end of the list.Hint 4To look up items without traversing the entire list, you can also add a hash table that maps from the key to the item's node.Challenge 1Whats the time complexity of your solution?This problem is challenging because of the combination of constraints such as key-value lookup, size limits, and time complexity. Ultimately, there is no single data structure that fulfills all of these needs, so we have to get creative! Our solution is to implement the cache class with a linked list and a hash table. The linked list is doubly-linked, which allows us to insert and remove items in constant time without traversing the entire list. Every time an item is set, updated, or retrieved, we just move its Node to the front of the list. When the cache reaches its size limit, we can easily make room by removing the item at the end of the list. The linked list gives us O(1) insertion time, but by itself it has O(n) lookup time because we have to traverse the entire list to find a particular item. That's not ideal since we want to be able to look up an item quickly using its key. That's where the hash table comes in: we can use it add a mapping from an item's key to its Node. Together, these data structures give us a time complexity of O(1) for insertion, deletion, and retrieval in the worst case, and a space footprint of O(n). No answers contributed by the community yet.

Trie and Prefix Indexing 1 questions

HARD | IN APP LC 648 | PracHub | Technical Screen | 41 of 196

Implement Longest-Match Text Replacement

STUDY: TALK | Google frequency on LeetCode: all time 31.9, last 30 days 0 | book priority 109 of 196
Topic: Trie and Prefix Indexing
Optimal answer from: the interview site's own solution
Also in your app (open the app for its Expose): LeetCode #648 Replace Words
LeetCode match: LC 648 Replace Words / LC 833 Find And Replace in String
Original: https://prachub.com/coding-questions/implement-longest-match-text-replacement

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed. Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2", "xy
": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **0**, both **"ab"** and **"abc"** match, so choose the longest token **"abc"** and output **"2"**. Then **"d"** has no match, so output it literally. Then **"xy"** matches and outputs **"3"**. Finally, **"z"** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?
Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints
0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.
Examples
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
Expected Output: ["2", "d", "3", "z"]
Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.
Hints
A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.
Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints
0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

APPROACH

Build a trie of tokens; scan the text and at each position walk the trie, remembering the longest terminal match; emit its id and jump ahead, else copy the char. Aho-Corasick if the text is large.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary token starting there. Checking every token costs the size of the dictionary per character, so share prefixes in a trie: walk the text from the current position down the trie, remember the last node that ends a token, and stop when the trie has no further edge. The remembered node is the longest match.

Basis: LeetCode 648 (Replace Words) and the longest-prefix matching used in tokenizers and routing tables. Scan left to right, at each position take the longest dictionary token that matches, else copy the character.

BRUTE FORCE - PSEUDO-CODE

```
i = 0; while i < len(text): best = the longest dictionary
token that text starts with
    at i (test every token)
if there is one: output its id, i += len(best); else outpu
t text[i], i += 1
```

Time: O(len(text) * tokens * token length) | Space: O(1) extra

Why it is slow

Each position tests every token. A trie of the tokens is walked from i, remembering the last token end, so a position costs at most the longest token.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(text, dictionary):
    END = "__end__"
    trie = {}

    for token, replacement_id in dictionary.items():
        if not token:
            continue
        node = trie
        for ch in token:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node[END] = replacement_id

    result = []
    i = 0
    n = len(text)

    while i < n:
        node = trie
        j = i
        best_id = None
        best_len = 0

        while j < n and text[j] in node:
            node = node[text[j]]
            j += 1
            if END in node:
```

```
                best_id = node[END]
                best_len = j - i

        if best_id is None:
            result.append(text[i])
            i += 1
        else:
            result.append(best_id)
            i += best_len

    return result
```

Time: O(S + N * L), where S is the sum of token lengths, N is the text length, and L is the maximum token length | Space: O(S)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

```
def replace_with_priority(text, entries, key):
    """Follow-up: the winner among matching tokens is chos
en by an arbitrary comparator key(token, id, order)."""
    out, i = [], 0
    while i < len(text):
        matches = [
            (tok, ident, order)
            for order, (tok, ident) in enumerate(entries)
            if text.startswith(tok, i)
        ]
        if matches:
            tok, ident, _ = max(matches, key=lambda m: key
(*m))
            out.append(ident)
            i += len(tok)
        else:
            out.append(text[i])
            i += 1
    return out
```

The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. Cost: still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

FUNCTION AND TEST CASES

Signature

```
def solution(text: str, dictionary: dict[str, str]) -> lis
t[str]:
```

Sample call

```
solution("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
```

Test cases

Input	Expected
`("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})`	`["2", "d", "3", "z"]`
`("aaaaa", {"a": "X", "aa": "Y"})`	`["Y", "Y", "X"]`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Greedy Longest-Match Dictionary Tokenizer | 1Point3Acres | phone-screen | asked 2026-06-15 | Low

https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.
Two natural shapes:
Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.
Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).
Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).
If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

Preparation: Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

Medium questions (135)

BFS/DFS Graph and Tree Traversal and Shortest Paths 35 questions

MEDIUM | IN APP LC 1650 | HackTheRounds | Phone Screen | 42 of 196

Lowest Common Ancestor of a Binary Tree III

STUDY: TALK | Google frequency on LeetCode: all time 14.7, last 30 days 0 | book priority 149 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #1650 Lowest Common Ancestor of a Binary Tree III

LeetCode match: LC 1650

QUESTION

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA). Each node has a parent pointer.

EXAMPLES

Example 1	
Input	root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output	3
Why	The LCA of nodes 5 and 1 is 3.

Example 2	
Input	root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output	5
Why	The LCA of nodes 5 and 4 is 5.

Example 3	
Input	root = [1,2], p = 1, q = 2
Output	1
Why	Node 1 is ancestor of node 2.

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

APPROACH

Parent pointers: two-pointer swap trick, or a set of ancestors

HOW TO THINK ABOUT IT

With parent pointers each node is the head of a linked list that ends at the root, so the question is where two linked lists meet. If the lists have different lengths, walk each pointer to the end and then restart it on the other node: both pointers then travel the same total distance and arrive at the meeting node together.

Basis: LeetCode 1650, Lowest Common Ancestor of a Binary Tree III (every node has a parent pointer).

BRUTE FORCE - PSEUDO-CODE

```
ancestors = {}; walk from p up to the root adding every no
de (including p)
walk from q up; the first node already in ancestors is the
 answer
```

Time: O(h) | Space: O(h)

Why it is slow

Uses O(h) extra space for the set. Two pointers that switch to the other node's start when they fall off the root meet at the answer with O(1) space.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: FIND LCA USING RECURSIVE DFS APPROACH

```
class Solution:
    def lowestCommonAncestor(self, arr, p_val, q_val):
        """
        Find LCA using recursive DFS approach.
        Returns the node if it matches target, or the LCA
if both targets found in subtrees.

        Args:
            arr: Binary tree in level-order array format
            p_val: Value of first node
            q_val: Value of second node

        Returns:
            Value of LCA node

        Time: O(n) - may need to visit all nodes
        Space: O(h) - recursion depth equals tree height
        """
        if not arr or arr[0] is None:
            return None

        class TreeNode:
            def __init__(self, val):
                self.val = val
                self.left = None
                self.right = None

        # Build tree
        nodes = [TreeNode(v) if v is not None else None fo
r v in arr]
        for i, node in enumerate(nodes):
            if node:
                left_idx = 2 * i + 1
                right_idx = 2 * i + 2
                if left_idx < len(nodes):
                    node.left = nodes[left_idx]
                if right_idx < len(nodes):
                    node.right = nodes[right_idx]
        root = nodes[0]

        def find_lca(node, p_val, q_val):
            if not node:
                return None
            if node.val == p_val or node.val == q_val:
```

```
            return node
        left = find_lca(node.left, p_val, q_val)
        right = find_lca(node.right, p_val, q_val)
        if left and right:
            return node
        return left or right

    result = find_lca(root, p_val, q_val)
    return result.val if result else None
```

Time: O(n) | Space: O(h)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Do it without the swap trick (it feels like magic).

Compute the depth of each node by walking up, lift the deeper one by the difference, then step both up together until they meet. Same O(h) time and O(1) space and easier to justify.

```
class PNode:
    def __init__(self, val):
        self.val, self.left, self.right, self.parent = val
, None, None, None


def _depth(n):
    d = 0
    while n.parent:
        n, d = n.parent, d + 1
    return d


def _lca_depth(p, q):
    dp, dq = _depth(p), _depth(q)
    while dp > dq:
        p, dp = p.parent, dp - 1
    while dq > dp:
        q, dq = q.parent, dq - 1
    while p is not q:
        p, q = p.parent, q.parent
    return p
```

Many LCA queries on the same tree.

Preprocess with binary lifting (up[k][v] is the 2^k-th ancestor): O(n log n) to build, O(log n) per query. Or an Euler tour with a range-minimum structure for O(1) queries.

The nodes might not be in the same tree.

Then the two pointers both end as None after two passes, and None is the correct answer. The loop terminates because both pointers reach None at the same step.

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 43 of 196

Count Islands in a Grid and Track the Count as Land Cells Are Added

STUDY: TALK | Google frequency on LeetCode: all time 36, last 30 days 0 | book priority 94 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with `m` rows and `n` columns in which every cell is land (`1`) or water (`0`). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[t
uple[int, int]]) -> list[int]:
```

Rules

- `grid[r][c]` is `1` for land and `0` for water, where `r` is the row index and `c` is the column index.
- An addition `(r, c)` turns cell `(r, c)` into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length `len(additions) + 1`. Element `0` is the number of islands in the initial grid. For `i >= 1`, element `i` is the number of islands after the first `i` additions have been applied.

Constraints

- `1 <= m <= 1000`, `1 <= n <= 1000` and `m * n <= 200000`
- Every row of `grid` has exactly `n` entries, each `0` or `1`.
- `0 <= len(additions) <= 200000`
- Every addition `(r, c)` satisfies `0 <= r < m` and `0 <= c < n`. The same cell may appear more than once.
- Every count is at most `m * n`, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions
 = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

A map is a grid with m rows and n columns in which every cell is land (1) or water (0). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map grid and a sequence of additions, each of which turns one cell into land. Each addition is given as a pair [r, c]. Report the number of islands before any addition, and again after each addition. Rules: - grid[r][c] is 1 for land and 0 for water, where r is the row index and c is the column index.

- An addition [r, c] turns cell (r, c) into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length len(additions) + 1. Element 0 is the number of islands in the initial grid. For i >= 1, element i is the number of islands after the first i additions have been applied. Constraints:
- 1 <= m <= 1000, 1 <= n <= 1000 and m * n <= 200000 - Every row of grid has exactly n entries, each 0 or 1.
- 0 <= len(additions) <= 200000 - Every addition [r, c] satisfies 0 <= r < m and 0 <= c < n.

APPROACH

Union-find over cells: start with initial islands, then each addition creates a component and unions with the four neighbors; track the component count. Ignore repeated cells. O((mn + k) alpha).

HOW TO THINK ABOUT IT

Re-running a flood fill after every addition costs O(mn) each time. Land only ever appears, so islands only merge: Union-Find fits exactly. A new land cell starts as a new island (count + 1) and every union with a neighbouring land cell that belongs to a different island removes one island (count - 1).

Basis: LeetCode 305, Number of Islands II. Report the count before any addition and after each; adding a cell that is already land changes nothing.

BRUTE FORCE - PSEUDO-CODE

```
after every addition: set the cell to land and run a full
flood-fill count of islands
```

Time: O(A * m * n) | Space: O(m * n)

Why it is slow

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def island_counts(grid, additions):
    m = len(grid)
    n = len(grid[0]) if m > 0 else 0
    total = m * n
    parent = list(range(total))
    size = [1] * total
    land = [False] * total
    count = 0

    def find(x):
        root = x
        while parent[root] != root:
            root = parent[root]
        while parent[x] != root:
            nxt = parent[x]
            parent[x] = root
            x = nxt
        return root

    def unite(i, j):
        nonlocal count
        if not land[j]:
            return
        a = find(i)
        b = find(j)
        if a == b:
            return
        if size[a] < size[b]:
            a, b = b, a
        parent[b] = a
        size[a] += size[b]
        count -= 1

    def activate(r, c):
        nonlocal count
        idx = r * n + c
        if land[idx]:
            return
        land[idx] = True
        count += 1
        if r > 0:
            unite(idx, idx - n)
        if r + 1 < m:
            unite(idx, idx + n)
        if c > 0:
            unite(idx, idx - 1)
        if c + 1 < n:
            unite(idx, idx + 1)

    for r in range(m):
        row = grid[r]
        for c in range(n):
            if row[c] == 1:
                activate(r, c)
    result = [count]
    for r, c in additions:
        activate(r, c)
        result.append(count)
    return result
```

Time: O((m*n + k) * alpha(m*n)), where k = len(additions) | Space: O(m*n + k)

Explanation

Treat every cell as a node of a disjoint-set (union-find) structure indexed by r * n + c, with path compression and union by size, and keep a running island count. Activating a cell that is already land does nothing. Activating a water cell marks it as land and increments the count (it starts as its own island); then, for each of its up to four in-bounds orthogonal neighbours that is land, the roots of the two cells are found and, if they differ, the two sets are merged and the count is decremented. The initial grid is built by activating its land cells in row-major order and the count is recorded; each addition is then activated in order and the count recorded again.

Invariant: the count always equals the number of disjoint sets among land cells, and two land cells share a set exactly when a horizontal/vertical path of land joins them. A new land cell can only connect itself to the islands of its orthogonal neighbours, so merging it with each distinct neighbouring set and decrementing once per successful merge preserves the invariant. Comparing roots before merging is what makes a cell that touches the same island from two or more sides decrement the count only once. Diagonal cells are never examined, so corner contact never connects islands, and neighbour indices are derived from (r, c) with explicit bounds checks against m and n, so the last cell of one row is never treated as adjacent to the first cell of the next.

Edge cases: empty additions returns only the initial count; repeated additions and additions on initial land repeat the previous count; an all-water grid starts at 0 and an all-land grid stays at 1. Every count is at most m * n <= 200000, so 32-bit integers suffice in every language.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. Cost: O(a * alpha) for a additions and O(a) memory, independent of the grid size.

Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. Cost: O(alpha) per addition with size tracking; the maximum is one running variable.

```
def largest_island_after_each(grid, additions):
    parent, size, best, out = {}, {}, 0, []

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def add(r, c):
        nonlocal best
        if (r, c) in parent:
            return
        parent[(r, c)], size[(r, c)] = (r, c), 1
        for nb in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if nb in parent:
                a, b = find((r, c)), find(nb)
                if a != b:
                    if size[a] < size[b]:
                        a, b = b, a
                    parent[b] = a
                    size[a] += size[b]
        best = max(best, size[find((r, c))])

    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1:
                add(r, c)
    out.append(best)
    for r, c in additions:
        add(r, c)
        out.append(best)
    return out
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[list[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

MEDIUM | IN APP LC 1102 | PracHub | Technical Screen | 44 of 196

Find A Threshold-Limited Path With Minimum Required Safety

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 124 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #1102 Path With Maximum Minimum Value

LeetCode match: LC 1102 Path With Maximum Minimum Value (inferred from overview)

Original: https://prachub.com/coding-questions/find-a-threshold-limited-path-with-minimum-required-safety

QUESTION

Overview: Practice a graph path interview problem that asks for the minimum required safety value along a threshold-limited path. The prompt targets minimax path reasoning, filtered edges, Dijkstra-style relaxation, disconnected graphs, and edge-case handling.

APPROACH

Minimax Dijkstra: heap keyed by the best bottleneck (min safety along the path) while skipping edges beyond the threshold; or binary search the safety value + BFS.

HOW TO THINK ABOUT IT

When the quality of a path is decided by its weakest edge, the best path does not need to be short, only strong. Two equivalent tools: add edges from strongest to weakest with Union-Find until source and target connect (that edge's value is the answer), or run Dijkstra with a max-heap where extending a path takes the minimum so far. Disconnected graphs must return the 'no path' value.

Basis: LeetCode 1102 (Path With Maximum Minimum Value) and the bottleneck path on a graph. Assumed form (confirm it): edges (u, v, safety); the safety of a path is its smallest edge; find the best achievable path safety from source to target, or -1 if disconnected.

BRUTE FORCE - PSEUDO-CODE

```
depth-first search over every simple path from the source
to the target
for each path take its smallest edge safety; answer = the
largest of those
```

Time: exponential | Space: O(path)

Why it is slow

Exponential. Adding edges from strongest to weakest in a Union-Find until the source and target connect is O(E log E).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(n, edges, start, end, threshold):
    if start == end:
        return 0
    adj = [[] for _ in range(n)]
    for u, v, w in edges:
        if w <= threshold:
            adj[u].append((v, w))
            adj[v].append((u, w))
    import heapq

    dist = [float("inf")] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        cost, node = heapq.heappop(heap)
        if node == end:
            return cost
        if cost != dist[node]:
            continue
        for nxt, w in adj[node]:
            nc = max(cost, w)
            if nc < dist[nxt]:
                dist[nxt] = nc
                heapq.heappush(heap, (nc, nxt))
    return -1
```

Time: O((n + m) log n) using a minimax Dijkstra variant. | Space: O(n + m).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

```python
def best_path_safety_dijkstra(n, edges, s, t):
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))
    best = {s: float("inf")}
    heap = [(-float("inf"), s)]
    while heap:
        neg, u = heapq.heappop(heap)
        if -neg < best.get(u, -1):
            continue
        if u == t:
            return -neg
        for v, w in adj[u]:
            val = min(-neg, w)
            if val > best.get(v, -1):
                best[v] = val
                heapq.heappush(heap, (-val, v))
    return -1
```

Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

FUNCTION AND TEST CASES

Sample call

```
solution(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)
```

Test cases

Input	Expected
(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)	5
(3, [(0,1,7),(1,2,3)], 0, 2, 5)	-1

MEDIUM | PracHub | Technical Screen | 45 of 196

Count Clusters of 2D Points Within a Radius

STUDY: TALK | Google frequency on LeetCode: all time 40.7, last 30 days 0 | book priority 81 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (inferred from overview)

Original: https://prachub.com/coding-questions/count-clusters-of-2d-points-within-a-radius

QUESTION

Overview: This question tests graph traversal skills, specifically identifying connected components among a set of 2D points using distance-based adjacency. It assesses the ability to model a geometric problem as a graph and apply DFS, BFS, or Union-Find, a common evaluation of algorithmic thinking in coding interviews. It also probes awareness of efficiency trade-offs when scaling to large point sets.

APPROACH

Union-find over points with distance <= r; count roots. Use squared distance; spatial grid hashing for speed.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the distance is Euclidean, whether the radius is inclusive (a distance of at most r connects), whether points are integers, and the number of points.

Brute force to say out loud

Union-Find over all pairs of points, comparing every pair. O(n^2) distance checks.

Optimal idea

Compare squared distances to r squared (no square roots, exact for integers). For many points hash them into grid cells of side r so each point only checks its own and the eight neighboring cells; use Union-Find to merge and count the components with distinct roots. About O(n) expected for evenly spread points.

Dry-run these cases

No points, one point, two points exactly r apart (connected if inclusive), a chain of points each close to the next (one cluster), duplicate points.

Likely follow-ups

Dynamic points, a very large radius, or reporting the largest cluster.

Pitfalls

Using the cell size smaller than r so close points fall in non-adjacent cells. Comparing floating-point distances for equality.

HOW TO THINK ABOUT IT

Model it as a graph where points within r are neighbours and count connected components with Union-Find. The all-pairs check is O(n^2); to speed it up, hash points into square cells of side r so that only points in the same or an adjacent cell can be within r. Compare squared distances to r * r to avoid square roots.

Basis: Connected components of a distance graph (LeetCode 547 idea). Two points are connected when their Euclidean distance is at most r; a cluster is a connected component.

BRUTE FORCE - PSEUDO-CODE

```
union-find with one set per point
for every pair of points: if their distance is at most r:
union them
answer = the number of distinct roots
```

Time: O(n^2) | Space: O(n)

Why it is slow

n^2 / 2 distance checks. Hashing points into grid cells of side r means each point is compared only with points in its own and neighbouring cells.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def countClusters(points, r):
    n = len(points)
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra != rb:
            parent[ra] = rb

    r2 = r * r
    for i in range(n):
        xi, yi = points[i][0], points[i][1]
        for j in range(i + 1, n):
            dx = xi - points[j][0]
            dy = yi - points[j][1]
            if dx * dx + dy * dy <= r2:
                union(i, j)

    return sum(1 for i in range(n) if find(i) == i)
```

Time: O(n^2 * alpha(n)) — every pair is checked with near-constant Union-Find operations. This matches the brute-force approach the interviewer preferred; a spatial grid/index can reduce the pairwise cost when points are sparsely distributed. | Space: O(n) for the Union-Find parent array.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(i)). The unions are the same.

```python
def _find_factory(n):
    parent = list(range(n))
```

```
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    return parent, find


def largest_cluster(points, r):
    parent, find = _find_factory(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if (points[i][0] - points[j][0]) ** 2 + (
                points[i][1] - points[j][1]
            ) ** 2 <= r * r:
                parent[find(i)] = find(j)
    return max(Counter(find(i) for i in range(len(points))
).values(), default=0)
```

Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. Cost: grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

FUNCTION AND TEST CASES

Signature

```
def countClusters(points: list[list[int]], r: int) -> int:
```

Sample call

```
countClusters([[0, 0], [1, 0], [10, 10]], 2)
```

Test cases

Input	Expected
`([[0, 0], [1, 0], [10, 10]], 2)`	`2`
`([[0, 0], [3, 0], [6, 0]], 3)`	`1`

ALSO REPORTED - 2 MORE REPORTS OF THIS QUESTION

Count Connected Clusters of Two-Dimensional Points | PracHub | Technical Screen

https://prachub.com/coding-questions/count-connected-clusters-of-two-dimensional-points

2D Points Connected Components by Radius | 1Point3Acres | phone-screen | asked 2026-08-04 | Low

https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

DFS, BFS, or Union-Find all fit the core connected-components model.

For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.

Clarify squared-distance comparison to avoid unnecessary square roots.

Preparation: Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

MEDIUM | PracHub | Technical Screen | 46 of 196

Match people to questions using tags and priorities

STUDY: TALK | **Google frequency on LeetCode: all time 5, last 30 days 0 | book priority 158 of 196**
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: the interview site's own solution
LeetCode match: LC 1820 Maximum Number of Accepted Invitations (bipartite matching) (inferred from overview)
Original: https://prachub.com/coding-questions/match-people-to-questions-using-tags-and-priorities

QUESTION

Overview: This question evaluates competencies in bipartite matching and combinatorial optimization, along with practical data-structure design for scalable tag-based assignment and priority-aware selection.

APPROACH

Model as bipartite matching: edges where tags overlap, weights by priority; use Kuhn's/Hopcroft-Karp for max matching, or greedy by priority with an inverted index tag -> candidates.

INTERVIEW-DAY PLAN

Clarify first

Ask whether matching is one-to-one or a question can take many people, what priority means (higher wins, or a numeric weight to maximize), whether a person needs only one shared tag, and whether it must be maximum cardinality or maximum total priority.

Brute force to say out loud

Try every assignment: exponential. State it and move on.

Optimal idea

Build an inverted index tag to candidate questions. If only the number of matches matters it is bipartite matching: Kuhn's augmenting-path algorithm in O(V * E) is enough, Hopcroft-Karp for scale. If priorities are weights use min-cost flow or the Hungarian algorithm. If priority is a strict order, process people by priority and augment, because matchings form a matroid and greedy by priority is optimal.

Dry-run these cases

A person with no tag overlap. Two people competing for one question. A chain where the first greedy choice blocks a larger matching (shows why plain greedy fails without augmenting).

Likely follow-ups

Capacities per question (turns into max flow), online arrivals, or fairness constraints.

Pitfalls

Assuming plain greedy gives a maximum matching. Not building the index and doing all-pairs tag checks.

HOW TO THINK ABOUT IT

'Assign each person at most one question, each question at most one person, allowed pairs given by tags' is maximum bipartite matching. The tag index only builds the edges faster. For maximum count use augmenting paths: try to give a person a question, and if taken ask its current owner to move. If priorities decide who is served, process people in priority order with the same augmenting step.

Basis: LeetCode 1820, Maximum Number of Accepted Invitations (bipartite matching). Assumed rules (confirm them): a person and a question can be matched if they share a tag; each person gets at most one question and each question at most one person; priority decides who is served first when not everyone can be matched.

BRUTE FORCE - PSEUDO-CODE

```
go(person): either skip this person, or give them any unus
ed question that shares a
    tag
answer = the largest number of matches found
```

Time: O((Q + 1)^P) | Space: O(P)

Why it is slow

Each person branches over all questions: exponential. Augmenting paths (move an earlier person to another question when needed) solve it in O(people * edges).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(people, question_tags, priorities):
    from collections import deque

    n = len(people)
    m = len(question_tags)
    people_sets = [set(tags) for tags in people]
    question_sets = [set(tags) for tags in question_tags]

    source = 0
    sink = 1 + n + m
    graph = [[] for _ in range(sink + 1)]

    def add_edge(u, v, cap, cost):
        graph[u].append([v, len(graph[v]), cap, cost])
        graph[v].append([u, len(graph[u]) - 1, 0, -cost])

    for i in range(n):
        add_edge(source, 1 + i, 1, 0)

    for j in range(m):
        add_edge(1 + n + j, sink, 1, -priorities[j])

    for i in range(n):
        if not people_sets[i]:
            continue
        for j in range(m):
            if question_sets[j] and not people_sets[i].isd
isjoint(question_sets[j]):
                add_edge(1 + i, 1 + n + j, 1, 0)

    flow = 0
    cost = 0
    INF = 10**18

    while True:
        dist = [INF] * (sink + 1)
        in_queue = [False] * (sink + 1)
        parent = [(-1, -1)] * (sink + 1)

        dist[source] = 0
        q = deque([source])
        in_queue[source] = True

        while q:
            u = q.popleft()
            in_queue[u] = False
            for ei, edge in enumerate(graph[u]):
                v, rev, cap, edge_cost = edge
                if cap > 0 and dist[u] + edge_cost < dist[
v]:
                    dist[v] = dist[u] + edge_cost
                    parent[v] = (u, ei)
                    if not in_queue[v]:
                        q.append(v)
                        in_queue[v] = True

        if dist[sink] == INF:
            break

        add = INF
        v = sink
        while v != source:
            u, ei = parent[v]
            add = min(add, graph[u][ei][2])
            v = u

        v = sink
        while v != source:
            u, ei = parent[v]
            edge = graph[u][ei]
            edge[2] -= add
            graph[v][edge[1]][2] += add
            cost += edge[3] * add
            v = u

        flow += add

    return (flow, -cost)
```

Time: O(F * V * E + N * M * L), where F is the matching size, V = N + M + 2, E is the number of flow edges, and L is the average cost of a set-disjointness check. | Space: O(E + N + M)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

```
def match_by_priority(adj, priority):
    """Serve people in priority order (higher first); the
matched set is the best possible for any priority-consiste
nt weights."""
    owner = {}

    def try_assign(p, seen):
```

```
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            if q not in owner or try_assign(owner[q], seen
):
                owner[q] = p
                return True
        return False

    order = sorted(range(len(adj)), key=lambda p: -priorit
y[p])
    matched = [p for p in order if try_assign(p, set())]
    return sorted(matched), {q: p for q, p in owner.items(
)}
```

Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[list[str]], question_tags: list[
list[str]], priorities: list[int]) -> list[int]:
```

Sample call

```
solution([['python', 'sql'], ['java'], ['ml']], [['sql'],
['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])
```

Test cases

Input	Expected
`([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])`	`(3, 16)`
`([['a'], ['a']], [['a'], ['a'], ['b']], [1, 10, 100])`	`(2, 11)`

MEDIUM | PracHub | Technical Screen | 47 of 196

★ TOP 75: rank 27 in the Medium and Hard priority list

★ Solve Grid and Counting Problems

STUDY: TYPE | Google frequency on LeetCode: all time 64.8, last 30 days 33 | book priority 17 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #200 Number of Islands

LeetCode match: LC 200 Number of Islands (+ a counting task)

Original: https://prachub.com/coding-questions/solve-grid-and-counting-problems

QUESTION

A software engineering intern phone screen included two coding tasks: 1.

Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.

Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.
Constraints
0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length
Examples
Input: []
Expected Output:
Explanation: Empty grid contains no land, so there are no islands.
Input: [['1']]
Expected Output:
Explanation: A single land cell forms one island.
Hints
Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.
Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.
Constraints
0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type
Examples
Input: []
Expected Output:
Explanation: No letters means it is impossible to form GOOGLE.
Input: ['G','O','O','G','L','E']
Expected Output:
Explanation: There is exactly one way to choose all six letters.
Hints
First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1

L and 1 E.

APPROACH

Islands: DFS/BFS or union-find over '1' cells. Second counting task: use prefix-sum + hash map or math counting per the statement.

INTERVIEW-DAY PLAN

Clarify first

Two tasks. Islands: four-directional adjacency, grid up to 300 x 300, empty grid allowed. Counting: choose letters from an array so the chosen ones can be rearranged to spell GOOGLE, each index used at most once, different indices are different choices.

Brute force to say out loud

Islands: none needed, DFS is the standard. Counting: enumerate all 6-element subsets of the array and test each one. O(n^6).

Optimal idea

Islands: scan cells, start a DFS or BFS (iterative, to avoid recursion limits) from each unvisited land cell, count the starts. O(mn). Counting: GOOGLE needs G twice, O twice, L once, E once. The number of ways is C(g, 2) * C(o, 2) * l * e, where g, o, l, e are the counts of those letters in the array, since the chosen multiset is fixed and only the indices vary. O(n) to count.

Dry-run these cases

Islands: empty grid, a single land cell, diagonal-only neighbors (separate islands). Counting: missing a letter (0), exactly one G (0), lots of copies, an empty array.

Likely follow-ups

Islands: add land one cell at a time with Union-Find. Counting: a different target word, or count ways when letters have to appear in order (subsequence DP).

Pitfalls

Treating diagonal cells as connected. Counting arrangements instead of choices of indices.

HOW TO THINK ABOUT IT

Islands: every unvisited land cell starts a new island; mark everything connected to it. Counting: the target multiset is fixed (G twice, O twice, L once, E once), so the number of ways to choose the indices is a product of binomial coefficients, one factor per distinct letter. Do not enumerate arrangements.

Basis: Two tasks: LeetCode 200 (Number of Islands) and counting the ways to choose letters from an array that can be rearranged into GOOGLE (each index used at most once).

BRUTE FORCE - PSEUDO-CODE: ISLANDS: RECURSIVE FLOOD FILL

```
count = 0; for every cell: if it is unvisited land: count
= count + 1 and recursively
    mark all land connected to it
```

Time: O(m * n) | Space: O(m * n) recursion

Why it is slow

Time is O(m * n), but the recursion can be m * n deep on a snake-shaped island and overflow the stack. A queue or an explicit stack avoids that.

BRUTE FORCE - PSEUDO-CODE: COUNTING: ENUMERATE EVERY 6-ELEMENT SELECTION

```
for every set of 6 indices of the array: if their letters
are G, G, O, O, L, E in some
    order: count = count + 1
```

Time: O(C(n, 6) * 6) | Space: O(1)

Why it is slow

C(n, 6) selections. The letters needed are fixed, so the answer is C(#G, 2) * C(#O, 2) * #L * #E from one pass of counts.

OPTIMAL SOLUTION - ISLANDS: ITERATIVE BFS

```
def numIslands(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                seen.add((r, c))
                queue = deque([(r, c)])
                while queue:
                    x, y = queue.popleft()
                    for nx, ny in ((x + 1, y), (x - 1, y),
 (x, y + 1), (x, y - 1)):
                        if (
                            0 <= nx < m
                            and 0 <= ny < n
                            and grid[nx][ny] == "1"
                            and (nx, ny) not in seen
                        ):
                            seen.add((nx, ny))
                            queue.append((nx, ny))
    return count
```

Time: O(m * n) | Space: O(min(m, n)) queue

Why

Same scan, but spread with a queue instead of recursion, so the depth of an island cannot overflow the stack. Marking visited cells in place (or with a set) keeps each cell processed once.

OPTIMAL SOLUTION - COUNTING: PRODUCT OF BINOMIAL COEFFICIENTS

```
def google_ways(letters):
    c = Counter(letters)
    return math.comb(c["G"], 2) * math.comb(c["O"], 2) * c
["L"] * c["E"]
```

Time: O(n) | Space: O(1)

Why

Let g, o, l, e be the number of G, O, L, E in the array. The chosen letters must be exactly two G's, two O's, one L and one E, so the number of ways is C(g, 2) * C(o, 2) * l * e (indices make each choice distinct). One pass counts the letters.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

```
def ways_to_form_word(letters, word):
    need, have = Counter(word), Counter(letters)
    result = 1
    for ch, k in need.items():
        result *= math.comb(have[ch], k)
    return result
```

The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[str]]) -> int:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`0`
`[['1']]`	`1`

MEDIUM | IN APP LC 261 | PracHub | Technical Screen | 48 of 196

Validate parent array forms a tree

STUDY: TALK | Google frequency on LeetCode: all time 29, last 30 days 0 | book priority 116 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #261 Graph Valid Tree

LeetCode match: LC 261 Graph Valid Tree

Original: https://prachub.com/coding-questions/validate-parent-array-forms-a-tree

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2. **No cycles** (following parent pointers from any node must eventually reach the root). 3. **Connectivity** (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints

1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

Examples

Input: [-1, 0, 0, 1, 1]

Expected Output: True

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

Input: [2, -1, 1, 2]

Expected Output: True

Explanation: Node 1 is the root. The paths are 0->2->1, 2->1, and 3->2->1, so every node reaches the same root.

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

APPROACH

Exactly one root (parent = -1 or self), every other node has one parent; detect cycles by walking up with a visited set or union-find; all nodes must reach the root. O(n).

HOW TO THINK ABOUT IT

Exactly one -1 gives the root; then the rest is whether every node reaches it. Following parents from each node can loop (a cycle) or run out of range. Do it with memoised walks so every node is processed once: mark nodes on the current walk, and if the walk meets one of them again it is a cycle; if it meets a node already known to reach the root, all nodes on the walk reach it too.

Basis: LeetCode 261 (Graph Valid Tree) in parent-pointer form: exactly one root (-1), no cycles, every node reaches the root.

BRUTE FORCE - PSEUDO-CODE

```
check that exactly one entry is -1 and all others are in r
ange
for every node: follow parents for at most n steps; if the
 root (-1) is not reached:
    there is a cycle
```

Time: O(n^2) | Space: O(1)

Why it is slow

Every node walks its whole chain, O(n^2). Marking nodes as 'on this walk' or 'verified' lets every node be processed once, O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(parent):
    n = len(parent)
    if n == 0:
        return False

    root = -1
    root_count = 0
    children = [[] for _ in range(n)]

    for i, p in enumerate(parent):
        if p == -1:
            root_count += 1
            root = i
        else:
            if p < 0 or p >= n:
                return False
            children[p].append(i)

    if root_count != 1:
        return False

    visited = [False] * n
    stack = [root]
    visited[root] = True
    seen = 0

    while stack:
        node = stack.pop()
        seen += 1
        for child in children[node]:
            if visited[child]:
                return False
            visited[child] = True
            stack.append(child)

    return seen == n
```

Time: O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The graph is given as undirected edges (LeetCode 261).

A tree has exactly n - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(n alpha(n)).

```python
def valid_tree_by_edges(n, edges):
    """Undirected form (LeetCode 261): n - 1 edges and con
nected."""
    if len(edges) != n - 1:
        return False
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for u, v in edges:
        ru, rv = find(u), find(v)
        if ru == rv:
            return False
        parent[ru] = rv
    return True
```

Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

FUNCTION AND TEST CASES

Signature

```
def solution(parent: list[int]) -> bool:
SAMPLE INPUT
[-1, 0, 0, 1, 1]
```

Test cases

Input	Expected
[-1, 0, 0, 1, 1]	True
[2, -1, 1, 2]	True

MEDIUM | IN APP LC 127 | PracHub | Technical Screen | 49 of 196

Find shortest relationship path using BFS

STUDY: TALK | Google frequency on LeetCode: all time 41.8, last 30 days 0 | book priority 78 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #127 Word Ladder

LeetCode match: LC 127 Word Ladder (BFS shortest path) (inferred from overview)

Original: https://prachub.com/coding-questions/find-shortest-relationship-path-using-bfs

QUESTION

Overview: This question evaluates proficiency in graph representations and traversal algorithms—specifically BFS and adjacency-structure construction—for computing unweighted shortest paths and reasoning about reachability.

APPROACH

Build an adjacency list, BFS from the source with a parent map, then reconstruct the path; O(V+E). Consider bidirectional BFS.

HOW TO THINK ABOUT IT

Unweighted means every hop costs the same, so breadth-first search finds the fewest hops: it explores all people one hop away, then two hops, and the first time it reaches the target is the shortest path. Keep a parent map to rebuild the path and a visited set so nobody is queued twice.

Basis: LeetCode 127 (Word Ladder) idea: shortest path in an unweighted graph of people. Return the list of people from source to target, or None.

BRUTE FORCE - PSEUDO-CODE

```
depth-first search over every simple path from the source
keep the shortest path that reaches the target
```

Time: exponential | Space: O(path)

Why it is slow

The number of simple paths is exponential. Breadth-first search reaches the target along a shortest path first, O(V + E), and bidirectional BFS explores far fewer nodes on big graphs.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
from collections import defaultdict, deque


def solution(relationships, start, target):
    if start == target:
        return 0

    graph = defaultdict(list)
    for u, v in relationships:
        graph[u].append(v)
        graph[v].append(u)

    queue = deque([(start, 0)])
    visited = {start}

    while queue:
        node, dist = queue.popleft()
        if node == target:
            return dist

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, dist + 1))

    return -1
```

Time: O(V + E) | Space: O(V + E)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

```
def shortest_relationship_bidirectional(adj, s, t):
    if s == t:
        return [s]
    fwd, bwd = {s: None}, {t: None}
    qf, qb = deque([s]), deque([t])
    meet = None
    while qf and qb and meet is None:
        if len(qf) <= len(qb):
            for _ in range(len(qf)):
                u = qf.popleft()
                for v in adj.get(u, ()):
                    if v not in fwd:
                        fwd[v] = u
                        qf.append(v)
                        if v in bwd:
                            meet = v
                            break
                if meet is not None:
                    break
        else:
            for _ in range(len(qb)):
                u = qb.popleft()
                for v in adj.get(u, ()):
                    if v not in bwd:
                        bwd[v] = u
                        qb.append(v)
                        if v in fwd:
                            meet = v
                            break
                if meet is not None:
                    break
    if meet is None:
        return None
    left, x = [], meet
    while x is not None:
        left.append(x)
        x = fwd[x]
    right, x = [], bwd[meet]
    while x is not None:
        right.append(x)
        x = bwd[x]
    return left[::-1] + right
```

Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

FUNCTION AND TEST CASES

Signature

```
def solution(relationships: list[list[int]], start: int, target: int) -> int:
```

Sample call

```
solution([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')
```

Test cases

Input	Expected
`[['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')`	`3`
`[[1, 2], [2, 5], [1, 3], [3, 4], [4, 5]], 1, 5)`	`2`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Degrees of Friendship | Exponent

https://www.aced.io/questions/3628/degrees-of-friendship

MEDIUM | PracHub | Technical Screen | 50 of 196

★ TOP 75: rank 65 in the Medium and Hard priority list

★ Compute shortest paths with blocked nodes

STUDY: TYPE | Google frequency on LeetCode: all time 37.9, last 30 days 33 | book priority 53 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #743 Network Delay Time

LeetCode match: LC 743 Network Delay Time (Dijkstra with blocked nodes skipped)

Original: https://prachub.com/coding-questions/compute-shortest-paths-with-blocked-nodes

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

Examples

Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3], False)

Expected Output: [0, 1, 2, -1, -1]

Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)

Expected Output: [0, 1, 2, 3]

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

APPROACH

Run Dijkstra (BFS if unweighted) from s, never relaxing into blocked nodes; blocked/unreachable nodes stay at infinity. O((V+E) log V).

HOW TO THINK ABOUT IT

Blocked nodes are simply not in the graph, so skip them when you queue neighbours and never start from one. After that it is plain BFS from the source: first discovery gives the shortest distance. State the edge cases: a blocked source makes everything unreachable, and directed edges must only be added in their given direction.

Basis: Unweighted shortest distances (BFS) with some nodes removed from the graph. Output: a distance per node, -1 for unreachable or blocked nodes; directed=True uses edges one way only.

BRUTE FORCE - PSEUDO-CODE

```
remove the blocked nodes and their edges
d[i][j] = the shortest distance for every pair (triple loop); read the row of the
    source
```

Time: O(V^3) | Space: O(V^2)

Why it is slow

All-pairs distances cost O(V^3) and O(V^2) memory when one source is needed. A BFS from the source that never enters a blocked node is O(V + E).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
import collections


def solution(n, edges, source, blocked, directed=False):
    blocked = set(blocked)
    distances = [-1] * n
    if source in blocked or not (0 <= source < n):
        return distances
    graph = [[] for _ in range(n)]
    for u, v in edges:
        if 0 <= u < n and 0 <= v < n:
            graph[u].append(v)
            if not directed:
                graph[v].append(u)
    q = collections.deque([source])
    distances[source] = 0
    while q:
        node = q.popleft()
        for nei in graph[node]:
            if nei in blocked or distances[nei] != -1:
                continue
            distances[nei] = distances[node] + 1
            q.append(nei)
    for node in blocked:
        if 0 <= node < n:
            distances[node] = -1
    return distances
```

Time: O(n + e) | Space: O(n + e)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

```
def blocked_weighted(n, edges, source, blocked, directed=False):
    bl = set(blocked)
    if source in bl:
        return [-1] * n
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        if not directed:
            adj[v].append((u, w))
    dist = {source: 0}
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in bl and d + w < dist.get(v, float("inf")):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return [dist.get(i, -1) for i in range(n)]
```

Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

MEDIUM | IN APP LC 501 | PracHub | Technical Screen | 51 of 196

Find mode in a trinary search tree

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 125 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #501 Find Mode in Binary Search Tree

LeetCode match: LC 501 Find Mode in Binary Search Tree

Original: https://prachub.com/coding-questions/find-mode-in-a-trinary-search-tree

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all values > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []
Explanation: The tree is empty, so there are no modes.
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
Explanation: A single-node tree has that node's value as the only mode.
Hints
In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.
Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n -

1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []
Explanation: The tree is empty, so there are no modes.
Input: ([-2147483648], [-1], [-1], [-1], 0)
Expected Output: [-2147483648]
Explanation: A single-node tree has that node's value as the only mode.
Hints
Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

APPROACH

In-order traversal (left, middle, right) so equal values are consecutive; track current run length and the max; O(h) extra space.

HOW TO THINK ABOUT IT

In a valid search tree an in-order walk (left, middle, right) visits values in sorted order, so equal values are adjacent and you can count runs with a single running counter and O(height) memory instead of a dictionary. If the tree may be broken, the order guarantee is gone and you need a real frequency table.

Basis: LeetCode 501, Find Mode in Binary Search Tree, with a middle child for equal values. Valid case: left < val, middle = val, right > val.

BRUTE FORCE - PSEUDO-CODE

```
traverse the whole tree, counting how many times each valu
e occurs in a Counter
return the values with the highest count
```

Time: O(n) | Space: O(n)

Why it is slow

O(n) time but memory for every distinct value. In a valid tree an in-order walk (left, middle, right) gives equal values consecutively, so a running counter needs only O(height) space.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(values, left, middle, right, root):
    if root == -1:
        return []

    # Iterative inorder traversal for a trinary search tre
e:
    # left subtree, node itself, middle subtree, right sub
tree.
    stack = [(root, 0)]

    have_prev = False
    prev_value = None
    current_count = 0
    max_count = 0
    modes = []

    while stack:
        node, state = stack.pop()
        if node == -1:
            continue
        if state == 0:
            # Push in reverse order because stack is LIFO.
            stack.append((right[node], 0))
            stack.append((middle[node], 0))
            stack.append((node, 1))
            stack.append((left[node], 0))
        else:
            value = values[node]

            if not have_prev or value != prev_value:
                prev_value = value
                have_prev = True
                current_count = 1
            else:
                current_count += 1

            if current_count > max_count:
                max_count = current_count
                modes = [value]
            elif current_count == max_count:
                modes.append(value)

    return modes
```

Time: O(n), where n is the number of reachable nodes | Space: O(h + m), where h is the tree height used by the traversal stack and m is the number of modes returned. In the worst case, h can be O(n).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

```python
class TNode:
    def __init__(self, val, left=None, middle=None, right=
None):
        self.val, self.left, self.middle, self.right = val
, left, middle, right


def modes_counter(root):
    c, stack = Counter(), [root] if root else []
    while stack:
        x = stack.pop()
        c[x.val] += 1
        stack.extend(ch for ch in (x.left, x.middle, x.rig
ht) if ch)
    if not c:
        return []
    top = max(c.values())
    return sorted(v for v, k in c.items() if k == top)
```

Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. Cost: O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

FUNCTION AND TEST CASES

Signature

```
def solution(values: list[int], left: list[int], middle: l
ist[int], right: list[int], root: int) -> list[int]:
```

Sample call

```
solution([], [], [], [], -1)
```

Test cases

Input	Expected
([], [], [], [], -1)	[]
([42], [-1], [-1], [-1], 0)	[42]

MEDIUM | PracHub | Technical Screen | 52 of 196

Determine whether two nodes are related

STUDY: TALK | Google frequency on LeetCode: all time 24.5, last 30 days 0 | book priority 130 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

LeetCode match: LC 1971 Find if Path Exists in Graph (inferred from overview)

Original: https://prachub.com/coding-questions/determine-whether-two-nodes-are-related

QUESTION

Overview: This question evaluates understanding of graph representations, reachability and connectivity concepts for ancestry relationships, including handling disconnected components and potential cycles.

APPROACH

Build the ancestry graph and BFS/DFS (or union-find for undirected relatedness); handle disconnected components.

INTERVIEW-DAY PLAN

Clarify first

Ask what related means: one is an ancestor of the other, they share a common ancestor, or they are connected in any direction. Ask whether the graph is a tree, a DAG, or may have cycles, and whether it is disconnected.

Brute force to say out loud

DFS from the first node looking for the second, and again from the second. O(V + E) per query.

Optimal idea

Build the adjacency list once. For ancestry in a DAG compute the ancestor set of each node (DFS or BFS) and test for the target or a non-empty intersection. For undirected relatedness use union-find, which answers queries in near constant time after one pass. Track visited to survive cycles.

Dry-run these cases

A node related to itself, two nodes in separate components, a long chain, a cycle.

Likely follow-ups

Many queries (precompute components or ancestors), the lowest common ancestor, or updates to the graph.

Pitfalls

Missing a visited set. Treating direction as undirected, or the reverse.

HOW TO THINK ABOUT IT

Ask what 'related' means: connected in any direction (a connectivity question: union-find or one traversal), or ancestor/descendant, or share an ancestor. With many queries, preprocess once: components give O(1) answers. Always make the visited set part of the design, since relationship data can have cycles.

Basis: LeetCode 1971, Find if Path Exists in Graph, for connectivity, and ancestor queries on a parent map for the 'ancestry' reading. Assumed meaning (confirm it): related = connected by any path of relationships.

BRUTE FORCE - PSEUDO-CODE

```
build adjacency lists
for every query (a, b): run a DFS from a; answer whether b
 was reached
```

Time: O(V + E) per query | Space: O(V)

Why it is slow

Every query repeats the same traversal, O(V + E) each. A Union-Find built once answers each query in near-constant time.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(relations, a, b):
    if a == b:
        return True
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        root = x
        while parent[root] != root:
            root = parent[root]
        while parent[x] != root:
            parent[x], x = root, parent[x]
        return root

    def union(x, y):
        parent.setdefault(x, x)
        parent.setdefault(y, y)
        rx, ry = find(x), find(y)
        if rx != ry:
            parent[rx] = ry

    for p, c in relations:
        union(p, c)

    if a not in parent or b not in parent:
        return False
    return find(a) == find(b)
```

Time: O((E + Q) * alpha(N)) where E = number of relations, Q = number of queries (1 here), N = distinct chickens, and alpha is the inverse Ackermann function from union-find with path compression — effectively O(E). | Space: O(N) for the disjoint-set parent map over the distinct chicken IDs.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

'Related' means ancestor or descendant, or sharing an ancestor.
Store the data as a parent map and walk up from one node while recording ancestors, then walk up from the other until you hit one. Keep a seen set so a corrupt cycle cannot loop forever.

```
def share_ancestor(parent, a, b):
    chain, seen = set(), set()
    while a is not None and a not in seen:
        chain.add(a)
        seen.add(a)
        a = parent.get(a)
    seen = set()
    while b is not None and b not in seen:
        if b in chain:
            return True
        seen.add(b)
        b = parent.get(b)
    return False
```

Relationships arrive over time and you must answer queries between updates.
Union-Find handles incremental unions in near-constant time. If relationships can also be removed, you need offline processing or a dynamic-connectivity structure.

The graph is directed and 'related' means reachability from a to b.
Union-Find no longer works (it ignores direction). Use DFS/BFS per query, or compute strongly connected components and reachability on the condensed DAG for many queries.

FUNCTION AND TEST CASES

Signature

```
def solution(relations: list[list[int]], a: int, b: int) -
> bool:
```

Sample call

```
solution([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)
```

Test cases

Input	Expected
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)`	`True`
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 11)`	`False`

MEDIUM | IN APP LC 542 | PracHub | Technical Screen | 53 of 196

Compute distance to nearest taxi in grid

STUDY: TALK | Google frequency on LeetCode: all time 36.5, last 30 days 0 | book priority 92 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: the interview site's own solution
Also in your app (open the app for its Expose): LeetCode #542 01 Matrix
LeetCode match: LC 542 01 Matrix (inferred from overview)
Original: https://prachub.com/coding-questions/compute-distance-to-nearest-taxi-in-grid

QUESTION

Overview: This question evaluates understanding of graph traversal and shortest-path reasoning on grids, algorithmic problem-solving for computing distances from many targets, and the ability to manage time and space complexity on large inputs.

APPROACH

Multi-source BFS from all taxis simultaneously; the BFS level is the distance for each cell. O(mn).

HOW TO THINK ABOUT IT

Asking for the distance from every cell to its nearest source is one BFS started from all sources at once, not one BFS per cell. Put all taxis in the queue at distance 0; the first time the search reaches a cell is its shortest distance, because BFS visits cells in order of distance.

Basis: LeetCode 542, 01 Matrix, with 'T' for a taxi, '#' for a blocked cell and '.' for open ground. Distance counts moves through open cells (4-directional).

BRUTE FORCE - PSEUDO-CODE

```
for every open cell: run a BFS outward until the first tax
i is found; the BFS depth is
    the distance
```

Time: O((m * n)^2) | Space: O(m * n)

Why it is slow

A separate BFS per cell, O((m * n)^2). One BFS that starts from all taxis at once reaches every cell at its nearest-taxi distance, O(m * n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(grid):
    from collections import deque

    if not grid:
        return []

    rows = len(grid)
    cols = len(grid[0]) if rows > 0 else 0
    if cols == 0:
        return [[] for _ in range(rows)]

    dist = [[-1] * cols for _ in range(rows)]
    q = deque()

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
```

```
                dist[r][c] = 0
                q.append((r, c))

    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    while q:
        r, c = q.popleft()
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and dist[
nr][nc] == -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))

    return dist
```

Time: O(R*C) | Space: O(R*C)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

```
class TaxiMap:
    """A taxi appears: only cells that become closer chang
e, so relax outward from the new taxi."""

    def __init__(self, grid):
        self.grid, self.dist = [list(r) for r in grid], ta
xi_distances(grid)

    def add_taxi(self, r, c):
        self.grid[r][c] = "T"
        self.dist[r][c] = 0
        queue = deque([(r, c)])
        while queue:
            x, y = queue.popleft()
            for nx, ny in ((x + 1, y), (x - 1, y), (x, y +
 1), (x, y - 1)):
                if (
                    0 <= nx < len(self.grid)
                    and 0 <= ny < len(self.grid[0])
                    and self.grid[nx][ny] != "#"
                ):
                    if (
                        self.dist[nx][ny] == -1
                        or self.dist[nx][ny] > self.dist[x
][y] + 1
                    ):
                        self.dist[nx][ny] = self.dist[x][y
] + 1
                        queue.append((nx, ny))
```

Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0,0,0],[0,1,0],[0,0,0]])
```

Test cases

Input	Expected
`([[0,0,0],[0,1,0],[0,0,0]],)`	`[[2,1,2],[1,0,1],[2,1,2]]`
`([[1,0,0,0],[0,0,0,1],[0,0,0,0]],)`	`[[0,1,2,1],[1,2,1,0],[2,3,2,1]]`

MEDIUM | PracHub | Technical Screen | 54 of 196

★ TOP 75: rank 46 in the Medium and Hard priority list

★ Solve tree, graph, sliding-window problems

STUDY: TYPE | Google frequency on LeetCode: all time 51.9, last 30 days 33 | book priority 26 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #103 Binary Tree Zigzag Level Order Traversal, LeetCode #207 Course Schedule

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal + LC 207 Course Schedule

Original: https://prachub.com/coding-questions/solve-tree-graph-sliding-window-problems

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters. https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/ https://leetcode.com/problems/course-schedule/description/ https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form
Examples
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.
Input: ([1],)
Expected Output: [[1]]
Explanation: A single-node tree has only one level.
Hints
A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.
Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.
Constraints
1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses
Examples
Input: (2, [[1, 0]])
Expected Output: True
Explanation: Course 0 can be taken first, then course 1.
Input: (2, [[1, 0], [0, 1]])
Expected Output: False
Explanation: The two courses depend on each other, forming a cycle.
Hints
Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.
Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.
Constraints
0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters
Examples
Input: ("eceba", 2)
Expected Output: 3
Explanation: The longest valid substring is 'ece', which has length 3.
Input: ("aa", 1)
Expected Output: 2
Explanation: The whole string contains only one distinct character.
Hints
A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

APPROACH

BFS by level with reversed alternate levels; topological sort / DFS cycle detection; plus a sliding-window task.

INTERVIEW-DAY PLAN

Clarify first

Three LeetCode problems: 103 zigzag level order, 207 course schedule, 340 longest substring with at most K distinct characters. For each ask the usual edge questions: empty tree, duplicate prerequisites, K = 0.

Brute force to say out loud

Zigzag: do a normal level order and reverse alternate levels afterwards. Course schedule: DFS from every course to find cycles. Substring: check every substring.

Optimal idea

Zigzag: BFS level by level, reverse (or use a deque appended on the left or right) on every other level. O(n). Course schedule: Kahn's algorithm, with in-degree counts and a queue; if processed courses equal the course count there is no cycle. O(V + E). Substring (340): sliding window with a dict of counts; expand right, and while distinct characters exceed K shrink from the left, deleting a character at count 0. Track the maximum window. O(n).

Dry-run these cases

Zigzag: empty tree, a single node, a skewed tree. Course schedule: no prerequisites, a self loop, a two-course cycle. Substring: K = 0 (answer 0), K larger than distinct characters, a repeated character.

Likely follow-ups

Course Schedule II (return an order), longest substring with exactly K distinct, at most K distinct in a stream.

Pitfalls

Reversing the wrong levels. Not removing a key from the window dict when its count hits zero.

HOW TO THINK ABOUT IT

Three standard patterns. Level order needs a queue and a direction flag per level. 'Can everything be finished' is cycle detection in a directed graph: Kahn's algorithm removes nodes with no remaining prerequisites, and a cycle leaves some behind. 'Longest window with at most K of something' is a sliding window with a count map that shrinks from the left when the limit is exceeded.

Basis: Three LeetCode problems: 103 Binary Tree Zigzag Level Order, 207 Course Schedule and 340 Longest Substring with At Most K Distinct Characters.

BRUTE FORCE - PSEUDO-CODE: COURSE SCHEDULE: DFS FROM EVERY COURSE LOOKING FOR A CYCLE

```
for every course: follow prerequisite edges looking for a
path that returns to it; if
    found: there is a cycle
```

Time: O(V * (V + E)) | Space: O(V)

Why it is slow

Each search can walk the whole graph again, O(V * (V + E)). Kahn's algorithm (remove courses with no unmet prerequisites) is O(V + E).

BRUTE FORCE - PSEUDO-CODE: LONGEST WINDOW: CHECK EVERY SUBSTRING

```
for every start i: extend the end while the substring has
at most K distinct
    characters; record the longest
```

Time: O(n^2) | Space: O(k)

Why it is slow

O(n^2) in the worst case. A sliding window with a count map that shrinks from the left when it exceeds K distinct characters is O(n).

BRUTE FORCE - PSEUDO-CODE: ZIGZAG: LEVEL ORDER, THEN REVERSE THE ODD LEVELS

```
levels = a normal level-order traversal
reverse every level with odd index
```

Time: O(n) | Space: O(n)

Why it is slow

Same O(n) time with one extra copy of every second level; filling each level from alternating ends of a deque avoids it.

OPTIMAL SOLUTION - COURSE SCHEDULE: KAHN'S TOPOLOGICAL SORT

```
def canFinish(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    queue = deque(c for c in range(n) if indeg[c] == 0)
    done = 0
    while queue:
        c = queue.popleft()
        done += 1
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return done == n
```

Time: O(V + E) | Space: O(V + E)

Why

Count prerequisites per course and start from courses with none. Remove a course, decrease the count of the courses that depend on it, and queue those that reach zero. All courses can be taken exactly when every course is removed; a leftover means a cycle.

OPTIMAL SOLUTION - LONGEST WINDOW: SLIDING WINDOW WITH A COUNT MAP

```
def lengthOfLongestSubstringKDistinct(s, k):
    count, left, best = {}, 0, 0
    for right, ch in enumerate(s):
        count[ch] = count.get(ch, 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

Time: O(n) | Space: O(k)

Why

Add s[right] to the counts. While more than K distinct characters are present, remove s[left] (delete the key when its count hits 0) and advance left. The window is always valid after the shrink, so record its length. Each index enters and leaves once.

OPTIMAL SOLUTION - ZIGZAG: FILL EACH LEVEL FROM THE CORRECT END

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def zigzagLevelOrder(root):
    out, q, ltr = [], deque([root] if root else []), True
    while q:
        level = deque()
        for _ in range(len(q)):
            x = q.popleft()
            level.append(x.val) if ltr else level.appendle
ft(x.val)
            q.extend(c for c in (x.left, x.right) if c)
        out.append(list(level))
        ltr = not ltr
    return out
```

Time: O(n) | Space: O(width)

Why

Keep a direction flag; for each level build a deque, appending on the right when going left to right and on the left otherwise. Flip the flag after each level; no reversal is needed.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

```
def find_order(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    queue, order = deque(c for c in range(n) if indeg[c] =
= 0), []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []
```

Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

FUNCTION AND TEST CASES

Signature

```
def solution(root: list[str]) -> list[list[int]]:
```

Sample call

```
solution([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

MEDIUM | IN APP LC 207 | PracHub | Technical Screen | 55 of 196

★ TOP 75: rank 33 in the Medium and Hard priority list

★ Validate course catalog dependencies

STUDY: TYPE | Google frequency on LeetCode: all time 51.9, last 30 days 33 | book priority 27 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #207 Course Schedule

LeetCode match: LC 207 Course Schedule

Original: https://prachub.com/coding-questions/validate-course-catalog-dependencies

QUESTION

Design a function to validate an e-learning course catalog. You are given: (1) a set of course IDs, and (2) a list of prerequisite pairs (u, v) meaning course v requires course u. Determine if the catalog is valid: no prerequisite references a non-existent course, and there are no cycles in the dependency graph. If valid, return true and a possible ordering of courses; otherwise return false and report the set of missing course references and at least one cycle you detect. Specify your algorithm, time and space complexity.

APPROACH

Check every prerequisite refers to a known course ID, then Kahn's topological sort or DFS coloring to detect cycles.

HOW TO THINK ABOUT IT

Two separate validity rules: every course mentioned must exist, and the dependency graph must have no cycle. The first is a set lookup per pair. For the second, Kahn's algorithm removes courses with no unmet prerequisites; anything left over is on a cycle or depends on one, and a DFS restricted to the leftovers reveals an actual cycle to report.

Basis: LeetCode 207 / 210 (Course Schedule) plus a check for references to unknown courses. A pair (u, v) means course v requires u.

BRUTE FORCE - PSEUDO-CODE

```
report any prerequisite that mentions a course not in the
catalog
for every course: search along prerequisite edges for a pa
th back to it; if found:
    cycle
```

Time: O(V * (V + E)) | Space: O(V)

Why it is slow

Every course can trigger a walk of the whole graph, O(V * (V + E)). Kahn's algorithm is O(V + E) and its leftovers also reveal a cycle to report.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(course_ids, prereqs):
    courses = set(course_ids)
    # Find missing references: any course in a prereq pair
 not in the catalog
    missing = set()
    for u, v in prereqs:
        if u not in courses:
            missing.add(u)
        if v not in courses:
            missing.add(v)
    # Build graph only over known courses (edge u -> v mea
ns v requires u)
    adj = {c: [] for c in courses}
    indeg = {c: 0 for c in courses}
    for u, v in prereqs:
        if u in courses and v in courses:
            adj[u].append(v)
            indeg[v] += 1
    # Kahn's algorithm for topological ordering (process n
odes in sorted order for determinism)
    from collections import deque

    queue = deque(sorted([c for c in courses if indeg[c] =
= 0]))
    order = []
    local_indeg = dict(indeg)
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in sorted(adj[node]):
            local_indeg[nxt] -= 1
            if local_indeg[nxt] == 0:
                queue.append(nxt)
        # Re-sort queue to keep deterministic lexicographi
c order
        queue = deque(sorted(queue))
    if len(order) == len(courses):
        valid = len(missing) == 0
        if valid:
            return (True, order, [], [])
        # No cycle but there are missing references -> inv
alid
        return (False, order, sorted(missing), [])
    # A cycle exists. Detect one cycle via DFS over remain
ing nodes.
    remaining = set(c for c in courses if local_indeg[c] >
 0)
    cycle = []
    color = {c: 0 for c in courses}  # 0=white,1=gray,2=bl
ack
    parent = {}
    found = [False]

    def dfs(start):
        stack = [(start, iter(sorted(adj[start])))]
```

```
        color[start] = 1
        path = [start]
        in_path = {start}
        while stack:
            node, it = stack[-1]
            advanced = False
            for nxt in it:
                if nxt not in remaining:
                    continue
                if color[nxt] == 1 and nxt in in_path:
                    # Found a cycle; extract from path
                    idx = path.index(nxt)
                    cyc = path[idx:] + [nxt]
                    cycle.extend(cyc)
                    found[0] = True
                    return
                if color[nxt] == 0:
                    color[nxt] = 1
                    stack.append((nxt, iter(sorted(adj[nxt
])))
                    path.append(nxt)
                    in_path.add(nxt)
                    advanced = True
                    break
            if not advanced:
```

```
                color[node] = 2
                stack.pop()
                path.pop()
                in_path.discard(node)
        return

    for c in sorted(remaining):
        if color[c] == 0:
            dfs(c)
            if found[0]:
                break
    return (False, [], sorted(missing), cycle)
```

Time: O((V + E) log V), where V is the number of courses and E the number of prerequisite pairs. Building the graph is O(V + E); Kahn's topological sort visits every node and edge once but re-sorts the ready queue for deterministic tie-breaking, adding the log V factor; the DFS cycle search is O(V + E). If deterministic tie-breaking is not required, this reduces to O(V + E). | Space: O(V + E) for the adjacency list, in-degree map, BFS/DFS queue and stack, and the output ordering/cycle lists.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

```
def validate_catalog(courses, prereqs):
    cs = set(courses)
    missing = sorted({x for u, v in prereqs for x in (u, v
) if x not in cs}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in cs}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
```

```
    queue = deque(sorted((c for c in cs if indeg[c] == 0),
 key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(cs):
        return True, [], order
    left = {c for c in cs if indeg[c] > 0}
    pred = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            pred[v].append(u)
    start = min(left, key=str)
    walk, pos, x = [], {}, start
    while (
        x not in pos
    ):  # every leftover course has a leftover prerequisit
e, so this walk never stops early
        pos[x] = len(walk)
        walk.append(x)
        x = pred[x][0]
    cycle = walk[pos[x] :] + [x]
    return (
        False,
        [],
        cycle[::-1],
    )  # walked against the edges, so reverse to list prer
equisites first
```

Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

FUNCTION AND TEST CASES

Signature

```
def solution(course_ids: list[str], prereqs: list[tuple[st
r, str]]) -> tuple[bool, list[str], list[str], list[str]]:
```

Sample call

```
solution(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])
```

Test cases

Input	Expected
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])`	`(True, ['A', 'B', 'C'], [], [])`
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C'), ('C', 'A')])`	`(False, [], [], ['A', 'B', 'C', 'A'])`

MEDIUM | IN APP LC 103 | PracHub | Technical Screen | 56 of 196

Implement zigzag level-order traversal

STUDY: TALK | Google frequency on LeetCode: all time 38.7, last 30 days 0 | book priority 86 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #103 Binary Tree Zigzag Level Order Traversal

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal

Original: https://prachub.com/coding-questions/implement-zigzag-level-order-traversal

QUESTION

Given the root of a binary tree, return the node values in zigzag level order: visit nodes level by level, reading the first level from left to right, the next from right to left, and so on alternating each level. Output a list of lists of integers where each inner list contains the values for one level in the required direction. Aim for O(n) time and O(w) extra space, where n is the number of nodes and w is the maximum width of the tree.

APPROACH

BFS by level; reverse (or use a deque appending left/right) on alternate levels. O(n).

HOW TO THINK ABOUT IT

It is a plain level-order traversal where every second level is read in the other direction. Either collect each level and reverse the odd ones, or fill each level from the correct end of a deque so no reversal is needed.

Basis: LeetCode 103, Binary Tree Zigzag Level Order Traversal.

BRUTE FORCE - PSEUDO-CODE

```
levels = a normal level-order traversal
reverse every level with odd index
```

Time: O(n) | Space: O(n)

Why it is slow

Same O(n) time, plus an extra pass copying the reversed levels. Filling each level from the left or right end of a deque as it is built needs no reversal.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(values):
    if not values:
        return []
    n = len(values)
    root_val = values[0]
    if root_val is None:
        return []

    class Node:
        __slots__ = ("val", "left", "right")

        def __init__(self, v):
            self.val = v
            self.left = None
            self.right = None

    root = Node(root_val)
    from collections import deque
    q = deque([root])
    i = 1
    while q and i < n:
        node = q.popleft()
        if i < n:
            lv = values[i]
            i += 1
            if lv is not None:
                node.left = Node(lv)
                q.append(node.left)
        if i < n:
            rv = values[i]
            i += 1
            if rv is not None:
                node.right = Node(rv)
                q.append(node.right)
    result = []
    level = deque([root])
    left_to_right = True
    while level:
        size = len(level)
        cur = []
        for _ in range(size):
            node = level.popleft()
            cur.append(node.val)
            if node.left:
                level.append(node.left)
            if node.right:
                level.append(node.right)
        if not left_to_right:
            cur.reverse()
        result.append(cur)
        left_to_right = not left_to_right
    return result
```

Time: O(n) — each of the n nodes is built once and visited once during the BFS; reversing alternate levels is O(width) amortized to O(n) total. | Space: O(w) — the BFS queue holds at most one level at a time, bounded by the maximum width w of the tree (plus O(n) for the output, which is required).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def level_order_bottom_up(root):
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(c for c in (n.left, n.right) if c
)
    return out[::-1]
```

Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

FUNCTION AND TEST CASES

Signature

```
def zigzagLevelOrder(values: list[int | None]) -> list[list[int]]:
```

Sample call

```
zigzagLevelOrder([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
`([3, 9, 20, None, None, 15, 7],)`	`[[3], [20, 9], [15, 7]]`
`([1],)`	`[[1]]`

MEDIUM | IN APP LC 207 | Exponent | 57 of 196

★ TOP 75: rank 40 in the Medium and Hard priority list

★ Course Schedule

STUDY: TYPE | Google frequency on LeetCode: all time 51.9, last 30 days 33 | book priority 28 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #207)

LeetCode match: LC 207 Course Schedule

+1017 interviews

Original: https://www.aced.io/questions/3313/course-schedule

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

APPROACH

Kahn's topological sort (or DFS cycle detection); the answer is true iff every course is processed. O(V+E).

HOW TO THINK ABOUT IT

Finishing every course is possible exactly when the prerequisite graph has no cycle. Kahn's algorithm tests that by repeatedly taking courses with no unmet prerequisites; if you can take all of them there is no cycle, and the order you took them is a valid schedule.

Basis: LeetCode 207 (can you finish all courses) and 210 (return an order). A pair (a, b) means b must be taken before a.

BRUTE FORCE - PSEUDO-CODE

```
for every course: follow its dependencies looking for a path back to itself; if found:
    a cycle exists, so the courses cannot all be finished
```

Time: O(V * (V + E)) | Space: O(V)

Why it is slow

The same parts of the graph are walked again for many starting courses, O(V * (V + E)). Kahn's algorithm (take courses with no unmet prerequisites) is O(V + E).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #207 COURSE SCHEDULE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Can we finish all n courses given prerequisite pairs [a,b] meaning b must come before a?
Equivalent to: does the directed graph have no cycle? Right?"

"A few quick questions:
Return bool. If a cycle exists, some course can never be taken -> return false."

"Let me walk: n=2, prerequisites=[[1,0]] -> 0 before 1 -> no cycle -> true ok. [[1,0],[0,1]] -> cycle -> false ok

PHASE 3 — OPTIMIZE

Say: "Kahn's BFS topological sort — already O(V+E).
Walk-through of how cycle detection works:
Step 1: Build adjacency list and compute in-degree (number of incoming edges) for each node.
Step 2: Enqueue all nodes with in-degree == 0 (no prerequisites).
Step 3: BFS loop:
Pop node u. completed += 1.
For each neighbor v of u: indegree[v] -= 1.
If indegree[v] == 0 -> enqueue v (all its prerequisites are now done).
Step 4: If completed == numCourses -> no cycle, all courses can be taken. Else -> cycle detected.
WHY cycle detection works: nodes in a cycle NEVER reach indegree 0 (they depend on each other),
so they're never enqueued -> never counted -> completed < numCourses.

Time: O(V+E). Space: O(V+E).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict, deque
class _207_CourseSchedule:
    def canFinish(self, numCourses, prerequisites):
        graph = defaultdict(list)
        indegree = [0] * numCourses
        for a, b in prerequisites:
            graph[b].append(a)
            indegree[a] += 1
        queue = deque(i for i in range(numCourses) if in
degree[i] == 0)
        processed = 0
        while queue:
            node = queue.popleft()
            processed += 1
            for neighbor in graph[node]:
                indegree[neighbor] -= 1
                if indegree[neighbor] == 0:
                    queue.append(neighbor)
        return processed == numCourses
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace canFinish on n=4, prerequisites=[[1,0],[2,0],[3,1],[3,2]]."
Build graph: 0->[1,2], 1->[3], 2->[3].
Kahn's BFS: in-degree=[0,1,1,2]. queue=[0]. processed=0.
Pop 0: neighbors 1,2. in_degree[1]--=0->enqueue 1. in_degree[2]--=0->enqueue 2. processed=1.
Pop 1: neighbor 3. in_degree[3]--=1. processed=2.
Pop 2: neighbor 3. in_degree[3]--=0->enqueue 3. processed=3.
Pop 3: no neighbors. processed=4. 4==n -> True ok
Cycle case [[1,0],[0,1]]: in_degree=[1,1]. queue=[]. processed=0!=2 -> False ok
WHY processed==n: every course was reachable — no cycle trapped any node.
DFS cycle detect: visited[node]=1 (in current path), 2 (done). call stack: dfs(0)->dfs(1)->dfs(0): finds visited[0]=1 -> CYCLE -> False. pop stack, mark done.
DAG (Directed Acyclic Graph) check: component with cycle->course impossible. dist[node]=topo order. If back-edge found during DFS -> cycle -> return False. Reverse topo = valid course order.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(V+E). Space O(V+E).
DFS approach also O(V+E) — Kahn's is iterative (no stack overflow risk for large graphs).
Follow-up: Course Schedule II (#210) — same Kahn's BFS, collect the order.
Follow-up: Minimum Semesters (#1136) — count BFS levels to get minimum time.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the order of courses (LeetCode 210).

The same Kahn loop records the order; if fewer than n courses are taken return an empty list. O(V + E).

```
def find_order(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    queue, order = deque(c for c in range(n) if indeg[c] =
= 0), []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []
```

The minimum number of semesters if you can take any number of courses per semester.

Run Kahn's algorithm level by level: each level is one semester. The number of levels is the longest chain of prerequisites (return -1 if there is a cycle).

```
def semesters_needed(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    level, taken, rounds = [c for c in range(n) if indeg[c
] == 0], 0, 0
    while level:
        rounds += 1
        taken += len(level)
        nxt = []
        for c in level:
            for d in adj[c]:
                indeg[d] -= 1
                if indeg[d] == 0:
                    nxt.append(d)
        level = nxt
    return rounds if taken == n else -1
```

Report an actual cycle.

After Kahn's algorithm, the courses left over all still have an unmet prerequisite among themselves; walk backwards along prerequisites inside that set until a course repeats to extract a cycle.

OFFICIAL PAGE TEXT

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples

```
Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True
Explanation: You can complete course 0 first, then course
1, and finally course 2.
```

```
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1
, 0], [0, 3]]
Output: False
Explanation: The prerequisites form a cycle, making it imp
ossible to finish all courses.

Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2
, 0]]
Output: True
Explanation: You can complete the courses in order from co
urse 0 to course 4.
```

Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.Our solution processes the input data to determine if it's possible to finish all courses given the prerequisite constraints. It constructs a directed graph where nodes represent courses and edges represent the prerequisite relationships. The solution employs a queue to manage nodes with zero in-degrees (courses with no remaining prerequisites) and uses a list to keep track of the number of prerequisites (in-degrees) for each course. For each course with zero in-degrees, the solution dequeues it and processes its neighbors, reducing their in-degrees. If a neighbor's in-degree becomes zero, it is added to the queue. This approach ensures that all courses with no remaining prerequisites are processed and their dependencies are updated accordingly. By maintaining a count of the number of processed courses, the solution can determine if all courses can be finished. If the count of processed courses matches the total number of courses, it implies that there are no cyclic dependencies, and thus all courses can be completed. The solution uses a queue to efficiently manage courses with zero in-degrees and a vector to keep track of in-degrees. This method ensures that each course is processed only once, making the approach both clear and efficient. NoteWe are using a BFS (implemented by a queue) for this solution because it helps process nodes level by level (course by course), ensuring that we first handle courses with no dependencies before moving to those that rely on other courses. This approach ensures that we can detect cycles in the course prerequisites, which would make it impossible to complete all courses. Time Complexity: The solution has a time complexity of O(numCourses + numPrerequisites). This is because we process each course (vertex) and each prerequisite pair (edge) exactly once. The time complexity reflects the total number of courses (numCourses) and the number of prerequisite relationships (numPrerequisites). Space Complexity: The solution uses O(numCourses + numPrerequisites) space for the adjacency list and the in-degree vector. The space complexity accounts for the storage needed for both the graph representation and the queue, which in the worst case, contains all courses.No answers contributed by the community yet.

MEDIUM | IN APP LC 543 | Exponent | 58 of 196

Given the root of a binary tree, return the length of the diameter of the tree.

STUDY: TYPE | Google frequency on LeetCode: all time 44.4, last 30 days 0 | book priority 72 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #543)

LeetCode match: LC 543 Diameter of Binary Tree

+18 interviews

Original:

https://www.aced.io/questions/2705/binary-tree-diameter-length

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

APPROACH

DFS returning height; update diameter = max(diameter, left + right). O(n).

HOW TO THINK ABOUT IT

The longest path in a tree has a highest node, and the length of the best path through a node is the height of its left subtree plus the height of its right subtree. So compute heights bottom-up and, at every node, check left height + right height against a running best. One traversal does both jobs.

Basis: LeetCode 543, Diameter of Binary Tree (length counted in edges).

BRUTE FORCE - PSEUDO-CODE

```
for every node: leftHeight = height(left), rightHeight = h
eight(right) (each computed
    by a fresh traversal)
diameter = the largest leftHeight + rightHeight over all n
odes
```

Time: O(n^2) worst case | Space: O(h)

Why it is slow

Heights of the same subtrees are recomputed for every ancestor, O(n^2) on a skewed tree. One post-order pass returns heights and updates the best on the way, O(n).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #543 DIAMETER OF BINARY TREE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Diameter = length of the longest path between any two nodes (measured in edges).
The path may or may not pass through the root. Right?"

"A few quick questions:
Single node -> diameter 0? Yes. Two nodes -> diameter 1? Yes."

"Let me walk: root=[1,2,3,4,5] -> path 4->2->1->3 has 3 edges -> diameter=3 ok

PHASE 3 — OPTIMIZE

Say: "Post-order DFS: diameter through each node = left_depth + right_depth — O(n). Walk-through:
def depth(node):
If node is None: return 0.
left = depth(node.left). right = depth(node.right).
ans = max(ans, left + right). <- diameter through this node
return 1 + max(left, right). <- max depth for parent's calculation
WHY left + right: the diameter using this node as 'bend' = longest path from left subtree
through this node to right subtree = left_depth + right_depth.
WHY return 1 + max(left, right): for the parent, this node contributes its own height
(the longer of its two subtrees + 1 for itself).
Global max: maintained across all nodes.
Time: O(n). Space: O(h) call stack.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _543_DiameterOfBinaryTree:
    def diameterOfBinaryTree(self, root):
        best = [0]
        def depth(node):
            if not node:
                return 0
            left = depth(node.left)
            right = depth(node.right)
            best[0] = max(best[0], left + right)
            return 1 + max(left, right)
        depth(root)
        return best[0]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace diameterOfBinaryTree on [1,2,3,4,5]."
depth(4): L=0, R=0. diameter candidate=0+0=0. return 1.
depth(5): same -> return 1. diameter=0.
depth(2): L=depth(4)=1, R=depth(5)=1. candidate=1+1=2. ans=2. return 1+max(1,1)=2.
depth(3): L=0, R=0. candidate=0. return 1.
depth(1): L=depth(2)=2, R=depth(3)=1. candidate=2+1=3. ans=3. return 1+max(2,1)=3.
Answer: 3 ok (path 4->2->5->? No: 4->2->1->3 has length 3 edges)
Actually path is 4->2->5 or 4->2->1->3 — both length 3. ans=max=3 ok
WHY left+right at each node: diameter passing through this node = left depth + right depth.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n): single post-order DFS. Space O(h) call stack.
The single-element list best=[0] is the Python pattern for mutating a variable in a closure.
Follow-up: Binary Tree Maximum Path Sum (#124) — same pattern with sum instead of count.
Follow-up: Diameter of N-Ary Tree (#1522) — extend to multiple children.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

An n-ary tree.

Take the two largest child heights at each node (not just left and right): best = max(best, top1 + top2). Sorting the children's heights costs O(d log d) per node, or track the top two in O(d).

```python
class NaryNode:
    def __init__(self, val, children=None):
        self.val, self.children = val, children or []


def nary_diameter(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        tops = sorted((depth(c) for c in node.children), r
everse=True)[:2]
        best = max(best, sum(tops))
```

```
        return 1 + (tops[0] if tops else 0)

    depth(root)
    return best
```

Edges have weights, or you want the path itself.
Add the edge weight to each child's height before comparing; for the path, also store which child gave each of the two best heights and rebuild from the node that produced the best sum.

Count the diameter in nodes instead of edges.
Add one to the answer (a path of k edges has k + 1 nodes). State which convention the interviewer wants before you code.

OFFICIAL PAGE TEXT

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree:

```
// tree
        1
       / \
      2   3
     / \
    4   5
```

The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.

MEDIUM | IN APP LC 105 | Exponent | 59 of 196

Construct a binary tree given its preorder and inorder traversals.

STUDY: TALK | Google frequency on LeetCode: all time 38.3, last 30 days 0 | book priority 87 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #105)

LeetCode match: LC 105 Construct Binary Tree from Preorder and Inorder Traversal

+26 interviews

Original: https://www.aced.io/questions/3417/construct-binary-tree-preorder-inorder-traversal

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7] Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2] Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

APPROACH

Preorder gives the root; hash map of inorder indices splits left/right; recurse with index bounds. O(n).

HOW TO THINK ABOUT IT

Preorder tells you the root (its first element), and inorder tells you how many nodes go left (everything before the root in inorder). That cut gives two independent smaller problems. The only expensive step is locating the root in inorder, which a hash map of value to index makes O(1).

Basis: LeetCode 105, Construct Binary Tree from Preorder and Inorder Traversal (values must be unique).

BRUTE FORCE - PSEUDO-CODE

```
build(pre, in): if empty: return nothing
root = pre[0]; i = position of root in in (linear search)
left = build(pre[1..i], in[..i]); right = build(pre[i+1..]
, in[i+1..]) using list
    slices
```

Time: O(n^2) | Space: O(n^2) for the slices

Why it is slow

The linear search and the list copies make a skewed tree O(n^2) in time and space. A dictionary of inorder positions with index bounds is O(n).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #105 CONSTRUCT BINARY TREE FROM PREORDER AND INORDER TRAVERSAL)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Preorder: root first, then left subtree, then right subtree.
Inorder: left subtree, root, right subtree.
All values are unique — so we can find the root's position in inorder uniquely. Right?"

"preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]:
Root=3. In inorder, 3 is at index 1. Left subtree=[9], right subtree=[15,20,7].
preorder left=[9], right=[20,15,7]. Recurse."
Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 3 — OPTIMIZE

Say: "Use preorder[0] as root; split inorder with index map — O(n). Walk-through:
Build inorder_map: value -> index (for O(1) lookup).
build(pre_start, pre_end, in_start, in_end):
If pre_start > pre_end: return None.
root_val = preorder[pre_start]. root = TreeNode(root_val).
mid = inorder_map[root_val].
left_size = mid - in_start.
root.left = build(pre_start+1, pre_start+left_size, in_start, mid-1).

root.right = build(pre_start+left_size+1, pre_end, mid+1, in_end).
return root.
WHY preorder[0] is root: preorder visits root FIRST.
WHY inorder split at mid: left of mid = left subtree (in_start..mid-1), right = right subtree (mid+1..in_end).
WHY left_size: preorder's left subtree occupies next left_size positions.
Time: O(n). Space: O(n).

PHASE 4 — CLEAN CODE

```
class _105_BuildTree:
    def buildTree(self, preorder: List[int], inorder: Li
st[int]) -> Optional['TreeNode']:
        in_idx = {val: i for i, val in enumerate(inorder
)}
        pre    = iter(preorder)

        def build(left: int, right: int) -> Optional['Tr
eeNode']:
            if left > right:
                return None
            root_val = next(pre)
            root     = TreeNode(root_val)
            mid      = in_idx[root_val]
            root.left  = build(left, mid - 1)
            root.right = build(mid + 1, right)
            return root

        return build(0, len(inorder) - 1)
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace construction from preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]."
preorder[0]=3 -> root. inorder.index(3)=1. left_size=1.
Left call: pre[1:2]=[9], in[0:1]=[9] -> preorder[0]=9->root. No children -> Node(9).
Right call: pre[2:5]=[20,15,7], in[2:5]=[15,20,7]:
pre[0]=20->root. inorder.index(20)=1. left_size=1.
Left: pre[1:2]=[15],in[0:1]=[15]->Node(15).
Right: pre[2:3]=[7],in[2:3]=[7]->Node(7).
Tree: 3->left=9, right=20(left=15,right=7) ok
WHY preorder[0]=root, inorder split: pre gives root first; in tells where to split L vs R.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n). Space O(n) for the hash map and call stack.
Using iter(preorder) as a lazy pointer is cleaner than an index variable.
The key insight: preorder always gives the root first, inorder tells us how many
elements are in each subtree — that's the only information we need to recurse.
Given more time I'd ask: construct from postorder and inorder?
Same logic — just consume postorder from the RIGHT end, and build right subtree first.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

You are given postorder and inorder instead.

The root is the last postorder element, and because postorder reads left, right, root, reading it backwards yields root, right, left, so build the right subtree before the left.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def buildTree_post(inorder, postorder):
    where = {v: i for i, v in enumerate(inorder)}
    nxt = len(postorder) - 1

    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(postorder[nxt])
        nxt -= 1
        mid = where[root.val]
        root.right = build(
            mid + 1, hi
        )  # postorder read backwards visits the right sub
tree first
        root.left = build(lo, mid)
        return root

    return build(0, len(inorder))
```

Values can repeat.

Then the tree is not unique and the inorder index of the root is ambiguous. Ask for a rule (or for a preorder with null markers). Preorder plus postorder alone also cannot determine a general binary tree.

The tree is a 100,000-node chain.

The recursion is as deep as the tree. Use an explicit stack: push nodes while walking preorder, and pop back to the parent when the next node's inorder index shows you have finished the left side.

OFFICIAL PAGE TEXT

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples

```
preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7]
output: [3, 9, 20, null, null, 15, 7]
Explanation:
Given the two arrays, the binary tree constructed is:
        3
       / \
      9   20
         /  \
        15   7
```

```
preorder = [1, 2], inorder = [2, 1]
output: [1, 2]
Explanation:
Given the two arrays, the binary tree constructed is:
```

```
  1
 /
2
```

Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order. Our solution to construct a binary tree from pre-order and in-order traversal arrays uses a recursive approach. We first identify the root node from the pre-order array, then split the in-order array into left and right subtrees based on the root's position in the in-order array. We continue this process recursively to build the entire binary tree. We start by picking the first element of the preorder array as the root of the tree. The index of this root element in the inorder array helps us divide the tree into left and right subtrees. The elements to the left of the root in the inorder array form the left subtree, and the elements to the right form the right subtree. We then recursively build the left and right subtrees using the next available elements in the preorder array. To ensure efficient lookups of the root's index in the inorder array, we create a hash map (inorder_index_map) that maps values to their indices in the inorder array. This allows us to retrieve the position of the root in constant time. NoteRecursion: Base Case: If there are no elements to construct the tree (i.e., the preorder or inorder array is empty), we return nullptr. Recursive Step: We extract the root from the preorder array and find its position in the inorder array using the hash map. This splits the inorder array into left and right subtrees. We then recursively build the left and right subtrees and attach them to the root. By maintaining the correct indices for both the pre-order and in-order arrays, we can recursively construct the entire tree. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the tree. This is because we traverse both the preorder and inorder arrays once, and we use a hash map for constant-time lookups of root indices in the inorder array. Space Complexity: The solution uses O(n) space for the hash map that stores the indices of elements in the inorder array, and the recursion stack requires O(h) space, where h is the height of the tree. In the worst case (a skewed tree), the height of the tree can be O(n), so the total space complexity is O(n).No answers contributed by the community yet.

MEDIUM | IN APP LC 98 | Exponent | 60 of 196

Determine if a given binary tree is a binary search tree (BST).

STUDY: TYPE | Google frequency on LeetCode: all time 36, last 30 days 41.1 | book priority 46 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #98)

LeetCode match: LC 98 Validate Binary Search Tree (inferred from short statement)

+28 interviews

Original:

https://www.aced.io/questions/1158/determine-binary-tree-bst

QUESTION

Determine if a given binary tree is a valid Binary Search Tree (BST).

APPROACH

DFS with (low, high) bounds, or in-order traversal staying strictly increasing. O(n).

HOW TO THINK ABOUT IT

Checking only a node against its two children is not enough; every node in the left subtree must be smaller than the node, not just the left child. Pass down the allowed open interval (low, high) for each subtree, or use the fact that an in-order walk of a BST is strictly increasing.

Basis: LeetCode 98, Validate Binary Search Tree (strict: left < node < right, no duplicates).

BRUTE FORCE - PSEUDO-CODE

```
list = the in-order traversal of the tree
return whether the list is strictly increasing
```

Time: O(n) | Space: O(n)

Why it is slow

O(n) time but stores every value, O(n) space. Passing the allowed (low, high) range down the tree checks each node once with O(height) space and stops at the first violation.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #98 VALIDATE BINARY SEARCH TREE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Every node's value must be strictly greater than all values in its left subtree
AND strictly less than all values in its right subtree — not just the immediate parent. Right?
The classic trap: [5,1,4,null,null,3,6] — root=5, right child=4 — 4<5 -> invalid ok"

"[2,1,3]: 1<2<3 -> true ok. [5,1,4,null,null,3,6]: 4<5 -> false ok."
Any constraints on tree height or node count I should design around?

PHASE 3 — OPTIMIZE (bounds propagation)

Pass min/max bounds down the tree.
Every node must satisfy min_val < node.val < max_val.
Left child: max_val = node.val.
Right child: min_val = node.val.
Use float('-inf') and float('inf') as initial bounds. Use inner helper."
Time: O(log n) — binary search on answer space.
Key insight: passing (min_val, max_val) bounds down the recursion replaces inorder-then-compare — each node is validated in O(1) on entry, giving O(n) total with O(h) space.

PHASE 4 — CLEAN CODE

```
class _98_ValidateBST:
    def isValidBST(self, root: Optional['TreeNode']) ->
bool:
        def valid(node, min_val, max_val):
            if not node:
                return True
            if not (min_val < node.val < max_val):
                return False
            return (valid(node.left,  min_val,   node.va
l) and
                    valid(node.right, node.val,   max_va
l))
        return valid(root, float('-inf'), float('inf'))
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace BST validation on [5,1,4,null,null,3,6]."
validate(5, -inf, +inf): val=5 in (-inf,+inf) ok
validate(1, -inf, 5): val=1 in (-inf,5) ok -> left=None ok, right=None ok
validate(4, 5, +inf): val=4 NOT > 5 -> return False
Returns False. Root says 5>4>3 but 4 is in right subtree -> invalid BST ok
[2,1,3]:
validate(2,-inf,+inf)->validate(1,-inf,2): 1<2 ok. validate(3,2,+inf): 3>2 ok. -> True ok
Edge: [5,4,6,null,null,3,7] — right subtree 6 has left child 3 < 5 (root) -> invalid.
validate(6,5,+inf)->validate(3,5,6): 3 NOT > 5 -> False ok WHY: bound carries down from ancestor.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n). Space O(h) for the call stack.
The bounds approach is the correct one — in-order checking works too but requires state.
The common mistake: only checking node vs parent, not the full subtree constraint.
Given more time I'd ask: what if duplicates are allowed (left <= node < right)?
Change the strict inequalities accordingly.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Duplicates are allowed (all on the left, or all on the right).
Make one of the two bounds inclusive: for 'duplicates go left' every node must satisfy lo < val <= hi (the bound inherited from a left ancestor is inclusive, the one from a right ancestor is exclusive). Tested on trees built by inserting duplicates to the left.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def is_bst_duplicates_left(root):
    """Duplicates allowed on the left: left subtree <= nod
e < right subtree."""
    stack = [(root, float("-inf"), float("inf"))]
    while stack:
        node, lo, hi = stack.pop()
        if node is None:
            continue
        if not (
            lo < node.val <= hi
        ):  # lower bound exclusive (a right ancestor), up
per bound inclusive (a left ancestor)
            return False
        stack.append((node.left, lo, node.val))
        stack.append((node.right, node.val, hi))
    return True
```

Do it with an in-order walk that keeps only the previous value.
Walk in order with an explicit stack and compare each value with the previous one (strictly greater). O(h) space and early exit.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def isValidBST_morris_free(root):
    prev, stack, node = None, [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if prev is not None and node.val <= prev:
            return False
        prev, node = node.val, node.right
    return True
```

Recover a BST where exactly two nodes were swapped (LeetCode 99).
An in-order walk shows one or two descents; the first and last nodes involved in the descents are the swapped pair. O(n) time and O(h) space (O(1) with Morris traversal).

OFFICIAL PAGE TEXT

Determine if a given binary tree is a valid Binary Search Tree (BST). HintA binary tree is considered a "valid BST" if for every node: All the values in its left subtree are strictly less than the node's value. All the values in its right subtree are strictly greater than the node's value. Both the left and right subtrees must also be valid BSTs. Examples

```
All trees defined in level-order
root = [5, 2, 6]
output: true

root = [5, 2, 4, null, null, 3, 6]
output: false

root = []
output: true
```

In these examples, the output is true if the given binary tree root is a valid BST, and false otherwise.Our solution to validate whether a binary tree is a Binary Search Tree (BST) uses a recursive approach. We traverse the tree while maintaining a range for valid node values (min and max). At each node, we check whether its value lies within the given range: If the node's value is not within the allowed range, we return False, indicating the tree is not a valid BST. If the node's value is valid, we recursively check its left and right subtrees: The left subtree must have values strictly less than the current node's value. The right subtree must have values strictly greater than the current node's value. For the left subtree, the range becomes (min, current_node_value). For the right subtree, the range becomes (current_node_value, max). We use nullptr to indicate unbounded ranges when there is no minimum or maximum restriction (e.g., for the root node or outermost nodes). If all nodes satisfy their respective conditions, we return True, confirming that the tree is a valid BST. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the binary tree. This is because we traverse each node exactly once, performing constant-time checks at each node. Space Complexity: The space complexity is O(h), where h is the height of the tree. This space is used by the recursion stack. In the worst case (for an unbalanced tree), the height could be n, making the space complexity O(n). For a balanced tree, the height would be O(log n).No answers contributed by the community yet.

MEDIUM | IN APP LC 1376 | Exponent | 61 of 196

Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 126 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: your app's interview approach (LeetCode #1376)
LeetCode match: LC 1376 Time Needed to Inform All Employees
Original: https://www.aced.io/questions/2411/minimum-iterations-information-distribution-n-ary-tree

QUESTION

Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

APPROACH

DFS/BFS from the root computing the max time along any root-to-leaf path (each node passes the info with its own delay); iterative to avoid recursion depth on large trees.

HOW TO THINK ABOUT IT

Each node can call one child per round, so a node with several children must choose an order. The subtree that needs the most rounds should be called first, because it starts earliest and the others can wait. That makes the time at a node the maximum over children of (position in the calling order + the child's own time), a bottom-up computation.

Basis: Tree broadcasting (minimum rounds). Not LeetCode 1376, which uses per-employee delays; here every node can inform one child per iteration.

BRUTE FORCE - PSEUDO-CODE

```
T(v) = 0 if v has no children
else try every ordering of v's children; for an ordering,
child i (counting from 1)
    finishes at i + T(child)
T(v) = the smallest, over all orderings, of the largest fi
nish time
```

Time: O(product of d! over nodes) | Space: O(n)

Why it is slow

A node with d children has d! orderings, so the cost is the product of d! over all nodes. Sorting the children by decreasing T gives the same answer in O(d log d).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #1376 TIME NEEDED TO INFORM ALL EMPLOYEES)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
A manager informs their direct subordinates, each taking informTime[i] minutes to inform their reports.
Find the total time to inform all employees. Right?"

"A few quick questions:
Headcount is the root? Yes. Can have multiple root candidates? No — exactly one head.
Return the maximum total time from root to any leaf? Yes."

"Let me walk: n=6, headID=2, manager=[-1,2,2,6,6,2,2], informTime=[0,0,4,0,0,3,0] -> 7 ok

PHASE 3 — OPTIMIZE

Say: "DFS/BFS from headID accumulating propagation time — already O(n). Walk-through:
Build children tree: manager[i] = i's boss -> build children[boss] =

[subordinates].
DFS from headID:
dfs(employee, time_so_far):
max_time = max(max_time, time_so_far)
for child in children[employee]:
dfs(child, time_so_far + informTime[employee])
<- time for child to receive info = time headID->employee + informTime[employee].
HOW propagation works: headID informs at time=0. Its subordinates receive info at
time=informTime[headID]. Their subordinates receive at time=informTime[headID]+informTime[subordinate].
We accumulate total delay along each path from root to leaf.
Answer = maximum accumulated time across all employees.
Time: O(n). Space: O(n).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
class _1376_TimeNeededToInformAllEmployees:
    def numOfMinutes(self, n, headID, manager, informTim
e):
        reports = defaultdict(list)
        for emp, mgr in enumerate(manager):
            if mgr != -1:
                reports[mgr].append(emp)
        def dfs(emp_id):
            max_sub_time = 0
            for sub_id in reports[emp_id]:
                max_sub_time = max(max_sub_time, dfs(sub
_id))
            return informTime[emp_id] + max_sub_time
        return dfs(headID)
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace numOfMinutes on informTime=[1,1,1,1,0,0,0] with chain structure."
DFS from headID:
dfs(headID): total_time = informTime[headID] + max(dfs(sub) for sub in subordinates[headID]).
Leaf employees: dfs(leaf) = informTime[leaf] + 0 = 0 (leaf has 0 inform time).
Chain: head->a->b->c (each inform time = t):
dfs(c)=0. dfs(b)=t+0=t. dfs(a)=t+t=2t. dfs(head)=t+2t=3t.
WHY max not sum: manager informs all subordinates in PARALLEL. Total = manager's time + max subordinate chain.
n=6,headID=2,manager=[-1,2,2,2,2,2],informTime=[0,0,0,0,0,0]: headID=0,chain of depth 1. All informed at once=0 ok

WHY tree details: return values from children drive decisions at the parent (post-order), so base cases must be correct.
DFS: dfs(headID,0). visited: each employee node once (tree, no cycles). call stack: dfs(head)->dfs(sub)->...->leaf (no subs). Max time propagates up via return value.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n): visit each employee once. Space O(n) adjacency list + recursion.
Follow-up: BFS alternative — same O(n) with a queue, avoids stack overflow for deep trees.
Follow-up: Employee Importance (#690) — sum instead of max.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why is it optimal to serve the child with the largest subtree time first?

Exchange argument: if a child with time a is served in round i and one with time b >= a in round i + 1, swapping them changes the finish times from max(i + a, i + 1 + b) to max(i + b, i + 1 + a), and the second is never larger because b >= a. So sorting by decreasing T minimizes the maximum. Cost: sorting the children at each node makes the whole computation O(n log n); the exchange argument adds nothing.

What if each edge has a different transmission cost?

Replace the unit step by the cost: the child served in order position i can only start after the sum of the costs of the edges before it, so T(v) = max over i of (prefix cost up to child_i + T(child_i)). The best order is still decreasing (cost + T) only when costs are equal; in general it becomes a scheduling problem solved by sorting on T(child) - cost, i.e. Smith's rule.

Each call takes a different time per child.

The order still matters, and the right rule is Jackson's: serve children in decreasing order of their own finishing time T, whatever each call costs, because the finish of the child served at prefix cost P is P + T and the largest tail should start earliest. Checked against trying every order on random trees.

```
def inform_rounds_weighted(n, children, cost, root=0):
    """Calling child c takes cost[c] rounds. Serve childre
n in decreasing order of their own finishing time T (Jacks
on's rule)."""

    def t(v):
        kids = sorted(
            ((cost[c], t(c)) for c in children.get(v, []))
, key=lambda ct: -ct[1]
        )
        clock = finish = 0
        for c, tc in kids:
            clock += c
            finish = max(finish, clock + tc)
        return finish

    return t(root)
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

MEDIUM | 1Point3Acres | onsite-coding | 62 of 196

Minimum Direction Violations in a Directed Graph

STUDY: TALK | Google frequency on LeetCode: all time 10.6, last 30 days 0 | book priority 151 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

Examples:

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

APPROACH

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.

Compare the proof and complexity with Dijkstra on the same transformed graph.

Test unreachable destinations, parallel edges, cycles, and start == end.

INTERVIEW-DAY PLAN

Clarify first

Ask what a violation is (traversing an edge against its direction), whether edge costs are all 1, what to return when the end cannot be reached, and whether the graph has parallel edges or self loops.

Brute force to say out loud

Plain BFS ignoring direction minimizes edges, not violations, so say why it is wrong. Dijkstra with costs 0 and 1 works at O((n + m) log n).

Optimal idea

Add every edge twice: forward with weight 0, backward with weight 1. Run 0-1 BFS with a deque: relax a weight-0 edge by pushing to the front, a weight-1 edge to the back. The deque always holds nodes whose distances differ by at most 1, so O(n + m).

Dry-run these cases

Start equals end (0). A graph with no path. A path that needs every edge reversed. A shortcut that costs more edges but fewer violations.

Likely follow-ups

General edge costs (Dijkstra), shortest path itself (parent pointers), or negative costs (Bellman-Ford).

Pitfalls

Using visited flags instead of a distance check. Pushing weight-0 edges to the back.

HOW TO THINK ABOUT IT

Going against an edge costs 1 and going with it costs 0, so it is a shortest-path problem with costs of only 0 and 1. Plain BFS counts edges, not violations, so it is the wrong tool. A deque lets 0-cost moves jump the queue (push front) and 1-cost moves wait their turn (push back), which is Dijkstra without the heap.

Basis: Related to LeetCode 1466, Reorder Routes: a shortest path where traveling an edge backwards costs 1.

BRUTE FORCE - PSEUDO-CODE

```
give every edge a forward copy of cost 0 and a backward co
py of cost 1
dist[start] = 0, everything else infinity
repeat V - 1 times: for every edge copy (u, v, w): if dist
[u] + w < dist[v]: dist[v] =
    dist[u] + w
answer = dist[end]
```

Time: O(V * E) | Space: O(V)

Why it is slow

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION - 0-1 BFS ON THE DOUBLED GRAPH

```python
def minDirectionViolations(n, edges, start, end):
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [float("inf")] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                dq.appendleft(v) if w == 0 else dq.append(
v)
    return -1 if dist[end] == float("inf") else dist[end]
```

Time: O(n + m) | Space: O(n + m)

Why

Add every edge twice: forward with weight 0, backward with weight 1. Shortest path with weights in {0, 1} is a deque BFS: push a zero-weight neighbour to the front and a one-weight neighbour to the back. Plain BFS minimizes edge count, not violations, which is the common mistake.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

```
def min_cost_path(n, edges, start, end):
    """Edges carry costs: edges[i] = (u, v, forward_cost,
backward_cost). Dijkstra with a heap."""
    import heapq

    graph = defaultdict(list)
    for u, v, f, b in edges:
        graph[u].append((v, f))
        graph[v].append((u, b))
    dist = {start: 0}
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, float("inf")):
            continue
        for v, w in graph[u]:
            if d + w < dist.get(v, float("inf")):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist.get(end, -1)
```

NOTES FROM CANDIDATES

Represent each original edge twice: the forward direction with weight 0 and the reverse direction with weight 1.

A regular BFS minimizes the number of edges, not the number of direction violations.

The linear-time target is O(n + m) time and space.

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.

Compare the proof and complexity with Dijkstra on the same transformed graph.

Test unreachable destinations, parallel edges, cycles, and start == end.

MEDIUM | 1Point3Acres | phone-screen | 63 of 196

Longest Consecutive Path in Binary Tree

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 143 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1. Follow-up: allow the path to extend in any direction through the tree, not only from parent to child.SWEtreedfsrecursionmediumFrequencySingle reportLast asked2026-07-02Stagephone-screen

APPROACH

Write the parent-to-child traversal in under 10 minutes with a global best.

Practice the bidirectional tree follow-up where each node returns two lengths upward.

Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

INTERVIEW-DAY PLAN

Clarify first

Ask whether length counts nodes or edges, whether the path goes only parent to child (base) or can bend through a node (follow-up), and whether the sequence must increase by exactly one.

Brute force to say out loud

From every node walk down both children recording the longest run. O(n^2) on a skewed tree.

Optimal idea

Base: DFS passing the current run length; it grows by one when the child is exactly the parent plus one, otherwise it resets to 1; keep the global best. O(n). Follow-up: each node returns the longest increasing and longest decreasing chain going down from it; the best path through a node is inc + dec - 1.

Dry-run these cases

A single node, an empty tree, duplicate values, a split branch, a descending chain.

Likely follow-ups

The any-direction version (above), and an iterative version for very deep trees.

Pitfalls

Counting edges when the interviewer expects nodes. Resetting the run to 0 instead of 1.

HOW TO THINK ABOUT IT

Each node needs to know the longest run that ends at its parent, so pass the current length down and reset it to 1 when the value is not parent + 1. For the bending version each node must report two lengths upward (increasing and decreasing chains going down), and the best path through a node combines one of each.

Basis: LeetCode 298 (parent-to-child) and LeetCode 549 (any direction), Binary Tree Longest Consecutive Sequence I and II.

BRUTE FORCE - PSEUDO-CODE

```
for every node as a starting point:
    walk down while a child's value is exactly the current
 value + 1, taking the
        longer of the two children
answer = the longest walk found
```

Time: O(n * h) | Space: O(h)

Why it is slow

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION - TOP-DOWN DFS CARRYING THE CURRENT RUN

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def longestConsecutive(root):
    best = 0

    def dfs(node, parent_val, run):
        nonlocal best
        if node is None:
            return
        run = run + 1 if parent_val is not None and node.v
al == parent_val + 1 else 1
        best = max(best, run)
        dfs(node.left, node.val, run)
        dfs(node.right, node.val, run)

    dfs(root, None, 0)
    return best
```

Time: O(n) | Space: O(h)

Why

Pass the length of the run ending at the parent. If the child is exactly parent + 1 the run grows, otherwise it restarts at 1. Track the global best. Length is counted in nodes; state that choice out loud.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Follow-up: path may bend through a node (inc and dec chains)

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def longestConsecutiveAnyDirection(root):
    best = 0

    def dfs(node):
        nonlocal best
        if node is None:
            return 0, 0
        inc = dec = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            ci, cd = dfs(child)
            if child.val == node.val + 1:
                inc = max(inc, ci + 1)
            elif child.val == node.val - 1:
                dec = max(dec, cd + 1)
        best = max(best, inc + dec - 1)
        return inc, dec

    dfs(root)
    return best
```

Time: O(n) | Space: O(h)

Why

Each node returns two numbers: the longest chain going down that increases by 1 per step, and the longest chain going down that decreases by 1. A path that bends at a node is an increasing chain on one side joined to a decreasing chain on the other, so its length is inc + dec - 1 (the node is counted twice).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def longest_consecutive_path_values(root):
    best = []

    def dfs(node, parent_val, path):
        if node is None:
            return
        path = (
            path + [node.val]
            if parent_val is not None and node.val == pare
nt_val + 1
            else [node.val]
        )
        if len(path) > len(best):
            best[:] = path
        dfs(node.left, node.val, path)
        dfs(node.right, node.val, path)

    dfs(root, None, [])
    return best
```

The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

NOTES FROM CANDIDATES

In the base version, carry the current consecutive length while traversing downward and reset to 1 whenever the child is not exactly parent + 1.

For the any-direction follow-up, track increasing and decreasing chain lengths at each node, then combine through the node when the left/right child values support a consecutive bridge.

Clarify whether the answer is measured in nodes or edges before coding; Google candidates usually state node count.

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.

Practice the bidirectional tree follow-up where each node returns two lengths upward.

Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

MEDIUM | 1Point3Acres | onsite-coding | 64 of 196

Shortest Directed Cycle Through a Node

STUDY: TALK | Google frequency on LeetCode: all time 5, last 30 days 0 | book priority 160 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

QUESTION

Examples:

Edges [(0,1),(1,2),(2,0)], v = 0 -> 3.

Edges [(0,1),(1,2)], v = 0 -> -1.

APPROACH

Write the directed BFS-from-source version from memory in under 12 min.

Practice the "re-entry detection" pattern (don't mark v visited until after pop).

INTERVIEW-DAY PLAN

Clarify first

Ask whether edges are directed (yes), whether self loops count (cycle of length 1), what to return if there is none (-1), and whether edges have weights.

Brute force to say out loud

Enumerate all simple paths from v back to v with DFS. Exponential.

Optimal idea

BFS from v. The first time you see an edge from a visited node u back to v, the cycle length is dist[u] + 1, and BFS order makes it the shortest. Do not mark v as discovered before checking for re-entry. O(V + E).

Dry-run these cases

A triangle (3). A path with no return (-1). A self loop (1). Two cycles through v of different lengths.

Likely follow-ups

Shortest cycle in the whole graph (BFS from every node), weighted edges (Dijkstra), or undirected graphs.

Pitfalls

Marking v visited at the start, which hides the cycle. Counting nodes instead of edges.

HOW TO THINK ABOUT IT

A cycle through v is a path from v back to v, so find shortest distances from v and watch for an edge that points back at v. Breadth-first search discovers nodes in order of distance, so the first edge back to v closes the shortest cycle. Do not mark v as 'seen' in a way that hides the edge that returns to it.

Basis: Related to LeetCode 2608, Shortest Cycle in a Graph, but directed and anchored at a given node.

BRUTE FORCE - PSEUDO-CODE

```
depth-first search from v over simple paths (never revisit
 a node on the current path)
whenever an edge leads back to v, record the current path
length + 1
answer = the smallest recorded length, or -1
```

Time: exponential | Space: O(V)

Why it is slow

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

OPTIMAL SOLUTION - BFS FROM V, CLOSE THE CYCLE ON THE FIRST EDGE BACK TO V

```python
def shortest_cycle_through(n, edges, v):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
    dist = {v: 0}
    queue = deque([v])
    while queue:
        u = queue.popleft()
        for w in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1
```

Time: O(V + E) | Space: O(V)

Why

A directed cycle through v is a path v to u followed by an edge u to v. BFS visits nodes in nondecreasing distance, so the first time we see an edge back to v its length is dist[u] + 1 and is the shortest. A

self-loop gives 1. Return -1 if BFS ends without closing.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

```
def shortest_cycle_through(n, edges, v):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
    dist = {v: 0}
    queue = deque([v])
    while queue:
        u = queue.popleft()
        for w in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


def girth_directed(n, edges):
    return min(
        (c for c in (shortest_cycle_through(n, edges, v) f
or v in range(n)) if c != -1),
        default=-1,
    )
```

NOTES FROM CANDIDATES

Strictly directed; do not relax to undirected.

BFS layer 0 is v itself; the first re-discovery during expansion is the cycle length.

Common bug: marking v as visited immediately blocks the re-entry.

PREPARATION

Write the directed BFS-from-source version from memory in under 12 min.

Practice the "re-entry detection" pattern (don't mark v visited until after pop).

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 65 of 196

Rotten Oranges / Multi-Source BFS (taxis)

STUDY: TYPE | Google frequency on LeetCode: all time 51.2, last 30 days 0 | book priority 60 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app (open the app for its Expose): LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

Examples:

Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges variant): return number of minutes until all reachable 1s become 2 (4).

Taxi variant: for each empty cell, distance to nearest taxi.

APPROACH

Drill multi-source BFS from memory in under 10 min.

Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.

Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

Basis: LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

BRUTE FORCE - PSEUDO-CODE

```
minutes = 0; repeat:
    find every fresh orange with a rotten neighbour; if no
ne: stop
    turn all of them rotten at once; minutes = minutes + 1
if any fresh orange remains: return -1, else return minute
s
```

Time: O(minutes * m * n) | Space: O(m * n)

Why it is slow

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION - MULTI-SOURCE BFS BY MINUTE

```
def orangesRotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue, fresh = deque(), 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1
    seen = {cell for cell in queue}
    minutes = 0
    while queue and fresh:
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c +
 1), (r, c - 1)):
                if (
                    0 <= nr < rows
                    and 0 <= nc < cols
                    and grid[nr][nc] == 1
                    and (nr, nc) not in seen
                ):
                    seen.add((nr, nc))
                    fresh -= 1
                    queue.append((nr, nc))
        minutes += 1
    return -1 if fresh else minutes
```

Time: O(mn) | Space: O(mn)

Why

Push every rotten orange into the queue at minute 0 and expand level by level; each level is one minute. Count the fresh oranges and decrement as they rot, so the answer is -1 if any remain at the end. Processing level by level (the inner for loop over the current queue length) is what turns BFS depth into minutes.

OPTIMAL SOLUTION - TAXI VARIANT: DISTANCE FROM EVERY EMPTY CELL TO THE NEAREST TAXI

```
def nearest_taxi_distance(grid):
    rows, cols = len(grid), len(grid[0])
    dist = [[-1] * cols for _ in range(rows)]
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and grid[nr][nc] != "#"
                and dist[nr][nc] == -1
            ):
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist
```

Time: O(mn) | Space: O(mn)

Why

Seed the queue with all taxis ('T') at distance 0 and BFS through open cells ('.'); walls ('#') are never entered. The first time a cell is reached is its shortest distance because BFS expands in nondecreasing distance. Cells that no taxi can reach stay at -1. Running a BFS per taxi instead would cost O(mn * taxis).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

```
def signal_time(grid):
    """'S' sources, 'T' targets, '#' walls, '.' open: time
 until every target is reached; -1 if one is unreachable;
0 with no targets."""
    R, C = len(grid), len(grid[0])
    dist = {(r, c): 0 for r in range(R) for c in range(C)
if grid[r][c] == "S"}
    queue = deque(dist)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if (
                0 <= nr < R
                and 0 <= nc < C
                and grid[nr][nc] != "#"
                and (nr, nc) not in dist
            ):
                dist[(nr, nc)] = dist[(r, c)] + 1
                queue.append((nr, nc))
    targets = [(r, c) for r in range(R) for c in range(C)
if grid[r][c] == "T"]
    if not targets:
        return 0
    return -1 if any(t not in dist for t in targets) else
max(dist[t] for t in targets)
```

NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).
The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.
For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 66 of 196

★ TOP 75: rank 17 in the Medium and Hard priority list

★ Number of Islands

STUDY: TYPE | Google frequency on LeetCode: all time 64.8, last 30 days 33 | book priority 18 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: your app's interview approach (LeetCode #200)
LeetCode match: LC 200 Number of Islands
Frequency: Low | Last asked: 2026-08-29 | Roles: SWE, MLE
Original: https://www.1point3acres.com/interview/problems/company/google/number-of-islands

QUESTION

Examples:
[[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2.
Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1] (last addition merges the two land cells).

APPROACH

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

HOW TO THINK ABOUT IT

Each island is a connected group of land cells, so count how many times you start a fresh search at an unvisited land cell. The search itself (DFS or BFS) just marks everything reachable. If land is added over time, do not rerun the search: Union-Find keeps the count up to date.

Basis: LeetCode 200, Number of Islands, and LeetCode 305, Number of Islands II (streaming additions with Union-Find).

BRUTE FORCE - PSEUDO-CODE

```
count = 0; for every cell: if it is land and not visited:
count = count + 1, then mark
    all land connected to it by recursing into the four ne
ighbours
```

Time: O(m * n) | Space: O(m * n) recursion

Why it is slow

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #200 NUMBER OF ISLANDS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Count connected components of '1's in a binary grid. Diagonal connections don't count. Right?"
Is the graph guaranteed to be connected? Can it contain cycles or self-loops?

PHASE 3 — OPTIMIZE

Say: "DFS flood-fill IS already the optimal O(m*n) approach.
Walk-through of how DFS generates the island count:
For each unvisited '1': call dfs(r, c).
dfs(r, c):
Base cases: out of bounds OR cell is not '1' -> return immediately.
Mark grid[r][c] = '#' (visited in-place — avoids a separate visited set).
Recurse in all 4 directions: dfs(r-1,c), dfs(r+1,c), dfs(r,c-1), dfs(r,c+1).
Each dfs call floods one connected island component completely before returning.
After dfs returns, we increment count -> one full island discovered.
In-place marking means each cell is visited at most once across ALL dfs calls -> O(m*n) total.

PHASE 4 — CLEAN CODE

```
class _200_NumberOfIslands:
    def numIslands(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])

        def dfs(r, c):
            if r < 0 or r >= m or c < 0 or c >= n or gri
d[r][c] != '1':
                return
            grid[r][c] = '#'    # mark visited by floodi
ng
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                dfs(r + dr, c + dc)

        count = 0
        for r in range(m):
            for c in range(n):
                if grid[r][c] == '1':
                    dfs(r, c)
                    count += 1
        return count
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace numIslands on [['1','1','0'],['0','1','0'],['0','0','1']]."
DFS from each unvisited '1'. Mark visited by setting to '0'.
(0,0)='1' -> dfs(0,0): mark '0'. Visit (0,1)='1'->dfs(0,1): mark '0'.
(1,1)='1'->dfs(1,1): mark '0'. All neighbors of (1,1) are '0' or out of bounds -> return. Back to (0,1): no more live neighbors. count=1.
(0,2)='0' skip. (1,0)='0' skip. (1,1)='0' skip. (1,2)='0' skip.
(2,0)='0' skip. (2,1)='0' skip. (2,2)='1' -> dfs(2,2): mark '0'.
count=2.
Return 2 ok
Edge: all water -> return 0 ok. All land -> DFS spreads to entire grid -> 1 island ok
WHY mark '0' during DFS: prevents revisiting. No extra visited array needed (in-place O(1) space).
DFS: visited (mark cell '0' to sink it). island_count=0. For each unvisited '1': dfs->sink all connected land. visited via mutation: '1'->'0'. component per call=1 island.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(m*n). Space O(m*n) stack. Flooding in-place avoids a separate visited set.
BFS alternative: use a queue instead of recursion — better for very large grids (no stack overflow).
Union-Find alternative: O(m*n * alpha(m*n)) — useful when islands merge dynamically.
Given more time I'd ask: restore the grid after counting?
Keep a separate visited set or restore '#' -> '1' after each DFS.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Follow-up: land is added one cell at a time (Union-Find)

```
class IslandCounter:
    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        self.parent, self.size, self.count = {}, {}, 0

    def _find(self, x):
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:
            self.parent[x], x = root, self.parent[x]
        return root

    def add_land(self, r, c):
        cell = (r, c)
        if cell in self.parent:
            return self.count
        self.parent[cell], self.size[cell] = cell, 1
        self.count += 1
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if (nr, nc) in self.parent:
                a, b = self._find(cell), self._find((nr, n
c))
                if a != b:
                    if self.size[a] < self.size[b]:
                        a, b = b, a
                    self.parent[b] = a
                    self.size[a] += self.size[b]
                    self.count -= 1
        return self.count
```

Time: O(k * alpha(mn)) for k additions | Space: O(k)

Why

Keep a disjoint-set over land cells only. Adding a cell makes the island count go up by one, then each union with a land neighbour that is in a different set merges two islands and decrements the count. Re-adding an existing land cell must be ignored. Path compression plus union by size gives near-constant time per operation, far better than re-running the DFS (O(mn) per update).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

```
def largest_island(grid):
    m, n, best, seen = len(grid), len(grid[0]), 0, set()
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                size, stack = 0, [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    size += 1
                    for nx, ny in ((x + 1, y), (x - 1, y),
 (x, y + 1), (x, y - 1)):
                        if (
                            0 <= nx < m
```

```
                        and 0 <= ny < n
                        and grid[nx][ny] == 1
                        and (nx, ny) not in seen
                    ):
                        seen.add((nx, ny))
                        stack.append((nx, ny))
            best = max(best, size)
    return best
```

The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. Cost: O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

NOTES FROM CANDIDATES

For streaming, Union-Find with path compression is the natural choice — each addLand is amortized alpha(n).

In one onsite, the follow-up arrived after a working BFS solution and explicitly required a Union-Find rewrite; retrofitting the original traversal consumed the remaining time.

Common edge cases: diagonal connectivity (some prompts use 8-connectivity), grid with no land.

PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.

Be ready to switch between DFS / BFS / Union-Find on demand.

MEDIUM | IN APP LC 236 | HackTheRounds | Onsite | 67 of 196

Lowest Common Ancestor

STUDY: TALK | Google frequency on LeetCode: all time 41.5, last 30 days 0 | book priority 79 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #236 Lowest Common Ancestor of a Binary Tree

LeetCode match: LC 236

QUESTION

Given a binary tree and two nodes p and q, find their lowest common ancestor (LCA): the lowest node that has both p and q as descendants (a node can be a descendant of itself). Follow-up: what if it is an n-ary tree?

EXAMPLES

Example 1	
Input	root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output	3
Why	LCA of 5 and 1 is 3.

Example 2	
Input	root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output	5
Why	A node can be its own ancestor.

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

APPROACH

Recursive DFS; follow-up: n-ary tree

HOW TO THINK ABOUT IT

Ask each subtree 'do you contain p or q?'. The first node where both subtrees report a hit (or the node is p or q itself and a subtree has the other) is the answer. One pass returns the answer upward: a node returns itself if p or q, the child result if only one side found something, and itself if both sides did.

Basis: LeetCode 236, Lowest Common Ancestor of a Binary Tree (and its n-ary follow-up).

BRUTE FORCE - PSEUDO-CODE

```
path_p = the nodes from the root to p; path_q = the same f
or q
answer = the last node where the two paths are still the s
ame
```

Time: O(n) | Space: O(n)

Why it is slow

Two full paths stored, O(n) space, and two traversals. One recursive pass that reports whether each subtree contains p or q finds the answer in O(n) time and O(h) space.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: RECURSIVE DFS

```
from typing import Optional


class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None


class Solution:
    def lowestCommonAncestor(self, root: TreeNode, p_val:
int, q_val: int) -> int:
        # p_val and q_val are integer values, not TreeNode
 objects
        def dfs(node):
            if not node:
                return None
            if node.val == p_val or node.val == q_val:
                return node

            left = dfs(node.left)
            right = dfs(node.right)

            if left and right:
                return node
            return left if left else right

        result = dfs(root)
```

```
        return result.val if result else -1
```

Time: O(n) where n is number of nodes - we visit each node once | Space: O(h) where h is tree height - recursion stack depth

Explanation

Intuition

This is one of the most elegant recursive solutions you'll encounter. The key insight is understanding what each recursive call returns.

What Does Our Function Return? For any node, our function returns:

The node itself if it's p or q
The LCA if both p and q are found in its subtree
Whatever was found in left/right subtree (could be p, q, or null)
Null if neither p nor q is in its subtree

The Three Cases:

Case 1: Current node is p or q If we find p or q, return it immediately. This node could be the LCA, or it could be passed up for comparison.

Case 2: p and q are in different subtrees If left subtree returns something and right subtree returns something, the current node must be the LCA (p is on one side, q is on the other).

Case 3: p and q are in the same subtree If only one side returns something, both p and q must be in that subtree. Pass up whatever was found (which is the LCA).

Visual Example:

```
      / \
     5 1
    /\ /\
   6 2 0 8
    /\
   7 4
```

LCA(5, 1):

- Check node 3
- left(5): returns 5 (found p!)
- right(1): returns 1 (found q!)
- Both sides returned -> 3 is LCA!

LCA(5, 4):

- Check node 3
- left(5): 5 is p, but keep checking... eventually returns 5 (because 4 is also found in 5's subtree)
- right(1): returns null
- Only left returned -> LCA is in left subtree -> return 5

Why This Works: The recursion naturally "bubbles up" information. When both children return non-null, we know we're at the meeting point. When only one returns non-null, the answer is in that direction.

Algorithm
Base case: if root is null, return null
Base case: if root is p or q, return root
Recurse on left subtree
Recurse on right subtree
If both left and right returned non-null: root is LCA
If only left is non-null: return left (LCA is in left subtree)
If only right is non-null: return right (LCA is in right subtree)
If both null: return null

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The tree is n-ary.

The same recursion: collect the non-empty results from all children; if two or more children reported a hit this node is the answer, otherwise pass the single result (or None) up.

```
class NNode:
    def __init__(self, val, children=None):
        self.val, self.children = val, children or []


def lca_nary(root, p, q):
    def dfs(node):
        if node is None or node is p or node is q:
            return node
        hits = [r for r in (dfs(c) for c in node.children)
 if r]
        return node if len(hits) >= 2 else (hits[0] if hit
s else None)

    return dfs(root)
```

A very deep tree, or no recursion allowed.

Do one traversal that records each node's parent (a dict), then use the ancestor-set method from the parent-pointer version. O(n) time and space, no recursion.

```
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def lca_with_parent_map(root, p, q):
    parent, stack = {root: None}, [root]
    while stack:
        n = stack.pop()
        for c in (n.left, n.right):
            if c:
                parent[c] = n
                stack.append(c)
    ancestors = set()
    while p:
        ancestors.add(p)
        p = parent[p]
    while q not in ancestors:
        q = parent[q]
    return q
```

p or q might not be in the tree.

The recursive answer is wrong in that case (it returns the one that exists). Count how many of p and q were seen during the DFS, or first check both exist, and return None if not.

MEDIUM | PracHub | Technical Screen | 68 of 196

Count Prefix-Matching Simple Paths in a Labeled Graph

STUDY: TALK | Google frequency on LeetCode: all time 42.5, last 30 days 0 | book priority 75 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

APPROACH

DFS/backtracking from every start vertex, extending a path only while the labels match the next query character and the vertex is unvisited (visited bitmask); count paths for every prefix length.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the graph is directed, whether a path is counted as a vertex sequence, whether paths of every prefix length are counted (the statement says every nonempty prefix), and the size (small, since the state is a visited subset).

Brute force to say out loud

Enumerate all simple paths and then test labels. Exponential and wasteful.

Optimal idea

DFS with backtracking from every start vertex. Extend a path only if the next vertex is unvisited (a bitmask of visited vertices) and its label equals the next character of the query word. Every time the path length is L, count one for prefix length L. The pruning on labels keeps the search small. Time depends on the number of matching paths, bounded by n * d^(L-1).

Dry-run these cases

A query of length 1 (count of vertices with that label), a graph with a cycle (simple paths may not revisit), a label that appears nowhere, repeated letters in the query.

Likely follow-ups

Longer queries (memoize on (vertex, mask, position) only helps if counting, not listing), or undirected edges.

Pitfalls

Counting a path that revisits a vertex. Counting an undirected path twice (once in each direction) without agreeing on it.

HOW TO THINK ABOUT IT

A path is only worth extending while its labels still match the query, so prune on the label before going deeper. Distinctness is tracked with a visited set (a bitmask for small graphs). Count at every depth, since each prefix length has its own answer.

Basis: Written; backtracking on a small labelled graph. Assumed rules (confirm them): directed edges; a path is a sequence of distinct vertices; for each L from 1 to the length of the query, count the paths of L vertices whose labels spell the first L letters of the query.

BRUTE FORCE - PSEUDO-CODE

```
list every simple path of every length (depth-first, never
 revisiting a vertex)
for each path and each prefix length L: if the path's firs
t L labels equal the query's
    first L letters: count it
```

Time: exponential | Space: O(path)

Why it is slow

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def count_prefix_paths(labels, edges, word):
    vertex_count = len(labels)
    adjacency = [0] * vertex_count
    for first, second in edges:
        adjacency[first] |= 1 << second
        adjacency[second] |= 1 << first

    states = {}
    for vertex, label in enumerate(labels):
        if label == word[0]:
            states[(1 << vertex, vertex)] = 1

    counts = [sum(states.values())]
    for character in word[1:]:
        next_states = {}
        for (visited, endpoint), ways in states.items():
            available = adjacency[endpoint] & ~visited
            while available:
                bit = available & -available
                neighbor = bit.bit_length() - 1
                available -= bit
                if labels[neighbor] == character:
                    state = (visited | bit, neighbor)
                    next_states[state] = next_states.get(s
tate, 0) + ways
        states = next_states
        counts.append(sum(states.values()))
    return counts
```

Time: O(len(word) * n^2 * 2^n) in the worst case | Space: O(n * 2^n) frontier states

Explanation

Represent each partial path by its visited-vertex bitmask and endpoint. The state value is the number of distinct vertex sequences compressed into that pair, so different path orders remain counted. Initialize every vertex matching the first character, then extend only to adjacent, unvisited vertices matching the next character. Summing the frontier multiplicities after each step gives the corresponding prefix count.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

```
def prefix_paths(labels, edges, query):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 < len(query):
            for w in adj[u]:
                if not mask >> w & 1 and labels[w] == quer
y[depth + 1]:
                    go(w, depth + 1, mask | 1 << w)

    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts


def prefix_paths_undirected(labels, edges, query):
    both = list(edges) + [(v, u) for u, v in edges]
    return prefix_paths(labels, both, query)
```

Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`

MEDIUM | PracHub | Technical Screen | 69 of 196

Solve matrix groups and recipe inventory

STUDY: TALK | Google frequency on LeetCode: all time 33.8, last 30 days 0 | book priority 102 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #2115 Find All Possible Recipes from Given Supplies

LeetCode match: No direct match; related LC 2115 Find All Possible Recipes from Given Supplies (inferred from overview)

Original: https://prachub.com/coding-questions/solve-matrix-groups-and-recipe-inventory

QUESTION

Overview: This pair of problems evaluates block-wise grid processing and aggregation for minimum-value selection alongside dependency-resolution and resource-allocation reasoning for recipe inventory planning, testing competencies in array manipulation, tie-breaking and constraint handling, graph-based dependency management and cycle detection, and algorithmic complexity analysis; they are commonly asked to gauge an applicant's ability to implement correct, efficient solutions under input-size and edge-case constraints. They belong to the Coding & Algorithms domain with subdomains in data structures (arrays/grids), algorithm design and analysis, and graph theory, and primarily require practical application of algorithmic techniques with attention to time/space complexity rather than purely theoretical proofs.

APPROACH

Part 1: iterate block by block and take the min per group. Part 2: topological sort/DFS over recipe dependencies with an inventory map.

INTERVIEW-DAY PLAN

Clarify first

Two parts. Matrix groups: how the grid is split into blocks (group size) and what is taken per block (the minimum). Recipes: can a recipe be made from supplies, do recipes need other recipes, are amounts counted or only availability (LeetCode 2115 is availability), how are cycles treated.

Brute force to say out loud

Matrix: scan each block directly. Recipes: recursive search from each recipe without memory.

Optimal idea

Matrix: iterate block by block with nested loops and take the minimum of each block, O(mn). Recipes: build a graph from ingredients to recipes with in-degrees and run Kahn's algorithm starting from the supplies; a recipe becomes available when all its ingredients are; recipes on a cycle never become available. If quantities matter, DFS with memoization over recipes tracking inventory, and decide how to share a scarce ingredient between recipes.

Dry-run these cases

A grid not divisible by the block size, a recipe whose ingredient is another recipe, a cycle between two recipes, a missing ingredient, an empty supply list.

Likely follow-ups

Amounts per ingredient (resource allocation), the maximum number of each recipe, or the cheapest set of recipes.

Pitfalls

Treating availability as quantities. Not handling a cycle.

HOW TO THINK ABOUT IT

Blocks: visit each cell once and update the minimum of its block, indexed by (row // k, col // k). Recipes: this is a dependency graph; a recipe is makeable when all its ingredients are supplies or makeable recipes. Count unmet ingredients per recipe and process recipes as they become available (Kahn's algorithm); recipes on a cycle never become available.

Basis: Two parts (my copy has only the overview, so the rules are assumed): take the minimum of every k x k block of a matrix, and decide which recipes can be made from supplies where recipes may need other recipes (LeetCode 2115, Find All Possible Recipes).

BRUTE FORCE - PSEUDO-CODE: BLOCKS: COLLECT EACH BLOCK'S VALUES AND CALL MIN

```
for every k x k block: collect its cells into a list; the
answer for the block is the
    minimum
```

Time: O(m * n) | Space: O(m * n)

Why it is slow

Copies every block into a list. One pass that updates the minimum of cell (r // k, c // k) needs no copies.

BRUTE FORCE - PSEUDO-CODE: RECIPES: KEEP RE-SCANNING UNTIL NO RECIPE CHANGES

```
available = the supplies; repeat: for every recipe whose i
ngredients are all
    available: add the recipe to available
stop when a full pass adds nothing
```

Time: O(R^2 * I) | Space: O(R)

Why it is slow

Many passes over all recipes, O(R^2 * I). Counting each recipe's missing ingredients and processing items as they become available (Kahn's algorithm) is O(R + I).

OPTIMAL SOLUTION - BLOCKS: ONE PASS UPDATING A MINIMUM PER BLOCK

```
def block_mins(matrix, k):
    m, n = len(matrix), len(matrix[0])
    out = [[float("inf")] * ((n + k - 1) // k) for _ in ra
nge((m + k - 1) // k)]
    for r in range(m):
        for c in range(n):
            out[r // k][c // k] = min(out[r // k][c // k],
 matrix[r][c])
    return out
```

Time: O(m * n) | Space: O(blocks)

Why

Each cell updates the minimum of block (r // k, c // k). No block list is built; the last blocks may be smaller when k does not divide the size.

OPTIMAL SOLUTION - RECIPES: KAHN'S ALGORITHM ON THE INGREDIENT GRAPH

```
def findAllRecipes(recipes, ingredients, supplies):
    need, waiting = {}, defaultdict(list)
    for r, ing in zip(recipes, ingredients):
        need[r] = len(set(ing))
        for x in set(ing):
            waiting[x].append(r)
    queue, made = deque(supplies), []
    while queue:
        item = queue.popleft()
        for r in waiting[item]:
            need[r] -= 1
            if need[r] == 0:
                made.append(r)
                queue.append(r)
    return sorted(made)
```

Time: O(R + I) | Space: O(R + I)

Why

For each recipe count the ingredients it still lacks (those not in supplies) and record, for each ingredient, the recipes that need it. Start from the supplies; when an item becomes available decrease the counts of the recipes waiting for it, and queue a recipe when its count hits zero. Recipes on a cycle never reach zero.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

```
def recipes_with_quantities(recipes, needs, stock):
    """Follow-up: ingredients have amounts; make each reci
pe at most once, in the given priority order, if the stock
 allows (recipes can use the output of earlier ones)."""
    stock, made = dict(stock), []
    progress = True
    while progress:
        progress = False
        for r, need in zip(recipes, needs):
            if r not in made and all(stock.get(x, 0) >= q
for x, q in need.items()):
                for x, q in need.items():
                    stock[x] -= q
                stock[r] = stock.get(r, 0) + 1
                made.append(r)
                progress = True
    return made
```

Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours

over just those recipes tells the two cases apart.

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[int]:
```

Sample call

```
solution([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]])
```

Test cases

Input	Expected
`([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]],)`	`(0, 3, 3)`
`([[-1,5,6,-1,7,8],[9,10,11,12,13,14],[15,16,17,18,19,20],[21,22,23,24,25,26],[27,28,29,30,31,32],[33,34,35,36,37,38]],)`	`(-1, 0, 0)`

MEDIUM | Exponent | 70 of 196

Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 129 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1102 Path With Maximum Minimum Value, LeetCode #1730 Shortest Path to Get Food

LeetCode match: No direct match; related LC 1102 Path With Maximum Minimum Value, LC 1730 (inferred from short statement)

Original: https://www.aced.io/questions/5244/safest-path-maximize-minimum-distance-matrix

QUESTION

Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

APPROACH

Multi-source BFS from the cat to get each cell's distance from the cat, then maximize the minimum along a path from rat to bread: Dijkstra with a max-heap keyed on min distance (or binary search the distance + BFS).

INTERVIEW-DAY PLAN

Clarify first

Ask what distance means (steps through open cells), whether the rat can move four-way, whether the cat is fixed, whether the path must avoid the cat cell itself, and what to return when the bread is unreachable.

Brute force to say out loud

DFS over all simple paths and keep the best minimum distance. Exponential.

Optimal idea

Run a multi-source BFS from every cat cell to get each cell's distance from the cat. Then the answer is the path from the rat to the bread that maximizes the minimum cat distance along it (a bottleneck path): Dijkstra with a max-heap keyed on the best minimum so far, or binary search the threshold and check reachability with BFS. O(mn log mn).

Dry-run these cases

The rat adjacent to the bread, no path (obstacles), the cat next to the rat, several cats, a grid where the longest path is safer than the shortest.

Likely follow-ups

Tie-break by the shortest path, or a moving cat.

Pitfalls

Using the sum of distances instead of the minimum. Letting the cat walk through obstacles.

HOW TO THINK ABOUT IT

Two separate ideas. First, how far every cell is from the cat: that is one BFS from the cat (all cats at once). Second, choose the path whose worst cell is as far as possible: this is a 'bottleneck' (maximin) path. A max-heap search that always extends the path with the best bottleneck so far (Dijkstra with min instead of sum) solves it; binary search on the threshold plus a reachability check also works.

Basis: Written. Assumed rules (confirm them): 'R' rat, 'B' bread, 'C' cat (one or more), '#' obstacle, '.' open; 4-directional moves; distance is steps through non-obstacle cells; maximise the minimum cat distance over the cells of the rat's path (endpoints included); -1 if no path.

BRUTE FORCE - PSEUDO-CODE

```
dist_cat[cell] = distance from the nearest cat (BFS)
depth-first search over all simple paths from the rat to the bread; for each path take
    the minimum dist_cat along it
answer = the largest such minimum
```

Time: exponential | Space: O(path length)

Why it is slow

The number of simple paths grows exponentially with the grid. A max-heap search that extends the path with the best bottleneck so far (or binary search on the threshold) is O(m * n log(m * n)).

OPTIMAL SOLUTION - MULTI-SOURCE BFS FROM THE CAT, THEN A MAX-HEAP BOTTLENECK SEARCH

```
def safest_path(grid):
    R, C = len(grid), len(grid[0])
    cat = _cat_dist(grid)
    start, goal = _find(grid, "R"), _find(grid, "B")
    best = {start: cat.get(start, 10**9)}
    heap = [(-best[start], start)]
    while heap:
        neg, pos = heapq.heappop(heap)
        if -neg < best[pos]:
            continue
        if pos == goal:
            return -neg
        r, c = pos
```

```
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc
] != "#":
                value = min(-neg, cat.get((nr, nc), 10**9)
)
                if value > best.get((nr, nc), -1):
                    best[(nr, nc)] = value
                    heapq.heappush(heap, (-value, (nr, nc)
))
    return -1
```

Time: O(m * n log(m * n)) | Space: O(m * n)

Why

BFS from every cat gives the distance of each cell. Then run Dijkstra from the rat where the 'cost' of a path is the minimum cat distance along it and we maximise it: pop the cell with the best bottleneck so far, and extend to a neighbour with min(bottleneck, neighbour distance). The first time the bread is popped its bottleneck is optimal.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Use binary search instead of a heap.

Binary search the safety threshold d and test with BFS whether the rat can reach the bread using only cells at least d from the cat. The test is monotone in d. O(m * n log(m + n)).

```
def _cat_dist(grid):
    R, C = len(grid), len(grid[0])
    dist = {(r, c): 0 for r in range(R) for c in range(C)
if grid[r][c] == "C"}
    queue = deque(dist)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1),
 (r, c - 1)):
            if (
                0 <= nr < R
                and 0 <= nc < C
                and grid[nr][nc] != "#"
                and (nr, nc) not in dist
            ):
                dist[(nr, nc)] = dist[(r, c)] + 1
                queue.append((nr, nc))
    return dist


def _find(grid, ch):
    return next(
        (r, c)
        for r in range(len(grid))
        for c in range(len(grid[0]))
        if grid[r][c] == ch
    )


def safest_path_threshold(grid):
    """Alternative: binary search the threshold d and test
 reachability using only cells at distance >= d."""
    R, C = len(grid), len(grid[0])
    cat = _cat_dist(grid)
    start, goal = _find(grid, "R"), _find(grid, "B")

    def reachable(d):
        if cat.get(start, 10**9) < d:
            return False
        seen, stack = {start}, [start]
        while stack:
            r, c = stack.pop()
            if (r, c) == goal:
                return True
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c +
1), (r, c - 1)):
                if (
                    0 <= nr < R
                    and 0 <= nc < C
                    and grid[nr][nc] != "#"
                    and (nr, nc) not in seen
                    and cat.get((nr, nc), 10**9) >= d
                ):
                    seen.add((nr, nc))
                    stack.append((nr, nc))
        return False

    values = sorted(
        {
            cat.get((r, c), 10**9)
            for r in range(R)
            for c in range(C)
            if grid[r][c] != "#"
        }
    )  # candidate safety values
    lo, hi, ans = 0, len(values) - 1, -1
    while lo <= hi:
        mid = (lo + hi) // 2
        if reachable(values[mid]):
            ans, lo = values[mid], mid + 1
        else:
            hi = mid - 1
    return ans
```

Among the safest paths, return the shortest one.

First find the best safety value d*, then run an ordinary BFS restricted to cells at distance at least d*. The result is the shortest path with maximum safety.

The cat moves one step toward the rat each turn.

Then distances depend on time, and the problem becomes a game or a time-expanded graph search; ask for the cat's movement rules before designing anything.

MEDIUM | Exponent | 71 of 196

Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

STUDY: TYPE | Google frequency on LeetCode: all time 37.9, last 30 days 33 | book priority 54 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #787 Cheapest Flights Within K Stops

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 787 (inferred from short statement)

Original:
https://www.aced.io/questions/6331/find-earliest-train-sequence-route

QUESTION

Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

APPROACH

Dijkstra over (city, arrival time): from a city at time t, take a train whose start time >= t and relax the arrival time. Reconstruct the train sequence with parent pointers.

INTERVIEW-DAY PLAN

Clarify first

Ask whether you may wait at stations, whether transfers need a minimum time, whether the start time is given, and what to return (the sequence of trains and the arrival time).

Brute force to say out loud

DFS over all train sequences and keep the earliest arrival. Exponential.

Optimal idea

Dijkstra on arrival times. Keep earliest[city] and a min-heap of (time, city). From a city at time t, relax every train departing at or after t, giving arrival at the train's end time. Waiting is free, so earliest arrival is always better. Store the train used to reach each city and reconstruct the sequence by following parents. O((V + E) log V).

Dry-run these cases

No route, a train that departs just before you arrive (cannot be taken), two trains to the same city with different arrival times, the source equals the destination.

Likely follow-ups

Fewest transfers, cheapest fare, a minimum layover, or time zone changes.

Pitfalls

Using the train's duration as an edge weight and ignoring departure times. Allowing a train that has already left.

HOW TO THINK ABOUT IT

Waiting is free, so arriving earlier at a city is never worse than arriving later: that is exactly the property Dijkstra needs. Keep the earliest known arrival time per city in a heap, and from a city at time t relax only the trains that depart at or after t. Remember which train gave each improvement to rebuild the sequence.

Basis: Written; earliest-arrival path in a time-dependent graph (Dijkstra on arrival times, compare LeetCode 743 and 787). Assumed rules (confirm them): a train (from, to, depart, arrive) can be boarded if you are at its origin at or before its departure; waiting is free; no minimum transfer time.

BRUTE FORCE - PSEUDO-CODE

```
go(city, time): if city is the destination: record time
for every train from city that departs at or after time: g
o(train's destination,
    train's arrival)
answer = the earliest recorded arrival
```

Time: exponential | Space: O(trains)

Why it is slow

The number of train sequences can be exponential. Because waiting is free, an earlier arrival is never worse, so Dijkstra on earliest arrival times is O((T + C) log C).

OPTIMAL SOLUTION - DIJKSTRA ON EARLIEST ARRIVAL TIMES, WITH THE TRAIN SEQUENCE

```
def earliest_arrival(trains, src, dst, start=0):
    by_city = defaultdict(list)
    for tr in trains:
        by_city[tr[0]].append(tr)
    best, via, heap = {src: start}, {}, [(start, src)]
    while heap:
        t, city = heapq.heappop(heap)
        if t > best[city]:
            continue
        if city == dst:
            seq = []
            while city != src:
                tr, prev = via[city]
                seq.append(tr)
                city = prev
            return t, seq[::-1]
        for tr in by_city[city]:
            f, to, dep, arr = tr
            if dep >= t and arr < best.get(to, float("inf"
)):
                best[to], via[to] = arr, (tr, city)
                heapq.heappush(heap, (arr, to))
    return None
```

Time: O((T + C) log C) with trains sorted by departure per city | Space: O(T + C)

Why

best[city] is the earliest time you can be there. Pop the city with the smallest time, and for every train from it that departs at or after that time, improve best[to] to the train's arrival time and remember the train. Because waiting is free, a later arrival never beats an earlier one, so the first pop of the destination is optimal.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

A minimum layover is required when changing trains.

Replace the boarding test dep >= t with dep >= t + layover for every city except the origin. The earliest-arrival property still holds (a later arrival never helps because the layover is the same), so Dijkstra remains valid.

```
def earliest_arrival_with_layover(trains, src, dst, start=
0, layover=0):
    by_city = defaultdict(list)
    for tr in trains:
        by_city[tr[0]].append(tr)
    best, heap = {src: start}, [(start, src)]
    while heap:
        t, city = heapq.heappop(heap)
```

```
        if t > best[city]:
            continue
        if city == dst:
            return t
        for f, to, dep, arr in by_city[city]:
            ready = (
                t if city == src else t + layover
            )  # a layover is needed at every transfer, no
t at the origin
            if dep >= ready and arr < best.get(to, float("
inf")):
                best[to] = arr
                heapq.heappush(heap, (arr, to))
    return None
```

Fewest transfers instead of earliest arrival.

Run BFS over (city) counting trains taken, but track the earliest arrival for each count (a state is (city, number of trains)). Or Dijkstra with the pair (transfers, arrival time) compared lexicographically.

The schedule repeats daily.

Add the day to the time (departure + 24h * k) or treat times modulo 24h and carry the day count. The same Dijkstra works on absolute times.

MEDIUM | Exponent | 72 of 196

Find the shortest path in a graph.

STUDY: TYPE | Google frequency on LeetCode: all time 37.9, last 30 days 33 | book priority 55 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #1091 Shortest Path in Binary Matrix

LeetCode match: No direct match; related LC 743 Network Delay Time, LC 1091 (inferred from short statement)

Original: https://www.aced.io/questions/2094/find-shortest-path-graph

QUESTION

Find the shortest path in a graph.

APPROACH

Unweighted: BFS; weighted non-negative: Dijkstra with a heap; negative edges: Bellman-Ford. Clarify the graph type first.

INTERVIEW-DAY PLAN

Clarify first

Clarify the graph first: directed or undirected, weighted, whether weights can be negative, the size, and whether you need the path itself or only its length.

Brute force to say out loud

Enumerate all paths with DFS and take the shortest. Exponential.

Optimal idea

Unweighted: BFS with a parent map, O(V + E). Non-negative weights: Dijkstra with a min-heap, O((V + E) log V). Negative weights without a negative cycle: Bellman-Ford, O(V * E). All pairs on a small graph: Floyd-Warshall. A weights of 0 and 1 graph: 0-1 BFS. Reconstruct the path from the parent pointers.

Dry-run these cases

Source equals destination, no path, a graph with a cycle, parallel edges, a negative edge (shows why Dijkstra fails).

Likely follow-ups

A* with a heuristic, bidirectional BFS, or the k shortest paths.

Pitfalls

Using BFS on a weighted graph. Not stating the assumption about edge weights.

HOW TO THINK ABOUT IT

Ask the graph questions first: directed or undirected, weighted or not, can weights be negative, and do you want the length or the path. That choice picks the algorithm: unweighted means breadth-first search, non-negative weights mean Dijkstra with a heap, negative weights mean Bellman-Ford. Remember parent pointers if the path itself is needed.

Basis: The shortest-path family: BFS (unweighted), Dijkstra (non-negative weights, LeetCode 743), Bellman-Ford (negative weights).

BRUTE FORCE - PSEUDO-CODE

```
dist[source] = 0, others infinity
repeat V - 1 times: for every edge (u, v, w): if dist[u] +
 w < dist[v]: dist[v] =
    dist[u] + w
return dist[target]
```

Time: O(V * E) | Space: O(V)

Why it is slow

Each round relaxes every edge: O(V * E). It is the right tool for negative weights; for non-negative weights Dijkstra's heap is O((V + E) log V), and for unweighted graphs BFS is O(V + E).

OPTIMAL SOLUTION - DIJKSTRA WITH A HEAP (NON-NEGATIVE WEIGHTS), RETURNING THE PATH

```
def shortest_path(n, edges, s, t):
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    dist, parent, heap = {s: 0}, {s: None}, [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if u == t:
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return d, path[::-1]
        for v, w in graph[u]:
            if d + w < dist.get(v, float("inf")):
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    return None
```

Time: O((V + E) log V) | Space: O(V)

Why

Keep the best known distance to each node and a min-heap of (distance, node). Pop the closest unsettled node, relax its edges, and remember the parent of every improvement. The first time the

target is popped its distance is final; rebuild the path by following parents. It needs non-negative weights.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The graph is unweighted.

BFS from the source with a parent map gives a shortest path in O(V + E); Dijkstra would be correct but needlessly slow. Stop as soon as you pop the target.

```
def shortest_path_unweighted(n, edges, s, t):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        if u == t:
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return path[::-1]
        for v in graph[u]:
            if v not in parent:
                parent[v] = u
                queue.append(v)
    return None
```

Weights can be negative.

Dijkstra can give wrong answers. Use Bellman-Ford (O(V * E)); one more pass that still improves a distance proves a negative cycle reachable from the source, where shortest paths are undefined.

```
def shortest_bellman_ford(n, edges, s, t):
    INF = float("inf")
    dist = [INF] * n
    dist[s] = 0
    for _ in range(max(0, n - 1)):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v], changed = dist[u] + w, True
        if not changed:
            break
    return None if dist[t] == INF else dist[t]
```

Weights are only 0 or 1, or the graph is huge and you know the target.

0-1 BFS uses a deque in O(V + E). With a good distance estimate use A* (Dijkstra ordered by distance plus a heuristic that never overestimates); bidirectional search roughly squares the work saved on large graphs.

MEDIUM | 1Point3Acres | phone-screen | 73 of 196

Minimum-Cost Tree Disconnection

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 176 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

QUESTION

Examples:

Negative-weight follow-up:

```
A --(-1)--> B --(-1)--> C
Minimum total cut cost: -2
```

Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

APPROACH

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

INTERVIEW-DAY PLAN

Clarify first

Ask what must be disconnected (root from every leaf), whether the tree is rooted and directed, whether weights can be negative, and what to return for an empty or single-node tree.

Brute force to say out loud

Try every subset of edges and keep those that separate the root from all leaves. Exponential.

Optimal idea

Define best(v) as the cheapest set of cuts in v's subtree that disconnects v from every leaf below it. For each child c with edge weight w, either cut the edge, paying w, or keep it and require best(c) (not allowed if c is a leaf). Sum over children. With non-negative weights that is the answer. With negative weights, once you cut an edge the whole subtree below becomes free, and you may still cut any negative edge there, so the cut option costs w plus the sum of min(0, edge) over that subtree.

Dry-run these cases

A single edge, the negative chain A to B to C (answer -2), a leaf at different depths, a star.

Likely follow-ups

Negative weights, many roots, or recursion depth for a chain tree.

Pitfalls

Treating every negative edge as a mandatory cut. Recursing deeply on a chain.

HOW TO THINK ABOUT IT

On a tree, every decision about a subtree depends only on the edge above it, so define a function of the node that returns the best cost inside its subtree. For each child you either cut the edge above it (pay its weight) or keep it and make the child itself disconnected from the leaves below. With negative weights, a cut edge frees the whole subtree, so you may cut every negative edge in it for free gain.

Basis: Written. Assumed setup (confirm it): a rooted tree with edge weights (cuts cost the weight, weights may be negative); cut edges so the root is disconnected from every leaf; a leaf is a non-root node with no children; with no leaves the cost is 0.

BRUTE FORCE - PSEUDO-CODE

```
for every subset of edges: remove it
if the root can no longer reach any leaf: cost = sum of th
e removed weights; keep the
    smallest
```

Time: O(2^E * n) | Space: O(n)

Why it is slow

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

OPTIMAL SOLUTION - TREE DP: BEST(V) = CHEAPEST CUTS INSIDE V'S SUBTREE THAT DISCONNECT V FROM ITS LEAVES

```python
def disconnect_cost(n, edges):
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    INF = float("inf")

    def solve(v):
        free = 0
        best = 0
        for c, w in kids[v]:
            c_best, c_free = solve(c)
            free += min(0, w) + c_free
            keep = INF if not kids[c] else c_best
            best += min(w + c_free, keep)
        return best, free

    return solve(0)[0] if kids[0] or n > 1 else 0
```

Time: O(n) | Space: O(h)

Why

For a child c with edge weight w there are two options: cut the edge, paying w plus whatever extra negative-weight edges you can still cut below c (free(c) = sum of min(0, weight) over its subtree), or keep the edge and require best(c), which is impossible when c is a leaf. best(v) is the sum of the cheaper option over its children. With non-negative weights free(c) is 0 and this is the classic recurrence.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

```python
def disconnect_cost_nonneg(n, edges):
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))

    def solve(v):
        return sum(w if not kids[c] else min(w, solve(c))
for c, w in kids[v])

    return solve(0)
```

The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

NOTES FROM CANDIDATES

Clarify the required result for an empty tree and a single-node tree. No final return convention is specified for these cases.

Include recursion-stack memory in space analysis. The tree can degenerate into a chain; balanced-tree depth is not a worst-case bound.

Coding took place directly in Google Docs with no supplied framework; the candidate also had to define the tree node structure. The interviewer asked the candidate to think aloud while developing the approach.

For nonnegative weights, use postorder DP. Let F(v) be the minimum cost to disconnect node v from every original leaf below it. An original leaf has F(v) = infinity; an internal node has F(v) = sum(min(weight(v,c), F(c))) over its children. Each branch must be blocked independently. Use a safe infinity representation rather than adding to a fixed-width maximum integer.

For negative weights, cutting an ancestor edge does not remove the opportunity to cut negative edges below it. Track A(v), the minimum optional cut cost when the subtree is already disconnected: A(v) = sum(min(0, weight(v,c)) + A(c)), with A(leaf) = 0. Then use F(v) = sum(min(weight(v,c) + A(c), F(c))), retaining F(leaf) = infinity. This includes beneficial redundant negative cuts without double counting edges.

Both recurrences process each edge once: O(n) time and O(h) recursive auxiliary space, where h can equal n. A single original root-leaf cannot be disconnected from itself by cutting edges; agree on its result convention before coding.

PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth.
Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 74 of 196

Compute File System Total Size

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 183 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: written and tested here for this exact question
Frequency: Low | Last asked: 2026-09-02 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

Examples:

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

Brute force to say out loud

Recursive DFS adding file sizes. Simple and usually the expected first answer.

Optimal idea

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

Dry-run these cases

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

Likely follow-ups

Cache totals, handle updates, or find the largest directory.

Pitfalls

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE - PSEUDO-CODE

```
size(node): if it is a file: return its size
return the sum of size(child) over the directory's childre
n
```

Time: O(n) | Space: O(depth) call stack

Why it is slow

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION - ITERATIVE TRAVERSAL WITH A VISITED SET

```python
class File:
    def __init__(self, size):
        self.size = size


class Dir:
    def __init__(self, *children):
        self.children = list(children)


def total_size(root):
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        if id(node) in seen:
            continue
        seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total
```

Time: O(n) | Space: O(n)

Why

An explicit stack removes the recursion limit, and a set of visited node ids makes each node count once, so shared directories are not double counted and cycles terminate.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

```python
class File:
    def __init__(self, size):
        self.size = size
```

```python
class Dir:
    def __init__(self, *children):
        self.children = list(children)


class CachedFS:
    """Directory totals cached; changing a file invalidate
s only its ancestors."""

    def __init__(self, root):
        self.parent, self.cache, stack = {id(root): None},
 {}, [root]
        while stack:
            n = stack.pop()
            for c in getattr(n, "children", []):
                self.parent[id(c)] = n
                stack.append(c)
        self.root = root

    def total(self, node=None):
        node = node or self.root
        if id(node) in self.cache:
            return self.cache[id(node)]
        result = (
            node.size
            if isinstance(node, File)
            else sum(self.total(c) for c in node.children)
        )
        self.cache[id(node)] = result
        return result

    def resize(self, file, size):
        file.size = size
        cur = file
        while cur is not None:
            self.cache.pop(id(cur), None)
            cur = self.parent[id(cur)]
```

Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. Cost: one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 75 of 196

Determine Reporting Relationships

STUDY: TYPE | Google frequency on LeetCode: all time 37.4, last 30 days 33 | book priority 56 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: No direct match; related LC 721 Accounts Merge (union-find)

Original: https://prachub.com/coding-questions/determine-reporting-relationships

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints
0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles
Examples
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
Expected Output: [True]
Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4.
Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

APPROACH

Manager edges as a directed graph/parent map; peers via union-find or shared-manager sets; queries answer reachability up the manager chain. Keep both structures in sync.

INTERVIEW-DAY PLAN

Clarify first

Ask whether is_manager means direct manager or any manager up the chain (the example implies the chain), whether peers can be added before their manager is known, and whether input is always consistent (the statement says yes).

Brute force to say out loud

Store all relations and recompute reachability with a graph search on each query. O(V + E) per query.

Optimal idea

Keep a Union-Find over employees for peer groups and attach one manager to each group. add_manager(a, b) sets the manager of b's group to a. add_peer(a, b) unions the two groups and, if both had managers, keeps the (necessarily identical) manager.
is_manager(a, b): look up the manager of b's group, return true if it is a, otherwise continue from that manager's group up the chain. Each query walks at most the depth of the hierarchy.

Dry-run these cases

add_manager(m, x) then add_peer(x, y) gives is_manager(m, y) true; add_peer before add_manager; a manager's manager (indirect); an employee with no manager; a query on unknown ids.

Likely follow-ups

Direct manager only, the closest common manager, or removing relationships.

Pitfalls

Forgetting to merge managers when two groups unite. Treating peers as managers of each other.

HOW TO THINK ABOUT IT

Peers form groups (a peer relation is transitive: same manager as a person who has the same manager as another). A group has at most one direct manager, so store the manager on the group. To answer is_manager(a, b) go from b's group to its manager, then that manager's group to its manager, and so on, looking for a. Union-find makes the groups cheap to merge.

Basis: Written (compare LeetCode 721, Accounts Merge, for the union-find idea). Operations: add_manager(a, b) (a is b's direct manager), add_peer(a, b) (same direct manager), is_manager(a, b) (a is anywhere above b). Inferences must follow from earlier calls.

BRUTE FORCE - PSEUDO-CODE

```
keep every add_manager and add_peer fact
is_manager(a, b): merge peers into groups, attach managers
 to groups until nothing
    changes, then walk up from b looking for a
```

Time: O(F * employees) per query | Space: O(F)

Why it is slow

Each query redoes the whole inference over all facts. Union-Find for peer groups with a manager stored on each group keeps every update near-constant and a query O(depth).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(operations):
    uf_parent = {}
    uf_size = {}
    members = {}
    group_manager = {}
    direct_parent = {}
    answers = []

    def make(x):
        if x not in uf_parent:
            uf_parent[x] = x
            uf_size[x] = 1
            members[x] = [x]
            group_manager[x] = None
            direct_parent[x] = None

    def find(x):
        while uf_parent[x] != x:
            uf_parent[x] = uf_parent[uf_parent[x]]
            x = uf_parent[x]
        return x

    for op, a, b in operations:
        make(a)
        make(b)

        if op == "add_manager":
            root = find(b)
            if group_manager[root] is None:
                group_manager[root] = a
                for emp in members[root]:
                    direct_parent[emp] = a
            # If the group already has the same manager, t
he input is consistent
            # and nothing new needs to be done.
```

```
        elif op == "add_peer":
            ra = find(a)
```

```
            rb = find(b)
            if ra == rb:
                continue

            ma = group_manager[ra]
            mb = group_manager[rb]

            # If one group already knows its manager and t
he other does not,
            # the unmanaged group's members learn that sam
e direct manager now.
            if ma is not None and mb is None:
                for emp in members[rb]:
                    direct_parent[emp] = ma
            elif ma is None and mb is not None:
                for emp in members[ra]:
                    direct_parent[emp] = mb

            # Union by size to keep DSU efficient.
            if uf_size[ra] < uf_size[rb]:
                ra, rb = rb, ra
                ma, mb = mb, ma

            uf_parent[rb] = ra
            uf_size[ra] += uf_size[rb]
            members[ra].extend(members[rb])
            group_manager[ra] = ma if ma is not None else
mb

            del members[rb]
            del group_manager[rb]
            del uf_size[rb]

        elif op == "is_manager":
            cur = b
            found = False
```

```
            while direct_parent[cur] is not None:
                cur = direct_parent[cur]
                if cur == a:
                    found = True
                    break
            answers.append(found)

        else:
            raise ValueError("Unknown operation")

    return answers
```

Time: O(m * alpha(k) + k * log k + q * h), where k is the number of distinct employees, q is the number of queries, and h is the maximum current management-chain length walked during a query | Space: O(k)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

```
class Org:
    def __init__(self):
        self.parent, self.boss = {}, {}

    def _find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self._find(a), self._find(b)
        if ra != rb:
            self.parent[rb] = ra
            if rb in self.boss and ra not in self.boss:
                self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        self._find(a)
        self.boss[self._find(b)] = a

    def is_manager(self, a, b):
        seen, cur = set(), b
        while cur not in seen:
            seen.add(cur)
            root = self._find(cur)
            if root not in self.boss:
                return False
            cur = self.boss[root]
            if cur == a:
                return True
        return False


def closest_common_manager(org, a, b):
    chain, cur = [], a
    seen = set()
    while cur not in seen:
        seen.add(cur)
        chain.append(cur)
        root = org._find(cur)
        if root not in org.boss:
            break
        cur = org.boss[root]
    anc, cur, seen = set(chain), b, set()
    while cur not in seen:
        if cur in anc:
            return cur
        seen.add(cur)
        root = org._find(cur)
        if root not in org.boss:
            return None
        cur = org.boss[root]
    return None
```

Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[tuple[str, int, int]]) -> li
st[bool]:
SAMPLE INPUT
([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager"
, 1, 3)])
```

Test cases

Input	Expected
`[["add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])`	`[True]`
`[["add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])`	`[False, True, True, False]`

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 76 of 196

Employee Hierarchy Add / Delete

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 195 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: written and tested here for this exact question
Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id. Follow-ups cover deleting managers, deleting the CEO, cycle handling, and choosing a replacement manager by seniority.SWEtreehashmapobject-designmediumFrequencySingle reportLast asked2026-06-10Stageonsite-coding

APPROACH

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

INTERVIEW-DAY PLAN

Clarify first

Ask what happens when a manager is deleted (promote a report, reassign to the grandparent, or reject), what happens when the CEO is deleted, whether adding can create a cycle, and how seniority is defined.

Brute force to say out loud

Store parent pointers only and search the whole tree to find children on every delete. O(n).

Optimal idea

Keep id to node, and for each node a parent pointer and a children set. Add: validate the manager exists and the new id is free. Delete: take the children of the removed node and attach them to a replacement (the deleted node's parent, or a chosen successor), update pointers, and remove it from its parent's children. CEO needs a stated policy. Compare candidates by (level descending, insertion order ascending) for the seniority rule.

Dry-run these cases

Delete a leaf, a middle manager, the CEO, an unknown id, adding with a missing manager.

Likely follow-ups

Cycle detection by walking up parent pointers, bulk deletes, or a history of changes.

Pitfalls

Leaving dangling children references. Not choosing a policy for the root.

HOW TO THINK ABOUT IT

A hierarchy is a tree you update, so store links in both directions (parent per node, children per node). Then every operation touches only the neighbours of one node instead of searching the tree. For each delete case (leaf, middle, root) say the policy first, because the data structure is easy once the rule is fixed.

Basis: Written. Assumed policies (confirm them): deleting a manager hands their reports to that manager's own manager; deleting the CEO is allowed only if exactly one report exists (that report becomes CEO), otherwise it is rejected; ids are unique.

BRUTE FORCE - PSEUDO-CODE

```
manager = dict from employee to manager
delete(e): find all reports of e by scanning every entry;
give them e's manager;
    remove e
```

Time: O(n) per delete | Space: O(n)

Why it is slow

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION - PARENT MAP PLUS CHILDREN SETS

```python
class Org:
    def __init__(self, ceo):
        self.manager = {ceo: None}
        self.reports = {ceo: set()}

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        return True

    def delete(self, emp):
        if emp not in self.manager:
            return False
        mgr, kids = self.manager[emp], self.reports[emp]
        if mgr is None and len(kids) != 1:
            return False
        for r in kids:
            self.manager[r] = mgr
            if mgr is not None:
                self.reports[mgr].add(r)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True
```

Time: O(r) per delete where r is the number of reports | Space: O(n)

Why

Keep manager[emp] and reports[emp] (a set). Add inserts into both. Delete reattaches each report to the deleted node's manager and removes the node from its manager's set. Cost depends only on the number of direct reports.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

```
class Org:
    def __init__(self, ceo):
        self.manager = {ceo: None}
        self.reports = {ceo: set()}

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        return True

    def delete(self, emp):
        if emp not in self.manager:
            return False
        mgr, kids = self.manager[emp], self.reports[emp]
        if mgr is None and len(kids) != 1:
            return False
        for r in kids:
            self.manager[r] = mgr
            if mgr is not None:
                self.reports[mgr].add(r)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True


def move_employee(org, emp, new_mgr):
    """Reject a move that would create a cycle: new_mgr mu
st not be inside emp's subtree."""
    cur = new_mgr
    while cur is not None:
        if cur == emp:
            return False
        cur = org.manager[cur]
    old = org.manager[emp]
    if old is not None:
        org.reports[old].discard(emp)
    org.manager[emp] = new_mgr
    org.reports[new_mgr].add(emp)
    return True
```

When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

NOTES FROM CANDIDATES

Use maps from id to node and id to children; keep parent pointers so deletes do not require a full traversal.
CEO deletion needs an explicit policy: reject, promote a child, or create a new root. State the policy and keep it consistent.
For the seniority follow-up, maintain insertion order on nodes and compare (level desc, insertion_order asc).

PREPARATION

Implement tree mutation operations with parent pointers.
Practice deleting an internal node while reconnecting children.
Add cycle checks with DFS or parent-chain walking before accepting a new edge.

Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns 43 questions

MEDIUM | PracHub | Technical Screen | 77 of 196

Sliding-Window Mean After Removing the Largest K Values

STUDY: TALK | Google frequency on LeetCode: all time 29.8, last 30 days 0 | book priority 114 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer

mean such as 5 is **`5/1`**.

- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the wi
ndow into the removed largest values and the retained valu
es. Keep their sizes correct and ensure no retained value
is larger than a removed value.
```

APPROACH

Keep the window in two structures (top-k multiset and the rest) with running sums, using two heaps with lazy deletion or a balanced BST; mean = rest_sum / (w - k). O(n log w).

HOW TO THINK ABOUT IT

The mean needs two sums: the whole window and the sum of its K largest values. Keep the window in a sorted container; the K largest are its last K elements. When one value enters or leaves, only one element crosses the boundary between 'kept' and 'removed', so the sum of the K largest can be updated in O(1) after an O(log) search.

Basis: LeetCode 480 (Sliding Window Median) style order statistics; the mean of the window after dropping exactly K occurrences of the largest values, returned as reduced fractions.

BRUTE FORCE - PSEUDO-CODE

```
for every window: sort a copy, drop the k largest, sum the
 rest and write the mean as
    a reduced fraction
```

Time: O(n * w log w) | Space: O(w)

Why it is slow

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def trimmed_window_means(nums, window, k):
    import heapq
    import math

    low, high, side = [], [], {}
    retained, low_size, total = window - k, 0, 0

    def prune(heap, group):
        while heap and side.get(heap[0][1]) != group:
            heapq.heappop(heap)

    def rebalance():
        nonlocal low_size, total
        prune(low, 0)
        prune(high, 1)
        target = min(retained, len(side))
        while low_size > target:
            negative, index = heapq.heappop(low)
            value = -negative
            side[index] = 1
            low_size -= 1
            total -= value
            heapq.heappush(high, (value, index))
            prune(low, 0)
        while low_size < target:
            value, index = heapq.heappop(high)
            side[index] = 0
            low_size += 1
            total += value
            heapq.heappush(low, (-value, index))
            prune(high, 1)
        if len(low) + len(high) > 4 * window:
            low[:] = [entry for entry in low if side.get(e
ntry[1]) == 0]
            high[:] = [entry for entry in high if side.get
(entry[1]) == 1]
            heapq.heapify(low)
            heapq.heapify(high)

    def add(index, value):
        nonlocal low_size, total
        prune(low, 0)
        if not low or value <= -low[0][0]:
            side[index] = 0
            low_size += 1
            total += value
            heapq.heappush(low, (-value, index))
        else:
            side[index] = 1
            heapq.heappush(high, (value, index))
        rebalance()

    def remove(index, value):
        nonlocal low_size, total
        if side.pop(index) == 0:
            low_size -= 1
            total -= value
        rebalance()

    result = []
    for index, value in enumerate(nums):
        if index >= window:
            remove(index - window, nums[index - window])
        add(index, value)
        if index >= window - 1:
            divisor = math.gcd(abs(total), retained)
            result.append(str(total // divisor) + "/" + st
r(retained // divisor))
    return result
```

Time: O(n log(window + 1)) amortized plus exact fraction formatting | Space: O(window) maintained state, excluding input and returned output

Explanation

Partition each window into low, the window-k smallest occurrences, and high, the removed k largest occurrences. Maintain low's sum and enforce max(low)<=min(high) whenever both groups are nonempty. Insert relative to low's largest boundary; remove the outgoing occurrence; then move boundary values until low has the required size. During initial fill and the brief removal step its target is min(window-k,current count). These boundary moves preserve ordering, so low is exactly the required retained multiset and its

tracked sum is correct. Equal boundary values are interchangeable; Java counted TreeMaps and C++ multisets can remove any equal occurrence. Python/JavaScript use index-tagged max/min heaps and active-index group membership to distinguish duplicates. Deletions adjust logical counts/sum immediately, and stale heap heads are discarded before boundary access. When physical heaps exceed four times the window, rebuilding from active entries removes hidden stale entries. This prevents monotone streams from growing memory with n: rebuilding requires enough intervening deletions to amortize its O(window) Python heapify or O(window log window) JavaScript reinsertion cost. Each update has O(log window) amortized work, and maintained state is O(window). Reduce sum/(window-k) by gcd(abs(sum),window-k); the denominator stays positive and zero becomes 0/1. Wide sums fit signed 64-bit and exact JavaScript integers. In a true stream, retain the two groups, retained sum, deletion bookkeeping, and a window-sized FIFO to identify the next outgoing value. The callable can read that outgoing value from its existing input; neither setting sorts each new window.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. Cost: unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

```
def trimmed_both_means(nums, window, k):
    s, total, top, bottom, out = [], 0, 0, 0, []

    def insert(x):
        nonlocal total, top, bottom
        L = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        total += x
        if L < k:
            top += x
            bottom += x
        else:
            if p >= L + 1 - k:
                top += x - s[L - k]
            if p < k:
                bottom += x - s[k]

    def remove(x):
        nonlocal total, top, bottom
        L = len(s)
        p = bisect_left(s, x)
        total -= x
        if L <= k:
            top -= x
            bottom -= x
        else:
            if p >= L - k:
                top += s[L - k - 1] - x
            if p < k:
                bottom += s[k] - x
        del s[p]

    for i, x in enumerate(nums):
        insert(x)
        if i >= window:
            remove(nums[i - window])
        if i >= window - 1:
            kept = window - 2 * k
            num = total - top - bottom
            g = math.gcd(abs(num), kept)
            out.append(f"{num // g}/{kept // g}")
    return out
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k: int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
([1, 9, 2, 8], 3, 1)	['3/2', '5/1']
([-1, 1, 3], 2, 0)	['0/1', '2/1']

ALSO REPORTED - 2 MORE REPORTS OF THIS QUESTION

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

MEDIUM | PracHub | Technical Screen | 78 of 196

Find the Largest Monotone Increasing Number

STUDY: TALK | Google frequency on LeetCode: all time 5, last 30 days 0 | book priority 157 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 738 Monotone Increasing Digits (inferred from overview)

Original: https://prachub.com/coding-questions/find-the-largest-monotone-increasing-number

QUESTION

Overview: Find the largest number no greater than n whose decimal digits are monotone increasing. Develop a linear-time greedy correction that revisits earlier decreasing boundaries and fills the remaining suffix optimally without introducing leading zeros.

APPROACH

Scan digits right to left; when digit[i-1] > digit[i], decrement digit[i-1] and mark position i as the point to fill with 9s. O(d).

INTERVIEW-DAY PLAN

Clarify first

Ask whether monotone means non-decreasing (equal digits allowed) or strictly increasing, whether n can be 0, and whether n is given as an integer or a very long string.

Brute force to say out loud

Decrement n until its digits are monotone. Correct but up to O(n * d).

Optimal idea

Scan the digits from right to left. When digit[i - 1] > digit[i], decrease digit[i - 1] by one and remember i as the first position to fill with 9s. After the scan set every digit from that marker to the end to 9. O(d) time.

Dry-run these cases

10 gives 9, 332 gives 299, 1234 stays 1234, 100 gives 99, 0 gives 0, 1000 gives 999.

Likely follow-ups

Strictly increasing digits variant, or return the largest number below n when n itself is already valid.

Pitfalls

Scanning left to right, which misses the cascading decrement. Setting 9s from the wrong index.

HOW TO THINK ABOUT IT

If a digit is smaller than the one before it, the earlier digit has to drop by one and everything after that point can become 9 (as large as possible). Dropping a digit can break the pair before it, so scan from the right: each repair is checked against the digit to its left. That is why one right-to-left pass is enough.

Basis: LeetCode 738, Monotone Increasing Digits (non-decreasing digits).

BRUTE FORCE - PSEUDO-CODE

```
while the digits of n are not non-decreasing: n = n - 1
return n
```

Time: O(n * d) | Space: O(1)

Why it is slow

n can be 10^9, so it may take a billion steps. A right-to-left scan that lowers a digit where order breaks and fills the rest with 9s is O(digits).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def largest_monotone_increasing(n):
    if not isinstance(n, int) or isinstance(n, bool) or n
< 0:
        raise ValueError("n must be nonnegative")
    digits = list(str(n))
    suffix_start = len(digits)
    for index in range(len(digits) - 1, 0, -1):
        if digits[index - 1] > digits[index]:
            digits[index - 1] = str(int(digits[index - 1])
 - 1)
            suffix_start = index
    for index in range(suffix_start, len(digits)):
        digits[index] = "9"
    return int("".join(digits))
```

Time: O(number of decimal digits). | Space: O(number of decimal digits).

Explanation

A right-to-left scan repairs every decrease by decrementing the digit on its left. That decrement can create an earlier decrease, which the continuing backward scan catches. Once the earliest repair is known, filling its suffix with 9 maximizes the result.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The number is given as a string with 100,000 digits.

The scan works directly on the digit list, so keep it as a list of characters and never convert to an integer until the end (or not at all). The algorithm is O(d).

```
def monotoneIncreasingDigits(n):
    d = list(str(n))
    mark = len(d)
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            mark = i
    for i in range(mark, len(d)):
        d[i] = "9"
    return int("".join(d))
```

Return the smallest monotone number that is at least n instead.

Scan left to right for the first digit that is larger than its right neighbour and copy that digit over the whole suffix (332 becomes 333). The prefix before it is already monotone.

```
def smallest_monotone_at_least(n):
    d = list(str(n))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            for j in range(i + 1, len(d)):
```

```
            d[j] = d[i]
        break
return int("".join(d))
```

Digits must be strictly increasing.

Same idea, but the suffix to fill is the largest strictly increasing run (for example ...6789), and you must check that the prefix leaves enough room. Say the extra care and write the check as a brute-force comparison first.

FUNCTION AND TEST CASES

Sample call

```
largest_monotone_increasing(0)
```

Test cases

Input	Expected
(0,)	0
(9,)	9

MEDIUM | PracHub | Technical Screen | 79 of 196

★ TOP 75: rank 51 in the Medium and Hard priority list

★ Filter Repeated Robot Status Messages

STUDY: TYPE | Google frequency on LeetCode: all time 51.2, last 30 days 33 | book priority 29 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (inferred from overview)

Original: https://prachub.com/coding-questions/filter-repeated-robot-status-messages

QUESTION

Overview: Filter a timestamped robot status stream so a message reappears only when strictly more than ten seconds have passed since its last displayed occurrence. Preserve input order, track each exact case-sensitive message independently, and ensure hidden repeats do not reset the interval.

APPROACH

Hash map message -> last displayed timestamp; display only if now - last > 10 (strictly), then update. Preserve input order. O(1) per message.

INTERVIEW-DAY PLAN

Clarify first

Ask for the exact rule: show a message again only if more than 10 seconds have passed since it was last displayed (strictly more), whether messages are compared case-sensitively (yes), and whether hidden repeats extend the interval (no).

Brute force to say out loud

For each message scan the already displayed ones for the last occurrence. O(n^2).

Optimal idea

Keep a dict from message to the time it was last displayed. For each message in order: if it is not in the dict or now - last > 10, display it and set last = now; otherwise skip it and do not update last. Output preserves input order. O(1) per message.

Dry-run these cases

A repeat at exactly 10 seconds (hidden), at 11 seconds (shown), a hidden repeat followed by another repeat 10 seconds after the first display (shown, since hidden repeats do not reset), different casing (different messages), the first message.

Likely follow-ups

Bounded memory (evict messages older than the interval), or a per-robot map.

Pitfalls

Updating the last time on a hidden message. Using at least instead of strictly more than.

HOW TO THINK ABOUT IT

The decision for a message depends only on when that exact message was last displayed, so a dictionary from message to last display time is all the state you need. The detail that decides correctness: the stored time changes only when a message is shown (a hidden repeat must not reset the clock), and the comparison is strict (> 10).

Basis: Written (compare LeetCode 359, Logger Rate Limiter). Rule: show a message again only when strictly more than 10 seconds have passed since it was last shown; hidden repeats do not extend the interval; messages are case-sensitive; input order is kept.

BRUTE FORCE - PSEUDO-CODE

```
shown = list of (time, message) that were displayed
for each new message: look backward in shown for the last
time this message was
    displayed
display it if there is none or now - last > 10
```

Time: O(n^2) | Space: O(n)

Why it is slow

Each message searches the whole history, O(n^2). A dictionary from message to its last displayed time makes each decision O(1).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def filter_messages(events):
    last_displayed = {}
    displayed = []
    previous_timestamp = -1
    for event in events:
        if (
            not isinstance(event, (list, tuple))
            or len(event) != 2
            or not isinstance(event[0], int)
            or isinstance(event[0], bool)
            or event[0] < 0
            or not isinstance(event[1], str)
            or not event[1]
        ):
            raise ValueError("invalid event")
```

```
        timestamp, message = event
        if timestamp < previous_timestamp:
            raise ValueError("timestamps must be nondecrea
sing")
        previous_timestamp = timestamp
        if message not in last_displayed or timestamp - la
st_displayed[message] > 10:
            displayed.append([timestamp, message])
            last_displayed[message] = timestamp
    return displayed
```

Time: Expected O(n). | Space: O(u), where u is the number of distinct messages.

Explanation

The dictionary records only successful display times, so suppressed events cannot extend the cooldown. Each event performs one expected constant-time lookup while retained events are appended in input order.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

```
class BoundedStatusFilter:
    """Streaming version that forgets messages older than
the interval, so memory stays bounded."""

    def __init__(self, gap=10):
        self.gap, self.last, self.heap = gap, {}, []

    def offer(self, ts, msg):
        while (
            self.heap and self.heap[0][0] + self.gap < ts
        ):  # entries older than the gap can never block a
 message again
            t, m = heapq.heappop(self.heap)
            if self.last.get(m) == t:
                del self.last[m]
        if msg in self.last and ts - self.last[msg] <= sel
f.gap:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True
```

Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

FUNCTION AND TEST CASES

Signature

```
def filter_messages(events: list[tuple[int, str]]) -> list
[tuple[int, str]]:
```

Sample call

```
filter_messages([])
```

Test cases

Input	Expected
`([],)`	`[]`
`([[10, "solar panel activated"], [13, "solar panel activated"], [20, "solar panel activated"], [21, "solar panel activated"]],)`	`[[10, "solar panel activated"], [21, "solar panel activated"]]`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Robot Status Message Deduplication Window | 1Point3Acres | onsite-coding | asked 2026-09-10 | Low

https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.

Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

Preparation: Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand. Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

MEDIUM | PracHub | Technical Screen | 80 of 196

Can board states be transformed?

STUDY: TALK | Google frequency on LeetCode: all time 40.3, last 30 days 0 | book priority 83 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (inferred from overview)

Original: https://prachub.com/coding-questions/can-board-states-be-transformed

QUESTION

Overview: This question evaluates reasoning about constrained piece movement, string transformation, and invariant-based correctness in the Coding & Algorithms domain, testing skills in string manipulation and combinatorial reasoning while examining both conceptual properties and practical algorithmic implementation.

APPROACH

Invariants: the sequence of non-empty pieces must match; each L can only move left and each R only right (compare positions with two pointers).

INTERVIEW-DAY PLAN

Clarify first

Ask what the pieces are, which direction each may move, whether pieces may jump over each other, and what the empty cell is called.

Brute force to say out loud

BFS over all reachable board strings. Exponential but clear.

Optimal idea

Invariants: pieces never cross, so the order of the non-empty pieces must be identical in both strings; an L can only move left, so in the start its index must be at least its index in the target; an R can only move right, so its start index must be at most its target index. Walk two pointers over the non-empty cells and check those conditions. O(n).

Dry-run these cases

Equal strings. A different piece order. An L that would have to move right. Same pieces but different counts.

Likely follow-ups

Add walls that cannot be passed or crossed, or ask for the minimum number of moves.

Pitfalls

Comparing counts only and not order. Skipping the empty cells incorrectly in the two-pointer loop.

HOW TO THINK ABOUT IT

Think invariants, not search. Pieces cannot jump each other, so the sequence of L and R letters (ignoring blanks) cannot change. Within that, an L can only go left and an R only right, so compare the position of each piece in the start and in the target. That reduces exponential BFS to one pass with two pointers.

Basis: LeetCode 777, Swap Adjacent in LR String: 'X' is an empty cell, 'L' only moves left, 'R' only moves right, pieces never pass each other.

BRUTE FORCE - PSEUDO-CODE

```
queue = [start]; while queue: pop a board; apply every leg
al move (XL becomes LX, RX
    becomes XR); add unseen boards
return whether the target board appears
```

Time: exponential in the length | Space: exponential

Why it is slow

The number of reachable boards grows exponentially. Pieces never cross, so compare the sequence of L and R pieces and check that no L moved right and no R moved left, in O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(start, target):
    if len(start) != len(target):
        return False

    for i in range(len(start)):
        if (start[i] == "|") != (target[i] == "|"):
            return False

    def check_segment(a, b):
        i = j = 0
        n = len(a)

        while True:
            while i < n and a[i] == "_":
                i += 1
            while j < n and b[j] == "_":
                j += 1

            if i == n or j == n:
                return i == n and j == n

            if a[i] != b[j]:
                return False

            if a[i] == "L" and i < j:
                return False
            if a[i] == "R" and i > j:
                return False

            i += 1
            j += 1

    start_parts = start.split("|")
    target_parts = target.split("|")

    for a, b in zip(start_parts, target_parts):
        if not check_segment(a, b):
            return False

    return True
```

Time: O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

```python
def canTransform(start, end):
    if len(start) != len(end):
        return False
    i = j = 0
    n = len(start)
    while i < n or j < n:
        while i < n and start[i] == "X":
            i += 1
        while j < n and end[j] == "X":
            j += 1
        if i == n or j == n:
            return i == j
        if start[i] != end[j]:
            return False
        if start[i] == "L" and i < j:
            return False
        if start[i] == "R" and i > j:
```

```
            return False
        i, j = i + 1, j + 1
    return True


def min_moves(start, end):
    """If the transformation is possible, every piece move
s independently, so the minimum is the sum of index differ
ences."""
    if not canTransform(start, end):
        return -1
    a = [i for i, c in enumerate(start) if c != "X"]
    b = [i for i, c in enumerate(end) if c != "X"]
    return sum(abs(x - y) for x, y in zip(a, b))
```

Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

FUNCTION AND TEST CASES

Signature

```
def solution(start: str, target: str) -> bool:
```

Sample call

```
solution("R_L|__L", "_RL|L__")
```

Test cases

Input	Expected
`("R_L|__L", "_RL|L__")`	`True`
`("L_|R", "L|_R")`	`False`

MEDIUM | PracHub | Technical Screen | 81 of 196

★ TOP 75: rank 35 in the Medium and Hard priority list

★ Find shared objects across two log files

STUDY: TYPE | Google frequency on LeetCode: all time 50.4, last 30 days 46.9 | book priority 22 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 349 Intersection of Two Arrays

Original: https://prachub.com/coding-questions/find-shared-objects-across-two-log-files

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields
Examples
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']], [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
Hints
Track which days each object appears in and its set of clients.

APPROACH

Load the smaller file's obj_ids into a hash set, stream the other and collect matches; if both are huge use external sort/merge or hash partitioning to disk (or a Bloom filter prefilter).

INTERVIEW-DAY PLAN

Clarify first

Ask whether to report distinct ids or every occurrence, how large each file is, whether order matters, and what to output (ids only or full lines).

Brute force to say out loud

For each id in file A scan file B. O(n * m), unusable at scale.

Optimal idea

Load the ids of the smaller file into a hash set, stream the larger file and collect hits (O(n + m) time). If both do not fit in memory, hash-partition both files by id into buckets on disk so each bucket pair fits, then join bucket by bucket; or external sort both and merge. A Bloom filter of one file prefilters the other.

Dry-run these cases

Empty file, duplicate ids, identical files, no overlap, ids that differ only in case or whitespace.

Likely follow-ups

Distributed version (shuffle by key), top N shared ids by frequency, restrict to a time window.

Pitfalls

Loading both files into memory. Not deduplicating output.

HOW TO THINK ABOUT IT

A common key in two files is a join. If one file fits in memory, load its keys into a set and stream the other. If neither fits, split both files by hash of the key into buckets small enough for memory; equal keys always land in the same bucket number, so you only compare bucket i of one file with bucket i of the other (Grace hash join).

Basis: Written; a join of two large relations (external hash join). Line format assumed: 'timestamp obj_id other'.

BRUTE FORCE - PSEUDO-CODE

```
for every line in file A: for every line in file B: if the
 object ids are equal:
    record the id
```

Time: O(n * m) | Space: O(1) extra

Why it is slow

O(n * m) comparisons. Loading the smaller file's ids into a hash set and streaming the other file is O(n + m); if neither fits, partition both by a hash of the id.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
import collections


def solution(day1, day2):
    days = collections.defaultdict(set)
    clients = collections.defaultdict(set)

    def process(rows, day):
        for row in rows:
            if isinstance(row, dict):
                obj, client = row["obj_id"], row["client_i
d"]
            else:
                obj, client = row[1], row[2]
            days[obj].add(day)
            clients[obj].add(client)

    process(day1, 1)
    process(day2, 2)
    return sorted(obj for obj in days if days[obj] == {1,
2} and len(clients[obj]) >= 2)
```

Time: O(n) | Space: O(number of objects + associations)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

```
def shared_objects_partitioned(path_a, path_b, buckets=4):
    """Neither file fits in memory: partition both by hash
(id) to disk, then join bucket by bucket."""

    def partition(path, tag):
        files = [open(f"{path}.{tag}{i}", "w") for i in ra
nge(buckets)]
        try:
            with open(path) as f:
                for line in f:
                    if line.strip():
                        oid = line.split()[1]
                        files[zlib.crc32(oid.encode()) % b
uckets].write(oid + "\n")
        finally:
            for fh in files:
                fh.close()

    partition(path_a, "a")
    partition(path_b, "b")
    out = set()
    for i in range(buckets):
        with open(f"{path_a}.a{i}") as fa:
            ids = {ln.strip() for ln in fa}
        with open(f"{path_b}.b{i}") as fb:
            out |= {ln.strip() for ln in fb if ln.strip()
in ids}
        os.remove(f"{path_a}.a{i}")
        os.remove(f"{path_b}.b{i}")
    return out
```

Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. Cost: O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Two-Day Log Intersection under Memory Limit | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.

Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).

The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n

log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

Preparation: Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

MEDIUM | PracHub | Technical Screen | 82 of 196

★ TOP 75: rank 13 in the Medium and Hard priority list

★ Solve array-sum and city-community problems

STUDY: TYPE | Google frequency on LeetCode: all time 66, last 30 days 46.9 | book priority 6 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K, LeetCode #200 Number of Islands

LeetCode match: LC 560 Subarray Sum Equals K + LC 200 Number of Islands (inferred from overview)

Original: https://prachub.com/coding-questions/solve-array-sum-and-city-community-problems

QUESTION

Overview: This question evaluates array-processing and grid/graph connectivity skills, covering counting subarrays with a target sum and identifying connected building communities in a grid; it belongs to the Coding & Algorithms domain and involves streaming/online algorithm reasoning as well as grid traversal concepts.

APPROACH

Prefix sum + hash map of counts for the subarray target; communities = connected components via DFS/union-find on the grid.

INTERVIEW-DAY PLAN

Clarify first

Two parts. Array: count subarrays whose sum equals K; are values negative (decides the method) and can the input be a stream? Grid: building communities as connected components, is adjacency four-way, and what marks a building. The scraped solution covers only the array part.

Brute force to say out loud

Array: all subarrays with running sums, O(n^2). Grid: none needed, a flood fill is the baseline.

Optimal idea

Array: prefix sums with a dict of how many times each prefix has occurred; for each element add the count of (prefix - K), then record the prefix. O(n) and it works online for a stream. A sliding window fails with negative values. Grid: DFS/BFS (iterative) or Union-Find to count connected components of building cells.

Dry-run these cases

Array: [1,1,1] with K = 2 gives 2, [1,2,3] with K = 3 gives 2, negatives, K = 0 with zeros, an empty array. Grid: no buildings, one building, diagonal buildings (separate).

Likely follow-ups

Array: longest subarray with sum K, or counting in a stream. Grid: the largest community, or communities as buildings are added.

Pitfalls

Not seeding the prefix dict with {0: 1}. Using a window technique with negative numbers.

HOW TO THINK ABOUT IT

Subarray sums with negatives allowed cannot use a sliding window; use prefix sums: a subarray ending here sums to K exactly when an earlier prefix equals (current prefix - K), so a dictionary of prefix counts answers it in one pass and works for streams. Communities are connected components, so flood fill or union-find over the building cells.

Basis: Two tasks: count subarrays with sum equal to K (LeetCode 560) and count connected building communities in a grid (LeetCode 200 idea). Assumed rule: buildings are '1' cells, communities connect horizontally and vertically.

BRUTE FORCE - PSEUDO-CODE: TASK 1: TRY EVERY SUBARRAY

```
count = 0; for every start i: running sum from i; each tim
e it equals K: count = count
    + 1
```

Time: O(n^2) | Space: O(1)

Why it is slow

O(n^2). A dictionary of prefix sums (how many earlier prefixes equal current - K) is O(n) and also works on a stream and with negative numbers.

BRUTE FORCE - PSEUDO-CODE: TASK 2: RELABEL CELLS UNTIL THE LABELS STOP CHANGING

```
give every building cell its own label
repeat: set every cell's label to the smallest label among
 itself and its neighbours;
    stop when nothing changes
answer = the number of distinct labels
```

Time: O((m * n)^2) worst case | Space: O(m * n)

Why it is slow

Labels spread one step per round, so a long snake needs about m * n rounds of O(m * n): O((m * n)^2). Union-Find or one flood fill is near-linear.

OPTIMAL SOLUTION - TASK 1: PREFIX SUMS IN A DICTIONARY

```
def subarraySum(nums, k):
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count
```

Time: O(n) | Space: O(n)

Why

Walk the array keeping the prefix sum. The number of subarrays ending at the current index with sum K equals how many earlier prefixes were equal to prefix - K, so look that up and then record the current prefix. Seed the dictionary with {0: 1} for subarrays that start at index 0. Works with negative numbers and online.

OPTIMAL SOLUTION - TASK 2: UNION-FIND OVER BUILDING CELLS

```
def communities(grid):
    cells = [
        (r, c)
        for r in range(len(grid))
        for c in range(len(grid[0]))
        if grid[r][c] == "1"
    ]
    parent = {p: p for p in cells}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for r, c in cells:
        for nb in ((r + 1, c), (r, c + 1)):
            if nb in parent:
                parent[find((r, c))] = find(nb)
    return len({find(p) for p in cells})
```

Time: O(m * n * alpha(m * n)) | Space: O(m * n)

Why

Make each building its own set and union it with the building to its right and below. The number of communities is the number of distinct roots. Union-Find also supports adding buildings later.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

```
def longest_subarray_sum_k(nums, k):
    first, prefix, best = {0: -1}, 0, 0
    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, i - first[prefix - k])
        first.setdefault(prefix, i)
    return best
```

Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], k: int) -> int:
```

Sample call

```
solution([1, 1, 1], 2)
```

Test cases

Input	Expected
([1, 1, 1], 2)	2
([1, 2, 3], 3)	2

MEDIUM | IN APP LC 2 | PracHub | Technical Screen | 83 of 196

★ TOP 75: rank 1 in the Medium and Hard priority list

★ Add two big integers from digit lists

STUDY: TYPE | Google frequency on LeetCode: all time 78.7, last 30 days 69.3 | book priority 1 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #2 Add Two Numbers

LeetCode match: LC 2 Add Two Numbers

Original: https://prachub.com/coding-questions/add-two-big-integers-from-digit-lists

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

Examples

Input: ([2, 4, 3], [5, 6, 4])

Expected Output: [7, 0, 8]

Explanation: 342 + 465 = 807, stored reversed as [7, 0, 8].

Input: ([9, 9, 9], [1])

Expected Output: [0, 0, 0, 1]

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

APPROACH

Walk both lists with a carry, build the result list. O(max(m, n)).

HOW TO THINK ABOUT IT

Addition by hand: start at the least significant end, add the two digits and the carry, write digit % 10, carry digit // 10. The only edge cases are different lengths and a final carry. Never convert to a machine integer in an interview about big numbers, since the point is that the numbers do not fit.

Basis: LeetCode 989 / 415 (Add to Array-Form of Integer, Add Strings). Assumed input: lists of decimal digits, most significant digit first.

BRUTE FORCE - PSEUDO-CODE

```
x = the integer formed by the first list's digits; y = the
 same for the second list
return the digits of x + y
```

Time: O(n^2) for the string-to-integer conversions | Space: O(n)

Why it is slow

It relies on arbitrary-size integers and the conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(l1, l2):
    result = []
    carry = 0
    i, j = 0, 0
    n, m = len(l1), len(l2)
    while i < n or j < m or carry:
        total = carry
        if i < n:
            total += l1[i]
            i += 1
        if j < m:
            total += l2[j]
            j += 1
        result.append(total % 10)
        carry = total // 10
    return result
```

Time: O(n + m) | Space: O(1) extra (excluding the output array, whose length is at most max(n, m) + 1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def add_two_numbers_reversed(l1, l2):
    """Digits stored least significant first in a linked l
ist (LeetCode 2)."""
    dummy = tail = ListNode()
    carry = 0
    while l1 or l2 or carry:
        total = carry + (l1.val if l1 else 0) + (l2.val if
```

```
 l2 else 0)
        tail.next = ListNode(total % 10)
        tail, carry = tail.next, total // 10
        l1, l2 = l1.next if l1 else None, l2.next if l2 el
se None
    return dummy.next
```

Multiply instead of add.

Schoolbook multiplication: the product of digits a[i] and b[j] lands in position i + j + 1, with the carry spilling to i + j. O(m * n); Karatsuba or FFT for very long numbers.

```
def multiply_digits(a, b):
    out = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            out[i + j + 1] += a[i] * b[j]
            out[i + j] += out[i + j + 1] // 10
            out[i + j + 1] %= 10
    while len(out) > 1 and out[0] == 0:
        out.pop(0)
    return out
```

Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

FUNCTION AND TEST CASES

Sample call

```
add_two_numbers([2, 4, 3], [5, 6, 4])
```

Test cases

Input	Expected
([2, 4, 3], [5, 6, 4])	[7, 0, 8]
([9, 9, 9], [1])	[0, 0, 0, 1]

MEDIUM | PracHub | Technical Screen | 84 of 196

★ TOP 75: rank 2 in the Medium and Hard priority list

★ Find median of merged RLE arrays

STUDY: TYPE | Google frequency on LeetCode: all time 74.2, last 30 days 61 | book priority 4 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #4 Median of Two Sorted Arrays

LeetCode match: LC 4 Median of Two Sorted Arrays (RLE variant) (inferred from overview)

Original: https://prachub.com/coding-questions/find-median-of-merged-rle-arrays

QUESTION

Overview: The question evaluates understanding of run-length encoding, order-statistics (median) computation, and efficient merging of frequency-annotated sequences, emphasizing handling large frequencies without explicit decompression.

APPROACH

Total length = sum of run counts; walk both run lists merged by value accumulating counts until the median index(es) are covered. Or binary search on value with counts.

INTERVIEW-DAY PLAN

Clarify first

Ask whether each array is sorted by value, how large counts can be (so decompression is impossible), and what to return for an even total (average of the two middle values).

Brute force to say out loud

Decompress both arrays, merge, take the middle. O(total count) memory, which counts can make huge.

Optimal idea

Let N be the sum of all counts and the median positions be (N - 1) // 2 and N // 2. Merge the two run lists by value with two pointers (like the merge step of merge sort), accumulating counts until the running total passes each median position, and read the value there. O(r1 + r2) with no decompression. Alternative: binary search on the value, counting how many elements are at most that value in each array.

Dry-run these cases

A single run, an even total, the median falling exactly on a run boundary, both arrays with the same values, an empty array.

Likely follow-ups

K arrays (heap merge), or arrays not sorted by value.

Pitfalls

Treating counts as lengths of different arrays instead of merging by value. Off-by-one at a run boundary.

HOW TO THINK ABOUT IT

Counts can be huge, so never expand a run. You only need the elements at ranks (N - 1) // 2 and N // 2 in the merged order. Merge the two run lists by value, counting how many elements you have passed, and read the value when the count reaches each rank.

Basis: Written; LeetCode 4 with run-length encoded inputs. Assumed input: two lists of (value, count) runs, each sorted by value; the median of the merged multiset (average of the two middle values for an even total).

BRUTE FORCE - PSEUDO-CODE

```
dense = every run (value, count) written out as count copi
es, from both arrays, sorted
return the middle element (or the average of the two middl
e elements)
```

Time: O(total count) | Space: O(total count)

Why it is slow

Needs time and memory proportional to the total count, which can be 10^9 or more. Walking the two run lists in value order and counting up to the middle rank costs only the number of runs.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(A, B):
    total = sum(f for _, f in A) + sum(f for _, f in B)
    targets = [(total - 1) // 2, total // 2]
    vals = []
    i = j = 0
    seen = -1
    while len(vals) < 2:
        if j >= len(B) or (i < len(A) and A[i][0] <= B[j][
0]):
            val, freq = A[i]
            i += 1
        else:
            val, freq = B[j]
            j += 1
        next_seen = seen + freq
        while len(vals) < 2 and targets[len(vals)] <= next
_seen:
            vals.append(val)
        seen = next_seen
    return vals[0] if total % 2 == 1 else (vals[0] + vals[
1]) / 2
```

Time: O(len(A)+len(B)) | Space: O(1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

K arrays instead of two.

Merge all runs by value (heap of K heads, or sort the combined run list if memory allows) and walk to the two middle ranks. O(R log K) with a heap, R the total number of runs.

```
def median_rle_k_arrays(arrays):
    runs = sorted((v, c) for arr in arrays for v, c in arr
)
    N = sum(c for _, c in runs)

    def kth(k):
        seen = 0
        for v, c in runs:
            seen += c
            if seen > k:
                return v

    return kth(N // 2) if N % 2 else (kth(N // 2 - 1) + kt
h(N // 2)) / 2
```

The arrays are huge and sorted, and you want O(log) time.

Binary search on the value: for a candidate value count how many elements are at most it in each array (binary search on prefix sums of counts) and find the smallest value with rank above the middle. O(log(range) * log(runs)).

Runs are not sorted by value.

Sort the runs by value first, O(R log R), then use the same walk.

FUNCTION AND TEST CASES

Sample call

```
solution([(1, 2), (5, 1)], [(2, 3)])
```

Test cases

Input	Expected
`([(1, 2), (5, 1)], [(2, 3)])`	`2.0`
`([(1, 1)], [(2, 1)])`	`1.5`

MEDIUM | PracHub | Technical Screen | 85 of 196

Check pivot after removing one element

STUDY: TALK | Google frequency on LeetCode: all time 33.2, last 30 days 0 | book priority 105 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #724 Find Pivot Index

LeetCode match: LC 724 Find Pivot Index (variant: one removal) (inferred from overview)

Original: https://prachub.com/coding-questions/check-pivot-after-removing-one-element

QUESTION

Overview: This question evaluates array-manipulation skills, cumulative-sum/prefix-sum reasoning, and robustness in handling negative values and duplicates, testing algorithmic problem-solving competency in the Coding & Algorithms domain and focusing on practical application rather than purely conceptual understanding.

APPROACH

Prefix sums plus a hash map of left-side sums; for each removal candidate, check whether some index has equal left and right sums after the removal.

INTERVIEW-DAY PLAN

Clarify first

The statement is not complete in my copy, so confirm: after removing exactly one element (or at most one), does there exist an index where the sum to the left equals the sum to the right (the pivot itself excluded)? Are negatives and duplicates allowed?

Brute force to say out loud

For each element removed, rebuild the array and test every pivot with prefix sums. O(n^2).

Optimal idea

Let P[i] be the sum before index i and S the total. If the removed element r is to the right of the pivot i, the two sides are equal exactly when a[r] = S - 2 * P[i] - a[i]. If r is to the left of i, the condition is a[r] = 2 * P[i] + a[i]

- S. So for every pivot candidate the value that must be removed is determined; look it up in a dict of value to last index (needs an index greater than i) and a dict of value to first index (needs an index less than i). One pass, O(n) time and space. Say the brute

force first, and mention you checked the formulas against it on random small arrays.

Dry-run these cases

A one-element array, two elements, negatives, duplicate values, a pivot that exists without any removal, removing the pivot itself (not allowed by most readings).

Likely follow-ups

Allow at most one removal or remove a pivot candidate, or count all pairs.

Pitfalls

Letting the removed element be the pivot. Forgetting that the prefix sums shift after a removal.

HOW TO THINK ABOUT IT

Use prefix sums. For a pivot at original index i and a removed element on one side, the balance equation fixes the value that has to be removed: it is a closed form in the prefix sum, the total and a[i]. So for every candidate pivot you only have to ask 'does that value occur on the correct side?', which a dictionary of first and last positions answers in O(1).

Basis: Variant of LeetCode 724 (Find Pivot Index). Assumed rule (confirm it): remove exactly one element; is there then an index where the sum on its left equals the sum on its right (the pivot excluded)?

BRUTE FORCE - PSEUDO-CODE

```
for every index r to remove: build the shorter array
for every position i in it: if sum(left of i) == sum(right
 of i): return true
return false
```

Time: O(n^3), O(n^2) with prefix sums | Space: O(n)

Why it is slow

n removals times n pivots times a sum: O(n^3) (O(n^2) with prefix sums). For each pivot the value to remove is determined by prefix sums, so a dictionary lookup makes it O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(nums):
    from collections import Counter

    total = sum(nums)
    right = Counter(nums)
    left = Counter()
    prefix = 0  # sum of elements strictly before the curr
ent index

    for x in nums:
        # Current element cannot be the removed one if it
is the pivot we are testing,
        # so exclude it from the right-side multiset befor
e checking.
        right[x] -= 1
        if right[x] == 0:
            del right[x]

        # If the current index stays, then:
        # left_sum = prefix
        # right_sum = total - prefix - x
        # imbalance = left_sum - right_sum
        imbalance = prefix - (total - prefix - x)

        # To balance around this index after one removal:
        # - remove 'imbalance' from the left side, or
        # - remove '-imbalance' from the right side.
        if left.get(imbalance, 0) > 0 or right.get(-imbala
nce, 0) > 0:
            return True

        left[x] += 1
        prefix += x

    return False
```

Time: O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

```
def pivot_index(a):
    S, left = sum(a), 0
    for i, x in enumerate(a):
        if left == S - left - x:
            return i
        left += x
    return -1
```

At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> bool:
```

Sample call

```
solution([1, 2])
```

Test cases

Input	Expected
([1, 2],)	True
([2, 1, 1, 2],)	True

MEDIUM | PracHub | Technical Screen | 86 of 196

Find longest increasing contiguous subarray

STUDY: TALK | Google frequency on LeetCode: all time 19.6, last 30 days 0 | book priority 134 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 674 Longest Continuous Increasing Subsequence (variant: one modification) (inferred from overview)

Original: https://prachub.com/coding-questions/find-longest-increasing-contiguous-subarray

QUESTION

Overview: This question evaluates proficiency in array processing and algorithmic optimization, focusing on recognition of contiguous subarray properties and handling a single-value modification to achieve strict monotonicity.

APPROACH

Compute run lengths ending at i (left) and starting at i (right); for each i try changing nums[i] to bridge: left[i-1] + 1 + right[i+1] if nums[i+1] - nums[i-1] >= 2. O(n).

INTERVIEW-DAY PLAN

Clarify first

Ask whether increasing is strict, whether the modified value can be any integer or any real, and whether you must change exactly one element or at most one.

Brute force to say out loud

Try changing each element to every useful value and recompute the run. O(n^2).

Optimal idea

Compute left[i], the length of the strictly increasing run ending at i, and right[i], the run starting at i. Changing nums[i] can bridge the neighbors if nums[i + 1] - nums[i - 1] >= 2 (integers), giving left[i - 1] + 1 + right[i + 1]. Otherwise you can only extend one side: left[i - 1] + 1 or right[i + 1] + 1. Take the maximum over all i, and also the run with no change. O(n).

Dry-run these cases

n = 1, a strictly decreasing array, a plateau like [1, 2, 2, 3], neighbors that differ by exactly 1 (cannot bridge with integers), the change at the first or last index.

Likely follow-ups

Allow k changes (needs a sliding window with at most k bad positions), allow non-strict increase, or return the subarray bounds.

Pitfalls

Forgetting the integer gap condition. Index errors at the array ends.

HOW TO THINK ABOUT IT

A run that has a single 'bad' spot can be repaired by changing one element, so look at each position i as the element to change. The best run through i joins the increasing run that ends just before it and the one that starts just after it, and the join is legal only if the values leave enough room between the two neighbours (for integers, a[i + 1] - a[i - 1] >= 2).

Basis: LeetCode 674 (Longest Continuous Increasing Subsequence) with one allowed modification. Assumed rule (confirm it): strictly increasing, you may change at most one element to any integer.

BRUTE FORCE - PSEUDO-CODE

```
for every subarray: count positions where an element is no
t larger than the previous
    one
if there are none, or one that a single change can repair
(the neighbours leave room),
    the subarray qualifies; keep the longest
```

Time: O(n^3) | Space: O(1)

Why it is slow

O(n^2) subarrays, each tested in O(n): O(n^3). Run lengths ending and starting at each index, then trying each index as the changed element, is O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(nums):
    n = len(nums)
    if n == 0:
        return 0
    max_len = 1
    cur = 1
    for i in range(1, n):
        if nums[i] > nums[i - 1]:
            cur += 1
            if cur > max_len:
                max_len = cur
        else:
            cur = 1
    return max_len
```

Time: O(n) | Space: O(1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when a[i] <= a[i - 1]. O(n), O(1).

```python
def longest_increasing_no_change(a):
    best, run = 0, 0
    for i, x in enumerate(a):
        run = run + 1 if i and x > a[i - 1] else 1
        best = max(best, run)
    return best
```

Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

FUNCTION AND TEST CASES

Signature

```
def longestIncreasing(nums: list[int]) -> int:
```

Sample call

```
longestIncreasing([1, 2, 2, 3, 4])
```

Test cases

Input	Expected
([1, 2, 2, 3, 4],)	3
([1, 2, 3, 4, 5],)	5

MEDIUM | PracHub | Technical Screen | 87 of 196

Decide string transform with directional tokens and walls

STUDY: TALK | Google frequency on LeetCode: all time 40.3, last 30 days 0 | book priority 84 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (with X walls)

Original: https://prachub.com/coding-questions/decide-string-transform-with-directional-tokens-and-walls

QUESTION

You are given two equal-length strings start and target of length n, each consisting only of the characters 'L', 'R', '.', and 'X'. A dot '.' represents an empty slot; 'X' represents an immovable wall; 'L' and 'R' are tokens that may move across empty slots subject to these rules: (1) An 'L' token may move any number of positions left into '.', but may never move right, pass through another token, or pass through an 'X'. (2) An 'R' token may move any number of positions right into '.', but may never move left, pass through another token, or pass through an 'X'. (3) Only one token can occupy a position at any time. Determine whether target can be obtained from start by a sequence of valid moves. If yes, return true; otherwise, return false. Design an algorithm that runs in O(n) time and O(1) extra space beyond a few pointers/counters. Explain your invariants, prove correctness informally, analyze time and space complexity, and walk through a few tricky edge cases (e.g., mismatched wall layouts, blocks separated by walls, consecutive tokens with no gaps).

APPROACH

Split both strings at 'X' walls (must align), then within each segment apply the L/R two-pointer rule: L only moves left, R only right, order of pieces preserved.

INTERVIEW-DAY PLAN

Clarify first

Ask what each symbol means (L moves left, R moves right, dot empty, X wall that never moves and cannot be crossed), whether a token can jump over another (no), and whether both strings have the same length.

Brute force to say out loud

BFS over all board states by moving tokens one step. Exponential.

Optimal idea

Compare the sequences of non-dot characters: they must be identical, which also forces every X to appear in the same order. Then with two pointers over those tokens check that each L has start index at least target index, each R has start index at most target index, and each X has the same index in both. Walls cannot be crossed because order is preserved. O(n).

Dry-run these cases

Identical strings, an L that would move right, an R blocked by a wall, a different number of tokens, a wall at a different index.

Likely follow-ups

Return the minimum number of moves (sum of index differences), or the move sequence.

Pitfalls

Letting a token cross a wall. Checking only counts of L and R.

HOW TO THINK ABOUT IT

Walls are pieces that never move. Ignore the dots and compare the sequences of L, R and X: they must match in order and in count, and each X must be at the same index in both strings. For the L and R pieces keep the same rule: L never moves right, R never moves left. Order is preserved, so no piece can get across a wall.

Basis: LeetCode 777 with walls: 'L' moves left, 'R' moves right, '.' is empty, 'X' is an immovable wall; no piece crosses another piece or a wall.

BRUTE FORCE - PSEUDO-CODE

```
queue = [start]; for each board popped: apply every legal
single step ('.L' becomes
    'L.', 'R.' becomes '.R'); walls never move; add unseen
 boards
return whether the target board appears
```

Time: exponential | Space: exponential

Why it is slow

The number of reachable boards grows exponentially with the length. Comparing the non-dot sequence (walls and pieces in order) with position rules (L never moves right, R never moves left, walls stay) is O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(start, target):
    n = len(start)
    if len(target) != n:
        return False
    i = 0
    j = 0
    while i < n or j < n:
        while i < n and start[i] == ".":
            i += 1
        while j < n and target[j] == ".":
            j += 1
        if i == n or j == n:
            return i == n and j == n
        if start[i] != target[j]:
            return False
        c = start[i]
        if c == "L":
            if i < j:
                return False
        elif c == "R":
            if i > j:
                return False
        else:
            if i != j:
                return False
        i += 1
        j += 1
    return True
```

Time: O(n) — each of the two pointers advances monotonically across the string exactly once. | Space: O(1) — only a constant number of integer pointers is used; the strings are scanned in place.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

```
def can_transform_walls(start, end):
    if len(start) != len(end):
        return False
    i = j = 0
    n = len(start)
    while True:
        while i < n and start[i] == ".":
            i += 1
        while j < n and end[j] == ".":
            j += 1
        if i == n or j == n:
            return i == j
        if start[i] != end[j]:
            return False
        if start[i] == "X" and i != j:
            return False
        if start[i] == "L" and i < j:
            return False
        if start[i] == "R" and i > j:
            return False
        i, j = i + 1, j + 1


def min_moves_walls(start, end):
    if not can_transform_walls(start, end):
        return -1
    a = [i for i, c in enumerate(start) if c != "."]
    b = [i for i, c in enumerate(end) if c != "."]
    return sum(abs(x - y) for x, y in zip(a, b))
```

FUNCTION AND TEST CASES

Signature

```
def canTransform(start: str, target: str) -> bool:
```

Sample call

```
canTransform('R...L', 'RL...')
```

Test cases

Input	Expected
`('R...L', 'RL...')`	`True`
`('R.L', 'R..L')`	`False`

MEDIUM | PracHub | Technical Screen | 88 of 196

Reconstruct original array from doubled shuffle

STUDY: TALK | Google frequency on LeetCode: all time 29.8, last 30 days 0 | book priority 115 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 2007 Find Original Array From Doubled Array (inferred from overview)

Original: https://prachub.com/coding-questions/reconstruct-original-array-from-doubled-shuffle

QUESTION

Overview: This question evaluates array manipulation, multiset/frequency reasoning, pairing logic, and careful handling of edge cases such as zeros and negative numbers.

APPROACH

Sort (by absolute value) and use a counter: for each smallest x with count > 0, pair with 2x; handle zeros (need even count) and negatives. O(n log n).

INTERVIEW-DAY PLAN

Clarify first

Ask whether the output may be in any order, whether negatives and zeros are possible, and what to return if it is impossible (empty array).

Brute force to say out loud

Try every pairing recursively. Exponential.

Optimal idea

If the length is odd return empty. Count values. Process numbers in order of absolute value; for each value x with remaining count, match it with 2x, decrement both counts, and fail if 2x has no count left. Zeros need an even count. Sorting by abs makes the smallest value always the original, never a double. O(n log n).

Dry-run these cases

[1,3,4,2,6,8] gives [1,3,4]. An odd length. [0,0]. [0,0,0] fails. Negatives such as [-2,-4,1,2].

Likely follow-ups

Linear time when the value range is small (counting array), or unsorted output.

Pitfalls

Sorting by value instead of absolute value for negatives. Mishandling zeros.

HOW TO THINK ABOUT IT

The element with the smallest absolute value cannot be the double of anything, so it is an original; its double must be present too. Pair them off, take the next smallest, and so on. Process values in order of absolute value so negatives and zeros follow the same rule, and handle zero separately (it is its own double, so it needs an even count).

Basis: LeetCode 2007, Find Original Array From Doubled Array.

BRUTE FORCE - PSEUDO-CODE

```
sort by absolute value; originals = []
while the list is not empty: x = the smallest; if 2x is no
t in the list: fail; remove
    x and one copy of 2x; add x to originals
```

Time: O(n^2) | Space: O(n)

Why it is slow

list.remove is a linear search, so each step is O(n) and the loop is O(n^2). A Counter processed in order of absolute value is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(changed):
    from collections import Counter

    n = len(changed)
    if n % 2 == 1:
        return []

    freq = Counter(changed)
    original = []

    for x in sorted(freq.keys(), key=abs):
        count = freq[x]
        if count == 0:
            continue

        if x == 0:
            if count % 2 == 1:
                return []
            original.extend([0] * (count // 2))
            freq[0] = 0
        else:
            if freq[2 * x] < count:
                return []
            original.extend([x] * count)
            freq[2 * x] -= count
            freq[x] = 0

    if len(original) * 2 != n:
        return []

    original.sort()
    return original
```

Time: O(n log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

```python
def findOriginalArray(changed):
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out


def find_original_in_input_order(changed):
    base = Counter(findOriginalArray(changed))
    if not base and changed:
        return []
    out = []
    for x in changed:
        if base[x] > 0:
            base[x] -= 1
```

```
            out.append(x)
    return out
```

Values are small non-negative integers.
Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

Why does processing by absolute value work with negatives?
For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

FUNCTION AND TEST CASES

Signature

```
def solution(changed: list[int]) -> list[int]:
```

Sample call

```
solution([2, 1, 4, 8])
```

Test cases

Input	Expected
`([2, 1, 4, 8],)`	`[1, 4]`
`([-4, -2, -2, -1, 1, 2],)`	`[-2, -1, 1]`

MEDIUM | PracHub | Technical Screen | 89 of 196
★ TOP 75: rank 12 in the Medium and Hard priority list

★ Solve Banana Speed and Interval Merge

STUDY: TYPE | Google frequency on LeetCode: all time 65.2, last 30 days 41.1 | book priority 9 of 196
Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns
Optimal answer from: written and tested here for this exact question
This question has several parts, so each part has its own answer below.
Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas, LeetCode #56 Merge Intervals
LeetCode match: LC 875 Koko Eating Bananas + LC 56 Merge Intervals
Original: https://prachub.com/coding-questions/solve-banana-speed-and-interval-merge

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`** Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints
1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9
Examples
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4: ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) = 1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 = 10 > 8, so 4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.
Hints
If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

APPROACH

Binary search the speed with ceil(pile/speed) sum <= h; sort intervals by start and merge overlaps in one pass.

INTERVIEW-DAY PLAN

Clarify first

Bananas: piles and h are positive, h is at least the number of piles (otherwise impossible), one pile per hour, and an hour finishes at most one pile. Intervals: overlap includes touching (next start at most the current end).

Brute force to say out loud

Bananas: try every speed from 1 up. Intervals: compare every pair.

Optimal idea

Bananas (LeetCode 875): the time function is monotone in the speed, so binary search the smallest speed in [1, max(piles)] with sum(ceil(p / s)) <= h. O(n log max). Intervals: sort by start and sweep, merging while the next start is at most the current end. O(n log n).

Dry-run these cases

Bananas: h equal to the pile count (speed is the maximum), a single pile, large piles (overflow). Intervals: touching, nested, unsorted, one interval.

Likely follow-ups

Bananas with a variable h or a floating-point speed. Intervals: insert one interval or count the merged result.

Pitfalls

Using floor instead of ceil. Merging with strict less-than.

HOW TO THINK ABOUT IT

Bananas: the time needed is non-increasing in the speed, so the smallest speed that finishes in h hours can be found by binary search over speeds, with a simple sum of ceil(pile / speed) as the check. Intervals: sort by start and sweep with one 'current' interval; the sort guarantees nothing earlier can still overlap.

Basis: LeetCode 875 (Koko Eating Bananas; one pile per hour, h at least the number of piles) and LeetCode 56 (Merge Intervals; overlapping when the next start is at most the current end).

BRUTE FORCE - PSEUDO-CODE: BANANAS: TRY EVERY SPEED UPWARD

```
for speed = 1, 2, 3, ...: hours = sum of ceil(pile / speed
) over the piles; if hours
    <= h: return speed
```

Time: O(max * n) | Space: O(1)

Why it is slow

Up to max(piles) speeds, each costing O(n); piles can be 10^9. Hours are non-increasing in the speed, so binary search over speeds is O(n log max).

BRUTE FORCE - PSEUDO-CODE: INTERVALS: MERGE ANY OVERLAPPING PAIR UNTIL NONE IS LEFT

```
repeat: find two intervals that overlap and replace them w
ith their union
stop when no pair overlaps
```

Time: O(n^2) | Space: O(n)

Why it is slow

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION - BANANAS: BINARY SEARCH THE SPEED

```python
def minEatingSpeed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if sum(-(-p // mid) for p in piles) <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Time: O(n log max) | Space: O(1)

Why

Hours needed are non-increasing in the speed, so keep lo = 1 and hi = max(piles) and halve: if speed mid finishes within h, try slower, otherwise faster. Use ceiling division (an hour finishes at most one pile).

OPTIMAL SOLUTION - INTERVALS: SORT AND SWEEP

```python
def merge_intervals(intervals):
    out = []
    for s, e in sorted(intervals):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Time: O(n log n) | Space: O(n)

Why

Sort by start. If an interval starts at or before the end of the last merged one, extend that end with max; otherwise start a new interval.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Bananas: h varies per query, or the piles change.

The check is independent of the search; for many queries keep the piles sorted with prefix sums so the hours needed at speed s can be computed in O(log n) per distinct pile size.

Intervals: insert one new interval into an already merged list (LeetCode 57).

Copy intervals ending before the new one, merge the ones that overlap it, then copy the rest. O(n), no sort needed.

```
def insert_interval(intervals, new):
    out, i, n = [], 0, len(intervals)
    while i < n and intervals[i][1] < new[0]:
        out.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] <= new[1]:
        new = [min(new[0], intervals[i][0]), max(new[1], i
ntervals[i][1])]
        i += 1
    out.append(new)
    out.extend(intervals[i:])
    return out
```

Bananas: the guards come back in fewer hours than piles.

Then no speed works (an hour can finish at most one pile): check h >= len(piles) first and return -1 otherwise.

FUNCTION AND TEST CASES

Signature

```
def solution(piles: list[int], h: int) -> int:
```

Sample call

```
solution([3, 6, 7, 11], 8)
```

Test cases

Input	Expected
([3, 6, 7, 11], 8)	4
([30, 11, 23, 4, 20], 5)	30

MEDIUM | IN APP LC 56 | Exponent | 90 of 196

★ TOP 75: rank 11 in the Medium and Hard priority list

★ Merge Intervals

STUDY: TYPE | Google frequency on LeetCode: all time 65.2, last 30 days 41.1 | book priority 10 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #56)

LeetCode match: LC 56 Merge Intervals

+1219 interviews

Original: https://www.aced.io/questions/1602/merge-intervals

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in th

APPROACH

Sort by start and merge overlapping intervals in one pass. O(n log n).

HOW TO THINK ABOUT IT

After sorting by start, an interval can overlap only the interval just before it in the output, because everything earlier ends before that one starts. So one pass with a 'current merged interval' is enough: extend its end, or start a new one.

Basis: LeetCode 56, Merge Intervals (intervals sharing a point merge).

BRUTE FORCE - PSEUDO-CODE

```
repeat: find two intervals that overlap and replace them w
ith their union
stop when no pair overlaps; return the intervals sorted
```

Time: O(n^2) or worse | Space: O(n)

Why it is slow

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #56 MERGE INTERVALS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Merge all overlapping intervals. Two intervals overlap if one starts before or when the other ends.
[1,4] and [4,5] overlap at 4. Sort by start, then merge. Return non-overlapping result. Right?"

"[[1,3],[2,6],[8,10],[15,18]] -> [[1,6],[8,10],[15,18]] ok (1–3 and 2–6 overlap at 2-3)"
Are there duplicate elements? Should the sort be stable?

PHASE 3 — OPTIMIZE

Sort by start time — O(n log n). Then one pass:
Keep a result list. For each interval, if it overlaps with the last result interval
(last.end >= current.start), extend the last interval's end.
Otherwise, append as a new separate interval.
Time: O(n log n). Space: O(n)."
Key insight: sorting by start time guarantees that intervals that can merge are adjacent — one linear pass then extends or appends, making the merge O(n) after the O(n log n) sort.

PHASE 4 — CLEAN CODE

```
class _56_MergeIntervals:
    def merge(self, intervals: List[List[int]]) -> List[
List[int]]:
        intervals.sort()
        merged = [intervals[0]]
        for start, end in intervals[1:]:
            if merged[-1][1] >= start:              # ov
erlap
                merged[-1][1] = max(merged[-1][1], end)
            else:
                merged.append([start, end])
        return merged
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace merge([[1,3],[2,6],[8,10],[15,18]])."
Sort by start (already sorted). result=[[1,3]].
[2,6]: 2<=3 -> overlap. Merge: [1,max(3,6)]=[1,6]. result=[[1,6]].
[8,10]: 8>6 -> no overlap. Append. result=[[1,6],[8,10]].
[15,18]: 15>10 -> no overlap. Append. result=[[1,6],[8,10],[15,18]] ok
[[1,4],[4,5]]: 4<=4 -> overlap -> [1,5] ok
WHY sort first: ensures we only need to compare with the last merged interval.
If current start > last end: new group. Else: extend last end to max of both ends.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n log n). Space O(n).
The sort is the bottleneck — the merge pass itself is O(n).
If the input were already sorted, this runs in O(n) total.

Given more time I'd ask: how many meeting rooms needed?
That's Meeting Rooms II (#253) — use a min-heap of end times.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Insert one interval into an already merged list (LeetCode 57).
Copy the intervals that end before it, merge the ones that overlap, copy the rest: O(n) with no sort.

```
def insert_interval(intervals, new):
    out, i = [], 0
    while i < len(intervals) and intervals[i][1] < new[0]:
        out.append(intervals[i])
        i += 1
    while i < len(intervals) and intervals[i][0] <= new[1]
:
        new = [min(new[0], intervals[i][0]), max(new[1], i
ntervals[i][1])]
        i += 1
    return out + [new] + intervals[i:]
```

Intervals of touching integers ([1, 2] and [3, 4]) should merge.
Change the overlap test to start <= last_end + 1. For real-valued ranges that would be wrong, so state which you assume.

Too many intervals for memory.
Sort externally (external merge sort) and stream the merge: the one-pass sweep needs only the current interval in memory. Cost: O(n log n) external sort with bounded memory runs, then an O(n) streaming merge with O(1) memory.

OFFICIAL PAGE TEXT

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
output: [[1,6], [8,10], [15,18]]

intervals = [[1,4], [4,5]]
output: [[1,5]]

intervals = [[1,4], [2,3]]
output: [[1,4]]

intervals = [[1,2], [3,4], [5,6], [7,8]]
output: [[1,2], [3,4], [5,6], [7,8]]
```

The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent

intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in the result list. If the current interval overlaps with the last interval in the result (i.e., the start of the current interval is less than or equal to the end of the last interval), merge them by updating the end of the last interval in the result to the maximum end time of both intervals. If there is no overlap, add the current interval to the result list as a separate interval. Return the Result: After processing all intervals, the result list contains the merged intervals. Time Complexity: The dominant factor in the time complexity is the sorting step, which takes O(n log n), where n is the number of intervals. The merging process involves a single pass through the sorted intervals, which takes O(n) time. Therefore, the overall time complexity is O(n log n). Space Complexity: The space complexity is O(n), which accounts for the space required to store the result list. This is because, in the worst case, none of the intervals overlap, and the result list will contain all the original intervals.In this mock interview video, Lewin (Dropbox SWE), answers the interview question, "Given an array of intervals, merge the overlapping intervals and return an array of the non-overlapping intervals that cover all intervals in the input."No answers contributed by the community yet.

MEDIUM | IN APP LC 49 | Exponent | 91 of 196

★ TOP 75: rank 23 in the Medium and Hard priority list

★ Group anagrams

STUDY: TYPE | Google frequency on LeetCode: all time 57.7, last 30 days 51.4 | book priority 14 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #49)

LeetCode match: LC 49 Group Anagrams (inferred from short statement)

Original: https://www.aced.io/questions/4676/group-anagrams

QUESTION

Group anagrams

APPROACH

Hash map keyed by sorted letters (or a 26-count tuple) to lists of words. O(n * k log k).

HOW TO THINK ABOUT IT

Anagrams are strings with the same letters in any order, so they have the same canonical form: the sorted letters, or the count of each letter. Use that as a dictionary key and group in one pass; you never compare two strings directly.

Basis: LeetCode 49, Group Anagrams.

BRUTE FORCE - PSEUDO-CODE

```
groups = []; for each word: compare its sorted letters wit
h those of the first word of
    every group
add it to the group that matches, else start a new group
```

Time: O(n * g * L log L) | Space: O(n)

Why it is slow

Each word is compared with many groups, O(n * groups * L log L). Using the sorted letters (or a letter-count tuple) as a dictionary key groups all words in one pass.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #49 GROUP ANAGRAMS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Group strings that are anagrams. Return groups in any order. Same chars, same frequencies -> anagrams. Empty string is its own group. Right?"

"['eat','tea','tan','ate','nat','bat'] -> 3 groups ok"
Are string comparisons case-sensitive?

PHASE 3 — OPTIMIZE

Say: "Sort each word as key, group by key — O(n * k log k).
Walk-through:
For each word in strs:
key = ''.join(sorted(word)) <- anagrams share the same sorted key.
groups[key].append(word).
Return list(groups.values()).
WHY sorted key: anagrams are permutations of each other -> sorting them gives identical strings.
'eat','tea','ate' all sort to 'aet' -> same group.
Alternative: count-tuple key (26-char frequency) -> O(n*k) but more code.
Time: O(n * k log k) where k = max word length. Space: O(n*k).

PHASE 4 — CLEAN CODE

```python
class _49_GroupAnagrams:
    def groupAnagrams(self, strs: List[str]) -> List[Lis
t[str]]:
        groups: defaultdict = defaultdict(list)
        for word in strs:
            groups[tuple(sorted(word))].append(word)
        return list(groups.values())
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace groupAnagrams(['eat','tea','tan','ate','nat','bat'])."
Key each word by sorted characters.
'eat'->'aet'. 'tea'->'aet'. 'tan'->'ant'. 'ate'->'aet'. 'nat'->'ant'. 'bat'->'abt'.
Groups: {'aet':['eat','tea','ate'], 'ant':['tan','nat'], 'abt':['bat']}.

Return [['eat','tea','ate'],['tan','nat'],['bat']] ok
WHY sorted key: anagrams have identical character multisets -> same sorted string.
O(nklogk) where k=max word length (sort each word).
WHY sorted string as key: anagrams are permutations -> same character multisets -> sort normalizes.
Counter key alt: tuple(sorted(Counter(word).items())) works too but slower than sorted string.
Trace: ["eat","tea","tan","ate","nat","bat"] -> sort each: "aet","aet","ant","aet","ant","abt" -> groups: {"aet":["eat","tea","ate"],"ant":["tan","nat"],"abt":["bat"]}.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n * m log m). Space O(n * m).
Frequency tuple alternative: O(n * m), avoids log m sort. For short words (typical)
sorting is fine; for very long words, frequency tuple wins.
Given more time I'd ask: Unicode input? sorted() handles it; frequency array breaks.
Use Counter-based key: tuple(sorted(Counter(word).items())).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Make the key O(L) instead of O(L log L).

Use a tuple of 26 letter counts as the key (no sort). Total O(n * L); for large alphabets use a Counter turned into a frozenset of items or a sorted tuple of (char, count).

```
def group_anagrams_counts(words):
    groups = defaultdict(list)
    for w in words:
        key = [0] * 26
        for ch in w:
            key[ord(ch) - 97] += 1
        groups[tuple(key)].append(w)
    return list(groups.values())
```

Case-insensitive, Unicode, or ignore spaces and punctuation.

Normalise first (casefold, strip non-letters, Unicode normalisation) and then compute the key. Say what counts as a letter.

The words are in many files.

Compute the key on each machine and hash-partition by key so every anagram group lands on one worker, which then groups locally (a MapReduce shuffle).

OFFICIAL PAGE TEXT

Given an array of strings, group together the words that are anagrams (e.g. gas, sag) of each other. The order of the groups or the words within the groups does not matter in the final result.

MEDIUM | IN APP LC 271 | Exponent | 92 of 196

Write functions to serialize and deserialize a list of strings.

STUDY: TALK | Google frequency on LeetCode: all time 28.2, last 30 days 0 | book priority 117 of 196
Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns
Optimal answer from: your app's interview approach (LeetCode #271)
LeetCode match: LC 271 Encode and Decode Strings
2 interviews
Original:
https://www.aced.io/questions/1307/serialize-deserialize-list-strings

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

APPROACH

Length-prefix encoding (e.g. '5#hello'), decode by reading the length and slicing; safe for any characters including delimiters.

HOW TO THINK ABOUT IT

Any text can appear inside a string, so a delimiter is never safe by itself. The fix is to say how long each string is before you write it (length prefix), so the decoder never has to look for a separator inside the data. Always check two corner cases: the empty list and a list containing the empty string.

Basis: LeetCode 271, Encode and Decode Strings.

BRUTE FORCE - PSEUDO-CODE

```
serialize: join the strings with ','
deserialize: split the text on ','
```

Time: O(total length) | Space: O(total length)

Why it is slow

It is fast (O(total length)) but wrong: a string that contains the delimiter breaks it, and an empty list cannot be told from a list of one empty string. Writing each string's length before it fixes both.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #271 ENCODE AND DECODE STRINGS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Design encode(strs) and decode(s) that roundtrip correctly for ANY list of strings —
including strings with special characters, empty strings, or strings containing
whatever delimiter we might choose. Right?
We can't use eval or pickle."

"['Hello','World'] -> encode -> some string -> decode -> ['Hello','World'] ok
[''] -> [''] ok (empty string must survive the roundtrip)"
Are string comparisons case-sensitive?

PHASE 3 — OPTIMIZE (length-prefix encoding)

Say: "Encode with length prefix, decode by parsing length — O(total chars). Walk-through:
encode(strs): for each s: result += str(len(s)) + '#' + s.
Example: ['abc','de'] -> '3#abc2#de'.
decode(s): i=0. While i < len(s):
j = s.index('#', i). <- find next '#'
length = int(s[i:j]). <- parse length
words.append(s[j+1 : j+1+length]). <- extract word
i = j+1+length. <- advance past this word
WHY '#' separator + length: '#' alone is not unique (words may contain '#').
Length prefix lets decoder know exactly how many chars to read for each word.
WHY this beats delimiter-only: handles arbitrary characters in strings.
Time: O(n). Space: O(n).

PHASE 4 — CLEAN CODE

```python
class _271_Codec:
    def encode(self, strs: List[str]) -> str:
        return ''.join(f'{len(s)}#{s}' for s in strs)

    def decode(self, s: str) -> List[str]:
        result = []
        i = 0
        while i < len(s):
            j = s.index('#', i)
            length = int(s[i:j])
            result.append(s[j + 1: j + 1 + length])
            i = j + 1 + length
        return result
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace encode/decode on ['Hello','World']."
Encode: for each word, prefix with len#: '5#Hello5#World'
Decode: scan string, read number up to '#', then read that many chars
i=0: read '5', skip '#', read 5 chars 'Hello'. i=7
i=7: read '5', skip '#', read 5 chars 'World'. i=14. Done.
Result: ['Hello','World'] ok
WHY length-prefix: handles strings containing any chars (spaces, #, slashes).
Edge: empty string '' -> encoded as '0#'; decoded back to ''. ok
Edge: word with '#' in it: '3#a#b' -> len=3, read 'a#b' correctly.
Decode: scan encoded string left-to-right. stack=[] of decoded chunks. Find digits before '#': length=4,chunk='lint'.
stack=['lint','code','love','you']. Decoded=['lint','code','love','you'] ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

encode O(total chars). decode O(total chars). Space O(n).
Length-prefix is the standard solution — no special characters, no escaping needed.
The '#' separator is arbitrary — any fixed separator works since we always read
the length first and know exactly how many chars to consume.
Given more time I'd ask: what if strings can contain binary data (null bytes)?
Use a 4-byte big-endian integer prefix instead of a text number — fully binary-safe.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Use escaping instead of a length prefix.

Escape the separator and the escape character inside each string, then terminate each item with the unescaped separator. The decoder tracks whether the previous character was an escape. It is streamable, while the length prefix can be read in O(1) jumps.

```python
def decode_escape(data):
    out, cur, i = [], [], 0
    while i < len(data):
        ch = data[i]
        if ch == "\\":
            cur.append(data[i + 1])
            i += 2
```

```
        elif ch == ";":
            out.append("".join(cur))
            cur = []
            i += 1
        else:
            cur.append(ch)
            i += 1
    return out


def encode_escape(strs):
    return "".join(s.replace("\\", "\\\\").replace(";", "\
\;") + ";" for s in strs)
```

The strings are long or binary.

Use bytes and a fixed-width length (for example a 4-byte big-endian integer from struct.pack). Fixed-width lengths avoid the digit-scanning loop and work for any byte value.

Just use JSON?

Yes if the format is free to choose: json.dumps handles quoting and Unicode correctly. Mention it as the practical answer, but code the length-prefix version when the interviewer wants to see you handle the delimiter problem yourself.

```
def encode_json(strs):
    return _json.dumps(strs)
```

OFFICIAL PAGE TEXT

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples

```
lst = ['hello', 'world']
serialized = serialize(lst)
# serialized output: "\x05hello\x05world"
deserialized = deserialize(serialized)
# deserialized output should be: ['hello', 'world']

lst = ['']
serialized = serialize(lst)
# serialized output: "\x00"
deserialized = deserialize(serialized)
# deserialized output should be: ['']

lst = ['a', 'bc', 'def']
serialized = serialize(lst)
# serialized output: "\x01a\x02bc\x03def"
deserialized = deserialize(serialized)
# deserialized output should be: ['a', 'bc', 'def']
```

Our solution serializes and deserializes a list of strings by encoding the length of each string as a single character and concatenating it with the original string. The serialize function processes the input list of strings by iterating through each string. For every string, it first encodes the length of the string as a single character using the encodeNumber function. The encoded length is then appended to a result list, followed by the string itself. The result list is ultimately joined into a single string, which represents the serialized form of the list of strings. The deserialize function takes the serialized string and reconstructs the original list of strings. It starts by iterating through the serialized string, decoding the length of each string using the decodeNumber function, then extracting the string of that length from the serialized data. These strings are collected in a list and returned as the final deserialized result. Time Complexity: The time complexity of both serialize and deserialize is O(n), where n is the total number of characters across all strings in the input list. In serialize, we iterate through every string, performing constant-time operations (encoding the length and appending the string) for each string. Similarly, in deserialize, we iterate through the serialized string, decoding lengths and extracting substrings, which are also constant-time operations for each step. Space Complexity: The space complexity is O(n) for both functions since we store the serialized form of the input list (which has a size proportional to the total number of characters across all strings) and reconstruct the list in deserialize.Daniel answers "Write a pair of functions to serialize and deserialize a list of strings."No answers contributed by the community yet.Related questionsWrite a function to do Excel-style operations.Write a function to check whether a string contains a substring.Given a list of bounding boxes and annotations, write a function to compute Intersection over Union.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

MEDIUM | Exponent | 93 of 196

Determine if two sentences are similar.

STUDY: TALK | Google frequency on LeetCode: all time 21, last 30 days 0 | book priority 132 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1813 Sentence Similarity III

1 interview

Original:
https://www.aced.io/questions/2280/determine-similar-sentences

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

APPROACH

Two pointers: match the longest common prefix and suffix of the word lists; similar iff the shorter sentence is fully covered by prefix + suffix.

INTERVIEW-DAY PLAN

Clarify first

Ask which version: similarity from a list of word pairs (LeetCode 734 or 737), and whether similarity is transitive (737) or not (734). If instead one sentence is the other with a block inserted in the middle (LeetCode 1813), say so and solve that. Ask about case and equal length.

Brute force to say out loud

For each position check the pair against the list by scanning it. O(n * P).

Optimal idea

Put the pairs in a set (both directions) for the non-transitive version: lengths must match and each word pair is equal or in the set, O(n + P). For the transitive version build a union-find over words and compare roots. For the insert-a-block version use two pointers to match the common prefix and the common suffix; the sentences are similar if the shorter one is fully covered.

Dry-run these cases

Different lengths, identical sentences, a pair that is only transitively similar (decides the version), duplicate pairs.

Likely follow-ups

Make similarity transitive, add weights, or allow at most one unmatched word.

Pitfalls

Assuming similarity is symmetric when the pairs are one-directional. Using lists instead of sets.

HOW TO THINK ABOUT IT

Ask which similarity you have: from a pair list, transitive or not. Without transitivity it is a set lookup per word pair. With transitivity, similar words form groups, which is union-find (or BFS on a word graph). The variant where a block is inserted is a different problem: match the common prefix and suffix of the word lists.

Basis: LeetCode 734 (similarity from a list of word pairs), 737 (the same with transitive similarity) and 1813 (one sentence is the other with a block inserted); the question text matches 734/737.

BRUTE FORCE - PSEUDO-CODE

```
if the sentences differ in length: return false
for each position: the words are similar if equal, or the
pair appears in the list
    (checking both orders by scanning the list)
```

Time: O(n * P) | Space: O(1)

Why it is slow

Each word pair scans the whole pair list, O(n * P). Putting the pairs into a set makes each check O(1).

OPTIMAL SOLUTION - SET OF PAIRS (NOT TRANSITIVE)

```
def areSentencesSimilar(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    known = set()
    for a, b in pairs:
        known.add((a, b))
        known.add((b, a))
    return all(a == b or (a, b) in known for a, b in zip(s
1, s2))
```

Time: O(n + P) | Space: O(P)

Why

Put each pair into a set in both directions, so a word pair is similar if the words are equal or the pair is in the set. Sentences of different length are never similar.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Similarity is transitive (a ~ b and b ~ c means a ~ c).

Union-find over the words: union each pair, then two words are similar if they are equal or have the same root. Path compression keeps it near O(n + P).

```
def areSentencesSimilarTwo(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    parent = {}

    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for a, b in pairs:
        parent[find(a)] = find(b)
    return all(a == b or find(a) == find(b) for a, b in zi
p(s1, s2))
```

One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813).

Match the longest common prefix and the longest common suffix of the word lists; the shorter list must be covered by prefix plus suffix.

```
def areSentencesSimilar_insert(s1, s2):
    if len(s1) > len(s2):
        s1, s2 = s2, s1
    i = 0
    while i < len(s1) and s1[i] == s2[i]:
        i += 1
    j = 0
    while j < len(s1) - i and s1[-1 - j] == s2[-1 - j]:
        j += 1
    return i + j >= len(s1)
```

Case should not matter, or words can have synonyms with scores.

Normalise with casefold before hashing. For scores, a graph with weighted edges and a threshold query turns it into reachability along strong edges.

OFFICIAL PAGE TEXT

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples

```
sentence1 = ["Let's", "code", "in", "Python"]
sentence2 = ["Let's", "program", "in", "Python"]
similarPairs = [
    ("code", "program"),
]
output: true

sentence1 = ["I", "love", "to", "play", "football"]
sentence2 = ["I", "love", "playing", "soccer"]
similarPairs = [("play", "playing"), ("football", "soccer"
)]
output: false, different sentence lengths

sentence1 = ["Do", "you", "like", "coffee"]
sentence2 = ["Do", "you", "love", "coffee"]
similarPairs = [
    ("like", "enjoy"),
    ("coffee", "tea"),
]
output: false  # "like" is not similar to "love" based on
the given pairs.

sentence1 = ["I", "really", "love", "leetcode", "and", "ap
ples"]
sentence2 = ["I", "so", "like", "codesignal", "and", "oran
ges"]
similarPairs = [
    ("very", "so"),
    ("love", "adore"),
    ("really", "very"),
    ("leetcode", "codesignal"),
    ("apples", "oranges"),
    ("like", "adore"),
]
output: true
```

The provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are connected if they are similar according to the provided pairs. Then, it performs a depth-first search (DFS) to check if each pair of corresponding words from the two sentences are connected in the graph. The Depth-First Search (DFS) algorithm is well-suited for this question as it efficiently explores the graph structure created from the similar word pairs. DFS is ideal for checking connectivity between words, which aligns with the task of determining if two words are similar based on the provided pairs. Its memory efficiency is beneficial for handling potentially large sets of sentences and word pairs. Additionally, DFS offers a simple implementation, either through recursion or using a stack, which aids in writing clean and understandable code. Its flexibility allows for easy modification to incorporate additional constraints or optimizations if needed. Therefore, DFS provides an elegant and effective solution for traversing the graph and determining sentence similarity. Remember to handle potential off-by-one errors when implementing this solution due to index-based iteration over the sentences. Time Complexity: Let n be the maximum number of words in the sentences, and m be the number of similar word pairs. Constructing the graph takes O(m) time. Then, for each word pair, the DFS takes O(n) time in the worst case, resulting in a total time complexity of O(n + m). Space Complexity: In the worst case, the graph can contain O(m) edges. Additionally, in the DFS, the space complexity is O(n) due to the recursion stack or auxiliary data structures to track visited nodes. Therefore, the overall space complexity is O(n + m).Watch Simon, software engineer at Google, answer the question "determine if two sentences are similar."No answers contributed by the community yet.Related questionsDetermine if a number is a power of two.Determine if two words are sorted in lexicographic order.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior

candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

---

MEDIUM | Exponent | 94 of 196

Count the number of ways to split a string.

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 163 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1573 Number of Ways to Split a String (likely) (inferred from short statement)

Original:
https://www.aced.io/questions/3256/count-ways-to-split-string

QUESTION

Count the number of ways to split a string.

APPROACH

Count the target character; split positions are products of the gaps between required groups. If it is a generic split count, use DP on prefixes. Clarify the problem first.

INTERVIEW-DAY PLAN

Clarify first

This prompt is vague, so ask first. The common version is LeetCode 1573: split a binary string into three non-empty parts with the same number of ones. Others are splits into equal parts, or splits into palindromes. State which you are solving.

Brute force to say out loud

Try every pair of cut positions and count ones in each part. O(n^3), or O(n^2) with prefix sums.

Optimal idea

For the three-part binary version count the total ones. If it is not divisible by 3 the answer is 0; if there are none the answer is C(n - 1, 2). Otherwise let k be ones per part; the number of zeros between the k-th and (k+1)-th one (gap1) and between the 2k-th and (2k+1)-th one (gap2) give (gap1 + 1) * (gap2 + 1) ways, taken modulo 10^9 + 7. O(n).

Dry-run these cases

All zeros of length 3, 4 and 5. A string with 3 ones. Ones not divisible by 3. Large n for modulo.

Likely follow-ups

Split into k parts, or count palindromic splits with DP.

Pitfalls

Forgetting the all-zeros special case. Not applying the modulus to the product.

HOW TO THINK ABOUT IT

Ask which split question it is before solving. For 'equal ones' the answer depends only on where the ones are: the cuts have to fall inside the stretches of zeros that separate the groups of ones, so the count is a product of (zeros in the gap + 1). All-zero strings are a special case (choose any two cut points).

Basis: The prompt is vague; the usual version is LeetCode 1573, Number of Ways to Split a String (a binary string into three non-empty parts with equal numbers of ones, modulo 10^9 + 7).

BRUTE FORCE - PSEUDO-CODE

```
prefix[i] = number of ones in the first i characters
for every pair of cuts i < j: if the three parts have equa
l numbers of ones: count =
    count + 1
```

Time: O(n^2) | Space: O(n)

Why it is slow

O(n^2) pairs. The cuts only depend on where the ones are, so the answer is a product of two gap sizes, computed in O(n).

OPTIMAL SOLUTION - COUNT THE ONES, MULTIPLY THE GAP SIZES

```
def numWays(s):
    ones = [i for i, ch in enumerate(s) if ch == "1"]
    n, m = len(s), len(ones)
    if m % 3:
        return 0
    if m == 0:
        return ((n - 1) * (n - 2) // 2) % MOD if n >= 3 el
se 0
    k = m // 3
    return ((ones[k] - ones[k - 1]) * (ones[2 * k] - ones[
2 * k - 1])) % MOD
```

Time: O(n) | Space: O(n)

Why

Let ones be the total number of ones. If it is not divisible by 3 the answer is 0; if it is 0, any two of the n - 1 cut positions work, C(n - 1, 2). Otherwise each part holds k = ones / 3 ones; the first cut can fall anywhere in the run of zeros between the k-th and (k + 1)-th one, and the second between the 2k-th and (2k + 1)-th one, so the answer is (gap1 + 1) * (gap2 + 1).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Split into m parts with equal ones.

The same argument: each of the m - 1 cut points sits in the gap after a multiple of k ones, so multiply the gap sizes; with no ones at all it is C(n - 1, m - 1).

```
def num_ways_m_parts(s, m):
    ones = [i for i, ch in enumerate(s) if ch == "1"]
    n, total = len(s), len(ones)
    if total % m:
        return 0
    if total == 0:
        return math.comb(n - 1, m - 1) % MOD if n >= m els
e 0
```

```
    k, result = total // m, 1
    for part in range(1, m):
        result = result * (ones[part * k] - ones[part * k
- 1]) % MOD
    return result
```

Count ways to split into palindromes.
That is a different problem: dp[i] counts splits of the prefix, adding dp[j] for every palindromic s[j:i]; precompute the palindromic substrings with expand-around-center or a DP table in O(n^2).

The numbers get huge.
Return the answer modulo 10^9 + 7 and apply the modulus after every multiplication; in languages with fixed-width integers use 64-bit intermediates. Cost: O(n) with modular arithmetic; no big integers are needed.

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

MEDIUM | IN APP LC 76 | Exponent | 95 of 196

★ TOP 75: rank 55 in the Medium and Hard priority list

★ Find the minimum window substring.

STUDY: TYPE | Google frequency on LeetCode: all time 42.5, last 30 days 33 | book priority 32 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #76)

LeetCode match: LC 76 Minimum Window Substring (inferred from short statement)

+36 interviews

Original:
https://www.aced.io/questions/1687/minimum-window-substring

QUESTION

Find the minimum window substring.

APPROACH

Sliding window with need/have counts: expand right until valid, then shrink left while valid, tracking the best window. O(n).

HOW TO THINK ABOUT IT

Expand the right end until the window covers everything in t, then shrink from the left as far as it stays valid, record the length, and continue. A count of 'characters still missing' makes the validity check O(1): it goes down when a needed character is added and up when one is removed. Each index enters and leaves the window once.

Basis: LeetCode 76, Minimum Window Substring: the shortest substring of s containing every character of t with multiplicity.

BRUTE FORCE - PSEUDO-CODE

```
for every substring of s: if it contains every character o
f t with the right counts:
    keep it if it is the shortest
```

Time: O(n^2 * m) | Space: O(m)

Why it is slow
O(n^2) substrings, each checked in O(m) or more. A sliding window that grows until valid and then shrinks from the left is O(n + m).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #76 MINIMUM WINDOW SUBSTRING)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Smallest window in s containing all characters of t (with multiplicity).
Return '' if no such window. Right?
t may have duplicate chars — window must have at least that many."

"s='ADOBECODEBANC', t='ABC' -> 'BANC' (contains A,B,C, length 4) ok"
Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 3 — OPTIMIZE (sliding window with 'formed' counter)

Say: "Sliding window with formed counter — O(m+n).
Walk-through:
required = Counter(t). window = {}. formed=0. left=0.
best=(inf,-1,-1).
For right in 0..m-1:
c = s[right]. window[c] = window.get(c,0)+1.
If c in required and window[c] == required[c]: formed++.
While formed == len(required): <- window is valid
Update best if right-left+1 < best[0].
Shrink left: lc=s[left]. window[lc]-=1. If lc in required and window[lc]<required[lc]: formed--.
left++.
Return s[best[1]:best[2]+1] or ''.
WHY 'formed' counter: tracks how many chars in t have their required count satisfied.
When formed == len(distinct chars in t): all requirements met -> valid window.
WHY shrink while valid: find the minimum window.
Time: O(m+n). Space: O(|charset|).

PHASE 4 — CLEAN CODE

```python
class _76_MinWindowSubstring:
    def minWindow(self, s: str, t: str) -> str:
        if not t or not s:
            return ''
        need    = Counter(t)
        have    : Counter = Counter()
        formed  = 0
        required = len(need)
        left    = 0
        best    = (float('inf'), 0, 0)    # (length, left, right)

        for right in range(len(s)):
            c = s[right]
            have[c] += 1
            if c in need and have[c] == need[c]:
                formed += 1
            while formed == required:
                if right - left + 1 < best[0]:
                    best = (right - left + 1, left, right)
                have[s[left]] -= 1
                if s[left] in need and have[s[left]] < need[s[left]]:
                    formed -= 1
                left += 1

        return '' if best[0] == float('inf') else s[best[1]: best[2] + 1]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace minWindow('ADOBECODEBANC','ABC')."
need={A:1,B:1,C:1}. window={}. have=0. left=0. min_len=inf.
Expand right: A->have=1. D,O,B->have=2. E,C->have=3 (formed!).
At right=5 (C): window='ADOBEC'. Shrink: remove A->have=2. left=1.
Expand: O,D,E,B->window='DOBECODEB'. Right hits A->have=3. Window='DOBECODEBANC'. Shrink: D(not needed),O,B->now have=2.
... eventually window='BANC' -> min_len=4.
Return 'BANC' ok
WHY have/need: track how many distinct chars are fully satisfied in window. O(n) via lazy shrink.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(m+n). Space O(m+n). The 'formed' counter avoids comparing the full frequency maps.
A window is valid exactly when formed==required — elegant O(1) validity check.
Given more time I'd ask: find all minimum windows?
Collect all windows where length == best length during the BFS scan.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The smallest window containing every distinct character of s itself.
Use t = the set of distinct characters of s and run the same window.
A related problem: the longest substring with at most k distinct characters uses a shrink-when-too-many window.

```python
def minWindow(s, t):
    if not t:
        return ""
    need, missing = Counter(t), len(set(t))
    best, left = "", 0
    for right, ch in enumerate(s):
        if ch in need:
            need[ch] -= 1
            if need[ch] == 0:
                missing -= 1
        while missing == 0:
            if best == "" or right - left + 1 < len(best):
                best = s[left : right + 1]
            out = s[left]
            if out in need:
                need[out] += 1
                if need[out] > 0:
                    missing += 1
            left += 1
    return best


def smallest_window_all_distinct(s):
    return minWindow(s, "".join(sorted(set(s))))
```

Return all minimum windows, or the number of valid substrings.
For all minimum windows record every window equal to the best length. For counting, when the window [left, right] is valid, every extension to the right is valid too, so add len(s) - right for each left.

Order matters: t must appear as a subsequence.
That is a different problem (LeetCode 727): DP over positions, or a two-pass scan (forward to match, backward to shrink) for each possible end.

OFFICIAL PAGE TEXT

Upgrade to see our video answerBecome a member today100+ expert answer videosAccess to all coursesPeer-to-peer practicePrivate Discord communityFull question databaseUpgrade nowLewin (Dropbox SWE) answers the minimum window substring interview question.

MEDIUM | IN APP LC 249 | Exponent | 96 of 196

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

STUDY: TALK | Google frequency on LeetCode: all time 35.4, last 30 days 0 | book priority 98 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #249)

LeetCode match: LC 249 Group Shifted Strings (inferred from short statement)

Original: https://www.aced.io/questions/6356/group-buddy-strings-same-distance

QUESTION

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

APPROACH

Normalize each string by its successive character differences (mod 26) as a tuple key, then group by key in a hash map. O(total chars).

HOW TO THINK ABOUT IT

Buddies are shifted copies of each other, so what they share is the shape: the gaps between neighbouring letters. Turn each string into its shape (a tuple of gaps mod 26) and group by it with a dictionary; you never compare two strings directly. Mention that single letters all have the same (empty) shape.

Basis: LeetCode 249, Group Shifted Strings.

BRUTE FORCE - PSEUDO-CODE

```
groups = []
for each string s: look through the groups; if s has the same length and a constant
    letter shift from the group's first string, add s to it
if no group fits: start a new group containing s
```

Time: O(n * groups * L) | Space: O(n)

Why it is slow

Each string is compared with the representative of every group, O(n * groups * L). Using the sequence of gaps between neighbouring letters as a dictionary key groups everything in one pass.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #249 GROUP SHIFTED STRINGS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Two strings are in the same shifting sequence if one can become the other
by consistently shifting every character by the same amount (cyclically, a->z wraps).
Group them. Return order doesn't matter. Right?
'abc' and 'xyz' are in the same group — same relative gaps between characters."

"['abc','bcd','xyz','az','ba','a','z']:
'abc','bcd','xyz' -> all have gaps [1,1] -> same group.
'az','ba' -> gaps [25] and [25] (b-a=1… wait: 'az': z-a=25. 'ba': a-b=-1+26=25) -> same group.
'a','z' -> single char, empty gap string -> same group."
Are string comparisons case-sensitive?

PHASE 3 — OPTIMIZE

Say: "Normalize each word to shift-0 key, group by key — O(nL).
Walk-through:
For each word:
key = tuple(((ord(c)-ord(word[0]))%26) for c in word).
groups[key].append(word).
Return list(groups.values()).
WHY shift relative to first char: words in the same group have the same relative differences.
'abc' -> (0,1,2). 'bcd' -> (0,1,2). Same key -> same group.
WHY %26: wrap around alphabet (e.g., 'xyz' -> 'z'-'x'=2, 'a'-'x'=3, 'b'-'x'=4 -> (0,3,4) — wraps correctly).
Wait: (ord('a')-ord('z'))%26 = (-25)%26 = 1. Correct wraparound!
Time: O(nL) where L = max word length. Space: O(nL)."
Time: O(n) — optimal single-pass or O(n²) DP. Linear preferred.

PHASE 4 — CLEAN CODE

```python
class _249_GroupShiftedStrings:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for s in strings:
            key = tuple((ord(s[i]) - ord(s[i - 1])) % 26 for i in range(1, len(s)))
            groups[key].append(s)
        return list(groups.values())
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace groupStrings(['abc','bcd','acef','xyz','az','ba','a','z'])."
Normalize each string to a 'delta' key: differences between consecutive chars mod 26.
'abc': (b-a)%26=1, (c-b)%26=1 -> key=(1,1).
'bcd': (c-b=1),(d-c=1) -> key=(1,1). Same group as 'abc' ok
'xyz': (y-x=1),(z-y=1) -> key=(1,1). Same group!
'az': (z-a=25) -> key=(25,). 'ba': (a-b+26=25) -> key=(25,). Same group ok (shift by 1).
'a','z': key=() (single chars). Same group ok
Groups: {(1,1):['abc','bcd','xyz'],(25,):['az','ba'],():['a','z'],(...):['acef']}

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n*L) where L = max string length. Space O(n*L).
The %26 mod handles the wraparound: 'z'->'a' gives (ord('a')-ord('z'))%26 = (-25)%26 = 1.

Without the mod, 'az' and 'ba' wouldn't match — the mod is the key insight.
Given more time I'd ask: how do you efficiently check if two strings are in the same group
at query time? Build a map string->canonical_key at init, then O(L) per query.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if shifting is not cyclic (z does not wrap to a)?

Drop the % 26 and use the raw differences as the key. Two strings with the same raw gaps differ by one fixed integer shift, and both are already inside 'a'..'z', so that shift is automatically legal: the gap tuple alone decides buddies, with no extra range check. What changes is that 'az' and 'ba' are no longer buddies (gaps 25 and -1 instead of 25 and 25).

How would you do this with strings that are too many for one machine?

Compute the gap key on each machine, hash-partition the strings by key (so every buddy lands on the same worker), and group locally. The key is the only thing exchanged, so shuffle cost is one pass over the data. Cost: O(total characters / machines) per machine to compute the keys, plus one shuffle of the keys; each group is then built locally in O(group size).

Shifting does not wrap around (z stays z).

Use the raw gaps (no % 26) as the key. Equal raw gaps mean one fixed integer shift, which is automatically legal because both strings are already letters, so no extra check is needed.

```
def group_buddies_noncyclic(strings):
    groups = defaultdict(list)
    for s in strings:
        groups[tuple(ord(s[i]) - ord(s[i - 1]) for i in ra
nge(1, len(s)))].append(s)
    return list(groups.values())
```

MEDIUM | 1Point3Acres | onsite-coding | 97 of 196

Cutting Ribbons (LC 1891)

STUDY: TALK | Google frequency on LeetCode: all time 19.6, last 30 days 0 | book priority 135 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.S WEarraybinary-searchcomplexity-analysismediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

Clarify first

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

Brute force to say out loud

Try every length from the maximum down to 1 and count pieces. O(n * max).

Optimal idea

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

Dry-run these cases

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

Likely follow-ups

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

Pitfalls

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE - PSEUDO-CODE

```
for L from the longest ribbon down to 1:
    pieces = sum of (ribbon // L) over all ribbons
    if pieces >= K: return L
return 0
```

Time: O(n * max) | Space: O(1)

Why it is slow

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION - BINARY SEARCH THE ANSWER (INTEGER LENGTHS)

```
def maxLength(ribbons, k):
    lo, hi, best = 1, max(ribbons), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if sum(r // mid for r in ribbons) >= k:
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best
```

Time: O(n log(max length)) | Space: O(1)

Why

If you can cut k pieces of length L you can cut k pieces of any shorter length, so feasibility is monotone. Binary search the largest L in [1, max(ribbons)] with sum(r // L) >= k, and return 0 if even L = 1 fails.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Floating-point lengths with a fixed iteration count

```
def maxLengthFloat(ribbons, k, iterations=60):
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if mid > 0 and sum(int(r // mid) for r in ribbons)
 >= k:
            lo = mid
        else:
            hi = mid
    return lo
```

Time: O(n * iterations) | Space: O(1)

Why

With real-valued lengths there is no 'next integer', so search the real line and stop after a fixed number of halvings (each halves the error) instead of looping on an epsilon comparison, which can fail to terminate near rounding noise. Feasibility uses floor(r / L) as before.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. Cost: each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

```
def maxLength_tight(ribbons, k):
    lo, hi, best = 1, min(max(ribbons), sum(ribbons) // k)
, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if sum(r // mid for r in ribbons) >= k:
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied? Expect the follow-ups, not the base implementation, to consume most of the round.

For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.

Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.

Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

MEDIUM | 1Point3Acres | onsite-coding | 98 of 196

Union of Sorted Interval Lists

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 127 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

Clarify first

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

Brute force to say out loud

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

Optimal idea

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

Dry-run these cases

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

Likely follow-ups

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

Pitfalls

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE - PSEUDO-CODE

```
all = list1 + list2, sort all by start
merged = []; for each interval: if it starts at or before
the end of the last merged
    one, extend that end; else append it
```

Time: O((m + n) log (m + n)) | Space: O(m + n)

Why it is slow

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION - TWO-POINTER MERGE

```
def union_two(a, b):
    out, i, j = [], 0, 0
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i][0] <= b[j][
0]):
            cur = a[i]
            i += 1
        else:
            cur = b[j]
            j += 1
        if out and cur[0] <= out[-1][1]:
            out[-1][1] = max(out[-1][1], cur[1])
        else:
            out.append([cur[0], cur[1]])
    return out
```

Time: O(m + n) | Space: O(m + n) for the output

Why

Always take the interval with the smaller start next. If it touches or overlaps the last output interval (start <= last end, closed intervals) extend that one, otherwise append it. Invariant: everything before the pointers is already folded into a canonical output.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

K lists with a heap (streaming, O(K) memory)

```
def union_k(lists):
    last = None
    for start, end in heapq.merge(*lists):
        if last and start <= last[1]:
            last[1] = max(last[1], end)
        else:
            if last:
                yield tuple(last)
            last = [start, end]
    if last:
        yield tuple(last)
```

Time: O(N log K) | Space: O(K) beyond the output

Why

heapq.merge keeps one pointer per list in a heap and yields intervals in start order without materializing the inputs, so it works for lists too large for memory. The same fold step builds the union lazily.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

```
def union_sort(a, b):
    out = []
    for s, e in sorted(a + b):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out


def union_two(a, b):
    out, i, j = [], 0, 0
    while i < len(a) or j < len(b):
        if j == len(b) or (i < len(a) and a[i][0] <= b[j][
0]):
            cur = a[i]
            i += 1
        else:
            cur = b[j]
            j += 1
        if out and cur[0] <= out[-1][1]:
            out[-1][1] = max(out[-1][1], cur[1])
        else:
            out.append([cur[0], cur[1]])
    return out


def union_unsorted_lists(a, b):
    """Follow-up: a list may overlap itself and need not b
e sorted: sort each list first (O(n log n)), then merge as
 before."""
    return union_two(union_sort(a, []), union_sort(b, []))
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap.
For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

MEDIUM | 1Point3Acres | phone-screen | 99 of 196

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

STUDY: TYPE | Google frequency on LeetCode: all time 41.1, last 30 days 33 | book priority 41 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search — given an `isBadVersion(v)` oracle over versions `1..n`, find the first bad version with a single binary search. The graded follow-up requires parallelizing the search: partition the range into buckets and search them concurrently (bucket + binary search), with working code expected. SWEbinary-searchparallelismconcurrencymediumFrequencySingle reportLast asked2026-06-15Stagephone-screen

APPROACH

Write the single-threaded binary search first and verify it on small n. Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

INTERVIEW-DAY PLAN

Clarify first

Ask what the oracle is (isBadVersion is monotone), what concurrency primitive to use (ThreadPoolExecutor, Java's ExecutorService), and which metric matters: wall-clock time or total calls.

Brute force to say out loud

Check every version from 1. O(n) calls.

Optimal idea

Base: binary search, O(log n) calls. Parallel: each round fires up to k probes concurrently at evenly spaced points, finds the first probe that is bad, and narrows the interval to the gap between it and the previous probe. Each round shrinks the interval by k + 1, so there are about log base (k + 1) of n sequential rounds. Use a thread pool and map the probes in one call.

Dry-run these cases

n = 1, the first version bad, the last version bad, n smaller than the number of workers.

Likely follow-ups

Rate limiting, retries when a probe fails, and caching results.

Pitfalls

Mutating shared state from threads. Assuming more threads always means fewer total calls (they increase it).

HOW TO THINK ABOUT IT

'First bad version' is a boundary in a sorted yes/no sequence, which is binary search. The parallel part is a different trade: instead of one probe per round, send several probes in the same round and keep the gap that contains the boundary. Fewer rounds, more total calls.

Basis: LeetCode 278, First Bad Version, plus a concurrent k-ary search follow-up.

BRUTE FORCE - PSEUDO-CODE

```
for v = 1, 2, 3, ...: if isBadVersion(v): return v
```

Time: O(n) calls | Space: O(1)

Why it is slow

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION - BASE PROBLEM: BINARY SEARCH

```python
def first_bad_version(n, is_bad):
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Time: O(log n) calls | Space: O(1)

Why

isBad is monotone (all good, then all bad), so keep the invariant 'the first bad version is in [lo, hi]' and halve the interval. Use lo + (hi - lo) // 2 in languages with fixed-width integers.

OPTIMAL SOLUTION - PARALLEL: PROBE K POINTS AT ONCE, THEN RECURSE INTO THE BUCKET THAT FLIPS

```python
def first_bad_version_parallel(n, is_bad, workers=4):
    lo, hi = 1, n  # invariant: the answer is in [lo, hi]; hi is bad
    with ThreadPoolExecutor(max_workers=workers) as pool:
        while lo < hi:
            step = max(1, (hi - lo + 1) // (workers + 1))
            points = sorted({p for p in range(lo + step - 1, hi, step)})[:workers]
            if not points:
                points = [lo]
            results = list(pool.map(is_bad, points))
            first_true = next((i for i, r in enumerate(results) if r), None)
            if first_true is None:
                lo = points[-1] + 1
            else:
                hi = points[first_true]
                if first_true > 0:
                    lo = points[first_true - 1] + 1
    return lo
```

Time: O(log_{k+1} n) rounds | Space: O(k)

Why

Each round fires up to `workers` probes concurrently at evenly spaced points. Because the predicate is monotone, exactly one gap between consecutive probes contains the flip, so the interval shrinks by a factor of workers + 1 per round. This trades more total calls for fewer sequential rounds, which is the right trade when each probe is slow (a network call).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

```python
def first_bad_version_counting(n, is_bad, workers=4):
    rounds = calls = 0
    lo, hi = 1, n
    while lo < hi:
        step = max(1, (hi - lo + 1) // (workers + 1))
        points = sorted(set(range(lo + step - 1, hi, step)
))[:workers] or [lo]
        results = [is_bad(p) for p in points]
        rounds += 1
        calls += len(points)
        first_true = next((i for i, r in enumerate(results
) if r), None)
        if first_true is None:
            lo = points[-1] + 1
        else:
            hi = points[first_true]
            if first_true:
                lo = points[first_true - 1] + 1
    return lo, rounds, calls
```

NOTES FROM CANDIDATES

The base is trivial; budget your time for the parallel follow-up and aim to get a complete, compiling version through even if it has minor bugs — finishing the parallel version is what passes this round.
Be explicit about what the parallel design optimizes: wall-clock latency (fire many isBadVersion probes at once) versus total number of calls. Splitting into k buckets and probing each boundary concurrently localizes the transition in one parallel round, then a binary search finishes it.
Common bugs: off-by-one when mapping a bucket index back to an absolute version range; the transition sitting exactly on a bucket boundary; integer overflow in mid if you write (lo + hi) / 2 instead of lo + (hi - lo) // 2.

PREPARATION

Write the single-threaded binary search first and verify it on small n. Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 100 of 196

Run-Length Vector Storage + Dot Product

STUDY: TALK | Google frequency on LeetCode: all time 5, last 30 days 0 | book priority 159 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, Infra Eng

Original: https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

QUESTION

Examples:

```
Logical A = [3, 3, 3, 0, 0, 0, 4, 4]
Logical B = [1, 1, 1, 1, 2, 2, 2, 2]
Encoded A (RLE): [(3,3),(0,3),(4,2)]
Encoded B (RLE): [(1,4),(2,4)]
Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 +
0 + 16 = 25
```

APPROACH

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

INTERVIEW-DAY PLAN

Clarify first

Ask for the encoding format (value, count pairs), whether runs of zeros are stored, the logical length of both vectors (must be equal), and whether updates are needed.

Brute force to say out loud

Expand both vectors and multiply element by element. O(n) memory.

Optimal idea

Two pointers over the run lists. Take the overlap, which is min of the two remaining counts, add value_a * value_b * overlap, subtract the overlap from both, and advance whichever run is exhausted. O(r1 + r2), O(1) extra space.

Dry-run these cases

The example giving 25. One vector of a single run. Runs that end at the same time. All zeros.

Likely follow-ups

A sparse vector against an RLE vector, setting an element, or dot product of k vectors.

Pitfalls

Advancing only one pointer when both runs finish together. Overflowing the product for large counts.

HOW TO THINK ABOUT IT

A run is a stretch where both numbers are constant, so the dot product of two runs is value times value times the length of their overlap. Walk the two run lists like the merge step of merge sort: consume the smaller remaining length from both, and advance whichever run you finished.

Basis: LeetCode 1570, Dot Product of Two Sparse Vectors, with run-length encoding as the compression.

BRUTE FORCE - PSEUDO-CODE

```
dense_a = every run (value, count) written out as count co
pies of value; same for b
answer = sum of dense_a[i] * dense_b[i] over all positions
```

Time: O(logical length) | Space: O(logical length)

Why it is slow

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

OPTIMAL SOLUTION - TWO-POINTER OVER RUNS

```
def rle_dot(a, b):
    i = j = 0
    ra = a[0][1] if a else 0
    rb = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        take = min(ra, rb)
        total += a[i][0] * b[j][0] * take
        ra -= take
        rb -= take
        if ra == 0:
            i += 1
            ra = a[i][1] if i < len(a) else 0
        if rb == 0:
            j += 1
            rb = b[j][1] if j < len(b) else 0
    return total
```

Time: O(r1 + r2) | Space: O(1)

Why

Each run is (value, count). Compare the remaining lengths of the two current runs, multiply the values by the overlap, then advance whichever run was exhausted (or both). Never expand the vectors: the work depends on the number of runs, not the logical length.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

```
def dot_sparse_rle(sparse, runs):
    """sparse: {index: value}; runs: RLE list. Binary sear
ch the run that contains each index."""
    import bisect
    starts, pos = [], 0
    for _, c in runs:
        starts.append(pos)
        pos += c
    total = 0
    for idx, v in sparse.items():
        i = bisect.bisect_right(starts, idx) - 1
        if 0 <= i < len(runs) and idx < starts[i] + runs[i
][1]:
            total += v * runs[i][0]
    return total
```

NOTES FROM CANDIDATES

Two-pointer walk over the two RLE encodings: at each step compute the overlap length min(remainA, remainB), multiply values, advance the pointer whose run is exhausted.

Mention compression-friendly variations: sparse format wins when most entries are 0 (just enumerate non-zero indices and skip zeros entirely in the dot product).

Interviewer cared more about clean data-structure choice and the walk over both encodings than about absolute optimality.

PREPARATION

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).

Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).

Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

MEDIUM | 1Point3Acres | onsite-coding | 101 of 196

Rotate Any m×n 2D Matrix

STUDY: TYPE | Google frequency on LeetCode: all time 56.3, last 30 days 0 | book priority 50 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

Examples:

[[1,2,3],[4,5,6]] (2×3) rotated 90° CW -> [[4,1],[5,2],[6,3]] (3×2).

APPROACH

Write square in-place rotation from memory in under 8 min.

Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

INTERVIEW-DAY PLAN

Clarify first

Ask which direction and angle, whether in place is required, and the dimensions. A non-square matrix cannot be rotated in place in the same buffer, since its shape changes.

Brute force to say out loud

Create a new matrix and copy each cell to its rotated position with an index formula. O(mn) space.

Optimal idea

Clockwise 90 degrees: reverse the rows and transpose, [list(r) for r in zip(*matrix[::-1])]. Counter-clockwise: transpose then reverse the rows. 180 degrees: reverse the rows and each row. For a square matrix rotate in place by cycling four cells per layer.

Dry-run these cases

The 2 x 3 example giving [[4,1],[5,2],[6,3]], 1 x 1, 1 x n, an empty matrix, four rotations returning the original.

Likely follow-ups

In place square rotation, rotation by k times 90 degrees, or non-square in place with a flat array and cycles.

Pitfalls

Mixing up the index formula direction. Trying to rotate a rectangle in place in a 2D list.

HOW TO THINK ABOUT IT

A 90 degree clockwise rotation sends the cell at row i, column j to row j, column (m - 1 - i) of the new n x m matrix. That formula is the whole problem; 'reverse the rows then transpose' is just a short way to write it. A rectangle cannot be rotated in place because its shape changes.

Basis: Generalizes LeetCode 48, Rotate Image, from an n x n matrix to any m x n matrix.

BRUTE FORCE - PSEUDO-CODE

```
new = an n x m matrix
for every cell (i, j): new[j][m - 1 - i] = old[i][j]
return new
```

Time: O(m * n) | Space: O(m * n)

Why it is slow

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION - ROTATE 90 DEGREES CLOCKWISE BY REVERSE + TRANSPOSE

```python
def rotate_cw(matrix):
    return [list(row) for row in zip(*matrix[::-1])]
```

Time: O(mn) | Space: O(mn)

Why

Rotating clockwise puts the last row first, so reverse the rows and transpose (zip with *). A non-square matrix changes shape from m x n to n x m, which is why it cannot be done in place the way LeetCode 48 is.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Other angles, and the in-place square case

```python
def rotate_ccw(matrix):
    return [list(row) for row in zip(*matrix)][::-1]
```

Time: O(mn) | Space: O(1) for the in-place square version

Why

Counter-clockwise: transpose then reverse the rows (or reverse each row, then transpose). 180 degrees: reverse the rows and each row. Square matrices can be rotated in place by cycling four cells at a time, layer by layer.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

```python
def rotate_cw(matrix):
    return [list(row) for row in zip(*matrix[::-1])]


def rotate_k(matrix, k):
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix
```

NOTES FROM CANDIDATES

The in-place rectangle rotation is essentially cycle-decomposition on the index permutation — explain it but don't burn time implementing. Reported follow-up: rotate inside a window of a larger matrix while keeping the rest untouched.

PREPARATION

Write square in-place rotation from memory in under 8 min.
Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

MEDIUM | HackTheRounds | Onsite | 102 of 196

★ TOP 75: rank 66 in the Medium and Hard priority list

★ Shortest Subarray with K Distinct Integers

STUDY: TYPE | Google frequency on LeetCode: all time 42.5, last 30 days 33 | book priority 33 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #992 Subarrays with K Different Integers, LeetCode #76 Minimum Window Substring

LeetCode match: LC 992 / LC 76 flavor

QUESTION

Given an array nums and an integer k, find the length of the shortest contiguous subarray that contains at least k distinct integers. If no such subarray exists, return -1.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 1, 2, 3], k = 3`
Output	`3`
Why	`The shortest contiguous subarray with 3 distinct integers is [1, 2, 3].`

Example 2	
Input	`nums = [1, 1, 1, 1], k = 2`
Output	`-1`
Why	`Only 1 distinct integer exists.`

Example 3	
Input	`nums = [1, 2, 3, 4, 5], k = 3`
Output	`3`
Why	`Any 3 consecutive elements give 3 distinct integers.`

CONSTRAINTS

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^9`
- `1 <= k <= nums.length`

APPROACH

Sliding window with a frequency hash map, O(n); shrink while distinct >= k

INTERVIEW-DAY PLAN

Clarify first

Ask for the output if none exists (-1), whether k can exceed the number of distinct values, and whether the subarray must be contiguous (yes).

Brute force to say out loud

Check every subarray and count distinct values. O(n^3), or O(n^2) with a running set.

Optimal idea

Sliding window with a count dict. Move right to add elements; whenever the window has at least k distinct values, record its length and then shrink from the left while it still has at least k distinct, updating the answer each time. Each index enters and leaves once, so O(n).

Dry-run these cases

k = 1 (answer 1), k larger than distinct values (-1), all equal values, the shortest window in the middle, k = 0.

Likely follow-ups

Longest subarray with at most k distinct, count of subarrays with exactly k distinct (LeetCode 992 using at-most minus at-most).

Pitfalls

Shrinking the window before recording the answer. Not deleting keys when counts reach zero.

HOW TO THINK ABOUT IT

Once a window has at least k distinct values, shrinking it from the left can only keep or lose distinct values, so the shortest valid window ending at each right end is found by shrinking as far as possible. A frequency map tells you when a value leaves the window, so each index enters and leaves once.

Basis: Sliding window (compare LeetCode 992 and 340). Shortest contiguous subarray with at least k distinct values; -1 if none.

BRUTE FORCE - PSEUDO-CODE

```
for every start i: extend the end j one element at a time,
 keeping a set of the values
    seen
when the set reaches k values: record j - i + 1 and stop e
xtending
```

Time: O(n^2) | Space: O(n)

Why it is slow

O(n^2) in the worst case. A sliding window with a count map expands on the right and shrinks on the left, so each element is handled twice: O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: SLIDING WINDOW

```python
from typing import List
from collections import defaultdict


class Solution:
    def shortestSubarrayWithKDistinct(self, nums: List[int
], k: int) -> int:
        if len(nums) < k:
            return -1

        count = defaultdict(int)
        left = 0
        result = float("inf")

        for right, num in enumerate(nums):
            count[num] += 1

            while len(count) >= k:
                if len(count) == k:
                    result = min(result, right - left + 1)
```

```
                count[nums[left]] -= 1
                if count[nums[left]] == 0:
                    del count[nums[left]]
                left += 1

        return result if result != float("inf") else -1
```

Time: O(n) | Space: O(k)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Count the subarrays with exactly k distinct values (LeetCode 992).
exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding window that shrinks while the distinct count exceeds the limit. Both are O(n).

```
def count_subarrays_exactly_k(nums, k):
    def at_most(m):
        count, left, total = {}, 0, 0
        for right, x in enumerate(nums):
            count[x] = count.get(x, 0) + 1
            while len(count) > m:
                y = nums[left]
                count[y] -= 1
                if count[y] == 0:
                    del count[y]
                left += 1
            total += right - left + 1
        return total

    return at_most(k) - at_most(k - 1)
```

Longest subarray with at most k distinct values.
The same window, but keep the longest window whose distinct count never exceeds k (shrink while it does). O(n).

The values are strings or large objects.
The frequency map works on any hashable value; for huge objects hash them first or map them to integer ids so the window comparisons stay cheap. Cost: O(n) plus hashing each element; mapping values to integer ids costs O(n) once.

ALSO REPORTED - 2 MORE REPORTS OF THIS QUESTION

Find shortest subarray with at least k distinct | PracHub | Technical Screen
https://prachub.com/coding-questions/find-shortest-subarray-with-at-least-k-distinct

Find shortest subarray with >=k distinct integers | PracHub | Technical Screen
https://prachub.com/coding-questions/find-shortest-subarray-with-k-distinct-integers

---

MEDIUM | PracHub | Technical Screen | 103 of 196

Check if all substrings are dictionary words

STUDY: TALK | **Google frequency on LeetCode: all time 43.5, last 30 days 0 | book priority 74 of 196**
Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns
Optimal answer from: the interview site's own solution
Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break
LeetCode match: No direct match; related LC 139 Word Break
Original: https://prachub.com/coding-questions/check-if-all-substrings-are-dictionary-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

APPROACH

Enumerate all substrings of length >= 3 and look each up in the set; exit on the first failure. Prune: if any length-3 window fails return false immediately. O(n^2) lookups.

INTERVIEW-DAY PLAN

Clarify first
Ask for the dictionary access (a set or a trie), the maximum length of s, and whether case matters. A string of length n has about n^2 / 2 substrings of length at least 3, and each is a separate requirement.

Brute force to say out loud
Enumerate every substring of length at least 3 and look it up. O(n^2) lookups, each costing O(length) to hash, so O(n^3) characters.

Optimal idea
Exit on the first failure; checking all length-3 windows first prunes most inputs. To avoid the length factor, walk a trie: for each start i descend the trie along s[i:], and at every depth of at least 3 require the end-of-word flag; if the path breaks, return false. That is O(n^2) node steps in total.

Dry-run these cases
A string shorter than 3 (true vacuously), cats (needs cat, ats, cats), a failure on the first window, a failure only on the longest substring.

Likely follow-ups
Anagram version (next question), or return the first failing substring.

Pitfalls
Checking only the whole string. Treating a string shorter than 3 as false.

HOW TO THINK ABOUT IT

Every substring is a separate requirement, so no clever skipping exists: you must test about n^2 / 2 substrings. What you can improve is the cost per test and the early exit. A trie lets you extend a start position one character at a time without rehashing the whole substring, and the first failure ends everything.

Basis: Written; every contiguous substring of length >= 3 must be a dictionary word. The dictionary is a set (optionally preprocessed into a trie).

BRUTE FORCE - PSEUDO-CODE

```
for every substring of length 3 or more: if it is not in t
he dictionary set: return
    false
return true
```

Time: O(n^2) lookups, O(n^3) characters hashed | Space: O(1) extra

Why it is slow

About n^2 / 2 lookups and each hashes a whole substring, O(n^3) characters. Walking a trie from each start, one character at a time, is O(n^2) with no hashing of long strings.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(s, dictionary):
    word_set = set(dictionary)
    n = len(s)

    def backtrack(start, path):
        if len(path) >= 3 and path not in word_set:
            return False

        used = set()
        for i in range(start, n):
            ch = s[i]
            if ch in used:
                continue
            used.add(ch)
            if not backtrack(i + 1, path + ch):
                return False
        return True

    return backtrack(0, "")
```

Time: O(U * n), where U is the number of distinct subsequences of s | Space: O(n) auxiliary recursion space, excluding the dictionary set

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

```python
def first_failing_substring(s, words):
    for length in range(
        3, len(s) + 1
    ):  # shortest first, so the report is the smallest co
unterexample
        for i in range(len(s) - length + 1):
            if s[i : i + length] not in words:
                return s[i : i + length]
    return None
```

The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. Cost: still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

FUNCTION AND TEST CASES

Signature

```python
def solution(s: str, dictionary: list[str]) -> bool:
```

Sample call

```python
solution('cat', ['cat'])
```

Test cases

Input	Expected
`('cat', ['cat'])`	`True`
`('cats', ['cat', 'cas', 'cts', 'ats', 'cats'])`	`True`

MEDIUM | PracHub | Technical Screen | 104 of 196

★ TOP 75: rank 16 in the Medium and Hard priority list

★ Find Containing Range

STUDY: TYPE | Google frequency on LeetCode: all time 65.2, last 30 days 41.1 | book priority 11 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: No direct match; related LC 56 Merge Intervals + binary search (inferred from overview)

Original: https://prachub.com/coding-questions/find-containing-range

QUESTION

Overview: This question evaluates the ability to design appropriate data structures and apply algorithmic reasoning for efficient interval containment queries, emphasizing preprocessing, correctness, and time-space trade-offs.

APPROACH

Sort/merge ranges once, then answer each query with binary search on starts and check the end. O(n log n) preprocessing, O(log n) per query.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the given ranges can overlap, what to return when a point lies in several ranges (the original range, the merged range, or all of them), and what to return when none contains it.

Brute force to say out loud

For each query scan all ranges. O(n) per query.

Optimal idea

Sort the ranges by start and merge overlaps once, O(n log n). For each query binary search for the last merged range whose start is at most the point, then check that the point is at most its end. O(log n) per query. If the original (unmerged) range is required and ranges may overlap, use an interval tree or keep the maximum end seen in a prefix array.

Dry-run these cases

A point before all ranges, after all ranges, in a gap, exactly on an endpoint, inside two overlapping ranges.

Likely follow-ups

Insert and delete ranges (a balanced tree), or many point queries at once (sort and sweep).

Pitfalls

Binary searching without merging when ranges overlap. Mishandling the endpoint rule.

HOW TO THINK ABOUT IT

Merge the ranges once so they are disjoint and sorted; then 'which range contains x' is 'the last range whose start is at most x, and does its end reach x'. That is one binary search per query. Be clear whether the answer is the original range or the merged one, since they differ when ranges overlap.

Basis: Written (compare LeetCode 56 and a binary search for the range). Assumed rule (confirm it): given possibly overlapping ranges, answer queries 'which merged range contains x?', or None.

BRUTE FORCE - PSEUDO-CODE

```
for each query x: merge the ranges (or use them as given);
 test every range for start
    <= x <= end; return the match
```

Time: O(n) per query | Space: O(n)

Why it is slow

Every query walks all n ranges. Merging once (O(n log n)) and binary searching the sorted starts makes each query O(log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(ranges, queries):
    import bisect

    sorted_ranges = sorted(ranges, key=lambda r: r[0])
    starts = [r[0] for r in sorted_ranges]

    result = []
    for x in queries:
        idx = bisect.bisect_right(starts, x) - 1
        if idx >= 0 and sorted_ranges[idx][0] <= x <= sort
ed_ranges[idx][1]:
            result.append(sorted_ranges[idx][:])
        else:
            result.append(None)

    return result
```

Time: O(n log n + q log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

```python
def containing_original(ranges, x):
    """Return the original ranges (not merged) that contai
n x: sort by start and keep a running prefix max of ends i
s not enough, so scan the candidates."""
    starts = sorted(ranges)
    cut = bisect_right([s for s, _ in starts], x)
    return [(s, e) for s, e in starts[:cut] if e >= x]
```

Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

FUNCTION AND TEST CASES

Signature

```python
def solution(ranges: list[list[int]], queries: list[int])
-> list[list[int]]:
```

Sample call

```
solution([[1, 3], [50, 100]], [2, 50, 10])
```

Test cases

Input	Expected
`([[1, 3], [50, 100]], [2, 50, 10])`	`[[1, 3], [50, 100], None]`
`([[5, 8], [-10, -5], [0, 0]], [-7, 0, 8, 9, -11])`	`[[-10, -5], [0, 0], [5, 8], None, None]`

MEDIUM | PracHub | Technical Screen | 105 of 196

Check Whether Digits Are Monotone Increasing

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 172 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy digit scan) (inferred from overview)

Original: https://prachub.com/coding-questions/check-whether-digits-are-monotone-increasing

QUESTION

Overview: Check whether a non-negative integer's decimal digits are non-decreasing from left to right. Cover zero, single-digit values, repeated digits, and an O(d) implementation using either arithmetic digit extraction or string traversal.

APPROACH

Compare each digit with the previous one using % 10 and // 10 from the right (or string scan); handle 0 and single digits. O(d).

INTERVIEW-DAY PLAN

Clarify first

Ask whether equal neighbors are allowed (non-decreasing), whether the input can be negative, and whether to use the integer or string form.

Brute force to say out loud

Convert to a string and compare each pair. Fine and O(d).

Optimal idea

Without strings: peel digits from the right with % 10 and // 10, keeping the previous digit; fail if the current digit is greater than the previous one seen to its right. O(d), O(1) space.

Dry-run these cases

0, a single digit, 1234, 1224, 1243, 100.

Likely follow-ups

Strictly increasing, or find the next monotone number.

Pitfalls

Comparing in the wrong direction when scanning right to left.

HOW TO THINK ABOUT IT

Compare neighbouring digits. You can read the digits from the right with % 10 and // 10, remembering the previous (right-hand) digit: the number is monotone if no digit is larger than the one to its right. Say first whether equal neighbours are allowed.

Basis: The check behind LeetCode 738 (non-decreasing digits).

BRUTE FORCE - PSEUDO-CODE

```
s = the digits as text
return s == the characters of s sorted
```

Time: O(d log d) | Space: O(d)

Why it is slow

Sorting costs O(d log d) and copies the digits. Comparing each digit with its neighbour while peeling digits with % 10 is O(d) with O(1) space.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def has_monotone_increasing_digits(n):
    digits = str(n)
    return all(digits[i] <= digits[i + 1] for i in range(l
en(digits) - 1))
```

Time: O(d) | Space: O(d) for the decimal string

Explanation

Decimal digit characters have the same order as their numeric values. Scanning adjacent pairs rejects the first decrease and accepts empty adjacent-pair sets for single digits.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

```
def is_strictly_monotone_digits(n):
    prev = 10
    while n:
        d = n % 10
        if d >= prev:
            return False
        prev = d
        n //= 10
    return True
```

Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

FUNCTION AND TEST CASES

Signature

```
def has_monotone_increasing_digits(n: int) -> bool:
SAMPLE INPUT
0
```

Test cases

Input	Expected
0	True
7	True

MEDIUM | PracHub | Technical Screen | 106 of 196

★ TOP 75: rank 22 in the Medium and Hard priority list

★ Check if all substrings are anagrams of words

STUDY: TYPE | Google frequency on LeetCode: all time 57.7, last 30 days 51.4 | book priority 15 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #49 Group Anagrams

LeetCode match: No direct match; related LC 49 Group Anagrams

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-anagrams-of-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **>= 3** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - “Can be rearranged into a word” means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

APPROACH

Precompute sorted-letter (or count-vector) signatures of dictionary words into a set; for each substring of length >= 3 compute its signature and look it up.

INTERVIEW-DAY PLAN

Clarify first

Ask whether every substring of length at least 3 must be rearrangeable into some dictionary word, the alphabet (letters only), and the dictionary size and word lengths.

Brute force to say out loud

For every substring, compare against every dictionary word by sorted letters. Very slow.

Optimal idea

Preprocess the dictionary into a set of signatures, where a signature is the sorted letters or a 26-count tuple, and also record the set of word lengths present. For each start i extend j one letter at a time, updating a count vector incrementally; once the length is at least 3, look up its signature (skip lengths that no word has) and return false at the first miss. O(n^2 * 26) time.

Dry-run these cases

A string shorter than 3, a failing length that has no dictionary words, a long repeated letter, all substrings valid.

Likely follow-ups

Return the failing substring, or a larger alphabet (use a hash of counts).

Pitfalls

Sorting each substring from scratch, which costs an extra log factor. Forgetting that a signature must match the length too (a count tuple already does).

HOW TO THINK ABOUT IT

Anagrams share a signature: the multiset of letters (a sorted string or a 26-count tuple). So turn the dictionary into a set of signatures once, and test each substring by its signature. Extending a substring by one letter changes its count vector in O(1), so you do not rebuild the signature from scratch.

Basis: Written; every substring of length >= 3 must be a permutation of some dictionary word (compare Group Anagrams, LeetCode 49).

BRUTE FORCE - PSEUDO-CODE

```
for every substring of length 3 or more: sort its letters
compare with the sorted letters of every dictionary word;
if none matches: return
    false
```

Time: O(n^2 * W * L log L) | Space: O(1) extra

Why it is slow

Each substring is compared with every word and sorted again: O(n^2 * W * L log L). A set of word signatures, with the substring's count vector updated incrementally, makes each check O(alphabet).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(s, words):
    from collections import defaultdict

    n = len(s)
    if n < 3:
        return True

    def signature(text):
        counts = [0] * 26
        for ch in text:
            counts[ord(ch) - ord("a")] += 1
        return tuple(counts)

    valid = defaultdict(set)
    for word in words:
        if len(word) >= 3:
            valid[len(word)].add(signature(word))

    base = ord("a")

    for length in range(3, n + 1):
        if length not in valid or not valid[length]:
            return False

        window = [0] * 26
        for i in range(length):
            window[ord(s[i]) - base] += 1

        if tuple(window) not in valid[length]:
            return False

        for right in range(length, n):
            left = right - length
            window[ord(s[left]) - base] -= 1
            window[ord(s[right]) - base] += 1
            if tuple(window) not in valid[length]:
```

```
                return False
    return True
```

Time: O(total_chars_in_words + 26 * n^2) | Space: O(26 * U), where U is the number of unique dictionary signatures stored

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. Cost: O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

```
def first_failing_anagram(s, words):
    sigs = {tuple(sorted(w)) for w in words}
    for length in range(3, len(s) + 1):
        for i in range(len(s) - length + 1):
            if tuple(sorted(s[i : i + length])) not in sig
s:
                return s[i : i + length]
    return None
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, words: list[str]) -> bool:
```

Sample call

```
solution("act", ["cat", "dog"])
```

Test cases

Input	Expected
("act", ["cat", "dog"])	True
("abca", ["cab", "abca", "zzz"])	True

MEDIUM | PracHub | Technical Screen | 107 of 196

Maximize 1D deliveries within distance and minimize total distance

STUDY: TALK | Google frequency on LeetCode: all time 36.9, last 30 days 0 | book priority 91 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #455 Assign Cookies, LeetCode #475 Heaters

LeetCode match: No direct match; related LC 455 Assign Cookies / LC 475 Heaters (inferred from overview)

Original: https://prachub.com/coding-questions/maximize-1d-deliveries-within-distance-and-minimize-total-distance

QUESTION

Overview: This question evaluates combinatorial optimization and constrained matching skills, focusing on reasoning about distance-based pairings, lexicographic objective prioritization, and edge-case handling such as duplicates and empty inputs.

APPROACH

Sort both lists; two-pointer greedy matching each order to the nearest available driver within the limit, maximizing count first, then minimizing total distance as the tie-break.

INTERVIEW-DAY PLAN

Clarify first

Ask what an order and a driver are (points on a line), the distance limit, whether each driver takes one order, and the objective order: first the maximum number of deliveries, then the smallest total distance among those.

Brute force to say out loud

Try every matching of orders to drivers. Exponential.

Optimal idea

Sort both lists. On a line there is an optimal matching that does not cross, so a DP over (i, j), the first i orders and first j drivers, works: dp[i][j] is the best pair (maximum matches, minimum distance), taking the maximum of skipping an order, skipping a driver, or matching order i with driver j if their distance is within the limit. O(n * m), reducible to O(n * window) because only drivers within the limit matter. A two-pointer greedy that matches the nearest available driver gives the maximum count in O(n log n) but not necessarily the minimum total distance, so say that.

Dry-run these cases

No feasible pairs, duplicates, empty inputs, an order that could match two drivers equally, more drivers than orders.

Likely follow-ups

Minimizing the maximum distance instead, or drivers with capacities.

Pitfalls

Assuming greedy nearest matching minimizes the total. Comparing the two objectives as a sum instead of lexicographically.

HOW TO THINK ABOUT IT

On a line there is always an optimal matching that does not cross itself, so sort both lists and match in order: a DP over (orders considered, drivers considered) with the pair (matches, total distance) compared lexicographically solves both objectives together. A greedy that matches each order to the nearest free driver gets the count right but not always the smallest total distance.

Basis: Written; matching on a line. Assumed rules (confirm them): orders and drivers are positions on a line; an order can be served by a driver within distance D; each driver takes at most one order; first maximise the number of deliveries, then minimise the total distance.

BRUTE FORCE - PSEUDO-CODE

```
for each order: either skip it, or match it to any unused
driver within distance D
keep the assignment with the most matches, breaking ties b
y the smaller total distance
```

Time: O(drivers^orders) | Space: O(n)

Why it is slow

Each order branches over all drivers: exponential. After sorting, a DP over (orders considered, drivers considered) that compares (matches, minus distance) is O(n * m).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(people, shops, D):
    people = sorted(people)
    shops = sorted(shops)
    n, m = len(people), len(shops)

    if n == 0 or m == 0 or D < 0:
        return (0, 0)

    def better(a, b):
        if a[0] != b[0]:
            return a if a[0] > b[0] else b
        return a if a[1] < b[1] else b

    prev = [(0, 0)] * (m + 1)

    for i in range(1, n + 1):
        curr = [(0, 0)] * (m + 1)
        p = people[i - 1]

        for j in range(1, m + 1):
            best = better(prev[j], curr[j - 1])
            dist = abs(p - shops[j - 1])

            if dist <= D:
                candidate = (prev[j - 1][0] + 1, prev[j -
1][1] + dist)
                best = better(best, candidate)

            curr[j] = best

        prev = curr

    return prev[m]
```

Time: O(N log N + M log M + N*M) | Space: O(M)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Only the number of deliveries matters.

Two pointers on the sorted lists: match when within D, otherwise advance whichever is further left. O(n log n). It maximises the count (checked against exhaustive search) but not the total distance.

```
def max_deliveries_greedy(orders, drivers, D):
    """Maximum number of matches only: two pointers (does
not minimise distance)."""
    o, d, i, j, count = sorted(orders), sorted(drivers), 0
, 0, 0
    while i < len(o) and j < len(d):
        if abs(o[i] - d[j]) <= D:
            count, i, j = count + 1, i + 1, j + 1
        elif d[j] < o[i]:
            j += 1
        else:
            i += 1
    return count
```

A driver can take several orders (capacity c).

Replicate each driver c times (as separate sorted points) and use the same DP, or treat it as min-cost flow with capacities.

Minimise the largest single distance instead of the total.

Binary search the largest allowed distance D and test whether all orders can be served with the two-pointer matching; the check is monotone in D. Cost: O(n log n + (n + m) log(range)): sorting plus a binary search whose check is a two-pointer pass.

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[int], shops: list[int], D: int)
-> tuple[int, int]:
```

Sample call

```
solution([1, 7, 10], [2, 6, 12], 2)
```

Test cases

Input	Expected
([1, 7, 10], [2, 6, 12], 2)	(3, 4)
([0, 4], [2, 5], 3)	(2, 3)

MEDIUM | PracHub | Technical Screen | 108 of 196

Minimize calls to find all bad test pairs

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 173 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (adaptive group testing)

Original: https://prachub.com/coding-questions/minimize-calls-to-find-all-bad-test-pairs

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:
- (n): the total number of tests, and - (k): the total number of bad pairs.
3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:
- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints

0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

Examples

Input: (0, [])

Expected Output: ([], 0)

Explanation: There are no tests and therefore no oracle calls are needed.

Input: (4, [])

Expected Output: ([], 3)

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

APPROACH

Binary search to find a minimal failing prefix (its last element is in a bad pair), then binary search among earlier tests for its partner; remove and repeat. About O(p log n) calls for p pairs.

INTERVIEW-DAY PLAN

Clarify first

Ask what runTest returns (true if the set contains no incompatible pair), whether a test can be in more than one bad pair, and whether you may call it on any subset.

Brute force to say out loud

Call runTest on every pair of tests. O(n^2) calls.

Optimal idea

Find a minimal failing prefix by binary search: add tests one at a time conceptually and binary search for the smallest prefix that fails; the last element of that prefix is part of a bad pair. Then binary search over the earlier tests to find its partner. Record the pair, remove one of the two as needed, and repeat. About O(p log n) calls for p bad pairs.

Dry-run these cases

No bad pairs (one call). One bad pair. A test that conflicts with many others. Two disjoint bad pairs.

Likely follow-ups

Batch calls, noisy tests, or a limit on subset size.

Pitfalls

Removing both members of a pair and missing other pairs that involve them. Binary searching without the monotone property of prefixes.

HOW TO THINK ABOUT IT

Each call is a yes/no question about a whole set, so binary search is possible: a failing set can be narrowed by halving. Keep the tests you have processed in groups that each pass runTest (independent sets). A new test only needs to be checked against each group, and if a group plus the new test fails, binary search finds its partners in that group in about log(size) calls each.

Basis: Written; group testing / finding all edges of a hidden graph with independent-set queries. runTest(S) is True when S contains no incompatible (bad) pair.

BRUTE FORCE - PSEUDO-CODE

```
for every pair {a, b} of tests: if runTest({a, b}) is fals
e: record (a, b) as bad
```

Time: n(n - 1) / 2 calls | Space: O(1)

Why it is slow

n(n - 1) / 2 calls even when nothing is bad. Keeping groups of mutually compatible tests and binary searching only when a group fails needs about n + p log n calls for p bad pairs.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(n, bad_pairs):
    # Normalize the hidden bad pairs and build bit-mask ad
jacency for the oracle.
    bad_masks = [0] * n
    normalized_bad = set()
    for a, b in bad_pairs:
        if a == b:
            continue
        if a > b:
            a, b = b, a
        normalized_bad.add((a, b))

    for a, b in normalized_bad:
        bad_masks[a] |= 1 << b
        bad_masks[b] |= 1 << a

    calls = 0

    def run_test(vertices):
        """Return True iff vertices contains no hidden bad
 pair."""
        nonlocal calls
        calls += 1

        mask = 0
        for x in vertices:
            mask |= 1 << x

        for x in vertices:
            if bad_masks[x] & mask:
                return False
        return True

    discovered_masks = [0] * n
    found = []

    for v in range(n - 1, -1, -1):
```

```
        # Greedily color the already-processed suffix usin
g discovered bad pairs.
        color_classes = []
        color_masks = []

        for u in range(v + 1, n):
            placed = False
            for idx, cmask in enumerate(color_masks):
                # u can join this color class if it has no
discovered conflict inside it.
                if discovered_masks[u] & cmask == 0:
                    color_classes[idx].append(u)
                    color_masks[idx] |= 1 << u
                    placed = True
                    break

            if not placed:
                color_classes.append([u])
                color_masks.append(1 << u)

        def search_color_class(class_vertices):
            if not class_vertices:
                return

            query_vertices = [v] + class_vertices
            if run_test(query_vertices):
                return

            if len(class_vertices) == 1:
                u = class_vertices[0]
                found.append([v, u])
                discovered_masks[v] |= 1 << u
                discovered_masks[u] |= 1 << v
                return

            mid = len(class_vertices) // 2
            search_color_class(class_vertices[:mid])
```

```
            search_color_class(class_vertices[mid:])

        for color_class in color_classes:
            search_color_class(color_class)

    found.sort()
    return (found, calls)
```

Time: O(n * min(n, k + 1) + k log n) oracle calls, where k is the number of bad pairs. The provided simulation also spends time constructing and checking oracle subsets. | Space: O(n^2) bits for adjacency bit masks plus O(k) output space

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How many calls does it need, and can you prove a lower bound?

With p bad pairs among n tests about p log n calls are needed in the worst case (each pair carries log n bits of information about where it is), and the group method uses O(n * g + p log n). With no bad pairs it uses n - 1 calls, one per test after the first.

Only one bad pair exists.

Binary search for the smallest failing prefix (its last element is in the pair), then binary search among the earlier elements for the partner: about 2 log n calls.

runTest is expensive but can run in parallel.

Test all groups for the new element in the same round, or process several elements at once with their own groups, trading a few extra calls for fewer sequential rounds.

Only one bad pair exists.

Binary search the shortest failing prefix (its last element is in the pair), then binary search the earlier tests for the partner: about 2 log2(n) calls instead of n(n - 1) / 2 (verified by counting calls).

```
def find_single_bad_pair(tests, runTest):
    lo, hi = 1, len(tests)
    if runTest(set(tests)):
        return None
    while lo < hi:
        mid = (lo + hi) // 2
        if runTest(set(tests[:mid])):
            lo = mid + 1
        else:
            hi = mid
    last = tests[lo - 1]
    earlier = tests[: lo - 1]
    a, b = 1, len(earlier)
    while a < b:
        mid = (a + b) // 2
        if runTest(set(earlier[:mid]) | {last}):
            a = mid + 1
        else:
            b = mid
    return earlier[a - 1], last
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, bad_pairs: list[list[int]]) -> tuple[
list[list[int]], int]:
```

Sample call

```
solution(0, [])
```

Test cases

Input	Expected
(0, [])	([], 0)
(4, [])	([], 3)

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Find All Bad Pairs Using runTest Oracle | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition.
Algorithm sketch:
Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.
Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive $O(N^2)$ pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

Preparation: Read the Wikipedia article on group testing and Dorfman's two-stage scheme.

Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

MEDIUM | PracHub | Technical Screen | 109 of 196

Min deletions to avoid overlap in first k

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 174 of 196
Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns
Optimal answer from: the interview site's own solution
LeetCode match: No direct match (inferred) (inferred from overview)
Original: https://prachub.com/coding-questions/min-deletions-to-avoid-overlap-in-first-k

QUESTION

Overview: This question evaluates array and sequence manipulation skills, set-based reasoning about value overlap, and the ability to compute minimal edits while preserving relative order constraints.

APPROACH

Likely: compare the first k elements with the rest via sets and delete the minimum elements to remove value overlap while preserving order; confirm details on the PracHub page.

INTERVIEW-DAY PLAN

Clarify first

This prompt is vague, so ask for the exact rule first. A common reading: the first k elements and the remaining elements must share no value, delete the fewest elements. Ask whether deleted elements must be removed from the first part or either part, and whether order matters.

Brute force to say out loud

Try every subset of deletions. Exponential.

Optimal idea

Under that reading values are independent. For each value that appears in both parts, you must remove all its copies from one side, so you delete the smaller of its two counts. Sum over shared values. Count with two hash maps. O(n).

Dry-run these cases

No shared values (0), every value shared, k = 0 or k = n, repeated values on one side only.

Likely follow-ups

Delete only from the first part, weights on elements, or k given as a range.

Pitfalls

Starting to code before the rule is confirmed. Deleting elements one at a time greedily.

HOW TO THINK ABOUT IT

Values do not interact with each other, so each value can be decided on its own. A value that appears on both sides must disappear from one side entirely, and the cheapest side is the one with fewer copies. The answer is the sum, over values that appear on both sides, of the smaller of the two counts.

Basis: The statement is vague in my copy. Assumed rule (confirm it): given an array and k, the first k elements and the remaining elements must share no value; delete the fewest elements (from either part) to achieve that.

BRUTE FORCE - PSEUDO-CODE

```
for size = 0, 1, 2, ...: for every set of that many elemen
ts to delete: if the first-k
    part and the rest no longer share a value: return size
```

Time: O(2^n * n) | Space: O(n)

Why it is slow

2^n sets to try: exponential. Each shared value must disappear from one side, so the cost is the sum over shared values of the smaller of its two counts, in O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(list1, list2, k):
    if k == 0:
        return 0

    forbidden = set(list1[:k])
    deletions = 0
    valid_kept = 0

    for value in list2:
        if value in forbidden:
            deletions += 1
        else:
            valid_kept += 1
            if valid_kept == k:
                return deletions

    return -1
```

Time: O(k + n), where n = len(list2) | Space: O(k)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Only elements of the first part may be deleted.

Then every shared value must be removed from the first part entirely, so the cost is the number of first-part elements whose value occurs in the rest (a set lookup per element).

```python
def min_deletions_first_part_only(a, k):
    right = set(a[k:])
    return sum(1 for x in a[:k] if x in right)
```

Elements have weights (deleting some is more costly).

Replace counts by the sum of weights per side for each value; the choice per value is still independent, so take the cheaper side. Cost: unchanged O(n); the per-value choice sums weights instead of counts.

The statement may differ from this reading.

Ask whether deletions can come from both parts, whether duplicates count separately, and whether order matters. Everything above relies on those three answers.

FUNCTION AND TEST CASES

Signature

```python
def solution(list1: list[int], list2: list[int], k: int) -
> int:
```

Sample call

```
solution([1, 2, 3], [2, 1, 4, 5], 2)
```

Test cases

Input	Expected
([1, 2, 3], [2, 1, 4, 5], 2)	2
([1, 2, 3, 4], [2, 5, 3, 6], 2)	1

MEDIUM | Exponent | 110 of 196

★ TOP 75: rank 24 in the Medium and Hard priority list

★ Given a list with response time for each server, calculate how long time it will take to process n requests.

STUDY: TYPE | Google frequency on LeetCode: all time 59.1, last 30 days 46.9 | book priority 16 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas

LeetCode match: No direct match; related LC 875 Koko Eating Bananas (inferred from short statement)

Original: https://www.aced.io/questions/5485/calculate-time-to-process-given-server-response-time

QUESTION

Given a list with response time for each server, calculate how long time it will take to process n requests.

APPROACH

Binary search on time T: servers process floor(T / response_i) requests each; find the smallest T where the sum >= n. O(S log(maxT)).

INTERVIEW-DAY PLAN

Clarify first

Ask whether servers work in parallel and independently, what response time means (time to finish one request), and whether n can be zero.

Brute force to say out loud

Simulate each request assigned to the server that frees first with a heap. O(n log S).

Optimal idea

Binary search the answer T: in time T, server i finishes T // r_i requests, so feasibility is sum(T // r_i) >= n, which is monotone in T. Search T in [0, min(r) * n]. O(S log(min(r) * n)).

Dry-run these cases

One server, equal servers, n = 0, a very fast server and very slow servers, large n (overflow).

Likely follow-ups

Servers that are not independent, or the time to finish exactly the n-th request.

Pitfalls

Using ceil instead of floor. Searching with an upper bound that is too small.

HOW TO THINK ABOUT IT

'Smallest time T so that the servers can finish n requests' has a monotone yes/no check: if T is enough, any larger time is too. The check is a sum of T // response_time over the servers. Whenever the answer is a threshold on a monotone condition, binary search the threshold instead of simulating.

Basis: Written; binary search on the answer (compare LeetCode 875, Koko Eating Bananas, and LeetCode 1011).

BRUTE FORCE - PSEUDO-CODE

```
heap of 'time each server finishes its next request'
repeat n times: take the earliest finishing server, record
 that time, and push back
    (time + its response time)
answer = the largest recorded time
```

Time: O(n log S) | Space: O(S)

Why it is slow

One heap operation per request, O(n log S); with n up to 10^9 requests that is too slow. The finish time is a monotone threshold, so binary search on T with sum(T // response time) >= n is O(S log answer).

OPTIMAL SOLUTION - BINARY SEARCH ON THE TOTAL TIME

```python
def time_for_requests(times, n):
    if n == 0:
        return 0
    lo, hi = 1, min(times) * n
    while lo < hi:
        mid = (lo + hi) // 2
        if sum(mid // r for r in times) >= n:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Time: O(S log(min(times) * n)) | Space: O(1)

Why

In time T server i completes T // r_i requests, so the servers finish at least n requests exactly when sum(T // r_i) >= n, which is monotone in T. Search T between 0 and min(times) * n (one fastest server alone is enough).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Servers may fail, or there is a startup time per server.

Add the startup time to the check: server i completes max(0, (T - start_i) // r_i) requests. The check stays monotone in T, so binary search still applies.

The inverse question: how many servers are needed to finish within a given time?

Divide: each server finishes time // r requests, so you need ceil(n / that). No search needed.

```python
def servers_needed(time_budget, r, n):
    """Different question: how many identical servers of r
esponse time r are needed to finish n requests within time
_budget."""
    per_server = time_budget // r
    return -(-n // per_server) if per_server else None
```

Requests have different sizes.

Then it is a scheduling problem (minimise the makespan), NP-hard in general; the greedy 'longest request first to the least-loaded server' is within 4/3 of optimal.

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

MEDIUM | Exponent | 111 of 196

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

STUDY: TALK | Google frequency on LeetCode: all time 26.5, last 30 days 0 | book priority 121 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #475 Heaters

LeetCode match: No direct match; related LC 475 Heaters (inferred from short statement)

Original: https://www.aced.io/questions/5138/minimum-distance-kids-ice-cream-sellers

QUESTION

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

APPROACH

Sort sellers; for each kid binary search the nearest seller; E = max over kids of the nearest distance. O((N + M) log M). Alternative: binary search E + two pointers.

INTERVIEW-DAY PLAN

Clarify first

Ask whether positions are integers, whether several sellers can share a position, and whether E may be fractional (the answer is a distance, so it is one of the kid-seller distances).

Brute force to say out loud

Try every E over the range and check all kids. O(N * M * range).

Optimal idea

Sort the sellers. For each kid binary search the nearest seller (check the neighbors on both sides of the insertion point). E is the maximum of those nearest distances. O((N + M) log M). Alternatively, binary search on E and check coverage with two pointers (LeetCode 475, Heaters).

Dry-run these cases

A kid at a seller's position (0), all kids left of every seller, one seller, duplicates, kids between two sellers.

Likely follow-ups

Each seller can serve at most C kids, which turns it into a matching problem solved by binary search plus greedy.

Pitfalls

Checking only the seller to the right of the kid. Forgetting to sort.

HOW TO THINK ABOUT IT

Each kid only cares about its nearest seller, and the radius E must cover the worst kid. So E is the maximum over kids of the distance to the nearest seller. With sellers sorted, the nearest seller of a kid is one of the two neighbours around its insertion point, which is a binary search.

Basis: LeetCode 475, Heaters (houses = kids, heaters = sellers); the answer is the largest distance from a kid to the nearest seller.

BRUTE FORCE - PSEUDO-CODE

```
for each kid: nearest = the smallest |kid - seller| over a
ll sellers
answer = the largest nearest over all kids
```

Time: O(N * M) | Space: O(1)

Why it is slow

N * M distance checks. With the sellers sorted, binary search finds each kid's nearest seller among two neighbours, O((N + M) log M).

OPTIMAL SOLUTION - SORT THE SELLERS, BINARY SEARCH EACH KID

```
def smallest_radius(kids, sellers):
    sellers = sorted(sellers)
    worst = 0
    for k in kids:
        i = bisect_left(sellers, k)
        near = min(abs(k - sellers[j]) for j in (i - 1, i)
 if 0 <= j < len(sellers))
        worst = max(worst, near)
    return worst
```

Time: O((N + M) log M) | Space: O(1) extra

Why

With the sellers sorted, bisect_left gives the first seller at or after the kid; the nearest seller is that one or the one before it. E is the largest of these nearest distances.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Both lists are already sorted, or you want to avoid the binary search.

Walk the kids in order with one pointer on the sellers that only moves forward while the next seller is at least as close. O(N + M) after sorting.

```
def smallest_radius_two_pointers(kids, sellers):
    kids, sellers = sorted(kids), sorted(sellers)
    j, worst = 0, 0
    for k in kids:
        while j + 1 < len(sellers) and abs(sellers[j + 1]
- k) <= abs(sellers[j] - k):
            j += 1
        worst = max(worst, abs(sellers[j] - k))
    return worst
```

Each seller can serve at most C kids.

Now it is an assignment problem. Binary search the radius E and test feasibility with a greedy left-to-right matching that gives each seller up to C of the leftmost uncovered kids in its range.

Sellers can be placed anywhere (minimise the radius with S sellers).

That is the 1D k-center problem: binary search the radius and greedily place a seller at the farthest point covering the leftmost uncovered kid. O(N log range).

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

MEDIUM | 1Point3Acres | phone-screen | 112 of 196

Progressive Array Partitioning

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 177 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

Examples:
Stage 1
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size one.
Stage 2
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

Clarify first

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

Brute force to say out loud

Try every way to group the elements and test the rule. Exponential.

Optimal idea

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

Dry-run these cases

The four examples in the statement, an empty array, a single value, many values with equal counts.

Likely follow-ups

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

Pitfalls

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE - PSEUDO-CODE

```
counts = how many times each distinct value occurs
for every pair of distinct values: if their counts are equ
al: return false
return true
```

Time: O(d^2) where d is the number of distinct values | Space: O(d)

Why it is slow

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION - COUNTER, THEN COMPARE THE SET OF COUNTS WITH THE NUMBER OF VALUES

```python
def distinct_counts(nums):
    counts = Counter(nums)
    return len(set(counts.values())) == len(counts)
```

Time: O(n) | Space: O(d)

Why

All counts are different exactly when the set of counts has as many elements as there are distinct values. One pass to count, one pass over the counts.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

```python
def can_split_into_straights(nums, min_len=3):
    """One candidate reading of Stage 2 (LeetCode 659): sp
lit a sorted array into runs of consecutive values, each a
t least min_len long."""
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True
```

Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 113 of 196

Fence Painting Minimum Operations (Gemini Team)

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 179 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

QUESTION

Examples:

```
n = 5, a = [2, 2, 1, 2, 1]
Optimal:
  1) Horizontal stroke at h=1 across [1..5]   (covers row
1 across whole fence)
  2) Horizontal stroke at h=2 across [1..2]
  3) Vertical stroke at plank 4 (height 2 still missing ro
w 2)
Optimal answer: 3.
```

APPROACH

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

INTERVIEW-DAY PLAN

Clarify first

Ask whether vertical strokes paint one plank fully, whether horizontal strokes must cover consecutive planks at one height, and the sizes involved.

Brute force to say out loud

Try all combinations of strokes. Exponential.

Optimal idea

Divide and conquer on the minimum height: for a range with base height h, painting every plank vertically costs the number of planks; otherwise paint one horizontal band up to the minimum height m (cost 1) and recurse on the segments above m, split where planks are exactly m. The answer is the minimum of the two. O(n^2) worst case, O(n log n) with a sparse table or a min-stack for range minimum queries.

Dry-run these cases

The example giving 3. All equal heights (1). Strictly increasing heights. A single plank.

Likely follow-ups

A brush of width W, or return the strokes.

Pitfalls

Forgetting the vertical option, so a tall single plank is overcounted. Off-by-one on the band height.

HOW TO THINK ABOUT IT

The shortest plank in a range decides the shape: a horizontal band along the bottom, up to that plank's height, is free of regret because every plank in the range needs it. That splits the range at the shortest planks, so recurse on the pieces above the band. The other choice for a range is to paint every plank vertically. Take whichever is cheaper.

Basis: Codeforces 448C, Painting Fence. Strokes: a vertical stroke paints one whole plank; a horizontal stroke paints one row across consecutive planks that all reach that height.

BRUTE FORCE - PSEUDO-CODE

```
paint(l, r, base): if l > r: return 0
m = the minimum height in l..r (found by scanning)
options: paint every plank vertically: r - l + 1 strokes;
or one horizontal band of
    height m - base, then paint(...) on each piece of plan
ks above m
return the smaller of the two
```

Time: O(n^2) worst case | Space: O(n) recursion

Why it is slow

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

OPTIMAL SOLUTION - SAME RECURSION WITH A SPARSE TABLE FOR RANGE MINIMUM

```
def paint(a):
    n = len(a)
    if n == 0:
        return 0
    table = [list(range(n))]
    j = 1
    while (1 << j) <= n:
        prev, row = table[-1], []
        for i in range(n - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1

    def argmin(l, r):
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        if l > r:
            return 0
        m = argmin(l, r)
        return min(
            r - l + 1, a[m] - base + solve(l, m - 1, a[m])
 + solve(m + 1, r, a[m])
        )

    return solve(0, n - 1, 0)
```

Time: O(n log n) | Space: O(n log n)

Why

The recursion is correct; only the minimum lookup is slow. A sparse table answers range-minimum-index queries in O(1), and each plank becomes the split point at most once, so the recursion visits n ranges. Splitting at one minimum plank (instead of all of them) keeps ranges disjoint and makes the count obvious.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

```
def paint_brush_width(a, w):
    """Follow-up: a horizontal brush paints exactly at mos
t w consecutive planks per stroke."""

    def solve(l, r, base):
        if l > r:
            return 0
        m = min(a[l : r + 1])
        cost = 0
        i = l
        while i <= r:
            if a[i] == m:
                i += 1
                continue
            j = i
            while j <= r and a[j] != m:
                j += 1
            cost += solve(i, j - 1, m)
            i = j
        # a band of height (m - base) over r - l + 1 plank
s needs ceil(width / w) strokes per row
        band = (m - base) * (-(-(r - l + 1) // w))
        return min(r - l + 1, band + cost)

    return solve(0, len(a) - 1, 0)
```

Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. Cost: the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

NOTES FROM CANDIDATES

Classic recursive divide-and-conquer: for range [l..r] with base height h:
Option A: paint every plank vertically -> cost r - l + 1.
Option B: paint a horizontal band of height min - h, then recurse on the sub-ranges above the band split by planks at height min.
Take the minimum.
Equivalent to Cartesian-tree DP; runs in O(N log N) with min-stack or sparse table, or O(N²) with the recursive form.
Verbalize the recursion's correctness: the band optimally covers all planks >= min in the range, then each "taller column block" is solved independently.

PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

MEDIUM | 1Point3Acres | onsite-coding | 114 of 196

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 181 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

Examples:

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space woul
d be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo bar b
az" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

APPROACH

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.

Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.

Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the width counts characters in a monospace font, how to treat explicit newlines, multiple spaces and leading spaces, what to do with a word longer than the width, and whether to return the number of lines or the lines themselves.

Brute force to say out loud

Build the text line by line by trying to add each word and measuring. That is already the greedy algorithm.

Optimal idea

Tokenize into words and newline tokens. Keep the current line length; for each word, if it fits (with a space unless the line is empty) add it, otherwise start a new line. A newline token always ends the current line. Count lines (empty lines count). Greedy is expected; mention Knuth's minimum-raggedness DP only if asked. O(n).

Dry-run these cases

The example giving 4. width = 1. Consecutive newlines. A word exactly the width. A word longer than the width.

Likely follow-ups

Return the lines, justify the text, or minimize raggedness.

Pitfalls

Dropping empty lines. Adding a trailing space at the end of a line.

HOW TO THINK ABOUT IT

Greedy is optimal for the question 'fewest lines': put as many words on a line as fit. The decision per word is only 'fits with a space', 'fits as the first word on the line', or 'start a new line', so track the current line length, not the line text. Treat newline as a hard break and decide up front what empty lines and long words do.

Basis: Written; greedy word wrap (LeetCode 68, Text Justification, without the justification step). Assumed rules (confirm them): monospace width in characters; words are separated by whitespace; '\n' always ends a line and an empty line counts; a word longer than the width gets its own line.

BRUTE FORCE - PSEUDO-CODE

```
for each paragraph (split on newlines): line = ''
for each word: if line + ' ' + word still fits the width:
line = line + ' ' + word,
    else finish the line and start a new one with the word
```

Time: O(total characters * words per line) | Space: O(width)

Why it is slow

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION - GREEDY WITH A RUNNING LINE LENGTH

```python
def wrap_count(text, width):
    total = 0
    for paragraph in text.split("\n"):
        cur = 0
        lines = 1
        for w in paragraph.split():
            if cur == 0:
                cur = len(w)
            elif cur + 1 + len(w) <= width:
                cur += 1 + len(w)
            else:
                lines += 1
                cur = len(w)
        total += lines
    return total
```

Time: O(n) | Space: O(1) extra beyond the output

Why

Split on newlines first (each paragraph ends its line), then for each word compare cur_len + 1 + len(word) with the width. Only the length is tracked, never the joined line. The empty paragraph produces one empty line, and a word longer than the width sits alone on a line.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

```python
def wrap_lines_brute(text, width):
    lines = []
    for paragraph in text.split("\n"):
        words = paragraph.split()
        cur = ""
        for w in words:
```

```
            trial = (cur + " " + w) if cur else w
            if len(trial) <= width or not cur:
                cur = trial
            else:
                lines.append(cur)
                cur = w
        lines.append(cur)
    return lines
```

A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

```
def wrap_lines_split_long(text, width):
    lines = []
    for paragraph in text.split("\n"):
        cur = ""
        for w in paragraph.split():
            while len(w) > width:  # break words that can
never fit
                if cur:
                    lines.append(cur)
                    cur = ""
                lines.append(w[:width])
                w = w[width:]
            trial = (cur + " " + w) if cur else w
            if len(trial) <= width:
                cur = trial
            else:
                lines.append(cur)
                cur = w
        lines.append(cur)
    return lines
```

Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

NOTES FROM CANDIDATES

Single-pass tokenizer + accumulator: track curLen; on each token, decide "fits with leading space", "fits without space" (start of line), or "flush and start new line".

Watch out for \n right after another \n (empty line counts), \n at the end of input, leading whitespace, multiple consecutive spaces inside a line.

Don't try to handle font / kerning / hyphenation — the prompt assumes monospace width counted in characters.

Greedy packing is the expected answer (LC 68 style). Mention DP word-wrap (Knuth's algorithm) only if the interviewer pushes on 'minimum raggedness'; greedy is optimal-enough and what graders look for. Watch edge cases: a single word longer than W, an explicit newline forcing a break mid-pack, trailing whitespace handling.

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.

Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.

Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

MEDIUM | HackTheRounds | Onsite | 115 of 196

Sum of Two Squares in Two Ways

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 185 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

QUESTION

Given an integer n, find all integers s < n such that s can be expressed as the sum of two squares in exactly two distinct unordered ways. For example: 50 = 1^2 + 7^2 = 5^2 + 5^2.

EXAMPLES

Example 1

Input	`n = 100`
Output	`[50, 65, 85]`
Why	`50 = 1^2 + 7^2 = 5^2 + 5^2, 65 = 1^2 + 8^2 = 4^2 + 7^2, 85 = 2^2 + 9^2 = 6^2 + 7^2`

Example 2

Input	`n = 50`
Output	`[]`
Why	`No number less than 50 has exactly two representations.`

CONSTRAINTS

- `1 <= n <= 2^31 - 1`

APPROACH

Taxicab-number style: count a^2 + b^2 over a <= b with a hash map, keep sums with exactly two representations

INTERVIEW-DAY PLAN

Clarify first

Ask whether pairs are unordered with a <= b, whether zero is allowed, and whether the bound is strict (s < n).

Brute force to say out loud

For each s below n try every a and b. O(n * sqrt(n)).

Optimal idea

Loop over a <= b with a^2 + b^2 < n and count each sum in a hash map. Return the sums whose count is exactly two. There are about pi * n / 8 such pairs, so O(n) time and space. For the example, 50 = 1^2 + 7^2 = 5^2 + 5^2, so 50 qualifies when n > 50.

Dry-run these cases

n = 50 (50 itself is excluded because the bound is strict), a small n with no answers, a sum with three representations (excluded), a perfect square such as 25 = 0 + 25 = 9 + 16 if zero is allowed.

Likely follow-ups

Exactly k ways, or counting without a dictionary using a sieve.

Pitfalls

Counting (a, b) and (b, a) as different. Including s equal to n.

HOW TO THINK ABOUT IT

Do not test each s separately. Generate every pair (a, b) once, add its sum to a counter, and read the answer from the counter. The loop is bounded by a^2 + b^2 < n, so a and b are at most sqrt(n); a <= b removes order duplicates.

Basis: Taxicab-number style counting. The examples fix the rule: a and b are positive integers with a <= b, so 25 = 3^2 + 4^2 has only one representation (0^2 + 5^2 is not allowed).

BRUTE FORCE - PSEUDO-CODE

```
for every s from 1 to n - 1: count the pairs a <= b with a
^2 + b^2 = s
keep the s whose count is exactly 2
```

Time: O(n * sqrt(n)) | Space: O(1)

Why it is slow

Every s searches all pairs again, O(n * sqrt(n)). Generating each pair once and counting sums in a hash map is O(n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: COUNT REPRESENTATIONS

```python
from typing import List
from collections import defaultdict


class Solution:
    def sumOfTwoSquaresTwoWays(self, n: int) -> List[int]:
        # Count how many ways each sum can be represented
        count = defaultdict(int)

        # For each pair (a, b) where a <= b, count a^2 + b
^2
        a = 1
        while a * a < n:
            b = a
            while a * a + b * b < n:
                s = a * a + b * b
                count[s] += 1
                b += 1
            a += 1

        # Return sums with exactly 2 representations
        result = [s for s in sorted(count.keys()) if count
[s] == 2]
        return result
```

Time: O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Exactly k ways instead of two.

Change the comparison from == 2 to == k; the counting loop is the same. For k = 3 the smallest is 325.

```python
def two_squares_exactly_k(n, k):
    ways = Counter()
    a = 1
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            ways[a * a + b * b] += 1
            b += 1
        a += 1
    return sorted(s for s, c in ways.items() if c == k)
```

Allow zero (0^2 + b^2).

Start a at 0; then perfect squares like 25 gain a representation, so some answers appear or disappear. Check against the examples to see which convention the question uses (here, positive only).

n is huge (10^12) and you only need to test one number.

Factorise the number: the count of representations as a sum of two squares depends only on the primes congruent to 1 mod 4 (Jacobi's formula), so a factorisation answers it in sublinear time.

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Find numbers with exactly two sum-of-squares forms | PracHub | Technical Screen

https://prachub.com/coding-questions/find-numbers-with-exactly-two-sum-of-squares-forms

---

MEDIUM | IN APP LC 362 | DESIGN-STYLE (skippable) | HackTheRounds | Phone Screen | 116 of 196

Hit Counter

STUDY: TALK | Google frequency on LeetCode: all time 33.2, last 30 days 0 | book priority 104 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #362 Design Hit Counter

LeetCode match: LC 362

QUESTION

Design a hit counter that counts the number of hits received in the past 5 minutes (300 seconds). hit(timestamp) records a hit; getHits(timestamp) returns the number of hits in the past 300 seconds.

EXAMPLES

Example 1	
Input	`hit(1), hit(2), hit(3), getHits(4), hit(300), getHits(300), getHits(301)`
Output	`[null, null, null, 3, null, 4, 3]`
Why	`At 301, hit at timestamp 1 expires.`

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

APPROACH

Queue/deque of timestamps; evict entries older than 300 seconds

HOW TO THINK ABOUT IT

Only the last 300 seconds matter, so anything older is dead weight: evict from the front of a queue of timestamps. If hits arrive at very high rates, store counts per second in a circular array of 300 buckets so memory does not depend on the number of hits.

Basis: LeetCode 362, Design Hit Counter.

BRUTE FORCE - PSEUDO-CODE

```
hit(t): append t to a list
getHits(t): count the timestamps greater than t - 300
```

Time: O(n) per getHits | Space: O(n)

Why it is slow

Memory grows with every hit forever, and each query scans them all, O(n). A queue that drops timestamps older than 300 seconds (or 300 per-second buckets) keeps both bounded.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: HIT COUNTER USING QUEUE TO TRACK TIMESTAMPS

```python
from collections import deque


class Solution:
    """
    Hit Counter using queue to track timestamps.

    Time: O(1) amortized for hit, O(n) worst case for getH
its
    Space: O(n) where n is number of hits in 300 seconds
    """

    def __init__(self):
        self.hits = deque()

    def hit(self, timestamp: int) -> None:
        """Record a hit at given timestamp."""
        self.hits.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        """Return number of hits in past 300 seconds."""
        while self.hits and self.hits[0] <= timestamp - 30
0:
            self.hits.popleft()
        return len(self.hits)
```

Time: O(1)/O(n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Millions of hits per second.

Use 300 buckets (one per second) instead of one entry per hit: hit is O(1), getHits is O(300) = O(1), and memory is constant. A bucket whose stored second is older than the window is reset on reuse.

```python
class HitCounterBuckets:
    """O(1) memory: one counter per second modulo the wind
ow, reset when the slot belongs to an older second."""

    def __init__(self, window=300):
        self.window, self.times, self.counts = window, [0]
 * window, [0] * window

    def hit(self, t):
        i = t % self.window
        if self.times[i] != t:
            self.times[i], self.counts[i] = t, 0
        self.counts[i] += 1

    def getHits(self, t):
        return sum(c for ts, c in zip(self.times, self.cou
nts) if ts > t - self.window)
```

Timestamps can arrive out of order.

A deque no longer works. Keep a sorted structure (a sorted list with bisect or a balanced tree) and count with two binary searches, or use the buckets, which are order-insensitive within the window.

Thread safety.

Guard the queue with a lock (or use per-thread counters merged on read). The bucket version needs the compare-then-reset of a slot to be atomic. Cost: one lock per call, O(1) extra; contention rather than complexity is the concern.

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Design a rolling hit counter API | PracHub | Technical Screen

https://prachub.com/coding-questions/design-a-rolling-hit-counter-api

---

MEDIUM | IN APP LC 380 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 117 of 196

Design set with O(1) random access

STUDY: TYPE | Google frequency on LeetCode: all time 46.6, last 30 days 0 | book priority 66 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1)

Original: https://prachub.com/coding-questions/design-set-with-o-1-random-access

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.
- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

Examples

Input:
(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
Expected Output: [True, True, 1, True, 2, False, 2]
Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

Input: (['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True, None]
Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

APPROACH

Array of values + hash map value -> index; remove by swapping with the last element and popping.

HOW TO THINK ABOUT IT

A hash set gives O(1) add and delete but cannot pick a random element in O(1); an array can pick a random element but deleting from the middle is O(n). Combine them: keep the values in an array and a dict from value to array index. To delete, swap the element with the last one, pop the end, and update the moved element's index.

Basis: LeetCode 380, Insert Delete GetRandom O(1) (FancySet). Choice made: getRandom on an empty set raises an error (a defined, loud failure).

BRUTE FORCE - PSEUDO-CODE

```
add(x): if x is not already in the list (scan): append it
delete(x): find it by scanning and remove it from the list
getRandom(): return a random element of the list
```

Time: O(n) for add and delete, O(1) getRandom | Space: O(n)

Why it is slow

Membership checks and deletion are O(n). An array plus a dictionary from value to index, deleting by swapping with the last element, makes all three operations O(1).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(operations, values, seed):
    items = []
    pos = {}
    state = seed & 0xFFFFFFFF
    result = []

    def next_state():
        nonlocal state
        state = (1664525 * state + 1013904223) & 0xFFFFFFF
F
        return state

    for op, arg in zip(operations, values):
        if op == "add":
            x = arg[0]
            if x in pos:
                result.append(False)
            else:
                pos[x] = len(items)
                items.append(x)
                result.append(True)

        elif op == "delete":
            x = arg[0]
            if x not in pos:
                result.append(False)
            else:
                idx = pos[x]
                last = items[-1]

                items[idx] = last
                pos[last] = idx

                items.pop()
                del pos[x]
                result.append(True)

        elif op == "getRandom":
```

```
            if not items:
                result.append(None)
            else:
                idx = next_state() % len(items)
                result.append(items[idx])

        else:
            raise ValueError("Unknown operation: " + str(o
p))

    return result
```

Time: O(n) total for n operations; each add, delete, and getRandom is O(1) on average | Space: O(k) where k is the number of stored elements, O(n) in the worst case

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

```
class FancySet:
    def __init__(self):
        self.a, self.pos = [], {}

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.a)
        self.a.append(x)
        return True

    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.a[-1]
        self.a[i] = last
        self.pos[last] = i
        self.a.pop()
        del self.pos[x]
        return True

    def getRandom(self):
        if not self.a:
            raise IndexError("empty set")
        return self.a[random.randrange(len(self.a))]


class FancySetThreadSafe(FancySet):
    def __init__(self):
        super().__init__()
        self.lock = threading.Lock()

    def add(self, x):
        with self.lock:
            return super().add(x)

    def delete(self, x):
        with self.lock:
            return super().delete(x)

    def getRandom(self):
        with self.lock:
            return super().getRandom()
```

What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[str], values: list[list[int]
], seed: int) -> str:
```

Sample call

```
solution(['add','add','getRandom','delete','getRandom','ad
d','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
```

Test cases

Input	Expected
`(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)`	`[True, True, 1, True, 2, False, 2]`
`(['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)`	`[None, True, False, False, True, None]`

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 118 of 196

Design Tic-Tac-Toe With K Players

STUDY: TALK | Google frequency on LeetCode: all time 18.1, last 30 days 0 | book priority 140 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

LeetCode match: LC 348 Design Tic-Tac-Toe

Original: https://prachub.com/coding-questions/design-tic-tac-toe-with-k-players

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.

- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.
- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

Examples

Input: (3, 2, [])

Expected Output: []

Explanation: No moves are made, so there are no statuses to report.

Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])

Expected Output: ['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

APPROACH

Per-player row, column, diagonal and anti-diagonal counters; a player wins when any counter reaches n. O(1) per move.

INTERVIEW-DAY PLAN

Clarify first

Ask whether a win needs a full row, column or diagonal (n in a row) or only m in a row, how invalid moves are reported, what happens on a full board, and the API (move(row, col, player) returns winner or none).

Brute force to say out loud

After each move scan the row, column and both diagonals of the board. O(n) per move.

Optimal idea

Keep counters per player for each row, each column, the main diagonal and the anti-diagonal. A move increments up to four counters; a player wins when any of their counters reaches n. O(1) per move, O(k * n) memory using a dict of counters. Reject moves on occupied cells.

Dry-run these cases

A diagonal win, an anti-diagonal win, a row win by the last player, a draw, a move on an occupied cell, a move after the game has ended.

Likely follow-ups

Win length m smaller than n (scan four directions from the last move for up to m cells), undo a move, or an unbounded sparse board.

Pitfalls

Counting an occupied cell twice. Not ending the game after a win.

HOW TO THINK ABOUT IT

Do not look at the board after each move. A win needs n marks of the same player on one line, so keep a counter per player and per line (each row, each column, the two diagonals) and bump the at-most-four lines that the move touches. The move wins if any of those counters reaches n.

Basis: LeetCode 348, Design Tic-Tac-Toe, generalized to k players. Assumed rule (confirm it): a player wins by filling a whole row, a whole column or a whole diagonal of the n x n board.

BRUTE FORCE - PSEUDO-CODE

```
place the mark on the board
check the move's row, its column, and the two diagonals if
 the move is on one: does a
    line hold n marks of this player?
```

Time: O(n) per move | Space: O(n^2)

Why it is slow

Each move rescans up to four lines, O(n). Keeping a counter per player per row, column and diagonal makes each move O(1).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(n, k, moves):
    board = {}
    result = []
    expected_player = 1
    game_over = False
    directions = ((0, 1), (1, 0), (1, 1), (1, -1))

    for move in moves:
```

```
        player, row, col = move

        if game_over:
            result.append("Game Over")
            continue

        if (
            player < 1
            or player > k
            or player != expected_player
            or not (0 <= row < n and 0 <= col < n)
            or (row, col) in board
        ):
            result.append("Invalid move")
            continue

        board[(row, col)] = player

        won = False
        for dr, dc in directions:
            count = 1

            for sign in (1, -1):
                nr = row + sign * dr
                nc = col + sign * dc
                steps = 0

                while steps < 2 and board.get((nr, nc)) ==
 player:
                    count += 1
                    if count >= 3:
                        won = True
                        break
                    nr += sign * dr
                    nc += sign * dc
                    steps += 1

                if won:
                    break

            if won:
                break

        if won:
            result.append(f"{player} won")
            game_over = True
        else:
            result.append("Continue")
            expected_player = expected_player + 1 if expec
ted_player < k else 1

    return result
```

Time: O(m), where m is the number of move attempts | Space: O(v), where v is the number of valid moves stored on the board

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(m) per move.

```
class TicTacToeM:
    """Win by m in a row (not the whole board): look along
 4 directions from the last move, up to m cells each way."
""

    def __init__(self, n, m):
        self.n, self.m, self.cell = n, m, {}

    def move(self, r, c, player):
        if (r, c) in self.cell:
            return -1
        self.cell[(r, c)] = player
        for dr, dc in ((0, 1), (1, 0), (1, 1), (1, -1)):
            run = 1
            for sign in (1, -1):
                i = 1
                while (
                    i < self.m
                    and self.cell.get((r + sign * i * dr,
c + sign * i * dc)) == player
                ):
                    run += 1
                    i += 1
            if run >= self.m:
                return player
        return 0
```

Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. Cost: O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, k: int, moves: list[list[int]]) -> li
st[str]:
```

Sample call

```
solution(3, 2, [])
```

Test cases

Input	Expected
`(3, 2, [])`	`[]`
`(5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])`	`['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Tic-Tac-Toe System with K Players and 3-in-a-Row Win | 1Point3Acres | phone-screen, tech-screen | asked 2026-05-18 | Single report

https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.

Avoid scanning the whole board after every move unless n is tiny.

The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.
Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

Preparation: Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 119 of 196

Active Communication Top Users

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 196 of 196
Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns
Optimal answer from: written and tested here for this exact question
Frequency: Low | Last asked: 2026-09-22 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

Process a continuous stream of user pairs such as **A-B** and **B-C**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

APPROACH

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

INTERVIEW-DAY PLAN

Clarify first

Ask whether a pair A-B counts for both users, whether repeated pairs count once, how self communication is treated, how ties order, and whether k changes over time.

Brute force to say out loud

Recount from all edges and sort users on every query. O(E log U).

Optimal idea

Normalize pairs (sorted tuple). Keep an edge set for deduplication, a dict of user to count, and either a sorted structure of (count, user) or a bucket map count to ordered users so an update moves a user by one bucket. Top-K reads the highest buckets. Note the edge set can use quadratic space in the number of users; say so.

Dry-run these cases

A-B then B-A (duplicate), a self pair, ties on count, a new user, k larger than the number of users.

Likely follow-ups

Constant-time most active user (track the maximum count with buckets), a changing k, or expiry of old edges.

Pitfalls

Claiming linear space while storing all pairs. Counting a duplicate edge twice.

HOW TO THINK ABOUT IT

Normalize the pair first (sorted tuple) so A-B and B-A are one edge, then keep an edge set for dedup and a count per user. The ranking part is a separate question: top-K comes from a heap, and the single most active user can be maintained in O(1) because counts only ever go up. Mention that the edge set uses quadratic space in the worst case.

Basis: Written. Assumed rules (confirm them): a pair 'A-B' is an unordered communication edge; repeated pairs count once; a user's activity is the number of distinct partners; order by activity descending, then name; self pairs (A-A) are ignored unless a flag says otherwise.

BRUTE FORCE - PSEUDO-CODE

```
keep the set of distinct communication pairs
on every query: count the distinct partners of each user a
nd sort users by (count,
    name)
```

Time: O(E + U log U) per query | Space: O(E)

Why it is slow

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION - EDGE SET, COUNTS PER USER, COUNT BUCKETS FOR THE MAXIMUM

```
class Active:
    def __init__(self, count_self=False):
        self.edges, self.cnt, self.buckets, self.top, self
.count_self = (
            set(),
            Counter(),
            defaultdict(set),
            0,
            count_self,
        )

    def _bump(self, u):
        c = self.cnt[u]
        if c:
            self.buckets[c].discard(u)
        self.cnt[u] = c + 1
        self.buckets[c + 1].add(u)
        self.top = max(self.top, c + 1)

    def add(self, pair):
        a, b = _norm(pair)
        if (a == b and not self.count_self) or (a, b) in s
elf.edges:
            return
        self.edges.add((a, b))
        self._bump(a)
        if a != b:
```

```
            self._bump(b)

    def top_k(self, k):
        return heapq.nsmallest(k, self.cnt, key=lambda u:
(-self.cnt[u], u))

    def most_active(self):
        return set(self.buckets[self.top]) if self.top els
e set()
```

Time: O(1) per pair, O(U log k) for top-K, O(1) for the most active user | Space: O(E)

Why

A new edge increments the count of both users. Buckets map each count to the set of users with it, so a user moves from bucket c to bucket c + 1. Because counts never decrease, the highest non-empty bucket index only grows, which makes the most active users available in O(1). Top-K uses a heap over the count dict.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

```
def _norm(pair):
    a, b = pair.split("-")
    return (a, b) if a <= b else (b, a)


class Active:
    def __init__(self, count_self=False):
        self.edges, self.cnt, self.buckets, self.top, self
.count_self = (
            set(),
            Counter(),
            defaultdict(set),
            0,
            count_self,
        )

    def _bump(self, u):
        c = self.cnt[u]
        if c:
            self.buckets[c].discard(u)
        self.cnt[u] = c + 1
        self.buckets[c + 1].add(u)
        self.top = max(self.top, c + 1)

    def add(self, pair):
        a, b = _norm(pair)
        if (a == b and not self.count_self) or (a, b) in s
elf.edges:
            return
        self.edges.add((a, b))
        self._bump(a)
        if a != b:
            self._bump(b)

    def top_k(self, k):
        return heapq.nsmallest(k, self.cnt, key=lambda u:
(-self.cnt[u], u))

    def most_active(self):
        return set(self.buckets[self.top]) if self.top els
e set()
```

Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

NOTES FROM CANDIDATES

A Java solution used an ordered map / tree structure to keep counts sortable.

The interviewer asked many clarification questions and expected dry runs.

The first version did not require self-communication handling; it was added as a coded follow-up.

Account for the per-user contact sets when explaining space use: the distinct relationship state can take quadratic space in the number of users. The interviewer explicitly challenged an initial linear-space claim.

A fixed-size priority-queue proposal did not finish the changing-K discussion; the interviewer asked for further optimization.

Maintaining the API variant

With registrations only and any tied maximum acceptable, normalize undirected pairs and store the distinct edges. For a new edge, increment each distinct endpoint once; an accepted self-edge changes one count, while an ignored self-edge changes none. Clarify that choice and the empty-state response.

Cache a user with the largest count. Counts never decrease, so only the endpoints of a new edge can displace the cached winner; duplicate edges do not change it. Reading this cache makes mostActiveUser() O(1), with expected O(1) registration work using hash-based edge membership and counters under unit-cost user identifiers. A stricter tie rule requires separate handling.

For changing K, one option is a balanced ordered index of all users keyed by count and an agreed tie-breaker. Remove and reinsert the affected users when counts change. Updates cost O(log U); starting at the highest-ranked user and walking entries in descending-count order costs O(log U + K) for at most U returned users. This supports different K values without rebuilding a fixed-capacity heap and is a preparation option, not a specified interviewer solution.

Counters, the maximum cache, and the ranking index use O(U) space. Distinct relationship storage adds O(E), giving O(U + E) overall and O(U^2) in a dense graph.

PREPARATION

Implement the pair-normalization layer separately from count maintenance.

Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.

Dry-run duplicate edges and self-edges before discussing top-K optimization.

Dynamic Programming and State-Space Optimization 15 questions

MEDIUM | IN APP LC 198 | PracHub | Technical Screen | 120 of 196

★ TOP 75: rank 61 in the Medium and Hard priority list

★ Maximize sum without choosing adjacent elements

STUDY: TYPE | Google frequency on LeetCode: all time 57.2, last 30 days 0 | book priority 44 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #198 House Robber

LeetCode match: LC 198 House Robber (inferred from overview)

Original: https://prachub.com/coding-questions/maximize-sum-without-choosing-adjacent-elements

QUESTION

Overview: This question evaluates a candidate's understanding of array-based optimization and dynamic programming concepts for computing a maximum sum under non-adjacent selection constraints.

APPROACH

dp: take = skip_prev + x, skip = max(take_prev, skip_prev); O(n) time, O(1) space.

HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

Basis: LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

BRUTE FORCE - PSEUDO-CODE

```
best(i): if i is past the end: return 0
return the larger of best(i + 1) (skip) and nums[i] + best
(i + 2) (take)
```

Time: O(2^n) | Space: O(n)

Why it is slow

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(nums):
    incl = 0
    excl = 0
    for x in nums:
        new_incl = excl + x
        new_excl = max(excl, incl)
        incl = new_incl
        excl = new_excl
    return max(incl, excl)
```

Time: O(n) | Space: O(1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

```python
def _house_dp(nums):
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


def _house_circular(nums):
    if len(nums) == 1:
        return nums[0]
    return max(_house_dp(nums[1:]), _house_dp(nums[:-1]))
```

Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

```python
def _house_choice(nums):
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    chosen, i = [], 0
    while i < n:
        if nums[i] + dp[i + 2] >= dp[i + 1] and nums[i] +
dp[i + 2] == dp[i]:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return chosen
```

Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FUNCTION AND TEST CASES

Signature

```python
def solution(nums: list[int]) -> int:
```

Sample call

```
solution([2, 7, 9, 3, 1])
```

Test cases

Input	Expected
([2, 7, 9, 3, 1],)	12
([1, 2, 3, 1],)	4

MEDIUM | IN APP LC 198 | Exponent | 121 of 196

★ TOP 75: rank 74 in the Medium and Hard priority list

★ Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

STUDY: TYPE | Google frequency on LeetCode: all time 57.2, last 30 days 0 | book priority 45 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #198)

LeetCode match: LC 198 House Robber (inferred from short statement)

Original: https://www.aced.io/questions/4646/house-robber

QUESTION

Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

APPROACH

dp: rob = skip_prev + x, skip = max(rob_prev, skip_prev); O(n) time, O(1) space.

HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

Basis: LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

BRUTE FORCE - PSEUDO-CODE

```
best(i): if i is past the end: return 0
return the larger of best(i + 1) (skip) and nums[i] + best
(i + 2) (take)
```

Time: O(2^n) | Space: O(n)

Why it is slow

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #198 HOUSE ROBBER)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Rob houses in a row; can't rob adjacent houses. Maximize stolen amount. Right?"

"A few quick questions:
Can we skip multiple houses? Yes — skip any number between robberies.
All amounts non-negative? Yes."

"Let me walk: nums=[1,2,3,1] -> rob 1+3=4 ok. nums=[2,7,9,3,1] -> 2+9+1=12 ok

PHASE 3 — OPTIMIZE

Approach: optimal approach
"The bottleneck is recomputing overlapping subproblems.
DP with O(1) space — only need last two values:
at each house: rob=nums[i]+prev2; skip=prev1. Take max.

Time: O(n). Space: O(1)."
Key insight: rob[i] = max(nums[i] + rob[i-2], rob[i-1]) — only the previous two states matter, so two rolling variables replace an O(n) array.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _198_HouseRobber:
    def rob(self, nums):
        prev2 = prev1 = 0
        for amount in nums:
            prev2, prev1 = prev1, max(prev1, prev2 + amo
unt)
        return prev1
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace rob on [2,7,9,3,1]."
dp[0]=2. dp[1]=max(2,7)=7.
dp[2]=max(dp[1], dp[0]+9)=max(7,11)=11.
dp[3]=max(dp[2], dp[1]+3)=max(11,10)=11.
dp[4]=max(dp[3], dp[2]+1)=max(11,12)=12.
Return 12 ok (rob houses 1,3,5: 2+9+1=12).
[1,2,3,1]: dp=[1,2,4,4]. Return 4 ok (rob houses 1,3: 1+3=4).
WHY dp[i]=max(dp[i-1], dp[i-2]+nums[i]): either skip house i (take dp[i-1]) or rob it (skip i-1).
Two decisions per house, track the running optimum.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n): single pass. Space O(1): two rolling variables.
Follow-up: House Robber II (#213) — circular arrangement; run two passes: [0..n-2],[1..n-1].
Follow-up: House Robber III (#337) — tree structure; DP on tree nodes.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

```python
def _house_dp(nums):
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


def _house_circular(nums):
```

```
    if len(nums) == 1:
        return nums[0]
    return max(_house_dp(nums[1:]), _house_dp(nums[:-1]))
```

Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

```
def _house_choice(nums):
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    chosen, i = [], 0
    while i < n:
        if nums[i] + dp[i + 2] >= dp[i + 1] and nums[i] +
dp[i + 2] == dp[i]:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return chosen
```

Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

OFFICIAL PAGE TEXT

You are given an array representing the value in the houses e.g. [100, 20, 40, 70, 80]. Return 100 + 40 + 80 = 220

MEDIUM | IN APP LC 494 | Exponent | 122 of 196

Explain how to find a target sum in an array.

STUDY: TALK | Google frequency on LeetCode: all time 41.1, last 30 days 0 | book priority 80 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: your app's interview approach (LeetCode #494)
LeetCode match: LC 494 Target Sum
3 interviews
Original: https://www.aced.io/questions/1439/find-target-sum-array

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

APPROACH

Count assignments: DP on sums (subset-sum transform: P = (total + target) / 2), or memoized DFS on (index, sum). O(n * sum).

HOW TO THINK ABOUT IT

Put a + in front of the numbers in a set P and a - in front of the rest N. Then sum(P) - sum(N) = S and sum(P) + sum(N) = total, so sum(P) = (total + S) / 2. The problem becomes 'count the subsets with this sum', a one-dimensional knapsack count. If (total + S) is odd or negative the answer is 0.

Basis: LeetCode 494, Target Sum: assign + or - to every number and count the assignments that add up to S (the data matches this reading; a Two Sum reading is given as a follow-up).

BRUTE FORCE - PSEUDO-CODE

```
go(i, total): if i == len: count it if total == S
else go(i + 1, total + nums[i]) + go(i + 1, total - nums[i
])
```

Time: O(2^n) | Space: O(n)

Why it is slow

Two branches per number: O(2^n). The numbers with a + sign form a subset with sum (total + S) / 2, so a subset-sum counting table is O(n * sum).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #494 TARGET SUM)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Assign + or - to each number in nums so they sum to target.
Count the number of ways. Right?"

"A few quick questions:
nums can include 0s (still counted as separate +/- choices even if value is 0)? Yes."

"Let me walk: nums=[1,1,1,1,1], target=3 -> 5 ways ok

PHASE 3 — OPTIMIZE

Say: "DP on sum offset — O(n * sum). Walk-through:
Map: sum ranges from -total to +total. Use offset=total to shift to [0, 2*total].
dp[s] = number of ways to reach sum s.
Initial: dp[total] = 1 (sum=0, shifted by total).
For each num: new_dp[s+num] += dp[s]; new_dp[s-num] +=

dp[s].
Return dp[total + target].
WHY sum offset: dp array can't have negative indices; shift by total to handle -sum..+sum.
WHY new_dp each step: assignments for current num must not interfere with each other.
Optimization: observe P-N=target and P+N=total -> P=(total+target)/2 -> subset sum problem.
Time: O(n * sum). Space: O(sum).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _494_TargetSum:
    def findTargetSumWays(self, nums, target):
        total = sum(nums)
        if abs(target) > total or (total + target) % 2 !
= 0:
            return 0
        subset_sum = (total + target) // 2
        dp = [0] * (subset_sum + 1)
        dp[0] = 1
        for num in nums:
            for s in range(subset_sum, num - 1, -1):
                dp[s] += dp[s - num]
        return dp[subset_sum]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace findTargetSumWays([1,1,1,1,1], target=3)."
Sum=5. Subset with + signs: P. Subset with -: N=5-P. P-N=3 -> P=4. Subset sum = 4.
dp=[0]*6. dp[0]=1. Process each 1:
After 1st '1': dp[1]+=dp[0]=1. dp=[1,1,0,0,0,0].
After 2nd '1': dp[2]+=dp[1]=1. dp[1]+=dp[0]=1. dp=[1,2,1,0,0,0].
After 3rd '1': dp[3]+=dp[2],dp[2]+=dp[1],dp[1]+=dp[0]. dp=[1,3,3,1,0,0].
After 4th '1': dp=[1,4,6,4,1,0]. After 5th '1': dp=[1,5,10,10,5,1].
dp[4]=5 ok (number of ways to choose 4 from 5 ones for '+' sign = C(5,4)=5).

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n*sum). Space O(sum).
Follow-up: Partition Equal Subset Sum (#416) — same subset-sum DP, simpler target.
Follow-up: Coin Change II (#518) — unbounded version (reuse allowed), forward iteration instead.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The question was really 'find two numbers that add up to the target' (Two Sum).

One pass with a dictionary from value to index: for each x look for target - x among earlier values. O(n) time, O(n) space; with a sorted array use two pointers in O(1) space.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return None
```

Only the existence of a subset with the target sum.

The same subset-sum table with booleans (or a bitset: reach |= reach << x) answers existence in O(n * sum / word size).

Negative numbers or a huge sum.

With negatives shift all values to be non-negative, or use a dictionary from reachable sum to count (memoised DFS on (index, sum)); with a huge sum the dictionary is only as large as the number of distinct reachable sums. Cost: O(n * R) with a dictionary over R reachable sums; shifting for negatives adds O(n).

OFFICIAL PAGE TEXT

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples

```
nums = [1, 1, 1, 1, 1]
targetSum = 3
output: 5

nums = [1, 2, 1]
targetSum = 2
output: 2

nums = [1, 2, 7, 1, 5, 3]
targetSum = 8
output: -1 (Explanation: Since the total sum of the number
s plus the target is odd, it's not possible to reach the t
arget sum.)
```

The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\text{nums} = [a_1, a_2, ..., a_n]nums=[a1,a2,...,an], we need to assign either a +++ or a --- to each number so that the total sum equals a target SSS.Let's denote the sum of the entire list as sum(nums)=totalSum\text{sum(nums)} = \text{totalSum}sum(nums)=totalSum.The problem can be rephrased as finding subsets of nums\text{nums}nums such that the difference between the sum of elements in the subset and the sum of elements not in the subset equals SSS.Formulating the Equation: Let PPP be

the sum of elements with a +++ sign and NNN be the sum of elements with a --- sign. We need P-N=SP - N = SP-N=S. We also know that P+N=totalSumP + N = \text{totalSum}P+N=totalSum, where totalSum\text{totalSum}totalSum is the sum of all elements in nums\text{nums}nums. Combining the Equations: Adding these two equations:P-N+P+N=S+totalSum2P=S+totalSumP - N + P + N = S + \text{totalSum}\\ 2P = S + \text{totalSum} P-N+P+N=S+totalSum2P=S+totalSumTherefore, P=S+totalSum2P = \frac{S + \text{totalSum}}{2}P=2S+totalSum.The problem now reduces to finding subsets of nums\text{nums}nums whose sum equals S+totalSum2\frac{S + \text{totalSum}}{2}2S+totalSum. Approach We define a recursive function find_subset that takes an index, the list of numbers, the target subset sum, and a memoization table (dp). The function recursively explores two possibilities at each step: Include the current number in the subset. Exclude the current number from the subset. The base cases for the recursion are: If the target subset sum (s) is zero, it means we found a valid subset, so we return 1. If we reach the end of the list without finding the subset sum, we return 0. If we have already computed the value for the current state (index and subset sum), we return the stored value from the memoization table to avoid redundant calculations. The changeSigns function calculates the total sum of the numbers and checks if it is possible to partition them to meet the required target sum S. If it is possible, it initializes the memoization table and calls the find_subset function to compute the number of ways to achieve the target sum. Solution Time Complexity: The time complexity of this solution is O(n x (totalSum + S)) / 2, where n is the number of elements in the input list. This is because each state in the memoization table can be computed in constant time, and there are n x (totalSum + S) / 2 states to fill. Space Complexity: The space complexity of this solution is O(n x (totalSum + S) / 2) due to the memoization table used to store intermediate results. Additionally, the recursive call stack can go up to O(n) in the worst case, but this is dominated by the space required for the memoization table.Watch our mock Flipkart SWE (software engineering) interview. Neamah asks Rohan (Flipkart SWE) the Target Sum leetcode question.No answers contributed by the community yet.Related questionsFind a triplet in an array with a given sum.Given an array, find the two sum.Find quadruplets in an array with a given sum.Related coursesCourseCourseSoftware Engineering Interview PrepLand your dream software engineering role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates—practice data structures, algorithms, system design, people management, behavioral interviews, and more.Start learningCourseCourseSystem Design InterviewsLearn how to answer the latest system design questions across product, infrastructure, and AI domains. Features in-depth video examples, written breakdowns, and helpful reference patterns. Watch senior engineers and managers answer these questions in mock interview videos.Start learningPythonJavaScriptJavaC++PythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

---

MEDIUM | IN APP LC 22 | Exponent | 123 of 196

★ TOP 75: rank 19 in the Medium and Hard priority list

★ Generate Parentheses

STUDY: TYPE | Google frequency on LeetCode: all time 61.6, last 30 days 46.9 | book priority 12 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #22)

LeetCode match: LC 22 Generate Parentheses

+36 interviews

Original: https://www.aced.io/questions/1594/generate-parentheses

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(()())()", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also

APPROACH

Backtracking: add '(' while open < n and ')' while close < open. Catalan-many outputs.

HOW TO THINK ABOUT IT

Build the string left to right and only make moves that can still lead to a valid result: you may add '(' while fewer than n have been opened, and ')' only while there are more opens than closes. Because invalid prefixes are never extended, every leaf is a valid answer, and the count is the Catalan number.

Basis: LeetCode 22, Generate Parentheses: all well-formed strings with n pairs.

BRUTE FORCE - PSEUDO-CODE

```
for every sequence of n '(' and n ')': if it is balanced (
no prefix has more ')' than
    '(': keep it
```

Time: O(2^(2n) * n) | Space: O(2^(2n))

Why it is slow

Builds exponentially many invalid strings. Backtracking only adds '(' while fewer than n are used and ')' while fewer than the opens, so it

builds only valid prefixes (the Catalan number of results).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #22 GENERATE PARENTHESES)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Given n pairs of parentheses, generate all combinations of well-formed parentheses.
Well-formed means every '(' is closed before any preceding '(' closes. Right?"

"A few quick questions:
n=0 -> []? n=1 -> ['()']? Yes to both. Any ordering of output is fine."

"Let me walk: n=2 -> ['(())','()()'] ok. n=3 -> 5 combinations ok

PHASE 3 — OPTIMIZE

The bottleneck is generating invalid strings then filtering them.
Backtracking with validity pruning — only place characters that keep the string valid:
Add '(' if open count < n. Add ')' if close count < open count.
This generates ONLY valid strings — no wasted work.

Time: O(4^n / sqrt(n)) — the nth Catalan number. Space: O(n) call depth."
Key insight: enforcing open < n and close < open at each step generates only valid strings — the pruning shrinks the search tree to exactly the n-th Catalan number leaves.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _22_GenerateParentheses:
    def generateParenthesis(self, n):
        result = []
        def backtrack(path, open_count, close_count):
            if len(path) == 2 * n:
                result.append(''.join(path)); return
            if open_count < n:
                path.append('('); backtrack(path, open_c
ount + 1, close_count); path.pop()
            if close_count < open_count:
                path.append(')'); backtrack(path, open_c
ount, close_count + 1); path.pop()
        backtrack([], 0, 0)
        return result
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace generateParentheses n=3."
backtrack(s='',open=0,close=0):
Add '(': (1,0)->'('. Add '(': (2,0)->'((('. Add '(': (3,0)->'(((('. Add ')': close<open -> (3,1)->'((()'.
Add ')': (3,2)->'((()'->')'. Add ')': (3,3)->'((()))'. len==6->append ok.
Backtrack.
From (3,2): done. From (3,1)->Add ')': (3,2)->'(()('->? No: backtrack path '((' with open=2,close=1.
Full trace: open tracks '(' added, close tracks ')' added. Invariant: close<=open, open<=n.
Leaves (len=6): '((()))','(()())','(())()','()(())','()()()' — exactly 5=Catalan(3) ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(4^n / sqrt(n)): Catalan number of valid strings, each takes O(n) to record.
Brute O(2^(2n)) — exponentially worse.
Follow-up: count valid sequences without generating — Catalan(n) = C(2n,n)/(n+1).
Follow-up: Longest Valid Parentheses (#32) — find the longest valid substring using DP or stack."
Space: O(n) recursion depth + O(result) for storing all combinations.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return only the k-th string, or just the number of strings.

The count is the Catalan number C(n) = (2n choose n) / (n + 1). For the k-th string count the completions that start with '(' at each step and skip them if k is larger (unranking), using a small DP over (opens left, balance).

```
def kth_parenthesization(n, k):
    """The k-th (1-based, in the order of generateParenthe
sis) well-formed string, without generating the others."""

    @lru_cache(maxsize=None)
    def count(opened_left, balance):
        if opened_left == 0:
            return 1
        total = count(opened_left - 1, balance + 1)
        if balance > 0:
            total += count(opened_left, balance - 1)
        return total

    out, opened, balance = [], 0, 0
    for _ in range(2 * n):
        if opened < n:
            c = count(n - opened - 1, balance + 1)
            if k <= c:
                out.append("(")
                opened, balance = opened + 1, balance + 1
                continue
            k -= c
        out.append(")")
        balance -= 1
    return "".join(out)
```

Several bracket types, such as () [] {}.

Multiply the choices: at each position you may open any of the types (until its count is used) and may close only the type on top of a stack. The pruning rule (never close more than you have opened) becomes 'close only the most recent unclosed bracket'.

Avoid recursion.

Use an explicit stack of (string so far, opens, closes) states or build the strings by length with a queue. The pruning conditions stay the same.

OFFICIAL PAGE TEXT

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples

```
n = 1
output: ["()"]

n = 2
output: ["(())", "()()"]

n = 3
output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

n = 4
output should contain: ["(((())))", "((()()))", "((())())"
, "((()))()", "(()(()))", "(()()())", "(()())()", "(())(()
)", "(())()()", "()((()))", "()(()())", "()(())()", "()()(
())", "()()()()"]
```

The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also maintains the current combination of the string being formed. Base Case: If no more parentheses are remaining to add (both remaining_open and remaining_close are zero), the current combination is a valid sequence and is added to the result list. Recursive Case: Add an opening parenthesis (if there are any remaining and recursively call the function with one less opening parenthesis. Add a closing parenthesis) if the number of remaining closing parentheses is greater than the remaining opening parentheses and recursively call the function with one less closing parenthesis. Pruning Invalid Paths: If at any point the number of closing parentheses left is less than the number of opening parentheses left, it means the current sequence cannot be completed to a valid sequence, so we stop exploring that path. Implementation Steps Initialize: Create a result list to store all valid sequences. Call the recursive helper function with initial values (n opening and n closing parentheses). Recursive Helper Function: Track the number of remaining opening and closing parentheses. Track the current combination being formed. Append valid combinations to the result list when the base case is reached. Backtrack: After adding an opening or closing parenthesis, remove it (backtrack) before trying the next possibility. This ensures that we explore all possible valid combinations. Example Let's consider n = 3: Start with an empty combination "", 3 opening, and 3 closing parentheses. Add an opening parenthesis: "(", 2 opening, and 3 closing. Continue adding opening parentheses until you can no longer add without violating the rules. Then add closing parentheses and ensure that at no point the closing parentheses exceed the opening ones. Once you form a valid combination like "((()))", add it to the result list. Backtrack and explore other combinations. Solution Time Complexity: The time complexity of this solution is O(22n)O(2^{2n})O(22n). This is because each position in the string can either be an opening or closing parenthesis, and we need to generate all possible sequences of length 2n2n2n. Space Complexity: The space complexity of the solution is O(2n)O(2n)O(2n) which simplifies to O(n)O(n)O(n). This is because: The recursion depth is 2n2n2n in the worst case (each recursive call adds either an opening or a closing parenthesis). The combination string being formed is of length 2n2n2n. In this mock interview video, Aman (Google SWE) answers the interview question, "Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses."No answers contributed by the community yet.

MEDIUM | IN APP LC 53 | Exponent | 124 of 196

★ TOP 75: rank 20 in the Medium and Hard priority list

★ Find the maximum subarray sum.

STUDY: TYPE | Google frequency on LeetCode: all time 64.4, last 30 days 41.1 | book priority 13 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #53)

LeetCode match: LC 53 Maximum Subarray

+1522 interviews

Original: https://www.aced.io/questions/973/maximum-subarray-sum

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

APPROACH

Kadane's: cur = max(x, cur + x); best = max(best, cur). O(n).

HOW TO THINK ABOUT IT

The best subarray ending at position i either extends the best one ending at i - 1 or starts fresh at i; it extends only if the previous best is positive. So keep one number, the best sum ending here, and one answer: cur = max(x, cur + x), best = max(best, cur).

Basis: LeetCode 53, Maximum Subarray (Kadane's algorithm). The subarray must be non-empty.

BRUTE FORCE - PSEUDO-CODE

```
best = -infinity; for every start i: running sum over the
subarray; best = max(best,
    running sum)
```

Time: O(n^2) | Space: O(1)

Why it is slow

O(n^2) subarrays. Kadane's algorithm keeps the best sum ending at each position (extend or restart) in O(n) time and O(1) space.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #53 MAXIMUM SUBARRAY)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Find the contiguous subarray with the largest sum and return that sum.
Array can have negative numbers; at least one element. Right?"

"A few quick questions:
Can all elements be negative? Yes — return the largest (least negative) single element.
Return sum, not the subarray? Yes."

"Let me walk: nums=[-2,1,-3,4,-1,2,1,-5,4] -> [4,-1,2,1] sum=6 ok

PHASE 3 — OPTIMIZE

The bottleneck is the O(n²) nested scan.
Kadane's algorithm: track current_sum and global max.
At each element: current_sum = max(num, current_sum + num).
If adding previous sum makes it worse than starting fresh, reset.

Time: O(n). Space: O(1).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _53_MaximumSubarray:
    def maxSubArray(self, nums):
        current_sum = max_sum = nums[0]
        for num in nums[1:]:
            current_sum = max(num, current_sum + num)
            max_sum = max(max_sum, current_sum)
        return max_sum
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace maxSubArray on [-2,1,-3,4,-1,2,1,-5,4]."
Kadane's: curr=nums[0]=-2. best=-2.
i=1: curr=max(1,-2+1)=max(1,-1)=1. best=1.
i=2: curr=max(-3,1-3)=max(-3,-2)=-2. best=1.
i=3: curr=max(4,-2+4)=max(4,2)=4. best=4.
i=4: curr=max(-1,4-1)=max(-1,3)=3. best=4.
i=5: curr=max(2,3+2)=5. best=5.
i=6: curr=max(1,5+1)=6. best=6.
i=7: curr=max(-5,6-5)=1. best=6.
i=8: curr=max(4,1+4)=5. best=6.
Return 6 ok (subarray [4,-1,2,1]).
WHY curr=max(nums[i], curr+nums[i]): restart subarray if extending would make it worse.

WHY DP terms: using state + recurrence/transition to combine prior results efficiently.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n): single pass. Space O(1): two variables.
Follow-up: Maximum Product Subarray (#152) — track both max and min (negatives flip).
Follow-up: Maximum Sum Circular Subarray (#918) — max(kadane, total_sum - min_kadane).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the subarray itself.

Remember where the current run started: when starting fresh at i set start = i, and when cur beats best record (start, i). Same single pass.

```python
def max_subarray_with_indices(nums):
    cur, start, best, bs, be = nums[0], 0, nums[0], 0, 0
    for i in range(1, len(nums)):
        if nums[i] > cur + nums[i]:
            cur, start = nums[i], i
        else:
            cur += nums[i]
        if cur > best:
            best, bs, be = cur, start, i
    return best, bs, be
```

The array is circular (LeetCode 918).

The best circular subarray is either an ordinary one or the whole array minus the worst ordinary one (the minimum subarray): max(kadane max, total - kadane min), taking care of the all-negative case.

```python
def max_subarray_circular(nums):
    total, cur_max, best_max, cur_min, best_min = (
        sum(nums),
        0,
        float("-inf"),
        0,
        float("inf"),
    )
    for x in nums:
        cur_max, cur_min = max(cur_max + x, x), min(cur_mi
n + x, x)
        best_max, best_min = max(best_max, cur_max), min(b
est_min, cur_min)
    return best_max if best_max < 0 else max(best_max, tot
al - best_min)
```

Divide and conquer instead of Kadane.

Split the array; the best subarray is entirely left, entirely right, or crosses the middle (best suffix of the left plus best prefix of the right). T(n) = 2T(n/2) + O(n) = O(n log n); useful when the data is split across machines.

OFFICIAL PAGE TEXT

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples

```
Input: nums = [2, 3, -2, 4]
Output: 7
Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.

Input: nums = [1, -1, -5, -4]
Output: 1
Explanation: The maximum sum is 1, which is the single ele
ment with the highest value.
```

Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?Our solution for finding the maximum subarray sum processes the input array by iterating through it while maintaining two variables: max_current and max_global. The max_current variable keeps track of the maximum sum of the subarray ending at the current position, while max_global stores the maximum sum encountered so far. For each element in the array, we update max_current by taking the maximum of the current element alone or the sum of the current element and max_current. This decision ensures that max_current always holds the maximum sum of the subarray that ends at the current element. We then update max_global to be the maximum of max_global and max_current. If the input array is empty, the function returns 0 since there are no subarrays to consider. This approach uses a simple iteration and a couple of variables, making it both straightforward and efficient. Time Complexity: The solution has a time complexity of O(n), where n is the number of elements in the array. This is because we iterate through the array once, performing constant-time operations (comparison and addition) for each element. Space Complexity: The solution uses O(1) space since it only requires a fixed amount of extra space for the max_current and max_global variables, regardless of the input size. This makes the solution very efficient in terms of space usage. NoteThis algorithm is known as Kadane's Algorithm.No answers contributed by the community yet.

MEDIUM | IN APP LC 416 | Exponent | 125 of 196

Partition an array into two sub-arrays with equal sum.

STUDY: TYPE | Google frequency on LeetCode: all time 46.6, last 30 days 0 | book priority 67 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: your app's interview approach (LeetCode #416)
LeetCode match: LC 416 Partition Equal Subset Sum
+26 interviews
Original: https://www.aced.io/questions/969/partition-array-equal-sum

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

APPROACH

Subset-sum DP to total/2 (boolean array or bitset); odd totals are impossible. O(n * sum/2).

HOW TO THINK ABOUT IT

Two equal parts mean each sums to total / 2, so if the total is odd the answer is no. Otherwise the question is whether some subset reaches total / 2, which is subset-sum. A bitset where bit s says 'sum s is reachable' updates with one shift and one OR per number.

Basis: LeetCode 416, Partition Equal Subset Sum (the two parts are subsets, not contiguous sub-arrays; the statement says sub-arrays, so the contiguous reading is given as a follow-up).

BRUTE FORCE - PSEUDO-CODE

```
total = sum of all numbers; if total is odd: false
for every subset: if its sum equals total / 2: return true
```

Time: O(2^n * n) | Space: O(n)

Why it is slow

2^n subsets. Subset-sum as a reachability table (or a bitset shifted by each number) is O(n * sum).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #416 PARTITION EQUAL SUBSET SUM)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Partition nums into two subsets with equal sum.
Equivalent: can any subset sum to total/2? Return bool. Right?"

"A few quick questions:
If total is odd, immediately false? Yes — can't split odd into two equal integers.
All values positive? Yes."

"Let me walk: nums=[1,5,11,5] -> total=22, target=11. [1,5,5] sums to 11 -> true ok

PHASE 3 — OPTIMIZE

The bottleneck is recomputing overlapping subset decisions.
0/1 Knapsack DP: dp[j] = can we form sum j using a subset of nums seen so far.
Process each number: update dp from right to left to avoid reusing the same number.

Time: O(n*target). Space: O(target).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _416_PartitionEqualSubsetSum:
    def canPartition(self, nums):
        total = sum(nums)
        if total % 2: return False
        target = total // 2
        dp = [False] * (target + 1)
        dp[0] = True
        for num in nums:
            for j in range(target, num - 1, -1):
                dp[j] = dp[j] or dp[j - num]
        return dp[target]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace canPartition([1,5,11,5])."
sum=22, target=11. dp[i]=can we make sum i.
dp[0]=True. Process coin 1: dp[1]=dp[0]=T.
Process 5: dp[6]=dp[1]=T. dp[5]=dp[0]=T.
Process 11: dp[11]=dp[0]=T ok -> already found!
Process 5: dp[10]=dp[5]=T. dp[16]=dp[11]=T. dp[11] already T.
Return dp[11]=True ok ([1,5,5]=11, [11]=11).
[1,2,3,5]: sum=11, odd -> return False immediately ok
WHY iterate coin in outer, sum in inner (reverse): 0/1 knapsack — each item used at most once.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n*target). Space O(target): 1D boolean array.
Right-to-left is essential — left-to-right would be unbounded knapsack.
Follow-up: Subset Sum (exact sum) — same DP, return dp[target].
Follow-up: Last Stone Weight II (#1049) — minimize |S1-S2|; same partition approach.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the two subsets.

Keep, for each reachable sum, the list of indices (or a parent pointer) that first reached it; read off the subset for total / 2 and give the rest to the other side.

```
def partition_subsets(nums):
    total = sum(nums)
    if total % 2:
        return None
    half = total // 2
    reach = [None] * (half + 1)
    reach[0] = []
    for i, x in enumerate(nums):
        for s in range(half, x - 1, -1):
            if reach[s] is None and reach[s - x] is not No
ne:
                reach[s] = reach[s - x] + [i]
    if reach[half] is None:
        return None
    chosen = set(reach[half])
    return [nums[i] for i in sorted(chosen)], [
        nums[i] for i in range(len(nums)) if i not in chos
en
    ]
```

Minimise the difference between the two parts (LeetCode 1049).

Compute all reachable subset sums with the same bitset and choose the reachable s closest to total / 2: the answer is total - 2s.

```
def min_subset_difference(nums):
    total = sum(nums)
    reach = 1
    for x in nums:
        reach |= reach << x
    return min(abs(total - 2 * s) for s in range(total + 1
) if reach >> s & 1)
```

The parts must be contiguous sub-arrays (a split point).

Then only one split can work: compute prefix sums and check whether some prefix equals total / 2. O(n), O(1) space; no DP is needed.

OFFICIAL PAGE TEXT

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples

```
Input: [2, 4, 11, 5]
Output: True
Explanation: The array can be partitioned into two subsets
 [2, 4, 5] and [11] with equal sum.

Input: [2, 7, 3, 1]
Output: False
Explanation: The array cannot be partitioned into two subs
ets with equal sums.
```

Our solution processes the input array by first calculating the total sum of the array elements and then determining if it can be partitioned into two subsets with equal sums. The core idea is based on dynamic programming. Calculate Total Sum: We start by computing the total sum of the array elements. If the total sum is odd, it is immediately impossible to partition it into two subsets with equal sums, so we return false. Define Target Sum: If the total sum is even, we calculate the target sum for each subset, which is half of the total sum. Dynamic Programming Array: We use a boolean array dp where dp[i] indicates whether a subset sum of i is achievable. We initialize dp[0] to true because a subset sum of 0 is always achievable (by taking no elements). Iterate Through the Array: For each number in the array, we update the dp array in reverse order (from target down to the current number). This reverse iteration ensures that each number is only used once in the current update. If a subset sum i - num was achievable, then adding the current number num makes dp[i] achievable. We update dp[i] to true accordingly. Check Final Result: After processing all numbers, if dp[target] is true, it means there is a subset with the required sum equal to target, indicating that the array can be partitioned into two subsets with equal sums. Otherwise, we return false. The use of a

boolean array ensures that we efficiently keep track of achievable subset sums. Time Complexity: The solution has a time complexity of O(n * target), where n is the number of elements in the array and target is half of the total sum. This is because we iterate through the array and update the dp array for each element. Space Complexity: The solution uses O(target) space for the dp array, where target is half of the total sum. This space is needed to store the achievable subset sums.No answers contributed by the community yet.

MEDIUM | IN APP LC 46 | Exponent | 126 of 196

Permutations

STUDY: TYPE | Google frequency on LeetCode: all time 49.8, last 30 days 0 | book priority 62 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #46)

LeetCode match: LC 46 Permutations

+25 interviews

Original:

https://www.aced.io/questions/2212/leetcode-medium-permutations-dp

QUESTION

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

APPROACH

Backtracking with a used[] array (or swap-based in place). n! outputs.

HOW TO THINK ABOUT IT

The output has n! items, so no algorithm can beat O(n * n!); the question is only whether you generate them without waste. Build a permutation one position at a time, marking which elements are used, and undo the choice when you come back (backtracking). Every follow-up (duplicates, k-th, next) is a variation on 'what are the legal choices at this position'.

Basis: LeetCode 46 (Permutations), 47 (with duplicates), 31 (Next Permutation) and 60 (Permutation Sequence).

BRUTE FORCE - PSEUDO-CODE

```
for every sequence of length n over the values: if all its
 values are distinct: keep
    it
```

Time: O(n^n * n) | Space: O(n!)

Why it is slow

n^n sequences when only n! are valid, so almost all the work is wasted. Backtracking with a used marker only builds valid prefixes, O(n * n!).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #46 PERMUTATIONS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly. Given an array of distinct integers, return all possible permutations in any order. Right?"

"A few quick questions:
n is at most 6 so 6!=720 permutations is fine. All elements are distinct? Yes."

"Let me walk: nums=[1,2,3] -> 6 permutations in any order ok

PHASE 3 — OPTIMIZE

The bottleneck is the O(n) visited array — we can eliminate it. Swap-based backtracking: maintain a 'start' pointer. Everything before start is already placed; swap any element from [start, n) into position start,
recurse, then swap back.

Same O(n! * n) time but O(n) space and no visited array.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _46_Permutations:
    def permute(self, nums):
        result = []
        def backtrack(start):
            if start == len(nums):
                result.append(list(nums)); return
            for i in range(start, len(nums)):
                nums[start], nums[i] = nums[i], nums[sta
rt]
                backtrack(start + 1)
                nums[start], nums[i] = nums[i], nums[sta
rt]
        backtrack(0)
        return result
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace permutations([1,2,3])."
backtrack(path=[], used=[F,F,F]):
Pick 1(i=0, used[0]=T): backtrack([1],[T,F,F]):
Pick 2(i=1, used[1]=T): backtrack([1,2],[T,T,F]):
Pick 3(i=2): path=[1,2,3] len=3 -> append ok. Return.
used[1]=F. Pick 3(i=2, used[2]=T): backtrack([1,3],[T,F,T]):
Pick 2(i=1): path=[1,3,2] -> append ok. Return.
used[0]=F. Pick 2(i=1): backtrack([2],...): ->[2,1,3],[2,3,1].
Pick 3(i=2): ->[3,1,2],[3,2,1].
Result: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]] ok (3!=6 permutations).
Call stack depth=n. At each level: n choices -> (n-1) -> ... -> 1.
Total calls: n!×n.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n! * n): n! permutations, each O(n) to copy. Space O(n): call depth.
Visited-array approach: O(n) extra space for the boolean array.
Follow-up: Permutations II (#47) — duplicates in input. Sort first, skip duplicate
values at the same level: if i>start and nums[i]==nums[start] after sorting, skip.
Follow-up: Next Permutation (#31) — generate just the lexicographically next one.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The input has duplicates (Permutations II).

Sort first, and skip a value that equals the previous one unless the previous one is already used in the current prefix. That forces equal values to appear in a fixed order and removes duplicates without a set.

```
def permute_unique(nums):
    nums = sorted(nums)
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if used[i] or (i and x == nums[i - 1] and not
used[i - 1]):
                continue
            used[i] = True
            cur.append(x)
            go()
            cur.pop()
            used[i] = False

    go()
    return out
```

Return the next lexicographic permutation in place.

Find the last position i with a[i] < a[i + 1], swap a[i] with the smallest larger value to its right, then reverse the suffix. O(n), O(1) space. Repeat to enumerate in order.

```
def next_permutation(a):
    i = len(a) - 2
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1
    if i >= 0:
        j = len(a) - 1
        while a[j] <= a[i]:
            j -= 1
        a[i], a[j] = a[j], a[i]
    a[i + 1 :] = reversed(a[i + 1 :])
    return a
```

Return only the k-th permutation of 1..n.

Use the factorial number system: the first digit is index (k - 1) // (n - 1)! of the remaining digits, then continue with the remainder. O(n^2), no enumeration.

```
def kth_permutation(n, k):
    digits, fact, out = list(range(1, n + 1)), math.factor
ial(n - 1) if n else 1, []
    k -= 1
    for i in range(n, 0, -1):
        idx, k = divmod(k, fact)
        out.append(digits.pop(idx))
        if i > 1:
            fact //= i - 1
    return "".join(map(str, out))
```

OFFICIAL PAGE TEXT

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example

```
Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1
, 2], [3, 2, 1]]
```

Our solution generates all permutations of a given array by utilizing a recursive backtracking approach. The idea is to explore every possible way of arranging the elements by swapping them at each recursive call. Here's a breakdown of how the algorithm works: We define a helper function (backtrack) that recursively builds the permutations. This function takes two parameters: start (the index from where we start permuting) and end (the length of the array). The base case occurs when start == end, meaning that a complete permutation has been formed. We add a copy of the current permutation (stored in nums) to the result. In each recursive call, we iterate over the elements starting from the start index. For each element at position i, we: Swap the elements at start and i to bring a new element into the current position. Recursively call backtrack with start + 1, moving to the next index to fill. After the recursive call, we swap back the elements to restore the original order and continue exploring other possibilities. This process continues until all permutations are generated. The swapping ensures that we explore every possible combination of the elements. Time Complexity: The time complexity of this solution is O(n * n!), where n is the number of elements in the array. The reason for this is that generating each permutation takes O(n) time, and there are n! permutations. Space Complexity: The space complexity is O(n!) due to the storage of all permutations in the result list. Additionally, the recursion stack uses O(n) space for the recursive calls, but this is dominated by the result storage.No answers contributed by the community yet.

MEDIUM | IN APP LC 516, 5 | Exponent | 127 of 196

★ TOP 75: rank 45 in the Medium and Hard priority list

★ Find the longest palindromic subsequence using dynamic programming.

STUDY: TYPE | Google frequency on LeetCode: all time 69.4, last 30 days 0 | book priority 24 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #516)

LeetCode match: LC 516 Longest Palindromic Subsequence (title) / LC 5 Longest Palindromic Substring (statement)

+37 interviews

Original: https://www.aced.io/questions/1169/longest-palindromic-subsequence-dynamic-programming

QUESTION

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples Input: "abacc" Output: "aba" Explanation: "aba" is longer than "cc"

APPROACH

Subsequence: dp[i][j] over substrings (LCS of s and reverse(s)). Substring: expand around centers O(n^2) or Manacher O(n).

HOW TO THINK ABOUT IT

Look at the two ends of a range. If they are equal they both belong to the palindrome and you add 2 to the answer for the inside; if not, one of them is dropped, so take the better of dropping the left or the right end. The state is a range (i, j), giving an O(n^2) table filled from short ranges to long ones.

Basis: LeetCode 516, Longest Palindromic Subsequence (not a substring: letters may be skipped).

BRUTE FORCE - PSEUDO-CODE

```
lps(i, j): if i > j: return 0; if i == j: return 1
if s[i] == s[j]: return 2 + lps(i + 1, j - 1)
else return the larger of lps(i + 1, j) and lps(i, j - 1)
```

Time: O(2^n) | Space: O(n)

Why it is slow

Two branches per step and the same (i, j) range recurs: O(2^n). A table over all ranges is O(n^2) time, with a rolling row for O(n) memory.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #516 LONGEST PALINDROMIC SUBSEQUENCE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Find the length of the longest palindromic subsequence (characters need not be contiguous). Right?"

"A few quick questions:
Length 1 is always a palindrome? Yes. Empty string -> 0? Yes."

"Let me walk: s='bbbab' -> 'bbbb' len=4 ok. s='cbbd' -> 'bb' len=2 ok

PHASE 3 — OPTIMIZE

Say: "Interval DP — dp[i][j] = longest palindromic subsequence in s[i..j]. O(n²).
Base: dp[i][i] = 1 (single char is palindrome of length 1).
Fill diagonally (increasing length):
If s[i] == s[j]: dp[i][j] = dp[i+1][j-1] + 2
<- outer chars match -> wrap them around the inner palindrome.
Else: dp[i][j] = max(dp[i+1][j], dp[i][j-1])
<- skip either s[i] or s[j], take the better result.
HOW to fill: outer loop on length l=2..n; inner loop i from 0 to n-l; j=i+l-1.
WHY diagonal fill: dp[i][j] depends on dp[i+1][j-1] (smaller subproblem) which must
already be computed -> fill from shorter to longer substrings.
Answer = dp[0][n-1]. Time: O(n²). Space: O(n²).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _516_LongestPalindromicSubsequence:
    def longestPalindromeSubseq(self, s):
        n = len(s)
        dp = [1] * n
        for length in range(2, n + 1):
            new_dp = [0] * n
            for i in range(n - length + 1):
                j = i + length - 1
                if s[i] == s[j]:
                    new_dp[i] = (2 + dp[i+1] if i+1 <= j
-1 else 2)
                else:
                    prev_i = dp[i] if i < n-1 else 0
                    prev_j = dp[i+1] if i+1 < n else 0
                    new_dp[i] = max(prev_i, prev_j)
            dp = new_dp
        return dp[0]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace longestPalindromeSubseq on 'bbbab'."
dp[i][j] = LPS of s[i..j]. dp[i][i]=1 (single char). Fill diagonally.
len=2: dp[0][1]:'bb'->b==b->dp[0][1]=2. dp[1][2]:'bb'->2.
dp[2][3]:'ba'->1. dp[3][4]:'ab'->1.
len=3: dp[0][2]:'bbb'->b==b->dp[1][1]+2=3.
dp[1][3]:'bba'->b!=a->max(dp[2][3],dp[1][2])=max(1,2)=2.
dp[2][4]:'bab'->b==b->dp[3][3]+2=3.
len=4: dp[0][3]:'bbba'->b!=a->max(dp[1][3],dp[0][2])=max(2,3)=3.
dp[1][4]:'bbab'->b==b->dp[2][3]+2=3.
len=5: dp[0][4]:'bbbab'->b==b->dp[1][3]+2=4.
Return 4 ok ('bbbb').

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n²). Space O(n) with rolling array.
Follow-up: Longest Palindromic Substring (#5) — contiguous, expand-around-center O(n²).
Follow-up: Palindrome Partitioning II (#132) — min cuts using this palindrome check.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the subsequence itself.

Keep the full table and walk from (0, n - 1): equal ends are taken (both go into the answer), otherwise move toward the side with the larger value. O(n^2) memory is needed for the walk back.

```
def lps_string(s):
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            dp[i][j] = (
                dp[i + 1][j - 1] + 2
                if s[i] == s[j]
                else max(dp[i + 1][j], dp[i][j - 1])
            )
    left, right, i, j = [], [], 0, n - 1
    while i < j:
        if s[i] == s[j]:
            left.append(s[i])
            right.append(s[j])
            i, j = i + 1, j - 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    mid = [s[i]] if i == j else []
    return "".join(left + mid + right[::-1])
```

Minimum number of insertions (or deletions) to make the string a palindrome.

It is n - LPS(s): keep the longest palindromic subsequence and add (or remove) the other characters.

```
def longestPalindromeSubseq(s):
    n = len(s)
    if n == 0:
        return 0
    prev = [0] * n  # dp[i + 1][*]
    for i in range(n - 1, -1, -1):
        cur = [0] * n
        cur[i] = 1
        for j in range(i + 1, n):
            cur[j] = prev[j - 1] + 2 if s[i] == s[j] else
max(prev[j], cur[j - 1])
        prev = cur
    return prev[n - 1]


def min_insertions_to_palindrome(s):
    return len(s) - longestPalindromeSubseq(s)
```

Longest palindromic substring instead (contiguous).

Expand around each of the 2n - 1 centres in O(n^2), or use Manacher's algorithm in O(n). A subsequence DP is not needed.

OFFICIAL PAGE TEXT

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples

```
Input: "abacc"
Output: "aba"
Explanation: "aba" is longer than "cc"
```

Our solution finds the longest palindromic substring by expanding around potential centers in the input string. For each character in the string, we consider it as the center of a palindrome and expand outwards to determine the maximum length of the palindrome centered at that position. We repeat this for both odd-length and even-length palindromes. For each character, we use a helper function expandAroundCenter that takes two indices (left and right) and expands outwards as long as the characters at these indices are equal and within the bounds of the string. We calculate the lengths of palindromes centered at the current character (odd-length palindromes) and between the current character and the next one (even-length palindromes). We track the starting and ending indices of the longest palindromic substring found so far. By comparing the lengths of the palindromes found in each iteration, we can determine the longest palindromic substring. The use of expansion around centers ensures that we examine all potential palindromes efficiently. Time Complexity: The solution has a time complexity of O(n^2), where n is the length of the input string. This is because, for each character in the string, we potentially expand outwards in both directions, resulting in quadratic time complexity in the worst case. Space Complexity: The solution uses O(1) space beyond the input string. The only extra space used is for a few variables, making it very space-efficient. NoteCheck out our lesson on the two-pointer approach to learn more about what it is and how to use it!No answers contributed by the community yet.

MEDIUM | IN APP LC 322 | Exponent | 128 of 196

Coin Change

STUDY: TYPE | Google frequency on LeetCode: all time 48.5, last 30 days 0 | book priority 64 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: your app's interview approach (LeetCode #322)
LeetCode match: LC 322 Coin Change
+27 interviews
Original: https://www.aced.io/questions/1142/coin-change

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).

Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

APPROACH

Bottom-up dp[a] = min(dp[a - c] + 1) over coins; -1 if unreachable. O(amount * coins).

HOW TO THINK ABOUT IT

The fewest coins for amount a is one coin plus the fewest for a - c, for the best coin c; that is a table over amounts built from 0 upward. Greedy (always take the biggest coin) is wrong for general coin systems, which is the classic trap: with coins 1, 3, 4 the amount 6 needs two coins (3 + 3), not three (4 + 1 + 1).

Basis: LeetCode 322, Coin Change: the fewest coins that sum to the amount, or -1; unlimited coins of each value.

BRUTE FORCE - PSEUDO-CODE

```
fewest(a): if a == 0: return 0
return 1 + the smallest fewest(a - c) over all coins c <=
a (infinity if none)
```

Time: O(coins^amount) | Space: O(amount)

Why it is slow

The same amount is recomputed through many coin orders, exponential. Filling fewest[a] for a from 0 upward is O(amount * coins).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #322 COIN CHANGE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Minimum coins to make up amount. Unlimited supply of each coin. Return -1 if impossible. Right?
Coin values can be any positive integer — not necessarily power of 2 (so greedy fails)."

"coins=[1,2,5], amount=11: 5+5+1=3 coins ok"
What should I return if there is no valid answer (0, -1, empty)?

PHASE 3 — OPTIMIZE (bottom-up DP)

Say: "DP: dp[i] = min coins to make amount i — O(n*amount).
Walk-through:
dp = [inf] * (amount+1). dp[0] = 0.
For i in 1..amount:
For coin in coins:
If i >= coin: dp[i] = min(dp[i], dp[i-coin]+1).
Return dp[amount] if dp[amount] != inf else -1.
WHY dp[i] = min over all coins: try every coin denomination.
If using coin c to complete amount i: need dp[i-c] coins for the remaining amount,
plus 1 more coin (coin c) = dp[i-c]+1.
WHY initialize dp[0]=0: amount 0 needs 0 coins — base case.
WHY return -1 if inf: amount is not reachable with given coins.
Time: O(n*amount). Space: O(amount).

PHASE 4 — CLEAN CODE

```python
class _322_CoinChange:
    def coinChange(self, coins: List[int], amount: int)
-> int:
        dp = [float('inf')] * (amount + 1)
        dp[0] = 0
        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[amount] if dp[amount] != float('inf')
else -1
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace coinChange([1,5,11], amount=15)."
dp[0]=0. dp[i]=inf for i>0.
i=1: min(dp[0]+1)=1. i=2: dp[1]+1=2. ... i=5: min(dp[4]+1=5, dp[0]+1=1)=1.
i=6: min(dp[5]+1=2, dp[1]+1=2)=2. i=10: min(..., dp[5]+1=2, dp[-1]=inf)=2.
i=11: min(dp[10]+1=3, dp[6]+1=3, dp[0]+1=1)=1. (11=1×11!)
i=15: min(dp[14]+1, dp[10]+1=3, dp[4]+1=5)=min(inf,3,5)...
dp[14]=dp[3]+1=4.
dp[15]=min(dp[14]+1=5, dp[10]+1=3, dp[4]+1=5)=3 ok (11+...
wait 11+4? no: 11+1+1+1+1? But 15-11=4, dp[4]=4. Or 5+5+5=3 coins).
Retrace:
dp[15]=min(dp[14]+1,dp[10]+1,dp[4]+1)=min(4+1,2+1,4+1)=3 ok (three 5s).

WHY BFS: the queue processes nodes in increasing distance/level, so the first time we reach a node gives the shortest path for that level.
BFS/DP: visited={amount} to avoid re-processing. queue=[0].
Pop amount a: try each coin c: if a+c unvisited -> enqueue.
dp[a+c]=dp[a]+1. visited prevents revisit.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(amount * coins). Space O(amount).
BFS alternative: treat amounts as nodes, coins as edges. BFS from 0 to amount.
DP is preferred — BFS visits each amount once but rebuilds from scratch.
Given more time I'd ask: how many ways to make the amount?

Coin Change 2 (#518) — same DP but count combinations instead of min count.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Count the number of combinations that make the amount (LeetCode 518).

Loop over coins in the outer loop and amounts in the inner loop, adding ways[a - c]; the order of the loops makes combinations (not permutations) count once.

```
def coin_change_ways(coins, amount):
    ways = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
    return ways[amount]
```

Return which coins are used.

Remember the last coin chosen for each amount (a parent array) and walk back from the target.

```
def coin_change_coins(coins, amount):
    INF = amount + 1
    best, pick = [0] + [INF] * amount, [0] * (amount + 1)
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a], pick[a] = best[a - c] + 1, c
    if best[amount] == INF:
        return None
    out, a = [], amount
    while a:
        out.append(pick[a])
        a -= pick[a]
    return out
```

Does greedy work?

Only for special coin systems (like US coins). For general coins it can be wrong (coins 1, 3, 4, amount 6). Mention the counterexample, then give the DP.

OFFICIAL PAGE TEXT

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples

```
PythonInput: coins = [1, 2, 5], amount = 11
Output: 3
Explanation: The minimum number of coins needed is 3 (11 =
 5 + 5 + 1).

Input: coins = [4, 5], amount = 7
Output: -1
Explanation: It is not possible to make up the amount 7 wi
th coins of denominations 4 and 5.
```

Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the minimum number of coins found or -1 if it is not possible to make the target amount with the given coins.

```
PseudocodeFUNCTION coin_change_naive(coins, amount):
    IF amount == 0:
        RETURN 0  // Base case: No coins are needed to mak
e amount 0

    IF amount < 0:
        RETURN infinity  // Amount is negative, path is no
t feasible

    min_coins = infinity

    FOR EACH coin IN coins:
        result = coin_change_naive(coins, amount - coin)
// Recursively solve for the remaining amount
        IF result != infinity:
            min_coins = MIN(min_coins, result + 1)  // Upd
ate min_coins with the minimum value

    RETURN min_coins IF min_coins != infinity ELSE -1  //
Return the minimum number of coins or -1 if no solution ex
ists
```

Time Complexity: The solution has a time complexity of O(m^n), where n is the amount and m is the number of coin denominations. The exponential time complexity arises from the recursive exploration of all possible combinations of coins. Each recursive call generates multiple further calls, leading to an exponential growth in the number of computations required. The total number of recursive calls can be as large as mnm^nmn due to the need to try each coin denomination for each sub-amount. Space Complexity: The space complexity of the naive approach is O(n), where n is the target amount. The space complexity is determined by the maximum depth of the recursion stack. In the worst case, the recursion depth can be as deep as the target amount n (when using coins of value 1). Therefore, the space required for the call stack is proportional to the target amount. As you can see, this solution does not perform very well. Let's take a look at a better solution. Solution 2: Dynamic programming approach Our solution employs a dynamic programming approach to determine the fewest number of coins needed to make up a given amount. We use a DP array, where each index represents a specific amount, and its value represents the minimum number of coins required to achieve that amount. The array is initialized with a value greater than the maximum possible number of coins needed (infinity), except for the base case where dp[0] is set to 0, indicating that zero coins are needed to make amount 0. For each amount from 1 to the target amount, we iterate

through each coin denomination and update the DP array. If the current amount minus the coin value is non-negative and the previous amount can be formed, we update the current amount's value to be the minimum of its current value or one more than the value of the previous amount. This ensures that we are considering the fewest coins needed by checking each possible coin combination. After processing all amounts, if the value for the target amount is still set to infinity, it means it's not possible to form the target amount with the given coins, and we return -1. Otherwise, we return the value at dp[amount], which represents the minimum number of coins needed. Time Complexity: The solution has a time complexity of O(n * m), where n is the amount and m is the number of coin denominations. This is because we iterate over each amount from 1 to the target amount, and for each amount, we iterate over all coin denominations to update the DP array. Space Complexity: The solution uses O(n) space for the DP array, where n is the target amount. The array stores the minimum number of coins required to make each amount up to the target amount. This space complexity is necessary to hold the results for all subproblems up to the target amount.No answers contributed by the community yet.

MEDIUM | 1Point3Acres | phone-screen | 129 of 196

Count Word Decompositions into Chemical Element Symbols

STUDY: TALK | Google frequency on LeetCode: all time 43.5, last 30 days 0 | book priority 73 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

QUESTION

Examples:

Physics has four stated decompositions:

P - H - Y - Si - C - S

P - H - Y - Si - Cs

P - H - Y - S - I - C - S

P - H - Y - S - I - Cs

The returned count is 4.

APPROACH

Implement a suffix-count or prefix-count formulation and state what each state represents.

Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

INTERVIEW-DAY PLAN

Clarify first

Ask whether matching is case-insensitive, whether to count decompositions or just test for one, and for the dictionary (the periodic table symbols of one and two letters).

Brute force to say out loud

Recursive search that tries both a one-letter and a two-letter symbol at each position. Exponential in the worst case.

Optimal idea

Suffix-count DP: ways[i] is the number of decompositions of word[i:], with ways[n] = 1. ways[i] adds ways[i + 1] if word[i] is a symbol and ways[i + 2] if word[i:i + 2] is a symbol. O(n). Physics gives 4.

Dry-run these cases

Physics (4), a word with no decomposition (hello), one with exactly one, the empty string, overlapping one- and two-letter symbols such as Si versus S + I.

Likely follow-ups

List the decompositions, handle a larger dictionary with a trie, or count modulo a prime.

Pitfalls

Forgetting to lowercase before the lookup. Starting the DP with ways[n] = 0.

HOW TO THINK ABOUT IT

Counting the ways to cut a string into dictionary pieces is Word Break with a count instead of a yes/no. Define ways[i] as the number of decompositions of the suffix starting at i; it depends only on ways[i + 1] and ways[i + 2], because symbols have one or two letters. Compute from the end so every value you need already exists.

Basis: A counting version of LeetCode 139, Word Break, with the periodic-table symbols as the dictionary.

BRUTE FORCE - PSEUDO-CODE

```
ways(i): if i is the end of the word: return 1
total = 0; if word[i] is a symbol: total += ways(i + 1)
if word[i..i+1] is a symbol: total += ways(i + 2)
return total
```

Time: exponential (Fibonacci growth) | Space: O(n)

Why it is slow

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

OPTIMAL SOLUTION - SUFFIX-COUNT DP

```python
def count_element_decompositions(word, symbols=ELEMENTS):
    w = word.lower()
    n = len(w)
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i : i + 2] in symbols:
            ways[i] += ways[i + 2]
```

```
    return ways[0]
```

Time: O(n) | Space: O(n)

Why

ways[i] is the number of decompositions of word[i:]. A symbol has one or two letters, so ways[i] = ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if word[i:i + 2] is a symbol). ways[n] = 1 for the empty suffix. Matching is case-insensitive.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

```
def list_decompositions(word, symbols=ELEMENTS):
    w = word.lower()
    out = []

    def go(i, parts):
        if i == len(w):
            out.append("-".join(p.capitalize() for p in pa
rts))
            return
        for k in (1, 2):
            if i + k <= len(w) and w[i : i + k] in symbols
:
                go(i + k, parts + [w[i : i + k]])

    go(0, [])
    return out
```

A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

NOTES FROM CANDIDATES

Confirm whether matching is case-sensitive and whether the caller supplies the symbol set or the implementation may treat the periodic table as fixed.

The result counts decompositions, not merely whether at least one decomposition exists.

PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.

Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

MEDIUM | 1Point3Acres | onsite-coding | 130 of 196

Jump Game with Profit

STUDY: TYPE | Google frequency on LeetCode: all time 56, last 30 days 0 | book priority 51 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #55 Jump Game

LeetCode match: LC 55 Jump Game (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/jump-game-with-profit

QUESTION

Examples:

Variant-dependent; the principal recurrence is the one above.

APPROACH

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.

Practice clarifying "can I skip without consuming a turn" before coding.

INTERVIEW-DAY PLAN

Clarify first

This is underspecified, so ask: is arr[i] the maximum jump (a cap) or an exact jump, is profit collected on landing or when leaving, can you skip a cell for free, are revisits allowed, and what is the goal (reach the end, or maximize profit)?

Brute force to say out loud

Recursively try every jump from every cell. Exponential.

Optimal idea

Pick a reading and say it. Cap reading: best[i] is the best profit standing on i; best[i] = profit[i] + max best[j] over j that can reach i; O(n * max jump), or O(n log n) with a heap and lazy deletion. Exact jump with optional skipping: dp[i] = max(dp[i + 1], profit[i] + dp[i + arr[i]]) from the right, O(n). Plain reachability (no profit) is a greedy farthest-reach scan in O(n).

Dry-run these cases

A cell with jump 0, an unreachable end, negative profits, a single cell.

Likely follow-ups

Minimum jumps (LeetCode 45), profits that can be negative, or a limit on the number of jumps.

Pitfalls

Treating the jump as a cap when it is exact, or the reverse. Not defining what skipping costs.

HOW TO THINK ABOUT IT

This statement is underspecified, so pin it down first (cap or exact jump, when profit is collected, can you skip, what is the goal). The reading with a maximum jump and profit on landing is a DP over positions: the best profit at i is its own profit plus the best of all positions that can reach it. The reading with an exact jump and a free skip is a one-line DP from the right.

Basis: Variant of LeetCode 55 (Jump Game) and LeetCode 45 (Jump Game II). The prompt is underspecified, so two readings are given; confirm which one with the interviewer.

BRUTE FORCE - PSEUDO-CODE: READING A: RECURSION OVER EVERY JUMP LENGTH

```
go(i, total): if i is the last index: record total
for every jump length s from 1 to arr[i]: go(i + s, total
+ profit[landing cell])
answer = the best recorded total
```

Time: O(max jump ^ n) | Space: O(n)

Why it is slow

Every cell branches into up to max-jump choices, so the paths multiply (max jump to the power n). A table of the best profit per cell, filled left to right, is O(n * max jump).

BRUTE FORCE - PSEUDO-CODE: READING B: RECURSION OVER 'JUMP OR SKIP' CHOICES

```
best(i): if i is past the end: return 0
skip = best(i + 1); jump = profit[i] + best(i + arr[i]) if
 arr[i] > 0
return the larger of skip and jump
```

Time: O(2^n) | Space: O(n)

Why it is slow

Two branches at every cell and the same suffix is solved repeatedly: O(2^n). Computing best(i) once per index from the right is O(n).

OPTIMAL SOLUTION - READING A: ARR[I] IS A MAXIMUM JUMP, PROFIT[I] IS GAINED ON LANDING, MAXIMIZE TOTAL PROFIT TO THE LAST INDEX

```
def max_profit_jump(arr, profit):
    n = len(arr)
    NEG = float("-inf")
    best = [NEG] * n
    best[0] = profit[0]
    for i in range(1, n):
        for j in range(i):
            if best[j] != NEG and j + arr[j] >= i:
                best[i] = max(best[i], best[j] + profit[i]
)
    return -1 if best[-1] == NEG else best[-1]
```

Time: O(n * maxJump) | Space: O(n)

Why

Let best[i] be the largest profit achievable when standing on i. Landing on i from any j < i with j + arr[j] >= i gives best[i] = profit[i] + max(best[j]). Unreachable cells stay at minus infinity. Return best[n - 1], or -1 if the end cannot be reached. Pull-style DP keeps the reachability test explicit.

OPTIMAL SOLUTION - READING B: ARR[I] IS AN EXACT JUMP; AT EACH INDEX YOU MAY JUMP (GAINING PROFIT[I]) OR SKIP ONE CELL FOR FREE

```
def max_profit_exact_jump(arr, profit):
    n = len(arr)
    dp = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        nxt = i + arr[i]
        jump = profit[i] + (dp[nxt] if nxt < n else 0) if
arr[i] > 0 else float("-inf")
        dp[i] = max(dp[i + 1], jump)
    return dp[0]
```

Time: O(n) | Space: O(n)

Why

Memoize from the right. dp[i] is the best profit starting at i, with dp[i] = 0 for i >= n (past the end ends the game). dp[i] = max(skip: dp[i + 1], jump: profit[i] + dp[i + arr[i]]). This is the 'exact jump, not range cap' reading the original report warns about.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

```
def can_reach_end(arr):
    far = 0
    for i, x in enumerate(arr):
        if i > far:
            return False
        far = max(far, i + x)
    return True
```

Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

NOTES FROM CANDIDATES

Standard intern coding round, single source — clarification with interviewer is key (does "skip" advance by 1 or 0? are revisits allowed?).
Common bug: treating arr[i] as range cap rather than exact jump.

PREPARATION

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

MEDIUM | IN APP LC 465 | HackTheRounds | Onsite | 131 of 196

Trip Expense Settlement

STUDY: TALK | Google frequency on LeetCode: all time 25.5, last 30 days 0 | book priority 128 of 196

Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: the interview site's own solution
Also in your app (open the app for its Expose): LeetCode #465 Optimal Account Balancing
LeetCode match: LC 465 (Optimal Account Balancing)

QUESTION

A group of friends go on a trip. Each friend may pay for others. Given payment records, compute minimal payments required to settle all debts.

EXAMPLES

Example 1

Input	`[{"payer": "alice", "amount": 4000, "payees": ["bob", "jess", "alice", "sam"]}, {"payer": "jess", "amount": 2000, "payees": ["jess", "alice"]}]`
Output	`[{"payer": "bob", "amount": 1000, "payee": "alice"}, {"payer": "sam", "amount": 1000, "payee": "alice"}]`
Why	`Calculate net balance for each person, then settle with minimum transactions.`

Example 2

Input	`expenses = [{"payer": "Alice", "amount": 40, "participants": ["Alice", "Bob"]}, {"payer": "Bob", "amount": 20, "participants": ["Alice", "Bob"]}]`
Output	`["Bob pays Alice 10.00"]`

CONSTRAINTS

- `1 <= payments.length <= 1000`
- `1 <= amount <= 10^6`
- `1 <= payees.length <= 100`

APPROACH

Compute each person's net balance, then match debtors and creditors with heaps

HOW TO THINK ABOUT IT

First reduce the records to one number per person: net balance = paid - owed. Only that matters; who owed whom originally is irrelevant. Then settling is moving money between people with positive and negative balances. Matching largest creditor with largest debtor is quick and uses at most n - 1 payments, but only searching groups of people who sum to zero gives the true minimum.

Basis: LeetCode 465, Optimal Account Balancing. Each record is split equally among the listed payees (the payer may be one of them).

BRUTE FORCE - PSEUDO-CODE

```
net[person] = paid - owed from all records
settle(i): skip people with zero balance; for every later
person j with the opposite
    sign: move balance i to j, settle(i + 1), undo
answer = the fewest payments found
```

Time: O(n!) worst case | Space: O(n)

Why it is slow

It tries every pairing, up to n! in the worst case. Matching the largest creditor with the largest debtor (heaps) is O(n log n) and uses at most n - 1 payments, though not always the minimum.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: GREEDY MATCHING WITH SORTING

```python
from typing import List, Dict


class Solution:
    def settleDebts(self, expenses: List[Dict]) -> List[str]:
        """
        Settle trip expenses using greedy matching approach.
        Matches largest debtors with largest creditors.

        Args:
            expenses: List of expense dicts with payer, amount, participants

        Returns:
            List of settlement strings "X pays Y Z.ZZ"

        Time: O(n log n) for sorting
        Space: O(n) for balance tracking
        """
        balance = {}

        for expense in expenses:
            payer = expense["payer"]
            amount = expense["amount"]
            participants = expense["participants"]
            share = amount / len(participants)
```

```
            balance[payer] = balance.get(payer, 0) + amoun
t
            for p in participants:
                balance[p] = balance.get(p, 0) - share

        creditors = [(p, b) for p, b in balance.items() if
 b > 0.01]
        debtors = [(p, -b) for p, b in balance.items() if
b < -0.01]

        creditors.sort(key=lambda x: -x[1])
        debtors.sort(key=lambda x: -x[1])

        result = []
        i, j = 0, 0

        while i < len(debtors) and j < len(creditors):
            debtor, debt = debtors[i]
            creditor, credit = creditors[j]

            amount = min(debt, credit)
            result.append(f"{debtor} pays {creditor} {amou
nt:.2f}")

            debtors[i] = (debtor, debt - amount)
            creditors[j] = (creditor, credit - amount)

            if debtors[i][1] < 0.01:
                i += 1
            if creditors[j][1] < 0.01:
                j += 1

        return result
```

Time: O(n log n) | Space: O(n)

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Exact minimum with a bitmask DP over zero-sum groups

```
def settle_min_dp(records):
    vals = list(balances(records).values())
    n = len(vals)
    total = [0] * (1 << n)
    dp = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        total[mask] = total[mask & (mask - 1)] + vals[low]
        best = 0
        for i in range(n):
            if mask >> i & 1:
                best = max(best, dp[mask ^ (1 << i)])
        dp[mask] = best + (1 if total[mask] == 0 else 0)
    return n - dp[(1 << n) - 1]
```

Time: O(2^n * n) | Space: O(2^n)

Why

A group of k people whose balances sum to zero can settle among themselves with k - 1 payments, so the minimum is (people with a balance) minus (the maximum number of disjoint zero-sum groups). dp[mask] is the largest number of zero-sum groups inside mask: add one person at a time and gain 1 when the mask's total is zero. Feasible for about 20 people.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Do you need the true minimum number of payments?

That is NP-hard in general (it is the zero-sum partition problem). The greedy answer is at most n - 1 payments and is usually accepted; for exact answers use the bitmask DP above (about 20 people) or backtracking with pruning.

```
def settle_min_dp(records):
    vals = list(balances(records).values())
    n = len(vals)
    total = [0] * (1 << n)
    dp = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        total[mask] = total[mask & (mask - 1)] + vals[low]
        best = 0
        for i in range(n):
            if mask >> i & 1:
                best = max(best, dp[mask ^ (1 << i)])
        dp[mask] = best + (1 if total[mask] == 0 else 0)
    return n - dp[(1 << n) - 1]
```

Amounts that do not divide evenly (cents).

Work in integer cents and decide where the remainder goes (to the payer, or round-robin among payees). Never use floats for money, and make the rule deterministic.

Different shares per person instead of equal split.

Replace the equal share with an explicit amount per payee in each record; the net balance per person is computed the same way.

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Compute minimal transfers to settle group expenses | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimal-transfers-to-settle-group-expenses

---

MEDIUM | HackTheRounds | Onsite | 132 of 196

Reverse Coin Change

STUDY: TALK | Google frequency on LeetCode: all time 37.9, last 30 days 0 | book priority 90 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #518 Coin Change II

LeetCode match: Inverse of LC 518

QUESTION

In standard coin change, given a coins array, we build dp where dp[i] = number of ways to make amount i. Reverse problem: given a dp array, find the original coins array that produced it.

EXAMPLES

Example 1	
Input	dp = [1, 1, 2, 2, 3]
Output	[1, 2]
Why	Coins [1, 2] produce this dp array.

Example 2	
Input	dp = [1, 1, 1, 2, 2, 2, 3]
Output	[1, 3]

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

APPROACH

Rebuild coins left to right: the first amount whose dp exceeds the dp built from coins so far is a new coin

HOW TO THINK ABOUT IT

Think of building the ways table forward: adding a coin c changes dp[c] by exactly one (the single coin c). So scanning amounts upward, the first amount where the target has more ways than your table can only be explained by a new coin of exactly that value. Add it, rebuild, and continue; at the end compare with the target to detect an impossible input.

Basis: The inverse of LeetCode 518, Coin Change II (number of ways to make each amount with unlimited coins).

BRUTE FORCE - PSEUDO-CODE

```
for every subset of {1..n} as a candidate coin set: build
its ways-to-make-amount
    table
return the subset whose table equals the given dp array
```

Time: O(2^n * n * coins) | Space: O(n)

Why it is slow

2^n candidate sets, each with an O(n * coins) table. Scanning amounts upward and adding a coin whenever the target has one more way than the table is O(n^2).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: INCREMENTAL DETECTION

```python
from typing import List


class Solution:
    def reverseCoinChange(self, dp: List[int]) -> List[int
]:
        n = len(dp)
        coins = []

        # Build coins incrementally
        # current_dp tracks what we can explain with found
 coins
        current_dp = [0] * n
        current_dp[0] = 1

        for c in range(1, n):
            # If dp[c] != current_dp[c], coin c might exis
t
            # Try adding coin c and see if it helps match
the target

            # Test what happens if we add coin c
            test_dp = current_dp.copy()
            for i in range(c, n):
                test_dp[i] += test_dp[i - c]

            # Check if test_dp matches dp up to this point
            matches = True
            for i in range(n):
                if test_dp[i] > dp[i]:
                    matches = False
                    break

            if matches and test_dp[c] == dp[c]:
                # Adding this coin is valid
                coins.append(c)
                current_dp = test_dp

        # Verify final result matches
        if current_dp == dp:
            return coins
        return coins
```

Time: O(n^2) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The array might not come from any set of coins.

Return None (or raise) when dp[0] != 1, when an amount needs more than one extra way, or when the rebuilt table differs from the input at the end; the code does all three checks.

```python
def ways_table(coins, n):
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


def coins_from_dp(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(a, n + 1):
            cur[x] += cur[x - a]
    return coins if cur == list(dp) else None
```

Order of the coins matters (the dp counts permutations, not combinations).

That is a different recurrence: dp[a] = sum of dp[a - c] over coins. Inverting it is a linear-algebra style problem: at each amount the new coins explain the excess; ask which counting is used.

Each coin may be used at most once.

The forward recurrence iterates amounts downward. The inverse still scans upward adding a coin when the target exceeds the table, but the table update must use the 0/1 form.

MEDIUM | PracHub | Technical Screen | 133 of 196

Determine if a 14-tile hand is winning

STUDY: TALK | Google frequency on LeetCode: all time 19.6, last 30 days 0 | book priority 136 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #659 Split Array into Consecutive Subsequences

LeetCode match: No direct match; related LC 659 Split Array into Consecutive Subsequences

Original: https://prachub.com/coding-questions/determine-if-a-14-tile-hand-is-winning

QUESTION

You are given an integer array **`tiles`** of length 14 representing a Mahjong-like hand. Each integer is a tile value from 1 to 9 (single suit). You may reorder the tiles. A hand is **winning** if it can be partitioned into:

- **1 pair**: two identical tiles, and - **4 melds** (each meld uses 3 tiles), where each meld is either: - a **triplet**: three identical tiles (e.g., 7,7,7), or - a **sequence**: three consecutive values (e.g., 2,3,4). Each tile can be used at most once. Return **`true`** if the hand is winning, otherwise return **`false`**. Assumptions/constraints:
- **`tiles.length == 14`**
- **`1 <= tiles[i] <= 9`**
- Each value can appear multiple times (as in a real tile set).

APPROACH

Count array of tiles 1..9; try each value as the pair, then backtrack: take the smallest remaining tile as a triplet or a run of three. Tiny search space.

INTERVIEW-DAY PLAN

Clarify first

Ask whether sequences are only consecutive in one suit (values 1 to 9), whether a tile can be used only once, and whether other special hands exist (assume not).

Brute force to say out loud

Try every permutation of the 14 tiles and test the grouping. Far too slow.

Optimal idea

Keep a count array of tiles 1 to 9. Try each value that has at least two tiles as the pair, remove it, then backtrack on the remaining 12 tiles: take the smallest value that still has tiles and either remove a triplet of it or a run of three consecutive values starting at it; if neither is possible, fail. Always handling the smallest value makes the search small. Time is tiny for 14 tiles.

Dry-run these cases

Four triplets and a pair, four runs and a pair, a hand with two possible groupings, a hand with a pair but impossible melds, all 14 tiles equal (invalid since each value has at most four copies).

Likely follow-ups

More suits, seven pairs, or all possible winning tiles for a 13-tile hand.

Pitfalls

Greedy without backtracking: a run versus a triplet choice can matter. Forgetting to restore counts.

HOW TO THINK ABOUT IT

Work with counts per value (at most nine numbers), not with the tile list. Try every value as the pair, then handle the lowest remaining value: it must belong to a triplet or to a sequence starting at it (nothing smaller is left to pair with it). Always processing the smallest value makes the search tiny and avoids trying the same grouping in different orders.

Basis: Written; a small exhaustive search like the Mahjong 'standard hand' check. Rule: 14 tiles of one suit with values 1..9 split into one pair and four melds, each meld a triplet (x, x, x) or a sequence (x, x + 1, x + 2).

BRUTE FORCE - PSEUDO-CODE

```
for every value with at least two tiles: take it as the pa
ir
try removing any triplet or any run of three from anywhere
 among the counts, in any
    order, recursively; the hand wins if all tiles can be
removed
```

Time: exponential but small for 14 tiles | Space: O(depth)

Why it is slow

It tries the same groupings in many different orders (still tiny for 14 tiles). Always grouping the smallest remaining tile tries each grouping once.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(tiles):
    counts = [0] * 10
    for tile in tiles:
        counts[tile] += 1

    def can_form_melds():
        i = 1
        while i <= 9 and counts[i] == 0:
            i += 1

        if i == 10:
            return True

        if counts[i] >= 3:
            counts[i] -= 3
            if can_form_melds():
                counts[i] += 3
                return True
            counts[i] += 3

        if i <= 7 and counts[i + 1] > 0 and counts[i + 2]
> 0:
            counts[i] -= 1
            counts[i + 1] -= 1
            counts[i + 2] -= 1
            if can_form_melds():
                counts[i] += 1
                counts[i + 1] += 1
                counts[i + 2] += 1
                return True
            counts[i] += 1
            counts[i + 1] += 1
            counts[i + 2] += 1

        return False
```

```
    for value in range(1, 10):
        if counts[value] >= 2:
            counts[value] -= 2
            if can_form_melds():
                counts[value] += 2
                return True
            counts[value] += 2

    return False
```

Time: O(1) under the fixed constraints (14 tiles, values 1 through 9) | Space: O(1)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

```
def is_winning_hand(tiles):
    cnt = [0] * 12

    for t in tiles:
        cnt[t] += 1

    def melds(v):
        while v <= 9 and cnt[v] == 0:
            v += 1
        if v > 9:
            return True
        if cnt[v] >= 3:
            cnt[v] -= 3
            ok = melds(v)
            cnt[v] += 3
            if ok:
                return True
        if v <= 7 and cnt[v + 1] and cnt[v + 2]:
            cnt[v] -= 1
            cnt[v + 1] -= 1
            cnt[v + 2] -= 1
            ok = melds(v)
            cnt[v] += 1
            cnt[v + 1] += 1
            cnt[v + 2] += 1
            if ok:
                return True
        return False

    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds(1)
            cnt[pair] += 2
            if ok:
                return True
    return False


def waiting_tiles(thirteen):
    """Follow-up: which tile values complete a 13-tile hand?"""
    out = []
    for v in range(1, 10):
        if thirteen.count(v) < 4 and is_winning_hand(thirteen + [v]):
            out.append(v)
    return out
```

More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

FUNCTION AND TEST CASES

Signature

```
def solution(tiles: list[int]) -> bool:
```

Sample call

```
solution([1,1,1,2,3,4,2,3,4,5,6,7,9,9])
```

Test cases

Input	Expected
`([1,1,1,2,3,4,2,3,4,5,6,7,9,9],)`	`True`
`([2,2,2,3,3,3,4,4,4,5,5,5,9,9],)`	`True`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Mahjong Winning Hand Detection (14 Tiles) | 1Point3Acres | onsite-coding | asked 2026-07-20 | Low

https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:
Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.
For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

Preparation: Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

MEDIUM | 1Point3Acres | onsite-coding | 134 of 196

Gold Chain Split into Two Equal Halves

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 180 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEarrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

APPROACH

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.

Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.

Watch the boundary: remove + cut should not yield an empty side.

INTERVIEW-DAY PLAN

Clarify first

Ask whether removing a segment keeps the rest as a chain joined end to end, whether the cut after removal must leave two non-empty parts, and whether to return a boolean, one answer, or all valid pairs.

Brute force to say out loud

For each removed index, rebuild the array and try every cut with prefix sums. O(n^2).

Optimal idea

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

Dry-run these cases

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

Likely follow-ups

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

Pitfalls

Allowing an empty half. Forgetting the shift of indices after the removed segment.

HOW TO THINK ABOUT IT

After removing segment r the target half is (S - a[r]) / 2, and a cut means some prefix of the remaining chain has exactly that weight. A prefix of the remaining chain is either an ordinary prefix of the original array (stops before r) or an original prefix minus a[r] (passes over r). Each is one value to look up in a hash of prefix sums, so every r costs O(1).

Basis: Written; prefix sums with a hash lookup (like LeetCode 724 and 560). Assumed rule (confirm it): remove exactly one segment, reconnect the remaining chain end to end, then cut it once into two non-empty parts of equal total weight.

BRUTE FORCE - PSEUDO-CODE

```
for every segment r: build the chain without it and comput
e its total
walk the chain with a running sum; if the sum equals half
the total at some cut:
    return true
return false
```

Time: O(n^2) | Space: O(n)

Why it is slow

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION - PREFIX SUMS WITH FIRST AND LAST OCCURRENCE MAPS

```
def gold_chain(a):
    n = len(a)
    if n < 3:
        return False
    P_ = [0]
    for x in a:
        P_.append(P_[-1] + x)
    S = P_[-1]
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P_[j], j)
        last[P_[j]] = j
    for r in range(n):
        rem = S - a[r]
        if rem % 2:
            continue
        half = rem // 2
        if half in first and first[half] <= min(r, n - 2):
            return True
        if (half + a[r]) in last and last[half + a[r]] > r
:
            return True
    return False
```

Time: O(n) | Space: O(n)

Why

Let P[j] be the sum of the first j segments. For removed index r with half = (S - a[r]) / 2 (skip if odd): a cut before r exists if half equals some P[j] with 1 <= j <= r; a cut after r exists if half + a[r] equals some P[k] with r < k <= n - 1. Keep the first index and the last index of every prefix value, so each check is one lookup.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

```
def gold_chain_all_pairs(a):
    out = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1 :]
        total, run = sum(rest), 0
        for c in range(len(rest) - 1):
            run += rest[c]
            if run * 2 == total:
                out.append(
                    (r, c + 1)
                )  # remove segment r, cut before position
 c + 1 of the remaining chain
    return out
```

The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

NOTES FROM CANDIDATES

Precompute prefix sums; for each candidate removed index r, the remaining total is S - a[r]. If that is odd, skip. Otherwise check if there is a split point achieving (S - a[r]) / 2.

Use a hashmap from prefix-sum value to index for O(1) lookups -> overall O(N) for the boolean answer, O(N^2) worst case for enumerating all pairs (output-bound).

Watch out for cut positions adjacent to the removed link — must still respect contiguous-chain semantics.

PREPARATION

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.

Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.

Watch the boundary: remove + cut should not yield an empty side.

Expression Parsing and Evaluation 6 questions

MEDIUM | 1Point3Acres | onsite-coding | 135 of 196

Nested List Weight Sum (LC 339) with Follow-Ups

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 142 of 196

Topic: Expression Parsing and Evaluation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

Examples:

"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2 + 2×2 + 91×3 = 320.

Use [2, [91]] as a manual trace for bracket depth and the final-number flush.

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.

Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.

Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

Brute force to say out loud

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

Optimal idea

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

Dry-run these cases

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

Likely follow-ups

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

Pitfalls

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE - PSEUDO-CODE

```
parse the text into nested lists (for example with a JSON
parser)
depthSum(list, depth): return the sum over items of (item
* depth if it is a number,
    else depthSum(item, depth + 1))
answer = depthSum(parsed, 1)
```

Time: O(n) | Space: O(depth)

Why it is slow

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION - SINGLE PASS OVER THE STRING WITH A DEPTH COUNTER

```python
def nestedWeightSumFromString(s):
    depth = total = 0
    num, sign, have = 0, 1, False
    for ch in s + "]":  # a sentinel flushes a trailing nu
mber
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        if have:
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

Time: O(n) | Space: O(1)

Why

Every integer is weighted by the number of '[' currently open, so no tree needs to be built. Accumulate digits in a buffer and flush the buffer when a non-digit arrives or the string ends. Forgetting that last flush is the classic bug.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Parsed lists with BFS (no recursion depth limit)

```python
def depthSumBFS(nested):
    total, depth, level = 0, 1, list(nested)
    while level:
        nxt = []
        for item in level:
            if isinstance(item, list):
                nxt.extend(item)
            else:
                total += item * depth
        level, depth = nxt, depth + 1
    return total
```

Time: O(n) | Space: O(width)

Why

Once the string is parsed into Python lists, process one depth level at a time with a queue. BFS never recurses, so a pathologically deep input cannot overflow the call stack.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

```python
def depth_sum_inverse(nested):
    """LeetCode 364: deeper integers weigh less. weight =
maxDepth - depth + 1, where maxDepth is the deepest level
holding an integer."""
    sums, counts, level = [], [], list(nested)
    while level:
        s = c = 0
        nxt = []
        for x in level:
            if isinstance(x, list):
                nxt.extend(x)
            else:
                s, c = s + x, c + 1
        sums.append(s)
        counts.append(c)
        level = nxt
    while (
        counts and counts[-1] == 0
    ):  # levels made only of empty lists do not count tow
ard the maximum depth
        counts.pop()
        sums.pop()
    total = running = 0
    for s in sums:
        running += (
            s  # integers on shallower levels are added ag
ain at every deeper level
        )
        total += running
    return total
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

MEDIUM | 1Point3Acres | onsite-coding | 136 of 196

Target Expressions from Ordered Digits

STUDY: TALK | Google frequency on LeetCode: all time 36.5, last 30 days 0 | book priority 93 of 196
Topic: Expression Parsing and Evaluation
Optimal answer from: written and tested here for this exact question
LeetCode match: LC 282 Expression Add Operators
Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

Examples:
For digits [2, 3, 4] and target 14, valid outputs include:
2 * (3 + 4)
2 + 3 * 4

APPROACH

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

INTERVIEW-DAY PLAN

Clarify first

Ask whether digits may be concatenated into multi-digit numbers, which operators are allowed, whether parentheses are free, whether the digit order is fixed, and whether to return all expressions or a boolean.

Brute force to say out loud

Enumerate every operator placement and every parenthesization for the three or four digits and evaluate each. That is fine for a handful of digits and is what the first stage asks for.

Optimal idea

Interval DP: for each interval of digits compute a map from value to the expressions that achieve it by splitting at every point and combining left and right with plus, minus and times. Memoize by interval. If only reachability is needed store just the set of values per interval, which is much smaller than the strings.

Dry-run these cases

[2, 3, 4] with target 14 gives 2*(3+4) and 2+3*4. A target that is unreachable. A single digit. Zeros.

Likely follow-ups

N digits (the DP above), division with fractions, or concatenation.

Pitfalls

Missing parentheses in the output string. Evaluating with floats.

HOW TO THINK ABOUT IT

Any expression over a run of digits is two smaller expressions joined by an operator at some split point, so the set of values for an interval is built from the sets of its two halves. Compute values bottom-up per interval and only build text for the target. Be explicit about parentheses: they come from the split structure, so every split is a candidate.

Basis: Related to LeetCode 282, Expression Add Operators, but with parentheses allowed and digits not concatenated.

BRUTE FORCE - PSEUDO-CODE

```
values(digits): if one digit, return {that digit}
for every split point: for each pair (a, b) of values of t
he left and right parts: add
    a + b, a - b and a * b to the result
the target is reachable if it is in values(all digits)
```

Time: exponential (Catalan(n) * 3^(n-1) expressions) | Space: O(n) recursion plus the value set

Why it is slow

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION - INTERVAL DP: ALL EXPRESSIONS FOR DIGITS[L:R]

```python
def expressions_for_target(digits, target):
    memo = {}

    def build(l, r):
        if (l, r) in memo:
            return memo[(l, r)]
        res = defaultdict(set)  # value -> {(text, precede
nce)}
        if r - l == 1:
            res[digits[l]].add((str(digits[l]), 3))
        for m in range(l + 1, r):
            for lv, lset in build(l, m).items():
                for rv, rset in build(m, r).items():
                    for lt, lp in lset:
                        for rt, rp in rset:
                            for op, prec, val in (
                                ("+", 1, lv + rv),
                                ("-", 1, lv - rv),
                                ("*", 2, lv * rv),
                            ):
                                left = f"({lt})" if lp < p
```

```
rec else lt
                                right = (
                                    f"({rt})"
                                    if (rp < prec or (rp =
= prec and op == "-"))
                                    else rt
                                )
                                res[val].add((f"{left}{op}
{right}", prec))
        memo[(l, r)] = res
        return res

    return sorted(text for text, _ in build(0, len(digits)
).get(target, ()))
```

Time: exponential in the number of results; polynomial in the number of distinct values | Space: O(distinct values per interval)

Why

Every expression over digits[l:r] is two sub-expressions over a split point joined by +, - or *. Memoize, for each interval, a map from numeric value to the set of expression strings that produce it. Keep expression construction separate from evaluation, and add parentheses only where precedence demands them.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

```
def target_reachable(digits, target):
    memo = {}

    def values(l, r):
        if (l, r) in memo:
            return memo[(l, r)]
        if r - l == 1:
            res = {digits[l]}
        else:
            res = set()
            for m in range(l + 1, r):
                for a in values(l, m):
                    for b in values(m, r):
                        res |= {a + b, a - b, a * b}
        memo[(l, r)] = res
        return res

    return target in values(0, len(digits))
```

NOTES FROM CANDIDATES

The base case is small enough for direct enumeration, but a complicated ad hoc implementation becomes difficult to extend. Keep expression construction separate from numeric evaluation while negotiating the N-digit contract.

If the agreed contract uses every digit exactly once in order, memoize each interval [l, r) as a map from attainable numeric values to the expression strings producing them. For every split point, combine left and right entries with + and *, wrapping composite children as needed; the full-interval target bucket is the answer. Because every distinct expression must be retained, both time and storage are output-sensitive and can grow exponentially even though interval states are memoized.

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.

Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.

Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 137 of 196

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

STUDY: TYPE | Google frequency on LeetCode: all time 57.7, last 30 days 0 | book priority 39 of 196

Topic: Expression Parsing and Evaluation

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

Examples:

"3[a]2[bc]" -> "aaabcbc".

"2[abc]3[cd]ef" -> "abcabccdcdcdef".

"3[a2[c]]" -> "accaccacc".

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE - PSEUDO-CODE

```
parse(): result = ''; while not at ']' or the end:
    if the next character is a digit: read the whole numbe
r n, skip '[', inner =
        parse(), skip ']', result += inner * n
    else: result += the character
return result
```

Time: O(output length) | Space: O(depth)

Why it is slow

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #394 DECODE STRING)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Decode 'k[encoded_string]' — repeat encoded_string k times.
Nesting is allowed. k is always a positive integer. Input is always valid. Right?"

"A few quick questions:
Can k be multi-digit? Yes — e.g. '12[a]' -> 'aaa...a'(12 times).
Can encoded_string be empty? Yes — e.g. '3[]' -> ''."

"Let me walk: '3[a]2[bc]' -> 'aaabcbc' ok. '3[a2[c]]' -> 'accaccacc' ok

PHASE 3 — OPTIMIZE

The bottleneck is the repeated string scans.
Stack-based single pass:
On digit: build the multiplier k (can be multi-digit).
On '[': push (current_string, k) onto the stack, reset both to empty/0.
On ']': pop (prev_string, k), set current = prev_string + k * current.
On letter: append to current string.

Time: O(n * max_k) for building the expanded output. Space: O(n) stack depth."
Key insight: on '[' push (current_result, multiplier) to freeze the outer context; on ']' pop and prepend the outer result — nested encodings unwind naturally in reverse order.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _394_DecodeString:
    def decodeString(self, s):
        stack = []
        current = ''
        k = 0
        for ch in s:
            if ch.isdigit():
                k = k * 10 + int(ch)
            elif ch == '[':
                stack.append((current, k))
                current, k = '', 0
            elif ch == ']':
                prev, repeat = stack.pop()
                current = prev + repeat * current
            else:
                current += ch
        return current
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace decodeString('3[a2[c]]')."
stack=[]. result=''. k=0.
'3': k=3. '[': push(result='',k=3). result='',k=0.
'a': result='a'. '2': k=2. '[': push(result='a',k=2). result='',k=0.
'c': result='c'. ']': pop->(prev='a',times=2). result='a'+'c'*2='acc'.
']': pop->(prev='',times=3). result=''+'acc'*3='accaccacc'.
Return 'accaccacc' ok
'2[abc]3[cd]ef': [2,'abc'] -> 'abcabc'. [3,'cd'] -> 'cdcdcd'. Append 'ef'. Return 'abcabccdcdcdef' ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n * max_k): building the output string can be large.
Space O(d): d = nesting depth of the stack.
Follow-up: Encode String with Shortest Length (#471) — reverse direction, DP + KMP.
Follow-up: if nesting is guaranteed absent, a simple split-on-bracket regex suffices.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Encode (the reverse direction, greedy run-length)

```python
def encodeRuns(s):
    return "".join(f"{len(list(g))}[{k}]" for k, g in iter
tools.groupby(s))
```

Time: O(n) | Space: O(n)

Why

Run-length encode the string into count[char] groups. This is the simple flat inverse; producing the shortest nested encoding (LeetCode 471) needs interval DP and is a hard follow-up.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

```
def decoded_length(s):
    """Length of the decoded string without building it.""
"
    stack, length, num = [], 0, 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            stack.append((length, num))
            length, num = 0, 0
        elif ch == "]":
            prev, times = stack.pop()
            length = prev + length * times
        else:
            length += 1
    return length
```

Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

MEDIUM | 1Point3Acres | onsite-coding | 138 of 196

Nested Function Expression Evaluator

STUDY: TALK | Google frequency on LeetCode: all time 5, last 30 days 0 | book priority 161 of 196
Topic: Expression Parsing and Evaluation
Optimal answer from: written and tested here for this exact question
LeetCode match: LC 770 Basic Calculator IV (related)
Frequency: Low | Last asked: 2026-06-10 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

Examples:

```
add(1, sub(1, 0)) -> 2
add(2, mul(3, pow(4, 5))) -> 3074
```

APPROACH

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

INTERVIEW-DAY PLAN

Clarify first

Ask which functions exist, whether they take a fixed or variable number of arguments, whether numbers can be negative, how division behaves, and how errors are reported.

Brute force to say out loud

Repeatedly find the innermost call with a regex and replace it with its value. Quick but fragile.

Optimal idea

Tokenize into identifiers, integers, commas and parentheses. Write parse(i) returning (value, next index): a number returns itself; a name must be followed by '(', then loop reading arguments until ')', then apply the function. A function table keeps new operators one line each. O(n), recursion depth equals nesting depth.

Dry-run these cases

add(1, sub(1, 0)) is 2. add(2, mul(3, pow(4, 5))) is 3074. A bare number. Variable argument counts. Unknown function names.

Likely follow-ups

Variables, iterative parsing with two stacks, or division semantics.

Pitfalls

Losing the next index after parsing an argument. Treating '-' in '-5' as an operator.

HOW TO THINK ABOUT IT

An expression with function calls inside function calls has a grammar: expression = number | name '(' expression, expression, ... ')'. Each call can be evaluated only after its arguments, so write a function that parses one expression and reports where it stopped (recursive descent). A tokenizer first, then parse(i) returning (value, next index), keeps the bookkeeping small.

Basis: Recursive descent parsing, in the spirit of LeetCode 224 / 726; closest on LeetCode is 770, Basic Calculator IV.

BRUTE FORCE - PSEUDO-CODE

```
while the text still contains a call:
    find a call whose arguments contain no parentheses, e.
g. add(2, 3)
    evaluate it and replace that piece of text with the re
sult
return the remaining number
```

Time: O(n^2) | Space: O(n)

Why it is slow

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION - TOKENIZER + RECURSIVE DESCENT OVER NAME(ARG, ARG, ...)

```
def evaluate(expr):
    tokens = re.findall(r"-?\d+|[A-Za-z_]\w*|[(),]", expr)
    ops = {
        "add": lambda *a: sum(a),
        "sub": lambda a, b: a - b,
        "mul": lambda *a: math.prod(a),
        "pow": lambda a, b: a**b,
        "div": lambda a, b: int(
            a / b
        ),  # truncates toward zero; ask the interviewer a
bout negatives
        "min": min,
        "max": max,
    }

    def parse(i):
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        if tok not in ops:
            raise ValueError(f"unknown function {tok!r}")
        if tokens[i + 1] != "(":
            raise ValueError("expected '('")
        i += 2
        args = []
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return ops[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value
```

Time: O(n) | Space: O(depth)

Why

Tokenize into identifiers, integers, commas and parentheses. parse(i) returns (value, next index): a number returns itself; an identifier must be followed by '(', then arguments are collected until ')' so any number of arguments works. Apply the function table to the collected values. Keeping the parser and the function table separate makes new functions a one-line change.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

```
def evaluate_env(expr, env):
    """Variables: an identifier that is not followed by '(
' is looked up in env."""
    tokens = re.findall(r"-?\d+|[A-Za-z_]\w*|[(),]", expr)

    def parse(i):
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        if i + 1 < len(tokens) and tokens[i + 1] == "(":
            i += 2
            args = []
            while tokens[i] != ")":
                value, i = parse(i)
                args.append(value)
                if tokens[i] == ",":
                    i += 1
            return _OPS[tok](*args), i + 1
        return env[tok], i + 1

    return parse(0)[0]
```

NOTES FROM CANDIDATES

Tokenizer + recursive descent is the cleanest approach: parse operator, consume (, parse comma-separated child expressions, consume), then evaluate.
Iterative parsing works but is easy to make off-by-one errors under interview pressure; one candidate was specifically flagged for minor syntax and bounds issues.
Clarify division behavior for negative values and non-divisible integers.

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

MEDIUM | PracHub | Technical Screen | 139 of 196

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

STUDY: TYPE | Google frequency on LeetCode: all time 57.7, last 30 days 0 | book priority 40 of 196

Topic: Expression Parsing and Evaluation

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

APPROACH

DFS expansion with memoization per key and a 'visiting' set to detect cycles; leave unknown keys per the spec; keep literal text unchanged.

INTERVIEW-DAY PLAN

Clarify first

Ask for the placeholder syntax (%KEY%), what to do with an unknown key (leave it as is or raise), how to write a literal percent sign (escaping such as %%), and how to handle cycles (raise an error).

Brute force to say out loud

Repeatedly run a regex replace on the whole string until nothing changes. It loops forever on a cycle.

Optimal idea

Resolve each key with memoized DFS: resolve(key) expands the value of that key by scanning for placeholders and recursively resolving them, with a visiting set to detect cycles and a cache so each key is expanded once. Expand the template by scanning once and replacing each placeholder with resolve(name), leaving literal text untouched. Linear in the sizes of text and values.

Dry-run these cases

A value that refers to another key, a cycle (A refers to B, B to A), an unknown key, a template with no placeholders, two placeholders adjacent, an escaped percent.

Likely follow-ups

Escaping, recursion depth limit (iterative expansion), or limiting the output size to prevent blow-up.

Pitfalls

Re-expanding a key every time. Treating an unknown key and a cycle the same way.

HOW TO THINK ABOUT IT

Expanding one key can require expanding others, so define expand(key) recursively and remember results (memoization) so a key used many times is expanded once. A 'currently expanding' set detects cycles: meeting a key that is already on the stack means a cycle. Decide the policy for unknown keys and for literal percent signs up front.

Basis: Written. Assumed rules (confirm them): placeholders look like %KEY%; the value of a key may itself contain placeholders; an unknown key is left as is; a cycle raises an error.

BRUTE FORCE - PSEUDO-CODE

```
repeat up to a limit: replace every %KEY% in the text by i
ts value
stop when the text no longer changes; if the limit is hit:
 report a cycle
```

Time: O(depth * length) | Space: O(length)

Why it is slow

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def expand_template(values, template):
    memo = {}
    state = {}

    def resolve(root_key):
        if root_key in memo:
            return memo[root_key]
        if root_key not in values:
            raise KeyError(root_key)
        state[root_key] = 1
        stack = [[root_key, values[root_key], 0, []]]
        while stack:
            key, text, cursor, pieces = stack[-1]
            start = text.find("%", cursor)
            if start == -1:
                pieces.append(text[cursor:])
                expanded = "".join(pieces)
                memo[key] = expanded
```

```
                state[key] = 2
                stack.pop()
                if stack:
                    stack[-1][3].append(expanded)
                continue
            end = text.find("%", start + 1)
            pieces.append(text[cursor:start])
            stack[-1][2] = end + 1
            dependency = text[start + 1 : end]
            if dependency in memo:
                pieces.append(memo[dependency])
                continue
            if state.get(dependency) == 1:
                raise ValueError("cyclic placeholder depen
dency")
            if dependency not in values:
                raise KeyError(dependency)
            state[dependency] = 1
            stack.append([dependency, values[dependency],
0, []])
        return memo[root_key]

    def expand_text(text):
        pieces = []
        cursor = 0
        while cursor < len(text):
            start = text.find("%", cursor)
            if start == -1:
                pieces.append(text[cursor:])
                break
            end = text.find("%", start + 1)
            pieces.append(text[cursor:start])
            pieces.append(resolve(text[start + 1 : end]))
            cursor = end + 1
        return "".join(pieces)

    return expand_text(template)
```

Time: O(I + O), where I is the reachable input text and O is the produced output size | Space: O(D + M + O) for recursion depth D, memoized expansions M, and output

Explanation

Scan any template fragment from left to right, copying literal spans and resolving the key between each matched percent pair. Resolution recursively expands the mapped value, memoizes the finished string, and tracks active keys to guard the contract. Shared dependencies are therefore expanded once.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

```
def expand_with_escapes(template, mapping):
    """'%%' means a literal percent sign; everything else
as above."""
    cache, visiting = {}, set()

    def expand_text(text):
        res, j = [], 0
        while j < len(text):
            if text.startswith("%%", j):
                res.append("%")
                j += 2
                continue
            m = _PH.match(text, j)
            if m and m.group(1) in mapping:
                key = m.group(1)
                if key in visiting:
                    raise ValueError(f"cycle at {key}")
                if key not in cache:
                    visiting.add(key)
                    cache[key] = expand_text(mapping[key])
                    visiting.discard(key)
                res.append(cache[key])
                j = m.end()
            else:
                res.append(text[j])
                j += 1
        return "".join(res)

    return expand_text(template)
```

An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.

Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

MEDIUM | 1Point3Acres | onsite-coding | 140 of 196

Expression Tree Single-Leaf Mutation

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 182 of 196
Topic: Expression Parsing and Evaluation
Optimal answer from: written and tested here for this exact question
Frequency: Low | Last asked: 2026-09-27 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

APPROACH

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

INTERVIEW-DAY PLAN

Clarify first

Ask which operators exist (AND, OR, XOR, NOT), whether XOR and others are binary only, whether nodes have parent pointers, and whether leaves are flipped one at a time and then restored or cumulatively.

Brute force to say out loud

For each leaf flip it, evaluate the whole tree, and flip it back. O(n^2).

Optimal idea

Evaluate once and cache each node's value. During that traversal record each leaf's path to the root (or parent links). To flip a leaf, recompute only the nodes on its path upward, stopping early when a parent's value does not change because the root value is then the same as before. That is O(depth) per flip.

Dry-run these cases

A single leaf tree, a NOT at the root, XOR chains where every flip changes the root, OR with a true sibling (early stop).

Likely follow-ups

Several flips applied together, or an undo.

Pitfalls

Mutating the cached values without restoring them between independent flips.

HOW TO THINK ABOUT IT

Flipping one leaf can only change the nodes on its path to the root, so cache every node's value and recompute only that path. The path is found through parent links you build during the first traversal. Stop early when a recomputed node keeps its old value, because nothing above it can change.

Basis: Written; incremental evaluation on a tree. Assumed rules (confirm them): leaves hold booleans; internal nodes are AND, OR, XOR (binary) or NOT (unary); report the root value after flipping each leaf on its own (flip, read the root, restore).

BRUTE FORCE - PSEUDO-CODE

```
for each leaf: flip it, evaluate the whole tree to get the
 root value, flip it back
```

Time: O(n^2) | Space: O(h)

Why it is slow

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION - CACHE VALUES AND PARENTS, RECOMPUTE ONLY THE PATH WITH EARLY STOP

```
class XNode:
    def __init__(self, op=None, left=None, right=None, val
=None):
        self.op, self.left, self.right, self.val = op, lef
t, right, val


def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()


def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(node.op, _eval_full(node.left), _eval_fu
ll(node.right))


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
    return out
```

```
def flips(root):
    value, parent = {}, {}

    def build(node, par):
        parent[id(node)] = par
```

```
        if node.op is None:
            value[id(node)] = node.val
        elif node.op == "NOT":
            build(node.left, node)
            value[id(node)] = not value[id(node.left)]
        else:
            build(node.left, node)
            build(node.right, node)
            value[id(node)] = _apply(
                node.op, value[id(node.left)], value[id(no
de.right)]
            )

    build(root, None)
    base_root = value[id(root)]
    out = []
    for leaf in _leaves(root):
        saved = []
        value[id(leaf)] = not value[id(leaf)]
        saved.append((id(leaf), not value[id(leaf)]))
        cur = parent[id(leaf)]
        changed = True
        while cur is not None and changed:
            if cur.op == "NOT":
                new = not value[id(cur.left)]
            else:
                new = _apply(cur.op, value[id(cur.left)],
value[id(cur.right)])
            changed = new != value[id(cur)]
            if changed:
                saved.append((id(cur), value[id(cur)]))
                value[id(cur)] = new
```

```
            cur = parent[id(cur)]
        out.append(value[id(root)])
        for key, old in saved:
            value[key] = old
    assert value[id(root)] == base_root
    return out
```

Time: O(n + L * depth) | Space: O(n)

Why

One traversal stores every node's value and its parent. To flip a leaf, toggle it, then walk up recomputing each ancestor from its children's cached values, stopping as soon as a node's value does not change (then the root is unchanged too). Undo by restoring the saved values on the walked path.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

```
class XNode:
    def __init__(self, op=None, left=None, right=None, val
=None):
        self.op, self.left, self.right, self.val = op, lef
t, right, val


def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(node.op, _eval_full(node.left), _eval_fu
ll(node.right))


def sensitive_leaves(root):
    """Top-down sensitivity: a leaf matters when every anc
estor lets its value through."""

    def val(n):
        return _eval_full(n)

    out = []

    def go(node, matters):
        if node.op is None:
            if matters:
                out.append(node)
            return
        if node.op == "NOT":
            go(node.left, matters)
            return
        a, b = val(node.left), val(node.right)
        if node.op == "AND":
            go(node.left, matters and b)
            go(node.right, matters and a)
        elif node.op == "OR":
            go(node.left, matters and not b)
            go(node.right, matters and not a)
        else:
            go(node.left, matters)
            go(node.right, matters)

    go(root, True)
    return {id(x) for x in out}
```

NOTES FROM CANDIDATES

Brute force is: collect leaves by inorder traversal, flip each leaf, and re-evaluate the whole tree.

The natural optimization is to cache subtree values and only recompute the path from the flipped leaf to the root.

Clarify whether XOR is binary-only and whether the tree is guaranteed valid.

Early-stop optimization is expected: after a leaf flip, recompute upward only until a parent value stays unchanged, then reuse the previous root value.

Settle how intermediate update state will be represented before implementation. In an L4 onsite, uncertainty about that representation caused a promising approach to fall back to brute-force code and contributed to negative coding feedback.

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

Other: Design, Heaps, Intervals and Misc 32 questions

MEDIUM | HackTheRounds | Phone Screen | 141 of 196

★ TOP 75: rank 59 in the Medium and Hard priority list

★ Meeting Rooms III Variant

STUDY: TYPE | Google frequency on LeetCode: all time 58, last 30 days 0 | book priority 34 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 (cyclic variant)

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- `1 <= intervals.length <= 10^4`
- `0 <= start < end <= 24 * 60 (minutes in a day)`
- `Meetings may wrap around (e.g., [23:00, 01:00])`

APPROACH

Sweep line; split intervals that wrap past midnight

INTERVIEW-DAY PLAN

Clarify first

Ask whether the 24-hour day repeats cyclically, whether a meeting that wraps midnight must stay in one room the whole time, and whether a meeting ending exactly when another starts conflicts (usually not).

Brute force to say out loud

Try assigning meetings to rooms one by one with backtracking. Exponential.

Optimal idea

Split a wrapping meeting [s, e] with e < s into [s, 24h] and [0, e] and run a sweep line over the events: sort start and end events, +1 at a start, -1 at an end (process ends first at equal times), and take the maximum running count. That gives the minimum number of rooms when pieces may use different rooms. If a wrapping meeting must stay in a single room the exact answer is circular-arc coloring, which can be larger than the maximum overlap (five arcs forming an odd cycle need three rooms with overlap two) and is hard in general, so state the assumption and mention a heuristic.

Dry-run these cases

No wrap, one wrapping meeting, a wrap that overlaps the beginning of the day, back-to-back meetings, all meetings the same.

Likely follow-ups

Return which room each meeting uses, or check feasibility with a given number of rooms.

Pitfalls

Counting back-to-back meetings as overlapping. Splitting a wrap but treating the two pieces as two meetings in the output.

HOW TO THINK ABOUT IT

Split a wrapped meeting into two pieces (from start to the end of the day, and from midnight to its end) and run the normal sweep line: the largest number of meetings active at once is the number of rooms needed. That is exact when pieces may sit in different rooms; if a meeting must keep one room, the arcs form a circular-arc graph and the true minimum can exceed the maximum overlap (five overlapping arcs around a circle need 3 rooms with overlap 2). Say which reading you assume.

Basis: LeetCode 253 (Meeting Rooms II) on a circular day. Meetings are half-open [start, end) in minutes; a meeting with end < start wraps past midnight. Two readings: the usual interview answer (split the wrapped meeting and take the maximum overlap) and the exact answer when a wrapped meeting must keep one room.

BRUTE FORCE - PSEUDO-CODE

```
turn each meeting into the set of minute slots it occupies
 (a wrapped meeting is two
    ranges)
for k = 1, 2, ...: try to assign each meeting one of k roo
ms so that no two meetings
    in a room share a slot (backtracking)
answer = the first k that works
```

Time: exponential | Space: O(n)

Why it is slow

Backtracking over room assignments is exponential. Splitting wrapped meetings and sweeping start and end events gives the maximum overlap in O(n log n); that equals the exact answer when meetings do not wrap.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: LINE SWEEP

```
from typing import List


class Solution:
    def minMeetingRooms(self, n: int, meetings: List[List[
int]]) -> int:
        if not meetings:
            return 0

        # Use line sweep: +1 at start, -1 at end
        events = []
        for start, end in meetings:
            events.append((start, 1))  # Meeting starts
            events.append((end, -1))  # Meeting ends

        # Sort by time, with ends (-1) before starts (+1)
at same time
        events.sort(key=lambda x: (x[0], x[1]))

        max_rooms = 0
        current_rooms = 0

        for time, delta in events:
            current_rooms += delta
            max_rooms = max(max_rooms, current_rooms)

        return max_rooms
```

Time: O(n log n) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

A wrapped meeting must stay in a single room.

Then it is circular-arc colouring, NP-hard in general. A common practical method: cut the circle at the minute with the fewest active meetings; the meetings crossing the cut use fixed rooms, and the rest are scheduled greedily with a heap. It can use more rooms than the maximum overlap, and the five-arc example shows why.

Return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room id); reuse the earliest-ending room if it is free, otherwise open a new one. For wrapped meetings schedule them first so their rooms are fixed.

Days can repeat (a wrapped meeting recurs every day).

A recurring meeting occupies its slots every day, so the circle model above is exactly right, and the no-wrap shortcut of ordering by start time is not.

Return which room each meeting uses.

Sort by start and keep a heap of (end time, room): reuse the earliest-ending room if it is free, otherwise open a new one. For meetings that do not wrap this uses exactly the minimum number of rooms (verified against the sweep); wrapped meetings need the cut-the-circle method.

```
def assign_rooms(meetings):
    free, out, rooms = [], {}, 0
    for i in sorted(range(len(meetings)), key=lambda i: me
etings[i][0]):
        s, e = meetings[i]
        if free and free[0][0] <= s:
            _, room = heapq.heappop(free)
        else:
            room, rooms = rooms, rooms + 1
        out[i] = room
        heapq.heappush(free, (e, room))
    return rooms, out
```

MEDIUM | HackTheRounds | Phone Screen | 142 of 196

Playlist Shuffle No Adjacent Artists

STUDY: TALK | Google frequency on LeetCode: all time 34.9, last 30 days 0 | book priority 99 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #767 Reorganize String

LeetCode match: LC 767 / LC 1054

QUESTION

Given a playlist of songs where each song has an artist, implement a shuffle function that minimizes (or eliminates) adjacent songs by the same artist.

EXAMPLES

Example 1	
Input	`[("A", "song1"), ("A", "song2"), ("B", "song3"), ("B", "song4"), ("C", "song5")]`
Output	`[("A", "song1"), ("B", "song3"), ("A", "song2"), ("B", "song4"), ("C", "song5")]`
Why	`No adjacent songs have same artist.`

CONSTRAINTS

- `1 <= songs.length <= 10^4`

APPROACH

Max-heap by remaining count per artist

INTERVIEW-DAY PLAN

Clarify first

Ask whether the shuffle must be random, how to behave when a perfect arrangement is impossible (the most frequent artist has more than half the songs, rounded up), and whether the result must be uniform among valid shuffles.

Brute force to say out loud

Shuffle randomly, then check for adjacent artists and retry. May never finish.

Optimal idea

Feasibility: let n be the song count; an arrangement exists if the largest artist group has at most ceil(n / 2) songs. Build it greedily with a max-heap of remaining songs per artist: repeatedly take from the artist with the most remaining songs who is not the previous artist, using random choice among songs of that artist (and optionally among equally big groups) to keep the result varied. If infeasible, place the leftovers at the end or spread them as evenly as possible, and say that adjacency is then unavoidable. O(n log A).

Dry-run these cases

One artist (impossible), two artists with equal counts (alternating), the largest artist at exactly half rounded up, three songs with two from the same artist.

Likely follow-ups

Uniform randomness among all valid shuffles (harder), or a minimum spacing of K songs between the same artist.

Pitfalls

Choosing the largest group even when it was just played. Assuming the greedy result is uniformly random.

HOW TO THINK ABOUT IT

A perfect arrangement exists exactly when the most frequent artist has at most ceil(n / 2) songs. Build it greedily: always play from the artist with the most songs left, except the artist who just played. A max-heap gives the most frequent artist; hold the previous artist back for one step. If it is impossible, the leftover songs of the dominant artist are the only unavoidable adjacencies.

Basis: LeetCode 767 (Reorganize String) and 1054 (Distant Barcodes) with songs and artists. Minimising adjacent same-artist pairs when a perfect arrangement does not exist.

BRUTE FORCE - PSEUDO-CODE

```
for every ordering of the songs: count neighbours with the
 same artist
answer = the ordering with the fewest such neighbours
```

Time: O(n! * n) | Space: O(n)

Why it is slow

n! orderings. A greedy that always plays the artist with the most songs left (not the one that just played), using a heap, is O(n log A).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: GREEDY WITH MAX HEAP

```
from typing import List
from collections import defaultdict
import heapq


class Solution:
    def shufflePlaylist(self, songs: List[List[str]]) -> L
ist[List[str]]:
        if not songs:
            return []

        artist_songs = defaultdict(list)
        for artist, song in songs:
            artist_songs[artist].append([artist, song])

        heap = [(-len(v), k) for k, v in artist_songs.item
s()]
        heapq.heapify(heap)

        result = []
        prev_artist = None
        waiting = None

        while heap or waiting:
            if not heap:
                if waiting:
                    result.append(artist_songs[waiting[1]]
.pop())
                    waiting = None
                continue

            count, artist = heapq.heappop(heap)

            if artist == prev_artist and heap:
                waiting = (count, artist)
                count, artist = heapq.heappop(heap)

            result.append(artist_songs[artist].pop())
            prev_artist = artist

            if artist_songs[artist]:
                heapq.heappush(heap, (count + 1, artist))

            if waiting:
                heapq.heappush(heap, waiting)
                waiting = None

        return result
```

Time: O(n log k) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The shuffle should look random, not always the same arrangement.

Break ties between equally frequent artists randomly and pick the song within the artist at random, as in the code. The result is valid but not uniform among all valid shuffles; a uniform sampler needs counting arrangements.

```
def shuffle_random_no_adjacent(artists, rng):
    """Random order among valid choices: break ties betwee
n equally frequent artists at random (valid, but not unifo
rm over all valid shuffles)."""
    counts = Counter(artists)
    out, prev = [], None
    for _ in range(len(artists)):
        options = [a for a in counts if counts[a] and a !=
 prev] or [
            a for a in counts if counts[a]
        ]
        top = max(counts[a] for a in options)
        a = rng.choice([x for x in options if counts[x] ==
 top])
        out.append(a)
        counts[a] -= 1
        prev = a
    return out
```

Songs by the same artist must be at least K apart.

Same idea with a waiting queue of length K: after playing an artist, hold them for K - 1 more songs before they are eligible again (the

LeetCode 358 construction).

What is the minimum number of unavoidable repeats?

With n songs and a dominant artist with m songs, the other n - m songs separate at most n - m + 1 groups, so at least m - (n - m + 1) = 2m - n - 1 neighbouring pairs of that artist are forced (if positive).

MEDIUM | PracHub | Technical Screen | 143 of 196

★ TOP 75: rank 28 in the Medium and Hard priority list

★ Implement frequency + distance top-K queries

STUDY: TYPE | Google frequency on LeetCode: all time 58.8, last 30 days 41.1 | book priority 19 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #347 Top K Frequent Elements, LeetCode #973 K Closest Points to Origin

LeetCode match: LC 347 Top K Frequent Elements + LC 973 K Closest Points to Origin

Original: https://prachub.com/coding-questions/implement-frequency-plus-distance-top-k-queries

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break)
You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **`p`** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **`k` distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **`x`** ascending, then **`y`** ascending

Input

- **`points`**: list of length **`n`**, each element **`[x, y]`**
- **`k`**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **`k`** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup
Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.
Constraints
1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points
Examples
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1] appears 2 times, so they are the top 2 distinct points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each appear twice. Their squared distances are 4, 5, and 5, so [0,2] comes first. Between [1,2] and [2,1], x is smaller for [1,2].
Hints
First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like **`(-frequency, distance, x, y)`**.
Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either **`['set', key, value, timestamp]`** or **`['getClosest', key, timestamp]`**. For every **`getClosest`** operation, return the stored value whose timestamp is closest to the query tim

APPROACH

Count frequencies in a hash map; use a size-k heap with key (frequency desc, distance asc) for Q1, and a max-heap of size k on distance for Q2.

INTERVIEW-DAY PLAN

Clarify first

Two questions. Q1 (top-K points): confirm the full ordering (higher frequency, then smaller distance, then smaller x, then smaller y) and that K is at most the number of distinct points. Q2 (time-based store): timestamps for a key arrive in non-decreasing order, and a tie in distance returns the smaller timestamp.

Brute force to say out loud

Q1: sort all distinct points with the full key. Q2: linear scan of a key's timestamps. Both are correct and easy to state.

Optimal idea

Q1: Counter over tuples, then heapq.nsmallest(k, distinct, key=(-freq, x*x + y*y, x, y)), O(n + d log k). Q2: per key keep parallel lists of timestamps and values; bisect_left finds the first timestamp at least the query; compare it with the previous one and return the closer value, the smaller timestamp on a tie. O(log m) per query.

Dry-run these cases

Q1: duplicates, ties on frequency and distance, K equal to the number of distinct points. Q2: query before the first timestamp, after the last, exactly on a timestamp, the tie example (a, 12 returns y), an unknown key.

Likely follow-ups

Q1: top-K in a stream. Q2: an out-of-order insert (then use a sorted container), deletes, and memory limits.

Pitfalls

Q1: sorting by distance only. Q2: off-by-one at the ends of the list.

HOW TO THINK ABOUT IT

Q1 is a ranking: say the full sort key out loud, count with a hash map, and then you only need the K best (heap), not a full sort. Q2 keeps each key's timestamps in a sorted list, so the nearest one is next to the insertion point: binary search, then compare the two neighbours.

Basis: Two tasks: top K points by (frequency desc, squared distance asc, x asc, y asc), a mix of LeetCode 347 and 973; and a time-based store whose getClosest returns the value at the nearest timestamp (ties to the smaller one), a variant of LeetCode 981.

BRUTE FORCE - PSEUDO-CODE: Q1: SORT ALL DISTINCT POINTS BY THE FULL KEY

```
count the points; sort the distinct points by (frequency d
escending, squared distance,
    x, y)
return the first K
```

Time: O(d log d) | Space: O(d)

Why it is slow

Sorts all d distinct points, O(d log d); a size-K heap with the same key is O(d log k).

BRUTE FORCE - PSEUDO-CODE: Q2: SCAN EVERY TIMESTAMP OF THE KEY

```
getClosest(key, t): walk the key's list of (timestamp, val
ue)
keep the entry with the smallest |timestamp - t| (smaller
timestamp on a tie); return
    its value
```

Time: O(m) per query | Space: O(total entries)

Why it is slow

Linear in the number of stored timestamps per query. Timestamps arrive in order, so binary search on the sorted list finds the neighbours in O(log m).

OPTIMAL SOLUTION - Q1: HEAP WITH THE FULL KEY

```python
def top_points(points, k):
    c = Counter(map(tuple, points))
    best = heapq.nsmallest(
        k, c, key=lambda p: (-c[p], p[0] ** 2 + p[1] ** 2,
 p[0], p[1])
    )
    return [list(p) for p in best]
```

Time: O(n + d log k) | Space: O(d)

Why

Count with a Counter, then heapq.nsmallest(k, ...) with the key (-frequency, squared distance, x, y). Squared distances avoid square roots and are exact for integers.

OPTIMAL SOLUTION - Q2: PARALLEL SORTED LISTS AND BISECT

```python
class ClosestStore:
    def __init__(self):
        self.times, self.values = defaultdict(list), defau
ltdict(list)

    def set(self, key, value, ts):
        self.times[key].append(ts)
        self.values[key].append(value)

    def getClosest(self, key, ts):
        t = self.times.get(key)
        if not t:
            return ""
        j = bisect_right(t, ts)  # t[:j] are at or before
ts, t[j:] after it
        if j == 0:
            return self.values[key][bisect_right(t, t[0])
- 1]
        if j == len(t):
            return self.values[key][-1]
        before, after = t[j - 1], t[j]
        return (
            self.values[key][j - 1]
            if ts - before <= after - ts
            else self.values[key][bisect_right(t, after) -
 1]
        )
```

Time: O(1) per set, O(log m) per getClosest | Space: O(total entries)

Why

Timestamps for a key arrive in non-decreasing order, so appending keeps the list sorted. bisect_right splits the list into timestamps at or before the query and after it; the answer is the last of the first part or the first of the second part, whichever is closer (the earlier one on a

tie). If a timestamp was set twice, the most recent value wins.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log k) per query), or maintain a sorted structure updated on every add if queries dominate.

```
class TopPointsStream:
    """Follow-up: points arrive one at a time; top_k reads
 from the counts without re-counting the history."""

    def __init__(self):
        self.c = Counter()

    def add(self, x, y):
        self.c[(x, y)] += 1

    def top_k(self, k):
        return [
            list(p)
            for p in heapq.nsmallest(
                k, self.c, key=lambda p: (-self.c[p], p[0]
 ** 2 + p[1] ** 2, p[0], p[1])
            )
        ]
```

Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

Q2 returns the closest value at or before the timestamp (LeetCode 981).

Use bisect_right(timestamps, ts)

- 1 and return that value or an empty string if the index is negative; there is no tie question.

FUNCTION AND TEST CASES

Signature

```
def solution(points: list[list[int]], k: int) -> list[list
[int]]:
```

Sample call

```
solution([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
```

Test cases

Input	Expected
([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)	[[2, 2], [1, 1]]
([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1] ,[3,0]], 3)	[[0, 2], [1, 2], [2, 1]]

MEDIUM | IN APP LC 621 | PracHub | Technical Screen | 144 of 196

Rearrange Tasks With Cooldown

STUDY: TALK | Google frequency on LeetCode: all time 40.3, last 30 days 0 | book priority 82 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #621 Task Scheduler

LeetCode match: LC 621 Task Scheduler (inferred from overview)

Original:
https://prachub.com/coding-questions/rearrange-tasks-with-cooldown

QUESTION

Overview: This question evaluates task-scheduling and constraint-handling skills, testing algorithmic reasoning about arranging repeated jobs with cooldown constraints along with analysis of schedule optimality and algorithmic complexity.

APPROACH

Greedy with max-heap + cooldown queue, or the formula (maxFreq-1)*(n+1)+countOfMax vs len(tasks). Rearranged output: always take the available task with highest remaining count.

HOW TO THINK ABOUT IT

The most frequent task is the bottleneck. Lay it out with gaps of n between its copies; every other task can fill gaps. Total length is the larger of 'that skeleton' and 'just the number of tasks' (when there are so many tasks the gaps are all filled and no idle time is needed). The skeleton has (maxcount - 1) full blocks of size n + 1 plus one last block holding every task tied for the maximum.

Basis: LeetCode 621, Task Scheduler (minimum time), with LeetCode 358, Rearrange String k Distance Apart as the construction follow-up. Assumed rule: the same task needs n idle-or-other slots between runs.

BRUTE FORCE - PSEUDO-CODE

```
each time unit: among tasks not cooling down, run the one
with the most copies left
    (idle if none)
put it on cooldown for n units; stop when all tasks are do
ne; answer = number of time
    units
```

Time: O(T log k) where T is the answer | Space: O(k)

Why it is slow

It simulates every slot, including long idle stretches, O(T log k). The answer follows from the most frequent task: max(len(tasks), (maxCount - 1) * (n + 1) + number of tasks with that count).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(tasks, cooldown):
    from collections import Counter, deque
    import heapq

    counter = Counter(tasks)
    if not counter:
        return ""

    heap = [(-count, task) for task, count in counter.item
s()]
```

```
    heapq.heapify(heap)

    waiting = deque()  # (ready_time, task, remaining_coun
t)
    time = 0
    schedule = []

    while heap or waiting:
        while waiting and waiting[0][0] <= time:
            _, task, remaining = waiting.popleft()
            heapq.heappush(heap, (-remaining, task))

        if heap:
            neg_count, task = heapq.heappop(heap)
            remaining = -neg_count - 1
            schedule.append(task)
            if remaining > 0:
                waiting.append((time + cooldown + 1, task,
 remaining))
        else:
            schedule.append("_")

        time += 1

    return "".join(schedule)
```

Time: O(L log U), where L is the returned schedule length and U is the number of distinct task types | Space: O(U)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

```
def rearrange_k_distance(s, k):
    """Return an arrangement with equal letters at least k
 apart, or '' if impossible (LeetCode 358)."""
    if k <= 1:
        return s
    heap = [(-c, ch) for ch, c in Counter(s).items()]
    heapq.heapify(heap)
    waiting, out = deque(), []
    while heap:
        c, ch = heapq.heappop(heap)
        out.append(ch)
        waiting.append((c + 1, ch))
        if len(waiting) >= k:
            c2, ch2 = waiting.popleft()
            if c2:
                heapq.heappush(heap, (c2, ch2))
    return "".join(out) if len(out) == len(s) else ""
```

Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

FUNCTION AND TEST CASES

Signature

```
def solution(tasks: list[str], cooldown: int) -> str:
```

Sample call

```
solution(['A', 'A', 'A', 'B', 'B', 'C'], 2)
```

Test cases

Input	Expected
(['A', 'A', 'A', 'B', 'B', 'C'], 2)	'ABCAB_A'
(['A', 'A', 'A', 'B'], 2)	'AB_A__A'

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Design a task scheduler in Python. | Exponent

https://www.aced.io/questions/3504/design-task-scheduler-python

---

MEDIUM | PracHub | Technical Screen | 145 of 196

★ TOP 75: rank 69 in the Medium and Hard priority list

★ Solve Rooms and Top-K Streams

STUDY: TYPE | Google frequency on LeetCode: all time 58, last 30 days 0 | book priority 35 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II, LeetCode #703 Kth Largest Element in a Stream

LeetCode match: LC 253 Meeting Rooms II + LC 703 Kth Largest Element in a Stream (inferred from overview)

Original:

https://prachub.com/coding-questions/solve-rooms-and-top-k-streams

QUESTION

Overview: This question evaluates algorithmic problem-solving skills across interval scheduling/resource allocation and streaming top-k maintenance, testing competency in data structures for ordered queries, tie-breaking logic, stateful stream processing, and analysis of time and space trade-offs.

APPROACH

Rooms: min-heap of end times. Stream: size-k min-heap, push and pop when over k.

INTERVIEW-DAY PLAN

Clarify first

Two parts. Rooms: is a meeting that starts exactly when another ends a conflict (the scraped solution treats it as no conflict)? Stream: top-K means the K largest values or K most frequent, and what ties do. The scraped solution covers only the rooms part.

Brute force to say out loud

Rooms: assign meetings to rooms and try all orders. Stream: store everything and sort on each query.

Optimal idea

Rooms: sort starts and ends separately and sweep with two pointers: if the next start is at least the earliest end, reuse that room, otherwise open a new one; or keep a min-heap of room end times. O(n log n). Stream of the K largest (LeetCode 703): a min-heap of size K, push each value and pop when the size exceeds K, so the root is the K-th largest. O(log K) per value.

Dry-run these cases

Rooms: [[0,30],[5,10],[15,20]] gives 2, an empty list gives 0, back-to-back meetings. Stream: fewer than K values so far, duplicates, K = 1.

Likely follow-ups

Return the room assignment, or a sliding window of the last N values for the stream.

Pitfalls

Sorting starts and ends together by start. Returning from the heap before it holds K items.

HOW TO THINK ABOUT IT

Rooms: the number of rooms needed is the largest number of meetings happening at the same time, so sort the starts and the ends separately and sweep: a start that comes before the earliest end needs a new room. Stream: only the K largest values can matter, so keep exactly those in a min-heap; the root is the K-th largest.

Basis: Two tasks: LeetCode 253 (minimum meeting rooms; back-to-back meetings may share a room) and LeetCode 703 (Kth Largest Element in a Stream).

BRUTE FORCE - PSEUDO-CODE: ROOMS: COUNT THE OVERLAP AT EVERY TIME UNIT

```
for every integer time t from the earliest start to the la
test end: count the meetings
    with start <= t < end
answer = the largest count
```

Time: O(range * n) | Space: O(1)

Why it is slow

Depends on the length of the time range, not only on n: O(range * n). Sorted starts and ends with two pointers (or a heap of end times) are O(n log n).

BRUTE FORCE - PSEUDO-CODE: STREAM: SORT EVERYTHING ON EVERY QUERY

```
keep all values; after every add: sort descending and read
 index K - 1
```

Time: O(n log n) per add | Space: O(n)

Why it is slow

Every add re-sorts the whole history, O(n log n). A min-heap of size K keeps the K-th largest at its root in O(log K) per add.

OPTIMAL SOLUTION - ROOMS: SORTED STARTS AND ENDS WITH TWO POINTERS

```
def minMeetingRooms(intervals):
    starts, ends = sorted(s for s, _ in intervals), sorted
(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            j += 1
        else:
            rooms += 1
    return rooms
```

Time: O(n log n) | Space: O(n)

Why

Sort starts and ends separately. For each start: if it is at or after the earliest unfinished end, that room is reused (advance the end pointer); otherwise a new room is needed. The count of rooms opened is the maximum overlap.

OPTIMAL SOLUTION - STREAM: MIN-HEAP OF SIZE K

```
class KthLargestStream:
    def __init__(self, k, nums=()):
        self.k, self.heap = k, []
        for v in nums:
            self.add(v)

    def add(self, v):
        heapq.heappush(self.heap, v)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0] if len(self.heap) == self.k el
se None
```

Time: O(log k) per add | Space: O(k)

Why

Push each value; if the heap has more than K items pop the smallest. The root is then the K-th largest. Memory is K, regardless of how many values have been seen.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

```
def meeting_room_assignment(intervals):
    """Which room each meeting uses: min-heap of (end time
, room id)."""
    free, out, rooms = [], {}, 0
    for i in sorted(range(len(intervals)), key=lambda i: i
ntervals[i][0]):
        s, e = intervals[i]
        if free and free[0][0] <= s:
            _, room = heapq.heappop(free)
        else:
            room, rooms = rooms, rooms + 1
        out[i] = room
        heapq.heappush(free, (e, room))
    return rooms, out
```

Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. Cost: a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 30], [5, 10], [15, 20]])
```

Test cases

Input	Expected
`([[0, 30], [5, 10], [15, 20]],)`	`2`
`([],)`	`0`

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 146 of 196

★ TOP 75: rank 58 in the Medium and Hard priority list

★ Compute servers needed for daily recurring jobs

STUDY: TYPE | Google frequency on LeetCode: all time 58, last 30 days 0 | book priority 36 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Original: https://prachub.com/coding-questions/compute-servers-needed-for-daily-recurring-jobs

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.
Examples
Input: ([],)
Expected Output: 0
Explanation: There are no jobs, so no servers are needed.
Input: ([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)
Expected Output: 1
Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.
Hints
At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id

may use the same dedicated server. For a given target_day, compute the total number of

APPROACH

Max concurrent jobs = servers needed; sweep line over start/end events, or min-heap of end times. Handle intervals wrapping midnight by splitting.

HOW TO THINK ABOUT IT

The number of servers is the largest number of jobs running at the same time. First remove the twist: overlapping intervals of the same job are really one longer interval, so merge the intervals of each job_id. Then it is the plain maximum overlap: sort start and end events and keep a running count.

Basis: LeetCode 253 (Meeting Rooms II) with a twist from the statement: intervals of the same job_id may overlap on one server and count once. Intervals are half-open [start, end) within one day.

BRUTE FORCE - PSEUDO-CODE

```
for every time t in the day: jobs = the set of job ids tha
t have an interval covering
    t
answer = the largest set size
```

Time: O(T * m) | Space: O(m)

Why it is slow

Depends on the length of the day, O(T * m). Merge each job's own intervals first, then sort start and end events and keep a running count, O(m log m).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(intervals):
    from collections import defaultdict

    intervals_by_job = defaultdict(list)
    for job_id, start, end in intervals:
        intervals_by_job[job_id].append((start, end))

    events = defaultdict(int)

    for ranges in intervals_by_job.values():
        ranges.sort()
        current_start, current_end = ranges[0]

        for start, end in ranges[1:]:
            if start <= current_end:
                current_end = max(current_end, end)
            else:
                events[current_start] += 1
                events[current_end] -= 1
                current_start, current_end = start, end

        events[current_start] += 1
        events[current_end] -= 1

    active = 0
    answer = 0

    for time in sorted(events):
        active += events[time]
        answer = max(answer, active)

    return answer
```

Time: O(m log m), where m is the number of input intervals. Sorting intervals within each job_id group plus sorting event times dominates. | Space: O(m), for grouped intervals and sweep events.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

```python
def servers_needed(intervals):
    by_job = defaultdict(list)
    for j, s, e in intervals:
        by_job[j].append((s, e))
    events = []
    for ivs in by_job.values():
        cur_s, cur_e = None, None
        for s, e in sorted(ivs):
            if cur_e is not None and s <= cur_e:
                cur_e = max(cur_e, e)
            else:
                if cur_e is not None:
                    events += [(cur_s, 1), (cur_e, -1)]
                cur_s, cur_e = s, e
        events += [(cur_s, 1), (cur_e, -1)]
    cur = best = 0
    for _, d in sorted(events):
        cur += d
        best = max(best, cur)
    return best


def servers_with_dedicated(intervals, long_jobs):
    """Follow-up: jobs listed in long_jobs reserve a dedic
ated server for the whole period; the others share servers
."""
    shared = [(j, s, e) for j, s, e in intervals if j not
in long_jobs]
    return len({j for j, _, _ in intervals if j in long_jo
bs}) + servers_needed(shared)
```

Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

FUNCTION AND TEST CASES

Signature

```python
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([])
```

Test cases

Input	Expected
`([],)`	`0`
`([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)`	`1`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Compute minimum servers for cyclic tasks | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-servers-for-cyclic-tasks

---

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 147 of 196

★ TOP 75: rank 50 in the Medium and Hard priority list

★ Compute minimum number of rooms needed

STUDY: TYPE | Google frequency on LeetCode: all time 58, last 30 days 0 | book priority 37 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Original: https://prachub.com/coding-questions/compute-minimum-number-of-rooms-needed

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of `n` pairs `[start, end]` where `0 <= start <= end`.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- `1 <= n <= 200000`
- Times are integers in a reasonable range (e.g., `0 . 10^9`).

Example

- Input: `[[0,30],[5,10],[15,20]]`
- Output: `2` (One room can host `[0,30]`, the other can host `[5,10]` then `[15,20]`.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.

Constraints

0 <= n <= 2000000 <= start <= end <= 10^9

Examples

Input: ([[0,30],[5,10],[15,20]],)

Expected Output: 2

Explanation: [0,30] overlaps with both [5,10] and [15,20], so at most 2 rooms are needed at the same time.

Input: ([[7,10],[2,4],[4,7],[10,12]],)

Expected Output: 1

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

APPROACH

Sort by start, min-heap of end times (reuse a room when earliest end <= start), or two-pointer sweep on sorted starts/ends. O(n log n).

HOW TO THINK ABOUT IT

The rooms needed equal the largest number of meetings that overlap at one moment. Two ways to find that without checking every moment: sort by start and keep a min-heap of the end times of rooms in use (reuse the room that frees up first), or sort starts and ends separately and sweep with two pointers.

Basis: LeetCode 253, Meeting Rooms II (half-open intervals [start, end): a meeting ending at t does not conflict with one starting at t).

BRUTE FORCE - PSEUDO-CODE

```
for every integer time t: count the meetings with start <=
 t < end
answer = the largest count
```

Time: O(range * n) | Space: O(1)

Why it is slow

Depends on the length of the time range, O(range * n). A min-heap of room end times, or sorted starts and ends with two pointers, is O(n log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(intervals):
    events = []

    for start, end in intervals:
        # [start, end) with start == end is an empty inter
val.
        if start == end:
            continue
        events.append((start, 1))
        events.append((end, -1))

    # Process ending events before starting events at the
same time.
    events.sort(key=lambda event: (event[0], event[1]))
```

```
    rooms_in_use = 0
    max_rooms = 0

    for _, delta in events:
        rooms_in_use += delta
        if rooms_in_use > max_rooms:
            max_rooms = rooms_in_use

    return max_rooms
```

Time: O(n log n) | Space: O(n)

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Sorted starts and ends with two pointers

```
def min_meeting_rooms_two_pointers(intervals):
    real = [
        iv for iv in intervals if iv[0] < iv[1]
    ]  # zero-length meetings occupy no time
    starts, ends = sorted(s for s, _ in real), sorted(e fo
r _, e in real)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            j += 1
        else:
            rooms += 1
    return rooms
```

Time: O(n log n) | Space: O(n)

Why

Sort the starts and the ends separately. For each start, if it is at or after the earliest unfinished end, a room has been freed (move the end pointer); otherwise a new room is needed. No heap and no pairing of starts with ends is required.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

```
def can_attend_all(intervals):
    iv = sorted(intervals)
    return all(a[1] <= b[0] for a, b in zip(iv, iv[1:]))
```

Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0,30],[5,10],[15,20]])
```

Test cases

Input	Expected
`([[0,30],[5,10],[15,20]],)`	`2`
`([[7,10],[2,4],[4,7],[10,12]],)`	`1`

ALSO REPORTED - 4 MORE REPORTS OF THIS QUESTION

Find Minimum Rooms Needed | PracHub | Technical Screen

https://prachub.com/coding-questions/find-minimum-rooms-needed

Compute minimum rooms for time intervals | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-rooms-for-time-intervals

Given a list of car rental requests with start and end times, assign cars to rentals such | Exponent

https://www.aced.io/questions/5425/assign-cars-minimize-total-cars-rented

Meeting Rooms / Interval Scheduling Variants | 1Point3Acres | phone-screen, tech-screen, onsite-coding | asked 2026-09-20 | Very high

https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.
The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

Preparation: Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

MEDIUM | PracHub | Technical Screen | 148 of 196

★ TOP 75: rank 70 in the Medium and Hard priority list

★ Solve chat, grid paths, and car rentals

STUDY: TYPE | Google frequency on LeetCode: all time 58, last 30 days 0 | book priority 38 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words, LeetCode #62 Unique Paths, LeetCode #253 Meeting Rooms II

LeetCode match: LC 692 Top K Frequent Words + LC 62/63 Unique Paths + LC 253

Original: https://prachub.com/interview-questions/solve-chat-grid-paths-and-car-rentals

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:
- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

Tasks: 1.

Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.

Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

- An integer **`C`** = minimal number of cars needed.
- A mapping from each **`rental_id`** to a **`car_id`** in the range **`1..C`**, such that no two rentals assigned to the same **`car_id`** overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals **`[s1, e1)`** and **`[s2, e2)`** overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if **`e1 == s2`** they do **not** over

APPROACH

Top-K messages via counts + heap; grid paths via DP (with obstacles if present); car rentals = minimum fleet = max overlapping reservations (sweep line).

INTERVIEW-DAY PLAN

Clarify first

Three independent tasks. Chat: activity means the number of messages, ties resolved how? Grid paths: moves right or down only, with or without obstacles, modulo? Car rentals: minimum cars equals the maximum overlap of reservations; does a car returned at time t serve a rental starting at t?

Brute force to say out loud

Chat: sort the counts. Grid: recursion without memory. Cars: assign reservations to cars one by one.

Optimal idea

Chat: Counter and heapq.nsmallest with the key (count descending, user id ascending). Grid: dp over cells with a rolling row, dp[j] += dp[j - 1], and 0 on obstacles; O(mn) time, O(n) space. Cars: sort the reservations by start and keep a min-heap of end times; reuse the earliest-ending car if it is free, else add one. The heap size at the end is the answer.

Dry-run these cases

Chat: ties, fewer users than K. Grid: 1 x n, an obstacle on the start, no path. Cars: back-to-back rentals, one rental, all identical.

Likely follow-ups

Chat in a stream, grid paths with diagonal moves, cars with a fixed fleet size and rejecting requests.

Pitfalls

Treating back-to-back as overlapping. Forgetting the modulo in the grid count if the statement requires it.

HOW TO THINK ABOUT IT

Three unrelated tasks, so say so and give each a one-line plan. Chat: count, then take the top K with a heap. Grid: each column depends only on the previous one, so keep one column. Cars: the minimum number of cars is the largest number of rentals that overlap at once, which a heap of return times tracks while also giving an assignment.

Basis: Three independent tasks: top K frequent items (LeetCode 347 / 692), grid path counting (a Unique Paths variant) and minimum meeting rooms with assignment (LeetCode 253 plus the assignment step).

BRUTE FORCE - PSEUDO-CODE: Q1: SORT ALL USERS BY (COUNT, USER ID)

```
count the messages per user
sort all distinct users by (count descending, user id)
return the first K
```

Time: O(d log d) | Space: O(d)

Why it is slow

Sorting all d distinct users costs O(d log d) even though only K are needed; a heap of size K costs O(d log k).

BRUTE FORCE - PSEUDO-CODE: Q2: PLAIN RECURSION OVER THE MOVES

```
ways(r, c): if c is the last column: return 1 if r is the
bottom row else 0
return ways(r - 1, c + 1) + ways(r, c + 1) + ways(r + 1, c
 + 1), ignoring rows outside
    the grid
```

Time: O(3^n) | Space: O(n)

Why it is slow

Three calls per cell and the same cell is reached by many paths, so the call tree has about 3^n nodes. One column at a time with a rolling array is O(m * n).

BRUTE FORCE - PSEUDO-CODE: Q3: COUNT THE OVERLAP AT EVERY TIME AND ASSIGN GREEDILY WITHOUT A HEAP

```
cars = empty list of 'time each car becomes free'
for each rental sorted by pickup: scan all cars for one th
at is free by the pickup
    time
if found: reuse it (update its free time); else open a new
 car
answer = number of cars opened
```

Time: O(R^2) | Space: O(R)

Why it is slow

Each rental scans every car, so the cost is O(R * cars), up to O(R^2). A heap of free times finds the earliest-free car in O(log R).

OPTIMAL SOLUTION - Q1: TOP-K ACTIVE USERS WITH A SIZE-K HEAP

```python
def topKActiveUsers(messages, k):
    counts = Counter(messages)
    best = heapq.nsmallest(k, counts.items(), key=lambda k
v: (-kv[1], kv[0]))
    return [user for user, _ in best]
```

Time: O(n + d log k) for d distinct users | Space: O(d)

Why

Count messages in a hash map, then keep a min-heap of the best k (count, user). That beats sorting all d users when k is small. Ties break by smaller user id so the answer is deterministic. A second approach is bucket sort by count (O(n)), or quickselect on the counts (O(d) average).

OPTIMAL SOLUTION - Q2: COUNT CONSTRAINED PATHS WITH A ROLLING COLUMN

```python
def countPaths(m, n):
    ways = [0] * m
    ways[m - 1] = 1
    for _ in range(n - 1):
        nxt = [0] * m
        for r in range(m):
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total
        ways = nxt
    return ways[m - 1]
```

Time: O(m * n) | Space: O(m)

Why

Let ways[r] be the number of ways to be in row r of the current column. Moving one column right adds the three neighbours from the previous column, so only the previous column is ever needed. Answer is ways[m - 1] after n - 1 moves (exact integers in Python; take % 1_000_000_007 if a fixed width is needed).

OPTIMAL SOLUTION - Q3: MINIMUM CARS AND ONE VALID ASSIGNMENT

```
def minimumCars(rentals):
    """rentals: list of (rental_id, pickup, return). Retur
ns (cars, {rental_id: car})."""
    assignment, busy, cars = {}, [], 0
    for rental_id, pickup, ret in sorted(rentals, key=lamb
da r: r[1]):
        if busy and busy[0][0] <= pickup:
            _, car = heapq.heappop(busy)
        else:
            car = cars
            cars += 1
        assignment[rental_id] = car
        heapq.heappush(busy, (ret, car))
    return cars, assignment
```

Time: O(R log R) | Space: O(R)

Why

Sort rentals by pickup and keep a min-heap of (return time, car). If the earliest-returning car is free by the next pickup (return <= pickup, so back-to-back is allowed) reuse it, otherwise open a new car. The number of cars ever opened is the maximum number of overlapping rentals, which is the lower bound.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(m * n).

```
def paths_with_obstacles(m, n, blocked):
    ways = [0] * m
    if (m - 1, 0) not in blocked:
        ways[m - 1] = 1
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
            if (r, c) in blocked:
                continue
            nxt[r] = (
                ways[r] + (ways[r - 1] if r else 0) + (way
s[r + 1] if r + 1 < m else 0)
            )
        ways = nxt
    return ways[m - 1]
```

FULL QUESTION TEXT FROM THE SITE

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity.

Q1. Find top-K most active chat users

You are given a log of chat messages from a messaging application.

Input:

An array messages of length n , where each element messages[i] is the user_id (string or integer) of the sender of the i-th message.
An integer k (1 <= k <= number of distinct users).

Output:

Return a list of k user IDs corresponding to the users who have sent the most messages.

Details & requirements:

The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in messages).
If there are fewer than k distinct users, return all distinct users.
If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by user_id in ascending order.
Aim for an algorithm that is more efficient than sorting all users by count when k is much smaller than the number of distinct users.

Design and implement a function, for example:

List<UserId> topKActiveUsers(List<UserId> messages, int k)

Also explain:

At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
The time and space complexity of each approach.
Q2. Count paths in a 2D grid with constrained moves

You have a 2D grid with m rows and n columns. Index rows from top to bottom as 0..m-1 and columns from left to right as 0..n-1.

The start cell is the bottom-left corner: (m-1, 0) .
The end cell is the bottom-right corner: (m-1, n-1) .

From any cell (r, c), you may move only to the next column to the right, in one of up to three ways (if within bounds):

Right-up: (r-1, c+1)
Right: (r, c+1)
Right-down: (r+1, c+1)

You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

Given integers m and n , compute the total number of distinct paths from the start cell (m-1, 0) to the end cell (m-1, n-1) using only the allowed moves.

Requirements:

Implement a function such as long countPaths(int m, int n) .
The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct.
Q3. Minimum cars needed for rentals and assignment

You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

An array of R rental records. Each record has:
rental_id : a unique identifier (e.g., integer or string).
pickup_time : an integer start time.
return_time : an integer end time, with pickup_time < return_time .

Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time t and another starts at time t, the same car can serve both.

Tasks:

Capacity : Compute the minimum number of cars required so that all rentals can be fulfilled without time conflicts on any single car.
Assignment : Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

An integer C = minimal number of cars needed.
A mapping from each rental_id to a car_id in the range 1..C , such that no two rentals assigned to the same car_id overlap in time.

Requirements & clarifications:

Overlap definition: two rentals with intervals [s1, e1) and [s2, e2) overlap if their time intervals intersect with non-empty duration.
Using half-open intervals, if e1 == s2 they do not overlap.
Aim for an algorithm that is efficient for up to, say, 10^5 rental records.
Discuss a naive approach and then an optimized approach (for example, using a min-heap/priority queue keyed by the earliest return time to track currently in-use cars).
Analyze the time and space complexity of your final solution.

MEDIUM | IN APP LC 2402 | PracHub | Technical Screen | 149 of 196

Simulate room assignments for scheduled meetings

STUDY: TALK | Google frequency on LeetCode: all time 40, last 30 days 0 | book priority 85 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: your app's interview approach (LeetCode #2402)
LeetCode match: LC 2402 Meeting Rooms III (inferred from overview)
Original: https://prachub.com/interview-questions/simulate-room-assignments-for-scheduled-meetings

QUESTION

Overview: This question evaluates scheduling and event-simulation skills, including time-ordered processing, resource allocation with tie-breaking, and efficient tracking of room availability and usage counts using appropriate data structures.

APPROACH

Sort meetings by start, min-heap of free rooms (by index) and a heap of busy rooms (end time, room); assign the lowest free room, else wait for the earliest finish and delay.

HOW TO THINK ABOUT IT

Simulate in start order with two questions per meeting: which rooms are free now (lowest index first), and if none, which room frees up first (the meeting is delayed but keeps its length). Two heaps answer both quickly: one of free room ids, one of (end time, room id) for busy rooms.

Basis: LeetCode 2402, Meeting Rooms III.

BRUTE FORCE - PSEUDO-CODE

```
end[r] = time room r becomes free, for every room r
for each meeting in start order: free = the rooms with end
[r] <= start
if some room is free: use the lowest-numbered one and set
end[r] = meeting end
else: take the room with the earliest end (lowest number o
n ties) and push the meeting
    back by the wait
count meetings per room; return the busiest (lowest number
 on ties)
```

Time: O(M * N) | Space: O(N)

Why it is slow

Every meeting scans all N rooms, O(M * N). Two heaps (free rooms and busy rooms by end time) make each meeting O(log N).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #2402 MEETING ROOMS III)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
n meeting rooms. meetings[i]=[start,end]. Allocate smallest available room.
If none available, delay the meeting until earliest room frees up.
Return the room that held the most meetings (ties: smallest

index). Right?"

"A few quick questions:
Meetings must be sorted by start time before processing? Yes."

"Let me walk: n=2, meetings=[[0,10],[1,5],[2,7],[3,4]] -> room 0 held 2, room 1 held 2 -> 0 ok

PHASE 3 — OPTIMIZE

Say: "Two heaps: available rooms + in-use rooms — O(n log n).
Walk-through:
Sort meetings by start time.
available = min-heap of room numbers (initially all rooms: 0..n-1).
in_use = min-heap of (end_time, room_number).
count[i] = how many meetings held in room i.
For each meeting (start, end):
Free all rooms that ended by 'start': while in_use and in_use[0][0] <= start: free that room.
If available: assign room = heappop(available). Push (end, room) to in_use.
Else: wait for next available: (end_time, room) = heappop(in_use).
new_end = end_time + (end - start).
Push (new_end, room) to in_use.
count[room] += 1.
Return argmax(count). Time: O(n log n). Space: O(n)."
Key insight: two heaps (available rooms keyed by room number, in-use rooms keyed by end time) efficiently answer "which room finishes soonest?" and "which free room has lowest number?" — both in O(log n) per meeting.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _2402_MeetingRoomsIII:
    def mostBooked(self, n, meetings):
        meetings.sort()
        count = [0] * n
        available = list(range(n))
        heapq.heapify(available)
        busy = []
        for start, end in meetings:
            while busy and busy[0][0] <= start:
                finish_time, room = heapq.heappop(busy)
                heapq.heappush(available, room)
            if available:
                room = heapq.heappop(available)
                count[room] += 1
                heapq.heappush(busy, (end, room))
            else:
                finish_time, room = heapq.heappop(busy)
                count[room] += 1
                new_end = finish_time + (end - start)
                heapq.heappush(busy, (new_end, room))
        return count.index(max(count))
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace how the heap property guides which element comes next."

n=2, meetings=[[0,10],[1,5],[2,7],[3,4]]:
[0,10]->room0 (count[0]=1). [1,5]->room1 (count[1]=1). [2,7]->no room free: pop(5,1)->room1,count[1]=2,new_end=5+(7-2)=10.
[3,4]->no room free:
pop(10,0)->room0,count[0]=2,new_end=10+(4-3)=11.
count=[2,2]. index of max=0 ok

Delayed meetings inherit the freed room for their full duration from finish_time
WHY two heaps (free rooms by index, busy rooms by end time): greedily assign to lowest-index free room or wait for earliest-finishing busy room — O(n log k).

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(m log n). Space O(n).
Follow-up: Meeting Rooms II (#253) — minimum rooms needed; simpler heap.
Follow-up: if rooms have priorities, use a priority queue ordered by (priority, room_index).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

```python
def most_booked_by_time(n, meetings):
    free = list(range(n))
    heapq.heapify(free)
    busy, total = [], [0] * n
    for start, end in sorted(meetings):
        while busy and busy[0][0] <= start:
            heapq.heappush(free, heapq.heappop(busy)[1])
        if free:
            room, finish = heapq.heappop(free), end
        else:
            finish, room = heapq.heappop(busy)
            finish += end - start
        heapq.heappush(busy, (finish, room))
        total[room] += end - start
    return total.index(max(total))
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are given n meeting rooms labeled 0..n-1 and a list of meetings meetings, where each meeting is [start, end] with start < end. Meetings are processed in increasing order of start time.

Assign each meeting to a room using these rules:

If one or more rooms are free at start , assign the meeting to the free room with the smallest index .
If no rooms are free at start , the meeting must wait until a room becomes free. It is then scheduled in the room that becomes available earliest (break ties by smallest room index). If it waits, its duration stays the same , so if the original duration is end - start and the room becomes free at time t , the meeting runs on [t, t + (end - start)] .

Each time a meeting is assigned to a room (even after waiting), that room's meeting count increases by 1.

Task

Return the index of the room that hosted the most meetings. If there is a tie, return the smallest index.

Input/Output
Input: n (int), meetings (list of [start,end])
Output: an integer room index
Constraints (reasonable interview assumptions)
1 <= n <= 10^5
1 <= len(meetings) <= 10^5
Times fit in 32-bit signed integer; waiting can increase end times beyond original but still fits in 64-bit.

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Assign meetings to rooms with delays | PracHub | Technical Screen

https://prachub.com/interview-questions/assign-meetings-to-rooms-with-delays

---

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 150 of 196

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

STUDY: TYPE | Google frequency on LeetCode: all time 58.8, last 30 days 41.1 | book priority 20 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: your app's interview approach (LeetCode #347)
LeetCode match: LC 347 Top K Frequent Elements
Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE
Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

Examples:

```
Input:  chatLog = ["A says hi", "B says hi", "A says bye",
 "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

APPROACH

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

HOW TO THINK ABOUT IT

Count the speakers first; a log line is just a way to read the user id. Then you only need the K largest counts: a heap of size K does that without sorting all users. For logs spread over many machines the local top K are not enough on their own, so say what extra information you send.

Basis: LeetCode 347, Top K Frequent Elements (here the key is the speaker in each chat line).

BRUTE FORCE - PSEUDO-CODE

```
count messages per speaker
sort every distinct speaker by (count descending, name)
return the first K
```

Time: O(n + d log d) | Space: O(d)

Why it is slow

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #347 TOP K FREQUENT ELEMENTS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Return the k most frequent elements. The answer is unique (no ties at position k).
Return in any order. Must be better than O(n log n). Right?"

"A few quick questions:
Can k equal the number of unique elements? Yes. Values can be negative? Yes."

"Let me walk: nums=[1,1,1,2,2,3], k=2 -> 1(freq=3), 2(freq=2) -> [1,2] ok

PHASE 3 — OPTIMIZE

The bottleneck is the O(n log n) sort of all frequencies.
Bucket sort: create n+1 buckets indexed by frequency.
Each bucket at index f holds all numbers that appear exactly f

times.
Scan buckets from index n down to 0, collect until we have k elements.

Time: O(n). Space: O(n).
Does that make sense before I code it?"
Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _347_TopKFrequent:
    def topKFrequent(self, nums, k):
        from collections import Counter
        count = Counter(nums)
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in count.items():
            buckets[freq].append(num)
        result = []
        for freq in range(len(buckets) - 1, 0, -1):
            for num in buckets[freq]:
                result.append(num)
                if len(result) == k:
                    return result
        return result
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace topKFrequent([1,1,1,2,2,3], k=2)."
Count: {1:3,2:2,3:1}. Min-heap of size k.
Push (3,1). Heap=[(3,1)]. Push (2,2): heap=[(2,2),(3,1)]. Push (1,3): heap min=2>1 -> no push (heap full AND new<min -> skip).
Wait: push all then keep k largest.
Actually: for each (freq, val): if heap size < k -> push. Else if freq > heap[0][0] -> pop, push.
After (3,1): heap=[(3,1)]. After (2,2): heap=[(2,2),(3,1)]. After (1,3): 1<heap[0][0]=2 -> skip.
Return [val for freq,val in heap] = [1,2] ok
Bucket sort alt: buckets[freq].append(val). Iterate from high freq down, collect k. O(n).

WHY k-size min-heap: we evict the least frequent when the heap exceeds k — the surviving k elements ARE the top-k frequent, O(n log k) vs O(n log n) full sort.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Bucket sort: O(n) time, O(n) space. Heap approach: O(n log k) — worse but simpler.
Bucket sort is the optimal O(n) solution when frequencies <= n (always true here).
Follow-up: Top K Frequent Words (#692) — break frequency ties alphabetically.
Follow-up: streaming top-k — maintain a min-heap of size k as elements arrive.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Quickselect on the counts (average O(d))

```
def top_k_speakers_quickselect(chat_log, k):
    items = list(Counter(line.split(" ", 1)[0] for line in
 chat_log).items())
    items = [(-c, u) for u, c in items]  # smaller tuple =
 better
    lo, hi = 0, len(items) - 1
    k = min(k, len(items))
    while lo < hi:
        pivot = items[random.randint(lo, hi)]
        # three-way partition around the pivot
        lt, gt, i = lo, hi, lo
        while i <= gt:
            if items[i] < pivot:
                items[lt], items[i] = items[i], items[lt]
                lt += 1
                i += 1
            elif items[i] > pivot:
                items[gt], items[i] = items[i], items[gt]
                gt -= 1
            else:
                i += 1
        if k <= lt:
            hi = lt - 1
        elif k > gt + 1:
            lo = gt + 1
        else:
            break
    return [u for _, u in sorted(items[:k])]
```

Time: O(n + d) average, O(d^2) worst case | Space: O(d)

Why

Partition the (count, user) pairs around a random pivot until exactly the k largest sit on the left. Random pivots make the bad case unlikely; mention median-of-medians if a guaranteed O(d) bound is asked for.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. Cost: O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the

interviewer is grading the trade-off discussion.

The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

```
def misra_gries(stream, m):
    """Heavy hitters with m counters: any user with more t
han n / (m + 1) messages survives."""
    counters = {}
    for x in stream:
        if x in counters:
            counters[x] += 1
        elif len(counters) < m:
            counters[x] = 1
        else:
            for key in list(counters):
                counters[key] -= 1
                if counters[key] == 0:
                    del counters[key]
    return counters
```

NOTES FROM CANDIDATES

The interviewer typically wants a discussion of the three implementations + complexity trade-off, not just one working solution.

For the distributed follow-up, candidates that mention count-min sketch and topK per shard -> global merge got positive feedback.

The same family surfaces as "Top-K IPs", "Top-K chatters", and "Top-K words"; the technique is interchangeable.

A recent variant uses a gigabyte-scale IP list and explicitly requires exact time and space complexity analysis.

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).

Be able to state amortized vs worst-case time for each approach.

Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".

Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

MEDIUM | 1Point3Acres | onsite-coding | 151 of 196

Gate Entry / Exit Time Resolution with Tie-Break Rules

STUDY: TALK | Google frequency on LeetCode: all time 27.4, last 30 days 0 | book priority 119 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2534 Time Taken to Cross the Door

Frequency: Single report | Last asked: 2026-05-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gate-entry-exit-scheduler

QUESTION

Onsite coding round 2: given an array indexed by person id, each containing **`(timestamp, action in {enter, exit})`**, return per-person the actual time they passed through the gate after applying tie-break rules between simultaneous enters and exits.SWEsimulationalgorithmmediumFrequencySingle reportLast asked2026-05-04Stageonsite-coding

APPROACH

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.

Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).

Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

INTERVIEW-DAY PLAN

Clarify first

Ask for the exact tie-break rules in the four cases (previous second unused, entered, exited, and which wins), whether one person passes per second, and how ties between equal arrival times break (by id).

Brute force to say out loud

Simulate every second between the first and last timestamp. Works, but its cost grows with the size of the timestamps.

Optimal idea

Sort people by (time, id) and keep two queues. At each step apply the previous-state rule to choose a queue. When both queues are empty jump to the next arrival time and reset the previous state to idle if at least one second was skipped. Linear after the sort.

Dry-run these cases

Same-time enter and exit, an idle gap of exactly one second and of many seconds, all enters, all exits.

Likely follow-ups

Several gates, priority classes, or answering online.

Pitfalls

Forgetting to reset the state across an idle gap. Letting the clock go backward.

HOW TO THINK ABOUT IT

Write the rule as a small state machine first: the state is what the gate did in the previous second (entered, exited or idle), and the rule picks who goes next from that state. Then replace the second-by-second loop with a loop over people: when nobody is waiting jump the clock forward, and remember that an idle gap resets the state to idle.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumption: one person passes per second; if an enter and an exit compete, the door's previous-second use decides (enter if it was an enter, otherwise exit, and exit if the door was idle).

BRUTE FORCE - PSEUDO-CODE

```
t = 0; two queues: waiting to enter, waiting to exit; prev
 = what the gate did last
    second
each second: add the people who arrived; if both queues ar
e non-empty, pick by the
    tie-break rule using prev; else take whichever has som
eone
record the pass time for that person, set prev, t = t + 1
```

Time: O(T + n) where T is the last timestamp | Space: O(n)

Why it is slow

It runs through every second between the first and last timestamp, so a timestamp of 10^9 means a billion iterations. Jumping to the next arrival when the queues are empty makes it O(n log n).

OPTIMAL SOLUTION - EVENT-DRIVEN SIMULATION WITH TWO QUEUES

```
def gate_pass_times(events):
    n = len(events)
    order = sorted(range(n), key=lambda i: (events[i][0],
i))
    enter_q, exit_q = deque(), deque()
    res = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or enter_q or exit_q:
        if not enter_q and not exit_q:
            t = max(t, events[order[i]][0])
            if t > last_used + 1:
                prev = None
        while i < n and events[order[i]][0] <= t:
            p = order[i]
            (enter_q if events[p][1] == "enter" else exit_
q).append(p)
            i += 1
        if enter_q and exit_q:
            queue = enter_q if prev == "enter" else exit_q
        else:
            queue = enter_q or exit_q
        person = queue.popleft()
        res[person] = t
        prev = events[person][1]
        last_used = t
        t += 1
    return res
```

Time: O(n log n) for the sort, O(n) simulation | Space: O(n)

Why

Sort people by (arrival time, id) and keep one FIFO queue for waiting enters and one for exits. At each second pick from the queues using the previous-second state. When both queues are empty jump straight to the next arrival; if that jump leaves a gap of at least one idle second the previous state resets to 'none', which is the subtle part of the optimization.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why does the optimization need care across an empty gap?

If the door sits idle for even one second the previous state becomes 'none', which favours exits again. Jumping time forward without resetting the state would wrongly keep 'enter' priority from before the gap. Compare t with last_used + 1 to know whether the gap really was idle.

What is the complexity, and what if events come online?

Offline it is O(n log n) for the sort, then O(n) for the simulation, independent of how large the timestamps are. Online, keep the two queues and the previous-state variable and advance the clock on every arrival or tick; the work per person stays O(1).

Timestamps go up to 10^9.

That is why the event-driven loop jumps to the next arrival: its running time is O(n log n) and does not depend on the size of the timestamps. Remember to reset the previous state to idle across a gap.

The tie-break when the gate was idle differs in this version of the question.

Keep the simulation and make the idle winner a parameter: only the line that picks the queue changes. With 'exit first' it equals the standard rule; with 'enter first' an enter and an exit arriving together swap places.

```
def turnstile_times_priority(events, first="exit"):
    """Same simulation with a configurable idle priority (
some versions let entering go first when the door was idle
)."""
    n = len(events)
    order = sorted(range(n), key=lambda i: (events[i][0],
i))
    ent, ext, res = deque(), deque(), [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or ent or ext:
        if not ent and not ext:
            t = max(t, events[order[i]][0])
            if t > last_used + 1:
                prev = None
        while i < n and events[order[i]][0] <= t:
            (ent if events[order[i]][1] == "enter" else ex
t).append(order[i])
            i += 1
        if ent and ext:
            want = prev if prev is not None else first
            queue = ent if want == "enter" else ext
        else:
            queue = ent or ext
        person = queue.popleft()
        res[person], prev, last_used = t, events[person][1
], t
        t += 1
    return res


def gate_pass_times_priority(events, first="exit"):
    return turnstile_times_priority(events, first)
```

NOTES FROM CANDIDATES

The clean implementation uses two queues (pending enters, pending exits) plus a prev_state variable; at each tick peek both queues, apply the four rules, advance, and update prev_state.
A naive tick-by-tick simulation walks every integer time unit from min to max timestamp; the expected optimization is to jump directly to the next event timestamp in the input, dropping complexity to O(n).
The optimization has to carry the "prior moment" state across empty gaps correctly (a 100-tick gap still counts as prev_state = none).
Walk through one or two examples on the whiteboard before coding — interviewers in this family prefer candidates who fully verbalize the precedence rules before writing.

PREPARATION

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

MEDIUM | PracHub | Technical Screen | 152 of 196

Find a Horizontal Cut That Bisects Rectangle Area

STUDY: TALK | Google frequency on LeetCode: all time 8.1, last 30 days 0 | book priority 155 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: the interview site's own solution
Related in your app (a neighbouring problem, not this exact question): LeetCode #850 Rectangle Area II
LeetCode match: No direct match; related LC 850 Rectangle Area II (inferred from overview)
Original: https://prachub.com/coding-questions/find-a-horizontal-cut-that-bisects-rectangle-area

QUESTION

Overview: Solve a computational-geometry problem that asks for the horizontal line dividing the combined area of many disjoint rectangles in half. The exercise emphasizes numerical precision, large coordinate ranges, performance at 200,000 rectangles, and careful treatment of partially intersected shapes.

APPROACH

Compute total area and the covered width per y-slab via a sweep; walk slabs accumulating area until half, then interpolate inside the slab. Use exact arithmetic for precision.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the rectangles are disjoint (the overview says so), the coordinate range, whether the answer must be exact (use fractions) and which y to return if a gap makes many cuts valid (the lowest).

Brute force to say out loud

Binary search on y, summing the rectangle area below y each time. O(n log precision), but floating point error is a risk.

Optimal idea

Because the rectangles are disjoint, the area below y is a piecewise linear function whose slope is the sum of the widths of the rectangles spanning y. Create events at each rectangle's bottom (slope increases by its width) and top (slope decreases), sort by y, and walk the slabs accumulating area. When the next slab would pass half of the total, solve inside it: y = y0 + (half - accumulated) / slope, using integer or Fraction arithmetic. O(n log n) for 200,000 rectangles.

Dry-run these cases

One rectangle (the middle), two rectangles separated by a gap (slope zero there), the half exactly at a rectangle boundary, huge coordinates.

Likely follow-ups

Overlapping rectangles (LeetCode 850, needs a segment tree over x), or a vertical cut.

Pitfalls

Floating point error when comparing areas. Not handling slabs with slope zero.

HOW TO THINK ABOUT IT

Area below a line y = c is a piecewise linear function of c, and between two consecutive rectangle edges its slope is just the total width of rectangles crossing that height (they are disjoint, so widths add). Walk the slabs between events, accumulate area, and when half the area falls inside a slab solve a linear equation. Use exact fractions so the answer is exact.

Basis: Written; a sweep over y events (compare LeetCode 850, Rectangle Area II, which is the harder overlapping version). Assumed rule: axis-aligned rectangles (x1, y1, x2, y2) that do not overlap; return the lowest y with half the total area below it.

BRUTE FORCE - PSEUDO-CODE

```
total = the sum of all rectangle areas
lo = lowest edge, hi = highest edge; repeat many times: mi
d = (lo + hi) / 2
    if the area below mid is at least half the total: hi =
 mid, else lo = mid
```

Time: O(R * iterations) | Space: O(1)

Why it is slow

Each step sums over all rectangles and the answer is only as precise as floating point allows. A sweep over sorted edges with exact fractions finds the slab and the cut directly in O(R log R).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def horizontal_bisector(rectangles):
    # Every cake contributes a horizontal band [bottom, bo
ttom + height] of
    # constant width. Sweeping the bottom/top edges bottom
-up keeps the total
    # width of the currently open bands, so the area below
 the sweep line grows
    # linearly inside each band and the exact answer needs
 one division.
    events = []
    total = 0.0
    for rect in rectangles:
        bottom = float(rect[1])
        width = float(rect[2])
        height = float(rect[3])
        total += width * height
        events.append((bottom, width))
        events.append((bottom + height, -width))
    events.sort(key=lambda event: event[0])

    target = total / 2.0
    accumulated = 0.0
    active_width = 0.0
    index = 0
    count = len(events)
    while index < count:
        boundary = events[index][0]
```

```
        # Apply every edge at this height before measuring
 the band above it.
        while index < count and events[index][0] == bounda
ry:
            active_width += events[index][1]
            index += 1
        if index >= count:
            break
        if active_width > 0.0:
            band = (events[index][0] - boundary) * active_
width
            if accumulated + band >= target:
                return boundary + (target - accumulated) /
 active_width
            accumulated += band
    return events[count - 1][0]
```

Time: O(n log n) | Space: O(n)

Explanation

Only horizontal edges matter: between two consecutive y values at which some cake starts or ends, the set of cakes the sweep line crosses is fixed, so the combined width is constant and the area below the line grows linearly.

The reference turns every cake into two events, (bottom, +width) and (bottom + height, -width), sorts them by height, and sweeps upward. At each distinct height it first applies every event at that height - three cakes sharing a bottom edge must all open before the band above it is measured - then computes the area of the band up to the next event as (next - boundary) * active_width. It keeps a running total of area already passed; the first band whose area would carry the running total to or past half the grand total is the band containing the answer, and the exact answer inside it is boundary + (target - accumulated) / active_width.

Bands of zero active width (the empty gaps between distant cakes) are skipped, which is what makes coordinates spread across 2 * 10^9 cost nothing: the sweep visits at most 2n edges regardless of how far apart they are. The problem's uniqueness guarantee means the halfway point never falls inside such a gap, so the division is always by a positive width.

Areas are accumulated as double in all four languages. A single cake can have area 10^18 and the total can reach 2 * 10^23, which no 64-bit integer holds; the grading tolerance is on k, whose relative error stays far under 1e-6 because the final division rescales the accumulated area by the same active width that produced it.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by p * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

```
def cut_at_ratio(rects, p):
    total = sum((x2 - x1) * (y2 - y1) for x1, y1, x2, y2 i
n rects)
    target = Fraction(total) * p
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    ys = sorted(events)
    slope, acc = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        slope += events[y0]
        gain = slope * (y1 - y0)
        if slope and acc + gain >= target:
            return y0 + (target - acc) / slope
        acc += gain
    return Fraction(ys[-1])
```

FUNCTION AND TEST CASES

Signature

```
def horizontal_bisector(rectangles: list[list[float]]) ->
float:
```

Sample call

```
horizontal_bisector([[0, 0, 4, 2]])
```

Test cases

Input	Expected
`([[0, 0, 4, 2]],)`	`1.0`
`([[0, 0, 2, 4], [10, 1, 2, 2]],)`	`2.0`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Find where to cut cake into equal areas | PracHub | Technical Screen

https://prachub.com/coding-questions/find-where-to-cut-cake-into-equal-areas

MEDIUM | PracHub | Technical Screen | 153 of 196

Min boards to cover roof holes

STUDY: TALK | Google frequency on LeetCode: all time 19.6, last 30 days 0 | book priority 137 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #452 Minimum Number of Arrows to Burst Balloons

LeetCode match: No direct match; related LC 1326 Minimum Number of Taps / LC 452 (inferred from overview)

Original: https://prachub.com/coding-questions/min-boards-to-cover-roof-holes

QUESTION

Overview: This question evaluates competency in combinatorial optimization and problem modeling for coverage constraints, testing algorithmic reasoning, complexity analysis, and efficient representation of grid-based data.

APPROACH

Greedy interval cover: sort hole positions, place a board starting at the first uncovered hole, skip all holes within its length, repeat.

INTERVIEW-DAY PLAN

Clarify first

My copy has only the overview, so ask: are holes points on a line or cells in a grid, what is the board length, can boards be placed in either orientation, and must each board be axis aligned. The 1D case is a classic interval cover.

Brute force to say out loud

Try every placement of boards. Exponential.

Optimal idea

1D with boards of length L: sort the holes, place a board starting at the first uncovered hole so it covers [h, h + L - 1], skip every hole inside it, repeat. The count is the answer, and the exchange argument shows starting at the leftmost uncovered hole is optimal. O(n log n). For a grid with horizontal boards only, do this per row. If boards can be placed in either orientation covering rows or columns, it becomes a minimum vertex cover in a bipartite graph (Konig's theorem), solved with matching.

Dry-run these cases

No holes (0), one hole, holes spread more than L apart, adjacent holes, holes at the same position.

Likely follow-ups

Boards of different lengths, or a limit on the number of boards.

Pitfalls

Starting a board in the middle of an uncovered gap. Not sorting the holes first.

HOW TO THINK ABOUT IT

Cover the leftmost hole first: any board that covers it can be slid right until it starts exactly at that hole without uncovering anything to its right, so start a board at the leftmost uncovered hole and skip everything it covers. Repeating that is optimal because starting later never covers more.

Basis: Written; interval covering (compare LeetCode 452 and 1326). Assumed rule (confirm it): holes are integer positions on a line, a board of length L covers positions p .. p + L - 1, boards may be placed anywhere; minimise the number of boards.

BRUTE FORCE - PSEUDO-CODE

```
best[0] = 0; for i from 1 to the number of holes:
    best[i] = the smallest best[j] + 1 over every j such t
hat one board of length
        L covers holes j .. i - 1
```

Time: O(n^2) | Space: O(n)

Why it is slow

Every i tries every j, O(n^2). Starting a board at the leftmost uncovered hole and skipping everything it covers is optimal and O(n log n) including the sort.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(roof):
    from collections import deque

    if not roof or not roof[0]:
        return 0

    m = len(roof)
    n = len(roof[0])

    # Build bipartite graph: rows -> columns where roof[r]
[c] == 0 (holes)
    adj = [[] for _ in range(m)]
    hole_exists = False
    for r in range(m):
        for c in range(n):
            if roof[r][c] == 0:
                adj[r].append(c)
                hole_exists = True

    if not hole_exists:
        return 0

    pairU = [-1] * m  # row -> matched column
    pairV = [-1] * n  # column -> matched row
    dist = [-1] * m

    def bfs():
        q = deque()
        found = False
        for u in range(m):
            if pairU[u] == -1:
                dist[u] = 0
                q.append(u)
            else:
                dist[u] = -1
        while q:
```

```python
            u = q.popleft()
            du = dist[u]
            for v in adj[u]:
                w = pairV[v]
                if w == -1:
```

```
                    found = True
                else:
                    if dist[w] == -1:
                        dist[w] = du + 1
                        q.append(w)
        return found

    def dfs_iter(start_u, next_idx):
        stack_rows = [start_u]
        path_cols = []  # columns leading into each subseq
uent row on the path
        # Initialize next indices
        # next_idx already provided and zeroed outside per
 BFS layer
        while stack_rows:
            u = stack_rows[-1]
            idx = next_idx[u]
            if idx >= len(adj[u]):
                dist[u] = -1
                if len(stack_rows) >= 2:
                    path_cols.pop()
                stack_rows.pop()
                continue
            v = adj[u][idx]
            next_idx[u] = idx + 1
            w = pairV[v]
            if w == -1:
                # Augment along path: ... -> u - v (free)
                pairU[u] = v
                pairV[v] = u
                # Flip along stored path in reverse
                for k in range(len(path_cols) - 1, -1, -1)
:
```

```
                    row_k = stack_rows[k]
                    col_k = path_cols[k]
                    pairU[row_k] = col_k
                    pairV[col_k] = row_k
                return True
            else:
                if dist[w] == dist[u] + 1:
                    stack_rows.append(w)
                    path_cols.append(v)
                # else: try next neighbor
        return False

    matching = 0
    while bfs():
        next_idx = [0] * m
        for u in range(m):
            if pairU[u] == -1 and adj[u]:
                if dfs_iter(u, next_idx):
                    matching += 1
    return matching
```

Time: O(m*n + H*sqrt(m+n)), where H is the number of holes | Space: O(H + m + n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

```
def min_boards(holes, L):
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
        if p > covered_to:
            count += 1
            covered_to = p + L - 1
    return count


def min_length_for_k_boards(holes, k):
    h = sorted(set(holes))
    if not h:
        return 0
    lo, hi = 1, h[-1] - h[0] + 1
    while lo < hi:
        mid = (lo + hi) // 2
        if min_boards(h, mid) <= k:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

FUNCTION AND TEST CASES

Signature

```
def solution(roof: list[list[int]]) -> int:
SAMPLE INPUT
[[1,0,1],[0,1,1]]
```

Test cases

Input	Expected
[[1,0,1],[0,1,1]]	2
[[1,1],[1,1]]	0

MEDIUM | PracHub | Technical Screen | 154 of 196

Solve several coding interview problems

STUDY: TALK | Google frequency on LeetCode: all time 38.3, last 30 days 0 | book priority 88 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #160 Intersection of Two Linked Lists

LeetCode match: No direct match (multi-part); related LC 415 Add Strings, LC 160 Intersection of Two Linked Lists (inferred from overview)

Original: https://prachub.com/coding-questions/solve-several-coding-interview-problems

QUESTION

Overview: This set of problems evaluates proficiency in string processing and combinatorial parsing, arbitrary-precision arithmetic using string representations, pointer/ancestor-chain intersection detection, and computational geometry for overlap counting, emphasizing algorithmic design, correctness, and performance.

APPROACH

Multiple small tasks: string parsing, big-number string arithmetic, ancestor-chain intersection (hash set or depth alignment), etc. Solve each independently.

INTERVIEW-DAY PLAN

Clarify first

My copy lists four small tasks: string parsing with combinatorial counting, adding two big integers given as strings, finding where two ancestor chains intersect, and counting overlaps in a geometry setting. Confirm each task's exact input and output, because the overview is the only statement I have.

Brute force to say out loud

Do each directly: parse and count with recursion, add digit by digit with a carry, compare ancestor lists, and test every pair of shapes.

Optimal idea

Big integers: walk both digit strings from the right with a carry and build the result (O(max(m, n))), as in LeetCode 415. Ancestor chains: put one chain in a set and walk the other, or align by depth and step both up until they meet, as in LeetCode 160; O(n) time, O(1) space for the aligned version. Overlap counting: a sweep line over start and end events, taking the running maximum or counting pairs depending on the question. String parsing and counting: define the state and use memoized recursion.

Dry-run these cases

Big integers with different lengths, a final carry, leading zeros. Chains that never meet, meet at the head, are identical. Overlaps that touch at a boundary.

Likely follow-ups

Subtraction or multiplication of big integers, lowest common ancestor with parent pointers, or counting overlapping rectangles.

Pitfalls

Starting to code before the four statements are confirmed. Forgetting the final carry.

HOW TO THINK ABOUT IT

A set of unrelated small tasks: split them, restate each contract, and solve the smallest one first. Big numbers: add digit by digit with a carry. Two chains that meet: walk both and switch to the other head at the end so the lengths cancel. Overlap counting: sort start and end events and keep a running count.

Basis: Three of the small tasks the overview lists, with the standard versions (my copy has only the overview): adding two big integers given as strings (LeetCode 415), intersection of two chains such as ancestors or linked lists (LeetCode 160), and the largest number of overlapping intervals.

BRUTE FORCE - PSEUDO-CODE: BIG NUMBERS: CONVERT TO INTEGERS AND ADD

```
return str(int(a) + int(b))
```

Time: O(n^2) for the conversions | Space: O(n)

Why it is slow

Relies on arbitrary-precision integers (not in every language) and the string-to-integer conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

BRUTE FORCE - PSEUDO-CODE: CHAINS THAT MEET: STORE ONE CHAIN IN A SET

```
put every node of the first chain in a set
walk the second chain; return the first node that is in th
e set
```

Time: O(m + n) | Space: O(m)

Why it is slow

O(m + n) time but O(m) extra memory. Two pointers that switch to the other chain's head at the end meet at the shared node with O(1) memory.

BRUTE FORCE - PSEUDO-CODE: OVERLAP: TEST EVERY TIME POINT

```
for every integer time t: count the intervals with start <
= t < end
answer = the largest count
```

Time: O(range * n) | Space: O(1)

Why it is slow

Depends on the length of the time range, O(range * n). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION - BIG NUMBERS: DIGIT BY DIGIT FROM THE RIGHT WITH A CARRY

```
def addStrings(a, b):
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (int(a[i]) if i >= 0 else 0) + (in
t(b[j]) if j >= 0 else 0)
        out.append(str(total % 10))
        carry = total // 10
        i, j = i - 1, j - 1
    return "".join(out[::-1])
```

Time: O(max(m, n)) | Space: O(max(m, n))

Why

Walk both strings from the right, add the two digits and the carry, append total % 10, carry total // 10; after the loop add a final carry digit if needed and reverse. Never convert the whole string to an integer.

OPTIMAL SOLUTION - CHAINS THAT MEET: TWO POINTERS THAT SWITCH HEADS AT THE END

```
class LNode:
    def __init__(self, val, nxt=None):
        self.val, self.next = val, nxt


def getIntersectionNode(a, b):
    p, q = a, b
    while p is not q:
        p = p.next if p else b
        q = q.next if q else a
    return p
```

Time: O(m + n) | Space: O(1)

Why

Walk p along a and q along b; when a pointer reaches the end, restart it at the other chain's head. Both pointers then travel m + n steps in total, so they meet at the first shared node (or both become None together).

OPTIMAL SOLUTION - OVERLAP: SWEEP OVER SORTED START AND END EVENTS

```
def max_overlap(intervals):
    events = sorted([(s, 1) for s, _ in intervals] + [(e,
-1) for _, e in intervals])
    cur = best = 0
    for _, d in events:
        cur += d
        best = max(best, cur)
    return best
```

Time: O(n log n) | Space: O(n)

Why

Create +1 at each start and -1 at each end, sort the events (ends before starts at the same time for half-open intervals), and track the running count; its maximum is the answer.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Ancestors in a tree with parent pointers, not a linked list.

The same two-pointer trick works on parent chains (LeetCode 1650); with depths known, lift the deeper node first and then move both up together.

Add two big numbers stored as linked lists.

If the lists store the least significant digit first, walk both together with the carry (LeetCode 2); if most significant first, reverse them or use stacks.

Count overlapping rectangles instead of intervals.

Sweep over x with a data structure over y (a segment tree with range add and max) to find the largest point depth in O(n log n); the pairwise count is O(n^2) unless you use a sweep with an interval tree.

Add two big numbers stored as linked lists (least significant digit first).

Walk both lists together with a carry and build the result list as you go; no reversal needed. O(max(m, n)) time. If the most significant digit comes first, reverse the lists or use stacks.

```
class _L:
    def __init__(self, val, nxt=None):
        self.val, self.next = val, nxt


def add_reversed_lists(a, b):
    dummy = tail = _L(0)
    carry = 0
    while a or b or carry:
        total = carry + (a.val if a else 0) + (b.val if b
else 0)
        tail.next = _L(total % 10)
        tail, carry = tail.next, total // 10
        a, b = (a.next if a else None), (b.next if b else
None)
    return dummy.next
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> list[str]:
```

Sample call

```
solution('.-')
```

Test cases

Input	Expected
`('.-',)`	`['A', 'ET']`
`('...',)`	`['EEE', 'EI', 'IE', 'S']`

MEDIUM | PracHub | Technical Screen | 155 of 196

Choose optimal guesses for green-only Wordle

STUDY: TALK | Google frequency on LeetCode: all time 42.2, last 30 days 0 | book priority 77 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: No direct match; related LC 843 Guess the Word (inferred from overview)

Original: https://prachub.com/interview-questions/choose-optimal-guesses-for-green-only-wordle

QUESTION

Overview: This question evaluates probabilistic reasoning, expected-value optimization, information partitioning, and algorithmic efficiency in selecting guesses under partial (green-only) feedback; it belongs to the Coding & Algorithms domain, specifically search and probabilistic modeling, and emphasizes practical algorithm design and implementation rather than purely theoretical concepts. It is commonly asked to assess a candidate's ability to model uncertainty with weighted distributions, minimize the expected remaining search space, and manage time/space trade-offs for inputs up to several thousand words.

APPROACH

Filter candidates by green-position constraints, then pick the guess that maximizes expected information (partition sizes) over the remaining words.

INTERVIEW-DAY PLAN

Clarify first

Ask what feedback exists (only green: letter in the right position), whether answers are equally likely or weighted, whether guesses must come from the candidate list, and the objective (minimize the expected size of the remaining set).

Brute force to say out loud

For each candidate guess and each possible answer compute the feedback and count the survivors. O(W^2 * L).

Optimal idea

A guess g splits the candidates by its pattern against each answer, a bitmask of which positions are green. If groups have sizes s_1 ... s_k, the expected number of survivors is the sum of s_i^2 divided by W (weights replace sizes with weight sums). Pick the guess minimizing it. To speed up, store for each (position, letter) a bitset of candidate words so groups come from bit operations instead of pairwise comparisons.

Dry-run these cases

Two candidates (any guess), words that differ in one position, a guess that is itself the answer (pattern all green), a single candidate.

Likely follow-ups

Full Wordle feedback with yellow, multiple rounds, or an adaptive strategy.

Pitfalls

Minimizing the number of groups instead of the expected survivors. Recomputing the filter from scratch each round.

HOW TO THINK ABOUT IT

A guess splits the candidates into groups by which positions turned green. If the groups have sizes s_1 ... s_k out of W candidates, the expected size of what remains is the sum of s_i squared divided by W (a candidate falls in a group of size s with probability s / W, and then s remain). Pick the guess with the smallest value. To make it fast, store for each (position, letter) the set of words with that letter there as a bitset, so the groups come from a few AND operations.

Basis: Written. Assumed rules (confirm them): words of equal length, the answer is uniformly random among the candidates, a guess must be a candidate, feedback marks only the positions where the guess letter equals the answer letter (green). Goal: minimise the expected number of candidates left.

BRUTE FORCE - PSEUDO-CODE

```
for every candidate guess: for every candidate answer: com
pute the green pattern
    (which positions match)
group the answers by pattern; score = (sum of group sizes
squared) / number of
    candidates
pick the guess with the smallest score
```

Time: O(W^2 * L) | Space: O(W)

Why it is slow

Compares every pair of words, O(W^2 * L). Bitsets per (position, letter) split the candidates into pattern groups with a few AND operations.

OPTIMAL SOLUTION - BITSETS PER (POSITION, LETTER), SPLITTING GROUPS WITH AND

```
def best_guess(words):
    W = len(words)
    bits = [defaultdict(int) for _ in range(len(words[0]))
]
    for i, w in enumerate(words):
        for pos, ch in enumerate(w):
            bits[pos][ch] |= 1 << i
    best = None
    for g in words:
        groups = [(1 << W) - 1]
        for pos, ch in enumerate(g):
            mask = bits[pos][ch]
            nxt = []
            for grp in groups:
                a, b = grp & mask, grp & ~mask
                if a:
                    nxt.append(a)
                if b:
                    nxt.append(b)
            groups = nxt
        score = Fraction(sum(grp.bit_count() ** 2 for grp
in groups), W)
        if best is None or (score, g) < best:
            best = (score, g)
    return best[1]
```

Time: O(W * L * groups) with bit-parallel work per group | Space: O(W * L)

Why

Index the words: bits[pos][letter] is an integer whose bit i is set when word i has that letter at pos. For a guess, start with one group containing all words and split it position by position into 'green here' (AND with bits[pos][guess letter]) and 'not green here' (the remainder). Group sizes are popcounts; the score is sum of s^2 over W. Choose the minimum score, ties by the smaller word.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Play a whole game, using the feedback after each guess.

Keep the candidates that would have produced exactly the observed green pattern (filter_candidates), then choose the best guess among the remaining ones. Each round at least removes the guess itself, so the game ends.

```
def best_guess(words):
    W = len(words)
    bits = [defaultdict(int) for _ in range(len(words[0]))
]
    for i, w in enumerate(words):
        for pos, ch in enumerate(w):
            bits[pos][ch] |= 1 << i
    best = None
    for g in words:
        groups = [(1 << W) - 1]
        for pos, ch in enumerate(g):
            mask = bits[pos][ch]
            nxt = []
            for grp in groups:
                a, b = grp & mask, grp & ~mask
```

```
                if a:
                    nxt.append(a)
                if b:
                    nxt.append(b)
            groups = nxt
        score = Fraction(sum(grp.bit_count() ** 2 for grp
in groups), W)
        if best is None or (score, g) < best:
            best = (score, g)
    return best[1]


def filter_candidates(words, guess, mask):
    return [w for w in words if _green_mask(guess, w) == t
uple(mask)]


def _green_mask(guess, answer):
    return tuple(g == a for g, a in zip(guess, answer))


def play(words, answer):
    """Follow-up: play the whole game by always picking th
e best guess among the remaining candidates."""
    guesses, cand = [], list(words)
    while True:
        g = best_guess(cand) if len(cand) > 1 else cand[0]
        guesses.append(g)
        if g == answer:
            return guesses
        cand = filter_candidates(cand, g, _green_mask(g, a
nswer))
```

Answers are not equally likely (weights).
Replace group sizes by group weights: score = sum(weight(group)^2) / total weight. The grouping step is unchanged; use sums of weights instead of popcounts.

Real Wordle also gives yellow feedback.
The pattern becomes three-valued per position (green, yellow, grey) with the usual duplicate-letter rules, so the pattern space is 3^L; bitsets still work if you precompute patterns, but the computation is O(W^2) pattern evaluations for a full table.

FULL QUESTION TEXT FROM THE SITE

Problem

You are playing a simplified 5-letter word guessing game.

There is a hidden secret word of length 5 .
You have a dictionary of valid 5-letter words. Each word may also have an associated frequency/weight representing how likely it is to be the secret.
When you guess a 5-letter word, the game returns only “green” information (correct letter in the correct position). There is no information about letters that exist in the word but are in the wrong position (“yellow”), and no explicit info about letters that are absent.
Feedback definition

For a guess g and secret s, define the feedback as a 5-bit mask m where:

m[i] = 1 iff g[i] == s[i]
otherwise m[i] = 0

Example: g="crane", s="cared" -> mask is 1 0 0 0 0 (only position 0 matches).

Task

Given:

words : a list of N valid 5-letter words
weight[w] (optional): a positive number for each word (if omitted, assume uniform)
candidates : the current set of possible secret words consistent with feedback so far (initially all words)

Write an algorithm/function to choose the next guess that is optimal under the following objective:

Choose a guess word g (typically from words) that minimizes the expected size of the remaining candidate set after receiving the green-only mask, where the expectation is taken over the secret word drawn from candidates according to weight .

Formally, if C is the current candidate set and C_m(g) is the subset of candidates that would produce mask m for guess g, choose g that minimizes:

E
[
|
C
m
a
s
k
(
g
)
|
]
=
sum
m
in
{
0
,
1
}
5
P
(
m
a
s
k
=
m
)
*
|
C
m
(
g

)
|
E[|C
mask

(g)|]=sum
min{0,1}
5

P(mask=m)*|C
m

(g)|

where
P
(
m
a
s
k
=
m
)
P(mask=m) is induced by the weighted distribution over secrets in C.

Input/Output expectations
Input sizes: 1 <= N <= 5,000 (you may propose optimizations if needed), word length is always 5.
Output: the selected next guess word.
Follow-ups (optional)
How would you incorporate non-uniform word frequencies?
How would you scale if N is very large (e.g., 100k)?
How would you change the objective to minimize expected number of guesses instead of expected remaining candidates?

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Wordle-Style 5-Letter Minimum-Guess Strategy | 1Point3Acres | phone-screen, onsite-coding | asked 2025-12-15 | Single report
https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy
Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

Preparation: Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

MEDIUM | Exponent | 156 of 196

Write SQL code to publish the Fibonacci series.

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 175 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: written and tested here for this exact question
LeetCode match: No LC match (SQL) (inferred from short statement)
Original: https://www.aced.io/questions/2779/sql-code-fibonacci-series

QUESTION

Write SQL code to publish the Fibonacci series.

APPROACH

Use a recursive CTE producing (n, a, b) rows and select a; limit the row count with a WHERE clause.

INTERVIEW-DAY PLAN

Clarify first
Ask which database dialect, how many terms, and whether the series starts at 0 or 1.

Brute force to say out loud
A loop in a stored procedure. Works but is procedural.

Optimal idea
Use a recursive common table expression holding (n, a, b): the anchor row is (1, 0, 1) and the recursive step is (n + 1, b, a + b) while n is below the limit. Select a. Watch integer overflow for large n; use a wider type or stop at the type limit.

Dry-run these cases
Limit 1, limit 2, limit 10, a limit large enough to overflow a 64-bit integer.

Likely follow-ups
Return only even terms or the sum of the first n terms.

Pitfalls
Forgetting the termination condition, which makes the query recurse until the database stops it.

HOW TO THINK ABOUT IT

A sequence that depends on the previous rows is a recursive CTE: one anchor row, one recursive step, and a stopping condition in the step. Carry the two previous values in each row (a, b) so the next row is (b, a + b); a row-per-term table is built in one statement with no procedural loop.

Basis: Written; recursive common table expressions. Run and checked on SQLite; the same query works on PostgreSQL and MySQL 8 (SQL Server omits the word RECURSIVE).

BRUTE FORCE - PSEUDO-CODE

```
a = 0, b = 1; repeat n times: insert a as a new row, then
(a, b) = (b, a + b)
```

Time: O(n) statements | Space: O(n)

Why it is slow

One statement and one round trip per term, and a loop outside the query language. A recursive CTE produces all rows in one set-based statement.

OPTIMAL SOLUTION - RECURSIVE CTE CARRYING (N, A, B)

```
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < :limit
)
SELECT a AS fibonacci FROM fib ORDER BY n;
```

Time: O(n) | Space: O(1) working set (O(n) result)

Why

The anchor row is (1, 0, 1): the first term is 0 and the next pair is (0, 1). Each recursive step produces (n + 1, b, a + b) while n is below the limit, so the term in column a is the Fibonacci number. Without the WHERE n < limit the recursion never stops. Integer overflow appears around the 93rd term for 64-bit integers.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Only the even terms, or the sum of the first n terms.

Keep the CTE and filter or aggregate in the outer query: WHERE a % 2 = 0 for the even terms, SUM(a) for the total. The recursion stays the same.

Which database?

PostgreSQL and MySQL 8 use WITH RECURSIVE; SQL Server drops the word RECURSIVE and may need a CAST so the column type is wide enough (bigint) in the anchor row; Oracle accepts WITH without RECURSIVE too. State the dialect assumption first.

Large n overflows.

Use a wider type (BIGINT, NUMERIC, or DECIMAL) in the anchor row so the column type is wide enough from the start; arbitrary-size values need a language with big integers. Cost: O(n) rows regardless of the type; a wider type only costs more bytes per row.

Use it from application code with a parameter.

Bind the limit as a query parameter (never build the SQL by string concatenation) and fetch the single column. The recursion lives inside the database, so there is one round trip regardless of n.

```
def fibonacci_via_sql(limit):
    conn = sqlite3.connect(":memory:")
    q = """WITH RECURSIVE fib(n, a, b) AS (SELECT 1, 0, 1
UNION ALL SELECT n + 1, b, a + b FROM fib WHERE n < :limit
)
           SELECT a FROM fib ORDER BY n"""
    return [r[0] for r in conn.execute(q, {"limit": limit}
)]
```

MEDIUM | Exponent | 157 of 196

★ TOP 75: rank 41 in the Medium and Hard priority list

★ Sort a nearly sorted (or K sorted) array.

STUDY: TYPE | Google frequency on LeetCode: all time 55.2, last 30 days 33 | book priority 23 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #215 Kth Largest Element in an Array, LeetCode #23 Merge k Sorted Lists

LeetCode match: No direct match; related LC 215 Kth Largest Element, LC 23 (inferred from short statement)

Original: https://www.aced.io/questions/5392/k-sorted-array

QUESTION

Sort a nearly sorted (or K sorted) array.

APPROACH

Min-heap of size k+1: push the first k+1 elements, then for each next element pop the smallest into output and push the next. O(n log k).

INTERVIEW-DAY PLAN

Clarify first

Ask what K means (each element is at most K positions from its sorted place), whether in place, and whether K can be 0.

Brute force to say out loud

Library sort, O(n log n). It works but ignores the structure.

Optimal idea

Min-heap of size K + 1: push the first K + 1 elements, then repeatedly pop the minimum into the output and push the next element. The minimum of any K + 1 consecutive window is always the next sorted element. O(n log K) time, O(K) space.

Dry-run these cases

K = 0 (already sorted), K = 1, K at least n, duplicates, a reverse-sorted array with K equal n - 1.

Likely follow-ups

A very large K (use the library sort), or sorting a stream with bounded memory.

Pitfalls

A heap of size K instead of K + 1 (misses one element). Not draining the heap at the end.

HOW TO THINK ABOUT IT

If each element is at most K places from where it belongs, then the smallest element of the whole array must be among the first K + 1 elements. A min-heap holding a sliding window of K + 1 elements always has the next output on top, so you never need to sort the whole array.

Basis: GeeksforGeeks 'sort a nearly sorted array'; each element is at most K positions from its sorted position.

BRUTE FORCE - PSEUDO-CODE

```
return sorted(array)
```

Time: O(n log n) | Space: O(n)

Why it is slow

O(n log n) regardless of K. Each element is at most K places from its sorted position, so a min-heap holding a window of K + 1 elements emits the sorted order in O(n log K).

OPTIMAL SOLUTION - MIN-HEAP OF SIZE K + 1

```
def sort_nearly(a, k):
    heap = list(a[: k + 1])
    heapq.heapify(heap)
    out = []
    for x in a[k + 1 :]:
        out.append(heapq.heapreplace(heap, x))
    while heap:
        out.append(heapq.heappop(heap))
    return out
```

Time: O(n log K) | Space: O(K)

Why

Push the first K + 1 elements. Then for each remaining element, pop the smallest into the output and push the new element. At the end pop the heap empty. The smallest of any K + 1 consecutive pending elements is the next element of the sorted order.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

K is not given. How do you find it?

Sort a copy with the original index as a tie-break and take the largest |original index - sorted index|; that is the smallest valid K. O(n log n), so only worth it if you will sort many similar arrays.

```
def smallest_k(a):
    """Find the smallest K for which the array is K-sorted
: the largest distance between an element's position and i
ts sorted position."""
    return _max_displacement(a)
```

The data is a stream you cannot hold in memory.

The heap of K + 1 elements is already a streaming algorithm: it emits the next sorted element after each arrival once it holds K + 1 elements. Memory O(K).

K is close to n.

Then O(n log K) approaches O(n log n) and the heap has no advantage; use the library sort. State the crossover.

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

MEDIUM | 1Point3Acres | onsite-coding | 158 of 196

Streaming Points: Emit K Within Distance

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 178 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Process a stream of float positions. Whenever three points satisfy the distance condition, output and remove those three points; a follow-up generalizes from 3 points to K points. The important ambiguity is whether the condition means a span of length **d** or pairwise distance within **d**.SWEdata-structuresortingsliding-window mediumambiguous-specFrequencySingle reportLast asked2026-06-23Stageonsite-coding

APPROACH

Practice range queries over sorted points with insertion and deletion.

Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.

Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the condition means all K points fit in a span of length d or every pair is within d (on a line these are equivalent), whether to emit as soon as a group exists, and how to choose among overlapping groups.

Brute force to say out loud

Keep a list, and after each point sort it and scan for a group. O(n log n) per point.

Optimal idea

Keep points in a sorted list (bisect.insort). After inserting x only windows containing x can be new, so look at the K-wide windows of sorted neighbors around its index; if the largest minus the smallest in a window is at most d, emit and remove those K points. O(log n) to locate plus the window size.

Dry-run these cases

Exactly d apart, duplicates, points arriving out of order, two overlapping candidate groups, K = 1.

Likely follow-ups

General K (the same code), 2D points, or a time-based expiration.

Pitfalls

Using epsilon-free float comparison blindly. Checking only the K points directly before x.

HOW TO THINK ABOUT IT

On a line, K points are all within distance d exactly when the largest minus the smallest is at most d, so only K consecutive points in sorted order can form a group. Keep the points sorted and look only where something changed: a window that contains the new point, or the seam left by removed points.

Basis: Written. Assumed rules (confirm them): after each new point, while some K consecutive points (in sorted order) span at most d, emit the lowest such group and remove it; on a line 'all within d' and 'span at most d' are the same condition.

BRUTE FORCE - PSEUDO-CODE

```
on every new point: add it, sort all points
while some K consecutive points have max - min <= d: emit
the lowest such group and
    remove it
```

Time: O(n log n) per point | Space: O(n)

Why it is slow

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION - SORTED LIST, CHECK ONLY WINDOWS TOUCHING THE CHANGE

```
class PointStream:
    def __init__(self, k, d):
        self.k, self.d, self.pts = k, d, []

    def _first_window(self, pos):
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.pts) - self.k)
        for i in range(lo, hi + 1):
            if self.pts[i + self.k - 1] - self.pts[i] <= s
elf.d:
                return i
        return None

    def add(self, x):
        insort(self.pts, x)
        out, pos = [], bisect_left(self.pts, x)
        while True:
            i = self._first_window(pos)
            if i is None:
                return out
            out.append(self.pts[i : i + self.k])
            del self.pts[i : i + self.k]
            pos = i
```

Time: O(log n + K) per check, plus list shifting | Space: O(n)

Why

Insert with bisect. Because no window existed before, a new window must contain the new point, so test the K windows that include its position. After emitting a group, the points on both sides of the gap become neighbours, so test the windows around that seam too. A balanced tree (sorted container) makes the insert and delete O(log n).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

```
def all_pairs_within(points, d):
    return all(abs(a - b) <= d for a, b in itertools.combi
nations(points, 2))
```

NOTES FROM CANDIDATES

A balanced ordered set / TreeSet-style structure is a natural fit because each new point only needs nearby neighbors.
Be careful with floating-point comparison and epsilon policy.
A wrong clarification can lead to checking subset(target, target + d) when the interviewer expects a wider candidate range around the target. Confirm the predicate with a tiny numeric example before coding.

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

MEDIUM | 1Point3Acres | onsite-coding | 159 of 196

Piano Hand Movement Grouping

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 184 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

Examples:

With notes = [1, 3, 5, 7, 9, 11] and k = 4, the stated output is 2.

Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] -> 1, without stating the reach parameter.

APPROACH

Practice turning the physical reach rule into an explicit interval invariant before writing code.

Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

INTERVIEW-DAY PLAN

Clarify first

The statement is ambiguous, so ask: does the hand cover a fixed interval of keys [p, p + k], is the interval fixed until the hand lifts, does the first placement count as a move, and do notes arrive in the given order. Notice the examples are not all consistent, so ask which one is the reference.

Brute force to say out loud

Try every starting position for each group and take the minimum. Exponential or quadratic.

Optimal idea

Greedy one-pass scan: start a group at the first note with the hand covering [note, note + k]. For each next note, if it is inside the interval keep going; if not, start a new group at that note. The answer is the number of groups (or groups minus one if the first placement is free). Group boundaries come from the same scan. O(n).

Dry-run these cases

A single note, the example [1, 3, 5, 7, 9, 11] with k = 4, notes going down and then up, k = 0.

Likely follow-ups

A hand that can slide, two hands, or return the group boundaries.

Pitfalls

Counting the initial placement inconsistently. Assuming notes are sorted.

HOW TO THINK ABOUT IT

Turn the physical rule into an invariant: the current group is valid while (largest note - smallest note) <= k, because you can place the hand at the smallest. The first note that breaks it must start a new group, and starting later never helps, so a single greedy pass is optimal. Ask whether the first placement counts as a move.

Basis: Written; interval grouping, like LeetCode 452 and 1326. Assumed rule (confirm it, the examples disagree): a hand placed on key p covers p to p + k inclusive and stays put until it is lifted; the notes come in order; the answer is the minimum number of placements, and the number of lifts is one fewer.

BRUTE FORCE - PSEUDO-CODE

```
best[0] = 0; for j from 1 to n:
    best[j] = the smallest best[i] + 1 over every i < j wh
ere max - min of
        notes[i..j) is at most k
```

Time: O(n^2) | Space: O(n)

Why it is slow

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION - GREEDY SCAN, TRACKING THE GROUP'S SMALLEST AND LARGEST NOTE

```python
def groups(notes, k):
    if not notes:
        return 0
    count, lo, hi = 1, notes[0], notes[0]
    for x in notes[1:]:
        nlo, nhi = min(lo, x), max(hi, x)
        if nhi - nlo <= k:
            lo, hi = nlo, nhi
        else:
            count += 1
            lo = hi = x
    return count
```

Time: O(n) | Space: O(1)

Why

Keep lo and hi of the current group. Extend while hi - lo stays within k, otherwise close the group and start a new one at this note. Cutting as late as possible cannot be worse because any later cut leaves a suffix that is a subset of what an earlier cut would have to cover.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

```python
def group_boundaries(notes, k):
    out, start = [], 0
    lo = hi = notes[0] if notes else 0
    for i, x in enumerate(notes):
        nlo, nhi = min(lo, x), max(hi, x)
        if nhi - nlo <= k:
            lo, hi = nlo, nhi
        else:
```

```
            out.append(
                (start, i - 1, lo)
            )  # (first note index, last note index, key t
o place the hand on)
            start, lo, hi = i, x, x
    if notes:
        out.append((start, len(notes) - 1, lo))
    return out
```

Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

NOTES FROM CANDIDATES

Clarify whether a placement is fixed until the hand lifts, whether the covered interval may slide without counting as a lift, and whether the initial placement counts. These details materially change the answer.

The six-note example's explanation says a placement at 7 covers keys 7–11 but still counts another lift for 9 and 11. Treat that as an ambiguity to resolve before coding rather than silently assuming the output convention.

Maintain the feasible placement interval for the current group. The follow-up printout should reuse the same grouping state instead of duplicating the movement logic.

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.

Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 160 of 196

★ TOP 75: rank 36 in the Medium and Hard priority list

★ Design a logger that returns top-K messages

STUDY: TYPE | Google frequency on LeetCode: all time 51.2, last 30 days 33 | book priority 30 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter, LeetCode #692 Top K Frequent Words

LeetCode match: LC 359 Logger Rate Limiter + LC 692 Top K Frequent Words (inferred from overview)

Original: https://prachub.com/coding-questions/design-a-logger-that-returns-top-k-messages

QUESTION

Overview: This question evaluates skills in designing in-memory data structures and algorithms for frequency counting and interval querying, including extracting top-K elements with a heap, within the Coding & Algorithms domain.

APPROACH

Hash map message -> count (plus a deque of timestamps for interval queries); top-K with a size-k heap, ties broken lexicographically.

INTERVIEW-DAY PLAN

Clarify first

My copy has only the overview, so ask what the API is: log(timestamp, message), top-K over all time or over an interval, whether duplicates within a time gap are suppressed (the LeetCode 359 rate limiter), and how ties break (lexicographically, for LeetCode 692).

Brute force to say out loud

Store every entry in a list and sort or scan on each query. O(n log n) per query.

Optimal idea

A dict message to count (and, for interval queries, a dict message to a list of timestamps kept in order so a time window is two binary searches). Top-K uses a heap of size K with the key (count descending, message ascending): heapq.nsmallest(k, items, key=(-count, message)). The rate limiter part is a dict message to last printed time, allowing a message when the gap is at least 10 seconds. O(d log K) per top-K query.

Dry-run these cases

Fewer than K distinct messages, ties on count, the same message repeated inside the rate-limit window, an empty logger.

Likely follow-ups

Streaming top-K with bounded memory (Space-Saving), or evicting old entries.

Pitfalls

Not specifying the tie rule. Using the full sort when only K items are needed.

HOW TO THINK ABOUT IT

Store enough to answer both questions without scanning everything. A dict from message to its list of timestamps (appended in time order) answers 'how many in [start, end]' with two binary searches, and a heap of size K over (count, message) gives the top K. Say how the logger handles repeated messages and what 'top' means with ties.

Basis: Written (compare LeetCode 359 Logger Rate Limiter and 692 Top K Frequent Words). Assumed API (confirm it): log(timestamp, message); top_k(k, start, end) over an optional time window; ties by message alphabetically.

BRUTE FORCE - PSEUDO-CODE

```
keep all (timestamp, message) pairs
top_k(k, start, end): count the messages whose timestamp i
s in the window; sort by
    (count descending, message); return the first k
```

Time: O(n + d log d) per query | Space: O(n)

Why it is slow

Every query reads the whole log, O(n + d log d). Per-message sorted timestamps with binary search answer a window count in O(log n), and a heap picks the top k.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(logs, queries):
    from collections import defaultdict
    from bisect import bisect_left, bisect_right
    import heapq

    timestamps_by_message = defaultdict(list)
    for timestamp, message in logs:
        timestamps_by_message[message].append(timestamp)

    message_items = list(timestamps_by_message.items())
    answers = []

    for start_time, end_time, k in queries:
        if k <= 0 or start_time > end_time:
            answers.append([])
            continue

        heap = []
        for message, timestamps in message_items:
            left = bisect_left(timestamps, start_time)
            right = bisect_right(timestamps, end_time)
            count = right - left
            if count > 0:
                heap.append((-count, message))

        heapq.heapify(heap)
        current = []
        for _ in range(min(k, len(heap))):
            current.append(heapq.heappop(heap)[1])
        answers.append(current)

    return answers
```

Time: O(n + q * d * log n + total_extracted * log d), where n is the number of logs, q is the number of queries, and d is the number of distinct messages | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

```
class RateLimitedLogger:
    """LeetCode 359: a message may be printed again only a
fter `gap` seconds."""

    def __init__(self, gap=10):
        self.gap, self.last = gap, {}

    def should_print(self, ts, msg):
        if msg in self.last and ts - self.last[msg] < self
.gap:
            return False
        self.last[msg] = ts
        return True
```

Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. Cost: O(1) amortized to drop expired timestamps; memory O(messages inside the window).

Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

FUNCTION AND TEST CASES

Signature

```
def solution(logs: list[tuple[int, str]], queries: list[li
st[int]]) -> list[list[str]]:
```

Sample call

```
solution([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3
, 2), (1, 10, 3)])
```

Test cases

Input	Expected
`([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])`	`[['A', 'B'], ['A', 'B', 'C']]`
`([(1, 'dog'), (2, 'cat'), (2, 'dog'), (4, 'cat'), (5, 'ant')], [(1, 4, 2)])`	`[['cat', 'dog']]`

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 161 of 196

Find top/bottom-k words in list or stream

STUDY: TALK | Google frequency on LeetCode: all time 32.6, last 30 days 0 | book priority 106 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (+ stream variant)

Original: https://prachub.com/coding-questions/find-top-bottom-k-words-in-list-or-stream

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated
Examples
Input: (['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
Expected Output: [['i', 'love'], ['coding', 'leetcode']]
Explanation: **`i`** and **`love`** both appear twice, so lexicographical order gives **`['i', 'love']`**. **`coding`** and **`leetcode`** both appear once, so lexicographical order gives **`['coding', 'leetcode']`**.
Input: (['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]
Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.
Hints
Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).
Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.
Constraints
1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested
Examples
Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]
Expected Output: [['apple', 'banana'], ['banana', 'apple']]
Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.
Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]
Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]
Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.
Hints
Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

APPROACH

List: counts + heaps (min-heap size k for top, max-heap size k for bottom). Stream: keep counts in a map and maintain heaps with lazy updates, or approximate with Space-Saving / Count-Min when memory is bounded.

INTERVIEW-DAY PLAN

Clarify first

Ask how ties break, whether top and bottom are over distinct words, what topK(k) returns for k larger than the number of words, and whether memory is bounded for the stream.

Brute force to say out loud

Count words, sort all distinct words by (count, word), take the first and last K. O(d log d).

Optimal idea

List version: Counter, then heapq.nlargest and nsmallest with the key (count, word) in the right directions, O(n + d log K). Stream version: ingest increments a dict in O(1); topK and bottomK compute on demand with a heap in O(d log K). If queries are frequent, keep words grouped by count in buckets (a linked list of count buckets like an LFU cache) so topK walks from the highest bucket in O(K). If memory is bounded, the Space-Saving or Count-Min sketch approximates the top K; bottom-K cannot be approximated well, so say so.

Dry-run these cases

Fewer words than K, all counts equal (ties decide everything), one word repeated, an empty stream.

Likely follow-ups

Windowed counts (expire old words), or distributed counting with a merge step.

Pitfalls

Mixing up the direction for the bottom-K heap. Missing a deterministic tie-break.

HOW TO THINK ABOUT IT

Count first; then you only need the K best and K worst, which a heap does without sorting everything. For a stream, ingesting is just a counter increment, and the answer is computed from the counts on demand. If memory is bounded, only heavy hitters (top-K) can be approximated; the rarest words cannot.

Basis: LeetCode 692 (Top K Frequent Words) and its bottom-K counterpart, plus a streaming interface. Ties break alphabetically.

BRUTE FORCE - PSEUDO-CODE

```
count the words; sort the distinct words by (count, word)
most frequent K = the first K by count descending; least f
requent K = the first K by
    count ascending
```

Time: O(d log d) | Space: O(d)

Why it is slow

Sorts all d distinct words, O(d log d). Two heaps of size K are O(d log k); a stream only needs the counts, and a Space-Saving sketch bounds memory for the top K.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(words, k):
    from collections import Counter

    if k <= 0 or not words:
        return [[], []]

    counts = Counter(words)
    limit = min(k, len(counts))

    top_k = sorted(counts.keys(), key=lambda word: (-count
s[word], word))[:limit]
    bottom_k = sorted(counts.keys(), key=lambda word: (cou
nts[word], word))[:limit]

    return [top_k, bottom_k]
```

Time: O(n + m log m), where n is the number of words and m is the number of distinct words | Space: O(m)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / m times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

```
class SpaceSaving:
    """Approximate heavy hitters with m counters: any word
 with frequency above n / m is guaranteed to be tracked.""
"

    def __init__(self, m):
        self.m, self.c = m, {}

    def ingest(self, w):
        if w in self.c:
            self.c[w] += 1
        elif len(self.c) < self.m:
            self.c[w] = 1
        else:
            victim = min(self.c, key=lambda x: (self.c[x],
 x))
            self.c[w] = self.c.pop(victim) + 1

    def top(self, k):
        return heapq.nsmallest(k, self.c, key=lambda w: (-
self.c[w], w))
```

topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], k: int) -> list[list[str]]:
```

Sample call

```
solution(['i', 'love', 'leetcode', 'i', 'love', 'coding'],
 2)
```

Test cases

Input	Expected
`(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)`	`[['i', 'love'], ['coding', 'leetcode']]`
`(['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)`	`[['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]`

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 162 of 196

Implement iterator merging lists with filtering

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 141 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #281 Zigzag Iterator

LeetCode match: LC 281 Zigzag Iterator (+ filter/dedupe) (inferred from overview)

Original: https://prachub.com/coding-questions/implement-iterator-merging-lists-with-filtering

QUESTION

Overview: This question evaluates iterator/generator design, sequence merging, stateful filtering and deduplication, checking a candidate's ability to maintain input ordering while excluding blocked IDs and preventing duplicate yields.

APPROACH

Keep one pointer per list and a lookahead 'next valid' value for hasNext/next; skip filtered and duplicate values; or merge with a heap if sorted.

INTERVIEW-DAY PLAN

Clarify first

Ask how the lists are merged: round robin (like LeetCode 281 Zigzag Iterator), one after the other, or by sorted value; what is filtered (a blocked set of ids); whether duplicates must be removed across lists; and what next() does when the iterator is exhausted.

Brute force to say out loud

Build one merged list up front with the filter applied. Simple but not lazy and not memory friendly.

Optimal idea

A lazy iterator with one pointer per list and a precomputed next valid value (lookahead). hasNext checks the lookahead; next returns it and then advances to the following valid element: skip blocked ids and anything already in a seen set. For sorted merges use a heap of (value, list index, position). Each element is touched once, O(total) overall.

Dry-run these cases

Empty lists, all values blocked, duplicates across lists, calling next after the end, a list that becomes empty mid-way.

Likely follow-ups

Make the iterator re-iterable, support remove, or bound memory for the seen set when ids are unbounded (Bloom filter).

Pitfalls

Computing the lookahead lazily at the wrong moment so hasNext skips or repeats a value.

HOW TO THINK ABOUT IT

An iterator does its work lazily: hasNext() must be able to answer without consuming. So keep a lookahead, the next valid value already computed, and advance to the one after it when next() is called. Filtering and de-duplication live in the 'find the next valid value' step so both hasNext and next stay trivial.

Basis: LeetCode 281, Zigzag Iterator, with a blocked-id filter and no duplicate output. Assumed rules (confirm them): lists are read round robin; ids in the blocked set are skipped; each id is yielded at most once.

BRUTE FORCE - PSEUDO-CODE

```
merged = interleave the lists round robin; drop blocked id
s and ids already seen
the iterator just walks merged
```

Time: O(total) | Space: O(total)

Why it is slow

It reads everything up front and keeps it all in memory, so it is not lazy and never works on a long or unbounded source. A lookahead value per iterator computes only the next valid item.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(favorites, photos, blockedIds):
    blocked = set(blockedIds)
    seen = set()
    out = []

    def ident(item):
        return item.get("id") if isinstance(item, dict) el
se item

    for seq in (favorites, photos):
        for item in seq:
            x = ident(item)
            if x not in blocked and x not in seen:
                seen.add(x)
                out.append(x)
    return out
```

Time: O(n+m) | Space: O(unique ids)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

```
def sorted_merge_unique(lists, blocked=()):
    blocked, last = set(blocked), object()
    for v in heapq.merge(*lists):
        if v not in blocked and v != last:
            yield v
            last = v
```

Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

```
def concatenated_unique(lists, blocked=()):
    seen, blocked = set(), set(blocked)
    for lst in lists:
        for v in lst:
            if v not in blocked and v not in seen:
                seen.add(v)
                yield v
```

The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. Cost: O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false

suppressions.

FUNCTION AND TEST CASES

Sample call

```
solution([3, 1, 2, 3], [2, 4, 1, 5], {4})
```

Test cases

Input	Expected
`([3, 1, 2, 3], [2, 4, 1, 5], {4})`	`[3, 1, 2, 5]`
`([], [1, 1], set())`	`[1]`

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 163 of 196

Implement a Reusable Locker Allocation System

STUDY: TALK | Google frequency on LeetCode: all time 8.1, last 30 days 0 | book priority 153 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: the interview site's own solution
LeetCode match: LC 1845 Seat Reservation Manager
Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), s
tore("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remov
e and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports
validation and pickup.
```

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.
Constraints
1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.
Examples
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
Explanation: This is the source example; the released smallest locker is reused.
Input: (5, [])
Expected Output: []
Explanation: An empty batch produces no per-operation results.
Hints
Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

APPROACH

Min-heap of released locker numbers plus a counter for the next never-used number; store pops the smallest, pickup pushes back. Track the last successful store separately. O(log n) per op.

INTERVIEW-DAY PLAN

Clarify first

Ask what store and pickup return, what happens when a customer stores twice, when no locker is free, and whether numbers are reused smallest first.

Brute force to say out loud

Scan lockers 1..n for the first free one on every store: O(n) per operation.

Optimal idea

Keep a min-heap of released locker numbers and a counter next_new for lockers never used. store: if the customer already has one return -1; take the heap minimum if the heap is not empty, otherwise next_new if it is at most n, else return -1; remember customer to locker in a dict. pickup: free the customer's locker by pushing it on the heap. O(log n) per operation, and memory depends on used lockers, not n.

Dry-run these cases

Pickup then store reuses the smallest freed number. Double store by one customer. Store when full. Pickup with no locker.

Likely follow-ups

n up to a billion (still fine because of the counter), thread safety with a lock, or lockers with sizes.

Pitfalls

Initializing the heap with all n numbers. Not removing the customer record on pickup.

HOW TO THINK ABOUT IT

'Smallest free number' with numbers coming back means a min-heap of returned numbers. Numbers that were never handed out do not need to be stored: keep a counter for the next unused number and use the heap first. That keeps memory proportional to the number of lockers in use, not to n.

Basis: Written; the smallest-available-number pattern (LeetCode 1845 Seat Reservation Manager, 2336 Smallest Number in Infinite Set).

BRUTE FORCE - PSEUDO-CODE

```
store(c): if c already has a locker: -1; else find the fir
st free locker from 1 upward
    by scanning, assign it
pickup(c): free c's locker
```

Time: O(n) per operation | Space: O(n)

Why it is slow

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
import heapq


def process_locker_operations(n, operations):
    available = list(range(1, n + 1))
    heapq.heapify(available)
    assigned = {}
    last_stored = -1
    answers = []

    for operation in operations:
        if operation[0] == "store":
            customer = operation[1]
            if customer in assigned or not available:
                answers.append(-1)
            else:
                locker = heapq.heappop(available)
                assigned[customer] = locker
                last_stored = locker
                answers.append(locker)
        elif operation[0] == "pickup":
            customer = operation[1]
            if customer not in assigned:
                answers.append(-1)
            else:
                locker = assigned.pop(customer)
                heapq.heappush(available, locker)
                answers.append(locker)
        else:
            answers.append(last_stored)

    return answers
```

Time: O((n + o) log n) overall; each store or pickup is O(log n) and last is O(1) | Space: O(n)

Explanation

Maintain all currently available locker numbers in a min-heap and all current customer assignments in a hash map. A successful store removes the minimum locker, records it for the customer, and updates last. A successful pickup deletes the assignment and reinserts its locker into the heap. Failed operations leave both allocation state and last unchanged, and last simply returns the saved successful-store number. Consequently each allocation always chooses the smallest free locker and every release becomes immediately reusable.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. Cost: O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

```python
class Lockers:
    def __init__(self, n):
        self.n, self.next_new, self.free, self.of = n, 1,
[], {}

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            i = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            i, self.next_new = self.next_new, self.next_ne
w + 1
        else:
            return -1
        self.of[customer] = i
        return i

    def pickup(self, customer):
        i = self.of.pop(customer, None)
        if i is None:
            return -1
        heapq.heappush(self.free, i)
        return i


class LockersThreadSafe(Lockers):
    def __init__(self, n):
        super().__init__(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)

    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)
```

Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the

same.

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[list[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 164 of 196

Streaming Median (LC 295) with Follow-ups

STUDY: TYPE | Google frequency on LeetCode: all time 50.4, last 30 days 0 | book priority 61 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: your app's interview approach (LeetCode #295)
LeetCode match: LC 295 Find Median from Data Stream
Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

Clarify first

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

Brute force to say out loud

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

Optimal idea

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

Dry-run these cases

One element, two elements, duplicates, all equal, alternating very large and very small values.

Likely follow-ups

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

Pitfalls

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE - PSEUDO-CODE

```
keep all values in a sorted list
add(x): insert x at its sorted position (binary search to find it)
median: the middle element, or the average of the two middle elements
```

Time: O(n) per insert (list shift), O(1) median | Space: O(n)

Why it is slow

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #295 FIND MEDIAN FROM DATA STREAM)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

> Let me make sure I understand the problem correctly. Design addNum(int) and findMedian()->double. Median of a growing stream. Right?"

"A few quick questions:
For even total elements, median = average of two middle values? Yes.
addNum can be called up to 5*10^4 times — both ops need to be fast."

"Let me walk: addNum(1),addNum(2),findMedian()->1.5.
addNum(3),findMedian()->2.0 ok

PHASE 3 — OPTIMIZE

The bottleneck is the O(n) insert for the sorted list.
Two heaps — max-heap for lower half, min-heap for upper half: max_heap.top() <= min_heap.top() always.
Sizes differ by at most 1.
addNum: push to max_heap (negated), rebalance.
findMedian: if equal sizes -> average of tops; else -> top of larger heap.

Time: O(log n) addNum, O(1) findMedian. Space: O(n)."
Key insight: two heaps partition numbers into lower and upper halves — the max-heap top and min-heap top are always the two median candidates, rebalanced in O(log n) after each insertion.

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import heapq
class _295_FindMedianFromDataStream:
    def __init__(self):
        self._lo = []
        self._hi = []

    def addNum(self, num):
        heapq.heappush(self._lo, -num)
        heapq.heappush(self._hi, -heapq.heappop(self._lo
))
        if len(self._hi) > len(self._lo):
            heapq.heappush(self._lo, -heapq.heappop(self
._hi))

    def findMedian(self):
        if len(self._lo) > len(self._hi):
            return float(-self._lo[0])
        return (-self._lo[0] + self._hi[0]) / 2.0
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace MedianFinder with addNum(1),addNum(2),findMedian(),addNum(3),findMedian()."
Two heaps: max_heap (left half), min_heap (right half). left>=right by 0 or 1.
addNum(1): push to max_heap. max_heap=[-1] (negated).
len(max)=1>len(min)+1=1? No. median=1.0.
addNum(2): push to max_heap [-1,-2]. pop max -> 2 -> push to min_heap=[2]. Balanced.
findMedian(): sizes equal -> (max_heap top + min_heap top)/2 = (1+2)/2=1.5 ok
addNum(3): push to max_heap [-1,-3]. max_heap top=3>min top=2 -> pop 3, push to min=[2,3]. max=[-1]. Balance: len(min)=2>len(max)=1+1 -> pop min, push to max. max=[-2,-1].
min=[3].
findMedian(): len(max)=2>len(min)=1 -> return max top=2 ok

WHY two heaps: lower half in max-heap, upper half in min-heap; median is always at one or both tops — O(1) query, O(log n) insert.

PHASE 6 — COMPLEXITY & FOLLOW-UP

addNum O(log n). findMedian O(1). Space O(n).
Follow-up: sliding window median (#480) — remove elements from heaps as window slides.
Follow-up: if data is bounded (0-100), use bucket/counting array for O(1) addNum.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Small value domain: counting array

```
class BoundedMedian:
    def __init__(self, max_value):
        self.counts = [0] * (max_value + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value

    def median(self):
        if self.n % 2:
            return float(self.kth(self.n // 2 + 1))
        return (self.kth(self.n // 2) + self.kth(self.n //
 2 + 1)) / 2
```

Time: O(1) add, O(V) median | Space: O(V)

Why

If values are integers in [0, V), keep a count per value. Adding is O(1); the median scans the counts once (or maintains a moving pointer to make it amortized O(1)).

Any k-th smallest: Fenwick tree over the value range

```
class KthSmallestStream:
    def __init__(self, max_value):
        self.size = max_value + 1
        self.tree = [0] * (self.size + 1)
        self.n = 0
        self.top = 1 << self.size.bit_length()

    def add(self, x):
        self.n += 1
        i = x + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
        pos, step = 0, self.top
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
```

```
            step >>= 1
        return pos  # zero-based value
```

Time: O(log V) add and query | Space: O(V)

Why

A Fenwick (binary indexed) tree stores counts per value. The k-th smallest is found by descending the tree with binary lifting, so any rank, not just the median, costs O(log V). Compress coordinates if V is huge.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

```
class SlidingWindowMedian:
    """Median of the last k values, with a sorted window (
bisect for insert and remove)."""

    def __init__(self, k):
        self.k, self.queue, self.sorted = k, deque(), []

    def add(self, x):
        self.queue.append(x)
        bisect.insort(self.sorted, x)
        if len(self.queue) > self.k:
            old = self.queue.popleft()
            del self.sorted[bisect.bisect_left(self.sorted
, old)]
        n = len(self.sorted)
        return (
            float(self.sorted[n // 2])
            if n % 2
            else (self.sorted[n // 2 - 1] + self.sorted[n
// 2]) / 2
        )
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value. For an even-sized stream, average the two heap roots.

Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.

For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.

The base problem required complete working code plus a dry run and complexity analysis.

The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.

Sketch a bounded-domain alternative and state its update and query costs.

Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

MEDIUM | IN APP LC 703 | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 165 of 196

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

STUDY: TALK | Google frequency on LeetCode: all time 32.6, last 30 days 0 | book priority 107 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

Examples:

```
insert(3)
insert(3)
insert(2)
findLargest(0) -> 3
findLargest(1) -> 3
findLargest(2) -> 2
```

APPROACH

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.

Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).

Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

HOW TO THINK ABOUT IT

If k can change from query to query you need every value, in a structure that supports rank queries; if k is fixed you only need the k largest, and a min-heap of size k gives the answer at its root. Ask which of the two it is: it decides everything.

Basis: LeetCode 703, Kth Largest Element in a Stream, generalized so k is passed to each query (0-indexed in the examples).

BRUTE FORCE - PSEUDO-CODE

```
insert(x): append x to a list
findLargest(k): sort the list in descending order and retu
rn the element at index k
```

Time: O(n log n) per query | Space: O(n)

Why it is slow

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #703 KTH LARGEST ELEMENT IN A STREAM)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Design a class that maintains the kth largest element as numbers are continuously added via add(val). Right?"

"A few quick questions:
Initial nums array may have fewer than k elements? Yes, but it's guaranteed there will be at least k elements after enough add() calls before we need a valid answer."

"Let me walk: k=3, nums=[4,5,8,2] -> add(3)->4, add(5)->5, add(10)->5, add(9)->8 ok

PHASE 3 — OPTIMIZE

Say: "Min-heap of size k — O(log k) per add. Walk-through:
Init: heap = sorted k largest vals from nums (using a min-heap of size k).
Push all; pop until len == k.
add(val): heap.push(val). If len(heap) > k: heap.pop(). Return heap[0] (min = k-th largest).
WHY min-heap of size k: heap[0] = smallest of the k largest = k-th largest overall.
WHY pop when size > k: maintain exactly k elements.
WHY heap[0] is the answer: the minimum of the top-k is the k-th largest.
Time: O(log k) per add. Space: O(k).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import heapq
class _703_KthLargestElementInAStream:
    def __init__(self, k, nums):
        self.k = k
        self.heap = nums
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace how the heap property guides which element comes next."

k=3, nums=[4,5,8,2]: heapify->[2,4,5,8]. trim to size 3: pop 2 -> heap=[4,5,8].
add(3): push 3->[3,4,5,8], size4>3->pop min(3)->heap=[4,5,8]. return heap[0]=4 ok
add(5): push->[4,5,5,8], pop min(4)->[5,5,8]. return 5 ok

The min-heap of size k always has the kth largest at its root after trimming
WHY k-size min-heap: heap minimum IS the kth largest at all times; add and evict maintains exactly k elements with O(log k) per insertion.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(log k) per add, O(n log k) init. Space O(k).
Follow-up: Kth Largest Element in an Array (#215) — one-shot version, quickselect is faster O(n) avg.
Follow-up: Find Median from Data Stream (#295) — two-heap technique for streaming median.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Fixed k: min-heap of the k largest

```python
class KthLargest:
    def __init__(self, k, nums=()):
        self.k, self.heap = k, []
        for x in nums:
            self.add(x)

    def add(self, x):
        heapq.heappush(self.heap, x)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0] if len(self.heap) == self.k el
se None
```

Time: O(log k) per insert, O(1) query | Space: O(k)

Why

If k never changes, only the k largest values can ever matter. A min-heap of size k keeps them and the root is the k-th largest. Pop only when the heap would exceed k.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

```
class StreamKth:
    def __init__(self):
        self.a = []

    def insert(self, x):
        insort(self.a, x)

    def findLargest(self, k):
        return self.a[-1 - k] if 0 <= k < len(self.a) else
 None


class StreamKthWithDelete(StreamKth):
    def delete(self, x):
        i = bisect.bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False
```

NOTES FROM CANDIDATES

An order-statistics structure (size-augmented BST / balanced tree / skiplist) gives O(log n) for both ops.
An order-statistics Fenwick tree over a coordinate-compressed value range gives O(log V).
A naive sorted array gives O(log n) find but O(n) insert; mention as a baseline.
Two heaps (the canonical streaming-median pattern) only work for a fixed k; the interviewer wants arbitrary k per call.

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 166 of 196

★ TOP 75: rank 73 in the Medium and Hard priority list

★ Design a restaurant waitlist system

STUDY: TYPE | Google frequency on LeetCode: all time 57.3, last 30 days 0 | book priority 43 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #146 LRU Cache

LeetCode match: No direct match; related LC 146 LRU Cache (ordered map)

Original: https://prachub.com/coding-questions/design-a-restaurant-waitlist-system

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **c** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
- Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
- Returns how many parties are ahead of this party in the current waitlist order (0-based).
- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

APPROACH

Hash map partyId -> party plus an ordered queue (linked list/OrderedDict) for arrival order; on a free table of size s, seat the earliest waiting party that fits; cancel is O(1) with lazy deletion.

INTERVIEW-DAY PLAN

Clarify first

Ask for the exact API (add party, cancel, seat when a table of capacity c frees up), what happens when no party fits, and whether a larger table may seat a smaller party (yes, if size <= c).

Brute force to say out loud

Keep a list in arrival order; on a table of capacity c scan from the front for the first party that fits. O(n) per seating, O(n) per cancel.

Optimal idea

Keep id to party in a dict. For efficiency use a segment tree over arrival order storing the minimum party size in each range; the earliest party that fits is the leftmost position with size <= c, found by descending the tree in O(log n). Cancel and seat set that position's size to infinity. The simpler version uses an ordered dict plus lazy deletion and scans, O(n) in the worst case, which is fine to start with and to improve.

Dry-run these cases

A table that fits nobody, a party that cancels before being seated, ties on arrival, a table that fits the second party but not the first, duplicate party id.

Likely follow-ups

Large parties waiting too long (starvation), time-based expiry, or estimating wait time.

Pitfalls

Skipping a party that cancelled but is still in the queue. Choosing the best fit instead of the earliest fit.

HOW TO THINK ABOUT IT

'Earliest arrival whose size fits' is a query on an ordered sequence: find the leftmost position whose value is at most c. A scan does it in O(n); a segment tree that stores the minimum size over each range answers it in O(log n) by walking down to the leftmost child whose range minimum fits. Cancelling or seating simply sets that position to infinity.

Basis: Written design (compare LeetCode 146 for the ordered-map idea). API assumed (confirm it): add(id, name, size), cancel(id), seat(capacity) seats and returns the earliest-arrived waiting party whose size is at most the capacity, or None.

BRUTE FORCE - PSEUDO-CODE

```
keep parties in arrival order in a list
seat(capacity): scan from the front for the first party wh
ose size fits; remove and
    return it
cancel(id): remove the party with that id
```

Time: O(n) per seat and per cancel | Space: O(n)

Why it is slow

Both operations walk the list, O(n). A segment tree of the smallest size per range finds the leftmost party that fits in O(log n).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(operations):
    import heapq

    class Fenwick:
        def __init__(self, n):
            self.n = n
            self.bit = [0] * (n + 1)

        def add(self, idx, delta):
            while idx <= self.n:
                self.bit[idx] += delta
                idx += idx & -idx

        def sum(self, idx):
            total = 0
            while idx > 0:
                total += self.bit[idx]
                idx -= idx & -idx
            return total

    max_adds = sum(1 for op in operations if op and op[0]
== "addParty")
    bit = Fenwick(max_adds + 2)
    max_size = 20
    size_heaps = [[] for _ in range(max_size + 1)]
    party_to_arrival = {}
    arrival_to_party = [0] * (max_adds + 2)
    active = [False] * (max_adds + 2)

    next_party_id = 1
    next_arrival = 1
    answers = []

    for op in operations:
        kind = op[0]
```

```
        if kind == "addParty":
            _, name, size = op
            party_id = next_party_id
            next_party_id += 1

            arrival = next_arrival
            next_arrival += 1

            party_to_arrival[party_id] = arrival
            arrival_to_party[arrival] = party_id
            active[arrival] = True

            heapq.heappush(size_heaps[size], arrival)
            bit.add(arrival, 1)
            answers.append(party_id)

        elif kind == "cancelParty":
            _, party_id = op
            arrival = party_to_arrival.get(party_id, 0)
            if arrival == 0 or not active[arrival]:
                answers.append(False)
            else:
                active[arrival] = False
                bit.add(arrival, -1)
                answers.append(True)

        elif kind == "seatTable":
            _, capacity = op
            capacity = min(capacity, max_size)
            best_arrival = 0
            best_size = 0

            for size in range(1, capacity + 1):
```

```
                heap = size_heaps[size]
                while heap and not active[heap[0]]:
```

```
                    heapq.heappop(heap)
                if heap:
                    arrival = heap[0]
                    if best_arrival == 0 or arrival < best
_arrival:
                        best_arrival = arrival
                        best_size = size

            if best_arrival == 0:
                answers.append(None)
            else:
                heapq.heappop(size_heaps[best_size])
                active[best_arrival] = False
                bit.add(best_arrival, -1)
                answers.append(arrival_to_party[best_arriv
al])

        elif kind == "getPosition":
            _, party_id = op
            arrival = party_to_arrival.get(party_id, 0)
            if arrival == 0 or not active[arrival]:
                answers.append(-1)
            else:
                answers.append(bit.sum(arrival - 1))

        else:
            raise ValueError(f"Unknown operation: {kind}")

    return answers
```

Time: O(m log m) for m operations: addParty/cancelParty/getPosition are O(log m) (Fenwick update/query). seatTable does up to 20 O(1) heap peeks plus amortized lazy pops; each party is heap-pushed once and popped once over the whole run, so total pop cost is O(m log m), making seatTable O(20 + log m) = O(log m) amortized. | Space: O(m): the Fenwick tree, active/arrival arrays, the maps, and the per-size heaps together hold one entry per added party (sizes are bounded by 20 buckets, a constant).

Explanation

The waitlist is keyed by arrival order (a monotonically increasing arrival counter), so "earliest-arrived" is simply the smallest active arrival index. Two independent structures support the four operations efficiently.

1. getPosition — Fenwick tree (BIT). bit stores a 1 at each active party's arrival index. bit.sum(arrival - 1) counts active parties that arrived strictly before the target, which is exactly how many are ahead of it. Add/cancel/seat flip the corresponding index by ±1. Each update and query is O(log m).
2. seatTable — 20 per-size min-heaps with lazy deletion. Parties are bucketed by size (1..20) into size_heaps[size], each a min-heap of arrival indices. To seat for a capacity, the loop scans sizes 1..capacity, first discarding stale heap tops (not active[...]) — these are parties already cancelled or seated — then peeks the smallest valid arrival across those buckets. The bucket whose top has the globally smallest arrival wins, giving the earliest-arrived fitting party. That entry is popped, marked inactive, and removed from the BIT.

Bookkeeping. party_to_arrival / arrival_to_party map partyIds to arrival indices and back; active[] is the single source of truth for "still waiting," enabling O(1) cancellation (just flip the flag) and lazy heap cleanup.

Why correct: arrival order strictly reflects FIFO priority, the BIT prefix-sum is an exact ahead-count, and scanning size buckets 1..capacity considers every party that fits while the min-heaps + active check guarantee the earliest valid one is chosen. Lazy deletion is safe because a stale entry is never re-activated, so popping it can't drop a still-waiting party.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

```
class WaitlistBestFit2:
    def __init__(self):
        self.by_size = defaultdict(deque)
        self.sizes = []
        self.n = 0

    def add(self, pid, name, size):
        if not self.by_size[size]:
            insort(self.sizes, size)
        self.by_size[size].append(pid)

    def seat(self, capacity):
        i = bisect_right(self.sizes, capacity) - 1
        if i < 0:
            return None
        size = self.sizes[i]
        pid = self.by_size[size].popleft()
        if not self.by_size[size]:
            del self.sizes[i]
        return pid
```

FUNCTION AND TEST CASES

Test cases

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('seatTable', 2), ('seatTable', 4)]`	`[1, 2, 2, 1]`

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('addParty', 'C', 3), ('getPosition', 3), ('seatTable', 2), ('getPosition', 3), ('cancelParty', 1), ('getPosition', 3), ('cancelParty', 1), ('seatTable', 2), ('seatTable', 3), ('getPosition', 3)]`	`[1, 2, 3, 2, 2, 1, True, 0, False, None, 3, -1]`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Restaurant Waitlist: join / delete / find_first_match | 1Point3Acres | onsite-coding | asked 2026-02-01 | Single report

https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

Preparation: Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 167 of 196

Design server allocation for multi-type nodes

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 188 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match; related LC 1912 Design Movie Rental System

Original: https://prachub.com/interview-questions/design-server-allocation-for-multi-type-nodes

QUESTION

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

APPROACH

Map instance type -> ordered set of free nodes; allocating a node removes it from every type's set (lazy deletion with an allocated flag); release re-adds it. Pick by the rule in the prompt.

INTERVIEW-DAY PLAN

Clarify first

Ask what getServer returns when it cannot be satisfied, whether it must be atomic, what collectively meet the types means (every requested type is covered by at least one chosen server), and the scale (about 100,000 nodes).

Brute force to say out loud

Scan all nodes on every request and test their types. O(N) per call.

Optimal idea

Keep, for each instance type, a heap or sorted set of free node ids ordered by how few types the node supports (least flexible first). To serve, pick for each requested type the least flexible free node not already chosen, then fill the remaining count with least flexible nodes that match any requested type. Mark nodes allocated (lazy deletion in the other types' structures). Roll back if the request cannot be completed. Release re-adds the node to each of its types.

Dry-run these cases

Request larger than the free pool, a node shared by two requested types, return then re-request, releasing an id that is not allocated.

Likely follow-ups

Concurrency (per-type locks or a single allocator thread), starvation (FIFO queue), fragmentation, failure handling (leases and timeouts).

Pitfalls

Allocating a node and forgetting to remove it from the other type indexes. Greedy choices that are not optimal set cover; say so.

HOW TO THINK ABOUT IT

A request mentions a few types, so index the free servers by type and avoid scanning all 100,000. To keep versatile servers available for harder requests, always prefer the server that supports the fewest types. Allocation touches every type index of a server, so use lazy deletion: mark it allocated and let the stale heap entries be skipped when they surface.

Basis: Written design. Assumed contract (confirm it): getServer(count, types) returns `count` free server ids such that every requested type is supported by at least one returned server, or None and no change; releaseServer(ids) frees them. To limit fragmentation the least flexible servers (supporting the fewest types) are used first.

BRUTE FORCE - PSEUDO-CODE

```
getServer(count, types): for each requested type: scan all
 free servers for the least
    flexible one supporting it and not yet chosen
then scan again for the remaining count among servers supp
orting any requested type;
    if not enough: fail
```

Time: O(N * T) per request (N servers) | Space: O(N)

Why it is slow

Every request scans all N servers, O(N * types). Per-type heaps ordered by flexibility with lazy deletion make a request O((count + types) log N).

OPTIMAL SOLUTION - PER-TYPE HEAPS ORDERED BY FLEXIBILITY, WITH LAZY DELETION

```
class Servers:
    def __init__(self, servers):
        self.types = {s: frozenset(t) for s, t in servers.
items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        for s, ts in self.types.items():
            for t in ts:
                heapq.heappush(self.heaps[t], (len(ts), s)
)

    def _top(self, t, skip):
        heap, held = self.heaps[t], []
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
            else:
                break
        result = (
            heap[0] if heap else None
        )  # read the best valid entry before putting the
skipped ones back
        for item in held:
            heapq.heappush(heap, item)
        return result

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if any(t in self.types[c] for c in chosen):
                continue
            top = self._top(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        while len(chosen) < count:
            best = None
            for t in types:
                top = self._top(t, set(chosen))
                if top and (best is None or top < best):
                    best = top
            if best is None:
                return None
            chosen.append(best[1])
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def releaseServer(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                for t in self.types[s]:
                    heapq.heappush(self.heaps[t], (len(sel
f.types[s]), s))
```

Time: O((count + types) log N) amortized per request | Space: O(total type memberships)

Why

For each type keep a heap of (number of supported types, id) for the servers that support it. A request first covers each requested type (taking the best server not already chosen), then fills the rest from the best heap tops among the requested types. Allocating a server sets its allocated flag, so stale entries in other types' heaps are discarded when they reach the top. If the request cannot be met, every claim is rolled back.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

```
class Servers:
    def __init__(self, servers):
        self.types = {s: frozenset(t) for s, t in servers.
items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        for s, ts in self.types.items():
            for t in ts:
                heapq.heappush(self.heaps[t], (len(ts), s)
)

    def _top(self, t, skip):
        heap, held = self.heaps[t], []
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
            else:
                break
        result = (
            heap[0] if heap else None
        )  # read the best valid entry before putting the
skipped ones back
        for item in held:
            heapq.heappush(heap, item)
        return result

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if any(t in self.types[c] for c in chosen):
                continue
            top = self._top(t, set(chosen))
            if top is None:
```

```
                return None
            chosen.append(top[1])
```

```
        while len(chosen) < count:
            best = None
            for t in types:
                top = self._top(t, set(chosen))
                if top and (best is None or top < best):
                    best = top
            if best is None:
                return None
            chosen.append(best[1])
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def releaseServer(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                for t in self.types[s]:
                    heapq.heappush(self.heaps[t], (len(sel
f.types[s]), s))


class ServersLeased(Servers):
    def __init__(self, servers):
        super().__init__(servers)
        self.lease = {}

    def getServer(self, count, types, now=0, ttl=10):
        got = super().getServer(count, types)
        if got:
            for s in got:
                self.lease[s] = now + ttl
        return got

    def expire(self, now):
```

```
        gone = [s for s, t in self.lease.items() if t <= n
ow]
        for s in gone:
            del self.lease[s]
        self.releaseServer(gone)
        return sorted(gone)
```

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 168 of 196

Design an elevator control system

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 189 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (OOP / low-level design)

Original: https://prachub.com/interview-questions/design-an-elevator-control-system-2

QUESTION

Design the software for a multi-elevator control system in a high-rise building. Specify classes, scheduling algorithms for car assignments, handling peak traffic, fairness, fault tolerance, and safety constraints. Consider real-time constraints and a simulation and testing strategy.

APPROACH

Model Elevator, Request and Controller classes; scheduling by nearest-car or SCAN/LOOK direction-based assignment; queue targets per elevator, handle peak traffic, fairness, fault tolerance and safety; discuss simulation and testing strategy.

INTERVIEW-DAY PLAN

Clarify first

Ask how many elevators and floors, whether to design only the control logic or also the hardware, request types (hall call with direction, destination inside), and what to optimize (wait time, fairness, energy).

Brute force to say out loud

A single queue: serve requests first come first served with the nearest idle car. Easy but wasteful.

Optimal idea

Classes: Request, Elevator (floor, direction, state, a set of stops), Controller (assigns requests). Use SCAN/LOOK inside each car: keep moving in the current direction serving stops, then reverse. Assign a hall call to the car with the lowest cost, such as distance plus a penalty if it is moving away or is full. Add a step method that advances time so it can be simulated and tested.

Dry-run these cases

One request while idle, a request in the opposite direction, two cars equally close, a car out of service with pending stops, the top and bottom floors.

Likely follow-ups

Peak traffic modes, weight limits, fault tolerance (reassign stops on failure), and safety (doors, overload).

Pitfalls

Putting all logic in one class. Not saying how you will test it.

HOW TO THINK ABOUT IT

Separate three things: the data (Request), one car's behaviour (Elevator: floor, direction, stops, door/state) and the policy that decides which car takes a call (Controller). Say the scheduling rule out loud: a car keeps going in its direction serving stops, then reverses (SCAN/LOOK), which avoids the back-and-forth of first-come-first-served. Add a step() so the whole thing can be simulated and tested.

Basis: Written object-oriented design (the classic low-level design question). A SCAN (elevator) scheduling policy per car with a cost-based assignment of hall calls.

BRUTE FORCE - PSEUDO-CODE

```
keep a queue of requests in arrival order
for each request: send the nearest idle car to that floor,
 then to the destination,
    before taking the next request
```

Time: O(requests * cars) | Space: O(requests)

Why it is slow

Cars zig-zag across the building and requests wait behind far-away ones, so total travel and waiting are high. Serving stops in sweep

order (SCAN) and assigning by cost cuts both.

OPTIMAL SOLUTION - ELEVATOR (SCAN) PER CAR AND COST-BASED ASSIGNMENT OF CALLS

```
class Request:
    def __init__(self, floor, direction):
        self.floor, self.direction = floor, direction  # d
irection: +1 up, -1 down


class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor, self.direction, self.stops, s
elf.in_service = (
            car_id,
            floor,
            0,
            set(),
            True,
        )

    def cost(self, req):
        if not self.in_service:
            return float("inf")
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        ahead = (req.floor - self.floor) * self.direction
>= 0
        return (
            dist
            if ahead and req.direction == self.direction
            else dist
            + 2 * (max(self.stops) - min(self.stops) + 1 i
f self.stops else 0)
            + 10
        )

    def step(self):
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        if self.direction >= 0 and up or (self.direction <
 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served
```

Time: O(cars) per call, O(1) per car per step | Space: O(stops)

Why

Each car keeps a set of stops and a direction. step() moves one floor in its direction and removes a stop it reaches; when no stops remain ahead it reverses, and when none remain at all it goes idle. The controller assigns a hall call to the car with the lowest cost: distance, plus a penalty if the car is moving away or is out of service. Cars that fail drop out and their stops are reassigned.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

```
class Controller:
    def __init__(self, cars):
        self.cars, self.served = cars, []

    def request(self, floor, direction=0):
        req = Request(floor, direction)
        best = min(self.cars, key=lambda c: (c.cost(req),
c.id))
        best.stops.add(floor)
        return best.id

    def fail(self, car_id):
        car = next(c for c in self.cars if c.id == car_id)
        car.in_service = False
        pending, car.stops = car.stops, set()
        for floor in pending:
            self.request(floor)

    def step(self):
        for c in self.cars:
            f = c.step()
            if f is not None:
                self.served.append((c.id, f))

    def idle(self):
        return all(not c.stops for c in self.cars if c.in_
service)


class LobbyController(Controller):
    def park_idle(self, lobby=0):
        for c in self.cars:
            if c.in_service and not c.stops and c.floor !=
 lobby:
                c.stops.add(lobby)
```

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 169 of 196

Temperature Monitoring Data Structure

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 191 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

QUESTION

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days.SWEdata-structuresstreamingsliding-windowFrequencySingle reportLast asked2026-09-10Stageonsite-coding

APPROACH

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans.
Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the window is the last N readings or the last N days, what moving average should return before the window fills, and the number of readings.

Brute force to say out loud

Store everything in a list and compute the window average and max by scanning. O(window) per query.

Optimal idea

For a count window keep a deque of the last N readings and a running sum for the average, and a monotonic decreasing deque for the maximum: pop smaller values from the back on insert and drop the front when it expires. All operations O(1) amortized. For time windows store (time, value) and evict by timestamp.

Dry-run these cases

N = 1, duplicate maxima, negative readings, the maximum expiring, fewer readings than the window.

Likely follow-ups

Minimum too, median over a window, or out-of-order timestamps.

Pitfalls

Removing the wrong element from the max deque when equal values repeat. Floating-point drift in the running sum.

HOW TO THINK ABOUT IT

Three operations on the last N readings. The average only needs a running sum, and the maximum needs a monotonic deque that drops smaller values from the back as larger ones arrive, so the front is always the max of the window. Decide first whether the window is a count of readings or a span of time.

Basis: Written; a sliding window with an average (LeetCode 346) and a window maximum (LeetCode 239), here as one data structure.

BRUTE FORCE - PSEUDO-CODE

```
keep every reading in a list
average(): sum the last N readings and divide; maximum():
take the max of the last N
```

Time: O(N) per average or max | Space: O(N)

Why it is slow

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

OPTIMAL SOLUTION - RUNNING SUM PLUS A MONOTONIC DEQUE

```python
class Monitor:
    def __init__(self, n):
        self.n, self.window, self.total = n, deque(), 0
        self.maxq, self.seen = deque(), 0

    def insert(self, x):
        self.window.append(x)
        self.total += x
        if len(self.window) > self.n:
            self.total -= self.window.popleft()
        while self.maxq and self.maxq[-1][1] <= x:
            self.maxq.pop()
        self.maxq.append((self.seen, x))
        self.seen += 1
        while self.maxq[0][0] <= self.seen - 1 - self.n:
            self.maxq.popleft()

    def average(self):
        return self.total / len(self.window)

    def maximum(self):
        return self.maxq[0][1]
```

Time: O(1) amortized per operation | Space: O(N)

Why

Keep the window in a deque and a running sum; on insert add the value and drop the oldest when the window is full. For the maximum keep a second deque of (index, value) with decreasing values: pop smaller ones from the back before pushing, and pop the front when its index leaves the window. The front is the maximum.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

```python
class TimeMonitor:
    """Window by elapsed time: readings older than `span`
are dropped (timestamps non-decreasing)."""

    def __init__(self, span):
        self.span, self.window, self.total, self.maxq = sp
an, deque(), 0, deque()

    def insert(self, t, x):
        self.window.append((t, x))
        self.total += x
        while self.maxq and self.maxq[-1][1] <= x:
            self.maxq.pop()
        self.maxq.append((t, x))
        while self.window and self.window[0][0] <= t - sel
f.span:
            self.total -= self.window.popleft()[1]
        while self.maxq and self.maxq[0][0] <= t - self.sp
an:
            self.maxq.popleft()

    def average(self):
        return self.total / len(self.window)

    def maximum(self):
        return self.maxq[0][1]
```

Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

NOTES FROM CANDIDATES

The introductory description refers to the most recent N days, while the API requirements refer to the most recent N readings. Clarify the window definition before coding; the day-based variant does not specify how timestamps enter the interface.

Empty-window behavior and insertion complexity are unspecified.

For the reading-count window with positive N, retain the last N readings in a FIFO queue and maintain their sum. Add the new reading and subtract any expired reading. Agree whether a partially filled window returns its current mean or is unavailable.

Maintain a second deque of reading indices with nonincreasing temperatures. Expire old indices at the front and remove smaller or equal temperatures from the back before adding the new index. The front holds the current maximum; indices distinguish duplicate readings.

Average and maximum queries are O(1). Each reading enters and leaves each deque at most once, so insertion is amortized O(1) but can take O(N) for one update; auxiliary space is O(N). This implements the reading-count contract. A day-based contract additionally needs timestamps, expiration timing, and an ordering rule.

PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans.

Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 170 of 196

Next-Word Predictor

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 192 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.

Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

Clarify first

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

Brute force to say out loud

Store all sentences and scan them for each prediction. O(corpus) per query.

Optimal idea

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

Dry-run these cases

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

Likely follow-ups

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

Pitfalls

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE - PSEUDO-CODE

```
keep all training sentences
predict(word): count, over all adjacent pairs in all sente
nces, which word follows
    'word' most often; return the most frequent
```

Time: O(corpus size) per prediction | Space: O(corpus size)

Why it is slow

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION - COUNTS PLUS A MAINTAINED BEST SUCCESSOR

```
class Predictor:
    def __init__(self):
        self.counts = defaultdict(Counter)
        self.best = {}

    def train(self, sentence):
        words = sentence.split()
        for a, b in zip(words, words[1:]):
            self.counts[a][b] += 1
            cur = self.best.get(a)
            if (
                cur is None
                or self.counts[a][b] > self.counts[a][cur]
                or (self.counts[a][b] == self.counts[a][cu
r] and b < cur)
            ):
                self.best[a] = b

    def predict(self, word):
        return self.best.get(word)
```

Time: O(1) per pair when training and per prediction | Space: O(number of distinct pairs)

Why

counts[word][next] is incremented for every adjacent pair. After the increment compare that one count with the current best for the word (higher count wins, then the smaller word). Because only one count changed, no other successor can overtake the best, so one comparison keeps it correct.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

```
class Predictor:
    def __init__(self):
        self.counts = defaultdict(Counter)
        self.best = {}

    def train(self, sentence):
        words = sentence.split()
        for a, b in zip(words, words[1:]):
            self.counts[a][b] += 1
            cur = self.best.get(a)
            if (
                cur is None
                or self.counts[a][b] > self.counts[a][cur]
                or (self.counts[a][b] == self.counts[a][cu
r] and b < cur)
            ):
                self.best[a] = b

    def predict(self, word):
        return self.best.get(word)


class TopKPredictor(Predictor):
    def predict_k(self, word, k):
        c = self.counts.get(word, {})
        return heapq.nsmallest(k, c, key=lambda w: (-c[w],
 w))
```

Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

```
class TrigramPredictor:
    def __init__(self):
        self.counts = defaultdict(Counter)

    def train(self, sentence):
        w = sentence.split()
        for a, b, c in zip(w, w[1:], w[2:]):
            self.counts[(a, b)][c] += 1

    def predict(self, a, b):
        c = self.counts.get((a, b))
        return min(c, key=lambda x: (-c[x], x)) if c else
None
```

The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.
The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.
After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.
With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.
Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 171 of 196

Implement Python's itertools.tee

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 193 of 196
Topic: Other: Design, Heaps, Intervals and Misc
Optimal answer from: written and tested here for this exact question
Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE
Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement the behavior of Python's `tee`: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

APPROACH

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the source iterator can be consumed by one consumer first, what happens if one consumer is far ahead, and whether thread safety is needed.

Brute force to say out loud

Store the whole source in a list and give each consumer an index. Simple, but defeats the purpose for large or infinite iterators.

Optimal idea

Keep a deque per consumer. When a consumer needs an item and its deque is empty, pull the next item from the source and append it to every consumer's deque. Each item is held only until all consumers have read it. Build with a shared state object and a small iterator class with __iter__ and __next__. The linked-list variant uses one chain of nodes with each consumer holding a pointer, which frees nodes as the slowest consumer moves.

Dry-run these cases

Two consumers at different speeds, n = 1, n = 0, an empty source, an exhausted source.

Likely follow-ups

Thread safety (a lock around the pull), and memory growth if one consumer stalls.

Pitfalls

Pulling from the source once per consumer instead of once per item. Not appending to the other deques.

HOW TO THINK ABOUT IT

The source can only be consumed once, so someone must remember items until every consumer has read them. The state to keep is exactly 'items the slowest consumer has not read yet', so either give each consumer a queue that the leader fills, or keep one shared linked list with a pointer per consumer so memory is freed automatically.

Basis: The itertools.tee documentation's recipe; the linked-list variant is how CPython shares buffers.

BRUTE FORCE - PSEUDO-CODE

```
items = list(source)   # read everything now
each of the n consumers gets its own index into items and
reads items[index], index +=
    1
```

Time: O(n) up front | Space: O(n)

Why it is slow

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in

memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION - ONE DEQUE PER CONSUMER, FILLED LAZILY

```
def tee(iterable, n=2):
    it = iter(iterable)
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(it)
                except StopIteration:
                    return
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)
```

Time: O(n) total, O(1) amortized per item per consumer | Space: O(lag between fastest and slowest consumer)

Why

When a consumer's deque is empty it pulls the next item from the source and appends it to every consumer's deque, then pops its own. Each item is read from the source exactly once and is dropped by a consumer after being read, so memory is only the gap between the leader and the slowest follower.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

```
class _Link:
    __slots__ = ("value", "next")

    def __init__(self):
        self.value = self.next = None


def tee_linked(iterable, n=2):
    it = iter(iterable)
    head = _Link()
    state = {"tail": head, "done": False}

    def consumer(node):
        while True:
            if node.next is None:
                if state["done"]:
                    return
                try:
                    value = next(it)
                except StopIteration:
                    state["done"] = True
                    return
                link = _Link()
                link.value = value
                state["tail"].next = link
                state["tail"] = link
            node = node.next
            yield node.value

    return tuple(consumer(head) for _ in range(n))
```

Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. Cost: one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

NOTES FROM CANDIDATES

The round depends on Python iterator-protocol fluency as much as general data-structure reasoning. Clarify laziness, source exhaustion, invalid n, and whether concurrent consumption is in scope before implementing.

A correct baseline consumes the source lazily once and gives each returned iterator its own queue: when one queue is empty, pull one source value, append it to every queue, then return it from the requesting queue. If n iterators trail across a gap of g values, however, those queues can retain O(n·g) buffered copies. For the optimal shared-state design, keep one linked buffer (or indexed deque) of source values plus one cursor per iterator, and discard a buffered value only after every cursor has passed it. That preserves independent consumption with O(g+n) auxiliary state, where g is the distance between the fastest and slowest consumers.

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.

Trace two and three consumers advancing at different speeds and identify what state must remain available.

Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 172 of 196

Batch and Streaming Log Deduplication

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 194 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

APPROACH

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

INTERVIEW-DAY PLAN

Clarify first

Ask which entry to keep in each policy (first seen or latest timestamp), how ties on timestamp break, whether output is sorted by timestamp, and how the streaming interface returns results (get_next).

Brute force to say out loud

Sort all logs and dedupe with a set. Correct for the batch version.

Optimal idea

Batch: keep a dict message to chosen entry (keep first-seen, or replace if the new timestamp is later) and sort the values by (timestamp, message). Streaming: a dict for membership plus a min-heap of (timestamp, message); on update of a message push a new entry and mark the old one stale (lazy deletion), and get_next pops until it finds a non-stale one. Amortized O(log n).

Dry-run these cases

The same message twice at different times, equal timestamps, an empty stream, get_next on an empty queue.

Likely follow-ups

Watermarks for late data, bounded memory with eviction by age.

Pitfalls

Returning a stale heap entry. Forgetting deterministic tie-breaking.

HOW TO THINK ABOUT IT

Dedup is a dictionary keyed by the message. Sorting is only for the output order. For a stream, a heap gives the smallest timestamp quickly, but when an entry is replaced you cannot delete from the middle of a heap, so leave the stale entry and skip it when it surfaces (lazy deletion).

Basis: Written. Assumed rules (confirm them): dedupe on the message; policy 'first' keeps the earliest in input order, policy 'latest' keeps the largest timestamp; output sorted by (timestamp, message); get_next returns the retained entry with the smallest timestamp and removes it.

BRUTE FORCE - PSEUDO-CODE

```
for each log line: scan the other lines with the same mess
age to decide if this one is
    the one to keep (first seen or latest)
collect the kept lines and sort them by timestamp
```

Time: O(n^2) | Space: O(n)

Why it is slow

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION - DICTIONARY ON THE MESSAGE, THEN SORT

```python
def dedupe(logs, policy="first"):
    kept = {}
    for ts, msg in logs:
        if msg not in kept or (policy == "latest" and ts >
 kept[msg]):
            kept[msg] = ts
    return sorted(((ts, msg) for msg, ts in kept.items()),
 key=lambda e: (e[0], e[1]))
```

Time: O(n log n) | Space: O(n)

Why

One pass with a dict from message to the retained timestamp (first seen: keep the first; latest: replace when the new timestamp is larger), then sort the retained entries for the output.

OPTIMAL SOLUTION - STREAMING: MAP + MIN-HEAP WITH LAZY DELETION

```python
class LogStream:
    def __init__(self, policy="latest"):
        self.policy, self.current, self.heap = policy, {},
 []

    def add(self, ts, msg):
        if msg not in self.current or (
            self.policy == "latest" and ts > self.current[
msg]
        ):
            self.current[msg] = ts
            heapq.heappush(self.heap, (ts, msg))

    def get_next(self):
        while self.heap:
            ts, msg = heapq.heappop(self.heap)
            if self.current.get(msg) == ts:
                del self.current[msg]
                return ts, msg
        return None
```

Time: O(log n) amortized per add and get_next | Space: O(n)

Why

current[msg] holds the retained timestamp. add pushes (timestamp, msg) on the heap only when it becomes the retained value (new message, or a larger timestamp under the 'latest' policy), leaving the older heap entry behind. get_next pops entries until one matches current[msg] (the others are stale), returns it and removes the message.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. Cost: O(log n) per eviction with a heap; memory O(entries inside the retention window).

Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

```
class LogStream:
    def __init__(self, policy="latest"):
        self.policy, self.current, self.heap = policy, {},
 []

    def add(self, ts, msg):
        if msg not in self.current or (
            self.policy == "latest" and ts > self.current[
msg]
        ):
            self.current[msg] = ts
            heapq.heappush(self.heap, (ts, msg))

    def get_next(self):
        while self.heap:
            ts, msg = heapq.heappop(self.heap)
            if self.current.get(msg) == ts:
                del self.current[msg]
                return ts, msg
        return None


class WatermarkLogStream(LogStream):
    def release(self, watermark):
        out = []
        while self.heap:
            ts, msg = self.heap[0]
            if self.current.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            if ts > watermark:
                break
            heapq.heappop(self.heap)
            del self.current[msg]
            out.append((ts, msg))
        return out
```

NOTES FROM CANDIDATES

Empty input, identical entries, equal timestamps, out-of-order arrivals, updates, and removals all need explicit semantics.
For keep-latest streaming behavior, stale heap entries must be detected against the current timestamp stored for that message.
Arbitrarily late arrivals make globally final timestamp ordering impossible without a watermark or maximum-lateness guarantee.

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

Trie and Prefix Indexing 4 questions

MEDIUM | IN APP LC 208 | PracHub | Technical Screen | 173 of 196

Return dictionary words matching a prefix

STUDY: TALK | Google frequency on LeetCode: all time 36, last 30 days 0 | book priority 96 of 196
Topic: Trie and Prefix Indexing
Optimal answer from: the interview site's own solution
Also in your app (open the app for its Expose): LeetCode #208 Implement Trie (Prefix Tree)
LeetCode match: LC 208 Implement Trie (Prefix Tree)
Original: https://prachub.com/coding-questions/return-dictionary-words-matching-a-prefix

QUESTION

You are given a list of strings **`words`** (a dictionary) and a string **`prefix`**. Return all words in **`words`** that start with **`prefix`**.
Clarifications/constraints:

- Matching is case-sensitive.
- If no words match, return an empty list.
- If **`words`** contains duplicates, include duplicates in the output.
- You may return the result in any order unless the interviewer asks for a specific ordering (e.g., lexicographic).

APPROACH

Sort once and binary search the prefix range, or build a trie for repeated queries. O(log n + m) per query after preprocessing.

HOW TO THINK ABOUT IT

If the dictionary is sorted, all words with a prefix form one contiguous block, and its edges are found with binary search: the first word >= prefix, and the first word that is >= prefix with its last character incremented. For many queries on a static list this beats a trie in memory.

Basis: Search Suggestions System (LeetCode 1268) and prefix queries on a sorted list.

BRUTE FORCE - PSEUDO-CODE

```
for each query prefix: keep every dictionary word that sta
rts with it
```

Time: O(W * L) per query | Space: O(m)

Why it is slow

Each query scans the whole dictionary, O(W * L). With the words sorted, binary search finds the block of matches directly.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(words, prefix):
    result = []
    for word in words:
        if word.startswith(prefix):
            result.append(word)
    return result
```

Time: O(n * p), where n is the number of words and p is the length of the prefix | Space: O(k), where k is the number of matching words returned

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

```
class PrefixIndex:
    def __init__(self, words):
        self.words = sorted(words)

    def matches(self, prefix, limit=None):
        i = bisect_left(self.words, prefix)
        out = []
        while i < len(self.words) and self.words[i].starts
with(prefix):
            out.append(self.words[i])
            if limit is not None and len(out) == limit:
                break
            i += 1
        return out
```

Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

```
class LivePrefixIndex:
    def __init__(self):
        self.words = []

    def add(self, w):
        i = bisect_left(self.words, w)
        if i == len(self.words) or self.words[i] != w:
            self.words.insert(i, w)

    def matches(self, prefix, limit=None):
        i, out = bisect_left(self.words, prefix), []
        while (
            i < len(self.words)
            and self.words[i].startswith(prefix)
            and (limit is None or len(out) < limit)
        ):
            out.append(self.words[i])
            i += 1
        return out
```

Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], prefix: str) -> list[str]:
```

Sample call

```
solution(['apple', 'app', 'banana', 'apply'], 'app')
```

Test cases

Input	Expected
`(['apple', 'app', 'banana', 'apply'], 'app')`	`['apple', 'app', 'apply']`
`(['Cat', 'car', 'Cart', 'dog'], 'Ca')`	`['Cat', 'Cart']`

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 174 of 196

Build Prefix Lookup with a Trie

STUDY: TALK | Google frequency on LeetCode: all time 36, last 30 days 0 | book priority 95 of 196

Topic: Trie and Prefix Indexing

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree) (inferred from overview)

Original:
https://prachub.com/coding-questions/build-prefix-lookup-with-a-trie

QUESTION

Overview: Practice a Google coding interview problem focused on build prefix lookup with a trie. The prompt emphasizes edge cases, clean implementation, and verifiable test behavior without revealing the solution.

APPROACH

Trie of dict words; walk the prefix, then DFS the subtree to collect completions (optionally store counts at nodes).

HOW TO THINK ABOUT IT

Prefix questions repeat work when each query scans the whole word list. A trie stores each prefix once: walk down by the query, then everything below that node is a completion. Keep counters or a word flag in the nodes to answer follow-ups such as 'how many words start with this' or 'top K completions' without another walk.

Basis: LeetCode 208 (Implement Trie) and 1268 (Search Suggestions System).

BRUTE FORCE - PSEUDO-CODE

```
for each query prefix: for every dictionary word: if it st
arts with the prefix:
    collect it
return the collected words
```

Time: O(W * L) per query | Space: O(1) extra

Why it is slow

Every query touches every word, O(W * L). A trie walks only the prefix and then the subtree below it.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(words, queries):
    sw = sorted(words)
    return [[w for w in sw if w.startswith(q)] for q in qu
eries]
```

Time: Varies by input size; see reference solution. | Space: Varies by input size; see reference solution.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

```
class CountingTrie:
    """Each node counts the words passing through it, so '
how many words start with p' needs no DFS."""

    def __init__(self):
        self.root = {"n": 0, "c": {}}

    def insert(self, word):
        node = self.root
        node["n"] += 1
        for ch in word:
            node = node["c"].setdefault(ch, {"n": 0, "c":
{}})
            node["n"] += 1

    def count_prefix(self, prefix):
        node = self.root
        for ch in prefix:
            node = node["c"].get(ch)
            if node is None:
                return 0
        return node["n"]
```

Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. Cost: a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], queries: list[str]) -> list
[list[str]]:
SAMPLE INPUT
{"words":["car","cat","dog"],"queries":["ca","d"]}
```

Test cases

Input	Expected
`{"words":["car","cat","dog"],"queries":["ca","d"]}`	`[["car","cat"],["dog"]]`
`{"words":[],"queries":["a"]}`	`[[]]`

ALSO REPORTED - 1 MORE REPORT OF THIS QUESTION

Trie Prefix Search / Autocomplete | 1Point3Acres | tech-screen, onsite-coding | asked 2026-07-02 | Medium

https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

Standard implementation: each trie node has a children map and an is_end flag.

Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).

The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

Preparation: Write the trie class from memory in under 12 min. Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.

For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 175 of 196

Implement Trie Insert and Exact-Word Search

STUDY: TALK | Google frequency on LeetCode: all time 36, last 30 days 0 | book priority 97 of 196

Topic: Trie and Prefix Indexing

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Original: https://prachub.com/coding-questions/implement-trie-insert-and-exact-word-search

QUESTION

Problem

Implement a trie that supports **`insert(word)`** and **`search(word)`**. Insert adds a lowercase word. Search returns true only when the complete word was previously inserted; a stored word merely sharing the requested prefix is not a match. Process a sequence of operations and return the results of all searches.

Constraints & Assumptions

- There are at most 200,000 operations.
- Every word has length between 1 and 1,000.
- The total number of characters across all operations is at most 1,000,000.
- Words contain lowercase English letters.

Clarifications

- Inserting the same word repeatedly is allowed and has no additional effect.
- A node must distinguish an inserted word ending there from a prefix only.
- Return booleans in the same order as search operations appear.

Examples

```
operations = [insert("apple"), search("apple"), search("ap
p"), insert("app"), search("app")] output = [true, false,
true]
```

Hints

```
Represent prefixes Each edge consumes one character and ea
ch node represents the prefix along its path.
```

```
Mark complete words Reaching the final character is not en
ough unless the terminal node records an insertion.
```

APPROACH

Trie nodes with children and an end-of-word flag; search returns true only if the final node is marked as a word.

HOW TO THINK ABOUT IT

A trie needs a flag that marks 'a word ends here'; without it search('app') would be true whenever 'apple' was inserted. Walking a prefix and checking the flag are the two operations: search checks the flag, startsWith does not.

Basis: LeetCode 208, Implement Trie (Prefix Tree).

BRUTE FORCE - PSEUDO-CODE

```
insert(w): append w to a list
search(w): is w equal to some word in the list?  startsWit
h(p): does some word start
    with p?  (scan the list)
```

Time: O(W * L) per operation | Space: O(total characters)

Why it is slow

Each call scans every stored word, O(W * L). A trie with an end-of-word flag makes every call O(L).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def process_trie_operations(operations):
    children = [{}]
    terminal = [False]
    results = []

    for operation, word in operations:
        if operation == "insert":
            node = 0
            for character in word:
                next_node = children[node].get(character)
                if next_node is None:
                    next_node = len(children)
                    children[node][character] = next_node
                    children.append({})
                    terminal.append(False)
                node = next_node
            terminal[node] = True
        else:
            node = 0
            found = True
            for character in word:
                next_node = children[node].get(character)
                if next_node is None:
                    found = False
                    break
                node = next_node
            results.append(found and terminal[node])

    return results
```

Time: O(C), where C is the total number of characters across all operations | Space: O(I + r), where I is the number of distinct inserted-prefix characters and r is the number of searches

Explanation

Each trie node represents the prefix spelled by the path from the root and stores one child per next lowercase character plus a terminal flag. Insert follows or creates the word's edges and marks only its final node terminal. Search follows the requested edges; a missing edge proves the word was never inserted, while reaching all characters succeeds only when the final node is terminal. Therefore shared prefixes reuse nodes without confusing a prefix with an exact stored word.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

```
class Trie:
    def __init__(self):
        self.root = {"children": {}, "end": False}

    def _walk(self, text):
        node = self.root
        for ch in text:
            node = node["children"].get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node["children"].setdefault(ch, {"child
ren": {}, "end": False})
        node["end"] = True

    def search(self, word):
        node = self._walk(word)
        return node is not None and node["end"]

    def startsWith(self, prefix):
        node = self._walk(prefix)
        return node is not None and (
```

```
            node["end"] or bool(node["children"])
        )  # a node counts only if some word passes throug
h it


class WildcardTrie(Trie):
    """search('a.c') where '.' matches any one letter (Lee
tCode 211)."""

    def search(self, pattern):
        def go(node, i):
            if i == len(pattern):
                return node["end"]
            if pattern[i] == ".":
                return any(go(child, i + 1) for child in n
ode["children"].values())
            child = node["children"].get(pattern[i])
            return child is not None and go(child, i + 1)

        return go(self.root, 0)
```

Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. Cost: O(L) time and O(L) space for the path stack.

Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. Cost: O(L) per operation with dictionary children; memory O(total characters).

FUNCTION AND TEST CASES

Signature

```
def process_trie_operations(operations: list[list[str]]) -
> list[bool]:
```

Sample call

```
process_trie_operations([['insert', 'apple'], ['search', '
apple'], ['search', 'app'], ['insert', 'app'], ['search',
'app']])
```

Test cases

Input	Expected
`([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['search', 'app']],)`	`[True, False, True]`
`([],)`	`[]`

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 176 of 196

URL Request Handler Router

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 190 of 196

Topic: Trie and Prefix Indexing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-28 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

Examples:

Register /users/{userId}/photo/{photoId}. A lookup for /users/123/photo/456 matches that registered handler.

APPROACH

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.

In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.

In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

INTERVIEW-DAY PLAN

Clarify first

Ask how to choose between several matching patterns (literal over placeholder?), what to return on no match or duplicate registration, and how to treat trailing slashes, query strings and case.

Brute force to say out loud

Store the patterns in a list and compare segment by segment. O(H * L) per lookup. It is a correct baseline and gives a reference to test the trie against.

Optimal idea

Segment trie: each node has literal children and one wildcard child, with the original pattern stored at the end node. Lookup explores the literal child first and the wildcard child on failure, because a literal branch can die later. Accept only when all segments are consumed and a handler is stored.

Dry-run these cases

An exact route, a placeholder route, both literal and placeholder matching the same URL, segment-count mismatch, no match.

Likely follow-ups

Methods (GET and POST), wildcard at the end, priority rules, and many routes.

Pitfalls

Picking the literal branch greedily without falling back. Overwriting patterns that differ only in placeholder names.

HOW TO THINK ABOUT IT

Matching a path against many patterns is a prefix problem, so share the prefixes in a trie of segments. The catch is the wildcard: a literal branch can fail later, so you cannot pick it greedily. Try the literal child first and fall back to the wildcard child. That also encodes the precedence rule for free.

Basis: Written. Assumed rules (confirm them): patterns look like /users/{id}/photo; {name} matches exactly one segment; a literal beats a placeholder at the first segment where they differ; no match returns None; trailing slashes are ignored.

BRUTE FORCE - PSEUDO-CODE

```
split the URL into segments
for every registered pattern with the same segment count:
it matches if every segment
    is equal or the pattern's segment is a placeholder
among the matches choose by precedence (literal beats plac
eholder at the first
    difference)
```

Time: O(H * L) per lookup | Space: O(1) extra

Why it is slow

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION - SEGMENT TRIE WITH LITERAL-FIRST FALLBACK

```
class Router:
    def __init__(self, patterns=()):
        self.root = {"lit": {}, "wild": None, "handler": N
one}
        for p in patterns:
            self.add(p)

    def add(self, pattern):
        node = self.root
        for seg in _segments(pattern):
            if seg.startswith("{") and seg.endswith("}"):
                if node["wild"] is None:
                    node["wild"] = {"lit": {}, "wild": Non
e, "handler": None}
                node = node["wild"]
            else:
                node = node["lit"].setdefault(
                    seg, {"lit": {}, "wild": None, "handle
r": None}
                )
        if node["handler"] is None:
            node["handler"] = pattern

    def match(self, url):
        segs = _segments(url)

        def walk(node, i):
            if i == len(segs):
                return node["handler"]
            child = node["lit"].get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            if node["wild"] is not None:
                return walk(node["wild"], i + 1)
            return None

        return walk(self.root, 0)
```

Time: O(L * branching) per lookup, O(total pattern length) to build | Space: O(total pattern length)

Why

Each trie node has literal children and one wildcard child; the pattern is stored at its last node. Lookup tries the literal child first, and if that whole branch fails it backs up and tries the wildcard child. A match must consume every segment and end on a node that stores a pattern.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

```
class Router:
    def __init__(self, patterns=()):
        self.root = {"lit": {}, "wild": None, "handler": N
one}
        for p in patterns:
            self.add(p)

    def add(self, pattern):
        node = self.root
        for seg in _segments(pattern):
            if seg.startswith("{") and seg.endswith("}"):
                if node["wild"] is None:
                    node["wild"] = {"lit": {}, "wild": Non
e, "handler": None}
                node = node["wild"]
            else:
                node = node["lit"].setdefault(
                    seg, {"lit": {}, "wild": None, "handle
r": None}
                )
        if node["handler"] is None:
            node["handler"] = pattern

    def match(self, url):
        segs = _segments(url)

        def walk(node, i):
            if i == len(segs):
                return node["handler"]
            child = node["lit"].get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            if node["wild"] is not None:
                return walk(node["wild"], i + 1)
            return None

        return walk(self.root, 0)


def _segments(path):
    return [s for s in path.strip("/").split("/") if s !=
""]


class MethodRouter(Router):
    def add_route(self, method, pattern):
        self.add(f"{method}/{pattern.strip('/')}")

    def lookup(self, method, url):
        return self.match(f"{method}/{url.strip('/')}")
```

A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

NOTES FROM CANDIDATES

Clarify how to choose between multiple matching handlers, including a literal segment competing with a placeholder. The precedence rule is unspecified.

The no-match result, duplicate-registration behavior, and URL normalization rules are also unspecified; do not assume a canonical behavior before discussing them.

Matching approach

Start with a scan of registered patterns: compare segment counts, then require each position to contain either the same literal or a placeholder. This is a correctness baseline that can collect all matches before applying the agreed precedence rule.

A segment trie shares prefixes. Keep literal children and a separate wildcard child; at each URL segment explore both compatible branches. A literal branch can fail later, so choosing it greedily can miss a valid wildcard match.

Store the original registered patterns at terminal nodes. Patterns with different placeholder names can share the same trie path; retain each original rather than overwriting it. Accept terminals only after consuming the entire URL.

For tokenized paths of L segments and H handlers, the scan costs O(HL) segment comparisons in the worst case. A wildcard trie can branch, so lookup is not necessarily O(L): account for visited states, matching candidates, and string-processing costs separately. Keep normalization and winner selection tied to the clarified contract.

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.

In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.

In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

SIDE NOTE: marked problems that are not in this book

★ The top 75 Medium and Hard problems and the top 10 Easy problems by Google frequency are marked in this book. These 39 are marked too but are not questions here. Do them from your app. Rank is the position in the Medium and Hard list, or E1 to E10 for Easy.

Rank	LC	Problem	Lvl	Freq
3	3	Longest Substring Without Repeating	M	73/69.3
5	42	Trapping Rain Water	H	73.6/55.1
7	128	Longest Consecutive Sequence	M	70.3/58.3
8	15	3Sum	M	72.6/58.3
9	7	Reverse Integer	M	66.3/51.4
15	11	Container With Most Water	M	68.3/41.1
18	410	Split Array Largest Sum	H	57.7/46.9
21	17	Letter Combinations of a Phone Number	M	57.8/51.4
25	239	Sliding Window Maximum	H	54.9/46.9
26	31	Next Permutation	M	61/41.1
30	6	Zigzag Conversion	M	53.5/51.4
31	34	Find First and Last Position	M	53/46.9
32	54	Spiral Matrix	M	59.3/33
34	1944	Number of Visible People in a Queue	H	53.6/41.1
37	33	Search in Rotated Sorted Array	M	57.2/33
38	84	Largest Rectangle in Histogram	H	56.8/33
39	167	Two Sum II	M	49.8/46.9
42	155	Min Stack	M	50.8/41.1
43	19	Remove Nth Node From End of List	M	50.2/41.1
44	424	Longest Repeating Character Replacement	M	53.6/33
47	739	Daily Temperatures	M	47.8/41.1
48	78	Subsets	M	51.7/33
49	39	Combination Sum	M	47.6/41.1
52	16	3Sum Closest	M	50.6/33
53	131	Palindrome Partitioning	M	46/41.1
54	209	Minimum Size Subarray Sum	M	45.2/41.1
57	402	Remove K Digits	M	44.1/0
62	151	Reverse Words in a String	M	46/33
63	312	Burst Balloons	H	41.8/0
67	51	N-Queens	H	58.6/0
75	210	Course Schedule II	M	36.5/33
E1	20	Valid Parentheses	E	67.3/70.9
E3	14	Longest Common Prefix	E	73.6/55.1
E4	88	Merge Sorted Array	E	69.7/55.1
E5	26	Remove Duplicates from Sorted Array	E	65.7/55.1
E7	28	Find the Index of the First Occurrence	E	56.2/61
E8	485	Max Consecutive Ones	E	56/58.3
E9	169	Majority Element	E	66.5/33
E10	242	Valid Anagram	E	57.3/51.4

Freq is LeetCode's frequency for Google: all time / last 30 days.

Easy questions (20)

BFS/DFS Graph and Tree Traversal and Shortest Paths 4 questions

EASY | IN APP LC 199 | PracHub | Technical Screen | 177 of 196

Find right-side view of binary tree

STUDY: TALK | Google frequency on LeetCode: all time 34.9, last 30 days 0 | book priority 100 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #199 Binary Tree Right Side View

LeetCode match: LC 199 Binary Tree Right Side View

Original: https://prachub.com/coding-questions/find-right-side-view-of-binary-tree

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where `TreeNode` is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

APPROACH

BFS taking the last node of each level, or DFS visiting right before left and recording the first node seen at each depth. O(n).

HOW TO THINK ABOUT IT

The right side view is the last node of every level, or equivalently the first node you reach at each depth if you visit right children before left. Either way you need the depth of each node, so carry it in the traversal.

Basis: LeetCode 199, Binary Tree Right Side View.

BRUTE FORCE - PSEUDO-CODE

```
level-order traversal collecting a list of values for ever
y level
return the last value of each level
```

Time: O(n) | Space: O(n)

Why it is slow

Stores every node's value when only one per level is needed, O(n) space. A depth-first walk visiting the right child first and recording the first value at each depth needs O(h).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```python
def solution(level_order):
    # Build a binary tree from a LeetCode-style level-orde
r list
    # (None marks a missing child), then return its right-
side view.
    from collections import deque

    class TreeNode:
        def __init__(self, val):
            self.val = val
            self.left = None
            self.right = None

    def build(values):
        if not values or values[0] is None:
            return None
        root = TreeNode(values[0])
        q = deque([root])
        i = 1
        n = len(values)
        while q and i < n:
            node = q.popleft()
            if i < n:
                v = values[i]
                i += 1
                if v is not None:
                    node.left = TreeNode(v)
                    q.append(node.left)
            if i < n:
                v = values[i]
                i += 1
                if v is not None:
                    node.right = TreeNode(v)
                    q.append(node.right)
        return root

    root = build(level_order)
    if root is None:
        return []

    result = []
    q = deque([root])
    while q:
        size = len(q)
        for k in range(size):
            node = q.popleft()
            if k == size - 1:
                result.append(node.val)
            if node.left:
```

```
                q.append(node.left)
            if node.right:
                q.append(node.right)
    return result
```

Time: O(n) — each of the n nodes is enqueued and dequeued exactly once during the BFS (tree construction is also O(n)). | Space: O(n) — the queue can hold up to O(w) nodes where w is the maximum width of the tree (up to ~n/2 in the worst level), plus O(n) for the constructed tree itself.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def leftSideView(root):
    out = []

    def dfs(node, depth):
        if node is None:
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)

    dfs(root, 0)
    return out
```

Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

FUNCTION AND TEST CASES

Signature

```
def rightSideView(level_order: list[int | None]) -> list[int]:
```

Sample call

```
rightSideView([1, 2, 3, None, 5, None, 4])
```

Test cases

Input	Expected
`([1, 2, 3, None, 5, None, 4],)`	`[1, 3, 4]`
`([1, 2, 3, 4],)`	`[1, 3, 4]`

EASY | Exponent | 178 of 196

Given two trees, determine if the pre-order traversals of two trees are same or not.

STUDY: TYPE | Google frequency on LeetCode: all time 45.8, last 30 days 0 | book priority 70 of 196
Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: written and tested here for this exact question
Related in your app (a neighbouring problem, not this exact question): LeetCode #100 Same Tree
LeetCode match: LC 100 Same Tree (related: traversal comparison) (inferred from short statement)
Original: https://www.aced.io/questions/7182/compare-preorder-traversals-two-trees

QUESTION

Given two trees, determine if the pre-order traversals of two trees are same or not.

APPROACH

Generate both pre-order sequences (with null markers) and compare, or walk both trees together with recursion. O(n).

INTERVIEW-DAY PLAN

Clarify first

Ask whether they mean the same sequence of values or the same tree. Two different trees can have the same pre-order values.

Brute force to say out loud

Generate both pre-order lists and compare them. O(n) time and space.

Optimal idea

Generate each pre-order lazily with an explicit stack and compare the two streams in lockstep with zip_longest, so the first mismatch stops the work. O(min(n, m)) with early exit, O(h) space. For the structural version walk both trees together, or serialize pre-order with null markers.

Dry-run these cases

Same shape, different shape with the same pre-order, different lengths, empty trees, one empty.

Likely follow-ups

Same tree (structure and values), or O(1) space with a Morris traversal.

Pitfalls

Comparing without null markers when the structure matters.

HOW TO THINK ABOUT IT

Ask whether they want identical pre-order value sequences or identical trees; different trees can share a pre-order. For sequences, compare lazily so the first difference ends the work, instead of building two full lists.

Basis: LeetCode 100, Same Tree, is the structural version; this question asks only whether the pre-order sequences match.

BRUTE FORCE - PSEUDO-CODE

```
list_a = pre-order of tree a (node, then left, then right)
; list_b = the same for b
return list_a == list_b
```

Time: O(n + m) | Space: O(n + m)

Why it is slow

It always visits and stores both trees completely, O(n + m) time and space. Walking both pre-orders lazily in lockstep stops at the first difference with O(h) space.

OPTIMAL SOLUTION - LAZY PRE-ORDER GENERATORS COMPARED IN LOCKSTEP

```
class _N:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def same_preorder(a, b):
    def preorder(root):
        stack = [root] if root else []
        while stack:
            node = stack.pop()
            yield node.val
            if node.right:
                stack.append(node.right)
            if node.left:
                stack.append(node.left)

    sentinel = object()
    return all(
        x == y
        for x, y in itertools.zip_longest(preorder(a), pre
order(b), fillvalue=sentinel)
    )
```

Time: O(min(n, m)) with early exit, O(n + m) worst case | Space: O(h)

Why

Generate each pre-order lazily with an explicit stack (push right before left), and compare the two streams element by element with zip_longest, so the first mismatch stops the work and no list is built. Different lengths show up as a mismatch against the fill value. The two trees can have the same pre-order but different shapes, which is why this differs from Same Tree.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if the interviewer meant the trees must be identical?

Then compare structure as well as values: same_tree above walks both trees together and treats two missing nodes as equal. Equivalently, serialize pre-order with null markers and compare the strings, since pre-order with nulls is unique. Ask which definition they mean before coding.

Do it with O(1) extra space.

Morris traversal visits pre-order using temporary threads on right pointers, giving O(1) space, but it mutates the tree while running (restoring it after), so it is not safe for concurrent readers. Mention that trade-off.

They want identical trees, not just sequences.

Compare structure as well: walk both trees together and treat a missing child on one side only as a mismatch, or serialise pre-order with null markers and compare the strings.

They want identical trees (structure and values), not just the same pre-order values.

Compare the two trees together: a node must match in value and both children must match; two missing children match. Equivalently compare the pre-order serialisation with null markers (which is unique). O(min(n, m)) with early exit.

```
def same_tree_structure(a, b):
    stack = [(a, b)]
    while stack:
        x, y = stack.pop()
        if x is None and y is None:
            continue
        if x is None or y is None or x.val != y.val:
            return False
        stack.append((x.left, y.left))
        stack.append((x.right, y.right))
    return True
```

EASY | IN APP LC 104 | Exponent | 179 of 196

Calculate the height of a binary tree.

STUDY: TYPE | Google frequency on LeetCode: all time 40, last 30 days 33 | book priority 49 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: your app's interview approach (LeetCode #104)

LeetCode match: LC 104 Maximum Depth of Binary Tree (inferred from short statement)

Original:

https://www.aced.io/questions/1153/calculate-height-binary-tree

QUESTION

Calculate the height of a binary tree.

APPROACH

Recursive 1 + max(h(left), h(right)), or level-order BFS count. O(n).

HOW TO THINK ABOUT IT

Height is defined recursively: 1 plus the larger child height, and 0 for an empty tree. Say whether you count nodes or edges. A level-order traversal gives the same number without recursion, which is the safe choice for a very deep tree.

Basis: LeetCode 104, Maximum Depth of Binary Tree (and 543, 110 for the follow-ups).

BRUTE FORCE - PSEUDO-CODE

```
height(node): if empty: return 0
return 1 + the larger of height(left) and height(right)
```

Time: O(n) | Space: O(h) call stack

Why it is slow

Linear time, which is optimal; the call stack is as deep as the tree, so a chain of 100,000 nodes can overflow. Counting levels with a queue avoids recursion.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #104 MAXIMUM DEPTH OF BINARY TREE)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Maximum depth is the number of nodes on the longest root-to-leaf path.
An empty tree has depth 0. A single node has depth 1. Right?"

"[3,9,20,null,null,15,7]: longest path is 3->20->15 or 3->20->7, 3 nodes -> 3 ok
[1,null,2]: 1->2, 2 nodes -> 2 ok"
Any constraints on tree height or node count I should design around?

PHASE 3 — OPTIMIZE

Say: "Recursive DFS is already O(n) — let me walk through how depth propagates up.
The recursion computes depth BOTTOM-UP (post-order):
depth(null) = 0 <- base case: empty node has depth 0.
depth(node) = 1 + max(depth(node.left), depth(node.right))
How answers flow upward: each leaf returns 1. Its parent gets max(1,1)+1=2.
That parent's parent gets max(2, sibling_depth)+1, and so on up to the root.
The max() picks the longer branch. The +1 counts this node itself.
Time: O(n) — every node visited once. Space: O(h) — recursion stack depth.

PHASE 4 — CLEAN CODE

```
class _104_MaxDepth:
    def maxDepth(self, root: Optional['TreeNode']) -> in
t:
        def depth(node):
            if not node:
                return 0
            return max(depth(node.left), depth(node.righ
t)) + 1
        return depth(root)
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace maxDepth on [3,9,20,null,null,15,7]."
maxDepth(3): left=maxDepth(9), right=maxDepth(20).
maxDepth(9): left=maxDepth(None)=0, right=maxDepth(None)=0 -> 1+max(0,0)=1.
maxDepth(20): left=maxDepth(15)=1, right=maxDepth(7)=1 -> 1+max(1,1)=2.
maxDepth(3): 1+max(1,2) = 3 ok
Call stack (post-order): leaves return 1, their parents return 2, root returns 3.
Edge: root=None -> return 0 ok. Single node -> 1+max(0,0) = 1 ok
WHY 1+max(left,right): depth = this node + the deeper of its two subtrees.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n). Space O(h) call stack — O(log n) balanced, O(n) skewed.
BFS alternative: count levels with a queue — O(n) time, O(w) space (w=max width).
For very deep trees, BFS avoids recursion stack overflow.
Given more time I'd ask: what about minimum depth? Different — must reach a leaf,
so BFS is actually better there (stops at first leaf found).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Nodes or edges?

Height in nodes is what the code returns; in edges subtract one (an empty tree is -1 and a single node is 0). Say which you use before you code.

Diameter (the longest path between any two nodes).

Return the height from the recursion and update a global best with left + right at each node: one pass, O(n).

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def diameter(root):
    best = 0

    def depth(n):
        nonlocal best
        if n is None:
            return 0
        l, r = depth(n.left), depth(n.right)
        best = max(best, l + r)
        return 1 + max(l, r)

    depth(root)
    return best
```

Is the tree balanced?

Return the height, or -1 as soon as a subtree is unbalanced or the heights differ by more than one, so the whole check is one O(n) pass.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def is_balanced(root):
    def check(n):
        if n is None:
            return 0
        l = check(n.left)
        if l < 0:
            return -1
        r = check(n.right)
        if r < 0 or abs(l - r) > 1:
            return -1
        return 1 + max(l, r)

    return check(root) >= 0
```

OFFICIAL PAGE TEXT

Log in or sign up to answer this question.

EASY | Exponent | 180 of 196

Tree Node.

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 186 of 196

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (tree node classification)

+38 interviews

Original: https://www.aced.io/questions/3957/tree-node

QUESTION

Given a binary tree, identify which nodes are leaf nodes, root nodes or inner nodes. Leaf nodes: A node with no children Root nodes: A node with no parent Inner nodes: A node with both children and parent nodes You are given the following table tree_node_table: idp_idintegerinteger where p_id is the id of the parent node. Your output should contain the following columns: id, node_types ('Root', 'Inner', 'Leaf').No answers contributed by the community yet.Interview experiences7 shared‹›Amazon · 5 months ago"It went well and answered all questions. and did both leetcode questions asked one in each interview"Software Engineer · InternGoogle · 6 months ago"Pretty easy, just very long. For technical just need to do leetcode, no system design. Interviewers help you if you are stuck."Software Engineer · Entry Level / L3Microsoft · 6 months ago"What went well: The live coding and system architecture rounds went smoothly. I was able to articulate my design choices clearly, especially around API design, database strategy, and maintaining high availability for microservices. What didn't go well: The initial behavioral portion felt slightly rushed because we spent extra time discussing specific cross-functional project dynamics from my previous roles. How does this interview compare to other interviews you've done? It was significantly more interactive. Rather than giving static answers, the interviewers actively challenged my assumptions and asked "what if" scenarios, making the evaluat

APPROACH

Traverse the tree (DFS/BFS) and classify: root has no parent, leaf has no children, inner otherwise. O(n).

INTERVIEW-DAY PLAN

Clarify first

Ask about the table shape (id and p_id), whether a node with no parent is the root even if it has no children, and whether a single-node tree is Root.

Brute force to say out loud

Load the table into a program and classify with loops. It works but SQL is expected.

Optimal idea

A single query with CASE: Root when p_id is null; Inner when the node's id appears as someone's p_id and p_id is not null; otherwise Leaf. Use a subquery or EXISTS for the 'has children' test. O(n) with an index on p_id.

Dry-run these cases

A single-node tree, a chain, a star, two roots (bad data).

Likely follow-ups

Binary tree validation, or output in a specific order.

Pitfalls

Calling a root with children Inner. Using NOT IN with NULLs in the subquery (it returns nothing).

HOW TO THINK ABOUT IT

Each row is a node with its parent id. A node is the root if it has no parent, a leaf if nobody lists it as a parent, and inner otherwise. Order the tests so the root is decided first (a single-node tree is a root, not a leaf). Beware NOT IN with NULLs: it returns nothing if the subquery contains a NULL.

Basis: LeetCode 608, Tree Node. Run and checked on SQLite.

BRUTE FORCE - PSEUDO-CODE

```
select the nodes with no parent as 'Root'
select the nodes that have a parent and for which a child
exists as 'Inner' (EXISTS
    subquery per row)
select the nodes that have a parent and no child as 'Leaf'
; combine the three with
    UNION
```

Time: O(n^2) without an index on p_id | Space: O(n)

Why it is slow

The correlated subquery runs once per row and the table is scanned three times, O(n^2) without an index. One CASE expression with a single set of parent ids scans once.

OPTIMAL SOLUTION - ONE CASE EXPRESSION

```
SELECT id,
       CASE
           WHEN p_id IS NULL THEN 'Root'
           WHEN id IN (SELECT p_id FROM tree WHERE p_id IS
 NOT NULL) THEN 'Inner'
           ELSE 'Leaf'
       END AS node_types
FROM tree
ORDER BY id;
```

Time: O(n) with the set of parent ids built once | Space: O(n)

Why

CASE tests in order: no parent means Root, an id that appears as someone's parent means Inner, anything else is a Leaf. The subquery filters out NULL so the IN test is safe (NOT IN with a NULL would return no rows). The table is scanned once and the parent id set is built once.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why not use NOT IN for the leaves?

If the subquery returns a NULL (the root's p_id), id NOT IN (...) is never true, so you get no rows. Filter NULLs out of the subquery or use NOT EXISTS.

The table has no row for the root's parent and p_id can point to missing ids.

Validate with a LEFT JOIN to find orphans. A node whose p_id is not NULL but missing from the table is a data error, not a root; decide whether to report it.

Add the depth of each node.

A recursive CTE starting from the root rows and joining children on p_id, carrying depth + 1, gives every node's depth in one query.

Add the depth of each node.

A recursive CTE starts from the root rows (depth 0) and joins the table to itself on p_id, adding 1 at every step; each node appears once because every node has one parent. Runs in one query, O(n) with an index on p_id.

```
def node_depths(rows):
    conn = sqlite3.connect(":memory:")
    conn.execute("CREATE TABLE tree(id INTEGER PRIMARY KEY
, p_id INTEGER)")
    conn.executemany("INSERT INTO tree VALUES (?, ?)", row
s)
    q = """WITH RECURSIVE d(id, depth) AS (SELECT id, 0 FR
OM tree WHERE p_id IS NULL
                                        UNION ALL SELEC
T t.id, d.depth + 1 FROM tree t JOIN d ON t.p_id = d.id)
           SELECT id, depth FROM d ORDER BY id"""
    return conn.execute(q).fetchall()
```

OFFICIAL PAGE TEXT

Given a binary tree, identify which nodes are leaf nodes, root nodes or inner nodes. Leaf nodes: A node with no children Root nodes: A node with no parent Inner nodes: A node with both children and parent nodes You are given the following table tree_node_table: idp_idintegerinteger where p_id is the id of the parent node. Your output should contain the following columns: id, node_types ('Root', 'Inner', 'Leaf').No answers contributed by the community yet.

Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns 10 questions

EASY | IN APP LC 729 | PracHub | Technical Screen | 181 of 196

Check an Interval Against Existing Intervals for Conflicts

STUDY: TALK | Google frequency on LeetCode: all time 26.5, last 30 days 0 | book priority 120 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

APPROACH

Half-open intervals [s,e) conflict iff new.start < e and s < new.end (touching endpoints do not conflict). One-off check on an unsorted collection is a single O(n) scan; for repeated queries keep a sorted structure (TreeMap/balanced BST or sorted list + bisect) and check only the neighbors. O(log n) per query.

HOW TO THINK ABOUT IT

Two half-open intervals [s1, e1) and [s2, e2) overlap exactly when s1 < e2 and s2 < e1. If the booked intervals are kept sorted and disjoint, only the neighbours at the insertion point can conflict, so a binary search replaces the scan.

Basis: LeetCode 729, My Calendar I (booking succeeds only when the new half-open interval overlaps nothing), and 731 for double booking.

BRUTE FORCE - PSEUDO-CODE

```
book(s, e): for every booked interval (s2, e2): if s < e2
and s2 < e: reject
otherwise add (s, e) to the list
```

Time: O(n) per booking | Space: O(n)

Why it is slow

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def hasIntervalConflict(candidate, intervals):
    a, b = candidate
    return any(a < e and s < b for s, e in intervals)
```

Time: O(n) | Space: O(1)

Explanation

Overlap holds when each start is before the other end.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

```
class CalendarTwo:
    """At most two overlapping bookings at any time (LeetCode 731)."""

    def __init__(self):
        self.delta = defaultdict(int)

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        running = 0
        for t in sorted(self.delta):
            running += self.delta[t]
            if running > 2:
                self.delta[start] -= 1
                self.delta[end] += 1
                return False
        return True
```

Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: list[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
`([4,7],[[1,3],[7,9]])`	`False`
`([4,7],[[1,5],[8,9]])`	`True`

EASY | PracHub | Technical Screen | 182 of 196

Implement string formatting and request tracking tasks

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 145 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: LC 246 Strobogrammatic Number (+ formatting and tracking subtasks) (inferred from overview)

Original: https://prachub.com/interview-questions/implement-string-formatting-and-request-tracking-tasks

QUESTION

Overview: This multi-part question evaluates string manipulation, numeric formatting, character-mapping validation under 180° rotation, and event-tracking/order management competencies, assessing parsing, edge-case handling, mapping validation, and design of data structures for ordered output.

APPROACH

Multi-part: numeric/string formatting, validating a number that reads the same after a 180 degree rotation (digit map 0-0, 1-1, 6-9, 8-8, 9-6, two-pointer check), and an ordered request tracker (dict + insertion order). Solve each part independently.

INTERVIEW-DAY PLAN

Clarify first

This is four small tasks, so confirm each contract. For the rotation task ask what the rotation map contains and whether leading zeros are allowed. For the tracker ask what 'print in order' means when a later request can still have an earlier end time.

Brute force to say out loud

For each part write the direct version first: split and join, loop with commas, check both readings, and print immediately.

Optimal idea

Snake to camel: split on underscores, drop empty parts, capitalize all but the first. Thousands separators: format the absolute value in groups of three from the right and restore the sign. OCR check: validate as-is and after reversing and mapping each character, reject unmapped characters. Tracker: a map for the first timestamp, a min-heap of completed (end, id) and a drain with a watermark.

Dry-run these cases

snake: leading, trailing and repeated underscores. Commas: 0, negative, 64-bit minimum. OCR: unmapped letter, value 201, value 0. Tracker: end before begin, equal end times.

Likely follow-ups

Locale-aware formatting, rotation maps given at runtime, and bounded memory in the tracker.

Pitfalls

Negating the minimum 64-bit integer. Printing a request before it is safe to order.

HOW TO THINK ABOUT IT

Four independent tasks, so solve and test each separately and say the contract for each before coding: what an empty or negative input returns, what 'valid' means for the rotated number, and for the tracker when it is safe to print in order. For the tracker, completion is the second event of a pair, and order printing is a heap that is drained only below a watermark.

Basis: Four small tasks: snake_case to camelCase, thousands separators, a rotated-OCR check (LeetCode 246 / 1056 flavor), and a request begin/end tracker that prints in end-time order.

BRUTE FORCE - PSEUDO-CODE: TASK A BY WALKING CHARACTERS

```
result = ''; up = False; for each character: if it is '_':
 up = True, else append it
    (uppercased if up) and set up = False
```

Time: O(n) | Space: O(n)

Why it is slow

Linear and fine; the split-based version is shorter. The only care is leading and repeated underscores.

BRUTE FORCE - PSEUDO-CODE: TASK B WITH THE LANGUAGE'S FORMAT SPECIFIER

```
return format(n, ',') (the language's built-in thousands s
eparator)
```

Time: O(d) | Space: O(d)

Why it is slow

Correct and O(d), but it hides the algorithm; interviews usually ask for the grouping by hand: peel three digits at a time from the right.

BRUTE FORCE - PSEUDO-CODE: TASK C BY CHECKING ALL THREE READINGS WITH A STRAIGHTFORWARD HELPER

```
if the text is a number in 1..200: return true
if every character has a rotation: rotated = reverse the t
ext and map each character
return whether rotated is a number in 1..200
```

Time: O(L) | Space: O(L)

Why it is slow

It is already linear; there is little to speed up. The risk is correctness: unmapped characters make the rotation invalid, and leading zeros are not valid numbers.

BRUTE FORCE - PSEUDO-CODE: TASK D BY STORING ALL EVENTS AND SORTING WHEN ASKED

```
store begin and end times per request id as they arrive (i
n any order)
when asked to print: pair the ones with both times, sort b
y (end time, id), print and
    remove them
```

Time: O(n log n) per print | Space: O(n)

Why it is slow

Each print sorts everything pending, O(n log n); a min-heap of completed requests prints in order with O(log n) per event, releasing only entries below a safe watermark.

OPTIMAL SOLUTION - TASK A: SNAKE_CASE TO CAMELCASE

```python
def snake_to_camel(s):
    words = [w for w in s.split("_") if w]
    return words[0] + "".join(w.capitalize() for w in word
s[1:]) if words else ""
```

Time: O(n) | Space: O(n)

Why

Split on '_' and drop empty parts, which handles leading, trailing and repeated underscores in one move. The first word stays lowercase and every later word is capitalized.

OPTIMAL SOLUTION - TASK B: THOUSANDS SEPARATORS

```python
def format_thousands(n):
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts = []
    while digits:
        parts.append(digits[-3:])
        digits = digits[:-3]
    return sign + ",".join(reversed(parts))
```

Time: O(d) | Space: O(d)

Why

Format the absolute value digit by digit from the right, inserting a comma before every third digit, and put the sign back. Working on the absolute value of a Python int is safe for the full 64-bit range; in C++ or Java negate through a wider type because abs(INT64_MIN) overflows.

OPTIMAL SOLUTION - TASK C: VALID AS-IS OR AFTER A 180 DEGREE ROTATION

```python
def ocr_valid(t, rot):
    def in_range(x):
        return bool(re.fullmatch(r"[1-9]\d*", x)) and 1 <=
 int(x) <= 200

    def rotate(x):
        if any(c not in rot for c in x):
            return None
        return "".join(rot[c] for c in reversed(x))

    rotated = rotate(t)
    return in_range(t) or (rotated is not None and in_rang
e(rotated))
```

Time: O(L) | Space: O(L)

Why

Rotating a string means reversing it and mapping every character through the rotation table; any unmapped character makes the rotation invalid. The answer is true if the original or the rotated string is a base-10 integer in [1, 200]. Reject leading zeros and signs by requiring an exact digits-only match with no leading zero.

OPTIMAL SOLUTION - TASK D: REQUEST TRACKER THAT PRINTS COMPLETED REQUESTS IN END-TIME ORDER

```
class Server:
    def __init__(self):
        self.pending = {}  # id -> ("begin"|"end", ts)
        self.ready = []  # heap of (end, id, begin)
        self.printed = []

    def begin(self, rid, ts):
        self._event(rid, "begin", ts)

    def end(self, rid, ts):
        self._event(rid, "end", ts)

    def _event(self, rid, kind, ts):
        if rid not in self.pending:
            self.pending[rid] = (kind, ts)
            return
        other_kind, other_ts = self.pending.pop(rid)
        begin, end = (ts, other_ts) if kind == "begin" els
e (other_ts, ts)
        heapq.heappush(self.ready, (end, rid, begin))

    def drain(self, watermark=None):
        out = []
        while self.ready and (watermark is None or self.re
ady[0][0] <= watermark):
            end, rid, begin = heapq.heappop(self.ready)
            line = f"{rid} begin={begin} end={end}"
            out.append(line)
            self.printed.append(line)
        return out
```

Time: O(log n) per event | Space: O(n)

Why

Store the first timestamp seen per request id in a pending map, whatever order begin and end arrive in. When the second one arrives the request is complete: push (end, id, begin) onto a min-heap. Printing is a separate step: pop while the earliest end is at or below a watermark, because an unfinished request could still produce an earlier end. With no watermark, drain() prints everything in (end, id) order.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. Cost: O(log n) per event for the eviction heap; memory O(requests inside the retention window).

Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

```
def format_grouped(n, sep=",", group=3, first_group=None):
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts = []
    size = first_group or group
    while len(digits) > size:
        parts.append(digits[-size:])
        digits, size = digits[:-size], group
    parts.append(digits)
    return sign + sep.join(reversed(parts))
```

FULL QUESTION TEXT FROM THE SITE

You are given several small coding tasks. Implement each as described.

Task A — Convert snake_case to camelCase

Write a function that converts an identifier from snake_case to camelCase.

Input: a string s containing lowercase letters a-z and underscores _ .
Output: the camelCase version.
Rules/edge cases:
Consecutive underscores should be treated as a single word separator (ignore empty parts).
If s starts/ends with underscores, ignore those underscores.
The first word remains lowercase; each subsequent word is capitalized.

Example:

"hello_world" -> "helloWorld"
"__my__var__name_" -> "myVarName"
Task B — Format an integer with thousands separators

Write a function that formats a (possibly negative) integer with commas as thousands separators.

Input: an integer n (may be large; assume it fits in 64-bit signed integer).
Output: a string representation using commas every 3 digits from the right.

Examples:

123 -> "123"
1234 -> "1,234"
-12345678 -> "-12,345,678"
Task C — Validate an OCR string under 180° rotation

An OCR system outputs a string t that may contain digits and a few letters. You need to decide whether it can represent an integer

between 1 and 200 inclusive under either normal reading or after a 180-degree rotation.

Define a rotation mapping rot(c) for characters; rotation is valid only if every character in the string has a mapping. Rotating a whole string means:

reverse the string, and
replace each character c with rot(c) .

Return true if either:

t as-is is a valid base-10 integer in [1, 200] , OR
rotate180(t) is a valid base-10 integer in [1, 200] .

Assume the rotation map is provided as a dictionary (examples from the interview note included):

'6' <-> '9'
plus any additional pairs given at runtime (e.g., 'h' -> '4' , 'L' -> '7').
Characters not in the map are invalid for rotation.

Examples (assuming map includes 6<->9, 0->0, 1->1, 8->8):

"69" rotates to "69" and is valid (69 in range) -> true
"201" is out of range, rotate may or may not help -> depends on mapping -> compute
Task D — Track request begin/end times and print in a required order

You are implementing a Server class that receives events for requests. Each request has a unique requestId.

You must implement two methods:

begin(requestId, timestamp)
end(requestId, timestamp)
Part 1

Whenever both timestamps for a request are known, print/log a line:

requestId begin=<beginTs> end=<endTs>

Assumptions:

Events may arrive out of order (an end can arrive before begin).
There may be multiple in-flight requests.
Follow-up

Instead of printing immediately when a pair is complete, the server must always print completed requests in increasing order of end time (earliest end printed first). If multiple have the same end time, break ties by requestId.

Design the data structures and logic to support this ordering efficiently.

EASY | IN APP LC 125 | Exponent | 183 of 196

Is this a valid palindrome?

STUDY: TYPE | Google frequency on LeetCode: all time 52.8, last 30 days 51.4 | book priority 21 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #125)

LeetCode match: LC 125 Valid Palindrome

+612 interviews

Original:
https://www.aced.io/questions/4056/validate-palindrome-check

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

APPROACH

Two pointers from both ends skipping non-alphanumerics and comparing case-insensitively. O(n), O(1).

HOW TO THINK ABOUT IT

A palindrome is symmetric, so compare the two ends and move inward; there is no need to build the reversed string. Decide what to ignore (non-alphanumerics, case) and skip those characters with the pointers instead of copying a cleaned string.

Basis: LeetCode 125, Valid Palindrome, and 680 (Valid Palindrome II).

BRUTE FORCE - PSEUDO-CODE

```
t = lowercase letters and digits of s only
return t == reverse(t)
```

Time: O(n) | Space: O(n)

Why it is slow

O(n) time but two extra copies of the string, O(n) space. Two pointers moving inward, skipping punctuation, need O(1) space and

stop at the first mismatch.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #125 VALID PALINDROME)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
We strip everything except letters and digits, lowercase it all, then check if the result reads the same forwards and backwards. Right?"

"A few quick questions:
Alphanumeric means letters AND digits — not just letters?
So '0P' after cleaning is '0p', which is not a palindrome — confirmed?
What about a string of only spaces or punctuation?
Example 3 shows a single space returns true — after cleaning it's an empty string,
and an empty string is considered a palindrome. Good.
Can the string be length 1? Yes — a single alphanumeric character is a palindrome."

"Let me walk example 1:
'A man, a plan, a canal: Panama' -> 'amanaplanacanalpanama' -> palindrome ok
Example 2: 'race a car' -> 'raceacar' -> not a palindrome ok

PHASE 3 — OPTIMIZE

The brute force is already O(n) time, but uses O(n) space for the cleaned copy.
We can do this in O(1) extra space using two pointers directly on the original string.

Place left at 0 and right at len(s)-1.
Advance left past any non-alphanumeric characters.
Retreat right past any non-alphanumeric characters.
Compare s[left].lower() and s[right].lower().
If they differ, return False. Otherwise move both pointers inward.
If left meets or passes right, the string is a palindrome.

The key benefit: we never build a second string — we skip junk characters on the fly.

Pseudocode:
left, right = 0, len(s)-1
while left < right:
while left < right and not s[left].isalnum(): left += 1
while left < right and not s[right].isalnum(): right -= 1
if s[left].lower() != s[right].lower(): return False
left += 1; right -= 1
return True

Time: O(n). Space: O(1). Does that make sense before I code it?

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _125_ValidPalindrome:
    def isPalindrome(self, s: str) -> bool:
        left  = 0
        right = len(s) - 1

        while left < right:
            while left < right and not s[left].isalnum()
:   # skip non-alphanumeric
                left += 1
            while left < right and not s[right].isalnum(
):  # skip non-alphanumeric
                right -= 1

            if s[left].lower() != s[right].lower():
                return False

            left  += 1
            right -= 1

        return True
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace isPalindrome('A man, a plan, a canal: Panama')."
Filter alphanumeric, lowercase: 'amanaplanacanalpanama'.
Two pointers: left=0, right=20.
left='a', right='a' ok. left='m', right='m' ok. ... All match.
Return True ok
'race a car': filtered='raceacar'. left='r',right='r'ok.
left='a',right='a'ok. left='c',right='c'ok. left='e'!='a'->False ok

WHY filter+lowercase first: then standard palindrome two-pointer on clean string.
'': empty string after filtering -> True (vacuously palindrome) ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n) — each character is visited at most once by left or right pointer.
Space O(1) — no auxiliary storage beyond two integer pointers.

The brute force one-liner is perfectly valid for an interview too — it's readable
and still O(n) time. The two-pointer version wins purely on space: O(1) vs O(n).
If the interviewer asks about the follow-up, lead with the space trade-off.

isalnum() handles both letters and digits in one call — worth naming explicitly
so the interviewer knows you're not just checking isalpha().

Subtle trap: the inner while loops must guard left < right to avoid crossing
pointers — if you forget, you can read out-of-bounds when the string is all
non-alphanumeric (like the single-space example).

Given more time I'd ask: what if the string could contain unicode characters
like accented letters (é, ñ)? isalnum() in Python handles unicode correctly
by default, so our solution already works — worth calling out.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

You may delete at most one character (LeetCode 680).

Move inward until the first mismatch, then the answer is whether skipping the left character or skipping the right character leaves a palindrome. That is two more linear checks, so O(n) overall.

```
def valid_palindrome_one_deletion(s):
    def pal(i, j):
        while i < j:
            if s[i] != s[j]:
                return False
            i, j = i + 1, j - 1
        return True

    i, j = 0, len(s) - 1
    while i < j:
        if s[i] != s[j]:
            return pal(i + 1, j) or pal(i, j - 1)
        i, j = i + 1, j - 1
    return True
```

Unicode, accents, or locale-specific case.

Normalise first (unicodedata.normalize and casefold) and decide what counts as a letter. str.isalnum handles many scripts, but combining marks need normalisation.

Longest palindromic substring.

Expand around each of the 2n - 1 centers in O(n^2), or use Manacher's algorithm in O(n). The two-pointer comparison is the building block.

OFFICIAL PAGE TEXT

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples

```
Input: s = "H123!@321h"
Output: true

Input: s = "race a car"
Output: false
```

Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their reversed copy. This space is needed to hold the filtered string and its reversed version. Solution 2: Two-Pointer Approach This solution uses a two-pointer technique to efficiently check if the string is a palindrome. The idea is to use one pointer starting from the beginning of the string (left) and another pointer from the end (right). The pointers move towards each other, skipping non-alphanumeric characters and comparing characters that are alphanumeric. Pointer Initialization: Set two pointers: left at the beginning and right at the end of the string. Skipping Non-Alphanumeric Characters: Move the left pointer to the right while skipping non-alphanumeric characters, and similarly move the right pointer to the left. Comparison: Compare the characters at left and right in a case-insensitive manner. If characters do not match, the string is not a palindrome. Update Pointers: Move the pointers closer to each other and repeat until the pointers meet or cross. Time Complexity: The time complexity is O(n), where n is the length of the string. Each character is processed at most twice (once by each pointer). Space Complexity: The space complexity is O(1) as it only uses a constant amount of extra space for the two pointers, regardless of the size of the input string. NoteCheck out our lesson on the two-pointer approach to learn more about it and its applications!No answers contributed by the community yet.

EASY | IN APP LC 186 | Exponent | 184 of 196

Reverse a Sentence

STUDY: TALK | Google frequency on LeetCode: none listed (not on its Google list) | book priority 164 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #186)

LeetCode match: LC 186 Reverse Words in a String II

+38 interviews

Original: https://www.aced.io/questions/3062/reverse-a-sentence

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge

4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

APPROACH

Reverse the whole char array, then reverse each word in place. O(n) time, O(1) space.

HOW TO THINK ABOUT IT

Reversing the order of words but not the letters is the same as reversing everything and then reversing each word back. That is two passes of the same helper and needs no extra memory when the sentence is a mutable character array. First decide how to treat repeated spaces.

Basis: LeetCode 151 (Reverse Words in a String) and 557; the in-place version is the classic array trick.

BRUTE FORCE - PSEUDO-CODE

```
words = split the sentence on whitespace
reverse the list of words and join with single spaces
```

Time: O(n) | Space: O(n)

Why it is slow

O(n) time but extra space for the word list, and it collapses repeated spaces. Reversing the whole character array and then each word in place needs O(1) extra space.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #186 REVERSE WORDS IN A STRING II)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Given a character ARRAY (not a string), reverse the order of words in-place.
Words are separated by single spaces. No leading/trailing spaces. Right?
['t','h','e',' ','s','k','y'] -> ['s','k','y',' ','t','h','e'] ok"
Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 3 — OPTIMIZE

Say: "Two-pointer reverse entire array, then reverse each word — O(n). Walk-through:
In-place for char array:
1. Reverse whole array: swap s[0..n-1].
2. Find each word (bounded by spaces): reverse it in place.
For each word: lo = word start, hi = word end. While lo<hi: swap(s[lo],s[hi]); lo++; hi--.
WHY reverse-then-reverse-words: reversing the whole array puts words in right order but
each word itself is reversed. Re-reversing each word restores their individual characters.
Example: 'the sky is' -> reverse all -> 'si yks eht' -> reverse each -> 'is sky the' ok
Time: O(n). Space: O(1).

PHASE 4 — CLEAN CODE

```python
class _186_ReverseWordsII:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(left, right):
            while left < right:
                s[left], s[right] = s[right], s[left]
                left += 1; right -= 1

        reverse(0, len(s) - 1)
        start = 0
        for i in range(len(s) + 1):
            if i == len(s) or s[i] == ' ':
                reverse(start, i - 1)
                start = i + 1
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace how the left/right pointers move and why the invariant stays true."
s=['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']:
Step 1 reverse all: ['e','l','u','b',' ','s','i',' ','y','k','s',' ','e','h','t']
Step 2 reverse words:
i=0..3 'e','l','u','b' -> reverse -> 'b','l','u','e'
i=5..6 's','i' -> 'i','s'
i=8..10 'y','k','s' -> 's','k','y'
i=12..14 'e','h','t' -> 't','h','e'
-> ['b','l','u','e',' ','i','s',' ','s','k','y',' ','t','h','e'] ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n). Space O(1) — two reversal passes, no extra memory.
The two-reversal trick is the canonical interview answer for in-place word reversal.
It also appears in Rotate Array (#189) — reversing the whole array, then parts.
Given more time I'd ask: what if words can be separated by multiple spaces?
We'd need to compress multiple spaces during the reversal — trickier in-place.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The input may have leading, trailing or repeated spaces.

The split-based version already normalises them; the in-place version keeps them, so say which you want. To normalise in place add a two-pointer pass that writes single spaces.

Reverse the letters of each word but keep the word order (LeetCode 557).

Split on single spaces and reverse each word, or reverse each run in place without the first full reversal.

```python
def reverse_each_word(s):
    return " ".join(w[::-1] for w in s.split(" "))
```

The sentence is a linked list of characters.

Reverse each word segment, then reverse the whole list, using the pointer-flip routine for linked lists.

OFFICIAL PAGE TEXT

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example

```
input:  arr = [ 'p', 'e', 'r', 'f', 'e', 'c', 't', '  ',
                'm', 'a', 'k', 'e', 's', '  ',
                'p', 'r', 'a', 'c', 't', 'i', 'c', 'e' ]

output: [ 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e', '  ',
          'm', 'a', 'k', 'e', 's', '  ',
          'p', 'e', 'r', 'f', 'e', 'c', 't' ]
```

Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible solution is doing a linear iteration on arr while pushing a copy of every word to a stack and then pulling them in reverse order while copying the words back into arr. Another approach is initializing a new array at the same length, iterating over arr from right to left, and copying any sequence of characters to the new array from left to right. Both approaches take O(N) time and at least O(N) space. A more elegant and efficient approach is to reverse all the characters in arr and then reverse the characters in each word separately. While the first reverse gives us the words in the reverse order as we wanted, it also reverses the characters of each word. To fix that, we do the second reverse, which reverses each word separately. Reversing items in an array is done by a 'mirror' function, that swaps the items of every 2 indices with the same distance from the middle. Pseudocode:

```
function reverseWords(arr):
    # reverse all characters:
    n = arr.length
    mirrorReverse(arr, 0, n-1)

    # reverse each word:
    wordStart = null
    for i from 0 to n-1:
        if (arr[i] == ' '):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i-1)
                wordStart = null
        else if (i == n-1):
            if (wordStart != null):
                mirrorReverse(arr, wordStart, i)
        else:
            if (wordStart == null):
                wordStart = i
    return arr

# helper function - reverses the order of items in arr
# please note that this is language dependent:
# if are arrays sent by value, reversing should be done in
 place

function mirrorReverse(arr, start, end):
    tmp = null
    while (start < end):
        tmp = arr[start]
        arr[start] = arr[end]
        arr[end] = tmp
        start++
        end--
```

Time Complexity: traversing the array twice with a constant number of actions for each item is linear O(N). Space Complexity: using iteration indices and one temp variable takes constant O(1) memory. Answer to bonus question: The mirrorReverse function could be implemented with a single index with left to right linear iteration and swapping arr[i] and arr[n-1-i] as long as i < n-1-i. No answers contributed by the community yet.Related questionsReverse a linked list.Related coursesCourseCourseMachine Learning Engineer Interview PrepLand your dream machine learning role at Meta, Google, Amazon, Apple, Microsoft, Nvidia, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Explore ML system design, core concepts, coding, behavioral interviews, and more.Start learningCourseCourseData Science Interview PrepLand your dream data science role at Google, Amazon, Microsoft, Meta, Apple, and other top companies. Learn from mock interviews, frameworks, and advice from senior candidates. Practice statistics, experimentation, coding, SQL, machine learning, behavioral interviews, and more.Start learningPythonJavaScriptC#JavaC++CGoRubySwiftPythonEditorShareLoading...OutputTest ResultsConsoleSign up to run code

---

EASY | Exponent | 185 of 196

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 146 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 500 Keyboard Row (inferred from short statement)

Original:
https://www.aced.io/questions/7293/return-words-one-keyboard-row

QUESTION

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

APPROACH

Map each letter to its keyboard row; keep words whose letters all belong to one row. O(total chars).

INTERVIEW-DAY PLAN

Clarify first

Ask about case (treat as case-insensitive), empty strings, and non-letter characters.

Brute force to say out loud

For each word test all three rows with string containment. Fine and short.

Optimal idea

Map each letter to its keyboard row once (26 entries). A word qualifies if the set of rows of its lowercased letters has one element. O(total characters).

Dry-run these cases

Mixed case such as Alaska, a word using two rows, an empty word, a one-letter word.

Likely follow-ups

Different keyboard layouts as a parameter.

Pitfalls

Forgetting to lowercase before looking up rows.

HOW TO THINK ABOUT IT

Each letter belongs to exactly one row, so the question for a word is whether all its letters map to the same row. Build the letter-to-row lookup once and check each word with a set of rows.

Basis: LeetCode 500, Keyboard Row.

BRUTE FORCE - PSEUDO-CODE

```
rows = the three keyboard rows as sets of letters
for each word: keep it if all its lowercase letters are in
 row 1, or all in row 2, or
    all in row 3
```

Time: O(W * L * 3) | Space: O(1)

Why it is slow

Up to three scans per word; building one letter-to-row table makes each word a single scan, O(total characters).

OPTIMAL SOLUTION - MAP EACH LETTER TO ITS ROW AND COMPARE SETS

```
def findWords(words):
    rows = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]
    row_of = {ch: i for i, row in enumerate(rows) for ch i
n row}
    return [w for w in words if len({row_of[c] for c in w.
lower()}) <= 1]
```

Time: O(total characters) | Space: O(1)

Why

Build a dict from letter to row index (26 entries). A word qualifies if all of its lowercased letters map to the same row, i.e. the set of rows has size one. Lowercase first so 'Alaska' works.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?

Nothing changes except the row table, which is built from the layout argument. Build the letter-to-row map once and reuse it across queries.

Other layouts (Dvorak, AZERTY) or a phone keypad.

Pass the rows as a parameter and build the letter-to-row dict from it. Nothing else changes.

A different keyboard layout.

Pass the rows as a parameter and build the letter-to-row map from it (a phone keypad or Dvorak works the same way). Letters that are on no row make the word invalid, which the code checks.

```
def find_words_layout(words, rows):
    row_of = {ch: i for i, row in enumerate(rows) for ch i
n row}
    return [
        w
        for w in words
        if len({row_of.get(c) for c in w.lower()}) <= 1
        and all(c in row_of for c in w.lower())
    ]
```

EASY | Exponent | 186 of 196

Find the Duplicates

STUDY: TALK | Google frequency on LeetCode: all time 28.2, last 30 days 0 | book priority 118 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question):
LeetCode #350 Intersection of Two Arrays II
LeetCode match: LC 350 Intersection of Two Arrays II
+15 interviews

Original: https://www.aced.io/questions/3056/find-the-duplicates

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted

array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

APPROACH

Two pointers when sizes are similar (O(N+M), O(1) space). When one array is much smaller, binary search each element of the small array in the large one (O(M log N)). Skip repeats to keep output unique.

INTERVIEW-DAY PLAN

Clarify first

Ask whether duplicates within one array count once in the output, whether the output must be sorted (yes), the relative sizes N and M (the statement asks for two cases), and the value range.

Brute force to say out loud

Put one array in a set and scan the other. O(N + M) time, O(min(N, M)) space, fine but ignores the sorted input.

Optimal idea

Similar sizes: two pointers, advancing the smaller value, collecting equal values and skipping repeats. O(N + M) time, O(1) extra space. M much bigger than N: for each element of the small array binary search the large one, O(N log M), which beats O(N + M) when N is small. Exponential (galloping) search from the previous position gives O(N log(M / N)).

Dry-run these cases

No common values, all common, repeated values in one array, an empty array, one array a single element.

Likely follow-ups

Three arrays, unsorted arrays, or arrays too big for memory (merge from files).

Pitfalls

Using O(M log N) for the second case: you search the large array N times, so it is O(N log M). Returning the same value twice.

HOW TO THINK ABOUT IT

Both arrays are sorted, so the question is how the two sizes compare. If they are about the same size, walk both with two pointers and advance the smaller one. If one is much smaller, do not walk the big one: binary search each element of the small array in the big array. Always say the cost in terms of both sizes N and M.

Basis: Intersection of two sorted arrays (LeetCode 349 / 350). Output: values present in both arrays, sorted, each once.

BRUTE FORCE - PSEUDO-CODE

```
for each element x of the first array: for each element y
of the second: if x == y:
    record x (once)
sort the results
```

Time: O(N * M) | Space: O(1) extra

Why it is slow

N * M comparisons. Two pointers on the sorted arrays are O(N + M), and binary searching the big array for each element of the small one is O(N log M).

OPTIMAL SOLUTION - CASE 1 (M CLOSE TO N): TWO POINTERS

```python
def duplicates_two_pointers(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not out or out[-1] != a[i]:
                out.append(a[i])
            i, j = i + 1, j + 1
    return out
```

Time: O(N + M) | Space: O(1) extra

Why

Advance whichever pointer points at the smaller value; when equal, record the value once (skip repeats) and advance both. Each element is passed at most once.

OPTIMAL SOLUTION - CASE 2 (M MUCH BIGGER THAN N): BINARY SEARCH THE BIG ARRAY

```python
def duplicates_binary_search(small, big):
    out, lo = [], 0
    for x in small:
        if out and out[-1] == x:
            continue
        i = bisect_left(big, x, lo)
        if i < len(big) and big[i] == x:
            out.append(x)
        lo = i
    return out
```

Time: O(N log M) | Space: O(1) extra

Why

For each element of the small array, binary search the large array. This beats O(N + M) when N log M is smaller than M. Resume each search from the previous position (galloping search) for O(N log(M / N)).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Three or more arrays.

Intersect pairwise, smallest arrays first so the running result shrinks quickly: O(total size). With K arrays you can also keep K pointers and advance the smallest.

```
def duplicates_two_pointers(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not out or out[-1] != a[i]:
                out.append(a[i])
            i, j = i + 1, j + 1
    return out


def duplicates_in_all(arrays):
    out = arrays[0] if arrays else []
    for arr in arrays[1:]:
        out = duplicates_two_pointers(out, arr)
    return out
```

The arrays are too big for memory.

Read both as streams (two pointers need only the current element of each), or partition both by value range into files that fit in memory and intersect partition by partition. Cost: O(N + M) time and O(1) memory with two streaming pointers; partitioning by value range multiplies the disk passes by the number of partitions.

Which one do you choose when N = 1,000 and M = 10^9?

Binary search: N log M is about 30,000 steps while two pointers need 10^9. The break-even is when N log M is around M.

OFFICIAL PAGE TEXT

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example

```
input:  arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20]

output: [3, 6, 7] # since only these three values are both
 in arr1 and arr2
```

Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices, let's say i without any loss of generality, points to a value that is smaller than the value pointed by other index, we increment i. If arr1[i] == arr2[j], we add the value to the output array and increment both indices. Pseudocode:

```
function findDuplicates(arr1, arr2):
   duplicates = []
   i = 0
   j = 0

   while (i < arr1.length AND j < arr2.length):
      if (arr1[i] == arr2[j]):
         duplicates.push(arr1[i])
         i = i + 1
         j = j + 1
      else if (arr1[i] < arr2[j]):
         i = i + 1
      else:
         j = j + 1

   return duplicates
```

Time Complexity: O(N+M) since in the worst case scenario we traverse both arrays once. Note that O(N+M) is a linear time complexity because the input size is O(N+M) as well. Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. Case 2 (M >> N) When one array is substantially longer than the other, we should try to avoid traversing the longer one. Instead, we can traverse the shorter array and look up its values in the longer array by using the binary search algorithm. We explain why this approach is superior in this case to the previous one in the complexity analysis section. Pseudocode:

```
function findDuplicates(arr1, arr2):
    duplicates = []

    for number in arr1:
        if binarySearch(arr2, number) != -1:
            duplicates.push(number);

    return duplicates

function binarySearch(arr, num):
    begin = 0
    end = arr.length - 1

    while (begin <= end):
        mid = begin + floor((end-begin)/2)
        if arr[mid] < num:
            begin = mid + 1
        else if arr[mid] == num:
            return mid
        else:
            end = mid - 1

    return -1
```

Time Complexity: we running a binary search on arr2 N times. Hence the time complexity is O(N*log(M)). So why is this algorithm better than the previous one when M >> N? To demonstrate that let's analyze the case that M = N^C, where C is a constant such that C >= 2. In this case, the time complexity of the second algorithm is:

```
O(N*log(M)) = O(N*log(N^C)) = O(C*N*log(N)) = O(N*log(N))
```

And the time complexity of the first algorithm is:

```
O(N + M) = O(N + N^C) = O(N^C)
```

As we all know O(N^C) > O(N*log(N)). Space Complexity: the variable duplicates is the only dynamic auxiliary space we're using in the algorithm. In the worst case scenario, the size of duplicates is going to be as big as big as the smaller input array. For instance, when the smaller array is fully contained within the bigger one. The space complexity is therefore O(N), where N <= M. No answers contributed by the community yet.

EASY | Exponent | 187 of 196

★ TOP 10 EASY: rank 2 among Easy problems

★ Maximum Profit

STUDY: TYPE | Google frequency on LeetCode: all time 72, last 30 days 61 | book priority 5 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #121 Best Time to Buy and Sell Stock

LeetCode match: LC 121 Best Time to Buy and Sell Stock (likely) (inferred from short statement)

4 interviews

Original: https://www.aced.io/questions/3630/maximum-profit

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

APPROACH

Single pass tracking the minimum price seen so far and the best difference; adapt if the statement allows multiple trades. O(n).

INTERVIEW-DAY PLAN

Clarify first

The statement gives a dictionary city to price: ask whether the order of cities matters (it does not, since it is a dictionary), whether you may buy and sell in the same city, and what to return when fewer than two cities exist or all prices are equal.

Brute force to say out loud

Compare every ordered pair of cities. O(n^2).

Optimal idea

Since there is no time order, the maximum profit is the largest price minus the smallest price, so find the city with the minimum price and the city with the maximum price in a single pass. Return [buy city, sell city]. O(n) time, O(1) space. If cities were a time series you would track the running minimum instead (LeetCode 121).

Dry-run these cases

Two cities, ties for the minimum or maximum, all equal prices, a single city, negative prices.

Likely follow-ups

Multiple transactions, a transaction fee, or the same problem on an ordered list.

Pitfalls

Using a time-order solution on an unordered dictionary. Returning the cities in the wrong order.

HOW TO THINK ABOUT IT

Check whether order matters. A dictionary has none, so the profit is largest price minus smallest price, and the answer is just the two extremes. If this were a list of prices over time you would have to buy before you sell, which is a different one-pass problem with a running minimum.

Basis: A dictionary of city to price has no time order, so the best trade is the cheapest city to the most expensive one (compare LeetCode 121, where order matters). Tie rule assumed: smallest city name; with all prices equal, the first two names alphabetically.

BRUTE FORCE - PSEUDO-CODE

```
best = nothing; for every ordered pair (buy city, sell cit
y) of different cities:
    profit = price[sell] - price[buy]; keep the best pair
```

Time: O(n^2) | Space: O(1)

Why it is slow

n^2 pairs. Since city order does not matter, the best pair is the cheapest city and the dearest city, found in one O(n) scan.

OPTIMAL SOLUTION - ONE PASS FOR THE CHEAPEST AND THE DEAREST CITY

```
def max_profit(prices):
    if len(prices) < 2:
        raise ValueError("need at least two cities")
    lo = min(prices, key=lambda c: (prices[c], c))
    hi = min(prices, key=lambda c: (-prices[c], c))
    if lo == hi:
        return sorted(prices)[:2]
    return [lo, hi]
```

Time: O(n) | Space: O(1)

Why

Find the city with the lowest price and the city with the highest price in one scan (ties by name). They are the best pair because profit is sell minus buy and the order of cities does not matter. Handle all-equal prices and fewer than two cities.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The prices are a time series: you must buy before you sell (LeetCode 121).

Keep the minimum price seen so far and the best difference: O(n), O(1). The dictionary version ignores order, so the two extremes are enough.

```
def max_profit_series(prices):
    """The ordered version (LeetCode 121): buy before sell
, one pass with a running minimum."""
    best, low = 0, float("inf")
    for p in prices:
        low = min(low, p)
        best = max(best, p - low)
    return best
```

Allow many trades (LeetCode 122) or a limit of k trades.

Many trades: add every positive day-to-day rise. At most k trades: DP over (trades used, holding or not), O(n * k).

Transaction fee or cooldown.

A two-state DP (holding, not holding) per day: hold = max(hold, free - price), free = max(free, hold + price - fee).

OFFICIAL PAGE TEXT

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases.
Example

```
Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67,
 'Miami': 62}

max_profit(prices) # => ['Miami', 'London']
```

Hint 1You can think of this as two sub-problems: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return different cities?For our solution, we'll simply iterate through the prices list and simultaneously keep track of the minimum and maximum values we've seen so far. This only requires one iteration, so it's more efficient than the 'brute force' approach of calculating the profit for each pair of cities, which would require O(n^2) time. A more concise—but slightly less efficient solution—would be to sort the cities according to their prices and then return the first and last elements. In some languages like Python, you can also use the built-in min and max functions to compare dictionary values:

```
Pythondef max_profit(prices):

  # Check if the dictionary is empty
  if not prices:
    return None

  min_city = min(prices, key=prices.get)
  max_city = max(prices, key=prices.get)

  # Check if no profits earned
  if min_city == max_city:
    return None

  return [min_city, max_city]
```

Time Complexity: Iterating over prices.items() takes O(n) time, where n is the number of cities in the prices dictionary. Within the loop, there are constant-time operations, so the overall time complexity is O(n). Space Complexity: The space complexity is also O(1) because the size of the auxiliary array returned ([min_city, max_city]) is constant and independent of the input size.No answers contributed by the community yet.

EASY | Exponent | 188 of 196

Sort a binary array in linear time.

STUDY: TYPE | Google frequency on LeetCode: all time 53.6, last 30 days 0 | book priority 57 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #75 Sort Colors

LeetCode match: LC 75 Sort Colors (binary version) (inferred from short statement)

Original:
https://www.aced.io/questions/971/sort-binary-array-linear-time

QUESTION

Sort a binary array in linear time.

APPROACH

Two pointers swapping 0s to the front (or counting zeros); one pass, O(n), O(1).

INTERVIEW-DAY PLAN

Clarify first

Ask whether it must be in place and stable (for bare 0 and 1 stability does not matter).

Brute force to say out loud

Use the library sort. O(n log n).

Optimal idea

Two pointers: lo moves right past zeros, hi moves left past ones, and when lo points at a 1 and hi at a 0 swap them. One pass, O(n) time, O(1) space. Counting the zeros and refilling is the other simple option.

Dry-run these cases

Empty array, all zeros, all ones, already sorted, reverse sorted.

Likely follow-ups

Three values (Dutch national flag, LeetCode 75), or sort records by a binary key.

Pitfalls

Moving both pointers after a swap but forgetting the loop bound.

HOW TO THINK ABOUT IT

With only two values the sorted array is 'all zeros then all ones', so you only have to move zeros left and ones right. Two pointers from the two ends do it in one pass: swap whenever the left pointer sees a 1 and the right pointer a 0.

Basis: LeetCode 75, Sort Colors, with two values instead of three.

BRUTE FORCE - PSEUDO-CODE

```
return sorted(arr)
```

Time: O(n log n) | Space: O(n)

Why it is slow

A comparison sort is O(n log n) and needs a copy; with only two values two pointers (swap a 1 on the left with a 0 on the right) finish in one O(n) pass in place.

OPTIMAL SOLUTION - TWO POINTERS SWAPPING FROM BOTH ENDS

```python
def sort_binary(arr):
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            arr[lo], arr[hi] = arr[hi], arr[lo]
            lo += 1
            hi -= 1
    return arr
```

Time: O(n) | Space: O(1)

Why

Keep lo at the next position for a 0 and hi at the next position for a 1. Advance lo past zeros, retreat hi past ones, and when both are stuck swap them. One pass, in place.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Three values (Dutch national flag)

```python
def sort_012(arr):
    low, mid, high = 0, 0, len(arr) - 1
    while mid <= high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 1:
            mid += 1
        else:
            arr[mid], arr[high] = arr[high], arr[mid]
            high -= 1
    return arr
```

Time: O(n) | Space: O(1)

Why

Maintain three regions: [0, low) zeros, [low, mid) ones, (high, end] twos. Examine arr[mid]: swap a 0 to low, skip a 1, swap a 2 to high without advancing mid (the swapped-in value has not been inspected).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Why not just count zeros and fill?

Counting is also O(n) time and O(1) space and is fine for bare numbers. It needs two passes, and it cannot sort records that merely have a binary key, because it loses the objects. The swap version sorts the records themselves.

Count the zeros and rebuild (also O(n)).

Fine for bare numbers: one pass to count, one to fill. It cannot sort records that merely have a 0 or 1 key, because it loses the objects; the swapping version can.

```python
def sort_by_counting(arr):
    zeros = arr.count(0)
    return [0] * zeros + [1] * (len(arr) - zeros)
```

EASY | IN APP LC 953 | Exponent | 189 of 196

Determine if two words are sorted in lexicographic order.

STUDY: TALK | Google frequency on LeetCode: all time 8.1, last 30 days 0 | book priority 154 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #953)

LeetCode match: LC 953 Verifying an Alien Dictionary (inferred from short statement)

Original: https://www.aced.io/questions/3431/check-lexicographic-order-two-words

QUESTION

Determine if two words are sorted in lexicographic order.

APPROACH

Compare the two words character by character using the given order map; shorter prefix sorts first. O(L).

HOW TO THINK ABOUT IT

Compare character by character until the first difference; whichever character comes first in the alphabet decides, and if one word is a prefix of the other the shorter word comes first. A custom alphabet is just a lookup table from letter to rank.

Basis: LeetCode 953, Verifying an Alien Dictionary (a custom alphabet makes it the general case).

BRUTE FORCE - PSEUDO-CODE

```
rank[c] = the position of letter c in the alphabet
ranks_a = [rank[c] for c in a]; ranks_b = [rank[c] for c i
n b]
return ranks_a <= ranks_b (list comparison)
```

Time: O(L) | Space: O(L)

Why it is slow

It builds both full rank lists even when the first letters already decide the answer. Comparing character by character and stopping at the first difference is O(min(len a, len b)) with O(1) extra space.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #953 VERIFYING AN ALIEN DICTIONARY)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Given a custom alphabet order, verify if words are sorted according to that order. Right?"

"A few quick questions:
Standard lexicographic comparison rules apply (shorter prefix word comes first)? Yes."

"Let me walk: words=['hello','leetcode'], order='hlabcdefgijkmnopqrstuvwxyz' -> true (h before l) ok

PHASE 3 — OPTIMIZE

Say: "Build char-order map, compare adjacent strings — O(n * L).
Walk-through:
order_map = {c: i for i,c in enumerate(order)}.
For each adjacent pair (words[i], words[i+1]):
Compare character by character.
Find first position where they differ: if order_map[a] > order_map[b]: return False.
If one string is a prefix of the other AND words[i] is longer: return False.
(E.g., 'apple' before 'app' is wrong — longer string can't come first if it's a prefix)
Return True.
WHY check only adjacent pairs: if all adjacent pairs are in order, the whole list is sorted.
WHY prefix check: 'abc' > 'ab' in standard lex; with alien order, if shorter is prefix of longer,
the shorter should come first -> longer first = out of order.
Time: O(n * L). Space: O(1).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _953_VerifyingAnAlienDictionary:
    def isAlienSorted(self, words, order):
        rank = {ch: i for i, ch in enumerate(order)}
        def in_order(word1, word2):
            for c1, c2 in zip(word1, word2):
                if rank[c1] != rank[c2]:
                    return rank[c1] < rank[c2]
            return len(word1) <= len(word2)
        return all(in_order(words[i], words[i+1]) for i
in range(len(words) - 1))
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace the key invariant and walk through the example to show how the hash structure updates."

words=['hello','leetcode'], order='hlabcdefgijkmnopqrstuvwxyz':
rank['h']=0, rank['l']=1.
in_order('hello','leetcode'): c1='h',c2='l': rank[h]=0 != rank[l]=1 -> return 0<1=True ok
words=['word','world','row'], order='worldabcefghijkmnpqstuvxyz':
in_order('word','world'):
'w'==,'o'==,'r'==,'d' vs 'l': rank[d] vs rank[l] in custom order -> determines result correctly

Comparing by rank instead of raw character lets any custom alphabet ordering work transparently

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(total characters across all words). Space O(1) extra beyond the rank map.
Follow-up: Verifying an Alien Dictionary II — handle case-insensitivity or extra rules.
Follow-up: Alien Dictionary (#269) — INFER the order from a sorted word list (topological sort), the reverse problem.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

What about uppercase, accents, or case-insensitive order?

Normalize first (casefold, unicode normalization) or give the rank map entries for the extra characters. Python's own string comparison orders by code point, which is not locale order, so use locale.strxfrm or a collation library when real language order matters.

A whole list of words in an alien alphabet (LeetCode 953).

Check each adjacent pair; sortedness of a list is the AND of its adjacent pairs. Build the rank table once.

```
def is_sorted_list(words, order):
    rank = {c: i for i, c in enumerate(order)}
    return all(
        [rank[c] for c in a] <= [rank[c] for c in b] for a
, b in zip(words, words[1:])
    )
```

Case-insensitive or accented comparison.

Normalise (casefold, unicode normalisation) before building ranks, or use locale-aware collation keys; plain code-point order is not language order.

Prefix rule.

'app' comes before 'apple' but 'apple' does not come before 'app': compare lengths only after all shared positions are equal.

EASY | HackTheRounds | Onsite | 190 of 196

Parity Alternating Subarray Query

STUDY: TALK | Google frequency on LeetCode: all time 34.9, last 30 days 0 | book priority 101 of 196

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #303 Range Sum Query - Immutable

LeetCode match: No direct match; prefix-sum range queries (idea of LC 303)

QUESTION

Given an array nums and multiple queries, each query asks whether a subarray nums[l:r] satisfies the condition that adjacent elements alternate in parity (odd-even-odd-even or even-odd-even-odd). Preprocess the array to answer each query in O(1) time.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 4, 5], queries = [[0, 4], [1, 3]]`
Output	`[true, true]`
Why	`All adjacent pairs alternate in parity.`

Example 2	
Input	`nums = [1, 3, 2, 4], queries = [[0, 1], [0, 3]]`
Output	`[false, false]`
Why	`nums[0]=1 and nums[1]=3 are both odd.`

CONSTRAINTS

- `1 <= nums.length <= 10^5`
- `1 <= queries.length <= 10^5`
- `0 <= l <= r < nums.length`

APPROACH

Prefix count of adjacent pairs with equal parity; a query [l, r] alternates iff the count over (l..r-1) is 0. O(1) per query.

INTERVIEW-DAY PLAN

Clarify first

Ask whether the query range is inclusive or exclusive (the statement writes nums[l:r]), whether a single element counts as alternating, and the number of queries.

Brute force to say out loud

For each query scan the range checking adjacent parities. O(length) per query.

Optimal idea

Precompute a prefix count B where B[i] is the number of adjacent pairs (j, j + 1) with j < i whose parities are equal. A subarray from l to r (inclusive) alternates exactly when the number of equal-parity pairs among positions l to r - 1 is zero, i.e. B[r]

- B[l] == 0. O(n) preprocessing, O(1) per query.

Dry-run these cases

A single-element range, a range with two equal-parity neighbors, the whole array, l equal r, negative numbers (use x % 2 carefully).

Likely follow-ups

Update an element (Fenwick tree), or return the longest alternating subarray.

Pitfalls

Off-by-one on the inclusive end. Parity of negative numbers in languages where % can be negative.

HOW TO THINK ABOUT IT

A subarray alternates in parity exactly when it contains no pair of neighbours with the same parity. 'Count of bad pairs in a range' is a prefix sum question, so precompute once and answer every query with one subtraction. Mark each position i by whether (nums[i], nums[i + 1]) is a bad pair.

Basis: Prefix counts for range queries (compare LeetCode 303). Assumed rule: a query (l, r) is inclusive on both ends; a single element is alternating.

BRUTE FORCE - PSEUDO-CODE

```
for each query (l, r): for i from l to r - 1: if nums[i] a
nd nums[i + 1] have the same
    parity: answer false
otherwise answer true
```

Time: O(length) per query | Space: O(1)

Why it is slow

Each query scans up to n elements. A prefix count of equal-parity neighbour pairs answers every query in O(1) after one O(n) pass.

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE, APPROACH 1: PREFIX ARRAY PREPROCESSING

```
class Solution:
    def parityAlternatingQueries(self, nums, queries):
        """Check if subarray has alternating parities."""
        n = len(nums)

        # break[i] = True if parity alternation breaks at
index i
        breaks = [False] * n
        for i in range(1, n):
            if (nums[i] % 2) == (nums[i - 1] % 2):
                breaks[i] = True

        # prefix sum of breaks
        prefix = [0] * (n + 1)
        for i in range(n):
            prefix[i + 1] = prefix[i] + (1 if breaks[i] el
se 0)

        result = []
        for l, r in queries:
            # Check if any break exists in range (l+1, r]
            breaks_in_range = prefix[r + 1] - prefix[l + 1
]
            result.append(breaks_in_range == 0)

        return result
```

Time: O(n + q) | Space: O(n)

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The array also changes (point updates).

Keep the bad-pair indicators in a Fenwick tree: an update changes at most two indicators, each an O(log n) update, and a query is two prefix sums.

```
class FenwickParity:
    """Point updates: set nums[i] = v; the bad-pair prefix
 counts live in a Fenwick tree."""

    def __init__(self, nums):
        self.nums, self.n = list(nums), len(nums)
        self.tree = [0] * (self.n + 1)
        for i in range(self.n - 1):
            self._add(i, self._bad(i))

    def _bad(self, i):
        return int(self.nums[i] % 2 == self.nums[i + 1] %
2)

    def _add(self, i, d):
        i += 1
        while i <= self.n:
            self.tree[i] += d
            i += i & -i

    def _prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total

    def update(self, i, v):
        for j in (i - 1, i):
            if 0 <= j < self.n - 1:
                self._add(j, -self._bad(j))
        self.nums[i] = v
        for j in (i - 1, i):
            if 0 <= j < self.n - 1:
                self._add(j, self._bad(j))

    def alternates(self, l, r):
        return self._prefix(r) - self._prefix(l) == 0
```

Find the longest alternating subarray.

One pass with a running length that resets to 1 on a bad pair. O(n), O(1).

```
def longest_alternating(nums):
    best = run = 1 if nums else 0
    for i in range(1, len(nums)):
        run = run + 1 if nums[i] % 2 != nums[i - 1] % 2 el
se 1
        best = max(best, run)
    return best
```

Offline queries, different property (for example all elements distinct).

The prefix-sum trick applies whenever the property is 'no violating neighbour pair'. Properties about distinctness need Mo's algorithm or an offline sweep with last-occurrence tracking. Cost: Mo's algorithm is O((n + q) sqrt(n)); the prefix trick for 'no violating neighbour pair' stays O(n + q).

ALSO REPORTED - 2 MORE REPORTS OF THIS QUESTION

Answer parity-alternation range queries | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-parity-alternation-range-queries

Answer range queries for alternating parity subarrays | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-range-queries-for-alternating-parity-subarrays

Dynamic Programming and State-Space Optimization 2 questions

EASY | IN APP LC 70 | Exponent | 191 of 196

★ TOP 10 EASY: rank 6 among Easy problems

★ Climbing Stairs

STUDY: TYPE | Google frequency on LeetCode: all time 62.9, last 30 days 51.4 | book priority 7 of 196

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #70)

LeetCode match: LC 70 Climbing Stairs

+38 interviews

Original: https://www.aced.io/questions/3255/climbing-stairs

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.
Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:
1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

APPROACH

dp[i] = dp[i-1] + dp[i-2] with two variables. O(n), O(1).

HOW TO THINK ABOUT IT

The last move was either a 1-step from n - 1 or a 2-step from n - 2, so ways(n) = ways(n - 1) + ways(n - 2): the Fibonacci recurrence. Only the previous two values matter, so two variables replace the whole table.

Basis: LeetCode 70, Climbing Stairs: ways to climb n steps taking 1 or 2 at a time (a Fibonacci number).

BRUTE FORCE - PSEUDO-CODE

```
ways(n): if n <= 1: return 1
return ways(n - 1) + ways(n - 2)
```

Time: O(2^n) | Space: O(n)

Why it is slow

The same values are computed over and over: O(2^n). Keeping the last two values in a loop is O(n) time and O(1) space (and a matrix power gives O(log n)).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #70 CLIMBING STAIRS)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
You can climb 1 or 2 steps at a time. How many distinct ways to reach step n? Right?"

"A few quick questions:
n is at least 1? Yes. Order of steps matters — (1,2) and (2,1) are different? Yes."

"Let me walk: n=2 -> (1,1),(2) -> 2 ways ok. n=3 -> (1,1,1),(1,2),(2,1) -> 3 ways ok

PHASE 3 — OPTIMIZE

The bottleneck is recomputing the same subproblems — this is Fibonacci.
Track only the last two values — O(1) space:
a, b -> b, a+b each step.

Time: O(n). Space: O(1)."
Key insight: dp[i] = dp[i-1] + dp[i-2] is the Fibonacci recurrence — only two rolling variables are needed, cutting space from O(n) to O(1).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _70_ClimbingStairs:
    def climbStairs(self, n):
        a, b = 1, 1
        for _ in range(n - 1):
            a, b = b, a + b
        return b
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace climbStairs on n=5."
dp[0]=1 (0 steps: 1 way). dp[1]=1 (1 step: only 1). dp[2]=2 (1+1 or 2).
dp[3]=dp[2]+dp[1]=3. dp[4]=dp[3]+dp[2]=5. dp[5]=dp[4]+dp[3]=8. Return 8 ok
WHY dp[i]=dp[i-1]+dp[i-2]: to reach step i, you came from i-1 (took 1 step) or i-2 (took 2).
This is Fibonacci! dp = [1,1,2,3,5,8,13,...].

Space optimization: only need prev two values. a=1,b=1. Loop 2..n: c=a+b; a=b; b=c. return b.
Space-optimized: a,b=1,1. for i in range(2,n+1): a,b=b,a+b. Return b.
This is exactly Fibonacci(n+1). climbStairs(n)=fib(n+1) ok

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n): one pass. Space O(1): two rolling variables.
Follow-up: if you can climb 1, 2, or 3 steps — same DP, track last 3 values.
Follow-up: Min Cost Climbing Stairs (#746) — add cost per step; choose min(dp[n-1],dp[n-2]).

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

n is huge (10^18).

Use matrix exponentiation of [[1, 1], [1, 0]] (or fast doubling) for O(log n) multiplications; take the result modulo a prime if asked.

```
def climb_matrix(n):
    def mul(A, B):
        return [
            [
                A[0][0] * B[0][0] + A[0][1] * B[1][0],
                A[0][0] * B[0][1] + A[0][1] * B[1][1],
            ],
            [
                A[1][0] * B[0][0] + A[1][1] * B[1][0],
                A[1][0] * B[0][1] + A[1][1] * B[1][1],
            ],
        ]

    result, base, p = [[1, 0], [0, 1]], [[1, 1], [1, 0]],
n
    while p:
        if p & 1:
            result = mul(result, base)
        base = mul(base, base)
        p >>= 1
    return result[0][0]
```

You can take 1 to k steps (or a given set of step sizes).

ways[i] = sum of ways[i - s] over allowed step sizes s; for 1..k keep a sliding-window sum for O(n).

```
def climb_k_steps(n, k):
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        ways[i] = sum(ways[i - j] for j in range(1, min(k,
 i) + 1))
    return ways[n]
```

Some steps are broken (cannot be stood on).

Set ways[i] = 0 for broken steps while filling the table; everything else is unchanged.

OFFICIAL PAGE TEXT

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer.
Examples

```
Input: n = 1
Output: 1
Explanation: There is only 1 way to climb a staircase with
 1 step.

Input: n = 2
Output: 2
Explanation: There are 2 ways to climb a staircase with 2
steps:
1. Climb 1 step + 1 step
2. Climb 2 steps directly

Input: n = 3
Output: 3
Explanation: There are 3 ways to climb a staircase with 3
steps:
1. Climb 1 step + 1 step + 1 step
2. Climb 1 step + 2 steps
3. Climb 2 steps + 1 step

Input: n = 4
Output: 5
Explanation: There are 5 ways to climb a staircase with 4
steps:
1. Climb 1 step + 1 step + 1 step + 1 step
2. Climb 1 step + 1 step + 2 steps
3. Climb 1 step + 2 steps + 1 step
4. Climb 2 steps + 1 step + 1 step
5. Climb 2 steps + 2 steps
```

Challenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] = 1 (1 way to reach the first step, which is a single 1-step) For each subsequent step i, the number of ways to reach that step is the sum of the ways to reach the previous step (ways[i - 1]) and the step before that (ways[i - 2]). This is because you can get to step i either from step i - 1 with a 1-step or from step i - 2 with a 2-step. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we iterate through the steps from 2 to n, updating the number of ways for each step. Space Complexity: The space complexity is O(n) due to the additional array ways used to store the number of ways for each step. If space optimization is needed, this can be reduced to O(1) by keeping only the last two computed values. Solution 2: Combinatorial Approach Our solution leverages combinatorial mathematics to calculate the number of distinct ways to climb n stairs. The approach is based on counting the number of valid sequences of steps (1-step and 2-steps) that sum up to n. We use the binomial coefficient to count the number of ways to arrange a certain number of 1-steps and 2-steps. For each possible number of

2-steps (k), we compute the number of ways to choose k positions from n - k steps (the remaining steps will be 1-steps). This is done using the formula for combinations: C(n,k)C(n, k)C(n,k), where C(n,k)=n!k!*(n-k)!C(n, k) = \frac{n!}{k! \cdot (n-k)!}C(n,k)=k!*(n-k)!n!. The total number of ways is the sum of these combinations for all valid values of k. Time Complexity: The time complexity of this solution is O(n), where n is the number of steps. This is because we compute the combination for each k from 0 to n // 2, and each combination computation is efficient. Space Complexity: The space complexity is O(1) as we only use a few additional variables for storing intermediate results, regardless of the input size.No answers contributed by the community yet.

EASY | IN APP LC 509 | Exponent | 192 of 196

Fibonacci Numbers

STUDY: TYPE | Google frequency on LeetCode: all time 46.3, last 30 days 0 | book priority 68 of 196
Topic: Dynamic Programming and State-Space Optimization
Optimal answer from: your app's interview approach (LeetCode #509)
LeetCode match: LC 509 Fibonacci Number
+26 interviews
Original: https://www.aced.io/questions/3631/fibonacci

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

APPROACH

Iterative two-variable loop O(n), O(1) space; mention memoization and matrix exponentiation / fast doubling for O(log n).

HOW TO THINK ABOUT IT

Each term needs only the previous two, so a loop with two variables is O(n) time and O(1) space; the naive recursion does exponential repeated work. Since the recurrence is linear it can also be turned into a matrix power, or into the fast doubling identities, for O(log n).

Basis: LeetCode 509, Fibonacci Number (F(0) = 0, F(1) = 1).

BRUTE FORCE - PSEUDO-CODE

```
fib(n): if n < 2: return n
return fib(n - 1) + fib(n - 2)
```

Time: O(2^n) | Space: O(n)

Why it is slow

The call tree has about 1.6^n nodes because the same values are recomputed. A loop with two variables is O(n) time and O(1) space; fast doubling is O(log n).

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #509 FIBONACCI NUMBER)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Return the nth Fibonacci number. F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2). Right?"

"A few quick questions:
n can be 0? Yes -> return 0. n>=0 always? Yes."

"Let me walk: F(0)=0, F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5 ok

PHASE 3 — OPTIMIZE

Say: "Iterative bottom-up DP — O(n). Walk-through:
Base: fib(0)=0, fib(1)=1.
For i in 2..n: fib[i] = fib[i-1] + fib[i-2].
Space optimization: only keep last two values.
a, b = 0, 1.
For i in 2..n: a, b = b, a+b.
Return b.
WHY bottom-up beats naive recursion: naive recursion recomputes fib(k) for every k
multiple times -> O(2^n). Bottom-up computes each once -> O(n).
Memoized recursion also O(n) but O(n) stack space vs O(1) iterative.
Time: O(n). Space: O(1).

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _509_FibonacciNumber:
    def fib(self, n):
        if n <= 1:
            return n
        prev2, prev1 = 0, 1
        for _ in range(2, n + 1):
            prev2, prev1 = prev1, prev2 + prev1
        return prev1
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace fib(5)."
dp[0]=0, dp[1]=1, dp[2]=1, dp[3]=2, dp[4]=3, dp[5]=5 ok
Space-optimized: prev=0, curr=1
n=2: prev,curr=1,1; n=3: prev,curr=1,2; n=4: prev,curr=2,3; n=5: prev,curr=3,5
Return curr=5 ok
WHY two variables: fib(n) only needs fib(n-1) and fib(n-2); no need for full array.
Recursive: fib(5)->fib(4)+fib(3)->... exponential without memoization.
Edge: fib(0)=0, fib(1)=1 (base cases).

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n). Space O(1).
Matrix exponentiation achieves O(log n) — useful for very large n.
Follow-up: Climbing Stairs (#70) — same Fibonacci recurrence.
Follow-up: N-th Tribonacci Number (#1137) — three rolling variables.

OTHER WAYS TO SOLVE IT - ALTERNATIVES AND VARIANTS

Fast doubling

```
def fib_fast(n):
    def go(k):
        if k == 0:
            return 0, 1
        a, b = go(k // 2)
        c = a * (2 * b - a)
        d = a * a + b * b
        return (d, c + d) if k % 2 else (c, d)

    return go(n)[0]
```

Time: O(log n) big-integer operations | Space: O(log n) recursion depth

Why

Use F(2k) = F(k) * (2 * F(k + 1)

- F(k)) and F(2k + 1) = F(k)^2 + F(k + 1)^2. Recursing on n // 2 halves the problem each time, so only about log2(n) steps are needed.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

n is up to 10^18 and the answer is needed modulo a prime.

Fast doubling (or matrix power) with every multiplication reduced modulo the prime: O(log n) steps with small numbers.

```
def fib_mod(n, m):
    def go(k):
        if k == 0:
            return 0, 1
        a, b = go(k // 2)
        c = a * (2 * b - a) % m
        d = (a * a + b * b) % m
        return (d, (c + d) % m) if k % 2 else (c, d)

    return go(n)[0]
```

Why is the recursive version slow, and how do you fix it without changing the structure?

It recomputes the same values exponentially often (the call tree has about phi^n nodes). Memoisation (lru_cache) makes each value computed once, so O(n) time with O(n) memory; bottom-up with two variables drops the memory to O(1).

```
def fib_memo(n):
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)
```

Closed form (Binet's formula).

F(n) = round(phi^n / sqrt(5)) is exact only while floating point has enough precision (up to about n = 70); beyond that it gives wrong digits, so use the integer methods.

OFFICIAL PAGE TEXT

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern:

```
1, 1, 2, 3, 5, 8, 13 ...
```

Examples

```
Pythonfib(0)  # => 1
fib(1)  # => 1
fib(2)  # => 2
fib(3)  # => 3
...
fib(10) # => 89
fib(20) # => 10946
```

Hint 1The Fibonacci sequence is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

calculations. Space Complexity: The memo dictionary stores each Fibonacci number from 0 to n, which requires O(n) space.No answers contributed by the community yet.

Other: Design, Heaps, Intervals and Misc 3 questions

EASY | IN APP LC 373 | PracHub | Technical Screen | 193 of 196

Find k pairs with smallest sums

STUDY: TALK | **Google frequency on LeetCode: all time 32.6, last 30 days 0 | book priority 108 of 196**

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Also in your app (open the app for its Expose): LeetCode #373 Find K Pairs with Smallest Sums

LeetCode match: LC 373 Find K Pairs with Smallest Sums

Original:
https://prachub.com/coding-questions/find-k-pairs-with-smallest-sums

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **k pairs `(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -
> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17 The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

APPROACH

Min-heap seeded with (arr1[i]+arr2[0], i, 0) for the first min(k, len1) indices; pop, then push (i, j+1). O(k log k). Return all pairs if fewer than k exist.

HOW TO THINK ABOUT IT

Because both arrays are sorted, the pair (i, j + 1) is never smaller than (i, j), so each row i of the 'matrix of sums' is sorted. The k smallest pairs are the k smallest items of a collection of sorted rows, which is a heap problem: start with the first element of each row (or the first few) and each time you pop a pair, push its successor in the row.

Basis: LeetCode 373, Find K Pairs with Smallest Sums (two sorted arrays).

BRUTE FORCE - PSEUDO-CODE

```
pairs = every (x, y) with x from the first array and y fro
m the second
sort the pairs by x + y and return the first k
```

Time: O(m * n log(m * n)) | Space: O(m * n)

Why it is slow

m * n pairs sorted: with 10^5 elements in each array that is 10^10 pairs. Because both arrays are sorted, a min-heap that starts from each x's first pair and expands only the next pair in that row is O(k log k).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
import heapq


def solution(arr1, arr2, k):
    if not arr1 or not arr2 or k <= 0:
        return []
    heap = []
    for i in range(min(len(arr1), k)):
        heapq.heappush(heap, (arr1[i] + arr2[0], i, 0))
    result = []
    while heap and len(result) < k:
        _, i, j = heapq.heappop(heap)
        result.append([arr1[i], arr2[j]])
        if j + 1 < len(arr2):
            heapq.heappush(heap, (arr1[i] + arr2[j + 1], i
, j + 1))
    return result
```

Time: O(min(k, len(arr1)) + k log min(k, len(arr1))) | Space: O(min(k, len(arr1)))

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

```
def kSmallestPairs(a, b, k):
    if not a or not b or k <= 0:
        return []
    heap = [(a[i] + b[0], i, 0) for i in range(min(k, len(
a)))]
    heapq.heapify(heap)
```

```
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            heapq.heappush(heap, (a[i] + b[j + 1], i, j +
1))
    return out


def k_smallest_unsorted(a, b, k):
    return kSmallestPairs(sorted(a), sorted(b), k)
```

Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

```
def kSmallestPairs(a, b, k):
    if not a or not b or k <= 0:
        return []
    heap = [(a[i] + b[0], i, 0) for i in range(min(k, len(
a)))]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            heapq.heappush(heap, (a[i] + b[j + 1], i, j +
1))
    return out


def kth_smallest_pair_sum(a, b, k):
    pairs = kSmallestPairs(a, b, k)
    return None if len(pairs) < k else sum(pairs[-1])
```

Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. Cost: O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

EASY | IN APP LC 206 | Exponent | 194 of 196

Reverse a linked list.

STUDY: TYPE | Google frequency on LeetCode: all time 59.8, last 30 days 0 | book priority 31 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #206)

LeetCode match: LC 206 Reverse Linked List

+921 interviews

Original: https://www.aced.io/questions/3247/reverse-linked-list

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

APPROACH

Iterative: walk with prev/cur pointers flipping next; or recursion. O(n) time, O(1) space iterative.

HOW TO THINK ABOUT IT

Reversing means every next pointer has to flip, so hold three things: the node you are on, the one before it (what it will point to) and the one after it (so you do not lose the rest). Always save next before overwriting it. For 'reverse part of the list' remember the node before the part and the node that will end up last in it.

Basis: LeetCode 206 (Reverse Linked List), 92 (between positions) and 25 (in groups of k).

BRUTE FORCE - PSEUDO-CODE

```
read all values into an array
build a new list by pushing each value to the front
```

Time: O(n) | Space: O(n)

Why it is slow

Time is O(n), but it allocates a whole new list, O(n) extra space. Flipping each node's next pointer in place with three variables needs O(1) space.

OPTIMAL SOLUTION - YOUR APP'S INTERVIEW APPROACH (LEETCODE #206 REVERSE LINKED LIST)

INTERVIEW APPROACH · STAR-LC

PHASE 1 — CLARIFY

Let me make sure I understand the problem correctly.
Given the head of a singly linked list, reverse it in-place and return the new head.
We change the next pointers — not just the values. Right?"

"A few quick questions:
Can the list be empty? Yes — return None.
Can it have just one node? Yes — return that node unchanged.
The follow-up asks for both iterative and recursive — I'll do iterative first."

"Let me walk the example: [1,2,3,4,5] -> [5,4,3,2,1] ok

PHASE 3 — OPTIMIZE

The bottleneck is the O(n) extra space and the unnecessary node allocation.
We can reverse in-place by maintaining three pointers: prev, curr, next_node.
At each step we point curr.next backward to prev, then advance.

Pseudocode:
prev, curr = None, head
while curr:
next_node = curr.next
curr.next = prev
prev, curr = curr, next_node
return prev

Time: O(n). Space: O(1).
Does that make sense before I code it?

PHASE 4 — CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _206_ReverseLinkedList:
    def reverseList(self, head):
        prev, curr = None, head
        while curr:
            next_node = curr.next
            curr.next = prev
            prev, curr = curr, next_node
        return prev
```

Select text -> Cmd/Ctrl+C to copy.

PHASE 5 — TEST & DEBUG

Say: "Let me trace reverseList(1->2->3->4->5)."
Iterative: prev=None, curr=1.
curr=1: nxt=2. 1.next=None. prev=1. curr=2.
curr=2: nxt=3. 2.next=1. prev=2. curr=3.
curr=3: nxt=4. 3.next=2. prev=3. curr=4.
curr=4: nxt=5. 4.next=3. prev=4. curr=5.
curr=5: nxt=None. 5.next=4. prev=5. curr=None. Stop.
Return prev=5. 5->4->3->2->1 ok
Recursive: reverseList(5) returns 5. Then 4.next.next=4 -> 5->4->...->1. Each call reverses one link.

WHY: pointer moves preserve the list's integrity; each rewiring step only changes local next-links while keeping the head/reference valid.

PHASE 6 — COMPLEXITY & FOLLOW-UP

Time O(n): one pass. Space O(1): three pointer variables.
Recursive solution is elegant but O(n) stack space — mention this trade-off.
Follow-up: Reverse Linked List II (#92) — reverse between positions l and r.
Follow-up: Reverse Nodes in k-Group (#25) — apply this logic to groups of k.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Do it recursively.

Reverse the rest first, then make the next node point back at this one and cut this node's own pointer. An inner helper that carries prev (as coded) keeps it tail-recursive in spirit; the call stack is O(n), so mention the depth limit.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def reverse_recursive(head):
    def go(node, prev):
        if node is None:
            return prev
        nxt = node.next
        node.next = prev
        return go(nxt, node)

    return go(head, None)
```

Reverse only positions left to right (LeetCode 92).

Walk to the node before the segment, reverse right - left + 1 nodes with the same loop, then reconnect: the old first node of the segment must point to what followed the segment, and the node before it points to the new first.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def reverse_between(head, left, right):
    dummy = ListNode(0, head)
    before = dummy
    for _ in range(left - 1):
        before = before.next
    prev, cur = None, before.next
    for _ in range(right - left + 1):
        nxt = cur.next
        cur.next = prev
        prev, cur = cur, nxt
    before.next.next = cur  # the old first node of the se
gment now ends it
    before.next = prev
    return dummy.next
```

Reverse in groups of k and leave a short last group alone (LeetCode 25).

Check that k nodes exist before reversing a group, reverse them with the same loop, reconnect both ends, and move the group pointer to the old first node of the group.

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def reverse_k_group(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        kth = group_prev
        for _ in range(k):
            kth = kth.next
            if kth is None:
                return dummy.next
        group_next = kth.next
        prev, cur = group_next, group_prev.next
        while cur is not group_next:
            nxt = cur.next
            cur.next = prev
            prev, cur = cur, nxt
        tmp = group_prev.next
        group_prev.next = kth
        group_prev = tmp
```

OFFICIAL PAGE TEXT

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example

```
Input: 1 -> 2 -> 3 -> 4
Output: 4 -> 3 -> 2 -> 1
```

Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?Our solution reverses the linked list by iterating through it while maintaining pointers to keep track of the previous node and the current node. For each node, it saves the next node, then reverses the link by pointing the current node's next to the previous node. It then moves the prev pointer to the current node and advances the current pointer to the saved next node. This process continues until the end of the list is reached. At the end, the prev pointer points to the new head of the reversed list, which is returned. The solution uses two pointers, prev and current, to achieve this reversal. This approach ensures that each node is processed exactly once, making it both efficient and straightforward. Time Complexity: The solution has a time complexity of O(n), where n is the number of nodes in the linked list. This is because we iterate through the list once, performing constant-time operations (pointer adjustments) for each node. Space Complexity: The solution uses O(1) space, as it only uses a fixed amount of extra space (three pointers: prev, current, and nextNode). No additional data structures are used, making it very space-efficient. NoteCheck out our lesson on the two pointer approach to learn more about what it is and how to use it!No answers contributed by the community yet.

EASY | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 195 of 196

Implement Batched Undo/Redo Layer

STUDY: TALK | Google frequency on LeetCode: all time 16.5, last 30 days 0 | book priority 144 of 196

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: LC 1472 Design Browser History (related: undo/redo stacks)

Original: https://prachub.com/coding-questions/implement-batched-undo-redo-layer

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch.

Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value after th
e batch string old_value; // value before the batch bool h
as_old_value; // whether the key existed before the batch
}
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.

Constraints

0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

Examples

Input: [('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]

Expected Output: [{'color': 'red'}, {}, {'color': 'red'}]

Explanation: A single batch is committed, undone, and redone.

Input: [('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]

Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for

committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.
Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

APPROACH

Hold the open batch as a map of key -> (oldValue, newValue); endBatch pushes it onto the undo stack and clears the redo stack. undo applies the old values and moves the batch to the redo stack, redo reverses that. Handle an empty batch and repeated sets of one key in a batch (keep the first old value).

INTERVIEW-DAY PLAN

Clarify first

Ask what undo does on an empty stack, whether an empty batch counts as an action, whether new edits after an undo clear the redo history (yes, normally), and how nested batches behave.

Brute force to say out loud

Snapshot the whole map after every batch and keep a list of snapshots. Simple but memory heavy.

Optimal idea

Hold the open batch as a dict key to (old value, new value), recording the old value only the first time a key is touched in the batch. endBatch pushes it on the undo stack and clears the redo stack. undo pops, restores the old values (deleting keys that did not exist), and pushes onto redo. redo reapplies the new values. O(size of batch) each.

Dry-run these cases

Set the same key twice in one batch, set a new key then undo (it must disappear), undo and redo with nothing available, a new batch after an undo (redo cleared), an empty batch.

Likely follow-ups

Nested batches, bounded history, or undo of individual operations.

Pitfalls

Recording the old value on every set instead of only the first. Forgetting to clear redo on a new batch.

HOW TO THINK ABOUT IT

Undo and redo are two stacks of actions. The question is what an action is: a batch is one action, so record per batch only what is needed to reverse it (the old value of each key touched, and the new one), not a whole copy of the document. A new edit after an undo kills the redo history.

Basis: LeetCode 1472 (Design Browser History) for the stacks idea, plus transactional batching.

BRUTE FORCE - PSEUDO-CODE

```
on every committed batch: push a copy of the whole propert
y map onto the undo stack,
    clear redo
undo/redo: swap the current map with the copy from the oth
er stack
```

Time: O(size of document) per batch | Space: O(size * batches)

Why it is slow

A one-key change copies the whole document, O(size) per batch and O(size * batches) memory. Storing only the old and new value of each changed key makes each batch O(keys changed).

OPTIMAL SOLUTION - FROM THE INTERVIEW SITE

```
def solution(operations):
    doc = {}
    undo_stack = []
    redo_stack = []
    current_batch = None
    snapshots = []

    for op in operations:
        cmd = op[0]

        if cmd == "begin":
            current_batch = {}

        elif cmd == "set":
            _, key, value = op
            if key not in current_batch:
                if key in doc:
                    current_batch[key] = [value, doc[key],
 True]
                else:
                    current_batch[key] = [value, None, Fal
se]
            else:
                current_batch[key][0] = value
            doc[key] = value

        elif cmd == "end":
            if current_batch:
                batch = []
                for key, (new_value, old_value, had_old_va
lue) in current_batch.items():
                    batch.append((key, new_value, old_valu
e, had_old_value))
                undo_stack.append(batch)
                redo_stack.clear()
            current_batch = None

        elif cmd == "undo":
            if undo_stack:
                batch = undo_stack.pop()
                for key, new_value, old_value, had_old_val
ue in batch:
                    if had_old_value:
                        doc[key] = old_value
                    else:
                        doc.pop(key, None)
                redo_stack.append(batch)

        elif cmd == "redo":
            if redo_stack:
                batch = redo_stack.pop()
                for key, new_value, old_value, had_old_val
ue in batch:
                    doc[key] = new_value
                undo_stack.append(batch)
```

```
        elif cmd == "snapshot":
            snapshots.append(dict(doc))

    return snapshots
```

Time: O(1) per begin/set/snapshot, and O(k) per end/undo/redo where k is the number of distinct keys touched in that batch. | Space: O(d + h), where d is the current document size and h is the total number of distinct-key changes stored in undo/redo history.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

```
class Layer:
    def __init__(self):
        self.props, self.undo_s, self.redo_s, self.open =
{}, [], [], None

    def beginBatch(self):
        self.open = {}

    def setProperty(self, k, v):
        if self.open is None:
            self.beginBatch()
            self.setProperty(k, v)
            self.endBatch()
            return
        old = self.open[k][0] if k in self.open else self.
props.get(k, _MISSING)
        self.open[k] = (old, v)
        self.props[k] = v

    def endBatch(self):
        if self.open:
            self.undo_s.append(self.open)
            self.redo_s.clear()
        self.open = None

    def _apply(self, batch, use_new):
        for k, (old, new) in batch.items():
            value = new if use_new else old
            if value is _MISSING:
                self.props.pop(k, None)
            else:
                self.props[k] = value

    def undo(self):
        if self.undo_s:
            batch = self.undo_s.pop()
            self._apply(batch, False)
            self.redo_s.append(batch)

    def redo(self):
        if self.redo_s:
            batch = self.redo_s.pop()
            self._apply(batch, True)
            self.undo_s.append(batch)


class LayerBounded(Layer):
    def __init__(self, limit):
        super().__init__()
        self.limit = limit

    def endBatch(self):
        super().endBatch()
        if len(self.undo_s) > self.limit:
            self.undo_s.pop(0)
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[list[str]]) -> list[dict[str
, str]]:
SAMPLE INPUT
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot
',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
```

Test cases

Input	Expected
`[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'color': 'red'}, {}, {'color': 'red'}]`
`[('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]`

Trie and Prefix Indexing 1 questions

EASY | Exponent | 196 of 196

Return all unique substrings of a given string.

STUDY: READ | Google frequency on LeetCode: none listed (not on its Google list) | book priority 187 of 196

Topic: Trie and Prefix Indexing

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (substring enumeration) (inferred from short statement)

Original:
https://www.aced.io/questions/1223/unique-substrings-given-string

QUESTION

Return all unique substrings of a given string.

APPROACH

Double loop adding s[i:j] to a set, O(n^2) substrings (O(n^3) chars). Mention a suffix automaton/trie for large n.

INTERVIEW-DAY PLAN

Clarify first

Ask whether to return the substrings or just count them, whether the empty string counts, the maximum length of the string, and whether order matters.

Brute force to say out loud

Double loop adding s[i:j] to a set. O(n^2) substrings and up to O(n^3) characters in total.

Optimal idea

For small n the set is fine. To count without storing, build a suffix automaton or a suffix array with LCP: the number of distinct substrings is the sum of (n - suffix_start) minus the LCP values, O(n log n). A trie over all suffixes also works at O(n^2).

Dry-run these cases

Empty string, a single character, all equal characters (n substrings), a string with no repeats (n(n + 1) / 2).

Likely follow-ups

Count only, the k-th lexicographic substring, or the longest repeated substring.

Pitfalls

Counting the empty string by accident. Using a list instead of a set and returning duplicates.

HOW TO THINK ABOUT IT

If the output must list the substrings, its size already is quadratic, so a set over all slices is the honest answer. If only the count is needed, nothing needs to be stored: distinct substrings equal total suffix lengths minus the sum of longest-common-prefixes between adjacent sorted suffixes.

Basis: Written; counting distinct substrings is a classic (suffix array with LCP, suffix automaton).

BRUTE FORCE - PSEUDO-CODE

```
set = {}; for every i < j: add s[i:j] to the set
return the set
```

Time: O(n^3) | Space: O(n^3)

Why it is slow

There are about n^2 / 2 substrings and each slice copies up to n characters: O(n^3) time and space. If only the count is needed, a suffix array with LCP counts them in O(n log n) without storing any.

OPTIMAL SOLUTION - COUNT WITH A SUFFIX ARRAY AND LCP

```python
def count_unique_substrings(s):
    n = len(s)
    suffixes = sorted(s[i:] for i in range(n))
    total = n * (n + 1) // 2
    for a, b in zip(suffixes, suffixes[1:]):
        k = 0
        while k < min(len(a), len(b)) and a[k] == b[k]:
            k += 1
        total -= k
    return total
```

Time: O(n^2 log n) with naive sorting (O(n log n) with a real suffix array) | Space: O(n)

Why

Sort the suffixes; the number of distinct substrings is sum over suffixes of (length) minus sum of the LCP of neighbours in sorted order, because each new suffix adds only the prefixes that the previous one did not already contribute. This counts without storing a single substring.

WHEN THE QUESTION CHANGES - FOLLOW-UPS AND HOW THE ANSWER UPDATES

List them without building duplicates first.

Insert every suffix into a trie; a substring is new exactly when its trie node is created. O(n^2) time and nodes, and the substrings appear without duplicates.

```python
def unique_substrings_trie(s):
    trie, out = {}, []
    for i in range(len(s)):
        node, cur = trie, []
        for ch in s[i:]:
            cur.append(ch)
            if ch not in node:
                node[ch] = {}
                out.append("".join(cur))
            node = node[ch]
    return out
```

Return the k-th lexicographic distinct substring, or the longest repeated substring.

Both use the suffix array with LCP: the longest repeated substring is the maximum LCP value, and the k-th substring is found by walking the sorted suffixes and subtracting the new prefixes each one contributes.

The string has a million characters.

Quadratic is out. Build a suffix automaton (O(n)) and count distinct substrings as the sum of len(v) - len(link(v)) over states, or use a suffix array with O(n log n) construction.